

A Mathematical Atlas of Adaptive Intelligence

Grand Challenge Labs

2026-10-07

Contents

1	The Adaptive-System Thesis	1
1.1	What this book is trying to explain	1
1.2	An atlas, not a catalogue	2
1.3	The recurring shifts	3
1.3.1	Vectors to operators	3
1.3.2	Layers to flows	3
1.3.3	Parameters to geometry	4
1.3.4	Routing to decision making	4
1.3.5	Context to compiled memory access	4
1.3.6	Agents to distributed systems	4
1.3.7	Training to dynamical identification	4
1.3.8	Interpretability to intervention	5
1.3.9	Architecture to composition of contracts	5
1.3.10	Result to evidence object	5
1.4	A provisional system decomposition	5
1.5	Why the model boundary is too small	6
1.6	A useful thesis must be vulnerable	7
1.7	Description versus programme	7
1.8	Dependencies in the argument	8
1.9	Tests the thesis must pass	8
1.10	The map ahead	8
1.11	Closing view	9
1.12	References used in this chapter	9
2	How to Read a Mathematical Atlas	11
2.1	An atlas is not a road	11
2.2	Map and territory	12
2.3	Read at several resolutions	13
2.4	Stable identity	13
2.5	The map is not the dependency graph	14
2.6	Hard dependency	14
2.7	Soft cross-link	14
2.8	Narrative is not mathematical authority	15

2.9	A compact epistemic legend	15
2.10	Six ways through the Atlas	15
2.10.1	Route A — linear foundations	16
2.10.2	Route B — dependency-first	16
2.10.3	Route C — concept/theme	16
2.10.4	Route D — research-frontier	17
2.10.5	Route E — proof/replay	17
2.10.6	Route F — visual/Atlas-plate	18
2.11	Choosing a route	18
2.12	How to read a keystone	19
2.13	How to read a frontier chapter	19
2.14	Allegory as a bounded instrument	19
2.15	What changed in our picture?	20
2.16	Five reading errors to avoid	20
2.16.1	Chapter order treated as dependency order	20
2.16.2	Soft cross-link treated as prerequisite	20
2.16.3	Exact computation treated as proof automatically	20
2.16.4	Replay treated as certification	20
2.16.5	Schematic figure treated as measured geometry	21
2.17	A compact reader protocol	21
2.18	Closing view	21
2.19	References used in this chapter	22
3	States, Operators, Flows, and Interfaces	23
3.1	The same tensor can be four different things	23
3.2	Noun, verb, motion, handshake	24
3.3	State	24
3.4	Operator	25
3.4.1	First category error: array equals role	26
3.5	Flow and dynamics	26
3.5.1	Second category error: one step equals dynamics	27
3.5.2	Flow-like is weaker than flow	27
3.6	Interface	28
3.6.1	Third category error: matching shape equals composability	29
3.6.2	Interfaces are not interiors	29
3.7	State-dependent operators	30
3.7.1	Operator-valued state	30
3.7.2	Evidence is another object class	31
3.8	Why this vocabulary matters	31
3.8.1	The four questions	31
3.8.2	State	32
3.8.3	Operator	32
3.8.4	Flow	32
3.8.5	Interface	32
3.9	Handoff to linear algebra	32
3.9.1	Handoff to information	32

3.10	Closing view	33
3.11	References used in this chapter	33
4	Claims, Evidence, and Computational Witnesses	35
4.1	Start with the exact claim	36
4.2	The claim-support packet	36
4.3	Eleven labels, not eleven rungs	37
4.4	Definition is not discovery	38
4.5	Established results and Atlas derivations	39
4.6	Computational witnesses	39
4.6.1	What a witness can be excellent at	40
4.6.2	What a witness cannot acquire by presentation	40
4.7	Observation is not interpretation	40
4.8	Public project evidence and programme context	41
4.9	Conjecture and open problem	41
4.10	Institutional status is a different axis	42
4.11	Source identity is not source authority	42
4.12	Promotion requires new support	43
4.13	Five common promotion failures	43
4.13.1	Finite check to universal theorem	43
4.13.2	Repeatable observation to universal mechanism	43
4.13.3	Narrow proof to broad prose	44
4.13.4	Precise citation to excessive authority	44
4.13.5	Procedure to truth	44
4.14	What later chapters may now assume	44
4.15	The durable habit	45
4.16	References used in this chapter	45
5	Linear Maps and Decompositions	47
5.1	Linear algebra as transformation language	47
5.2	Changing lenses versus changing the machine	48
5.3	Bases and coordinate representations	48
5.4	Subspaces	49
5.5	Orthogonal projection	50
5.6	Eigendecomposition	50
5.7	Normality	51
5.8	Singular-value decomposition	51
5.9	Operator norm	52
5.10	Eigenvalues and singular values answer different questions	52
5.11	Pseudoinverse and least squares	53
5.12	Low-rank approximation	54
5.13	Conditioning	55
5.14	Condition number is not optimization difficulty	55
5.15	Block structure	56
5.16	Kronecker structure	56
5.17	What linearization does and does not buy us	56

5.18	Computational witness	57
5.19	Five failure modes	58
5.19.1	Coordinates mistaken for invariants	58
5.19.2	Eigenvalues mistaken for all amplification	58
5.19.3	Low rank mistaken for semantic sufficiency	58
5.19.4	Conditioning mistaken for algorithm failure	59
5.19.5	Local linearization mistaken for global behavior	59
5.20	Atlas connections	59
5.21	Closing view	59
5.22	References used in this chapter	60
6	Normality, Pseudospectra, and Transient Growth	61
6.1	Stable eigenvalues can tell an incomplete story	61
6.2	Normality	62
6.3	Non-normality breaks that alignment	63
6.4	The smallest useful counterexample	64
6.5	Singular values expose the amplification	65
6.6	A matched normal comparison	65
6.7	The primary figure	66
6.8	The pseudospectrum	67
6.9	Exact pseudospectrum of the 2×2 example	68
6.10	Why pseudospectra matter	69
6.11	Transient growth in broader non-normal systems	70
6.12	What the Wolfram witness establishes	70
6.13	Five mistakes to avoid	71
6.13.1	Mistake 1: “Non-normal means unstable.”	71
6.13.2	Mistake 2: “Stable eigenvalues imply monotone decay.” . .	71
6.13.3	Mistake 3: “A transient spike proves divergence.”	71
6.13.4	Mistake 4: “Pseudospectra replace eigenvalues.”	71
6.13.5	Mistake 5: “Broad pseudospectra explain a neural failure.”	71
6.14	Atlas connections	71
6.15	Closing view	72
6.16	References used in this chapter	72
7	Krylov Subspaces and Iterative Solves	73
7.1	Why operator-generated directions matter	74
7.2	Arnoldi: general matrices	74
7.3	Lanczos: the symmetric specialization	74
7.4	Galerkin projection	75
7.5	Minimum residual is a different projection rule	75
7.6	Residual and error are related, not identical	75
7.7	Polynomial viewpoint	76
7.8	Exact two-dimensional witness	76
7.9	The same witness in an orthonormal symmetric basis	77
7.10	A conditioning control	78
7.11	Matrix-free access	79

7.12	Preconditioning changes the problem seen by the iteration	79
7.13	Finite precision	79
7.14	Restarting is a tradeoff	80
7.15	What this chapter does not claim	80
7.16	Downstream handoff	80
7.17	References used in this chapter	80
8	Probability, Information, and Statistical Structure	83
8.1	Information requires a probability model	83
8.2	Uncertainty budget, not semantic content	84
8.3	Random variables and distributions	84
8.4	Conditional probability	85
8.5	Expectation	85
8.6	Entropy	86
8.7	Joint and conditional entropy	86
8.8	Cross-entropy	87
8.9	KL divergence	87
8.10	Cross-entropy decomposition	88
8.11	Mutual information	88
8.12	Exact correlated-bit witness	89
8.13	Same entropy, different structure	89
8.14	Concentration	91
8.15	Why concentration matters for machine learning	91
8.16	Sufficient statistics	91
8.17	Exponential families	92
8.18	Bernoulli as an exponential family	92
8.19	Information geometry waits for later	93
8.20	Three overinterpretations to avoid	93
8.20.1	Entropy equals meaning	93
8.20.2	Mutual information equals causality	93
8.20.3	KL equals distance	94
8.21	Atlas connections	94
8.22	Closing view	94
8.23	References used in this chapter	95
9	Geometry of Constrained State Spaces	97
9.1	The space you write in is not always the space you may move in	97
9.2	Ambient space and admissible space	98
9.3	The unit sphere	99
9.4	Retractions: step locally, return legally	99
9.5	Retraction is not exponential map	100
9.6	The primary geometric plate	101
9.7	Geodesics and spherical distance	101
9.8	SLERP: interpolate on the sphere	103
9.9	Stiefel geometry: orthonormal frames	104
9.10	Grassmann geometry: the frame is not the subspace	105

9.11	Geometry changes what an optimizer is allowed to do	105
9.12	Four mistakes to avoid	106
9.12.1	Mistake 1: “The tangent space is the manifold.”	106
9.12.2	Mistake 2: “Normalization is the exponential map.” . . .	106
9.12.3	Mistake 3: “A Stiefel point and a Grassmann point are the same object.”	106
9.12.4	Mistake 4: “Local Euclidean behavior means the space is globally Euclidean.”	106
9.13	Atlas connections	106
9.14	Closing view	107
9.15	References used in this chapter	107
10	Flows, Stability, and Bifurcation	109
10.1	A state is not yet a dynamics	109
10.2	Law of motion versus film frames	110
10.3	Autonomous ordinary differential equations	110
10.4	Trajectory	111
10.5	Flow map	111
10.6	Exact flow of a linear system	112
10.7	Equilibrium	112
10.8	Stability is a neighborhood question	113
10.9	Linearization	113
10.10	Hyperbolic local stability	114
10.11	Zero eigenvalue is not “neutral stability”	114
10.12	Lyapunov functions	115
10.13	Negative semidefinite versus negative definite	115
10.14	Exact scalar stable flow	116
10.15	Phase portrait	117
10.16	Parameters can change qualitative behavior	117
10.17	Saddle-node bifurcation	118
10.18	Saddle-node stability	119
10.19	Pitchfork bifurcation	119
10.20	Pitchfork stability exchange	120
10.21	Why bifurcation language matters for learning	121
10.22	Hopf bifurcation: the minimal picture	121
10.23	Periodic orbit is not convergence	122
10.24	Conservative dynamics	122
10.25	Hamiltonian conservation	123
10.26	Harmonic oscillator	123
10.27	Symplectic structure	124
10.28	Energy conservation and symplecticity are not synonyms	125
10.29	Dissipative versus conservative pictures	125
10.30	Exact flow versus numerical step	126
10.31	Discretization can change stability	126
10.32	Discretization can change geometry	127
10.33	Depth as computational time	127

10.34	Flow-like architectures	128
10.35	Optimizer dynamics	128
10.36	Reinforcement-learning dynamics	129
10.37	Bifurcation as diagnostic hypothesis	129
10.38	Lyapunov functions as certificates	129
10.39	Four failure modes	130
10.39.1	Local linearization treated as global truth	130
10.39.2	Stable equilibrium confused with stable orbit	130
10.39.3	Hamiltonian structure asserted from oscillation alone	130
10.39.4	Numerical method identified with continuous flow	130
10.40	Atlas connections	130
10.41	What would falsify a dynamical interpretation?	131
10.42	Closing view	131
10.43	References used in this chapter	132
11	Discretization, Stability, and Splitting	133
11.1	The continuous system is not the algorithm	133
11.2	Map versus terrain	134
11.3	One-step methods	134
11.4	Local defect	135
11.5	Global error	135
11.6	Consistency is not enough	136
11.7	Explicit Euler	136
11.8	Explicit Euler local defect	137
11.9	Implicit Euler	137
11.10	Explicit versus implicit is not fast versus slow	137
11.11	The scalar test equation	138
11.12	Explicit Euler stability function	138
11.13	The explicit stability disk	139
11.14	Implicit Euler stability function	139
11.15	A-stability	140
11.16	L-stability	141
11.17	Stability is not accuracy	141
11.18	The stiff scalar witness	142
11.19	What stiffness means	143
11.20	Multiple timescales	143
11.21	Implicit methods move the stability boundary	143
11.22	Absolute stability is not nonlinear stability	144
11.23	From one vector field to two	144
11.24	Lie–Trotter splitting	145
11.25	Noncommutativity is the obstruction	145
11.26	Commuting subflows	146
11.27	Exact noncommuting witness	146
11.28	Lie local defect	147
11.29	Strang splitting	147
11.30	Exact Strang witness	148

11.31 Symmetry is doing mathematical work	148
11.32 Splitting is not arbitrary modularity	148
11.33 Splitting can preserve structure	149
11.34 Structure-preserving means name the structure	149
11.35 Backward error viewpoint	149
11.36 Step size is a modeling parameter	150
11.37 Depth as number of steps	150
11.38 Adaptive depth as error control	151
11.39 Commutator as split diagnostic	151
11.40 Stiff learned dynamics	151
11.41 Implicit computation	152
11.42 Four numerical failure modes	152
11.42.1 Small local defect, unstable propagation	152
11.42.2 Stable but inaccurate coarse steps	152
11.42.3 High-order formula outside assumptions	152
11.42.4 Structure-preserving label without structure	152
11.43 What would falsify a numerical interpretation?	153
11.44 Atlas connections	153
11.45 Closing view	153
11.46 References used in this chapter	154
12 Local-to-Global Mathematics	155
12.1 Local validity is not global coherence	156
12.2 Start with incidence	156
12.3 A graph is not a sheaf	156
12.4 Local sections	157
12.5 Restriction maps	157
12.6 Why restriction is an interface map	158
12.7 Matching families	158
12.8 Compatibility is typed	158
12.9 A presheaf gives the restriction machinery	159
12.10 A sheaf adds unique gluing	159
12.11 Existence and uniqueness answer different questions	159
12.12 The equalizer picture	159
12.13 Why this is a useful computational form	160
12.14 Exact witness: three points	160
12.15 Sections are functions	161
12.16 The global-to-local map	161
12.17 Uniqueness is injectivity	161
12.18 The overlap-discrepancy map	162
12.19 Image equals kernel	162
12.20 A compatible family	162
12.21 An incompatible family	163
12.22 The failure is localized	163
12.23 Gluing is stronger than aggregation	163
12.24 The graph view of the witness	163

12.25	Cellular sheaves make graph-local algebra explicit	164
12.26	We do not need cohomology yet	164
12.27	Pairwise compatibility needs the sheaf context	164
12.28	Local-to-global claims are model-relative	165
12.29	Exact consistency is brittle by design	165
12.30	Approximate consistency is a second layer	165
12.31	An approximate glue is not a sheaf-theoretic equality	166
12.32	Local truth is not required	166
12.33	Global consistency is not semantic adequacy	166
12.34	Local-to-global structure can expose hidden assumptions	167
12.35	Graphs provide the skeleton	167
12.36	Covers provide another skeleton	167
12.37	Interface data can be lower-dimensional	168
12.38	Compatibility can be transformed rather than literal identity	168
12.39	Coordinate changes fit naturally	168
12.40	Composition requires more than adjacency	169
12.41	Obstruction certificates are valuable outputs	169
12.42	A larger graph produces a discrepancy vector	169
12.43	Zero discrepancy can have several meanings	170
12.44	Richer interfaces detect more and cost more	170
12.45	Sheaf language should earn its keep	170
12.46	A practical local-to-global ledger	171
12.47	Failure modes	171
12.47.1	Graph-equals-sheaf	171
12.47.2	Local-equals-global	171
12.47.3	Pairwise-plausible-equals-matching	172
12.47.4	Matching-equals-truth	172
12.47.5	Approximate-equals-exact	172
12.47.6	Zero-obstruction-equals-complete-model	172
12.47.7	Sheaf-as-decoration	172
12.48	What the exact witness establishes	172
12.49	Downstream handoff	172
12.50	References used in this chapter	173

13 Representations and Invariants 175

13.1	A representation is not merely a vector	175
13.2	Languages that preserve the same message	176
13.3	Representation map	176
13.4	Coordinates versus represented structure	177
13.5	Exact recoding witness	178
13.6	Invariance	178
13.7	Equivariance	179
13.8	Reflection witness	179
13.9	Invariance and equivariance are not synonyms	181
13.10	Geometry enters representation	181
13.11	Information enters representation	181

13.12	Identifiability	182
13.13	Disentanglement needs assumptions	182
13.14	Sparse representations	183
13.15	Dictionaries and features	183
13.16	Superposition	183
13.17	Representation equivalence	184
13.18	Quotient thinking	184
13.19	Representation collapse	184
13.20	Identical task performance does not imply identical mechanism	185
13.21	The Residual question	185
13.22	Five failure modes	186
13.22.1	Coordinates mistaken for meaning	186
13.22.2	Invariance mistaken for universal virtue	186
13.22.3	Equivariance claimed without group action	186
13.22.4	Disentanglement claimed without assumptions	186
13.22.5	Toy superposition promoted to universal mechanism	186
13.23	Atlas connections	186
13.24	Closing view	187
13.25	References used in this chapter	187
14	Normalized and Hyperspherical Representations	189
14.1	Normalization is a modeling decision	189
14.2	Direction without volume	190
14.3	The unit hypersphere	191
14.4	The normalization map	191
14.5	Radial and tangential decomposition	192
14.6	Exact witness at (3, 4)	193
14.7	A singularity at zero	194
14.8	Norm erasure is information erasure	195
14.9	Angular similarity	195
14.10	Chord distance and angle	196
14.11	Sixty-degree witness	196
14.12	SLERP	197
14.13	Retraction	198
14.14	Angular learning rate	198
14.15	Hyperspherical contrastive representations	199
14.16	nGPT	199
14.17	Geometry does not choose the task for us	200
14.18	Normalization can relocate information	200
14.19	Collapse on the sphere	201
14.20	Antipodal ambiguity	201
14.21	Five failure modes	202
14.21.1	Radial information assumed irrelevant	202
14.21.2	Unit norm mistaken for non-collapse	202
14.21.3	Cosine similarity mistaken for complete similarity	202
14.21.4	Retraction mistaken for exponential map	202

14.21.5 nGPT evidence overgeneralized	202
14.22 Atlas connections	202
14.23 Closing view	203
14.24 References used in this chapter	203

15 Equivalence and Quotient Geometry 205

15.1 Different parameters can mean the same thing	205
15.2 Many addresses, one place	206
15.3 Equivalence relations	206
15.4 Group actions and orbits	207
15.5 Grassmann: the canonical example	207
15.6 Parameter equality versus functional equality	208
15.7 Hidden-unit permutation symmetry	209
15.8 Exact two-unit example	210
15.9 Positive-rescaling symmetry	210
15.10 Exact replay	212
15.11 Why quotienting is tempting	212
15.12 Vertical and horizontal directions	212
15.13 Local null directions	213
15.14 Hidden symmetries	214
15.15 Symmetry can affect learning dynamics	215
15.16 Quotienting removes declared redundancy	215
15.17 Redundancy removed does not mean convexity	216
15.18 Singular quotients	216
15.19 Function space is an even stronger quotient	216
15.20 Quotienting by the wrong relation	217
15.21 Gauge language	217
15.22 Representation equivalence revisited	217
15.23 The benign-nonconvexity question	218
15.24 A reconstruction test	218
15.25 Six failure modes	219
15.25.1 Parameter difference mistaken for functional difference	219
15.25.2 Quotient relation too coarse	219
15.25.3 Quotient relation too narrow	219
15.25.4 Regular quotient assumed globally	219
15.25.5 Flat direction assumed to be symmetry	219
15.25.6 Quotienting assumed to solve optimization	219
15.26 Atlas connections	219
15.27 Closing view	220
15.28 References used in this chapter	220

16 The Residual 221

16.1 What survives when representation changes?	221
16.2 The thing that cannot be transformed away	222
16.3 The message that survives translation	222
16.4 From representation equivalence to reconstruction	223

16.5	Declaring the problem	223
16.6	Transformation invariance	224
16.7	Capability sufficiency	225
16.8	The factorization preorder	225
16.9	Least, not merely small	226
16.10	Why leastness is useful	227
16.11	The behavior-table trap	228
16.12	Descriptor admissibility is part of the theorem statement	228
16.13	Classical sufficiency as a model	229
16.14	Minimal sufficient representations	229
16.15	Information bottleneck	230
16.16	A calibration toy model	230
16.17	Invariance of the toy Residual	231
16.18	Sufficiency of the toy Residual	231
16.19	Leastness in the toy model	231
16.20	Invariance without sufficiency	232
16.21	Sufficiency without minimality	233
16.22	Representation change	233
16.23	Exact recoding witness	234
16.24	Task dependence	235
16.25	Transformation dependence	236
16.26	Residual as equivalence class	236
16.27	Residual as operator structure	237
16.28	Residual as program	237
16.29	Exact versus approximate Residuals	237
16.30	Reconstruction is stronger than correlation	238
16.31	Intervention strengthens the test	238
16.32	Finite replay can fool us	239
16.33	Minimal curriculum	239
16.34	Minimal reasoning basis	239
16.35	Model interoperability	240
16.36	Memory	240
16.37	Benign nonconvexity	240
16.38	Governed adaptation	241
16.39	What would falsify a proposed Residual?	241
16.39.1	Invariance failure	241
16.39.2	Sufficiency failure	241
16.39.3	Leastness failure	241
16.39.4	Transfer failure	242
16.39.5	Intervention failure	242
16.39.6	Probe failure	242
16.40	What the Residual is not	242
16.41	Atlas connections	242
16.42	Closing view	243
16.43	References used in this chapter	243

17 Reconstruction and Transfer	245
17.1 Transfer is not reuse	245
17.2 Moving a machine through a narrow doorway	246
17.3 Standard source and target framing	246
17.4 Transfer needs a target criterion	247
17.5 Negative transfer is not paradoxical	248
17.6 Source accuracy is not transferability	248
17.7 Parameter reuse is weaker than capability transfer	249
17.8 Representation transfer	249
17.9 The common-Residual reconstruction certificate	250
17.10 What the certificate says	251
17.11 Exact linear witness	251
17.12 Exact numeric reconstruction	252
17.13 Two target tasks	253
17.14 Exact recoding is the easy case	253
17.15 A lossy bottleneck	255
17.16 Exact bottleneck failure	255
17.17 The fiber criterion	256
17.18 Why the fiber criterion is necessary	257
17.19 Why the fiber criterion is sufficient	257
17.20 One-task sufficiency is not task-family sufficiency	258
17.21 Universal representation quality is underspecified	258
17.22 Information bottleneck connection	259
17.23 Minimal sufficient representation connection	259
17.24 Transfer across domains	259
17.25 Transfer across tasks	260
17.26 Transfer across representation systems	260
17.27 Exact information preservation does not imply easy learning	260
17.28 Transfer can fail at the interface	261
17.29 Architecture transfer versus feature transfer	261
17.30 Fine-tuning changes the object	262
17.31 A reconstruction certificate is not a training algorithm	262
17.32 Reconstruction tests	263
17.33 What would falsify a transfer claim?	263
17.33.1 Sufficiency failure	263
17.33.2 Domain failure	263
17.33.3 Adapter failure	263
17.33.4 Baseline failure	263
17.33.5 Robustness failure	263
17.33.6 Attribution failure	263
17.34 Transferable Residual versus universal Residual	264
17.35 Minimal curricula	264
17.36 Minimal reasoning basis	264
17.37 Model interoperability	265
17.38 Memory and transfer	265
17.39 Compression as a transfer probe	265

17.40	Closing view	266
17.41	References used in this chapter	266
18	From Layered Networks to Residual Systems	267
18.1	Architecture is a choice of mathematical object	267
18.2	From assembly line to evolving state	268
18.3	Layered composition	268
18.4	Composition creates a product of Jacobians	269
18.5	The early object: a hierarchy of representations	269
18.6	Convolution replaces arbitrary connectivity with a structured operator	270
18.7	Exact translation equivariance in the declared model	270
18.8	Exact convolution witness	271
18.9	Practical CNNs can break exact equivariance	272
18.10	Recurrent systems change the object again	272
18.11	Parameter sharing across time	272
18.12	Exact linear recurrence	273
18.13	Exact recurrence witness	274
18.14	Long-term memory is not guaranteed by recurrence	274
18.15	LSTM: persistent state plus gates	275
18.16	Gates are operators on information flow	275
18.17	Encoder–decoder architecture introduces an explicit interface	275
18.18	The interface is a contract	276
18.19	Encoder information-loss boundary	276
18.20	Bottleneck is a design decision	277
18.21	Highway Networks: transform or carry	277
18.22	Highway identity limit	277
18.23	Exact highway witness	278
18.24	Gated carry is not residual addition	279
18.25	Residual learning	279
18.26	Residual Jacobian	279
18.27	Identity transport	280
18.28	Exact residual witness	280
18.29	Structural lineage plate	281
18.30	Residual systems invite a dynamical lens	281
18.31	The ODE analogy has conditions	282
18.32	Neural ODEs make continuous depth explicit	282
18.33	Continuous depth changes where approximation lives	283
18.34	Architecture as transport	283
18.35	Architecture can encode invariance	284
18.36	Architecture also creates failure modes	284
18.37	History is not a ranking	284
18.38	What changes in the object of analysis?	285
18.38.1	Layered networks	285
18.38.2	CNNs	285
18.38.3	RNNs and LSTMs	285

18.38.4	Encoder–decoders	285
18.38.5	Highway systems	285
18.38.6	Residual systems	286
18.38.7	Neural ODEs	286
18.39	Bridge to Transformers	286
18.40	Bridge to adaptive depth	286
18.41	Bridge to network numerics	287
18.42	What would falsify an architectural analogy?	287
18.43	Closing view	287
18.44	References used in this chapter	288
19	The Transformer as Baseline Object	289
19.1	We need one reference object	289
19.2	Attention is a component, not the architecture	290
19.3	The shared blackboard allegory	290
19.4	Residual-stream state	291
19.5	One attention head	291
19.6	Scores are not weights	292
19.7	The original Transformer	292
19.8	Multi-head attention	293
19.9	Exact two-head witness	293
19.10	Multi-head does not mean one monolithic attention matrix	294
19.11	Causal masking is architectural access control	294
19.12	Mathematical mask versus implementation mask	295
19.13	Exact three-token causal witness	295
19.14	Causal mask does not imply causal understanding	296
19.15	Position-wise feed-forward network	296
19.16	Different kinds of sharing	297
19.17	Residual transport	297
19.18	Exact residual identity witness	297
19.19	Layer Normalization	298
19.20	LayerNorm is not bookkeeping	299
19.21	Post-LN ordering	299
19.22	Pre-LN ordering	300
19.23	Pre-LN and post-LN are not notation variants	300
19.24	Baseline pre-LN block	300
19.25	Positional information is a separate axis	301
19.26	Encoder-only topology	301
19.27	Decoder-only topology	302
19.28	Encoder–decoder topology	302
19.29	Cross-attention is an interface	303
19.30	Three topologies, three dependency graphs	303
19.30.1	Encoder-only	303
19.30.2	Decoder-only	303
19.30.3	Encoder–decoder	303
19.31	Architecture versus training objective	303

19.32	Architecture versus optimizer	304
19.33	Architecture versus scale	304
19.34	Architecture versus tokenizer	304
19.35	The residual stream as the persistent state	305
19.36	But “residual stream” is not an ontological primitive	305
19.37	Attention as communication, FFN as local transformation	306
19.38	Multi-head factorization creates multiple projected interaction spaces	306
19.39	Masking changes the operator support	306
19.40	Normalization placement changes the update law	307
19.41	Residual identity path is necessary context for depth	307
19.42	The baseline object in one equation	307
19.43	What the baseline does not contain	308
19.44	Failure modes of vague Transformer language	308
19.44.1	“Attention model”	308
19.44.2	“Causal Transformer”	308
19.44.3	“Standard Transformer”	309
19.44.4	“Transformer layer”	309
19.45	Atlas connections	309
19.46	What would falsify a baseline claim?	309
19.47	Closing view	310
19.48	References used in this chapter	310
20	Depth as Computational Time	311
20.1	Five notions of time	312
20.2	Fixed depth	312
20.3	Depth is not automatically physical time	313
20.4	Recurrent depth	313
20.5	A computation can stop itself	314
20.6	An exact depth-as-error witness	315
20.7	Compute is more than step count	316
20.8	Equilibrium depth	317
20.9	Convergence is an architectural question only after assumptions	317
20.10	Conditional depth	318
20.11	Soft gates are not free skips	318
20.12	Effective depth is input-dependent execution history	319
20.13	Parameter depth and execution depth	320
20.14	Halting is a decision under uncertainty	320
20.15	More depth can make things worse	321
20.16	Budget exhaustion is a first-class outcome	321
20.17	Training and inference can disagree	322
20.18	What recurrent, adaptive, equilibrium, and conditional depth share	322
20.19	What the next chapters may assume	323
20.20	The boundary to remember	323
20.21	References used in this chapter	324

21 Split-Operator Networks	325
21.1 The smallest place order can matter	326
21.2 A telescope with two lenses	327
21.3 The exact reference problem	327
21.4 The exact two-by-two witness	329
21.5 The combined flow is not either Lie step	330
21.6 Symmetry cancels the quadratic defect	331
21.7 A commuting control	331
21.8 From exact flows to neural sublayers	332
21.9 Normalization belongs inside the submap	333
21.10 Symmetry is not exact reversibility	334
21.11 Shared operators and changing operators	334
21.11.1 Shared operators	335
21.11.2 Layer-varying operators	335
21.12 Splitting error is only one error	335
21.13 Ordering as an architectural degree of freedom	336
21.14 A Transformer block through the split lens	336
21.15 What a neural commutator could mean	337
21.16 Failure modes	338
21.16.1 Calling every residual block an integrator	338
21.16.2 Treating Strang order as architectural magic	338
21.16.3 Ignoring normalization	338
21.16.4 Confusing symmetry and invertibility	338
21.16.5 Measuring one local commutator and claiming global control	339
21.16.6 Folding training error into splitting error	339
21.16.7 Assuming shared-operator formulas for layer-varying net- works	339
21.17 What later chapters may now assume	339
21.18 Epistemic status	339
21.19 References used in this chapter	340
22 Representation as Transport	341
22.1 The primary object is a sequence of maps	342
22.2 Residual transport	343
22.3 Continuous depth is a different declaration	344
22.4 Stage index is not automatically physical time	344
22.5 The constrained case	345
22.6 Feasible motion is not geodesic motion	346
22.7 Moving the point and moving an arrow are different operations .	347
22.8 Exact tangent-ownership witness	348
22.9 The local-compass allegory	349
22.10 Path order can matter even when every endpoint is feasible . . .	349
22.11 The executed maps determine the path	351
22.12 State-dependent transport	351
22.13 Differential pushforward is not automatically vector transport .	352
22.14 Transport can lose information	352

22.15	Transport can also relocate information	353
22.16	Shared and stage-dependent transport	353
22.17	Stability is a separate question	354
22.18	Invertibility is also separate	355
22.19	This is not optimal transport	355
22.20	This is not parallel transport	356
22.21	This is not vector transport either	356
22.22	Transport and split operators	356
22.23	Transport and residual identity paths	357
22.24	Transport as an interface contract	358
22.25	Handoff to Neural Krylov Transport	358
22.26	Five distinctions to keep visible	359
22.26.1	State transport versus tangent transport	359
22.26.2	Feasibility versus geodesic exactness	359
22.26.3	Discrete composition versus continuous flow	359
22.26.4	Constraint preservation versus information preservation	359
22.26.5	Order versus ingredients	359
22.27	Failure modes	359
22.28	Closing view	360
22.29	References used in this chapter	360
23	Attention as an Operator	361
23.1	From an attention picture to an attention action	361
23.2	Standard scaled dot-product attention	362
23.3	Why “operator” is the right word	363
23.4	Softmax gives a row-stochastic mixing stage	363
23.5	The full self-attention map is nonlinear	364
23.6	A small operator we can inspect completely	365
23.7	The operator changes when the query changes	366
23.8	Permutation equivariance before position enters	367
23.9	A mask is a constraint on the operator family	368
23.10	Multihead attention is a family of state-dependent operators	368
23.11	The kernel lens	369
23.12	Feature maps and linear attention	370
23.13	Attention sensitivity has a concrete Jacobian	370
23.14	Three distinctions that must remain visible	371
23.14.1	Score matrix versus mixing operator	371
23.14.2	Mixing operator versus full head	371
23.14.3	Operator versus explanation	371
23.15	Why this viewpoint matters for position	372
23.16	What the computational witness establishes	372
23.17	Atlas connections	372
23.18	Closing view	373
23.19	References used in this chapter	373
24	Approximate and Structured Attention	375

24.1	The attention stack	376
24.2	Five different approximation statements	377
24.2.1	Score approximation	377
24.2.2	Kernel approximation	377
24.2.3	Operator approximation	377
24.2.4	Output approximation	377
24.2.5	Alternative operator	377
24.3	Why raw kernel error can mislead	378
24.4	Kernel error needs denominator control	379
24.5	Positive random features	380
24.6	From an expectation identity to Performer	381
24.7	Why feature attention can avoid the dense matrix	381
24.8	Low-rank attention is another idea	382
24.9	Low rank is a hypothesis, not a universal property	382
24.10	Landmark reconstruction: Nyströmformer	383
24.11	Conditioning belongs in the landmark story	384
24.12	Sparse attention can mean a different normalization	384
24.13	An exact sparse-support witness	385
24.14	Sparsity and low rank are independent	386
24.15	Exact sparsity is not yet a complexity theorem	386
24.16	Three error surfaces	387
24.16.1	Kernel error	387
24.16.2	Operator error	387
24.16.3	Output error	387
24.17	A small exact operator approximation	388
24.18	The same operator error can look smaller after acting on values	389
24.19	Task performance is a fourth question	389
24.20	Complexity must expose the approximation budget	390
24.21	Accuracy-cost curves are more informative than single points	390
24.22	Masks must remain in the target definition	391
24.23	Position changes what is being approximated	392
24.24	Finite precision is not an afterthought	392
24.25	A useful comparison table in words	392
24.26	The Krylov analogy, used carefully	393
24.27	What not to infer	394
24.27.1	Small kernel error does not automatically give small operator error	394
24.27.2	Small output error on one value field does not give small operator error	394
24.27.3	Low rank does not imply sparsity	394
24.27.4	Sparsity does not imply low rank	394
24.27.5	Exact zeros do not imply fast execution	394
24.27.6	Few random features do not imply uniform accuracy	394
24.27.7	Good benchmark performance does not imply softmax fidelity	394
24.28	A practical evaluation protocol	395

24.29	Research bridges	395
24.29.1	Spectral diagnostics	395
24.29.2	Context compression	395
24.29.3	Relative-position operators	395
24.29.4	Systems	395
24.29.5	Adaptive approximation	396
24.30	What the computational witness establishes	396
24.31	Closing view	396
24.32	References used in this chapter	397
25	The Geometry of Position	399
25.1	Position is not content	399
25.2	Additive sinusoidal position	400
25.3	Rotary position moves the geometry into query and key space . .	400
25.4	What the identity actually says	401
25.5	An exact same-offset witness	401
25.6	Why the rotation structure matters	402
25.7	Multi-frequency position	403
25.8	Position as phase	403
25.9	Common translation	404
25.10	Multidimensional position	404
25.11	Position and attention support are separate	405
25.12	Exact algebra does not guarantee long-context generalization . .	405
25.13	Direct extrapolation	406
25.14	Position Interpolation	406
25.15	YaRN	407
25.16	Frequency scaling and index scaling are different interventions . .	407
25.17	The geometry of extrapolation	407
25.18	Failure modes	408
25.18.1	Additive equals rotary	408
25.18.2	Relative identity equals long-context guarantee	408
25.18.3	Any position transform works	408
25.18.4	Frequency schedule equals context policy	408
25.18.5	2D means two copies of 1D	409
25.18.6	Interpolation equals extrapolation	409
25.18.7	Position explains all long-context behavior	409
25.19	Downstream handoff	409
25.20	References used in this chapter	409
26	Relative-Position Operators	411
26.1	Two spaces, two frequency notions	411
26.2	Relative position as an operator over sequence index	412
26.3	Why the Fourier basis appears	412
26.4	The DC mode	413
26.5	RoPE can have a feature-space DC block too	413
26.6	Low-dimensional operator reduction	414

26.7	Relative position in actual attention architectures	415
26.8	Relative bias is not full attention	415
26.9	Head-specific positional mode profiles	416
26.10	Exact two-head witness	416
26.11	One head is DC-dominant	417
26.12	The other head is Nyquist-dominant	417
26.13	Singular values lose mode identity	418
26.14	Head specialization needs a qualifier	418
26.15	The DC component is not always desirable	418
26.16	Boundary conditions matter	418
26.17	Bucketing is an operator approximation too	419
26.18	Frequency truncation can be head-specific	419
26.19	Multi-frequency RoPE and sequence modes should not be collapsed	419
26.20	What a positional-operator diagnostic should record	420
26.21	Handoff to Latent Clocks	420
26.22	Durable lesson	421
26.23	References used in this chapter	421

27 Latent Clocks 423

27.1	The handoff from positional operators	424
27.2	The handoff from Dynamics	424
27.3	Two observed streams	425
27.4	Dynamic time warping	425
27.5	The path is monotone	426
27.6	A local discrepancy	426
27.7	Dynamic programming	426
27.8	The exact warped witness	427
27.9	The local-cost matrix	427
27.10	There are 25 admissible paths	428
27.11	The unique zero-cost alignment	428
27.12	Observed index has separated from alignment phase	429
27.13	A common alignment coordinate	430
27.14	Why “clock” is a dangerous word	430
27.15	Identity/no-warp control	430
27.16	The diagonal is uniquely optimal	431
27.17	The control prevents a false lesson	431
27.18	Constraints are part of the model	431
27.19	A unique optimum is still model-relative	432
27.20	Multimodal alignment changes the comparison problem	432
27.21	Generalized time warping	433
27.22	Asynchrony is not one phenomenon	433
27.23	Positional frequency is not a clock	433
27.24	RoPE frequency is not elapsed time either	433
27.25	Discrete alignment is not a continuous trajectory	434
27.26	Continuous-time latent clocks require more structure	434
27.27	Alignment is not causality	435

27.28	Low cost is not semantic identity	435
27.29	Non-unique alignments should stay non-unique	435
27.30	Partial and missing-data alignment	436
27.31	A useful latent-clock protocol	436
27.32	When the alignment coordinate is useful	436
27.33	Atlas non-implications	437
27.34	Atlas connections	437
27.35	Closing view	438
27.36	References used in this chapter	438
28	First-Order Optimization	439
28.1	The gradient is not the optimizer	439
28.2	Steering versus changing the road	440
28.3	Gradient descent	441
28.4	Exact quadratic calibration	441
28.5	Step-size stability	442
28.6	Curvature sets the useful scale	442
28.7	Full gradient versus stochastic gradient	444
28.8	Exact unbiased-SGD witness	444
28.9	Noise can be useful and harmful	445
28.10	Momentum adds state	446
28.11	Momentum conventions differ	446
28.12	Nesterov-style lookahead	447
28.13	Deep-learning momentum	447
28.14	AdaGrad	447
28.15	Adaptive scaling as a diagonal geometry	448
28.16	RMSPProp	448
28.17	Adam	449
28.18	Bias correction	449
28.19	Exact Adam first step	449
28.20	Adaptive methods are not one algorithm	450
28.21	Gradient clipping	451
28.22	Exact clipping geometry	451
28.23	Clipping changes the estimator	452
28.24	Weight decay and L2 regularization	452
28.25	Coupled regularization under adaptive scaling	453
28.26	Decoupled weight decay	453
28.27	Exact decay counterexample	453
28.28	Learning-rate schedules	454
28.29	Warmup	455
28.30	Initialization is part of optimization	455
28.31	Fan-in second moment	455
28.32	Linear second-moment preservation	456
28.33	ReLU second moment	456
28.34	Second moment is not variance after ReLU	457
28.35	Optimization is a state machine	457

28.36	Coordinate scaling already hints at geometry	458
28.37	Spectral effects are still hidden	458
28.38	Five failure modes	458
28.38.1	Large step mistaken for fast learning	458
28.38.2	Unbiased mistaken for low variance	459
28.38.3	Clipping mistaken for harmless rescaling	459
28.38.4	Coupled L2 mistaken for AdamW decay	459
28.38.5	Initialization heuristic mistaken for distribution theorem	459
28.39	What would falsify an optimizer story?	459
28.40	Atlas connections	459
28.41	Closing view	460
28.42	References used in this chapter	461

29 Curvature and Second-Order Structure 463

29.1	The gradient tells you a direction; curvature tells you how that direction changes	463
29.2	Local quadratic model	464
29.3	Newton's equation	464
29.4	Positive-definite curvature gives a descent direction	465
29.5	Exact positive-definite witness	465
29.6	Indefinite curvature breaks the simple Newton story	466
29.7	Curvature signatures	467
29.8	Damped and regularized Newton systems	468
29.9	Trust regions: constrain where the model is trusted	468
29.10	Exact trust-region control on the indefinite witness	469
29.11	Exact Hessians can be too large to store	470
29.12	Matrix-free second-order structure	470
29.13	Fisher information is a different matrix	471
29.14	Natural gradient	471
29.15	Exact metric witness	472
29.16	Hessian versus Fisher	472
29.17	Quasi-Newton methods	472
29.18	BFGS and curvature conditions	473
29.19	Secant replay	473
29.20	Proximal structure is not "more curvature"	474
29.21	Soft thresholding witness	474
29.22	Proximal gradient	475
29.23	What second-order methods actually buy	475
29.24	Local models and global optimization	476
29.25	Failure modes	476
29.25.1	Hessian equals Fisher	476
29.25.2	Natural gradient equals Newton	476
29.25.3	Invertible Hessian means safe Newton step	476
29.25.4	Trust region equals clipping	477
29.25.5	Quasi-Newton equals exact curvature	477
29.25.6	HVP means Hessian is cheap	477

29.25.7 Proximal method is second-order	477
29.25.8 Positive local curvature means global convexity	477
29.25.9 Exact local model means global optimum	477
29.26 Downstream handoff	477
29.27 References used in this chapter	478
30 Matrix-Aware Optimization	479
30.1 A matrix parameter is not merely a bag of scalars	479
30.2 Three ways to change a matrix gradient	480
30.3 Left and right multiplication	480
30.4 Shampoo: accumulated matrix geometry	480
30.5 Rectangular matrices change the algebra	481
30.6 The polar factor	482
30.7 Polar orthogonalization is singular-value flattening	482
30.8 “Steepest” depends on the norm	483
30.9 Why their local decreases are not optimizer scores	484
30.10A one-step inverse-root identity	484
30.11Muon adds state and numerical approximation	484
30.12Exact versus approximate matrix functions	485
30.13The exact 3×2 witness	485
30.14Polarizing the witness	486
30.15A diagonal control	486
30.16Flattening is not the whole spectral story	487
30.17Matrix-aware does not mean manifold-constrained	487
30.18Matrix-aware does not mean full second order	488
30.19Optimizer state is part of the trajectory	488
30.20What this chapter establishes	489
30.21Failure boundaries	489
30.22References used in this chapter	490
31 Optimization on Manifolds	491
31.1 Optimization changes when legal states form a curved space . . .	491
31.2 Tangent directions are local legal velocities	492
31.3 The gradient depends on the metric	493
31.4 Embedded manifolds and tangent projection	493
31.5 Sphere geometry	494
31.6 Tangent projection is not the update	494
31.7 Retractions	495
31.8 Sphere normalization as a retraction	495
31.9 Exact sphere witness	496
31.10Retraction versus exponential map	497
31.11The Stiefel manifold	497
31.12Stiefel tangent projection	498
31.13Polar retraction on the Stiefel manifold	498
31.14Exact 2×2 witness	499
31.15Orthogonal does not mean geodesically exact	500

31.16	A generic first-order manifold step	500
31.17	Momentum creates a tangent-space problem	501
31.18	Intrinsic and extrinsic viewpoints	501
31.19	Constraint preservation is not optimization quality	501
31.20	Stationarity on a manifold	502
31.21	Sphere versus Stiefel	502
31.22	Stiefel versus Grassmann	503
31.23	Why parameterization matters	503
31.24	Geometry can change steepest descent	504
31.25	Retraction choice can matter	504
31.26	Step size still matters	504
31.27	Relation to normalized models	505
31.28	Relation to variational optimization	505
31.29	Durable boundaries	506
31.30	References used in this chapter	506

32 Spectral Shaping 507

32.1	The handoff from Matrix-Aware Optimization	508
32.2	The handoff from Non-normality	508
32.3	A general shaping map	508
32.4	Exact witness matrix	509
32.5	Scalar normalization changes scale	509
32.6	Clipping can change conditioning	510
32.7	Polar flattening is stronger	510
32.8	A non-flat target is another operation	510
32.9	The four maps are not synonyms	511
32.10	Better condition number is not automatically better optimization	511
32.11	Flat is not automatically best	512
32.12	Conditioning is only one objective	512
32.13	Rank matters	512
32.14	Singular vectors matter too	513
32.15	Non-normality supplies the dynamical warning	513
32.16	Same eigenvalues, different one-step response	513
32.17	Pseudospectra separate them further	514
32.18	Why that matters for optimizer updates	515
32.19	Stateful optimizers enlarge the object	515
32.20	Spectral shaping is not optimizer-state dynamics	515
32.21	Better conditioning is not convergence	516
32.22	Task quality is another layer	516
32.23	A practical reporting protocol	516
32.24	What a flat spectrum actually certifies	517
32.25	What a non-flat target actually certifies	517
32.26	Atlas non-implications	517
32.27	Atlas connections	518
32.28	Closing view	518
32.29	References used in this chapter	518

33 Optimizer-State Dynamics	521
33.1 The optimizer is part of the state	521
33.2 Heavy-ball momentum as a state-space map	522
33.3 The characteristic equation belongs to the coupled system	523
33.4 A stable example with transient amplification	524
33.5 Four stable steps can amplify by more than $2.6\times$	525
33.6 The Wolfram plate	525
33.7 Why the augmented state matters	526
33.8 Curvature is not the same as optimizer dynamics	527
33.9 From an autonomous toy system to time-varying training	527
33.10 Adam makes the state richer	528
33.11 Optimization as a dynamical and control system	529
33.12 What a local Jacobian can and cannot tell us	529
33.13 The flywheel allegory, and where it fails	530
33.14 A handoff to Coupling-Phase Spectroscopy	530
33.15 Five mistakes to avoid	531
33.15.1 Mistake 1: “The Hessian is the optimizer dynamics.” . . .	531
33.15.2 Mistake 2: “Eigenvalues inside the unit disk mean every perturbation shrinks every step.”	531
33.15.3 Mistake 3: “A transient spike proves divergence.”	531
33.15.4 Mistake 4: “A local linearization predicts the whole train- ing run.”	531
33.15.5 Mistake 5: “Optimizer state is implementation detail.” . .	531
33.16 Atlas connections	531
33.17 Closing view	532
33.18 References used in this chapter	532
34 Coupling-Phase Spectroscopy	533
34.1 The object is the augmented training state	534
34.2 What is spectroscopy here?	534
34.3 The behavioral transition is defined elsewhere	535
34.4 Current GCL state changed since the first Atlas source lock . . .	535
34.5 What GSD has actually established	535
34.6 What GSD-WP04 asks	536
34.7 Why there is no WP04 signal result yet	536
34.8 Blocked is not negative	537
34.9 The Atlas can still formalize the protocol	537
34.10 Return to the exact momentum Jacobian	538
34.11 One invariant is constant	538
34.12 Four synthetic windows	539
34.13 Discovery comes before heldout evaluation	539
34.14 Exact heldout replay	540
34.15 Why one spectral scalar is not enough	540
34.16 Same spectral radius, different local maps	541
34.17 This is why CPS is multi-coordinate	541
34.18 Local Jacobians remain local	542

34.19	Retrospective description is the weakest positive CPS result . . .	542
34.20	Prediction requires temporal and selection discipline	542
34.21	Discovery must be separated from confirmation	543
34.22	A real CPS heldout protocol	543
34.23	What counts as PREDICTIVE_SIGNAL?	544
34.24	What counts as DESCRIPTIVE_ONLY?	544
34.25	What counts as NO_SIGNAL?	544
34.26	What counts as an artifact block?	545
34.27	Prediction is not cause	545
34.28	Causal promotion needs intervention	546
34.29	The causal hierarchy	546
34.29.1	C0 — behavioral transition	546
34.29.2	C1 — local diagnostic movement	546
34.29.3	C2 — heldout prediction	546
34.29.4	C3 — intervention sensitivity	546
34.29.5	C4 — mechanism-specific support	546
34.30	Current GSD is at C0 for CPS	547
34.31	One transition cannot validate a general predictor	547
34.32	Small model does not silently become large model	547
34.33	Artifact completeness is part of the experiment	548
34.34	Why exact identity matters	548
34.35	Time-varying products matter	548
34.36	JVP/VJP sketches can make the probe tractable	549
34.37	Training time is an important baseline	549
34.38	Parameter-only baselines matter	549
34.39	Stable capability controls matter too	550
34.40	Negative CPS evidence is useful	550
34.41	What the current chapter establishes	550
34.42	Non-implications	551
34.43	Atlas connections	551
34.44	Closing view	552
34.45	References used in this chapter	552

35 Variational and Divergence-Derived Optimization 553

35.1	Two inherited firewalls	554
35.2	Why divergences enter optimization	554
35.3	Divergence is not distance	555
35.4	Local geometry from a Hessian	555
35.5	Geometry can define an update	556
35.6	Euclidean descent is one special case	556
35.7	An entropic coordinate system	557
35.8	Divergence-derived does not mean descending	557
35.9	From geometry to dynamics	558
35.10	Bregman Lagrangian	559
35.11	Continuous flow still is not the algorithm	559
35.12	Discrete action	559

35.13	Why derive the discrete rule this way?	560
35.14	Exact oscillator construction	560
35.15	Discrete Euler-Lagrange equation	561
35.16	Momentum lift	561
35.17	Exact map	562
35.18	Symplectic is a precise property	562
35.19	Exact energy counterexample	563
35.20	Exact objective counterexample	563
35.21	Optimization and mechanics use different objectives	564
35.22	Manifold geometry re-enters	564
35.23	MODULUS as programme evidence	565
35.24	The exact MODULUS pipeline	565
35.25	Exact programme witness	566
35.26	The target-angle subtlety	566
35.27	Why this distinction matters	567
35.28	MODULUS is not being promoted to general theory	567
35.29	Geometry-derived is broader than variational	567
35.29.1	Divergence-derived	568
35.29.2	Variational	568
35.29.3	Constraint-geometric	568
35.30	A variational derivation is not a convergence certificate	568
35.31	A divergence is not automatically the right geometry	568
35.32	A symplectic optimizer would still need an optimization story	569
35.33	Bregman-Lagrangian acceleration remains source-scoped	569
35.34	The final architecture lesson	569
35.35	Evidence classes	570
35.36	Durable non-implications	570
35.37	Closing the architecture layer	571
35.38	References used in this chapter	571
36	Networks as Numerical Schemes	573
36.1	From architecture to update map	574
36.2	The bridge must be declared	575
36.3	Residual architecture is not a unique ODE	575
36.4	Autonomous versus nonautonomous depth	576
36.5	A layer is not yet a time step	576
36.6	Local truncation error requires an exact flow	577
36.7	Small residual does not mean small truncation error	577
36.8	Global error is not layerwise error added up	578
36.9	More layers do not automatically mean finer discretization	579
36.10	The scalar test equation enters neural depth	579
36.11	A stable continuous mode can become an unstable network step	580
36.12	Numerical stability is not training stability	581
36.13	The residual Jacobian is a perturbation propagator	581
36.14	Exact finite witness: same flow, different discretization behavior	582
36.15	Refinement is a family, not a single network	583

36.16	A modified-equation viewpoint	583
36.17	Residual networks and Neural ODEs are related but different . . .	584
36.18	The discretization can become part of the model	584
36.19	Reversibility has three meanings	585
36.19.1	Map invertibility	585
36.19.2	Computational reversibility	585
36.19.3	Time reversibility of a numerical method	585
36.20	Explicit Euler is invertible and not time reversible	585
36.21	Exact reversibility witness	586
36.22	Why this matters for reversible neural networks	587
36.23	Stability also has several meanings	587
36.24	Stiffness in networks needs a declared dynamical problem	588
36.25	The numerical lens suggests architecture questions	588
36.26	What Haber–Ruthotto adds	589
36.27	What Lu et al. adds	589
36.28	Architecture analogy versus numerical theorem	589
36.29	A practical numerical-network contract	590
36.30	Failure modes	591
36.30.1	Residual-equals-Euler	591
36.30.2	Depth-equals-time	591
36.30.3	Small-residual-equals-small-error	591
36.30.4	Stable-forward-equals-stable-training	591
36.30.5	Invertible-equals-reversible-integrator	591
36.30.6	More-layers-equals-convergence	591
36.31	Downstream handoff: Adaptive Depth	591
36.32	Downstream handoff: Boundary Probes	592
36.33	What this chapter establishes	592
36.34	References used in this chapter	592

37 Adaptive Depth as Error Control 595

37.1	The four control surfaces	596
37.1.1	Execution depth	596
37.1.2	Numerical step size	596
37.1.3	Local error estimate	596
37.1.4	Learned halting signal	597
37.2	What adaptive depth already gives us	597
37.3	Numerical error control is more specific	597
37.4	Embedded formulas	598
37.5	Estimator is not oracle	599
37.6	A one-step exact witness	599
37.7	Rejecting a step	601
37.8	Shrinking the step	601
37.9	Error control is a feedback loop	602
37.10	A learned halting signal	603
37.11	Learned stopping is not numerical stopping	603
37.12	Perfect ranking is still not calibration	604

37.13	An exact premature-halting witness	605
37.14	The missing map	606
37.15	Rank quality versus magnitude semantics	607
37.16	Expected calibration may still be too weak	607
37.17	Local error is not global error	608
37.18	More depth is not always refinement	608
37.19	Adaptive depth versus adaptive step size	609
37.19.1	Adaptive execution depth	609
37.19.2	Adaptive time stepping	609
37.20	A learned error estimator	610
37.21	Error control as an interface contract	610
37.21.1	State	610
37.21.2	Target error	610
37.21.3	Estimator	611
37.21.4	Relation	611
37.21.5	Tolerance	611
37.21.6	Controller	611
37.21.7	Budget	611
37.22	Budget exhaustion remains a separate state	611
37.23	Error control is not compute optimality	612
37.24	Spatially adaptive computation	613
37.25	Neural error indicators	613
37.26	Residuals are promising but not universal	614
37.27	Fixed-point stopping	614
37.28	Halting as decision making	615
37.29	A hierarchy of stopping semantics	615
37.29.1	Heuristic stop score	615
37.29.2	Empirically predictive score	615
37.29.3	Calibrated expected error	615
37.29.4	High-probability error control	616
37.29.5	Deterministic certified error bound	616
37.30	Why embedded pairs matter conceptually	616
37.31	Error estimators can fail outside their regime	616
37.32	Adaptive computation and distribution shift	617
37.33	Acceptance and rejection should be explicit	617
37.34	A minimal controller grammar	617
37.35	Failure modes	618
37.35.1	Halting score as error	618
37.35.2	Rank as magnitude	618
37.35.3	Local as global	618
37.35.4	Depth as mesh	618
37.35.5	Budget as convergence	618
37.35.6	Spatial depth as adaptive mesh refinement	618
37.35.7	Error control as optimal compute	619
37.36	Research programme	619
37.36.1	Learned a posteriori estimators	619

37.36.2 Two-resolution neural computation	619
37.36.3 Conformal stopping	619
37.36.4 Robust halting under shift	619
37.36.5 Resource-aware error control	619
37.37 Closing view	619
37.38 References used in this chapter	620

38 Neural Krylov Transport 621

38.1 What comes from classical Krylov theory	622
38.2 What comes from representation transport	622
38.3 The local-solve interface	622
38.4 The representation update is separate	623
38.5 A fixed left-preconditioned exact witness	623
38.6 Why the preconditioner matters	624
38.7 One operator-generated direction	624
38.8 The transformed residual is nonzero	625
38.9 The original residual is different	625
38.10 The true error is another object	626
38.11 One more direction changes everything in this toy	626
38.12 The exact solution is in the second Krylov space	627
38.13 What the positive witness actually shows	627
38.14 Short horizon can be computationally meaningful	627
38.15 A low-dimensional failure control	628
38.16 The best one-dimensional residual can still be poor	628
38.17 Dimension is not convergence	628
38.18 Conditioning and spectral structure still matter	629
38.19 Task quality can disagree with solve quality	629
38.20 Yet the one-step solve is still wrong	630
38.21 The converse also needs proof	630
38.22 Representation quality is another layer	630
38.23 The local Jacobian bridge	631
38.24 Local Jacobian is not global transport	631
38.25 Learned preconditioners make the distinction sharper	632
38.26 Fixed left versus right versus flexible	632
38.27 Matrix-free access fits the architecture	632
38.28 Matrix-free does not mean operator-free	633
38.29 Residual and error need conditioning	633
38.30 Preconditioned residual adds another layer	634
38.31 Finite precision still exists inside neural systems	634
38.32 Restarting and truncation are architectural choices	634
38.33 A Neural Krylov block as an interface	635
38.34 Diagnostics should remain plural	635
38.35 The right empirical question	636
38.36 The right theoretical question	636
38.37 Non-implications	636
38.38 Atlas connections	637

38.39	Closing view	637
38.40	References used in this chapter	638
39	Boundary Probes	639
39.1	Boundary object	639
39.2	Local linearization	640
39.3	Why derivative products matter	640
39.4	JVP	641
39.5	JVP versus finite differences	641
39.6	VJP	641
39.7	Pairing identity	642
39.8	Chain rule at an interface	642
39.9	Local Euclidean gain	642
39.10	Local does not mean global	642
39.11	Power iteration as a probe	643
39.12	JVP and VJP implement the power operator	643
39.13	Estimator and quantity are different objects	644
39.14	Initialization matters	644
39.15	Finite Rayleigh estimates can be low	644
39.16	Exact witness map	645
39.17	Exact singular structure	645
39.18	Exact JVP and nonlinear remainder	645
39.19	Exact VJP	646
39.20	Exact power matrix	646
39.21	A mixed start converges	646
39.22	The convergence has a closed form	647
39.23	A different start misses the dominant direction	647
39.24	One run is not a certificate	648
39.25	Restarts and subspace methods	648
39.26	Probe metadata	648
39.27	Boundary-probe contract	648
39.28	The perturbation class matters	649
39.29	Norm choice matters	649
39.30	Units precede numerical norms	649
39.31	Relation to NETNUM sensitivity	650
39.32	Composite interfaces	650
39.33	Small sensitivity is not semantic adequacy	650
39.34	Large sensitivity is not automatically failure	651
39.35	VJP magnitude is not causal attribution	651
39.36	Probe ensembles	651
39.37	Probe freshness	652
39.38	Operating-region coverage	652
39.39	Acceptance thresholds are policy	652
39.40	Numerical versus semantic conditions	652
39.41	Failure modes	653
39.41.1	Local-to-global promotion	653

39.41.2 Estimate-to-certificate promotion	653
39.41.3 Initialization erasure	653
39.41.4 Norm erasure	653
39.41.5 Semantic laundering	653
39.41.6 VJP-as-causality	653
39.41.7 Stale evidence	653
39.42 Practical probe ledger	653
39.43 What the witness establishes	654
39.44 Downstream handoff	654
39.45 References used in this chapter	655

40 Boundary Contracts 657

40.1 Composition fails at boundaries	657
40.2 The engineered flange	658
40.3 A provisional contract object	658
40.4 Interface signature versus boundary contract	659
40.5 Shape-compatible, semantically incompatible	660
40.6 Four obligation classes	661
40.6.1 Semantic obligations	661
40.6.2 Geometric obligations	661
40.6.3 Differential obligations	662
40.6.4 Numerical obligations	663
40.7 The chain rule is the first composition law	663
40.8 An exact two-module witness	664
40.9 Exact local amplification	665
40.10 The boundary figure	666
40.11 Power iteration as a boundary probe	667
40.12 Estimation is not certification	668
40.13 Separator variables	668
40.14 Local-to-global reasoning	669
40.15 Individually acceptable pieces can fail together	670
40.16 Public GCL project state	670
40.17 The GCL working slogan, properly bounded	671
40.18 Six failure modes	672
40.18.1 Shape without meaning	672
40.18.2 Geometry without preservation	672
40.18.3 Local sensitivity without global scope	672
40.18.4 Compressed boundary without sufficiency	672
40.18.5 Numerical tolerance without conditioning	672
40.18.6 Contract without support	672
40.19 What a mature contract might carry	672
40.20 Atlas connections	673
40.21 Closing view	673
40.22 References used in this chapter	674

41 Composition Without Catastrophe 675

41.1	Components do not compose by optimism	676
41.2	A composition has obligations of its own	676
41.3	The first composition law is the chain rule	677
41.4	The exact two-component witness	677
41.5	Chronological order matters	678
41.6	Product order is not chronological prose	679
41.7	The pair is genuinely noncommuting	680
41.8	The product norm bound can be loose	680
41.9	A commuting control	681
41.10	The lesson of the control	681
41.11	Error budgets also compose	682
41.12	The connecting domain is load-bearing	683
41.13	A finite error-budget failure	683
41.14	This is not the same as noncommutativity	684
41.15	Longer chains	685
41.16	Uniform local budgets can still grow badly	685
41.17	Four levels of certificate	686
41.17.1	Component certificate	686
41.17.2	Interface compatibility certificate	687
41.17.3	Composition certificate	687
41.17.4	System-level guarantee	687
41.18	Local does not mean global	687
41.19	Interface compatibility is not system safety	688
41.20	Shape compatibility is the weakest case	688
41.21	Invariants can fail at the boundary	688
41.22	Semantic mismatch and numerical error can interact	689
41.23	Order can change what the next module sees	689
41.24	Classical splitting results stay bounded	689
41.25	JVP/VJP probes remain local evidence	690
41.26	Conditioning remains a separate question	690
41.27	Feedback changes the problem	691
41.28	Repeated local validity can still miss reachability	691
41.29	Budget accounting should be explicit	692
41.30	Conservative bounds are not failures	692
41.31	Tightness can be order-dependent	693
41.32	A practical composition protocol	693
41.33	A practical order test	693
41.34	A practical error test	694
41.35	A practical chain test	694
41.36	What counts as catastrophe here?	694
41.37	What this chapter proves	695
41.38	What this chapter does not prove	696
41.39	Non-implications	696
41.40	Atlas connections	697
41.41	Closing view	697
41.42	References used in this chapter	698

42 Conditional Computation	699
42.1 From computational depth to conditional paths	700
42.2 A typed sparsity object	700
42.3 Static parameter sparsity	701
42.4 Sparsity ratio is not execution semantics	701
42.5 Conditional computation	701
42.6 Hard block skipping	702
42.7 Soft gates are not automatically sparse execution	702
42.8 Spatially adaptive computation	703
42.9 Token sparsity	703
42.10 Masking after dense work is not the same as removing before it	704
42.11 Structural sparsity and dynamic sparsity	704
42.11.1 Structural/static	704
42.11.2 Dynamic/input-dependent	704
42.12 Why when matters	704
42.13 Parameter count and active parameter count	705
42.14 Routing is a computation	705
42.15 Resource vectors	706
42.16 Average cost	706
42.17 Peak and tail cost	706
42.18 Exact witness: fixed baseline	707
42.19 Exact witness: conditional paths	707
42.20 Average arithmetic falls	708
42.21 Peak arithmetic does not fall	708
42.22 Launch count gets worse	708
42.23 Hardware assumptions determine the ordering	709
42.24 FLOPs do not determine latency	709
42.25 Dynamic irregularity can cost efficiency	710
42.26 Static sparsity can also fail to accelerate	710
42.27 Conditional depth	710
42.28 Early exit versus block skipping	711
42.29 Token dropping versus conditional depth	711
42.30 Training-time execution can be denser	712
42.31 Compute and quality form a joint claim	712
42.32 Dynamic computation can specialize effort	712
42.33 The router can be wrong	713
42.34 Routing confidence is not correctness	713
42.35 Batching changes the economics	713
42.36 Communication can dominate arithmetic	714
42.37 Capacity and active work can decouple	714
42.38 Static compression and conditional capacity are different strategies	714
42.39 Token sparsity can alter later complexity superlinearly	715
42.40 Measured speedup is contextual evidence	715
42.41 Conditional computation is an allocation policy	715
42.42 Failure modes	716
42.42.1 Sparsity/FLOP conflation	716

42.42.2 FLOP/latency conflation	716
42.42.3 Soft/hard conflation	716
42.42.4 Mean/peak conflation	716
42.42.5 Router-free accounting	716
42.42.6 Training/inference conflation	716
42.42.7 Capacity/active-work conflation	716
42.42.8 Quality omission	716
42.43A practical conditional-compute ledger	716
42.44What the exact witness establishes	717
42.45Downstream handoff	717
42.46References used in this chapter	718
43 Mixture-of-Experts Systems	719
43.1 From conditional computation to experts	720
43.2 Where experts enter a Transformer	720
43.3 Expert family	720
43.4 Router logits	721
43.5 Router probabilities	721
43.6 Preferred top-k route	721
43.7 Accepted dispatch	722
43.8 Gate weights	722
43.9 Top-1 is simpler, not trivial	722
43.10Capacity is not a probability threshold	723
43.11Nominal capacity	723
43.12Demand versus accepted load	723
43.13Overflow needs policy	724
43.14Dropping changes the computation	724
43.15Rerouting changes the meaning of top-1	724
43.16Expert token load	725
43.17Router probability mass	725
43.18Balance has multiple meanings	725
43.19Auxiliary balancing objectives	726
43.20Balance is a systems pressure	726
43.21Specialization is a different claim	726
43.22Equal traffic does not prove equal usefulness	727
43.23Preferred-router collapse versus accepted-load concentration	727
43.24Expert underuse	727
43.25Expert redundancy	728
43.26Capacity overload is different again	728
43.27Expert parallelism	728
43.28Dispatch communication	729
43.29All-to-all is not free	729
43.30Stragglers matter	729
43.31Equal token counts need not mean equal compute	730
43.32Training and serving need not use the same routing semantics	730
43.33The exact witness: six tokens	730

43.34Naive top-1 preferences	731
43.35Keep the strongest E1 assignments	731
43.36Reroute the overflow token	732
43.37Accepted load is perfectly balanced	732
43.38Router probability mass is not balanced	732
43.39Exact probability-imbalance value	733
43.40Equal traffic still does not imply equal usefulness	733
43.41The utility numbers are not router scores	734
43.42Drop instead of reroute	734
43.43Overflow policy changes arithmetic	734
43.44Balanced counts do not settle communication	735
43.45Shazeer et al.: sparse gating and balancing	735
43.46GShard: capacity meets distributed execution	735
43.47Switch: top-1 simplifies one axis	735
43.48ST-MoE: stability and transfer remain system properties	736
43.49A practical MoE ledger	736
43.50Failure modes	737
43.50.1 Probability-equals-dispatch	737
43.50.2 Top-k-equals-execution	737
43.50.3 Balanced-counts-equals-specialization	737
43.50.4 Balanced-counts-equals-cheap	737
43.50.5 Collapse-as-one-word	737
43.50.6 Auxiliary-loss-equals-task-objective	737
43.50.7 Sparse-parameters-equals-sparse-system-cost	737
43.51What the exact witness establishes	737
43.52Downstream handoff: Router Dynamics	738
43.53Downstream handoff: Systems	738
43.54References used in this chapter	739
44 Router Dynamics and Diagnostics	741
44.1 Four moving objects	741
44.2 Probability drift	742
44.3 Preferred-route churn	742
44.4 Accepted-dispatch churn	743
44.5 Load drift	743
44.6 Exact counterexample: perfect balance, total churn	743
44.7 Transition operators on expert identity	744
44.8 One-step transition is not automatically a Markov model	745
44.9 Route churn can be zero while probabilities move	745
44.10Routing stability and training stability are different claims	746
44.11Routing rules alter what can specialize	746
44.12Specialization needs a declared taxonomy	746
44.13Temporal specialization drift	747
44.14Router dynamics include optimizer memory	747
44.15Commutators detect order sensitivity	748
44.16Exact commutator witness	748

44.17Spectral router diagnostics are typed	749
44.18Churn is not automatically bad	749
44.19Capacity can hide preference dynamics	750
44.20A practical diagnostic record	750
44.21Handoff to Routing as Online Decision Making	750
44.22Durable lesson	751
44.23References used in this chapter	751
45 Routing as Online Decision Making	753
45.1 Router dynamics are not regret theory	754
45.2 Preferred route is not always the executed action	754
45.3 Capacity is part of the decision semantics	755
45.4 Loss happens after the accepted action	755
45.5 Feedback must be named	755
45.6 Comparator comes before regret	756
45.7 Static regret asks one question	756
45.8 Shifting regret asks another question	756
45.9 One common finite routing problem	757
45.10The shifting comparator	757
45.11A perfectly stable route can regret its stability	758
45.12A maximally churning route can be regret-optimal	758
45.13Churn is not a regret surrogate	759
45.14Static regret tells a different story	760
45.15Negative static regret is not an error	760
45.16Optionality is feasible choice	760
45.17Correction capacity is ability to change course	761
45.18Optionality and correction capacity are distinct	761
45.19The stay policy preserves correction capacity	761
45.20The tracking policy spends correction capacity	762
45.21Capacity should constrain the comparator too	762
45.22Preferred-route regret is a separate counterfactual	762
45.23Capacity can hide or create churn	762
45.24Load balance is not online optimality	763
45.25Specialization is not regret	763
45.26Probability drift is not regret	763
45.27If switching is costly, put it in the loss	763
45.28Nonstationarity must be typed	764
45.29Bandit feedback changes the online problem	764
45.30Delayed feedback weakens correction	764
45.31Hindsight comparator is not an executable oracle	765
45.32Downstream task loss is still separate	765
45.33A practical routing-decision record	765
45.34Durable non-implications	766
45.35Closing view	766
45.36References used in this chapter	767

46 A Taxonomy of Machine Memory	769
46.1 Memory is an interface, not merely a medium	770
46.2 Six roles, not six rungs	770
46.3 Parametric memory	771
46.4 Working memory	772
46.5 Context is not durable memory	773
46.6 Episodic memory	773
46.7 Semantic memory	774
46.8 Episodic and semantic are not storage media	775
46.9 Associative memory	775
46.10 One store, three memory behaviors	776
46.11 External memory	776
46.12 External does not mean trustworthy	777
46.13 Memory Networks and explicit long-term state	778
46.14 Differentiable external memory	778
46.15 Lifetime is a separate axis	778
46.16 Writes have different costs	779
46.17 Provenance can be preserved or destroyed	779
46.18 Shared memory introduces coordination	780
46.19 Failure by memory role	781
46.20 Memory is not knowledge	781
46.21 What Retrieval may assume	781
46.22 What Continual Learning may assume	782
46.23 The boundary to remember	783
46.24 References used in this chapter	783
47 Retrieval and Associative Access	785
47.1 Retrieval starts where the memory taxonomy stops	786
47.2 A retrieval contract	786
47.3 Exact-key lookup	787
47.4 Key-value retrieval is not semantic retrieval	787
47.5 Symbolic retrieval	788
47.6 A predicate is not a ranking	788
47.7 Vector retrieval	789
47.8 Similarity is representation-relative	790
47.9 A similarity score is not automatically relevance probability . . .	790
47.10 Top-k is a budgeted truncation operator	791
47.11 Tie rules matter for exact replay	791
47.12 Exact nearest neighbour	791
47.13 Approximate nearest neighbour	792
47.14 Search approximation and representation error are different . . .	792
47.15 Sparse and lexical retrieval	793
47.16 Symbolic, sparse, and dense are different axes	793
47.17 One store can have many indices	794
47.18 Multi-index retrieval	794
47.19 Hybrid retrieval requires composition semantics	795

47.20	Exact witness store	795
47.21	Exact vector distances	796
47.22	Exact-key read of the same store	796
47.23	Symbolic read of the same store	796
47.24	Hybrid retrieval does not generally commute	797
47.24.1	Filter then rank	797
47.24.2	Rank then filter	797
47.25	Why filter-then-rank often has cleaner semantics	797
47.26	Score fusion is another hybrid family	798
47.27	Retrieval can be staged	798
47.28	Attention and retrieval overlap at the operator level	799
47.29	Retrieval can exceed the active field	799
47.30	Retrieval can use hard identity	799
47.31	Retrieval is not causal explanation	800
47.32	Retrieval and provenance	800
47.33	Rank does not create authority	800
47.34	Retrieval correctness, relevance, and usefulness	801
47.35	Recall and precision require a relevance judgment	801
47.36	ANN recall is not semantic recall	802
47.37	Retrieval-augmented generation is downstream composition	802
47.38	Retrieval is not memory writing	803
47.39	Retrieval under updates	803
47.40	Deletion is also a retrieval property	803
47.41	Multi-index consistency	804
47.42	Retrieval budgets shape cognition	804
47.43	The nearest record need not be the best context item	804
47.44	Diversification changes the objective	805
47.45	Retrieval over governed records	805
47.46	Failure modes	806
47.46.1	Address conflation	806
47.46.2	Similarity laundering	806
47.46.3	Score calibration fiction	806
47.46.4	Top-k completeness fiction	806
47.46.5	ANN/exact conflation	806
47.46.6	Hybrid ambiguity	806
47.46.7	Index/record conflation	806
47.46.8	Provenance laundering	806
47.46.9	Retrieval/generation conflation	806
47.46.10	Read/write conflation	806
47.47	A practical retrieval ledger	807
47.48	What the exact witness establishes	807
47.49	Downstream handoff	808
47.50	References used in this chapter	808
48	Continual Learning and Forgetting	811
48.1	Sequential learning creates a history	812

48.2	An Atlas forgetting summary	813
48.3	Parameter movement is not forgetting	813
48.4	Why the word catastrophic exists	814
48.5	Continual learning is not one benchmark regime	815
48.5.1	Task-incremental learning	815
48.5.2	Domain-incremental learning	815
48.5.3	Class-incremental learning	815
48.6	Memory enters continual learning in different roles	815
48.7	Replay: put old evidence back into the objective	816
48.8	Replay does not guarantee retention	817
48.9	Consolidation: preserve important parameter directions	817
48.10	EWC is an approximation, not an oracle	818
48.11	The stability-plasticity tension	818
48.12	Exact witness: two tasks, one scalar	819
48.13	Unprotected sequential learning	819
48.14	Exact quadratic consolidation	820
48.15	Replay and EWC coincide here for a special reason	821
48.16	Gradient constraints: protect old losses at update time	822
48.17	First-order protection has a scope	822
48.18	Parameter isolation: stop sharing the contested parameters	823
48.19	Isolation pays with capacity and routing	823
48.20	Replay, consolidation, constraints, and isolation are different levers	824
48.20.1	Replay	824
48.20.2	Consolidation	824
48.20.3	Gradient constraints	824
48.20.4	Parameter isolation	824
48.21	Capacity must be reported	824
48.22	Average forgetting can hide a damaged context	825
48.23	Positive transfer should not be clipped out of existence	826
48.24	Final accuracy alone is also insufficient	826
48.25	Forgetting and transfer can be asymmetric	826
48.26	Multi-task learning is the useful counterfactual	827
48.27	Replay versus joint training	827
48.28	EWC versus ordinary L2 regularization	828
48.29	Consolidation and semantic memory are not synonyms	828
48.30	Catastrophic forgetting is partly an access problem	828
48.31	Failure modes	829
48.31.1	Forgetting-equals-parameter-drift	829
48.31.2	Zero-forgetting-by-zero-learning	829
48.31.3	Replay-equals-memory-architecture	829
48.31.4	EWC-equals-guarantee	829
48.31.5	Isolation-equals-free retention	829
48.31.6	Average-only reporting	829
48.31.7	Regime mixing	830
48.32	A practical continual-learning ledger	830
48.33	What the exact witness establishes	830

48.34Atlas handoff	831
48.35References used in this chapter	832
49 The External-Memory Thesis	833
49.1 The wrong question is parameters versus memory	834
49.2 What is parametric knowledge?	834
49.3 What is external memory?	834
49.4 Retrieval is the read path, not the memory itself	835
49.5 Continual learning is the parameter-update side	835
49.6 The placement descriptor	836
49.7 Why no scalar placement score?	836
49.8 Volatility favors explicit state	836
49.9 Provenance favors explicit state	837
49.10Deletion and supersession favor explicit records	837
49.11Sharing can favor external memory	838
49.12Addressability matters	838
49.13Parametric compression has value	838
49.14Low-latency ubiquitous structure can favor parameters	839
49.15RAG makes the hybrid idea concrete	839
49.16kNN-LM shows datastore substitution	839
49.17RETRO shows scale, not universal superiority	840
49.18ROME gives the necessary counterexample	840
49.19The exact parametric witness	840
49.20Updating one fact requires a coordinated edit here	841
49.21A naive coordinate-local edit interferes	841
49.22The external-store version	842
49.23Record-locality is not compute optimality	842
49.24Provenance is explicit in the store	842
49.25The stale-read counterexample	843
49.26Freshness needs a contract	843
49.27Caching creates local memory again	844
49.28Write correctness and read correctness differ	844
49.29Retrieval failure is not memory-write failure	845
49.30Poisoning is a memory-layer risk	845
49.31Access control becomes first-class	846
49.32Sharing is not global visibility	846
49.33Deletion is not trivial even externally	846
49.34Parametric erasure is a different problem	847
49.35Consolidation moves in the other direction	847
49.36Externalization can precede consolidation	847
49.37Parameter isolation and external memory solve different problems	848
49.38Replay uses external evidence without making it the final answer source	848
49.39A record is not automatically knowledge	848
49.40Multiple records can legitimately conflict	849
49.41Update frequency is only one axis	849

49.42	The “world in weights” failure mode	849
49.43	The “database as intelligence” failure mode	850
49.44	The hybrid architecture	850
49.45	A practical placement ledger	850
49.46	Failure modes	851
49.46.1	External-equals-fresh	851
49.46.2	External-equals-correct	851
49.46.3	Retrieved-equals-authorized	851
49.46.4	Parameter-change-equals-forgetting	851
49.46.5	External-equals-cheap	851
49.46.6	Parametric-equals-uneditable	851
49.46.7	Shared-equals-consistent	852
49.46.8	Externalize-everything	852
49.47	What the exact witness establishes	852
49.48	Downstream handoff: Context Compilation	852
49.49	Downstream handoff: Polity	852
49.50	References used in this chapter	853

50 Context Compilation 855

50.1	The external-memory handoff	856
50.2	The pipeline	856
50.3	Storage is not retrieval	856
50.4	Retrieval is not compilation	857
50.5	Ranking is one signal, not the whole contract	857
50.6	Candidate records	857
50.7	Compiled cost includes metadata	858
50.8	Provenance is not decoration	858
50.9	Authorization comes before optimization	859
50.10	Freshness comes before optional utility	859
50.11	Unresolved conflict is a failure state	859
50.12	Mandatory records	860
50.13	Mandatory overflow	860
50.14	Optional selection	861
50.15	Utility is policy input	861
50.16	Determinism	862
50.17	Serialization is part of compilation	862
50.18	The exact budget-8 witness	862
50.19	Freshness removes the rank-1 policy	863
50.20	Mandatory policy consumes half the budget	863
50.21	Exhaustive finite selection	864
50.22	The compiled set	864
50.23	What the witness actually proves	865
50.24	The naive rank-prefix control	865
50.25	Why the rank prefix is invalid	866
50.26	Truncating the policy is not a repair	866
50.27	Atomicity is policy-relative	867

50.28	Compiler validity	867
50.29	Valid context is not correct answer	867
50.30	Stored is not retrieved	868
50.31	Retrieved is not included	868
50.32	Included is not used	868
50.33	Budget is an abstract resource	869
50.34	Prompt construction becomes replayable	869
50.35	Replay does not validate retrieval quality	869
50.36	Freshness and ranking answer different questions	870
50.37	Provenance and relevance answer different questions	870
50.38	Shared memory needs invocation-local compilation	870
50.39	Context compilation is not memory deletion	871
50.40	Context compilation is not model editing	871
50.41	Practical compiler protocol	871
50.42	What this chapter does not claim	872
50.43	Non-implications	872
50.44	Atlas connections	873
50.45	Closing view	873
50.46	References used in this chapter	874
51	Tokenization and Representation Boundaries	875
51.1	A ladder of boundaries	875
51.2	Tokenization as a map	876
51.3	Bytes and codepoints are not the same unit	876
51.4	Why subwords became useful	877
51.5	BPE is an algorithm, not merely a vocabulary	877
51.6	Unigram tokenization treats segmentation differently	878
51.7	Exact ambiguity witness	878
51.8	SentencePiece moves the boundary again	879
51.9	Token count is a useful but narrow quantity	879
51.10	Token count is not description length	879
51.11	Fertility measures segmentation expansion	880
51.12	Exact fertility witness	880
51.13	Morphology gives a different diagnostic	881
51.14	Morphology and subword frequency optimize different objects	882
51.15	Byte-level models remove one tokenizer bottleneck	882
51.16	Byte-level does not mean text has no structure	882
51.17	Multilingual tokenization creates allocation problems	883
51.18	A multilingual tokenizer report should be vector-valued	883
51.19	Token IDs are coordinates, not meaning	883
51.20	Tokenization changes the learning problem	884
51.21	Comparing tokenizers requires controlled scope	884
51.22	What this chapter does not claim	885
51.23	Handoff to Tokenization as Compression and Interface	885
51.24	Durable lesson	885
51.25	References used in this chapter	886

52 Tokenization as Compression and Interface	887
52.1 The inherited boundary	888
52.2 Compression requires a code	888
52.3 Vocabulary size enters the accounting	889
52.4 Fixed-width length is not entropy coding	889
52.5 A tokenizer is part of the codebook	889
52.6 Compression ratio needs a baseline	890
52.7 Token count is not compute	890
52.8 Two exact compute proxies	890
52.9 The finite witness	891
52.10 Small-vocabulary tokenizer	891
52.11 Piece tokenizer	892
52.12 The orderings conflict	892
52.13 There is no scalar winner without an objective	893
52.14 Shorter can still matter	893
52.15 Vocabulary can buy reusable pieces	894
52.16 Corpus distribution matters	894
52.17 Tokenizer interoperability	894
52.18 Canonical transport domain	895
52.19 Lossless translation theorem	895
52.20 The witness translates exactly	896
52.21 Lossless does not mean representation-equivalent	896
52.22 Nor does lossless mean semantic equivalence	897
52.23 Direct ID interoperability is stronger	897
52.24 Special tokens require protocol semantics	897
52.25 Version identity matters	898
52.26 Three meanings of lingua franca	898
52.26.1 Canonical transport	898
52.26.2 Shared tokenizer	898
52.26.3 Translation layer	898
52.27 Canonical bytes as a bounded lingua franca	898
52.28 Shared tokenizers trade interoperability for constraints	899
52.29 Translation layers preserve local freedom	899
52.30 Compression and interoperability are different objectives	899
52.31 Compute and interoperability are different objectives	900
52.32 A practical comparison vector	900
52.33 Multilingual scope remains explicit	900
52.34 Modeled compute versus measured compute	901
52.35 What would an empirical comparison require?	901
52.36 Durable non-implications	902
52.37 Tokenization as interface design	902
52.38 References used in this chapter	903
53 Data Quality, Mixtures, and Contamination	905
53.1 Start with the data object	906
53.2 Why lineage matters	906

53.3	Exact duplicates	906
53.4	Near duplicates	907
53.5	Semantic redundancy is different again	907
53.6	Deduplication changes the training measure	907
53.7	Deduplication itself has policy	908
53.8	Cross-domain duplicates are especially revealing	908
53.9	“Clean” is not a mathematical primitive	908
53.10	Quality is multidimensional	909
53.11	Filters can trade one quality for another	909
53.12	Raw corpus proportions	909
53.13	Training mixture weights	910
53.14	Corpus composition and training composition differ	910
53.15	Mixture weights are an intervention	910
53.16	Synthetic provenance	911
53.17	Recursive generated-data failure mode	911
53.18	Synthetic data need lineage depth	911
53.19	Train/evaluation contamination	911
53.20	Exact overlap is one threat model	912
53.21	Near overlap is a different threat model	912
53.22	Benchmark examples in web corpora	912
53.23	Exposure is not memorization	913
53.24	Memorization is not causal score inflation	913
53.25	Benchmark pathology is broader than exact leakage	913
53.26	Benchmark version matters	913
53.27	Exact finite witness	914
53.28	Exact-match contamination in the witness	914
53.29	Near-overlap contamination in the witness	914
53.30	Why this does not make either rate false	915
53.31	Exact deduplication changes the corpus	915
53.32	Near-duplicate clustering changes it again	916
53.33	Pairwise near-duplication need not be transitive	916
53.34	Mixture weights in the witness	916
53.35	Why mixture records should be first-class	917
53.36	Data quality and model quality are not identical	917
53.37	Deduplication precision and recall	918
53.38	Contamination detection also has precision and recall	918
53.39	Source-level versus span-level overlap	918
53.40	Time can be part of contamination	919
53.41	Public benchmarks create a moving target	919
53.42	Synthetic data can carry contamination too	919
53.43	Recursive lineage is a graph	920
53.44	Data mixtures can evolve during training	920
53.45	“More data” is underspecified	920
53.46	A practical dataset ledger	921
53.47	Failure modes	921
53.47.1	Exact-equals-near	921

53.47.2 Near-equals-semantic	922
53.47.3 Dedup-equals-storage	922
53.47.4 Corpus-proportion-equals-mixture	922
53.47.5 Synthetic-equals-bad	922
53.47.6 Overlap-equals-memorization	922
53.47.7 Memorization-equals-score-inflation	922
53.47.8 Clean-equals-quality	922
53.48 What the exact witness establishes	922
53.49 Downstream handoff	923
53.50 References used in this chapter	923

54 Curriculum Learning 925

54.1 The question is not merely “easy or hard?”	926
54.2 Three states, not one	926
54.2.1 Model state	926
54.2.2 Optimizer state	926
54.2.3 Curriculum state	927
54.3 Curriculum as a policy	927
54.4 Open loop and closed loop	928
54.5 Difficulty is a score, not an essence	929
54.6 Competence schedules	930
54.7 Automated curricula and learning progress	930
54.8 Learning progress is not a mechanism readout	931
54.9 Selection is not the same as reweighting	931
54.10 A finite state-aware separation	932
54.10.1 Learner A	933
54.10.2 Learner B	934
54.11 What the witness does not say	934
54.12 Equal-budget two-step replay	935
54.13 Why immediate progress is not enough	936
54.14 Curriculum versus training dose	936
54.15 Curriculum and mixture optimization	937
54.16 Stochastic syllabi	937
54.17 When loss is a bad difficulty proxy	938
54.18 The fixed lesson plan and the teacher reading the room	938
54.19 Failure modes	939
54.19.1 Easy-to-hard as dogma	939
54.19.2 Competence by naming	939
54.19.3 Progress as mechanism	939
54.19.4 Greedy progress as global policy	939
54.19.5 Hidden dose changes	939
54.19.6 Controller overfitting	939
54.19.7 Support drift	939
54.19.8 Noise chasing	939
54.19.9 Endpoint-only interpretation	939
54.19.10 Ignoring controller cost	940

54.20	What later chapters may assume	940
54.21	Epistemic status	940
54.22	References used in this chapter	941
55	Learning Progress as a Search Operator	943
55.1	The inherited progress signal	943
55.2	From curriculum control to search	944
55.3	Why progress is attractive	944
55.4	Search needs exploration	945
55.5	Exact exploration witness	945
55.6	Generalization-state evidence is not one scalar	946
55.7	Heterogeneous evidence needs an ordering rule	946
55.8	Immediate progress can be the wrong horizon	947
55.9	Credit assignment is separate from search	947
55.10	Absolute learning progress needs care	947
55.11	Search changes the training intervention	948
55.12	Search can overfit its metric	948
55.13	Search does not mean permanent novelty	949
55.14	Search over generated experience	949
55.15	What search has not yet solved	949
55.16	Handoff to Minimal Curricula and Reasoning Bases	949
55.17	Durable lesson	950
55.18	References used in this chapter	950
56	Minimal Curricula and Reasoning Bases	951
56.1	Progress search is not yet minimality	952
56.2	The Residual supplies the right discipline	952
56.3	The smallest exact witness	952
56.4	A declared reconstruction rule	953
56.5	Two examples are sufficient	954
56.6	No strict smaller set works	954
56.7	This is teaching-set minimality	955
56.8	Machine teaching makes the optimization direction explicit	955
56.9	Minimality is conditional	955
56.10	Capability identity can be weaker than model identity	956
56.11	Now add a high-progress distractor	956
56.12	But it tells us nothing about the target class	957
56.13	Search value and basis value can disagree	957
56.14	Search asks where to learn next	958
56.15	Minimal set is not optimal sequence	958
56.16	Why reasoning bases are harder than teaching sets	958
56.17	Four distinct questions	959
56.18	Acquisition	959
56.19	Persistence	959
56.20	Acquisition does not prove persistence	960
56.21	Accessibility	960

56.22	Expression	960
56.23	Later expression does not prove early persistence	961
56.24	This matters for developmental narratives	961
56.25	A minimal reasoning basis must specify composition	961
56.26	Strict-smaller failure is essential	962
56.27	High progress can be auxiliary	962
56.28	Low progress can still be necessary	962
56.29	Minimal basis can change with capability scope	963
56.30	Minimal does not mean unique	963
56.31	Operational versus semantic reconstruction	963
56.32	A general machine-teaching formulation	964
56.33	Early mechanisms require a longitudinal protocol	964
56.34	A useful empirical disposition grammar	964
56.34.1	MINIMAL_RECONSTRUCTIVE_BASIS	964
56.34.2	SUFFICIENT_NOT_MINIMAL	965
56.34.3	SEARCH_RELEVANT_NOT_BASIS	965
56.34.4	PERSISTENT_INACCESSIBLE	965
56.34.5	BEHAVIOR_ONLY	965
56.35	The exact witness in one line	965
56.36	Non-implications	966
56.37	Atlas connections	966
56.38	Closing view	967
56.39	References used in this chapter	967

57 Reinforcement Learning and Control 969

57.1	A controlled stochastic dynamical system	969
57.2	The Markov boundary	970
57.3	Reward is not value	971
57.4	Bellman's recursion	971
57.5	A two-state control problem	972
57.6	Policy evaluation and policy improvement	973
57.7	Optimality is relative	974
57.8	When the model is known	975
57.9	Learning from sampled transitions	975
57.10	Q-learning and off-policy control	975
57.11	Policy gradients	976
57.12	Model-based reinforcement learning	977
57.13	Partial observability	977
57.14	What value leaves out	978
57.15	Model-free is not assumption-free	978
57.16	Control and uncertainty are different layers	979
57.17	What exploration adds	979
57.18	The boundary to remember	980
57.19	References used in this chapter	980

58 Exploration and Information Value 981

58.1	From control to learning while acting	982
58.2	A minimal Bayesian exploration object	982
58.3	Bayes value makes exploration explicit	983
58.4	The smallest useful example	984
58.5	Exploration cost and future information value	985
58.6	Information gain is not value of information	985
58.7	Epistemic uncertainty versus randomness	986
58.7.1	Epistemic uncertainty	986
58.7.2	Outcome stochasticity	987
58.8	Bandits: the cleanest exploration laboratory	987
58.9	Optimism: act as if plausible good outcomes matter	988
58.10	Posterior sampling: act according to posterior possibility	988
58.11	Information-directed exploration	989
58.12	Information is useful only through a future decision	989
58.13	Exploration in an MDP is harder than exploration in a bandit	990
58.14	Exploration is not randomization	991
58.15	Safe exploration starts by naming the constraint	991
58.16	Safety can make information expensive	992
58.17	Information acquisition can destroy optionality	992
58.18	Exploration and regret are related but distinct	993
58.19	Finite horizon versus asymptotic reasoning	993
58.20	Failure modes	994
58.20.1	Myopic exploitation	994
58.20.2	Undirected exploration	994
58.20.3	Uncertainty conflation	994
58.20.4	Information-value conflation	994
58.20.5	Algorithm conflation	994
58.20.6	Safety laundering	994
58.20.7	Horizon erasure	994
58.21	A practical exploration ledger	995
58.22	What the exact witness establishes	995
58.23	Atlas handoff	996
58.24	References used in this chapter	996
59	Regret	997
59.1	From exploration to opportunity cost	998
59.2	The stochastic-bandit object	998
59.3	Pathwise reward shortfall is random	999
59.4	Expected or pseudo-regret	999
59.5	Why expected pathwise regret equals pseudo-regret here	1000
59.6	Pull counts expose the cost of exploration	1000
59.7	Exact witness: same policy, different pathwise regret	1000
59.8	Negative pathwise regret is not paradoxical	1001
59.9	Bayesian regret adds a prior	1001
59.10	Worst-case regret removes prior weighting	1002
59.11	Minimax regret chooses the best worst-case policy	1002

59.12	Exact Bayes/minimax witness	1002
59.13	Bayes performance is bounded by worst-case performance	1003
59.14	Minimax is not prior-free truth	1004
59.15	Comparator class is part of regret	1004
59.16	Exact comparator witness	1004
59.17	The comparator can be too strong to be useful	1005
59.18	Cumulative regret measures interaction cost	1006
59.19	Simple regret measures final recommendation quality	1006
59.20	Exact cumulative/simple witness	1006
59.21	Exploration can raise cumulative regret and lower simple regret .	1007
59.22	Sublinear regret is an average guarantee	1007
59.23	A good asymptotic rate can hide finite-horizon cost	1008
59.24	Lai–Robbins: exploration has an asymptotic information price . .	1008
59.25	Finite-time analysis answers another question	1009
59.26	Regret does not explain the policy mechanism	1009
59.27	Regret and information are different units	1009
59.28	Low regret does not imply safety	1010
59.29	Low regret does not imply calibration	1010
59.30	Low regret does not imply fairness	1010
59.31	Low regret does not imply robustness	1010
59.32	Low regret does not imply recoverability	1011
59.33	Expected regret can ignore tail structure	1011
59.34	Bayesian regret can hide rare-environment failure	1011
59.35	Minimax regret can be conservative under a strong prior	1012
59.36	A practical regret ledger	1012
59.37	What the exact witnesses establish	1012
59.38	Downstream handoff	1013
59.39	References used in this chapter	1013

60 Optionality and Correction Capacity 1015

60.1	The door that information cannot reopen	1015
60.2	Optionality needs a contract	1016
60.3	Viable continuations come before valuable continuations	1017
60.4	Why counting buttons fails	1017
60.5	Cardinality is not value	1018
60.6	Irreversibility enters when uncertainty meets time	1019
60.7	Recoverability is relative, not mystical	1019
60.8	A bad outcome is not the same as irreversible action	1020
60.9	Correction cost	1020
60.10	Conditional correction and ex-ante correction capacity	1021
60.11	The exact equal-value witness	1021
60.12	Prior value does not separate the policies	1022
60.13	Bayesian regret can also tie	1023
60.14	The future feasible sets do separate them	1023
60.15	Correction capacity separates them more sharply	1024
60.16	Information is identical	1024

60.17	Empowerment is related, but not identical	1025
60.18	Policy optionality is not environmental branching	1025
60.19	Delay is not automatically recoverability	1026
60.20	More optionality can be too expensive	1026
60.21	More labels are not more options	1027
60.22	Regret and optionality answer different questions	1027
60.23	Optionality can be an objective, a constraint, or a diagnostic	1028
60.24	Hard and soft correction failure should not be merged	1028
60.25	Optionality does not imply safety	1029
60.26	Optionality does not imply knowledge	1029
60.27	A useful evaluation vector	1029
60.28	What the exact witness proves	1030
60.29	Downstream handoff	1030
60.30	References used in this chapter	1031
61	Joint Uncertainty Propagation	1033
61.1	The controlled-system handoff	1034
61.2	Information-theoretic discipline	1034
61.3	Four local coordinates	1034
61.4	Marginals are not enough	1035
61.5	A declared local decision-error functional	1035
61.6	Exact affine propagation	1036
61.7	The naive diagonal calculation	1036
61.8	Positive common-shock dependence	1036
61.9	Positive dependence amplifies uncertainty	1037
61.10	Cancellation dependence	1037
61.11	Dependence can also cancel	1038
61.12	Independence control	1038
61.13	One set of marginals, three different results	1039
61.14	The difference is dependence structure	1039
61.15	Independence is stronger than required	1040
61.16	Propagation depends on the decision functional	1040
61.17	Reward uncertainty remains a distinct object	1040
61.18	Transition uncertainty remains distinct too	1041
61.19	Observation uncertainty is not transition uncertainty	1041
61.20	Future-value uncertainty is downstream but not redundant	1041
61.21	Expected value is not uncertainty	1041
61.22	A risk rule must be declared	1042
61.23	Variance is useful but incomplete	1042
61.24	Nonlinear decision maps	1042
61.25	The exact remainder identity	1043
61.26	First-order propagation is local	1043
61.27	When the local approximation can fail	1043
61.28	Sequential propagation is harder than one step	1043
61.29	Why naive independent summation is dangerous	1044
61.30	Covariance is not causality	1045

61.31	Conditional structure matters	1045
61.32	Functional-relative uncertainty is an interface concept	1045
61.33	What JOINTUNC does not claim	1046
61.34	Non-implications	1046
61.35	Practical protocol	1047
61.36	Atlas connections	1047
61.37	Closing view	1047
61.38	References used in this chapter	1048
62	Uncertainty and Calibration	1049
62.1	Probability is not yet calibration	1050
62.2	Confidence is not calibration	1050
62.3	Calibration is not accuracy	1051
62.4	Calibration is not sharpness	1051
62.5	Proper scoring and calibration are related but distinct	1052
62.6	Empirical calibration is an estimate	1052
62.7	One word, several uncertainty objects	1053
62.8	Predictive entropy	1053
62.9	Aleatoric and epistemic uncertainty	1054
62.10	Entropy decomposition	1054
62.11	Same entropy, opposite decomposition	1054
62.11.1	Model A	1055
62.11.2	Model B	1055
62.12	Reducibility is relative to the information set	1056
62.13	Bayesian uncertainty	1056
62.14	Dropout as an approximate Bayesian construction	1056
62.15	Deep ensembles	1057
62.16	Ensemble spread and epistemic uncertainty	1058
62.17	Prediction sets are another object	1058
62.18	Split conformal rank logic	1058
62.19	Exact conformal rank witness	1059
62.20	Coverage is not calibration	1060
62.21	Distribution-free does not mean assumption-free	1060
62.22	Conditional coverage is stronger	1061
62.23	Abstention is another control surface	1061
62.24	Exact risk-coverage witness	1061
62.25	Abstention is not calibration	1062
62.26	Uncertainty and decisions	1062
62.27	Calibration can be local or global in different senses	1063
62.28	The model can be calibrated for the wrong world	1063
62.29	The uncertainty interface	1064
62.30	Handoff to Distribution Shift and Robustness	1064
62.31	Failure modes	1065
62.32	Closing view	1065
62.33	References used in this chapter	1065

63 Distribution Shift and Robustness	1067
63.1 Source law is not deployment law	1067
63.2 Distribution shift is not one phenomenon	1068
63.2.1 Covariate shift	1068
63.2.2 Label/prior shift	1068
63.2.3 Concept or conditional shift	1068
63.2.4 Structural shift	1068
63.3 Covariate shift permits a change of measure	1068
63.4 Target-only support is a real boundary	1069
63.5 Exact witness: calibration can fail under pure covariate shift	1069
63.6 Calibration failure need not mean every risk changed	1070
63.7 Exact importance weights	1070
63.8 A detected marginal shift need not be a performance failure	1071
63.9 Concept shift is different	1071
63.10 Calibration, coverage, and selective risk each have their own transfer problem	1072
63.11 Adversarial robustness asks a different question	1072
63.12 Exact adversarial separation	1073
63.13 Robustness is perturbation-set relative	1073
63.14 Robust optimization	1073
63.15 Structural sensitivity is not an adversarial synonym	1073
63.16 Average-case shift is not worst-case shift	1074
63.17 Shift magnitude is metric-relative	1074
63.18 Support overlap determines what reweighting can say	1074
63.19 The unchanged model can still face a changed theorem	1074
63.20 Metric firewall	1075
63.21 What SHIFT does not license	1075
63.22 Operational checklist	1075
63.23 Durable rule	1075
63.24 References used in this chapter	1075
64 Interlude: From Readability to Functional Evidence	1077
64.1 Exact finite separation	1077
64.2 Diagnostic record	1079
64.3 Redundancy boundary	1080
64.4 Downstream handoff	1082
64.5 References used in this chapter	1083
65 Spectral and Operator Diagnostics	1085
65.1 Different spectral objects answer different questions	1086
65.2 The non-normal boundary	1086
65.3 Exact equal-eigenvalue control	1086
65.4 Singular values are different from eigenvalues	1087
65.5 Equal singular values can hide interface differences	1087
65.6 Relative-position diagnostics	1088
65.7 Zero spectral drift need not mean zero behavioral drift	1088

65.8	Pseudospectra remain a separate diagnostic family	1089
65.9	Jacobian spectra are local	1089
65.10	Same local Jacobian, different nonlinear maps	1089
65.11	Transition-local signatures	1090
65.12	Hessian spectra are also local and incomplete	1090
65.13	Exact Hessian control	1091
65.14	Koopman changes the object being diagonalized	1091
65.15	Exact two-state Koopman witness	1092
65.16	Finite Koopman approximations need their own error language .	1092
65.17	Spectral drift is definition-relative	1093
65.18	Descriptive quality is not functional necessity	1093
65.19	Predictive utility is not mechanism	1093
65.20	Mechanistic significance requires intervention	1093
65.21	A matched-spectrum control is especially valuable	1094
65.22	Finite precision and conditioning	1094
65.23	No universal scalar diagnostic	1094
65.24	Reporting protocol	1095
65.25	Durable non-implications	1095
65.26	Closing view	1095
65.27	References used in this chapter	1096

66 Compression and Description Length 1097

66.1	Compression begins with a code	1097
66.2	Description length is not parameter count	1098
66.3	A finite MDL witness	1098
66.4	Lossless and lossy are different questions	1099
66.5	Exact low-rank compression	1099
66.6	Exact code-length comparison	1100
66.7	Low rank can also be lossy	1100
66.8	Pruning removes parameters	1101
66.9	A small weight can still matter	1101
66.10	Quantization changes precision	1102
66.11	Codebook overhead matters	1103
66.12	Weight sharing is different again	1103
66.13	Deep Compression combines mechanisms	1104
66.14	Entropy coding acts on symbol statistics	1104
66.15	Low rank and sparsity are different structures	1104
66.16	Structured parameterizations	1105
66.17	Distillation compresses behavior, not bits	1106
66.18	Exact equality on a finite dataset is not global equality	1106
66.19	Compression ratio is incomplete without retained behavior	1107
66.20	Parameter distortion can disagree with task distortion	1107
66.21	Compression and representation equivalence	1107
66.22	MDL is not “smallest network wins”	1108
66.23	Compression can reveal redundancy	1108
66.24	Compressibility is not interpretability	1109

66.25	Compressibility is not intelligence	1109
66.26	Failure modes	1109
66.26.1	Parameter count equals description length	1109
66.26.2	Pruning equals quantization	1109
66.26.3	Quantization equals weight sharing	1109
66.26.4	Low rank equals sparsity	1110
66.26.5	Distillation equals source coding	1110
66.26.6	High compression ratio means good compression	1110
66.26.7	Same outputs on one dataset means same function	1110
66.26.8	Compressible means interpretable	1110
66.26.9	Compressible means intelligent	1110
66.27	Downstream handoff	1110
66.28	References used in this chapter	1111
67	Compression as Discovery and Intelligence Probe	1113
67.1	Compression is tempting for a good reason	1114
67.2	The Atlas boundary	1114
67.3	Begin with the code	1115
67.4	A short code can be trivial	1115
67.5	Why training compression is too weak	1116
67.6	Held-out codelength	1116
67.7	The period-two witness	1117
67.8	A matched failure control	1118
67.9	The result is still codec-relative	1118
67.10	MDL probing	1119
67.11	Same final accuracy can hide different learning difficulty	1119
67.12	Same codelength can hide different behavior	1119
67.13	Random-label controls	1120
67.14	Shuffle controls	1120
67.15	Random-representation controls	1121
67.16	Probe capacity is part of the instrument	1121
67.17	Blier and Ollivier: coding method matters	1121
67.18	Solomonoff: compression and prediction in an idealized theory	1122
67.19	Compression progress	1122
67.20	When compression progress is fake	1123
67.21	Discovery needs controls	1123
67.22	Reuse is stronger than one-task prediction	1123
67.23	Compression and the Residual	1124
67.24	Invertible recoding	1125
67.25	Recoding stability	1125
67.26	Probe accessibility is not mechanism	1125
67.27	What mechanism evidence would add	1126
67.28	Codelength and task distortion	1126
67.29	An evidence ladder	1127
67.29.1	Level 0: training compression	1127
67.29.2	Level 1: held-out compression	1127

67.29.3 Level 2: control-separated held-out compression	1127
67.29.4 Level 3: recoding-stable compression	1127
67.29.5 Level 4: cross-context reuse	1127
67.29.6 Level 5: mechanistic intervention	1127
67.30 Intelligence is broader	1128
67.31 A practical COMPINTEL protocol	1128
67.32 Longitudinal discovery protocol	1129
67.33 Research use: finding reusable computation	1129
67.34 Research use: compression before interpretation	1130
67.35 Research use: scientific discovery systems	1130
67.36 Source boundaries	1131
67.37 What the computational witness establishes	1131
67.38 Non-implications	1131
67.39 Atlas connections	1132
67.40 Closing view	1133
67.41 References used in this chapter	1133

68 From Models to Agents 1135

68.1 The model is not the loop	1136
68.2 A minimal agent object	1136
68.3 Why interaction changes the problem	1138
68.4 Tools are transition channels	1139
68.5 Planning is computation before commitment	1139
68.6 Reflection is state update, not self-transcendence	1140
68.7 Decomposition requires interfaces	1141
68.8 Delegation is decomposition plus authority	1142
68.9 The return is not the truth	1143
68.10 Authority and capability are different coordinates	1143
68.11 Budgets turn “keep trying” into mathematics	1144
68.12 Termination is part of the agent	1145
68.13 Where agent loops fail	1145
68.13.1 Observation failure	1145
68.13.2 Proposal failure	1145
68.13.3 Controller failure	1145
68.13.4 Interface failure	1146
68.13.5 State-update failure	1146
68.13.6 Planning failure	1146
68.13.7 Reflection failure	1146
68.13.8 Delegation failure	1146
68.13.9 Termination failure	1146
68.14 Autonomy is not one number	1146
68.15 What the representative mechanisms establish	1147
68.16 From one loop to many	1148
68.17 The boundary to remember	1148
68.18 References used in this chapter	1149

69 Coordination Architectures	1151
69.1 From one loop to many	1152
69.2 Communication is not agreement	1153
69.3 Local state and shared state	1153
69.4 Six coordination families	1154
69.4.1 Direct message passing	1154
69.4.2 Rendezvous	1155
69.4.3 Blackboard coordination	1155
69.4.4 Tuple spaces	1156
69.4.5 Event-driven coordination	1156
69.4.6 Transactions	1157
69.5 Causality before total order	1157
69.6 A small race with a complete answer	1158
69.7 Atomicity changes the schedule space	1159
69.8 Delivery is not effect	1159
69.9 Retry is a semantic operation	1160
69.10 Shared state is not authority	1161
69.11 Deadlock and livelock	1161
69.12 Partial failure changes reasoning	1162
69.13 Coordination cost is real	1162
69.14 The AETHER example	1163
69.15 What coordination does not solve	1164
69.16 What the next chapters may assume	1164
69.17 The boundary to remember	1165
69.18 References used in this chapter	1166
 70 Evidence Exchange and Zero-Context Work	 1167
70.1 Evidence must survive handoff	1168
70.2 The dispatch packet	1168
70.3 What zero-context actually means	1169
70.4 Hidden context and explicit prerequisites	1170
70.5 A dispatch is a boundary object	1171
70.6 Source policy is part of the mathematics	1171
70.7 Context class is not a certificate of independence	1172
70.8 The return object	1172
70.9 Provenance asks how an object came to exist	1173
70.10 Provenance is not truth	1174
70.11 Identity has layers	1174
70.12 A tiny ambiguity with an exact answer	1175
70.13 Hashes solve one problem, not every problem	1176
70.14 FAIRness and exchangeability	1177
70.15 Return grammars force useful omissions into view	1177
70.16 The return route matters	1178
70.17 Receipt is not acceptance	1178
70.18 Structural validity is weaker than mathematical validity	1179
70.19 Replayability is graded	1180

70.20	Zero-context work is compiled context	1180
70.21	Why excluded assumptions are first-class	1181
70.22	Falsification conditions make returns cheaper to judge	1181
70.23	GCL as a bounded implementation example	1182
70.24	A return must point backward before synthesis can point forward	1183
70.25	Synthesis is a new reasoning step	1183
70.26	Stronger prose requires stronger support	1184
70.27	Conflict is an admissible output	1185
70.28	Duplicate delivery and evidence identity	1185
70.29	Evidence exchange and memory	1186
70.30	Evidence exchange and agents	1186
70.31	Minimal handoff, not maximal paperwork	1187
70.32	What Replay may assume	1187
70.33	What the Research State Machine may assume	1188
70.34	The boundary to remember	1188
70.35	References used in this chapter	1189

71 The Computational Polity 1191

71.1	From agent to polity	1191
71.2	The polity object	1192
71.3	Local state and shared state	1192
71.4	Capability is not authority	1192
71.5	Candidate is not conclusion	1193
71.6	Why humans remain inside the system boundary	1193
71.7	System-level cognition is an old idea	1194
71.8	Exact two-specialist witness	1194
71.9	The same components can still fail	1195
71.10	Shared memory as a polity institution	1195
71.11	Validators are roles, not oracles	1195
71.12	Governance is transition structure	1196
71.13	Tools and external effects	1196
71.14	Cost is heterogeneous	1196
71.15	Roles are not identities	1197
71.16	System-level competence needs a declared contract	1197
71.17	What this chapter does not claim	1197
71.18	Handoff to Beyond the Monolithic Model	1198
71.19	Durable lesson	1198
71.20	References used in this chapter	1198

72 The Machine Under the Mathematics 1199

72.1	Mathematics does not run in zero time	1199
72.2	The linear map survives; the implementation changes	1200
72.3	FLOPs are an accounting convention	1200
72.4	Throughput is not latency	1201
72.5	Data movement can dominate arithmetic	1201
72.6	The Roofline idea	1202

72.7 A roof is not a prediction	1203
72.8 The memory hierarchy	1204
72.9 Registers and local state	1204
72.10 Shared memory and reuse	1204
72.11 Global memory and bandwidth	1205
72.12 Warps and SIMT	1205
72.13 Parallelism is a resource, not a synonym for speed	1205
72.14 Occupancy	1206
72.15 Matrix and tensor hardware	1206
72.16 Tiles connect algebra to hardware	1207
72.17 Precision is several questions	1207
72.18 Mixed precision	1208
72.19 Kernels are physical programs	1208
72.20 Fusion can change the traffic graph	1209
72.21 Exact finite witness	1209
72.21.1 No cross-output reuse	1209
72.21.2 Perfect full-input reuse	1210
72.22 The bound moves even when the mathematics does not	1210
72.23 The witness is intentionally idealized	1211
72.24 Benchmarking can create false conclusions	1211
72.25 Peak versus sustained performance	1212
72.26 FLOP/s can reward doing more work	1212
72.27 Hardware-aware mathematics	1212
72.28 What belongs downstream	1212
72.29 Durable boundaries	1213
72.30 References used in this chapter	1213
73 Scaling, Parallelism, and Serving	1215
73.1 From hardware to systems	1216
73.2 A typed systems object	1216
73.3 Parallelism is not one thing	1217
73.4 Data parallelism	1217
73.5 Tensor parallelism	1218
73.6 Pipeline parallelism	1218
73.7 A pipeline bubble is schedule-relative	1219
73.8 Expert parallelism	1219
73.9 Collective semantics	1220
73.9.1 All-reduce	1220
73.9.2 Reduce-scatter	1220
73.9.3 All-gather	1220
73.9.4 All-to-all	1221
73.10 Logical topology and physical topology	1221
73.11 Exact four-rank witness	1221
73.12 Per-rank ring accounting	1222
73.13 Grouped placement	1222
73.14 Interleaved placement	1223

73.15	Equal arithmetic does not imply equal physical communication .	1223
73.16	A lower bound is not a latency measurement	1223
73.17	Communication can hide behind computation	1224
73.18	Stragglers	1225
73.19	Training scaling	1225
73.20	More devices can reduce efficiency	1225
73.21	Training and serving optimize different state	1226
73.22	Request latency	1226
73.23	Throughput	1227
73.24	Goodput	1227
73.25	Average latency and tail latency	1227
73.26	Iteration-level scheduling	1227
73.27	Batching is a scheduling policy	1228
73.28	KV-cache state	1228
73.29	Cache allocation can limit concurrency	1229
73.30	Prefill and decode are not the same workload	1229
73.31	Utilization	1229
73.32	Communication and memory interact	1229
73.33	Placement is an optimization variable	1230
73.34	Measured versus modeled performance	1230
73.35	Training benchmark record	1230
73.36	Serving benchmark record	1231
73.37	What the sources establish	1231
73.38	The systems layer is not one scalar	1232
73.39	Failure boundaries	1232
73.40	Systems reasoning	1233
73.41	References used in this chapter	1233

74 Experiments as Arguments 1235

74.1	From metric to argument	1236
74.2	An intervention is not an observation	1237
74.3	Hypothesis versus estimand	1238
74.4	Baselines are comparators; ablations are interventions	1239
74.5	The confounding trap	1240
74.6	Controls do not all play the same role	1241
74.7	Counterfactuals are the missing half of the comparison	1242
74.8	Seeds: useful, narrow, often overinterpreted	1242
74.9	Selection rules are part of the experiment	1243
74.10	Effect size is not statistical significance	1244
74.11	Benchmark score versus mechanism evidence	1245
74.12	Reproducibility is valuable and not sufficient	1246
74.13	Falsifiability belongs in the design	1247
74.14	Repetition, replication, and robustness	1247
74.15	The experiment table	1248
74.16	Failure modes	1248
74.16.1	Confounded comparison	1248

74.16.2 Baseline asymmetry	1248
74.16.3 Hidden multiplicity	1249
74.16.4 Metric substitution	1249
74.16.5 Seed laundering	1249
74.16.6 Ablation overclaim	1249
74.16.7 Reproducibility overclaim	1249
74.16.8 Benchmark overclaim	1249
74.16.9 Scope drift	1249
74.17 Experiments in adaptive-intelligence research	1249
74.18 What this chapter establishes	1250
74.19 Atlas handoff	1251
74.20 References used in this chapter	1251
75 Replayable Evidence Objects	1253
75.1 A result should be reconstructable	1253
75.2 The chain of custody	1254
75.3 Reproducibility, replication, and Atlas replay	1254
75.4 A provisional evidence object	1255
75.5 Why claim identity comes first	1256
75.6 Source identity	1256
75.7 Method identity	1257
75.8 Environment identity	1257
75.9 The tiny replay package	1258
75.10 What successful replay establishes	1259
75.11 A perfectly replayable wrong program	1259
75.12 A correct argument that is not replayable enough	1260
75.13 Mutable dependencies	1260
75.14 Seed is not environment	1260
75.15 Byte identity is not semantic identity	1261
75.16 Formal proof of the wrong statement	1261
75.17 Finite verification generalized too far	1262
75.18 Internal replay is not independent reproduction	1262
75.19 Provenance as a graph	1263
75.20 Evidence edges versus authority edges	1264
75.21 Current GCL technical-writing doctrine	1264
75.22 Forge, Solve, Cert as role separation	1265
75.23 The current OPENMATH lifecycle	1265
75.24 Replay evidence can exist while certification remains false	1266
75.25 Fail closed	1266
75.26 Evidence object versus archive	1267
75.27 The evidence object as a reconstruction test	1267
75.28 Five distinctions to keep visible	1268
75.28.1 Replay versus truth	1268
75.28.2 Replay versus replication	1268
75.28.3 Replay versus formal verification	1268
75.28.4 Replay versus certification	1268

75.28.5 Certification versus truth	1268
75.29 What a durable evidence object should make inspectable	1268
75.30 Atlas connections	1269
75.31 Closing view	1270
75.32 References used in this chapter	1270

76 Formal Methods and Machine-Checkable Claims 1271

76.1 The formal support graph	1272
76.2 Formal specification is not human intent	1272
76.3 Hoare logic: contracts as proof objects	1273
76.4 Invariants turn local preservation into global claims	1274
76.5 Exact witness: an infinite-state counter	1274
76.6 Three tests are not that theorem	1275
76.7 A passing implementation can still be wrong	1275
76.8 Proof of specification is not proof of implementation	1276
76.9 Tests and proofs are complementary	1276
76.10 Exhaustive finite checking is stronger than sampling	1277
76.11 Model checking	1277
76.12 The model can be the wrong model	1278
76.13 Trusted kernels	1278
76.14 Lean as a proof-assistant example	1279
76.15 Small trusted core does not mean no trust	1279
76.16 Proof terms versus theorem meaning	1280
76.17 Replayable proof is stronger than an unpinned proof artifact	1280
76.18 Replay is not proof	1281
76.19 Executable checkers and proof objects	1281
76.20 Static types are formal claims with bounded scope	1281
76.21 Runtime contracts are path-local evidence	1282
76.22 Property-based testing is still testing	1282
76.23 SMT and SAT trust paths	1283
76.24 Formal proof can expose missing requirements	1283
76.25 Invariants are contracts over evolution	1283
76.26 Safety and liveness are different	1284
76.27 Formal certification is overloaded language	1284
76.28 Machine-checked does not mean assumption-free	1285
76.29 The trust-base ledger	1285
76.30 Failure modes	1286
76.30.1 Proof-of-the-wrong-specification	1286
76.30.2 Model-equals-system	1286
76.30.3 Tests-equal-proof	1286
76.30.4 Automation-equals-trust	1286
76.30.5 Kernel-equals-no-assumptions	1286
76.30.6 Type-safe-equals-correct	1286
76.30.7 Machine-checked-equals-certified	1286
76.30.8 Formal-equals-replayable	1286
76.31 Why the exact witness matters	1286

76.32	Downstream handoff	1287
76.33	References used in this chapter	1287
77	Research as a State Machine	1289
77.1	From evidence objects to research state	1289
77.2	A bounded work package	1290
77.3	The research state is a product	1290
77.4	State change requires a declared condition	1291
77.5	Forge, Solve, and Cert are roles	1291
77.6	Receipt is not incorporation	1291
77.7	Idempotence belongs to canonical effect	1291
77.8	Duplicate is an identity relation	1292
77.9	Exact finite retry witness	1292
77.10	Advancement needs its own guard	1293
77.11	A missing programme disposition keeps the gate closed	1294
77.12	A complete witness vector opens the finite gate	1294
77.13	A missing required check also keeps the gate closed	1294
77.14	Advancement can be canonically idempotent too	1295
77.15	Idempotence is scoped, not magical	1295
77.16	Different evidence under one dispatch is not a retry	1296
77.17	Conflict is a scientific relation, not an intake shortcut	1296
77.18	Actor roles are not actor independence	1296
77.19	A narrow separation-of-duty predicate	1297
77.20	Same-actor review can fail a declared gate	1297
77.21	Certification is another state change	1298
77.22	Exact target identity matters	1298
77.23	Forge, Solve, and Cert can be understood as authority compartments	1298
77.24	Frontier advancement is not contributor authority	1299
77.25	Independent-intelligence intake is an authority boundary	1299
77.26	Zero-context is a work-context property	1299
77.27	Fail closed means preserve protected state	1300
77.28	Safety is not liveness	1300
77.29	A finite safety-without-liveness witness	1300
77.30	Recovery needs typed classes	1301
77.30.1	Retry	1301
77.30.2	New evidence	1301
77.30.3	Supersession	1301
77.30.4	Implementation repair	1301
77.30.5	Policy change	1301
77.31	Recoverable tooling failure is not scientific refutation	1301
77.32	Protected repair requires replay	1302
77.33	Public lifecycle case study	1302
77.34	The case study also exposes authority limits	1302
77.35	State machines make authority inspectable	1303
77.36	The machine does not prove the science	1303
77.37	Machine state should not collapse scientific uncertainty	1303

77.38	Bounded work packages protect handoff quality	1303
77.39	Independent actors need bounded interfaces too	1304
77.40	Idempotence enables automation under retries	1304
77.41	History should remain append-only where possible	1304
77.42	A practical research-state ledger	1304
77.43	Failure modes	1305
77.43.1	Done-as-one-state	1305
77.43.2	Retry-multiplies-authority	1305
77.43.3	Dispatch-key-overwrite	1305
77.43.4	Replay-equals-truth	1305
77.43.5	Reviewer-label-equals-independence	1306
77.43.6	Contributor-equals-scheduler	1306
77.43.7	Safe-equals-live	1306
77.43.8	Repair-reuses-stale-evidence	1306
77.44	What the finite witness establishes	1306
77.45	Downstream handoff	1306
77.46	References used in this chapter	1307

78 Governed Adaptation 1309

78.1	Adaptation as a governed transition	1310
78.2	Authority is typed	1310
78.3	Execution is not promotion	1311
78.4	Evidence belongs to exact targets	1311
78.5	Canonical state and history	1312
78.6	Correction capacity enters the gate	1312
78.7	A finite admissibility gate	1313
78.8	Equal immediate value, unequal correction capacity	1313
78.8.1	Candidate A	1314
78.8.2	Candidate B	1314
78.9	Why a utility-only gate misses the difference	1315
78.10	Raw action count is not enough	1315
78.11	A backup is not recovery	1316
78.12	Corrigibility and intervention precedents	1316
78.13	Preserving correction can have cost	1317
78.14	Bounded authority	1317
78.15	Safety is not liveness	1318
78.16	Governance conformance is not substantive truth	1318
78.17	Repairs invalidate exact-target evidence	1319
78.18	Recovery must name a target	1319
78.19	State-relative authority	1320
78.20	Why this matters for adaptive intelligence	1320
78.21	Failure modes	1320
78.21.1	Capability as authority	1320
78.21.2	Promotion by execution	1320
78.21.3	Stale green evidence	1321
78.21.4	Cosmetic rollback	1321

78.21.5 Option counting	1321
78.21.6 Corrigibility by naming	1321
78.21.7 Safety without progress	1321
78.21.8 Governance as truth	1321
78.21.9 Unbounded delegation	1321
78.21.10 Recovery without revalidation	1321
78.22 Downstream handoff	1321
78.23 Epistemic status	1322
78.24 References used in this chapter	1322
79 Frontier Questions of Adaptive Intelligence	1325
79.1 A frontier is not a result	1325
79.2 The three governing prerequisites	1326
79.3 Geometry-derived optimization	1326
79.4 Operator-valued position	1326
79.5 Adaptive depth as error control	1327
79.6 Persistent and shared memory	1327
79.7 Learning progress as a search operator	1328
79.8 Minimal curricula and reasoning bases	1328
79.9 The Residual remains open at frontier scale	1328
79.10 Benign nonconvexity	1329
79.11 Boundary contracts	1329
79.12 Source boundaries	1329
79.13 Cross-programme questions	1330
79.14 How a programme item advances	1330
79.15 Negative results are progress	1330
79.16 Preferred research pattern	1331
79.17 What this chapter does not claim	1331
79.18 Why collect the programme now?	1331
79.19 Downstream handoff	1331
79.20 References used in this chapter	1332
80 Beyond the Monolithic Model	1333
80.1 A typed synthesis object	1333
80.2 Frontier status survives synthesis	1334
80.3 Polity becomes a systems interface	1334
80.4 Exact finite composition	1335
80.5 Evidence and provenance	1335
80.6 Governance and commit	1336
80.7 Shared memory	1336
80.8 Router ablation	1336
80.9 Validator ablation	1337
80.10 Memory ablation	1337
80.11 Governance ablation	1337
80.12 Capability and authority	1338
80.13 Evidence and governance	1338

80.14Memory and truth	1338
80.15Adaptation and permission	1339
80.16Tools inside the system boundary	1339
80.17Heterogeneous cost	1339
80.18Specialization does not imply net benefit	1339
80.19The monolithic model remains legitimate	1340
80.20A monolith can be internally composite	1340
80.21A distributed system can still be poorly composed	1340
80.22Evidence levels remain visible	1340
80.23Open research remains open	1340
80.24Durable Atlas disciplines	1341
80.25Architecture remains a design choice	1341
80.26Systems claims need a contract	1341
80.27Ablation is part of synthesis	1342
80.28Research and engineering consequence	1342
80.29No universal architecture theorem	1342
80.30The durable synthesis	1342
80.31References used in this chapter	1343

Chapter 1

The Adaptive-System Thesis

Epistemic status: Atlas Synthesis

Specification: `manuscript/specifications/ATLAS-CH-THESIS-001.md`

Documentary packet: `mathematics/derivations/ATLAS-CH-THESIS-001-DERIVATIONS.md`

Source lock: `sources/source-locks/ATLAS-CH-THESIS-001.yaml`

1.1 What this book is trying to explain

Machine learning is often introduced as a catalogue of model classes.

Linear models lead to multilayer networks. Convolutional networks lead to attention. Attention leads to Transformers. Transformers lead to mixture-of-experts systems, retrieval, tools, agents, and increasingly elaborate training pipelines.

That chronology is useful, but it hides a structural fact.

The objects that matter are no longer contained inside one function approximator.

A capable system may include:

- learned states;
- transformations acting on those states;
- persistent or external memory;
- numerical solvers;
- routing and scheduling;
- tools and other models;
- interaction protocols;
- evidence stores;
- human or machine review;
- mechanisms governing how the system may change itself.

The Atlas begins from that observation.

Its governing thesis is:

Adaptive intelligence is more usefully studied here as organized computation over states, operators, dynamics, memory, interfaces, evidence, and governance than as a catalogue of model classes.

This is the **Adaptive-System Thesis**.

It is the thesis of this monograph.

It is not introduced as an externally established theorem.

1.2 An atlas, not a catalogue

A catalogue tells us what objects exist.

An atlas tells us how a territory is charted.

It contains:

- coordinate systems;
- local maps;
- overlaps;
- landmarks;
- routes;
- scales;
- boundaries;
- regions that remain poorly mapped.

This book is organized in the second way.

A chapter on geometry does not exist merely because manifolds are mathematically important.

It exists because later chapters need a language for legal motion in constrained state spaces.

A chapter on non-normality does not exist merely because pseudospectra are elegant.

It exists because asymptotic eigenvalue stability can miss finite-horizon amplification.

A chapter on replayable evidence does not exist merely because provenance is administratively desirable.

It exists because an adaptive research system must know which claim is supported by which reconstructable path.

The atlas allegory has a limit.

A research programme is not static geography.

The territory can change when new algorithms, new evidence, or new forms of computation appear.

The map therefore has to be versioned.

1.3 The recurring shifts

The source inventory that seeded this monograph records a recurring sequence of conceptual shifts.

They are not presented as discoveries of this chapter.

They are the programme-level pattern the Atlas is being built to examine.

1.3.1 Vectors to operators

A vector tells us where we are in a representation space.

An operator tells us how states are transformed.

Many modern mechanisms become clearer when the transformation is treated as the central object rather than the stored vector alone.

Attention is one example.

Relative position may also be better understood as an operator acting on representation space rather than as an appended coordinate.

1.3.2 Layers to flows

A stack of layers is a discrete sequence.

The same stack can sometimes be interpreted as an approximation to an evolution law.

That viewpoint introduces questions of:

- stability;
- step size;
- reversibility;
- stiffness;
- splitting;
- error control.

Depth can then be studied partly as computational time.

This does not mean every network is literally a discretized differential equation.

It means numerical dynamics can reveal structure that layer counting alone hides.

1.3.3 Parameters to geometry

Parameter values live in spaces with constraints, symmetries, scales, and equivalences.

If a state or parameter is constrained to a sphere, Stiefel manifold, simplex, quotient space, or other structured domain, the geometry changes what constitutes a legal or meaningful update.

The Geometry keystone develops this point formally.

1.3.4 Routing to decision making

A router does more than compute a score.

It allocates scarce computation under uncertainty and capacity constraints.

That opens the door to online decision theory, regret, optionality, and coordination costs.

1.3.5 Context to compiled memory access

A context window is one memory surface.

It is not the only one.

An adaptive system can combine:

- parametric memory;
- external retrieval;
- associative stores;
- symbolic records;
- shared institutional memory;
- compiled context.

The Atlas therefore treats memory as infrastructure as well as model anatomy.

1.3.6 Agents to distributed systems

Once multiple models, tools, memories, humans, and validators interact, the relevant object is no longer a lone agent.

It is a coordinated system.

Questions of concurrency, communication, provenance, shared state, failure isolation, and authority become mathematically and operationally relevant.

1.3.7 Training to dynamical identification

An optimizer has state.

A model has state.

The data stream and scheduler can have state.

The combined training process can therefore be studied as a dynamical system rather than only as repeated minimization of a scalar loss.

1.3.8 Interpretability to intervention

Description is weaker than controlled substitution.

If a proposed internal object is said to matter, the stronger question is whether changing, replacing, suppressing, or preserving it changes behavior in the predicted way.

The Atlas therefore treats intervention and substitution as important diagnostic standards.

1.3.9 Architecture to composition of contracts

A system can fail even when every tensor shape matches.

Composition requires assumptions about meaning, geometry, perturbations, and numerical behavior.

The Boundary Contracts keystone develops this as an interface problem.

1.3.10 Result to evidence object

A research conclusion is not only a sentence.

It is supported by sources, methods, environments, artifacts, observations, interpretations, and review state.

The Replayable Evidence Objects keystone turns that support path into an explicit object.

1.4 A provisional system decomposition

For orientation, the Atlas uses the explanatory decomposition

$$\mathcal{A} = (\mathcal{S}, \mathcal{O}, \mathcal{D}, \mathcal{M}, \mathcal{I}, \mathcal{E}, \mathcal{G}).$$

Here:

$\mathcal{S}$ = states,

$\mathcal{O}$ = operators,

$\mathcal{D}$ = dynamics,

$\mathcal{M}$ = memory,

$\mathcal{I}$ = interfaces,

$\mathcal{E}$ = evidence,

and

$\mathcal{G}$ = governance.

This tuple is not a universal ontology.

It is a map legend.

Later chapters will refine each term and, where necessary, expose the places where the decomposition becomes inadequate.

1.5 Why the model boundary is too small

Consider a retrieval-augmented system.

Its answer can depend on:

- the model weights;
- the tokenizer;
- the query transformation;
- the retrieval index;
- the current contents of the index;
- ranking;
- context compilation;
- tool outputs;
- runtime policies.

If we ask only what is stored in the model parameters, we have chosen a system boundary that excludes part of the causal machinery.

The same issue appears in agent systems.

A planner that calls a solver and writes to shared memory cannot be understood entirely by inspecting the planner weights.

Capability is partly distributed over the arrangement.

This motivates a broader unit of analysis:

$$\boxed{\text{model} \subseteq \text{adaptive computational system.}}$$

The inclusion can be strict.

1.6 A useful thesis must be vulnerable

A thesis that explains everything after the fact explains very little.

The Adaptive-System Thesis therefore has pressure points.

It should become less attractive if:

- the proposed object distinctions repeatedly obscure rather than clarify important mechanisms;
- memory, interfaces, or governance turn out to be accidental implementation details rather than load-bearing computational structure;
- the operator/dynamics viewpoint adds no predictive or design value;
- composition can be handled adequately by shape-level software interfaces alone;
- evidence provenance proves irrelevant to the reliability of adaptive research systems;
- later chapters require entirely different primitives that cannot be expressed naturally through the Atlas roles.

The book is not arranged to prevent these outcomes.

It is arranged to make them visible.

1.7 Description versus programme

Some claims in the Atlas are descriptive.

For example:

- attention uses state-dependent mixing operators;
- momentum introduces optimizer state;
- a source hash identifies bytes;
- a Jacobian describes local differential sensitivity.

Other claims are programme-level.

For example:

- intelligent systems should externalize more world knowledge into governed shared memory;
- adaptive depth should be treated as numerical error control;
- boundary contracts should become first-class architectural objects;

- research systems should preserve replayable evidence objects.

The second class is not smuggled into the first.

A research programme can be useful before it becomes established theory.

It must still be labeled correctly.

1.8 Dependencies in the argument

The dependency graph is more than a way to navigate chapters: it records which mathematical or documentary objects a later argument is entitled to assume. An editorial reading order need not coincide with that dependency order. When a chapter borrows a definition, theorem, example, or evidence object, the relevant prerequisite must remain inspectable. That principle is part of the Atlas's method, not evidence that its broader thesis is true.

1.9 Tests the thesis must pass

The decomposition is useful only where it changes the questions we can ask and the errors we can detect. Later chapters must put four propositions at risk: (1) geometry and operators expose behavior hidden by coordinate descriptions; (2) dynamics and optimization explain finite-time effects missed by static summaries; (3) explicit interfaces, memory and coordination expose system-level failure modes; and (4) evidence records delimit which conclusions can be reconstructed and reused.

These are research obligations, not results proved by announcing a taxonomy. The counterexamples and limits in the chapters ahead are part of the test.

1.10 The map ahead

The immediate next chapter introduces four recurring roles:

- states;
- operators;
- flows;
- interfaces.

After that, the mathematical substrate develops:

- linear maps;
- probability and information;
- geometry;
- dynamics;
- numerical analysis;
- local-to-global reasoning.

Only then does the Atlas move deeply into representation, architecture, optimization, memory, routing, agents, and governed adaptation.

The recurring conceptual spine is:

geometry → operators → dynamics → optimization → composition → memory → coordination → diagnostics → gov
--

This is a route through the atlas.

It is not the only possible route.

1.11 Closing view

The central wager of this book is that the next useful abstraction boundary for machine intelligence is larger than the model.

It includes the transformations the model performs, the geometry in which those transformations occur, the dynamics through which they accumulate, the memory they consult, the interfaces through which components compose, the evidence by which claims are supported, and the governance by which adaptation is constrained.

Whether that wager survives is the work of the chapters ahead.

The Atlas begins by naming the territory.

It will spend the rest of the book trying to deserve the map.

1.12 References used in this chapter

This chapter is Atlas synthesis. Its load-bearing documentary sources are the source inventory, Atlas Map, Editorial Profile, and Chapter Composition Protocol bound in `sources/source-locks/ATLAS-CH-THESIS-001.yaml`. External mathematical claims are deferred to the downstream chapters that source them directly.

Chapter 2

How to Read a Mathematical Atlas

Epistemic status: Atlas Documentary Synthesis

Specification: manuscript/specifications/ATLAS-CH-MAP-001.md

Documentary packet: mathematics/derivations/ATLAS-CH-MAP-001-DOCUMENTARY.md

Source lock: sources/source-locks/ATLAS-CH-MAP-001.yaml

2.1 An atlas is not a road

A road tells you where to go next.

An atlas gives you enough structure to choose.

That distinction matters for this book.

A Mathematical Atlas of Adaptive Intelligence has an editorial order, but it is not designed to have only one valid reading order.

Different readers arrive with different objectives:

- learn the foundations;
- understand one technical result;
- reconstruct a computational witness;
- evaluate a research frontier;
- follow one concept across several domains;
- enter through a figure and descend into the mathematics.

The Atlas must support all of these without allowing convenience to erase prerequisites.

The governing rule is simple:

reading order is flexible; dependency and claim boundaries are not.

Start here: choose a reading route

Readers seeking a connected introduction can follow the printed order. For a technical target, follow its hard prerequisites backward before reading optional cross-links. For a conceptual question, trace the relevant theme across parts. For an open research question, inspect the chapter's claim boundary, exact sources, witnesses and failure modes. For formal or computational work, begin with the derivation and replay record. For visual learning, start with a figure and its declared literal and nonliteral meanings. The fuller route descriptions and a chapter-entry checklist appear below.

2.2 Map and territory

The word **atlas** is more than decoration.

A geographic atlas contains:

- maps at several scales;
- legends;
- coordinates;
- routes;
- landmarks;
- survey records;
- boundaries.

The mathematical correspondence is:

- territory $\leftrightarrow$ the mathematical and scientific subject;
- map $\leftrightarrow$ an organized representation of that subject;
- scale $\leftrightarrow$ level of resolution;
- legend $\leftrightarrow$ notation and epistemic labels;
- coordinates $\leftrightarrow$ stable chapter and object IDs;
- route $\leftrightarrow$ a reader-selected path;
- landmark $\leftrightarrow$ a chapter of unusual structural importance;
- survey record $\leftrightarrow$ source locks, witnesses, and audits.

The limit is equally important.

The territory is not exhausted by the map.

A route is not a unique curriculum.

A landmark is not evidence for every claim around it.

A legend tells us how to read a mark; it does not make the mark true.

2.3 Read at several resolutions

The Atlas is designed for movement among scales.

At the largest scale:

Atlas

shows the whole conceptual landscape.

Then come:

Part $\rightarrow$ Chapter $\rightarrow$ claim $\rightarrow$ support object.

A reader may begin broadly and zoom inward.

Or begin with one exact derivation and zoom outward.

Changing scale changes the amount of visible detail.

It must not change the epistemic status of the underlying claim.

A conjecture does not become established because it is drawn on a clean overview plate.

A theorem does not become weaker because we first encounter it through intuition.

2.4 Stable identity

The Atlas is expected to evolve.

Chapters may move.

Parts may be reorganized.

A chapter may gain or lose neighbors.

For that reason, identity is not carried primarily by final numbering.

It is carried by stable IDs such as

ATLAS-CH-DYN-001.

The Atlas Map explicitly treats numbering and physical order as revisable while stable IDs survive reordering.

This gives the book something like a coordinate grid.

A citation to a stable object should remain meaningful even when the local page neighborhood changes.

2.5 The map is not the dependency graph

Two structures coexist.

The **Atlas Map** answers:

What exists, and what role does it play in the book?

The **Dependency Graph** answers:

What may this chapter assume without rebuilding it?

They overlap.

They are not identical.

A chapter can appear nearby in the narrative without being a hard prerequisite.

A concept can illuminate another chapter without authorizing it to assume the concept has already been established.

2.6 Hard dependency

Suppose chapter (A) is a hard dependency of chapter (B).

Then (B) may consume the declared content of (A) without reconstructing it from first principles.

Write this schematically as

$$A \longrightarrow B.$$

The arrow is permission.

It says:

the downstream chapter may stand on this upstream object.

That permission creates an authorial obligation.

The upstream chapter must actually contain what the downstream chapter is entitled to assume.

This is why dependency audits matter.

2.7 Soft cross-link

A soft cross-link means something weaker.

Two chapters may illuminate one another:

$$A \leftrightarrow B.$$

But neither is automatically permitted to assume the other has been read.

A soft link is a conceptual bridge.

It is not prerequisite authority.

This distinction lets the Atlas remain richly cross-connected without turning every conceptual resonance into a cycle of mandatory reading.

2.8 Narrative is not mathematical authority

The dependency graph states this explicitly:

the graph is a dependency structure, not the book's narrative.

The converse is also true.

Narrative order is not dependency authority.

This gives the author freedom to place an accessible orientation chapter early while still exposing the technical mathematics that later chapters truly require.

It gives the reader freedom to choose a route while still knowing when a detour is mathematically necessary.

2.9 A compact epistemic legend

The Atlas distinguishes definitions, established external results, Atlas derivations, bounded computational witnesses, observations, interpretations, public project evidence, programme context, conjectures, open problems, and institutional statuses. These are distinct roles, not steps on a single scale. A green audit, a reproducible computation, or polished typography does not by itself prove a theorem or upgrade the scope of a claim.

An exact figure represents declared mathematics; a data-derived figure summarizes observations within a stated scope; a schematic figure makes a structural relationship legible without pretending to be measured geometry. The figure manifest determines what is literal and what is not.

A source lock identifies the consumed source and what that source can actually support. Exact identity is not proof of adequate authority. A replay confirms reconstruction under stated conditions, not truth, independent replication or certification. For definitions, examples, promotion failures and the claim-support discipline, continue to the next chapter, *Claims, Evidence, and Computational Witnesses*.

2.10 Six ways through the Atlas

There is no single privileged route.

The best route depends on what you are trying to do.

2.10.1 Route A — linear foundations

Follow the editorial sequence.

Best for:

- broad learning;
- conceptual continuity;
- readers who want the argument to unfold gradually.

Risk:

you may read more prerequisite material than a targeted question requires.

2.10.2 Route B — dependency-first

Choose a target chapter.

Follow hard dependencies backward until the path closes.

Then read forward.

Best for:

- technical work;
- minimal prerequisite reconstruction;
- verifying that a later result is actually supported.

Risk:

the route can be mathematically efficient but narratively austere.

2.10.3 Route C — concept/theme

Choose a concept such as:

- geometry;
- operators;
- memory;
- optimization;
- coordination;
- governance.

Follow soft cross-links and recurring objects across Parts.

Best for:

- synthesis;
- comparative reading;
- seeing one idea change form.

Risk:

soft links do not replace hard prerequisites.

2.10.4 Route D — research-frontier

Start at an open problem, conjecture, or GCL programme chapter.

Then descend.

Read:

1. the claim boundary;
2. the source lock;
3. the immediate mathematical prerequisites;
4. the computational witnesses;
5. the failure modes;
6. the relevant audit records.

Best for:

- evaluating a live research idea;
- separating established substrate from programme ambition.

Risk:

frontier prose can feel more mature than the evidence if the epistemic labels are ignored.

2.10.5 Route E — proof/replay

Begin with the result you want to trust.

Trace:

claim $\rightarrow$ derivation/proof $\rightarrow$ witness $\rightarrow$ source lock $\rightarrow$ audit/replay record.

Best for:

- reconstruction;
- adjudication;
- technical reuse.

Risk:

replayability can be mistaken for correctness unless proof and authority are read separately.

2.10.6 Route F — visual/Atlas-plate

Begin with a figure or conceptual plate.

Read its:

1. representation class;
2. literal semantics;
3. nonliteral semantics;
4. supporting derivation or witness;
5. claim boundary.

Best for:

- geometric intuition;
- operator intuition;
- rapid orientation.

Risk:

visual fluency can create false certainty if schematic elements are read literally.

2.11 Choosing a route

A practical routing table is:

Goal	Start with	Then follow
Learn the field systematically	editorial order	hard dependencies as they appear
Understand one technical chapter	target chapter	hard dependencies backward
Compare one concept across fields	theme	soft cross-links plus local prerequisites
Evaluate a frontier proposal	open-problem/programme chapter	source lock, prerequisites, witnesses, failures
Reproduce a result	claim/support object	derivation, witness, provenance, audit
Build intuition first	figure/plate	manifest semantics, mathematics, source boundary

The table chooses a route.

It does not choose what is true.

2.12 How to read a keystone

A keystone chapter has unusually large downstream reach.

That makes it important.

It does not make it infallible.

When reading a keystone:

1. identify the formal object it stabilizes;
2. inspect its exact derivations;
3. read its failure boundaries;
4. note which downstream chapters consume it;
5. inspect its audit disposition.

A defect in a high-leverage chapter can propagate widely.

That is why keystones deserve disproportionate scrutiny.

2.13 How to read a frontier chapter

A frontier chapter should be read in the opposite direction from a theorem.

Do not begin by asking:

Is this exciting?

Begin by asking:

Which parts are already established?

Which parts are Atlas derivation?

Which parts are computational evidence?

Which parts are interpretation?

Which sentence is the actual conjecture or open problem?

The Atlas is intentionally allowed to be imaginative.

Its epistemic labels keep imagination from borrowing the authority of proof.

2.14 Allegory as a bounded instrument

This book uses allegory deliberately.

Examples include:

- maps and territories;
- flows and film frames;
- interfaces and engineering contracts;
- memory and civic archives;

- distributed intelligence and scientific institutions.

The rule is always:

allegory $\rightarrow$ structural correspondence $\rightarrow$ formal object $\rightarrow$ limit.

An allegory is successful when it makes the formal structure easier to see.

It has failed when the reader can no longer tell which part is metaphor.

2.15 What changed in our picture?

The Atlas thesis says adaptive intelligence is better understood as a system of geometry, operators, dynamics, memory, composition, coordination, diagnostics, and governed adaptation.

This chapter adds a second-order lesson.

The book itself has to be adaptive without becoming unstable.

It needs:

- stable identity;
- revisable architecture;
- explicit dependencies;
- bounded evidence;
- recoverable provenance.

That is why the Atlas is versioned like a technical system rather than treated as a frozen sequence of pages.

2.16 Five reading errors to avoid

2.16.1 Chapter order treated as dependency order

Narrative placement does not authorize hidden assumptions.

2.16.2 Soft cross-link treated as prerequisite

Conceptual resonance is not a dependency edge.

2.16.3 Exact computation treated as proof automatically

A witness establishes only its bounded claim.

2.16.4 Replay treated as certification

Reconstructability and authority are different dimensions.

2.16.5 Schematic figure treated as measured geometry

Layout is not data unless the manifest says it is.

2.17 A compact reader protocol

When entering any unfamiliar Atlas chapter, ask:

1. What problem is this chapter solving?
2. What are its hard prerequisites?
3. What formal or documentary object does it introduce?
4. What is established externally?
5. What is derived here?
6. What is only witnessed computationally?
7. What is interpretation or programme context?
8. What would falsify or limit the claim?
9. Which figures are literal, data-derived, or schematic?
10. What downstream chapter is allowed to assume this work?

If those ten answers are visible, the chapter is doing its job.

2.18 Closing view

A mathematical atlas should make exploration possible without making rigor optional.

The reader is free to choose a route.

The author is not free to hide a dependency.

The reader is free to begin from a figure.

The figure is not free to conceal which marks are schematic.

The book is free to propose conjectures and programmes.

They are not free to masquerade as established results.

That is the compact bargain of the Atlas:

many routes, stable coordinates, explicit legends, reconstructable evidence.

The territory will remain larger than the book.

The task of the Atlas is to help us move through it without confusing navigation for proof.

2.19 References used in this chapter

This chapter is a documentary synthesis of project-internal canonical objects.

Primary locked sources:

- `governance/ATLAS_MAP.md`;
- `governance/DEPENDENCY_GRAPH.md`;
- `governance/CHAPTER_COMPOSITION_PROTOCOL.md`;
- `governance/EPISTEMIC_STATUS.yaml`;
- `governance/ATLAS_EDITORIAL_PROFILE.md`;
- `governance/FIGURE_REGISTER.yaml`;
- `governance/SOURCE_REGISTER.yaml`.

See `sources/source-locks/ATLAS-CH-MAP-001.yaml` for exact Git blob identities and declared authority.

Chapter 3

States, Operators, Flows, and Interfaces

Epistemic status: Atlas Synthesis

Specification: `manuscript/specifications/ATLAS-CH-OBJECTS-001.md`

Derivation packet: `mathematics/derivations/ATLAS-CH-OBJECTS-001-DERIVATIONS.md`

Source lock: `sources/source-locks/ATLAS-CH-OBJECTS-001.yaml`

3.1 The same tensor can be four different things

Suppose a program contains a square array

$$A \in \mathbb{R}^{n \times n}.$$

What is it?

It might be:

- a covariance state;
- a learned parameter;
- a linear operator;
- an adjacency matrix;
- a metric tensor in coordinates;
- an attention score matrix;
- an evidence artifact.

The shape does not answer the question.

The Atlas therefore begins with a discipline:

identify an object by the role it plays in the argument, not only by the container used to store it.

Four roles recur throughout this book:

state, operator, flow, interface.

These are explanatory categories.

They are not a universal ontology.

3.2 Noun, verb, motion, handshake

A useful first analogy is grammatical.

A **state** behaves like a noun: something has a value.

An **operator** behaves like a verb: something is done to a state.

A **flow** behaves like unfolding motion: transformations accumulate through an evolution parameter.

An **interface** behaves like a handshake: two objects agree on what may cross a boundary.

The analogy is useful because it catches category errors.

It also fails quickly.

A mathematical state can itself encode an operator.

An operator can be state-dependent.

A family of operators may fail to satisfy the exact algebraic law required of a true flow.

An interface can be learned or adaptive.

Natural-language grammar is therefore only an entry point.

The mathematics determines the object.

3.3 State

A state is an information-bearing value

$$x \in \mathcal{X}.$$

The state space $\mathcal{X}$ may be:

- Euclidean;

- discrete;
- probabilistic;
- constrained;
- manifold-valued;
- a product space;
- a space of functions;
- a structured record.

A state answers:

What information is currently instantiated?

Examples include:

- a hidden activation;
- an optimizer momentum buffer;
- a memory record;
- a probability distribution;
- a graph;
- a model checkpoint;
- a tuple-space configuration.

The word “state” is deliberately broader than “vector.”

3.4 Operator

An operator is a transformation

$$F : \mathcal{X} \rightarrow \mathcal{Y}.$$

It answers:

What transformation is being applied?

A linear operator has the special form

$$F(x) = Ax.$$

But many operators in the Atlas are nonlinear or state-dependent.

Examples include:

- attention;
- normalization;
- a retrieval rule;
- an optimizer update;
- a solver step;
- a routing map;
- a projection or retraction.

A matrix is not automatically an operator.

It becomes an operator when the model assigns it a transformation role.

3.4.1 First category error: array equals role

Let

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}.$$

If A is a covariance matrix, it is a state describing second-order structure.

If A acts by

$$x \mapsto Ax,$$

it is a linear operator.

The stored entries are identical.

The semantics are different.

This distinction becomes important later when the Atlas moves from vectors to operators.

Without role discipline, the phrase “the representation matrix” can hide whether we mean data, a transformation, or a coordinate encoding of something basis-independent.

3.5 Flow and dynamics

A global autonomous flow is a family of maps $\Phi_t : \mathcal{X} \rightarrow \mathcal{X}$, indexed by $t \in \mathbb{R}$, such that $\Phi_0 = \text{id}_{\mathcal{X}}$ and

$$\Phi_{t+s} = \Phi_t \circ \Phi_s \quad (t, s \in \mathbb{R}).$$

If only $t, s \geq 0$ are admitted, with the same identity and composition law, the corresponding object is a *semiflow*. A *local flow* instead permits $\Phi_t(x)$ only on a state-dependent time domain; the composition law is required only when both sides are defined. Existence for all real times must not be inferred from local existence.

If a differential equation

$$\dot{x} = f(x)$$

has a well-defined evolution, $\Phi_t(x_0)$ denotes the state after time t .

This gives a new kind of object.

A single operator says:

transform this state once.

A flow says:

here is how transformations accumulate across an evolution parameter.

This distinction becomes central when network depth is treated as computational time.

3.5.1 Second category error: one step equals dynamics

Consider a residual update

$$x_{k+1} = x_k + hf(x_k).$$

One update map is

$$F_h(x) = x + hf(x).$$

The depth-indexed computation is the sequence

$$x_0, x_1, x_2, \dots$$

generated by repeated application.

These are not the same object.

The single-step operator can be locally well behaved while repeated application has different long-horizon behavior.

Later chapters on numerical intelligence and non-normality make this distinction quantitative.

3.5.2 Flow-like is weaker than flow

Neural networks often invite continuous-time language.

That language can be useful.

It must be scoped.

If layer k uses a different transformation from layer $k + 1$, the depth-indexed family need not satisfy

$$\Phi_{t+s} = \Phi_t \circ \Phi_s$$

for one autonomous flow.

The safer statement may be:

the architecture admits a dynamical or integrator interpretation.

That is weaker than claiming an exact continuous-time flow.

The Atlas will use the stronger word only when the stronger structure is present.

3.6 Interface

Suppose

$$F : \mathcal{X} \rightarrow \mathcal{Y}$$

feeds

$$G : \mathcal{Y} \rightarrow \mathcal{Z}.$$

At minimum, the codomain of F must be admissible input for G .

Software type systems often check part of this requirement.

But a meaningful interface can also depend on:

- units;
- scale;
- normalization;
- geometric constraints;
- semantic interpretation;
- uncertainty;
- precision;
- differential sensitivity.

An interface therefore answers:

What is another component allowed to assume about what crosses this boundary?

The Boundary Contracts chapter later turns this question into a formal programme.

3.6.1 Third category error: matching shape equals composability

Let

$$F(x) = \frac{x}{\|x\|_2}$$

return direction only.

Let

$$G(y) = \|y\|_2$$

interpret magnitude as meaningful amplitude.

The shapes match.

The semantics do not.

For

$$x = (3, 4),$$

the original magnitude is 5, but

$$G(F(x)) = 1.$$

The interface failed before any matrix multiplication failed.

3.6.2 Interfaces are not interiors

A contract can describe:

- accepted inputs;
- guarantees;
- error bounds;
- invariants;
- provenance.

It does not need to expose the full internal mechanism.

This separation is important for modular intelligence.

If every component must know every other component's interior, composition does not scale.

But abstraction has a price.

A contract can omit something the downstream system actually needs.

The problem is therefore not:

hide as much as possible.

It is:

expose enough for the property the composition is supposed to preserve.

3.7 State-dependent operators

The four roles are not rigid boxes.

Attention provides a useful example.

For fixed queries and keys, the normalized attention matrix A acts linearly on values:

$$V \mapsto AV.$$

But in the full layer,

$$A = A(X)$$

depends on the current state X .

The operator itself is state-dependent.

This is not a contradiction.

It is exactly why role language is useful.

We can distinguish:

- the current state;
- the operator instantiated from that state;
- the transformation applied to another part of the state.

3.7.1 Operator-valued state

Sometimes a state contains an object that later acts as an operator.

An optimizer state may contain a preconditioner.

A memory system may store a learned routing matrix.

A hypernetwork may emit weights that define another map.

The same mathematical object can therefore change role across a pipeline.

The rule is not:

every object has one eternal category.

The rule is:

state the role being used in the present argument.

3.7.2 Evidence is another object class

Later in the Atlas, evidence records become explicit.

An evidence object may describe an operator:

- which source defined it;
- which code implemented it;
- which run exercised it;
- which output was observed.

The evidence record is not the operator.

Confusing those categories produces a different kind of error.

A green replay tells us that an evidence path reconstructed under its declared conditions.

It does not transform the evidence record into the mathematical object it describes.

3.8 Why this vocabulary matters

The rest of the Atlas repeatedly changes levels of description.

Linear algebra moves between coordinates and operators.

Geometry distinguishes ambient coordinates from admissible states.

Attention distinguishes scores from mixing operators.

Optimization adds optimizer state to model state.

Numerical analysis distinguishes one update from accumulated dynamics.

Memory distinguishes stored content from retrieval operators.

Composition distinguishes component interiors from boundary obligations.

Scientific method distinguishes claims from their evidence paths.

Without a small amount of object discipline, these chapters would reuse the same words for incompatible roles.

3.8.1 The four questions

When encountering a new object, ask four questions.

3.8.2 State

What information is instantiated?

3.8.3 Operator

What transformation acts?

3.8.4 Flow

How do transformations accumulate?

3.8.5 Interface

What may another component assume at the boundary?

Not every object requires all four answers.

But asking the questions often exposes hidden assumptions.

3.9 Handoff to linear algebra

The next mathematical chapter specializes the operator role.

For finite-dimensional linear maps, bases turn operators into matrices.

That creates a powerful computational language:

- projections;
- eigendecompositions;
- singular values;
- pseudoinverses;
- norms;
- conditioning;
- low-rank structure.

The important point is already in place:

matrix coordinates $\neq$ operator role by definition.

Linear algebra will now make that distinction precise.

3.9.1 Handoff to information

Probability introduces another kind of state.

A distribution is an information-bearing mathematical object.

Entropy, KL divergence, and mutual information are functionals of such states.

Again, storage format is secondary.

What matters is the declared probability model and the operation being performed on it.

3.10 Closing view

The first conceptual discipline of the Atlas is simple:

Do not ask only what shape an object has.

Ask what role it plays.

A state is what is instantiated.

An operator transforms.

A flow organizes transformation through evolution.

An interface states what composition may assume.

These distinctions are not the final theory.

They are the grammar that lets the rest of the book say what kind of thing it is talking about.

3.11 References used in this chapter

This chapter is Atlas synthesis. Its project-local vocabulary is bound through `sources/source-locks/ATLAS-CH-OBJECTS-001.yaml` to the source inventory, Mathematical Lexicon, and Atlas Map. Standard mathematical instances are developed and externally sourced in downstream chapters.

Chapter 4

Claims, Evidence, and Computational Witnesses

Epistemic status: Atlas documentary synthesis with one exact finite computational witness.

Specification: manuscript/specifications/ATLAS-CH-EVIDENCE-001.md

Documentary packet: mathematics/derivations/ATLAS-CH-EVIDENCE-001-DOCUMENTARY.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-EVIDENCE-001.md

Source lock: sources/source-locks/ATLAS-CH-EVIDENCE-001.yaml

A mathematical atlas does not merely contain statements. It tells the reader why some statements may be used as foundations, why others are observations, why still others are proposals, and where the boundary between them lies.

That distinction matters because polished exposition can make unlike things look deceptively similar. A theorem and a numerical plot may occupy adjacent pages. A conjecture may be written in notation as crisp as a proved result. A project record may contain exact hashes and still say nothing about whether the mathematics is true. A computational witness may be exact down to the last bit and still establish only a finite claim.

The first discipline of evidence is therefore separation.

A claim is one object. Its support is another. The interpretation placed on that support is another. The institutional state attached to the work is another.

The Atlas will often connect these objects. It must not fuse them.

4.1 Start with the exact claim

Before asking whether evidence is strong, ask what the evidence is supposed to support.

Compare:

- this matrix has spectral norm greater than one;
- this optimizer can amplify perturbations in this toy system;
- this training run exhibited a transient spike;
- non-normality caused the spike;
- non-normal transient amplification is prevalent in frontier training.

These statements are not interchangeable. They differ in object, quantifier, domain, and ambition. A calculation sufficient for the first may merely illustrate the second. An experiment relevant to the third may be suggestive but insufficient for the fourth. None automatically establishes the fifth.

Evidence becomes intelligible only after the claim is made precise enough to fail.

For that reason the Atlas treats claim identity as part of the scientific object, not as decorative prose wrapped around a result.

4.2 The claim-support packet

The chapter will use a compact documentary object:

$$K = (q, \tau, S, \Omega, N, D).$$

Its fields are:

q — the exact claim.

τ — the declared epistemic class.

S — the support objects or support routes.

Ω — the scope: hypotheses, domain, dataset, run, environment, version, numerical convention, or other boundary that controls where the support applies.

N — explicit non-entailments: stronger conclusions the support does not license.

D — downstream permission: what a later chapter may consume without silently strengthening the claim.

We write

$$S \rightsquigarrow_{\Omega, \tau} q.$$

This records a scoped evidentiary support judgment, not a logical entailment. The support relation must be assessed against the particular claim and scope.

The arrow is intentionally not a symbol for logical entailment.

Sometimes the support route is a proof, and then a genuine logical relation can be stated. Sometimes it is a finite computation, an empirical observation, a source-locked public artifact, or a review record. These objects can support claims without all functioning as proofs.

The notation exists to prevent a common category error: treating every reason for belief as though it were the same kind of mathematical implication.

4.3 Eleven labels, not eleven rungs

The Atlas uses eleven canonical reader-facing epistemic classes.

Class	What it records	What it does not create by itself
Definition	A stipulated mathematical or documentary object	External existence or empirical truth
Established Result	A result supported by appropriate external scholarly or formal authority	Permission to drop the source hypotheses
Atlas Derivation	A derivation carried out in this book from stated assumptions	External attribution or independent validation
Computational Witness	A reproducible symbolic, numerical, finite-search, simulation, graphical, or replay object	Proof or certification merely by being exact or reproducible
Observation	What was empirically or computationally observed under stated conditions	Universal mechanism or prevalence
Interpretation	A reasoned reading of results, observations, or witnesses	Replacement for the underlying evidence
GCL Public Project Evidence	A claim grounded in an exact public GCL artifact	Authority beyond what that artifact actually contains

Class	What it records	What it does not create by itself
GCL Programme	A research direction, hypothesis, remembered programme term, or planned construction	Public implementation or completed result
Conjecture	A substantive proposed claim not yet sufficiently proved	Established result
Open Problem	A precise unresolved question or target	Existence of a solution
Institutional Status	A governed state such as reviewed, accepted, superseded, adjudicated, or certified	Mathematical truth by status alone

It is tempting to place these classes on a ladder from weak to strong. That would be a mistake.

A definition is not weak evidence; it is not evidence of that kind at all. An open problem is not a low-confidence theorem. Institutional status records a relation to an authority process, not a probability that a mathematical sentence is true. Interpretation sits on top of evidence without replacing it. Programme context can be valuable for choosing research directions while remaining deliberately barred from public implementation claims.

The vocabulary is therefore closer to a set of typed roles than to a single confidence score. The classes do not form a scalar ladder.

Within a particular role, support can certainly be better or worse. A proof can be correct or defective. An experiment can be well or poorly controlled. A source can be primary or remote. But those judgments do not turn the role taxonomy itself into one scalar order.

4.4 Definition is not discovery

Definitions are unusually powerful because they let a subject become precise.

Suppose the Atlas defines a boundary contract, a replayable evidence object, or a Residual. The definition can be exact and useful immediately. It can support derivations about any object satisfying the definition.

What it cannot do is establish that the defined object occurs in nature, in a deployed system, or in a frontier model.

This sounds obvious when stated explicitly. It is easy to forget when a definition is elegant.

A name gives a phenomenon a handle. It does not give the phenomenon empirical prevalence.

4.5 Established results and Atlas derivations

An established result enters the Atlas from an appropriate external scholarly or formal source. The source matters, but so do its hypotheses.

If a theorem holds for compact operators, smooth manifolds, convex objectives, independent samples, finite state spaces, or asymptotic limits, those conditions are part of the object being imported. Citation does not dissolve them.

An Atlas derivation is different. It is mathematics carried out in the book from declared assumptions or earlier results.

The distinction is not an insult to either class. It is provenance.

A correct Atlas derivation may be entirely rigorous. Its label tells the reader where the derivation was performed and where responsibility for the argument lies. If the same statement is later located in external literature, the source record can be enriched. The original derivation does not retroactively become someone else's result.

4.6 Computational witnesses

A computational witness is one of the Atlas's most useful evidence forms because many structures become visible only when symbolic, numerical, finite-search, graphical, simulation, or replay machinery is allowed to participate.

The rule is simple:

A witness must be as exact about its boundary as it is about its output.

Consider the elementary parity example recorded in ATLAS-CW-EVIDENCE-001.

We enumerate every integer from -10 through 10 and compute $n(n-1)$ modulo 2 using exact integer arithmetic. Every residue is zero.

That computation establishes a complete finite statement:

for each of those 21 integers, $n(n-1)$ is even.

It does not prove the universal statement:

for every integer n , $n(n-1)$ is even.

The universal statement has a separate proof. Two consecutive integers have opposite parity, so one is even, and their product is even.

The witness and the proof agree on the tested values. They do different logical work.

This is the central discipline of computational evidence. Exactness protects the computation from one class of doubt. Reproducibility protects the reconstruction route from another. Neither property silently enlarges the domain of the claim.

4.6.1 What a witness can be excellent at

A computational witness can:

- verify a finite family exhaustively;
- expose a counterexample;
- check an algebraic identity at a declared set of exact inputs;
- evaluate a numerical quantity under a stated convention;
- reconstruct an artifact from pinned sources;
- make geometry visible;
- replay a deterministic procedure;
- falsify a proposed universal statement with one valid counterexample.

4.6.2 What a witness cannot acquire by presentation

A witness does not become a theorem because the plot is attractive.

It does not become independent replication because another process reran the same internal pipeline.

It does not become certification because CI is green.

It does not become causal explanation because the observed pattern matches an intuition.

The support route remains what it is.

4.7 Observation is not interpretation

An observation records what happened under stated conditions.

For example:

- a loss curve rose sharply at a particular step;
- one intervention reduced a measured quantity;
- a router changed assignments under a perturbation;
- an ablation changed test accuracy on a declared benchmark;
- a diagnostic statistic concentrated in a particular layer.

Interpretation asks what the observation means.

Perhaps the spike reflects transient amplification. Perhaps the intervention removed a mechanism. Perhaps the router statistic indicates specialization. These can be good interpretations. They are still interpretations.

The distinction is especially important when several mechanisms can produce similar observables.

A useful discipline is to ask whether another mechanism could produce the same observation. If yes, the observation may constrain the explanation without identifying it.

This is why later diagnostic chapters will need interventions, substitutions, controls, counterfactuals, and competing explanations. The evidence grammar comes first so those chapters can state exactly which inferential step each experiment supplies.

4.8 Public project evidence and programme context

Research programmes develop language faster than every idea becomes a stable public artifact.

The Atlas therefore separates two project-local classes.

GCL Public Project Evidence refers to an exact public object: a repository file, commit, artifact, result, or other inspectable object whose identity can be locked.

GCL Programme refers to a research direction, remembered term, planned construction, hypothesis, or context that is not being asserted as exact public evidence.

The distinction protects both.

Public evidence should not be weakened by mixing it with recollection. Programme context should not be forced into false certainty merely to make the prose sound complete.

A programme idea can motivate a chapter, suggest an experiment, or identify a useful open problem. It simply cannot be laundered into a statement that an implementation or result exists publicly when no exact public object has been bound.

4.9 Conjecture and open problem

A conjecture and an open problem are related but different.

A conjecture says: here is a substantive statement we suspect may be true.

An open problem says: here is an unresolved question or target.

A good research programme often contains both.

For example, one may conjecture that a certain invariant controls transfer, while the corresponding open problem asks for necessary and sufficient conditions under which reconstruction succeeds.

Precision does not demote either object. A conjecture can be mathematically sharp. An open problem can have a detailed attack plan. What remains forbidden is pretending that a plausible route is already a solution.

4.10 Institutional status is a different axis

The Atlas also records institutional states.

A result may be reviewed, audited, accepted, superseded, adjudicated, certified, or merged.

These states matter. They tell the reader what process the object has passed through and which authority made the decision.

But process state and mathematical truth are not synonyms.

Green CI may show that declared automated checks passed.

A documentary audit may show that source identities, scope statements, and manuscript claims agree.

A formal certificate may establish something much stronger if the certification authority and formal statement are themselves specified.

The error is not in having institutional status. The error is in allowing the status word to float free of the authority and checks that give it meaning.

The same discipline applies in reverse. A true mathematical argument can exist before any institution has reviewed it. Lack of institutional status is not a proof of falsehood.

4.11 Source identity is not source authority

A source lock answers two different questions:

1. Which exact object did the chapter consume?
2. What is that object authoritative for?

These fields must remain separate.

A commit hash can identify a README perfectly. That does not make the README a primary authority for a historical theorem.

A DOI can identify a paper perfectly. That does not make every sentence in later secondary commentary a theorem of that paper.

A source can therefore be perfectly pinned and still be insufficient for the claim being made.

This is why the Atlas rejects citation laundering. Precision of identity is necessary for reconstructability. Adequacy of authority is a separate judgment.

4.12 Promotion requires new support

Suppose a chapter begins with an Observation and later acquires a proof of a general theorem.

That is a legitimate promotion because the support changed.

Suppose instead that the observation is rewritten in more confident prose, placed in a polished figure, cited repeatedly, and merged after green CI.

Nothing epistemic has changed.

Presentation can reveal status. It cannot create status.

In the claim-support packet, a genuine promotion changes τ only when S and usually Ω change in a way that licenses the stronger q.

This yields a useful editorial test:

If the claim sounds stronger after revision, point to the new support object that justifies the stronger wording.

If no such object exists, the prose has outrun the evidence.

4.13 Five common promotion failures

4.13.1 Finite check to universal theorem

A finite exhaustive search proves a finite exhaustive claim.

It proves an infinite claim only when there is an additional argument reducing the infinite domain to the finite search.

Timeout is not a nonexistence proof. Large coverage is not an infinite quantifier.

4.13.2 Repeatable observation to universal mechanism

Repeated observation within one family improves confidence that the observation is stable in that family.

It does not alone prove that one mechanism caused it, that the mechanism is unique, or that the phenomenon is prevalent outside the tested regime.

4.13.3 Narrow proof to broad prose

A proof can be flawless and still fail to support the sentence written above it.

Typical causes include a wider domain, missing hypothesis, stronger quantifier, causal language, or an implicit approximation-to-exactness jump.

The cure is not to distrust proof. It is to align q with the theorem actually proved.

4.13.4 Precise citation to excessive authority

A source can be exact and still be the wrong kind of authority.

Source identity and source authority must both survive inspection.

4.13.5 Procedure to truth

CI, replay, review, audit, merge, adjudication, acceptance, and certification are procedural or institutional facts.

Each can support important claims about what was checked.

None may be treated as a generic truth operator.

4.14 What later chapters may now assume

Four architectural consumers depend directly on this chapter.

ATLAS-CH-DATA-001 — Data Quality, Mixtures, and Contamination may assume that empirical claims must retain dataset, mixture, benchmark, contamination, and run scope.

ATLAS-CH-MECHDIAG-001 — Mechanistic Diagnostics may assume the distinction between Observation and Interpretation and may require stronger interventions before promoting correlation to mechanism.

ATLAS-CH-EVIDEX-001 — Evidence Extraction and Claim Discipline may assume the claim-support packet and can refine it into operational extraction and adjudication procedures.

ATLAS-CH-EXPERIMENT-001 — Experiments as Arguments may assume that controls, ablations, counterfactuals, effect sizes, seeds, and reproducibility are support routes whose force depends on the exact claim and scope.

The handoff is deliberately modest.

Later chapters inherit the grammar. They do not inherit domain evidence they have not yet produced.

4.15 The durable habit

The strongest practical lesson of this chapter is a question sequence.

When encountering a technical statement, ask:

What exactly is the claim?

What class is it in?

What supports it?

Under what scope?

What stronger inference is not licensed?

What may a later chapter safely reuse?

If those questions can be answered, evidence becomes inspectable rather than atmospheric.

That is enough structure to let a large monograph combine theorem, derivation, computation, experiment, interpretation, programme, and governance without pretending they are the same thing.

4.16 References used in this chapter

This chapter is a project-local documentary synthesis. Its load-bearing sources are exact Atlas governance and prerequisite objects pinned in:

`sources/source-locks/ATLAS-CH-EVIDENCE-001.yaml`

The chapter makes no claim that the Atlas taxonomy is a universal or uniquely correct theory of scientific epistemology.

Chapter 5

Linear Maps and Decompositions

Epistemic status: Established Theory + Atlas Derivation

Specification: manuscript/specifications/ATLAS-CH-LINALG-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-LINALG-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-LINALG-001.md

Source lock: sources/source-locks/ATLAS-CH-LINALG-001.yaml

5.1 Linear algebra as transformation language

A matrix is often introduced as a rectangular table of numbers.

That description is correct and insufficient.

For the Atlas, the important object is the linear map.

Given finite-dimensional vector spaces V and W , a linear transformation is a map

$$T : V \rightarrow W$$

satisfying

$$T(\alpha x + \beta y) = \alpha T(x) + \beta T(y).$$

Once bases are chosen, T is represented by a matrix A .

The matrix is a coordinate description of the map.

Changing basis changes the coordinates used to describe the same transformation.

Changing the operator changes the transformation itself.

This distinction is the first reason linear algebra matters here.

5.2 Changing lenses versus changing the machine

Imagine observing one machine through different camera angles.

The pictures change.

The machine does not.

A change of basis plays a similar role.

The coordinates of a vector and the matrix representing an operator change together.

The underlying linear relation can remain the same.

The analogy has a limit.

A basis change is an exact algebraic transformation, not merely a visual viewpoint.

And many learned transformations are nonlinear, state-dependent, or not globally representable by one fixed matrix.

The analogy is useful only for separating coordinate description from operator identity.

5.3 Bases and coordinate representations

Let

$$B = \{b_1, \dots, b_n\}$$

be a basis of V .

Any vector $x \in V$ has a unique coordinate vector

$$[x]_B.$$

If $T : V \rightarrow W$ and C is a basis of W , then there is a matrix

$$[T]_{C \leftarrow B}$$

such that

$$[T(x)]_C = [T]_{C \leftarrow B} [x]_B.$$

This equation is the bridge between abstract linear maps and matrix computation.

The Atlas will often work directly with matrices.

But the operator viewpoint remains useful because it prevents basis-dependent coordinates from being mistaken for invariant structure.

5.4 Subspaces

A subspace is a subset closed under linear combination.

Three recurring subspaces are:

$$\text{range}(A),$$

$$\text{null}(A),$$

and, for a matrix acting on an inner-product space, orthogonal complements of these spaces.

Subspaces matter because many later problems are really questions about which directions:

- survive a transformation;
- are erased;
- are amplified;
- are identifiable;
- are reachable.

Rank is the dimension of the image:

$$\text{rank}(A) = \dim \text{range}(A).$$

A low-rank operator therefore communicates through a lower-dimensional effective channel.

That can be useful compression.

It can also be information loss.

5.5 Orthogonal projection

Suppose $Q \in \mathbb{R}^{n \times k}$ has orthonormal columns:

$$Q^\top Q = I.$$

Then

$$P = QQ^\top$$

is the orthogonal projector onto the column space of Q .

Indeed,

$$P^2 = QQ^\top QQ^\top = QIQ^\top = P,$$

and

$$P^\top = P.$$

Projection is one of the simplest examples of an operator whose geometry is visible.

It keeps components inside one subspace and removes orthogonal components.

Later chapters use this language for tangent projections, subspace methods, low-rank structure, and constrained updates.

5.6 Eigendecomposition

For a square matrix A , an eigenpair satisfies

$$Av = \lambda v.$$

An eigenvector is a direction mapped back into its own span.

If A is diagonalizable,

$$A = V\Lambda V^{-1}.$$

Then powers satisfy

$$A^k = V\Lambda^k V^{-1}.$$

This is powerful.

It is also easy to overuse.

Eigenvalues directly describe asymptotic modal behavior only under assumptions that later chapters will make explicit.

If V is badly conditioned, eigenvalues can be a poor guide to finite-time amplification.

The Non-normality keystone exists largely because that distinction matters.

5.7 Normality

A real matrix is normal when

$$A^\top A = AA^\top.$$

In complex spaces the transpose is replaced by the adjoint.

Normal matrices admit orthonormal eigenvector bases.

This property greatly simplifies spectral reasoning.

A non-normal matrix need not.

The Atlas will therefore use the word “normal” carefully.

It does not mean statistically normal.

It describes an operator relation.

5.8 Singular-value decomposition

For any real matrix

$$A \in \mathbb{R}^{m \times n},$$

the singular-value decomposition is

$$\boxed{A = U\Sigma V^\top}$$

with orthonormal columns in U and V , and nonnegative singular values in Σ .

The right singular vectors identify input directions.

The left singular vectors identify corresponding output directions.

The singular values quantify amplification:

$$Av_i = \sigma_i u_i.$$

This makes the SVD one of the Atlas's central linear tools.

It works for rectangular matrices.

It does not require diagonalizability.

And it directly answers Euclidean gain questions.

Standard treatments appear in Trefethen and Bau (Trefethen and David Bau 1997), Horn and Johnson (Horn and Johnson 2012), and Golub and Van Loan (Golub and Loan 2013).

5.9 Operator norm

The induced Euclidean norm is

$$\|A\|_2 = \sup_{\|x\|_2=1} \|Ax\|_2.$$

The SVD gives

$$\|A\|_2 = \sigma_{\max}(A).$$

This is not merely notation.

It says the largest singular value is the greatest one-step amplification of a unit vector under the Euclidean norm.

Later chapters on boundary sensitivity, optimization, and transient growth use exactly this interpretation.

5.10 Eigenvalues and singular values answer different questions

Consider

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

Both eigenvalues equal 1.

But the singular values are

$$\sigma_1 = \frac{1 + \sqrt{5}}{2},$$

and

$$\sigma_2 = \frac{\sqrt{5} - 1}{2}.$$

Therefore

$$\|A\|_2 = \frac{1 + \sqrt{5}}{2} > 1.$$

A unit vector can be amplified in one step even though every eigenvalue has modulus 1.

This is the smallest important lesson:

spectral radius $\neq$ operator norm

in general.

The Non-normality chapter develops the consequences.

5.11 Pseudoinverse and least squares

If a linear system

$$Ax = b$$

has no exact solution, one may seek

$$x_\star = \arg \min_x \|Ax - b\|_2.$$

Using

$$A = U\Sigma V^\top,$$

define the Moore–Penrose pseudoinverse

$$A^+ = V\Sigma^+U^\top,$$

where each nonzero singular value is inverted.

Then

$$x_{\star} = A^{+}b$$

gives the minimum-norm least-squares solution under the standard finite-dimensional assumptions.

This object recurs in inverse problems, linear probes, regression, and local approximations.

5.12 Low-rank approximation

Suppose

$$A = U\Sigma V^{\top}$$

with singular values

$$\sigma_1 \geq \sigma_2 \geq \cdots.$$

Truncating after k singular components gives

$$A_k = U_k \Sigma_k V_k^{\top}.$$

The Eckart–Young theorem identifies A_k as a best rank- k approximation in the spectral and Frobenius norms.

In the spectral norm,

$$\|A - A_k\|_2 = \sigma_{k+1}.$$

For the witness matrix above, the best rank-one error is

$$\frac{\sqrt{5} - 1}{2}.$$

Low rank is therefore a precise approximation concept.

It is not automatically semantic compression.

A small matrix approximation error can still erase a downstream feature that matters.

5.13 Conditioning

Some problems are intrinsically sensitive.

Suppose

$$D = \begin{pmatrix} 1 & 0 \\ 0 & 1/100 \end{pmatrix}.$$

Then

$$\sigma_{\max}(D) = 1, \quad \sigma_{\min}(D) = 1/100.$$

The 2-norm condition number is

$$\kappa_2(D) = 100.$$

The weak singular direction is much more sensitive to inversion.

This is a property of the problem.

It must be distinguished from the stability of a particular algorithm used to solve it.

That distinction becomes central in Numerical Intelligence.

5.14 Condition number is not optimization difficulty

A large matrix condition number can matter greatly.

But it does not by itself prove that a nonlinear training problem will be difficult.

Training dynamics also depend on:

- parameterization;
- optimizer state;
- curvature variation;
- stochastic gradients;
- non-normality;
- constraints;
- numerical precision;
- data ordering.

The Atlas therefore treats conditioning as one diagnostic quantity, not a universal explanation.

5.15 Block structure

Large transformations often have meaningful block structure.

For example,

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

can represent coupling among subsystems.

Block Jacobians later make optimizer-state dynamics explicit.

Boundary composition can also be analyzed through block sensitivity.

The point is structural:

a matrix need not be an undifferentiated table.

Its partition can encode a model of interactions.

5.16 Kronecker structure

For matrices A and B , the Kronecker product

$$A \otimes B$$

constructs a larger structured linear map.

Kronecker products appear in:

- separable transformations;
- tensor-product bases;
- covariance approximations;
- structured preconditioners.

This chapter does not develop the full theory.

It establishes the notation and the idea that large operators may be described compositionally.

5.17 What linearization does and does not buy us

Even nonlinear systems have local linear structure.

If

$$F : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

is differentiable, then near x ,

$$F(x + \delta) \approx F(x) + J_F(x)\delta.$$

The Jacobian is a linear operator.

This lets the Atlas reuse:

- singular values;
- operator norms;
- projections;
- condition estimates.

But local linearization is local.

A Jacobian at one point does not automatically describe a global nonlinear system.

The Boundary Contracts and optimizer-dynamics chapters both rely on this limitation.

5.18 Computational witness

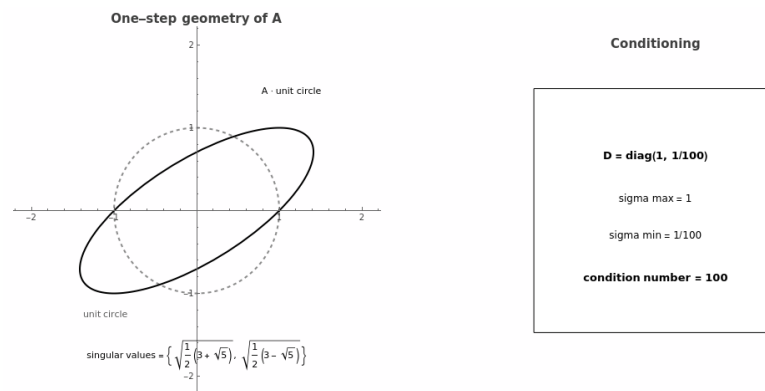


Figure 5.1: The unit circle mapped by a non-normal matrix into an ellipse beside an exact conditioning example with condition number 100.

The companion witness replays three exact facts.

For

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix},$$

Wolfram returns eigenvalues

$$1, 1$$

and singular values

$$\frac{1 + \sqrt{5}}{2}, \quad \frac{\sqrt{5} - 1}{2}.$$

For the projection

$$P = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix},$$

it verifies

$$P^2 = P.$$

For

$$D = \begin{pmatrix} 1 & 0 \\ 0 & 1/100 \end{pmatrix},$$

it verifies

$$\kappa_2(D) = 100.$$

The witness is exact finite-dimensional mathematics.

It does not imply a neural-network mechanism.

5.19 Five failure modes

5.19.1 Coordinates mistaken for invariants

A matrix entry can change under basis change while the operator remains the same.

5.19.2 Eigenvalues mistaken for all amplification

Non-normal operators can have large singular gain despite benign-looking eigenvalues.

5.19.3 Low rank mistaken for semantic sufficiency

A good norm approximation can still lose task-critical information.

5.19.4 Conditioning mistaken for algorithm failure

An ill-conditioned problem and an unstable algorithm are different objects.

5.19.5 Local linearization mistaken for global behavior

A Jacobian is a local differential object.

5.20 Atlas connections**Geometry.**

Tangent spaces are linear spaces attached to nonlinear manifolds.

Attention.

Attention matrices act as state-dependent linear mixing operators on value fields when the mixing weights are fixed.

Optimization.

Hessians, preconditioners, and update operators are linear-algebraic objects embedded in nonlinear dynamics.

Numerical intelligence.

Conditioning, projections, factorizations, and Krylov methods are core tools.

Memory and representation.

Low-rank and subspace structure can describe compressed state, but semantic adequacy is a separate question.

5.21 Closing view

Linear algebra is not here because neural networks contain matrices.

It is here because adaptive systems repeatedly ask linear questions inside larger nonlinear ones.

Which directions survive?

Which directions are amplified?

Which subspace matters?

Which information is lost?

Which coordinate change is merely descriptive?

Which operator is ill-conditioned?

The SVD, projections, pseudoinverses, norms, and decompositions give precise answers to those questions.

They become useful when we remember what kind of object they describe.

5.22 References used in this chapter

- (Trefethen and David Bau 1997)
- (Horn and Johnson 2012)
- (Golub and Loan 2013)

See `sources/source-locks/ATLAS-CH-LINALG-001.yaml` for exact source roles and claim scope.

Chapter 6

Normality, Pseudospectra, and Transient Growth

Epistemic status: mathematical exposition based on standard matrix analysis and pseudospectral theory, with Atlas-owned finite-dimensional derivations.

Primary figure: ‘ATLAS-FIG-PSPECTRUM-001‘

Derivation packet: ‘mathematics/derivations/ATLAS-CH-NONNORMAL-001-DERIVATIONS.md‘

6.1 Stable eigenvalues can tell an incomplete story

Suppose every eigenvalue of a matrix lies strictly inside the unit disk.

For a normal matrix, this strongly controls the powers of the matrix. For a non-normal matrix, it may not control what happens over the first few steps.

The key distinction is between:

asymptotic spectral behavior

and

finite-horizon amplification.

A matrix can satisfy

$$\rho(A) < 1$$

and still have

$$\|A^k\|_2 > 1$$

for some finite k .

This chapter explains exactly how.

The useful allegory is **aligned currents in a harbor**.

Each current may eventually decay, but if their directions are badly aligned, motion can transfer among them in a way that produces a temporary surge.

The correspondence is:

- modal decay $\leftrightarrow$ eigenvalues inside the unit disk;
- non-orthogonal modal geometry $\leftrightarrow$ non-normality;
- temporary surge $\leftrightarrow$ transient norm growth.

The limit is strict. A matrix need not represent a fluid. The analogy is only about geometric interaction among directions.

6.2 Normality

A real matrix A is normal when

$$A^\top A = AA^\top.$$

For complex matrices, transpose is replaced by conjugate transpose.

Normal matrices admit an orthogonal/unitary eigenbasis. In the real symmetric or complex normal setting,

$$A = U\Lambda U^*,$$

with U unitary.

Then

$$A^k = U\Lambda^k U^*,$$

so

$$\|A^k\|_2 = \|\Lambda^k\|_2 = \max_i |\lambda_i|^k = \rho(A)^k.$$

Thus if

$$\rho(A) < 1,$$

the operator norm decays monotonically with k .

For normal matrices, eigenvalue geometry and singular-value growth align unusually well (Horn and Johnson 2012).

6.3 Non-normality breaks that alignment

If A is diagonalizable but non-normal,

$$A = V\Lambda V^{-1},$$

then

$$A^k = V\Lambda^k V^{-1},$$

so

$$\|A^k\|_2 \leq \kappa_2(V)\rho(A)^k,$$

where

$$\kappa_2(V) = \|V\|_2\|V^{-1}\|_2.$$

A badly conditioned eigenvector matrix can therefore permit substantial finite-horizon amplification even when every eigenmode decays.

The deeper contrast is not simply

diagonalizable versus non-diagonalizable.

It is closer to

orthogonal modal geometry versus non-orthogonal modal geometry.

Normality removes the amplification associated with geometric misalignment of eigen-directions.

6.4 The smallest useful counterexample

Consider

$$A = \begin{pmatrix} a & K \\ 0 & a \end{pmatrix}, \quad |a| < 1.$$

Write

$$A = aI + KN, \quad N = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad N^2 = 0.$$

For each integer $n \geq 2$, the nilpotent binomial expansion truncates:

$$A^n = a^n I + na^{n-1}KN.$$

For these n , therefore

$$A^n = \begin{pmatrix} a^n & nKa^{n-1} \\ 0 & a^n \end{pmatrix}.$$

The low powers are $A^0 = I$ and $A^1 = A$. Treating them separately avoids the a^0 convention at $a = 0$.

The eigenvalue is always a , with algebraic multiplicity two, so

$$\rho(A) = |a|.$$

If $|a| < 1$, then $A^n \rightarrow 0$.

But the off-diagonal term

$$nKa^{n-1}$$

can first grow before eventual decay.

The matrix is asymptotically stable and can still amplify some perturbations strongly over a finite horizon.

6.5 Singular values expose the amplification

For

$$B = \begin{pmatrix} p & q \\ 0 & p \end{pmatrix},$$

we have

$$B^\top B = \begin{pmatrix} p^2 & pq \\ pq & p^2 + q^2 \end{pmatrix}.$$

Its eigenvalues are

$$\lambda_{\pm} = \frac{2p^2 + q^2 \pm |q|\sqrt{4p^2 + q^2}}{2}.$$

The spectral norm is therefore

$$\|B\|_2 = \sqrt{\frac{2p^2 + q^2 + |q|\sqrt{4p^2 + q^2}}{2}}.$$

For $B = A^n$ with $n \geq 2$,

$$p = a^n, \quad q = nKa^{n-1}.$$

This is an exact finite-horizon gain formula.

Nothing in the eigenvalue set alone displays the q -term.

6.6 A matched normal comparison

The Atlas witness fixes

$$a = \frac{4}{5}, \quad K = 4.$$

Then

$$A = \begin{pmatrix} 4/5 & 4 \\ 0 & 4/5 \end{pmatrix}.$$

Compare it with

$$N_0 = \frac{4}{5}I.$$

Both matrices have exactly the same eigenvalue set:

$$\left\{ \frac{4}{5}, \frac{4}{5} \right\}.$$

For the normal matrix,

$$\|N_0^n\|_2 = \left(\frac{4}{5}\right)^n.$$

For the non-normal matrix, Wolfram evaluation gives

$$\max_{0 \leq n \leq 40} \|A^n\|_2 = 8.2124290544 \dots$$

at

$$n = 4.$$

The two matrices have the same eigenvalue story and radically different finite-time stories.

6.7 The primary figure

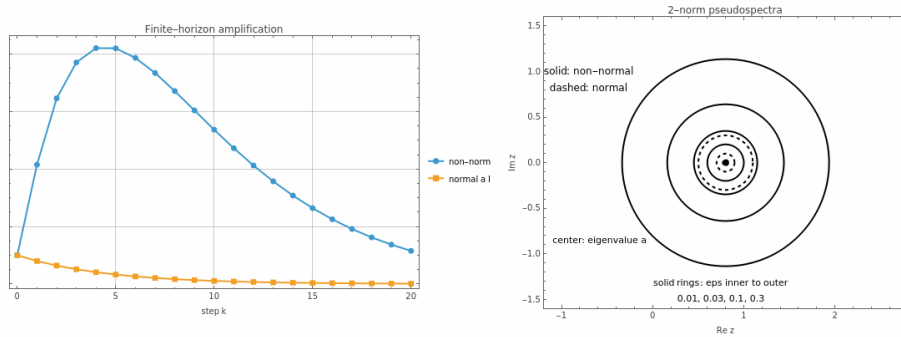


Figure 6.1: Matched-eigenvalue normal and non-normal matrices compared through transient 2-norm gain and pseudospectral contours.

The left panel compares

$$\|A^n\|_2$$

against

$$\|N_0^n\|_2.$$

The right panel compares their ε -pseudospectral boundaries.

The plate is not merely numerical decoration. For this 2×2 example, the pseudospectral radii are available in closed form.

6.8 The pseudospectrum

We use the closed 2-norm convention

$$\Lambda_\varepsilon(A) = \{z \in \mathbb{C} : \sigma_{\min}(zI - A) \leq \varepsilon\}.$$

Outside the spectrum,

$$\|(zI - A)^{-1}\|_2 = \frac{1}{\sigma_{\min}(zI - A)},$$

so equivalently,

$$\Lambda_\varepsilon(A) = \sigma(A) \cup \{z : \|(zI - A)^{-1}\|_2 \geq \varepsilon^{-1}\}.$$

Another equivalent finite-dimensional characterization is:

$$z \in \Lambda_\varepsilon(A)$$

if and only if z is an eigenvalue of $A + E$ for some perturbation satisfying

$$\|E\|_2 \leq \varepsilon.$$

These equivalent views connect pseudospectra to both resolvent growth and eigenvalue sensitivity (Trefethen and Embree 2005).

6.9 Exact pseudospectrum of the 2×2 example

For

$$A = \begin{pmatrix} a & K \\ 0 & a \end{pmatrix}, \quad K > 0,$$

write

$$z - a = \delta, \quad r = |\delta|.$$

Then

$$zI - A = \begin{pmatrix} \delta & -K \\ 0 & \delta \end{pmatrix}.$$

Its squared singular values are

$$\sigma_{\pm}^2 = \frac{2r^2 + K^2 \pm K\sqrt{K^2 + 4r^2}}{2}.$$

On the ε -pseudospectral boundary,

$$\sigma_{\min} = \varepsilon.$$

Because the product of the singular values is

$$|\det(zI - A)| = r^2,$$

we have

$$\sigma_{\max} = \frac{r^2}{\varepsilon}.$$

The sum of squared singular values is

$$\sigma_{\max}^2 + \sigma_{\min}^2 = 2r^2 + K^2.$$

Substituting gives

$$\frac{r^4}{\varepsilon^2} + \varepsilon^2 = 2r^2 + K^2.$$

Hence

$$(r^2 - \varepsilon^2)^2 = K^2 \varepsilon^2.$$

The admissible boundary branch is

$$r^2 = \varepsilon(\varepsilon + K).$$

Therefore

$$\Lambda_\varepsilon(A) = \left\{ z : |z - a| \leq \sqrt{\varepsilon(\varepsilon + K)} \right\}.$$

For the normal comparison $N_0 = aI$,

$$\Lambda_\varepsilon(N_0) = \{ z : |z - a| \leq \varepsilon \}.$$

The eigenvalue plot is identical.

The perturbation neighborhood is not.

For small ε ,

$$\sqrt{\varepsilon(\varepsilon + K)} \sim \sqrt{K\varepsilon},$$

which can be much larger than ε .

6.10 Why pseudospectra matter

Eigenvalues answer:

Where are the exact modal growth factors?

Pseudospectra ask:

How sensitive is that spectral picture, and how large can the resolvent become nearby?

For normal matrices, ε -pseudospectra are simply ε -neighborhoods of the spectrum in the 2-norm.

For non-normal matrices they can expand far beyond those neighborhoods.

This enlargement signals that small perturbations can move eigenvalues much farther than the ordinary eigenvalue plot suggests, and that resolvent response can be large.

Pseudospectra are therefore a way to visualize **spectral fragility**.

They are not a causal diagnosis by themselves.

6.11 Transient growth in broader non-normal systems

The importance of non-normal transient growth is not peculiar to the Atlas toy matrix. It has a substantial literature in hydrodynamic stability, where decaying eigenmodes can combine to produce substantial transient energy growth (Reddy et al. 1993; Trefethen et al. 1993).

Those examples motivate the general lesson without making a direct identification with neural optimization.

The Atlas will use the mathematics, not import the application wholesale.

Later, when an optimizer-state Jacobian or router linearization is non-normal, the correct question is not:

Is this secretly a fluid?

It is:

Can non-orthogonal dynamical directions create finite-horizon amplification that an eigenvalue-only analysis misses?

6.12 What the Wolfram witness establishes

For the exact matrix pair

$$A = \begin{pmatrix} 4/5 & 4 \\ 0 & 4/5 \end{pmatrix}, \quad N_0 = \frac{4}{5}I,$$

the witness establishes:

1. identical eigenvalues;
2. exact powers of A ;
3. large finite-horizon 2-norm gain for A ;
4. monotone decay for N_0 ;
5. exact non-normal pseudospectral radius

$$\sqrt{\varepsilon(\varepsilon + 4)};$$

6. normal pseudospectral radius

ε .

The witness does **not** establish:

- that every non-normal system exhibits large transient growth;
- that transient growth implies asymptotic instability;
- that broad pseudospectra identify a causal mechanism in a trained model;
- that one norm is universally privileged.

6.13 Five mistakes to avoid

6.13.1 Mistake 1: “Non-normal means unstable.”

No. The worked matrix satisfies $\rho(A) < 1$ and $A^n \rightarrow 0$.

6.13.2 Mistake 2: “Stable eigenvalues imply monotone decay.”

No. The worked matrix amplifies strongly before decaying.

6.13.3 Mistake 3: “A transient spike proves divergence.”

No. Finite-horizon gain and asymptotic divergence are different.

6.13.4 Mistake 4: “Pseudospectra replace eigenvalues.”

No. They supplement the spectral picture by adding perturbation and resolvent information.

6.13.5 Mistake 5: “Broad pseudospectra explain a neural failure.”

No. They expose a possible amplification mechanism. Causal attribution requires additional evidence.

6.14 Atlas connections

Spectral shaping.

An optimizer that changes singular structure may alter transient behavior even when eigenvalue summaries appear similar.

Optimizer-State Dynamics.

The augmented optimizer Jacobian can inherit exactly the stable-but-amplifying structure developed here.

Router dynamics.

Routing systems can possess non-normal local update operators and finite-horizon amplification.

Spectral diagnostics.

Eigenvalues, singular values, pseudospectra, and finite-horizon propagators become complementary diagnostic objects.

The recurring Atlas shift is:

Where are the eigenvalues? $\longrightarrow$ What can this operator do to perturbations over the horizon that matters?

6.15 Closing view

Eigenvalues are not wrong.

They are sometimes incomplete.

Normal matrices make eigenvalue geometry unusually transparent. Non-normal matrices can hide large finite-time behavior in the geometry of their directions.

Pseudospectra reveal part of that hidden sensitivity.

The aligned-currents allegory is only an intuition pump.

The operator norm, resolvent, singular values, and pseudospectrum are the objects.

6.16 References used in this chapter

- (Horn and Johnson 2012)
- (Trefethen and Embree 2005)
- (Reddy et al. 1993)
- (Trefethen et al. 1993)

See ‘sources/source-locks/ATLAS-CH-NONNORMAL-001.yaml’ for exact source identities and claim scope.

Chapter 7

Krylov Subspaces and Iterative Solves

Epistemic status: established numerical linear algebra + audited Atlas substrate + Atlas synthesis.

Specification: manuscript/specifications/ATLAS-CH-KRYLOV-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-KRYLOV-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-KRYLOV-001.md

Source lock: sources/source-locks/ATLAS-CH-KRYLOV-001.yaml

Large linear problems are often attacked without solving the whole space at once.

The key move is to build a sequence of informative directions from repeated application of the same operator to a starting residual. The resulting trial space is not arbitrary. It is generated by the operator itself.

For a square operator A and starting residual r_0 ,

$$K_m(A, r_0) = \text{span}\{r_0, A r_0, \dots, A^{(m-1)} r_0\}.$$

For $Ax=b$ with starting point x_0 and $r_0=b-Ax_0$, an iterative subspace method seeks

$$x_m \text{ in } x_0 + K_m(A, r_0).$$

This chapter develops the classical structure behind that idea.

7.1 Why operator-generated directions matter

A direct method tries to expose enough of the full matrix structure to solve the system outright.

An iterative subspace method instead asks:

- which directions become visible after repeated operator action;
- what reduced problem appears in those directions;
- whether the reduced correction meaningfully reduces the target error or residual;
- how numerical conditioning changes that interpretation.

This distinction matters when A is large, sparse, structured, or available only through a routine that computes $A v$.

7.2 Arnoldi: general matrices

Starting from a normalized residual v_1 , Arnoldi repeatedly applies A and orthogonalizes the result against the basis already built.

In exact arithmetic it produces orthonormal columns V_m and an upper-Hessenberg reduced relation

$$A V_m = V_{m+1} Hbar_m.$$

Three objects must remain distinct:

- the generated trial space;
- the orthonormal basis used to represent it;
- the reduced projected problem solved in that basis.

The Arnoldi relation is general for square matrices. It does not require symmetry.

7.3 Lanczos: the symmetric specialization

When A is symmetric, or Hermitian in the complex case, the Arnoldi projection becomes symmetric as well as upper Hessenberg.

A symmetric Hessenberg matrix is tridiagonal.

The exact-arithmetic recurrence therefore collapses to three terms:

$$\beta_{j-1} v_{j-1} + \alpha_j v_j + \beta_j v_{j+1} = A v_j.$$

This is the Lanczos structure.

The short recurrence is a structural consequence of symmetry in exact arithmetic. It is not a promise that floating-point basis vectors remain exactly orthogonal forever.

7.4 Galerkin projection

Write

$$\mathbf{x}_m = \mathbf{x}_0 + \mathbf{V}_m \mathbf{y}.$$

Then

$$\mathbf{r}_m = \mathbf{r}_0 - \mathbf{A} \mathbf{V}_m \mathbf{y}.$$

A Galerkin method chooses $\mathbf{y}$ so that the new residual is orthogonal to the trial space:

$$\mathbf{V}_m^T \mathbf{r}_m = 0.$$

This gives the reduced system

$$(\mathbf{V}_m^T \mathbf{A} \mathbf{V}_m) \mathbf{y} = \mathbf{V}_m^T \mathbf{r}_0.$$

The method therefore solves a smaller problem whose dimension is m rather than the ambient dimension.

7.5 Minimum residual is a different projection rule

A minimum-residual method instead chooses $\mathbf{y}$ to minimize

$$\|\mathbf{r}_0 - \mathbf{A} \mathbf{V}_m \mathbf{y}\|_2.$$

With an Arnoldi basis, this becomes a small least-squares problem involving $\mathbf{H}_{\text{bar}_m}$.

The distinction is important:

- Galerkin imposes an orthogonality condition;
- minimum residual explicitly minimizes residual norm;
- neither statement is automatically the same as minimizing solution error in Euclidean norm.

7.6 Residual and error are related, not identical

If $\mathbf{x}^*$ is the exact solution and

$$\mathbf{e}_m = \mathbf{x}^* - \mathbf{x}_m,$$

then

$$\mathbf{A} \mathbf{e}_m = \mathbf{r}_m.$$

For nonsingular $\mathbf{A}$,

$$\mathbf{e}_m = \mathbf{A}^{-1} \mathbf{r}_m.$$

Therefore

$$\|em\|_2 \leq \|A^{-1}\|_2 \|rm\|_2.$$

A small residual can support a useful error bound when the inverse operator is well behaved.

A badly conditioned operator can magnify residual-to-error conversion.

This is why convergence statements must say which norm and which quantity are being controlled.

7.7 Polynomial viewpoint

Every correction in the m -step generated subspace can be written as

$$q_{m-1}(A) r_0$$

for a polynomial of degree at most $m-1$.

The residual has the form

$$rm = p_m(A) r_0$$

with $p_m(0) = 1$.

This viewpoint connects iterative linear algebra to polynomial approximation.

It also explains why spectral distribution, clustering, non-normality, and the initial residual can strongly affect convergence.

7.8 Exact two-dimensional witness

Take

$$A = \text{diag}(1, 2, 4),$$

$$b = (1, 1, 1)^T,$$

$$x_0 = 0.$$

Then

$$r_0 = b,$$

$$Ab = (1, 2, 4)^T.$$

Use the two-column basis

$$V = [b, Ab] = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 4 \end{bmatrix}.$$

The reduced Galerkin system is

$$(V^T A V)c = V^T b.$$

7.9. THE SAME WITNESS IN AN ORTHONORMAL SYMMETRIC BASIS 77

Direct arithmetic gives

$$V^T A V = \begin{bmatrix} 7 & 21 \\ 21 & 73 \end{bmatrix},$$

$$V^T b = \begin{bmatrix} 3 \\ 7 \end{bmatrix},$$

and

$$c = \begin{bmatrix} 36/35 & -1/5 \end{bmatrix}^T.$$

Hence

$$x_2 = \begin{bmatrix} 29/35 & 22/35 & 8/35 \end{bmatrix}^T.$$

The residual is

$$r_2 = \begin{bmatrix} 6 & -9 & 3 \end{bmatrix}^T / 35.$$

Now check the defining orthogonality:

$$b^T r_2 = 0,$$

$$(Ab)^T r_2 = 0.$$

Thus

$$V^T r_2 = 0.$$

The squared residual norm is

$$\|r_2\|_2^2 = 18/175.$$

The exact solution is

$$x^* = \begin{bmatrix} 1 & 1/2 & 1/4 \end{bmatrix}^T.$$

The resulting error satisfies

$$A e_2 = r_2,$$

as it must.

7.9 The same witness in an orthonormal symmetric basis

Normalize

$$v_1 = (1, 1, 1)^T / \sqrt{3}.$$

Then

$$\alpha_1 = 7/3,$$

$$\beta_1 = \sqrt{14}/3,$$

and

$v_2 = (-4, -1, 5)^T / \sqrt{42}$.

The next projected diagonal coefficient is

$\alpha_2 = 59/21$.

So the two-dimensional projected matrix is

$T_2 = \begin{bmatrix} 7/3, \sqrt{14}/3 \\ \sqrt{14}/3, 59/21 \end{bmatrix}$.

This is the symmetric tridiagonal reduction in explicit form.

7.10 A conditioning control

Now take

$A_c = \text{diag}(1, 100, 10000)$,

with the same b and x_0 .

The one-dimensional SPD Galerkin/conjugate-gradient step has

$\alpha = 1/3367$.

Its Euclidean residual is

$r_1 = [3366, 3267, -6633]^T / 3367$,

with

$\|r_1\|_2^2 = 19602/3367$.

That is larger than the initial residual norm squared:

$\|r_0\|_2^2 = 3$.

So a valid conjugate-gradient step can increase Euclidean residual norm.

There is no contradiction.

For SPD systems, the relevant optimality statement for conjugate gradient concerns the A -norm of the error over the affine trial space, not monotone decrease of Euclidean residual norm.

The example therefore separates:

- residual norm;
- error norm;
- energy norm;
- method-specific optimality.

7.11 Matrix-free access

Nothing in the definition requires an explicitly materialized dense matrix.

If the implementation can compute

$$v \rightarrow A v,$$

it can generate operator-derived directions.

This matters for large sparse systems and for implicit operators.

The real cost then includes:

- operator-vector products;
- orthogonalization;
- projected solves;
- storage;
- preconditioner application;
- restarts.

7.12 Preconditioning changes the problem seen by the iteration

A left-preconditioned system uses

$$M^{-1} A x = M^{-1} b.$$

A right-preconditioned system uses

$$A M^{-1} y = b,$$

then recovers

$$x = M^{-1} y.$$

The generated space is therefore built from a different effective operator.

Preconditioning is not equivalent to merely taking more iterations.

A good preconditioner changes the geometry or spectrum of the problem in a way that makes the reduced iteration more effective.

7.13 Finite precision

Exact Arnoldi basis vectors are orthogonal.

Finite-precision implementations can lose orthogonality.

For symmetric recurrences, this can produce repeated or distorted spectral information if not controlled.

Possible responses include reorthogonalization, selective reorthogonalization, restarting, or other stabilization strategies.

Each changes cost and information retention.

7.14 Restarting is a tradeoff

Unrestarted Arnoldi stores more basis vectors and pays increasing orthogonalization cost.

Restarting limits memory and work per cycle.

But restarting can also discard useful directions accumulated earlier.

The right tradeoff depends on the operator, spectrum, target accuracy, and memory budget.

7.15 What this chapter does not claim

This chapter does not claim:

- that low-dimensional generated spaces always converge rapidly;
- that small residual always means small solution error;
- that eigenvalues alone determine convergence for non-normal operators;
- that exact-arithmetic orthogonality survives floating point automatically;
- that more iterations are equivalent to better preconditioning;
- that a learned nonlinear architecture inherits classical convergence results merely because it uses iterative transport or operator-generated directions.

7.16 Downstream handoff

Approximate and Structured Attention may inherit the principle that repeated operator action and projection can expose useful low-dimensional structure without full dense computation.

Neural iterative transport may inherit the classical trial-space, residual, projection, and preconditioning vocabulary.

It must independently justify any learned, nonlinear, adaptive, or representation-level extension.

7.17 References used in this chapter

- (Arnoldi 1951)
- (Lanczos 1950)
- (Saad 2003)
- (Trefethen and David Bau 1997)

- (Golub and Loan 2013)

See `sources/source-locks/ATLAS-CH-KRYLOV-001.yaml` for exact source identities and claim scope.

Chapter 8

Probability, Information, and Statistical Structure

Epistemic status: Established Theory + Atlas Derivation

Specification: `manuscript/specifications/ATLAS-CH-INFO-001.md`

Derivation packet: `mathematics/derivations/ATLAS-CH-INFO-001-DERIVATIONS.md`

Computational witness: `mathematics/computational-witnesses/ATLAS-CW-INFO-001.md`

Source lock: `sources/source-locks/ATLAS-CH-INFO-001.yaml`

8.1 Information requires a probability model

Machine learning uses the word “information” constantly.

The word can mean:

- a random variable reduces uncertainty about another;
- a code uses fewer bits;
- a hidden state preserves a task-relevant distinction;
- a message is semantically meaningful;
- a variable causally influences another;
- a dataset contains predictive signal.

These meanings are not interchangeable.

This chapter begins with the probabilistic meaning.

A probability model states how uncertainty is represented.

Information-theoretic quantities are then functions of that model.

The chapter’s first discipline is:

never ask what entropy or mutual information means before asking what probability model it is computed under.

8.2 Uncertainty budget, not semantic content

A useful allegory is an uncertainty budget.

Entropy says how uncertain a random variable is under a declared distribution.

Conditional entropy says how much uncertainty remains after another variable is known.

Mutual information says how much uncertainty is reduced, symmetrically, by knowing the other variable.

The allegory fails if “information” is taken to mean semantic significance.

A random identifier can have high entropy and almost no meaning to a user.

A one-bit alarm can carry enormous operational importance.

Information theory measures structure in a probability model.

It does not directly measure meaning.

8.3 Random variables and distributions

A random variable

$$X : \Omega \rightarrow \mathcal{X}$$

maps outcomes in a sample space to values.

In the discrete case, a mass function satisfies

$$p(x) \geq 0,$$

and

$$\sum_x p(x) = 1.$$

For continuous variables, probability is described relative to a measure through a density where one exists.

This distinction matters.

Expressions such as entropy and KL divergence behave differently in discrete and continuous settings.

The Atlas will not silently move between them.

8.4 Conditional probability

For events A and B with $P(B) > 0$,

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

For random variables, conditional distributions express how the law of one variable changes when another is known.

This is the foundation for:

- Bayesian inference;
- predictive modeling;
- filtering;
- probabilistic memory;
- uncertainty-aware decisions.

Conditional probability is not a causal operator by itself.

It is a relation inside a probability model.

8.5 Expectation

For discrete X ,

$$\mathbb{E}[X] = \sum_x x p(x).$$

Expectation is linear:

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y].$$

That algebraic simplicity is one reason expectations organize so much statistical learning.

Loss functions, gradients, risk, calibration, and uncertainty estimates are often expectations under one distribution or another.

The distribution must be named.

8.6 Entropy

For a finite random variable,

$$H(X) = - \sum_x p(x) \log_2 p(x).$$

Entropy is measured in bits when the logarithm has base 2.

A fair bit has

$$H(X) = 1.$$

A deterministic bit has

$$H(X) = 0.$$

These statements are exact.

They do not say that a fair bit is “more meaningful.”

They say its value is more uncertain before observation.

Standard information-theory definitions and identities are developed in Cover and Thomas (Cover and Thomas 2006).

8.7 Joint and conditional entropy

For jointly distributed X, Y ,

$$H(X, Y) = - \sum_{x, y} p(x, y) \log_2 p(x, y).$$

Conditional entropy satisfies

$$H(X | Y) = H(X, Y) - H(Y).$$

The entropy chain rule is

$$H(X, Y) = H(X) + H(Y | X).$$

This is one of the first examples of information decomposing according to conditional structure.

8.8 Cross-entropy

If data are distributed as P but a model assigns distribution Q , the cross-entropy is

$$H(P, Q) = -\mathbb{E}_{x \sim P} \log q(x).$$

In the discrete case,

$$H(P, Q) = -\sum_x p(x) \log q(x).$$

Cross-entropy is therefore a model-relative quantity.

It depends on both the data distribution and the proposed predictive distribution.

8.9 KL divergence

The Kullback–Leibler divergence is

$$D_{\text{KL}}(P \| Q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

when the support conditions make the expression well defined.

KL divergence is nonnegative.

It is not symmetric:

$$D_{\text{KL}}(P \| Q) \neq D_{\text{KL}}(Q \| P)$$

in general.

Therefore KL is not a metric.

It also can be infinite.

If P places positive mass where Q assigns zero mass, then

$$D_{\text{KL}}(P \| Q) = \infty.$$

The direction matters.

8.10 Cross-entropy decomposition

For discrete distributions,

$$H(P, Q) = H(P) + D_{\text{KL}}(P \| Q).$$

If P is fixed, minimizing cross-entropy with respect to Q is equivalent to minimizing

$$D_{\text{KL}}(P \| Q).$$

This identity is simple and widely used.

Its assumptions should remain visible.

The expression depends on the declared target distribution P and model distribution Q .

8.11 Mutual information

Mutual information is

$$I(X; Y) = D_{\text{KL}}(P_{XY} \| P_X P_Y).$$

Equivalently,

$$I(X; Y) = H(X) - H(X | Y),$$

and symmetrically,

$$I(X; Y) = H(Y) - H(Y | X).$$

If X and Y are independent,

$$P_{XY} = P_X P_Y,$$

so

$$I(X; Y) = 0.$$

Mutual information therefore measures statistical dependence.

It does not specify a causal direction.

8.12 Exact correlated-bit witness

Consider

$$P_{XY} = \begin{pmatrix} 3/8 & 1/8 \\ 1/8 & 3/8 \end{pmatrix}.$$

Both marginals are fair:

$$P_X = P_Y = (1/2, 1/2).$$

The companion Wolfram witness evaluates

$$I(X; Y) = \frac{\log(27/16)}{\log 16} \approx 0.1887218755408671$$

bits.

For deterministic equality $Y = X$ with a fair bit,

$$I(X; Y) = 1$$

bit.

The statistic is symmetric in X and Y .

The result therefore cannot by itself tell us whether X causes Y , Y causes X , or both share another cause.

8.13 Same entropy, different structure

Two distributions can have the same entropy while assigning probability to different events.

Entropy therefore does not uniquely identify a distribution.

Likewise, two learned representations can have similar aggregate information measures while differing dramatically in:

- geometry;
- localization;
- robustness;
- causal role;
- downstream accessibility.

Information-theoretic summaries are powerful.

They are not complete descriptions.

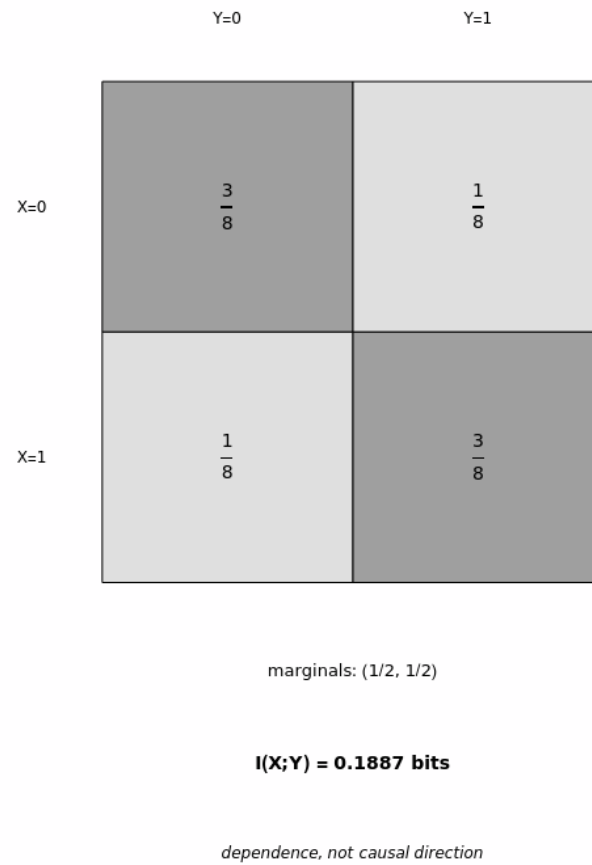


Figure 8.1: A 2 by 2 exact joint probability table with uniform marginals and mutual information about 0.1887 bits, explicitly labeled as dependence rather than causal direction.

8.14 Concentration

Expected behavior is not enough.

We often need to know how far a random quantity can deviate from its mean.

Concentration inequalities answer questions of the form:

$$P(|X - \mathbb{E}X| \geq t) \leq \text{tail bound}.$$

The assumptions matter.

For example, standard Hoeffding inequalities rely on bounded independent variables.

High-dimensional probability develops many concentration tools with different assumptions and regimes (Vershynin 2018).

The Atlas will carry those assumptions forward rather than quoting a tail formula detached from its conditions.

8.15 Why concentration matters for machine learning

Concentration appears in:

- generalization arguments;
- randomized embeddings;
- sketching;
- stochastic gradients;
- random-feature approximations;
- sampling-based estimators;
- high-dimensional geometry.

A concentration bound is an uncertainty statement about a random object.

It is not a guarantee that a learned system behaves well under distribution shift.

That requires a different argument.

8.16 Sufficient statistics

A statistic

$$T(X)$$

compresses observed data.

It is sufficient for a parameter θ in a statistical family when, informally, it retains all information in the sample relevant to θ .

Sufficiency is therefore relative to a model.

A statistic can be sufficient for one family and inadequate for another.

This is a useful early example of a theme that later reappears in boundary compression:

sufficient for what?

8.17 Exponential families

An exponential family has form

$$p_{\eta}(x) = h(x) \exp\{\eta^{\top} T(x) - A(\eta)\}.$$

Here:

- η is a natural parameter;
- $T(x)$ is a sufficient-statistic vector in the standard regular setting;
- $A(\eta)$ is the log-partition function.

Exponential-family structure connects:

- statistics;
- convexity;
- entropy;
- duality.

Wainwright and Jordan develop this framework systematically (Wainwright and Jordan 2008).

8.18 Bernoulli as an exponential family

For $x \in \{0, 1\}$,

$$p(x) = p^x (1 - p)^{1-x}.$$

Define

$$\eta = \log \frac{p}{1-p}.$$

Then

$$p(x) = \exp\{x\eta - A(\eta)\},$$

where

$$A(\eta) = \log(1 + e^\eta).$$

Thus

$$T(x) = x$$

is the natural sufficient statistic in this representation.

This small example is useful because it connects probability, sufficient statistics, and convex log-partition structure in one line.

8.19 Information geometry waits for later

Exponential families also support a geometric viewpoint.

The Hessian of the log-partition function is related to covariance and Fisher information under standard regularity conditions.

That leads toward information geometry.

This chapter does not develop that theory.

It establishes the probabilistic and statistical objects the later chapter will require.

8.20 Three overinterpretations to avoid

8.20.1 Entropy equals meaning

False.

Entropy quantifies uncertainty under a probability model.

8.20.2 Mutual information equals causality

False.

Mutual information is symmetric dependence information.

8.20.3 KL equals distance

False.

KL is asymmetric and can be infinite.

These are not pedantic distinctions.

They determine which downstream arguments are legal.

8.21 Atlas connections

Representation.

Information measures can characterize statistical dependence or compression, but representation geometry and identifiability remain separate questions.

Attention.

Softmax creates normalized weights, but probabilistic-looking normalization does not automatically make every attention object a probability model with the semantics assumed here.

Decision theory.

Expected utility and uncertainty require explicit distributions or uncertainty sets.

Memory.

Retrieval and compression can be studied in terms of sufficient statistics and information preservation, but semantic adequacy is task-relative.

Optimization.

KL and related divergences can induce optimization geometry.

8.22 Closing view

Probability gives the Atlas a language for uncertainty.

Information theory gives it a language for statistical dependence and coding structure.

Exponential families give it a language for structured statistical models.

Concentration gives it a language for deviations.

These tools are powerful precisely because their meanings are narrow.

The Atlas will use them most effectively by refusing to ask them to mean more than they do.

8.23 References used in this chapter

- (Cover and Thomas 2006)
- (Wainwright and Jordan 2008)
- (Vershynin 2018)

Chapter 9

Geometry of Constrained State Spaces

Epistemic status: mathematical exposition based on standard Riemannian and matrix-manifold geometry, with Atlas-owned elementary derivations.

Primary figure: ‘ATLAS-FIG-MANIFOLD-001’

Derivation packet: ‘mathematics/derivations/ATLAS-CH-GEOM-001-DERIVATIONS.md’

9.1 The space you write in is not always the space you may move in

Machine learning often presents objects in ambient Euclidean coordinates. A unit vector lives in $\mathbb{R}^d$. An orthonormal frame is stored as an array in $\mathbb{R}^{n \times p}$. A subspace may be represented by many different matrices.

Coordinates are convenient. They can also hide constraints.

If a state must satisfy

$$\|x\| = 1,$$

then an arbitrary Euclidean step $x \mapsto x - \eta g$ will generally leave the admissible set. If a matrix must satisfy

$$X^\top X = I,$$

then an arbitrary matrix update will generally destroy orthogonality.

The geometric question is therefore:

What are the legal local directions, and how do we turn them into legal finite moves?

The useful allegory is a **cartographer walking on a mountain**.

The map coordinates are the ambient coordinates. The mountain surface is the admissible state space. A tangent plane gives a locally linear set of legal directions. A geodesic stays intrinsically on the surface. A retraction takes a locally legal direction and returns a nearby ambient step to the constraint.

The limit of the allegory is important. A manifold need not be a visible surface in three-dimensional space, and a retraction need not be an orthogonal projection. The image teaches constrained motion, not all of differential geometry.

9.2 Ambient space and admissible space

A smooth embedded manifold $M \subseteq \mathbb{R}^N$ is locally well approximated by a linear space even though M need not itself be linear (Lee 2018).

At a point $x \in M$, the tangent space

$$T_x M$$

contains the velocities of differentiable curves through x .

If

$$\gamma : (-\varepsilon, \varepsilon) \rightarrow M, \quad \gamma(0) = x,$$

then

$$\dot{\gamma}(0) \in T_x M.$$

This gives a recurring Atlas pattern:

constrained object $\longrightarrow$ tangent linearization $\longrightarrow$ local update $\longrightarrow$ return to the constraint.

The tangent space is linear. The manifold is not.

That distinction will later let us separate “what direction should the optimizer choose?” from “how should the chosen direction be represented as a legal state update?”

9.3 The unit sphere

Consider

$$S^{d-1} = \{x \in \mathbb{R}^d : x^\top x = 1\}.$$

Let $\gamma(t) \in S^{d-1}$ with

$$\gamma(0) = x, \quad \dot{\gamma}(0) = v.$$

Because the curve remains on the sphere,

$$\gamma(t)^\top \gamma(t) = 1.$$

Differentiate at $t = 0$:

$$2x^\top v = 0.$$

Hence

$$\boxed{T_x S^{d-1} = \{v \in \mathbb{R}^d : x^\top v = 0\}}.$$

The tangent space is the hyperplane orthogonal to x .

If $g \in \mathbb{R}^d$ is an arbitrary ambient direction, its tangent component is

$$g_{\text{tan}} = g - (x^\top g)x.$$

The geometry has already changed the update rule. The radial component is not a legal first-order direction.

9.4 Retractions: step locally, return legally

A retraction $R_x : T_x M \rightarrow M$ is a smooth local map satisfying, at minimum,

$$R_x(0) = x$$

and

$$DR_x(0) = \text{id}_{T_x M}.$$

For the sphere, a simple retraction is normalization:

$$R_x(v) = \frac{x + v}{\|x + v\|}, \quad v \in T_x S^{d-1}.$$

Why is this a valid first-order retraction?

Set

$$r(t) = \frac{x + tv}{\|x + tv\|}.$$

Because $x^\top v = 0$ and $\|x\| = 1$,

$$r'(0) = v.$$

So the map agrees with the tangent direction to first order while returning every sufficiently small step to the sphere.

This is a common numerical idea: do the difficult reasoning in a linear tangent space, then use a controlled map back to the nonlinear constraint.

Matrix-manifold optimization develops this pattern systematically (Absil et al. 2008).

9.5 Retraction is not exponential map

The Riemannian exponential map moves along a geodesic.

On the unit sphere, for nonzero $v \in T_x S^{d-1}$,

$$\text{Exp}_x(v) = \cos(\|v\|)x + \sin(\|v\|)\frac{v}{\|v\|}.$$

The normalized retraction is

$$R_x(v) = \frac{x + v}{\sqrt{1 + \|v\|^2}},$$

because $x^\top v = 0$.

Both are legal maps back to the sphere. They are not the same map.

For small v ,

$$\text{Exp}_x(v) = x + v - \frac{\|v\|^2}{2}x + O(\|v\|^3),$$

and

$$R_x(v) = x + v - \frac{\|v\|^2}{2}x + O(\|v\|^3).$$

They agree through the first-order retraction requirement and share the same quadratic radial correction, but differ at higher order.

This distinction matters later. “Geometry-aware” should not become a vague synonym for “normalize after every step.”

9.6 The primary geometric plate

The Atlas figure fixes

$$x = (0, 0, 1), \quad v = (0.8, 0, 0).$$

Since

$$x^\top v = 0,$$

v is tangent to the sphere at x .

The figure shows:

- the unit sphere;
- the tangent plane at x ;
- the tangent vector v ;
- the spherical exponential-map endpoint;
- the normalized-retraction endpoint;
- the great-circle direction.

The plate is partly schematic. Perspective, tangent-plane extent, and label placement are pedagogical. The plotted points themselves are generated from the stated equations.

Its purpose is to make one fact visible:

a tangent vector belongs to a linear space, but a finite update must still be interpreted through the manifold.

9.7 Geodesics and spherical distance

On a Riemannian manifold, a geodesic generalizes the idea of a locally straight path.

For unit vectors $x, y \in S^{d-1}$, define their angle

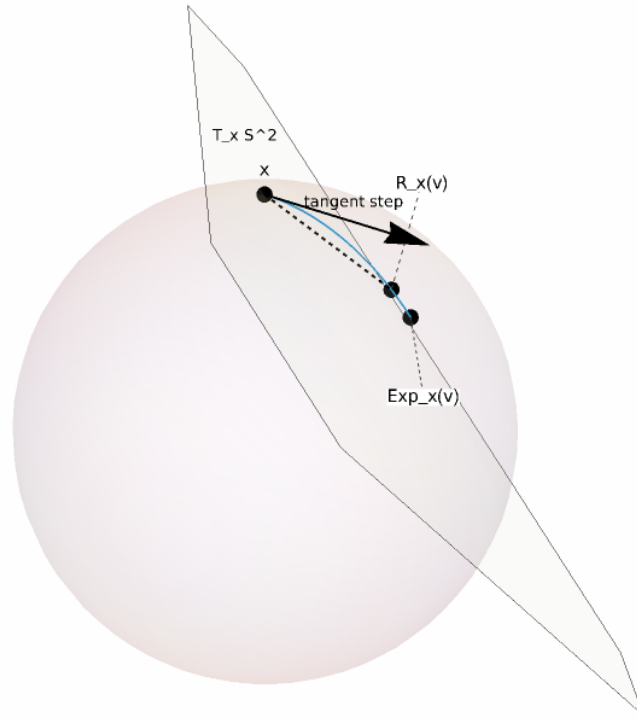


Figure 9.1: Unit sphere with the tangent plane at x , a tangent vector v , and separate exponential-map and normalized-retraction endpoints.

$$\theta = \arccos(x^\top y), \quad 0 \leq \theta \leq \pi.$$

For non-antipodal endpoints, the shorter great-circle distance is

$$d_S(x, y) = \theta.$$

The ambient chord distance is

$$\|x - y\| = 2 \sin \frac{\theta}{2}.$$

For small θ ,

$$2 \sin \frac{\theta}{2} = \theta + O(\theta^3),$$

so local Euclidean distance approximates geodesic distance.

Globally they are different.

This is a simple example of a recurring principle: local linearization can be excellent without making the global geometry flat.

9.8 SLERP: interpolate on the sphere

Let

$$x^\top y = \cos \theta, \quad 0 < \theta < \pi.$$

Spherical linear interpolation is

$$\boxed{\text{SLERP}(x, y; t) = \frac{\sin((1-t)\theta)}{\sin \theta} x + \frac{\sin(t\theta)}{\sin \theta} y, \quad 0 \leq t \leq 1.}$$

Shoemake introduced SLERP in quaternion interpolation for computer animation (Shoemake 1985). The underlying formula is simply great-circle interpolation on a sphere.

Writing

$$\alpha = \frac{\sin((1-t)\theta)}{\sin \theta}, \quad \beta = \frac{\sin(t\theta)}{\sin \theta},$$

we obtain

$$\|\alpha x + \beta y\|^2 = \alpha^2 + \beta^2 + 2\alpha\beta \cos \theta = 1.$$

So the path remains on the unit sphere exactly.

There are two important boundaries:

- as $\theta \rightarrow 0$, the apparent singularity is removable;
- at $\theta = \pi$, the shortest geodesic is not unique, so an additional directional choice is required.

The antipodal singularity is geometric information, not merely a numerical nuisance.

9.9 Stiefel geometry: orthonormal frames

For $X \in \mathbb{R}^{n \times p}$, the Stiefel manifold is

$$\text{St}(n, p) = \{X : X^\top X = I_p\}.$$

Its points are ordered orthonormal p -frames.

Let $X(t) \in \text{St}(n, p)$ and set

$$X(0) = X, \quad \dot{X}(0) = Z.$$

Differentiate

$$X(t)^\top X(t) = I_p.$$

At $t = 0$,

$$Z^\top X + X^\top Z = 0.$$

Therefore

$$\boxed{T_X \text{St}(n, p) = \{Z : X^\top Z + Z^\top X = 0\}.$$

This is the matrix analogue of the sphere condition $x^\top v = 0$.

The Stiefel manifold is especially important in optimization with orthogonality constraints; Edelman, Arias, and Smith provide a canonical geometric treatment (Edelman et al. 1998).

9.10 Grassmann geometry: the frame is not the subspace

A Stiefel matrix X stores an ordered orthonormal basis.

But if the object of interest is only the p -dimensional subspace spanned by its columns, then

$$X$$

and

$$XQ, \quad Q \in O(p),$$

represent the same subspace.

The Grassmann manifold identifies all such frames:

$$\boxed{\text{Gr}(n, p) \cong \text{St}(n, p)/O(p).}$$

This is the first important quotient-geometry example in the Atlas.

A parameterization can contain distinctions that the represented object does not.

Later, when we discuss semantic equivalence and benign nonconvexity, this pattern returns in a broader form:

if two parameter states represent the same functional object, should
the optimization landscape distinguish them?

The Grassmannian gives a clean finite-dimensional case where the answer is no.

9.11 Geometry changes what an optimizer is allowed to do

Suppose $f : M \rightarrow \mathbb{R}$.

In ambient coordinates, automatic differentiation may give an ordinary gradient ∇f . A manifold-aware method must convert that ambient information into a tangent direction and then map a finite step back to M .

Schematically,

$$\nabla f \longrightarrow \text{Proj}_{T_x M}(\nabla f) \longrightarrow v \longrightarrow R_x(v).$$

The exact Riemannian gradient depends on the chosen metric, and later chapters will treat that carefully.

The present chapter establishes the more basic point:

constraints are part of the state geometry, so they should appear in the update mathematics rather than only as after-the-fact repairs.

9.12 Four mistakes to avoid

9.12.1 Mistake 1: “The tangent space is the manifold.”

No. It is a local linearization at one point.

9.12.2 Mistake 2: “Normalization is the exponential map.”

No. On the sphere it is a useful retraction, not the same finite map as geodesic motion.

9.12.3 Mistake 3: “A Stiefel point and a Grassmann point are the same object.”

No. The first is an ordered frame; the second is a subspace modulo basis change.

9.12.4 Mistake 4: “Local Euclidean behavior means the space is globally Euclidean.”

No. Local tangent approximations coexist with global curvature and topological structure.

9.13 Atlas connections

Normalized representations.

Unit-norm states move naturally on spheres rather than unconstrained Euclidean space.

Optimization on manifolds.

Tangent gradients, retractions, and matrix manifolds turn the present geometry into algorithms.

Position geometry.

Rotations and spherical constructions recur in positional encodings.

Representation as transport.

A representation update can be treated as motion through a constrained state space.

Quotient geometry.

Grassmann geometry supplies the simplest durable example of removing representational redundancy.

The recurring Atlas shift is:

parameters as coordinates $\longrightarrow$ parameters as points in a structured state space.

9.14 Closing view

The mountain allegory is useful only until the mathematics becomes visible.

A constrained representation has an admissible state space. At each point, tangent geometry tells us which first-order motions are legal. Geodesics describe intrinsic motion. Retractions provide practical finite updates. Quotient geometry tells us when different coordinates describe the same underlying object.

The map is not the territory.

The ambient coordinates are not the admissible state space.

9.15 References used in this chapter

- (Lee 2018)
- (Absil et al. 2008)
- (Edelman et al. 1998)
- (Shoemake 1985)

See ‘sources/source-locks/ATLAS-CH-GEOM-001.yaml’ for exact source identities and claim scope.

Chapter 10

Flows, Stability, and Bifurcation

Epistemic status: Established Theory + Atlas Synthesis + Exact Computational Witness

Specification: manuscript/specifications/ATLAS-CH-DYN-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-DYN-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-DYN-001.md

Source lock: sources/source-locks/ATLAS-CH-DYN-001.yaml

10.1 A state is not yet a dynamics

A vector tells us where a system is.

A vector field tells us how the state wants to move.

A trajectory tells us one realized history.

A flow tells us exact transport through time.

These are different mathematical objects.

The distinction is simple enough to overlook and important enough to organize much of the Atlas.

Later chapters will discuss:

- residual depth;
- optimizer state;
- adaptive computation;
- recurrent systems;

- reinforcement learning;
- numerical solvers.

All of them can be described badly if “state,” “update,” and “dynamics” are treated as synonyms.

10.2 Law of motion versus film frames

A useful allegory is a film.

The vector field is the local law telling each point how to move.

A trajectory is one actor’s path through the scene.

A collection of sampled states is a sequence of film frames.

The flow map is the exact rule carrying every admissible initial state forward by a chosen amount of time.

A numerical integrator is different again.

It manufactures approximate frames according to its own discrete update rule.

The allegory fails if taken too literally.

Many neural layers do not arise from one autonomous continuous-time law.

The point is to distinguish the objects, not to force every architecture into an ODE.

10.3 Autonomous ordinary differential equations

Consider

$$\dot{x} = f(x),$$

with initial condition

$$x(0) = x_0.$$

The function

$$f : \mathcal{X} \rightarrow T\mathcal{X}$$

is the vector field, written here in Euclidean notation.

At a state x , the vector

$$f(x)$$

specifies the instantaneous direction of motion.

The differential equation asks for a curve

$$t \mapsto x(t)$$

whose derivative agrees with that field.

10.4 Trajectory

A trajectory is one solution beginning at one initial state:

$$x(t; x_0).$$

Changing x_0 can change the entire future path.

A trajectory is therefore not the vector field.

It is one realization generated by the field.

This matters in machine learning because a training run is a trajectory.

The optimizer update law is part of the dynamics that generates it.

One run does not identify the whole dynamical system.

10.5 Flow map

Where existence and uniqueness hold, define

$$\Phi_t(x_0) = x(t; x_0).$$

The map

$$\Phi_t$$

acts on initial states.

For an autonomous true flow,

$$\boxed{\Phi_{t+s} = \Phi_t \circ \Phi_s.}$$

The identity holds only for states and times on which the relevant compositions are defined. A forward-only system has a semiflow, with $s, t \geq 0$, rather than a globally defined two-sided flow.

This composition law is stronger than merely having a depth-indexed family of transformations.

The word **flow** should therefore not be used automatically for any sequence of neural layers.

10.6 Exact flow of a linear system

For a constant linear system

$$\dot{x} = Ax,$$

the exact flow is

$$\Phi_t(x_0) = e^{tA}x_0.$$

The matrix exponential therefore converts a generator A into finite-time transport.

This is one reason linear algebra and dynamics are tightly coupled.

Eigenvalues describe local modal growth for the linear system.

But finite-time behavior can still depend on non-normal structure, as the Non-normality chapter emphasized.

10.7 Equilibrium

An equilibrium x_* satisfies

$$f(x_*) = 0.$$

If the system starts there, it remains there.

Equilibria are fixed points of the flow:

$$\Phi_t(x_*) = x_*.$$

The important next question is not only whether an equilibrium exists.

It is what nearby trajectories do.

10.8 Stability is a neighborhood question

An equilibrium is Lyapunov stable if sufficiently small initial perturbations remain small for all future time.

Informally:

start close enough, remain close.

Asymptotic stability adds convergence:

start close enough, remain close, and eventually return.

Exponential stability strengthens the convergence rate:

$$\|x(t) - x_\star\| \leq Ce^{-\alpha t} \|x(0) - x_\star\|$$

for suitable positive constants C, α in a declared neighborhood.

These are not interchangeable labels.

10.9 Linearization

Suppose f is differentiable and

$$f(x_\star) = 0.$$

Write

$$x = x_\star + \delta.$$

Then

$$f(x_\star + \delta) = J_f(x_\star)\delta + o(\|\delta\|).$$

So the local perturbation dynamics are

$$\dot{\delta} = J_f(x_\star)\delta + o(\|\delta\|).$$

The little- o remainder is the general differentiability guarantee. A quadratic $O(\|\delta\|^2)$ bound needs additional regularity, such as a locally Lipschitz Jacobian.

The Jacobian provides the first local approximation to the nonlinear system.

10.10 Hyperbolic local stability

For a (C^1) vector field near a hyperbolic equilibrium, the Jacobian spectrum gives strong local information.

If every eigenvalue of

$$J_f(x_*)$$

has strictly negative real part, the equilibrium is locally exponentially stable.

If at least one eigenvalue has strictly positive real part, the equilibrium is unstable.

The difficult boundary is the imaginary axis.

If an eigenvalue has zero real part, linearization alone may be inconclusive.

Nonlinear terms can decide the outcome.

10.11 Zero eigenvalue is not “neutral stability”

This mistake is common.

Suppose

$$\dot{x} = -x^3.$$

At the origin,

$$f'(0) = 0.$$

The linearized system says nothing.

Yet the nonlinear system attracts nearby states to zero.

Now consider

$$\dot{x} = x^3.$$

Again,

$$f'(0) = 0.$$

But now the origin is unstable.

The same zero eigenvalue supports opposite nonlinear behavior.

This is why nonhyperbolic points require more than first-order spectral reasoning.

10.12 Lyapunov functions

A Lyapunov function is a scalar witness for stability.

For an equilibrium at the origin, choose

$$V(x)$$

such that

$$V(0) = 0$$

and

$$V(x) > 0$$

for nearby nonzero x .

Its derivative along trajectories is

$$\dot{V}(x) = \nabla V(x)^\top f(x).$$

If V decreases along trajectories, the system cannot wander arbitrarily outward while remaining in the certified neighborhood.

Khalil gives the standard nonlinear stability framework used here (Khalil 2002).

10.13 Negative semidefinite versus negative definite

The sign of

$$\dot{V}$$

matters.

If

$$\dot{V} \leq 0,$$

a standard Lyapunov theorem can establish stability under appropriate local hypotheses.

If

$$\dot{V} < 0$$

for every nearby nonzero state, the same framework can establish asymptotic stability.

Negative semidefinite derivative alone does not automatically prove convergence to the equilibrium.

Additional arguments, such as invariant-set reasoning, may be required.

The Atlas will keep that distinction visible.

10.14 Exact scalar stable flow

Consider

$$\dot{x} = -2x, \quad x(0) = 3.$$

The exact solution is

$$x(t) = 3e^{-2t}.$$

Choose

$$V(x) = \frac{1}{2}x^2.$$

Then

$$\dot{V} = x\dot{x} = -2x^2.$$

Therefore

$$\dot{V} < 0$$

for every nonzero state.

The origin is exponentially stable.

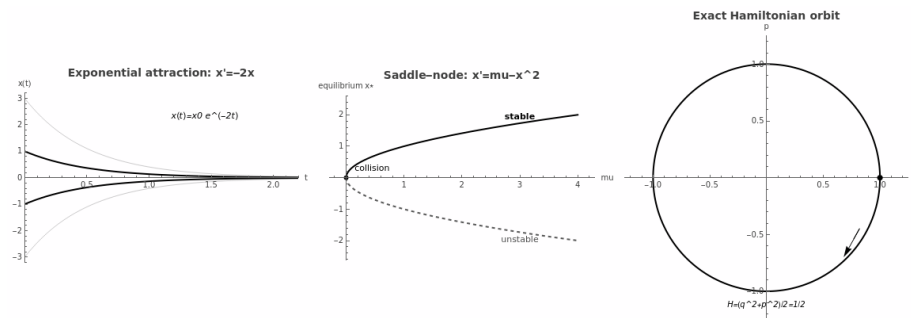


Figure 10.1: Three exact dynamics panels: exponential attraction under $x' = -2x$, stable and unstable saddle-node branches $x^* = \pm\sqrt{\mu}$, and a circular harmonic-oscillator energy orbit.

10.15 Phase portrait

A phase portrait shows trajectories or direction fields in state space.

It is a compressed picture of qualitative behavior.

It can reveal:

- attraction;
- repulsion;
- separatrices;
- periodic orbits;
- invariant regions.

But a phase portrait is still a representation.

Its appearance depends on:

- coordinates;
- projection;
- dimensional reduction;
- parameter choice.

A two-dimensional picture of a high-dimensional system can hide essential structure.

10.16 Parameters can change qualitative behavior

Consider a family

$$\dot{x} = f(x; \mu).$$

Changing the parameter μ may deform trajectories continuously.

Sometimes the qualitative structure changes.

Equilibria can:

- appear;
- disappear;
- change stability;
- split;
- give rise to cycles.

A bifurcation is a qualitative change of this kind.

Kuznetsov provides the formal bifurcation framework behind the normal forms used here (Kuznetsov 2004).

10.17 Saddle-node bifurcation

Take

$$\dot{x} = \mu - x^2.$$

Equilibria satisfy

$$x^2 = \mu.$$

For

$$\mu > 0,$$

there are two equilibria:

$$x_{\pm} = \pm\sqrt{\mu}.$$

For

$$\mu < 0,$$

there are no real equilibria.

At

$$\mu = 0,$$

the two branches meet.

This is the canonical saddle-node picture.

10.18 Saddle-node stability

Differentiate:

$$f'(x) = -2x.$$

At the positive branch,

$$f'(+\sqrt{\mu}) = -2\sqrt{\mu} < 0.$$

So the positive branch is locally stable.

At the negative branch,

$$f'(-\sqrt{\mu}) = 2\sqrt{\mu} > 0.$$

So the negative branch is unstable.

At the collision point,

$$f'(0) = 0.$$

The equilibrium is nonhyperbolic exactly where the qualitative structure changes.

10.19 Pitchfork bifurcation

Now consider

$$\dot{x} = \mu x - x^3.$$

Equilibria satisfy

$$x(\mu - x^2) = 0.$$

The central branch

$$x = 0$$

exists for all μ .

For

$$\mu > 0,$$

two additional branches appear:

$$x_{\pm} = \pm\sqrt{\mu}.$$

This is the standard supercritical pitchfork normal form.

10.20 Pitchfork stability exchange

The derivative is

$$f'(x) = \mu - 3x^2.$$

At the origin:

$$f'(0) = \mu.$$

Thus the origin is locally stable for

$$\mu < 0$$

and unstable for

$$\mu > 0.$$

At the nonzero branches:

$$f'(\pm\sqrt{\mu}) = -2\mu < 0.$$

Thus both new branches are locally stable.

A qualitative reorganization has occurred.

10.21 Why bifurcation language matters for learning

Training systems contain parameters that change over time:

- learning rate;
- momentum;
- weight decay;
- normalization scales;
- loss weights;
- routing temperature;
- data mixture;
- model state itself.

Qualitative changes in observed dynamics can resemble bifurcation phenomena.

The analogy can be useful.

It must not be overclaimed.

Observing a phase transition in training metrics does not by itself establish a mathematically identified bifurcation normal form.

That requires an actual reduced dynamical model.

10.22 Hopf bifurcation: the minimal picture

The supercritical Hopf normal form can be written in polar coordinates as

$$\dot{r} = \mu r - r^3,$$

$$\dot{\theta} = \omega.$$

For

$$\mu < 0,$$

the origin is radially stable.

For

$$\mu > 0,$$

the origin becomes unstable and a stable nonzero radius appears:

$$r_{\star} = \sqrt{\mu}.$$

The angular equation turns that radius into a periodic orbit.

This is the qualitative picture.

The chapter does not attempt the full Hopf theorem.

10.23 Periodic orbit is not convergence

A stable periodic orbit attracts nearby trajectories to a cycle.

It does not make the state converge to a fixed point.

This distinction becomes useful later when optimization dynamics oscillate.

A training system can settle into:

- a fixed point;
- a limit cycle;
- a slowly drifting attractor;
- more complicated recurrent behavior.

“Stable” therefore needs an object:

stable **what?**

10.24 Conservative dynamics

Not every useful system is dissipative.

Some dynamics preserve quantities.

Hamiltonian mechanics provides the canonical example.

Let

$$z = \begin{pmatrix} q \\ p \end{pmatrix}.$$

Define

$$J = \begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix}.$$

For Hamiltonian $H(z)$,

$$\dot{z} = J\nabla H(z).$$

The matrix J is skew-symmetric:

$$J^\top = -J.$$

10.25 Hamiltonian conservation

Along the exact autonomous Hamiltonian flow,

$$\frac{dH}{dt} = \nabla H^\top \dot{z}.$$

Substitute

$$\dot{z} = J\nabla H.$$

Then

$$\frac{dH}{dt} = \nabla H^\top J \nabla H.$$

For any real vector v ,

$$v^\top J v = 0$$

when J is skew-symmetric.

Therefore

$$\boxed{\frac{dH}{dt} = 0.}$$

The Hamiltonian is conserved along the exact flow.

10.26 Harmonic oscillator

Take

$$H(q, p) = \frac{1}{2}(q^2 + p^2).$$

Then

$$\dot{q} = p,$$

and

$$\dot{p} = -q.$$

In matrix form:

$$\dot{z} = Az,$$

with

$$A = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}.$$

Since

$$A^2 = -I,$$

the exact flow is

$$M(t) = e^{tA} = \begin{pmatrix} \cos t & \sin t \\ -\sin t & \cos t \end{pmatrix}.$$

The state rotates.

It does not decay.

10.27 Symplectic structure

A map M is symplectic in canonical coordinates when

$$M^\top JM = J.$$

For the exact harmonic-oscillator flow,

$$M(t)^\top JM(t) = J.$$

The computational witness verifies this identity symbolically.

Symplecticity means the canonical two-form is preserved.

In two dimensions, it also implies oriented phase-area preservation.

10.28 Energy conservation and symplecticity are not synonyms

The harmonic oscillator exact flow preserves:

- the symplectic form;
- the Hamiltonian energy.

But these are different properties.

A numerical method can be symplectic without preserving the original Hamiltonian exactly at every step.

Instead, symplectic integrators often exhibit favorable long-time behavior associated with a nearby modified Hamiltonian.

That belongs in the Numerics chapter.

The distinction matters because “structure-preserving” has more than one meaning.

10.29 Dissipative versus conservative pictures

The stable scalar flow satisfies

$$x(t) \rightarrow 0.$$

It loses the scalar Lyapunov energy

$$V(x) = x^2/2.$$

The harmonic oscillator does not converge.

It moves around a constant-energy orbit.

These are qualitatively different kinds of dynamics.

Later optimization chapters will encounter mixtures:

- damping;
- momentum;
- oscillation;
- dissipation;
- near-conservative transport.

One scalar notion of “stability” cannot describe all of them.

10.30 Exact flow versus numerical step

Suppose the exact flow over time h is

$$\Phi_h.$$

A numerical method defines another map:

$$\Psi_h.$$

A method of order p may satisfy locally

$$\Psi_h(x) = \Phi_h(x) + O(h^{p+1}).$$

That does not make the maps identical.

The numerical method is itself a discrete dynamical system.

It can possess behavior absent from the continuous system.

10.31 Discretization can change stability

Consider

$$\dot{x} = -\lambda x, \quad \lambda > 0.$$

The exact flow is stable for every positive λ .

Explicit Euler gives

$$x_{n+1} = (1 - h\lambda)x_n.$$

This discrete update is stable only when

$$|1 - h\lambda| < 1.$$

So the continuous system can be stable while the discretization explodes.

This is the bridge to numerical stability.

The next Numerics chapter will make it systematic.

10.32 Discretization can change geometry

A generic numerical step may fail to preserve:

- energy;
- symplectic form;
- norm;
- invariant sets;
- positivity;
- constraints.

Sometimes that failure is acceptable.

Sometimes it destroys the property the model was built to exploit.

This motivates structure-preserving methods.

Hairer, Lubich, and Wanner provide the standard geometric-numerical framework used in the later chapter (Hairer et al. 2006).

10.33 Depth as computational time

Residual architectures often take the form

$$x_{k+1} = x_k + hf_k(x_k).$$

If

$$f_k = f$$

and the interpretation is consistent, this resembles an explicit Euler step for

$$\dot{x} = f(x).$$

This makes depth interpretable as a form of computational time.

But the caveat is load-bearing.

Generic neural layers can vary with depth.

They can contain:

- normalization;
- attention;
- routing;
- stochasticity;
- discontinuous choices;
- data-dependent operators.

So:

residual update resemblance $\neq$ proof of one underlying autonomous ODE.

10.34 Flow-like architectures

The weaker claim is often the useful one.

An architecture can be studied with dynamical tools if:

- repeated updates accumulate state;
- local Jacobians matter;
- stability matters;
- long-horizon transport matters;
- step size or depth controls computation.

This does not require literal identification with classical mechanics.

It requires only that the dynamical viewpoint makes testable predictions or design constraints.

10.35 Optimizer dynamics

An optimizer can have state of its own.

Momentum methods introduce variables such as

$$v_t.$$

Adaptive optimizers introduce running statistics.

The combined training state can therefore be written schematically as

$$s_t = (\theta_t, m_t, v_t, \dots).$$

The update acts on this larger state.

The Optimizer-State Dynamics keystone already uses that principle.

The present chapter supplies the continuous-time language needed to interpret it.

10.36 Reinforcement-learning dynamics

Reinforcement learning adds further state:

- environment state;
- belief state;
- policy state;
- value estimates;
- replay buffers;
- target networks.

Learning itself becomes an evolving coupled system.

The RL foundations chapter will build on the same distinctions:

- state;
- flow/update;
- stability;
- fixed point.

10.37 Bifurcation as diagnostic hypothesis

Suppose a training process suddenly changes regime when a control parameter crosses a threshold.

Possible explanations include:

- true local bifurcation;
- numerical instability;
- data-mixture shift;
- optimizer-state transition;
- representation collapse;
- finite-sample noise.

Bifurcation language generates hypotheses.

It does not adjudicate them automatically.

The correct next step is to identify a reduced state and test the implied local dynamics.

10.38 Lyapunov functions as certificates

The same logic applies to Lyapunov ideas.

A training loss that decreases is not automatically a Lyapunov function for the full training state.

If the optimizer has hidden state, the scalar loss may rise temporarily while the full system remains stable.

A genuine Lyapunov certificate must be defined on the actual state whose stability is being claimed.

This becomes important in governed adaptation.

10.39 Four failure modes

10.39.1 Local linearization treated as global truth

The Jacobian describes first-order local behavior.

10.39.2 Stable equilibrium confused with stable orbit

Fixed points and cycles are different invariant objects.

10.39.3 Hamiltonian structure asserted from oscillation alone

Oscillation does not imply a canonical symplectic/Hamiltonian model.

10.39.4 Numerical method identified with continuous flow

An integrator is a separate discrete system.

These errors all arise from collapsing distinct mathematical objects.

10.40 Atlas connections

Numerics.

The next chapter studies how discrete methods approximate or distort exact flows.

Architecture history.

Residual, recurrent, and continuous-depth models can be organized by how they accumulate transformations.

Latent computational time.

Depth can become an adaptive integration horizon when the analogy is justified.

Optimization.

Momentum, damping, oscillation, and optimizer state are dynamical phenomena.

Reinforcement learning.

Bellman and policy updates define iterative dynamics on value/policy objects.

SPINDLE.

Operator splitting and adaptive depth require a clean distinction between flow generator, split update, and integrator.

10.41 What would falsify a dynamical interpretation?

A proposed dynamical model should make predictions.

Examples:

- perturbation decay rates;
- stability boundaries;
- conserved or slowly varying quantities;
- phase relationships;
- bifurcation thresholds;
- step-size sensitivity.

If those predictions fail systematically, the proposed state or flow model is wrong or incomplete.

The dynamical viewpoint earns its keep by constraining what should happen.

10.42 Closing view

The mathematical transition in this chapter is from static transformation to organized evolution.

We began with:

$$\dot{x} = f(x).$$

From that one object emerged:

- trajectories;
- exact flows;
- equilibria;
- stability;
- Lyapunov certificates;
- bifurcations;
- periodic orbits;
- Hamiltonian conservation;
- symplectic structure.

The most important lesson is not any one theorem.

It is object discipline.

The vector field is the law.

The trajectory is one history.

The flow is exact transport.

The integrator is a different discrete map.

Once those are separated, the later Atlas can ask a sharper question:

When machine computation unfolds through repeated updates, which properties belong to the intended dynamics, and which were created by the way we chose to discretize it?

That is where Numerical Intelligence begins.

10.43 References used in this chapter

- (Khalil 2002)
- (Kuznetsov 2004)
- (Hairer et al. 2006)

See ‘sources/source-locks/ATLAS-CH-DYN-001.yaml’ for exact source roles and claim boundaries.

Chapter 11

Discretization, Stability, and Splitting

Epistemic status: Established Theory + Atlas Synthesis + Exact Computational Witness

Specification: manuscript/specifications/ATLAS-CH-NUMERICS-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-NUMERICS-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-NUMERICS-001.md

Source lock: sources/source-locks/ATLAS-CH-NUMERICS-001.yaml

11.1 The continuous system is not the algorithm

The previous chapter separated the exact flow

$$\Phi_h$$

from a numerical update

$$\Psi_h.$$

That distinction is the starting point of numerical analysis.

A differential equation can be perfectly stable while a poor discretization explodes.

A numerical method can be stable while remaining inaccurate.

A high-order method can fail if used outside the regime for which its assumptions hold.

The numerical question is therefore not:

Does this update look like the differential equation?

It is:

In what precise sense does the discrete map approximate the exact flow, and what happens to errors as steps accumulate?

11.2 Map versus terrain

A useful allegory is navigation.

The exact flow is the true route through terrain.

A numerical method is a stepping rule based on a map.

Consistency says one small step points approximately where the true route points.

Stability says small errors in the stepping process are not catastrophically amplified.

Convergence says the full discrete route approaches the true route as step size shrinks.

The analogy has a limit.

Numerical error is not geographic error.

The actual theory requires norms, regularity assumptions, and method-specific bounds.

11.3 One-step methods

Consider

$$\dot{x} = f(t, x).$$

A one-step method has the form

$$x_{n+1} = \Psi_h(t_n, x_n).$$

The step size is

$$h = t_{n+1} - t_n.$$

The exact flow over one step is

$$\Phi_h(t_n, x).$$

The method is an approximation to this exact transport.

11.4 Local defect

Start one numerical step from the exact state.

Define

$$\delta_{n+1} = \Phi_h(t_n, x(t_n)) - \Psi_h(t_n, x(t_n)).$$

This is the **local defect**.

It asks:

If previous error were magically removed, how wrong would this one step be?

For an order- p method,

$$\delta_{n+1} = O(h^{p+1})$$

under the required smoothness assumptions.

11.5 Global error

The actual numerical state is not restarted from truth.

It starts from the previous approximation.

Define

$$e_n = x(t_n) - x_n.$$

This is the **global error**.

Global error combines:

- new local defect;
- propagation of old error.

That second term is where stability enters.

11.6 Consistency is not enough

A method can have a small one-step defect and still behave badly when errors are repeatedly amplified.

So:

$$\boxed{\text{consistency} \not\Rightarrow \text{convergence}}$$

without the relevant stability and regularity assumptions.

For a well-behaved one-step method over a finite time interval, a local defect

$$O(h^{p+1})$$

combined with suitable Lipschitz error propagation yields global error

$$O(h^p).$$

Iserles develops this numerical ODE viewpoint systematically (Iserles 2008).

11.7 Explicit Euler

Taylor-expand the exact solution:

$$x(t_n + h) = x(t_n) + hx'(t_n) + \frac{h^2}{2}x''(t_n) + O(h^3).$$

Using

$$x'(t_n) = f(t_n, x(t_n)),$$

drop the higher-order terms:

$$\boxed{x_{n+1} = x_n + hf(t_n, x_n)}.$$

This is explicit Euler.

11.8 Explicit Euler local defect

If the step begins from the exact state, then

$$\delta_{n+1}^{\text{EE}} = \frac{h^2}{2} x''(t_n) + O(h^3).$$

So the local defect is second order.

The accumulated global error is first order:

$$e_n = O(h)$$

under standard finite-time regularity and stability conditions.

The distinction between local h^2 and global h is fundamental.

11.9 Implicit Euler

Implicit Euler evaluates the vector field at the future state:

$$x_{n+1} = x_n + hf(t_{n+1}, x_{n+1}).$$

The future state appears on both sides.

So a step generally requires solving an equation.

That extra computational cost buys a different stability geometry.

11.10 Explicit versus implicit is not fast versus slow

“Explicit” means the new state can be computed directly from known quantities.

“Implicit” means the new state is defined by an equation that must be solved.

The distinction is algebraic.

It does not automatically determine:

- wall-clock cost;
- accuracy;
- parallelism;
- implementation quality.

A cheap implicit solve can outperform a tiny explicit step.

A difficult nonlinear implicit solve can dominate the computation.

11.11 The scalar test equation

To study numerical stability, use

$$y' = \lambda y.$$

Its exact one-step factor is

$$e^{h\lambda}.$$

Define

$$z = h\lambda.$$

A one-step numerical method gives

$$y_{n+1} = R(z)y_n,$$

where R is the **stability function**.

The method's behavior on this scalar problem reveals important properties of its discrete dynamics.

11.12 Explicit Euler stability function

Explicit Euler gives

$$y_{n+1} = y_n + h\lambda y_n.$$

Therefore

$$R_{\text{EE}}(z) = 1 + z.$$

Use the standard non-growth absolute-stability set

$$\mathcal{S} = \{z : |R(z)| \leq 1\}.$$

For explicit Euler,

$$|1 + z| \leq 1$$

is the absolute-stability disk.

For a decaying exact mode with

$$\operatorname{Re}(\lambda) < 0,$$

strict asymptotic decay of the discrete mode requires the interior condition

$$|1 + z| < 1.$$

11.13 The explicit stability disk

The non-growth condition

$$|1 + z| \leq 1$$

defines the closed disk of radius 1 centered at

$$-1$$

in the complex plane.

Its open interior

$$|1 + z| < 1$$

gives strict asymptotic decay.

A stable continuous eigenmode can therefore become unstable numerically if

$$h\lambda$$

falls outside this disk.

The differential equation did not become unstable.

The discretization did.

11.14 Implicit Euler stability function

Implicit Euler gives

$$y_{n+1} = y_n + h\lambda y_{n+1}.$$

Rearrange:

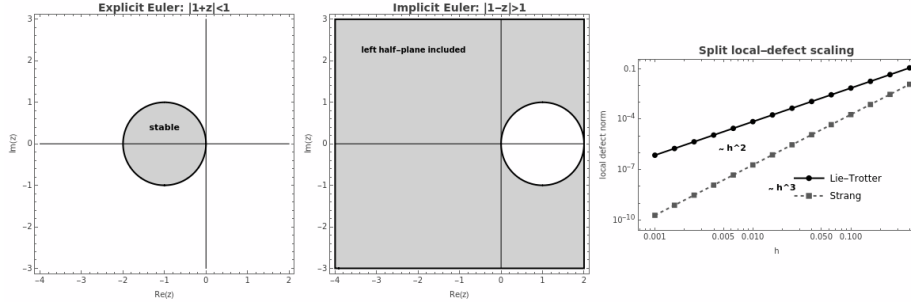


Figure 11.1: Explicit and implicit Euler absolute-stability regions beside log-log Lie-Trotter and Strang local-defect scaling for the exact split witness.

$$(1 - z)y_{n+1} = y_n.$$

Thus

$$R_{\text{IE}}(z) = \frac{1}{1 - z}.$$

The standard non-growth absolute-stability condition is

$$\left| \frac{1}{1 - z} \right| \leq 1.$$

Equivalently,

$$|1 - z| \geq 1.$$

Strict decay uses the corresponding strict inequalities.

11.15 A-stability

Every point in the closed left half-plane satisfies

$$|1 - z| \geq 1.$$

Therefore implicit Euler is A-stable in the standard non-growth sense.

For every point in the open left half-plane,

$$|1 - z| > 1,$$

so an exponentially decaying scalar mode remains strictly decaying for any positive step size.

That is a strong stability property.

It is not a claim of arbitrary accuracy.

11.16 L-stability

For implicit Euler,

$$R_{\text{IE}}(z) = \frac{1}{1-z}.$$

As

$$|z| \rightarrow \infty$$

within the left half-plane,

$$R_{\text{IE}}(z) \rightarrow 0.$$

Thus strongly damped continuous modes are also strongly damped numerically.

Implicit Euler is L-stable.

This matters in stiff problems, where very fast decaying modes can otherwise pollute a computation.

11.17 Stability is not accuracy

Take

$$y' = -100y.$$

Use

$$h = 0.05.$$

Then

$$z = -5.$$

The exact one-step factor is

$$e^{-5} \approx 0.00673795.$$

Explicit Euler gives

$$R_{\text{EE}}(-5) = -4.$$

The numerical mode grows in magnitude.

Implicit Euler gives

$$R_{\text{IE}}(-5) = \frac{1}{6} \approx 0.1667.$$

It decays.

But it is not close to the exact factor.

So:

stable $\neq$ accurate.

11.18 The stiff scalar witness

The same continuous mode can therefore produce three qualitatively different one-step behaviors:

$$\text{exact: } 0.0067,$$

$$\text{explicit Euler: } -4,$$

$$\text{implicit Euler: } 0.1667.$$

Explicit Euler is unstable.

Implicit Euler is stable.

Neither numerical factor equals the exact one.

This is why numerical analysis needs both stability and error analysis.

11.19 What stiffness means

Stiffness is not simply “large derivative.”

A useful operational picture is:

the system contains rapidly decaying modes that force a chosen numerical method to use a much smaller stable step than the slower behavior of interest would otherwise require.

The word is therefore partly problem-relative and partly method-relative.

Hairer and Wanner develop the stiff ODE framework in detail (Hairer and Wanner 1996).

11.20 Multiple timescales

Suppose

$$\dot{x} = \begin{pmatrix} -1 & 0 \\ 0 & -1000 \end{pmatrix} x.$$

The first mode evolves on timescale roughly 1.

The second decays on timescale roughly 10^{-3} .

After the fast mode has essentially vanished, an explicit method may still be forced to use tiny steps because of its stability restriction.

The numerical method is spending effort respecting a mode that no longer matters much to the observable slow trajectory.

This is the classic stiffness tension.

11.21 Implicit methods move the stability boundary

An implicit method can tolerate much larger stable steps for decaying modes.

But the price can include:

- linear solves;
- nonlinear solves;
- Jacobians;
- preconditioning;
- memory;
- implementation complexity.

Numerical method choice is therefore a systems tradeoff.

This becomes relevant when neural architectures contain implicit or equilibrium-style blocks.

11.22 Absolute stability is not nonlinear stability

The region

$$|R(z)| < 1$$

is defined for the scalar linear test equation.

It provides powerful local/mode-level information.

It is not a universal theorem about a nonlinear problem.

A nonlinear method can leave the test region satisfied while encountering:

- changing Jacobians;
- constraints;
- bifurcations;
- non-normal amplification;
- state-dependent stiffness.

The stability region is a diagnostic object, not a complete nonlinear certificate.

11.23 From one vector field to two

Suppose the dynamics decompose:

$$\dot{x} = (A + B)x.$$

Perhaps:

- A is easy to integrate;
- B is easy to integrate;
- $A + B$ is expensive;
- each subflow preserves a useful structure.

This motivates splitting.

Instead of approximating the full vector field in one step, we compose simpler exact or approximate subflows.

11.24 Lie–Trotter splitting

For constant matrices, define

$$S_{\text{LT}}(h) = e^{hA}e^{hB}.$$

This means the B subflow acts first on a state vector, followed by the A subflow, under the usual right-to-left composition convention.

The result approximates

$$e^{h(A+B)}.$$

How accurate is it?

The answer depends on commutation.

11.25 Noncommutativity is the obstruction

Expand both subflows:

$$e^{hA} = I + hA + \frac{h^2}{2}A^2 + O(h^3),$$

$$e^{hB} = I + hB + \frac{h^2}{2}B^2 + O(h^3).$$

Their product has second-order term

$$\frac{1}{2}A^2 + AB + \frac{1}{2}B^2.$$

The exact flow has second-order term

$$\frac{1}{2}(A^2 + AB + BA + B^2).$$

Subtracting gives

$$S_{\text{LT}}(h) - e^{h(A+B)} = \frac{h^2}{2}[A, B] + O(h^3),$$

where

$$[A, B] = AB - BA.$$

11.26 Commuting subflows

If

$$[A, B] = 0,$$

then for constant matrices

$$e^{hA}e^{hB} = e^{h(A+B)}$$

exactly.

The split introduces no error.

This is an important lesson for architecture.

Splitting quality is not determined only by the size of the components.

It depends on their algebraic interaction.

11.27 Exact noncommuting witness

Choose

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix},$$

and

$$B = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}.$$

Then

$$[A, B] = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

The operators do not commute.

The exact symbolic series therefore exposes a nonzero second-order Lie defect.

11.28 Lie local defect

For the declared witness,

$$S_{\text{LT}}(h) - e^{h(A+B)}$$

has leading term

$$\frac{h^2}{2} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

The off-diagonal terms begin at order

$$h^3.$$

The local defect is therefore

$$O(h^2).$$

Under the usual finite-time stability assumptions, Lie–Trotter is globally first order.

11.29 Strang splitting

A symmetric composition is

$$\boxed{S_{\text{S}}(h) = e^{hA/2} e^{hB} e^{hA/2}.$$

The half-step symmetry cancels the second-order local defect.

For sufficiently regular bounded operators,

$$S_{\text{S}}(h) - e^{h(A+B)} = O(h^3).$$

This yields second-order global accuracy under the usual stability assumptions.

11.30 Exact Strang witness

For the same A, B , the symbolic defect begins as

$$\begin{pmatrix} O(h^4) & h^3/12 \\ -h^3/6 & O(h^4) \end{pmatrix}.$$

The first nonzero terms are cubic.

The figure’s log–log plot evaluates exact matrix exponentials across a declared h -grid.

The asymptotic slopes reveal the expected local orders:

- Lie–Trotter: 2;
- Strang: 3.

11.31 Symmetry is doing mathematical work

The advantage of Strang is not merely that it performs more substeps.

The composition is time-symmetric in its outer A factors.

That symmetry removes an entire order of local defect.

Architectural ordering can therefore change approximation order even when the same primitive operations are used.

This is the core mathematical motivation for later split-operator neural architectures.

11.32 Splitting is not arbitrary modularity

Suppose a model has two modules.

Calling their composition a “split method” does not make numerical splitting theory applicable.

A genuine splitting interpretation requires a declared underlying evolution such as

$$\dot{x} = (A + B)x$$

or its nonlinear/operator analogue.

The substeps should correspond to meaningful component flows or approximations.

Otherwise “splitting” is metaphor, not derivation.

11.33 Splitting can preserve structure

Suppose each subflow is symplectic.

The composition of symplectic maps is symplectic.

Thus a split integrator can preserve the symplectic form exactly even when it only approximates the full exact flow.

This is one way discrete algorithms can preserve qualitative geometry.

McLachlan and Quispel survey this splitting viewpoint (McLachlan and Quispel 2002).

11.34 Structure-preserving means name the structure

A method may preserve:

- symplectic form;
- norm;
- positivity;
- mass;
- reversibility;
- constraint manifold;
- volume.

These are different promises.

A method can preserve one while violating another.

So the phrase

structure-preserving

is incomplete unless the structure is named.

Hairer, Lubich, and Wanner develop this geometric integration perspective (Hairer et al. 2006).

11.35 Backward error viewpoint

A powerful later idea is to ask whether a discrete method exactly follows a nearby modified differential equation.

This is backward error analysis.

Instead of saying

the numerical path is slightly wrong,

we ask

for which nearby dynamics is this path more nearly exact?

This can explain excellent long-time behavior of geometric methods.

The present chapter only introduces the viewpoint.

A later Numerical Intelligence chapter can develop it more deeply.

11.36 Step size is a modeling parameter

The step size

$$h$$

controls more than numerical precision.

It can alter:

- stability;
- damping;
- phase error;
- effective depth;
- computational cost.

In learned systems, scale parameters that resemble step sizes can therefore influence qualitative computation.

But an analogy becomes useful only when the update admits a coherent numerical interpretation.

11.37 Depth as number of steps

If a residual block approximates

$$x_{n+1} = x_n + hf(x_n),$$

then increasing depth while reducing h can resemble time refinement.

The natural questions become:

- Does the output converge as depth is refined?
- Is the method stable?
- Which invariant changes with depth?
- Is the architecture approximating one flow or changing the vector field itself?

The later Adaptive Depth chapter will inherit this language.

11.38 Adaptive depth as error control

Classical adaptive integration allocates more steps where error estimates demand them.

A learned analogue might use:

- residual magnitude;
- local defect proxy;
- uncertainty;
- contract violation;
- commutator magnitude.

to decide whether more computation is needed.

That is an architectural research programme.

This chapter supplies the numerical grammar.

It does not validate a specific controller.

11.39 Commutator as split diagnostic

For Lie–Trotter,

$$[A, B]$$

appears directly in the leading local defect.

This suggests a diagnostic principle:

if two learned operators are intended to be split, measure how strongly their order matters.

In nonlinear systems, Lie brackets or lifted commutators can play analogous roles.

This is one bridge to SPLICE-style diagnostics.

Again, the exact numerical theorem applies only after the operator model has been declared.

11.40 Stiff learned dynamics

A learned update can exhibit widely separated effective timescales.

Examples might include:

- fast normalization variables;
- slow feature evolution;
- rapidly decaying auxiliary states;

- momentum transients.

Calling such a system “stiff” requires an actual dynamical and numerical analysis.

The useful test is whether stability forces a step/depth scale much smaller than the behavior of interest requires.

11.41 Implicit computation

Implicit Euler solves for a future state satisfying an equation.

This creates a conceptual link to:

- equilibrium networks;
- implicit layers;
- fixed-point solvers;
- proximal updates.

But the relationship is not identity.

An implicit neural block can solve a different equation with different stability and convergence properties.

Numerical language should clarify the contract, not erase differences.

11.42 Four numerical failure modes

11.42.1 Small local defect, unstable propagation

Consistency without stability.

11.42.2 Stable but inaccurate coarse steps

The stiff implicit-Euler witness.

11.42.3 High-order formula outside assumptions

Formal order does not protect against nonsmoothness, singularity, or unbounded-operator domain failures.

11.42.4 Structure-preserving label without structure

A claim is meaningless until the preserved invariant or geometry is stated.

These are distinct failures.

11.43 What would falsify a numerical interpretation?

Suppose a neural architecture is claimed to approximate a particular flow.

Useful falsification tests include:

- depth refinement fails to converge;
- halving the apparent step size does not reduce error at the predicted order;
- stability boundaries disagree with the proposed test equation or Jacobian model;
- split ordering has no relation to predicted commutator effects;
- claimed invariants drift despite a supposed structure-preserving construction.

A numerical interpretation should generate quantitative predictions.

11.44 Atlas connections

Dynamics.

The exact flow is the object being approximated.

Adaptive Depth.

Depth can become an error-controlled step budget when the numerical model is justified.

Split-operator architectures.

Commutators and composition order become design variables.

Variational optimization.

Discrete updates can be derived from geometric or variational principles.

Numerical Intelligence.

Solvers, residuals, error estimates, and backward error become computational mechanisms rather than background implementation details.

SPINDLE.

The controlled-operator-splitting programme sits directly on the distinctions developed here.

11.45 Closing view

Numerical analysis asks a disciplined question:

What discrete computation faithfully represents the behavior we intend?

The answer is not contained in one error order.

A method must negotiate:

- local approximation;
- error propagation;
- stability;
- stiffness;
- computational cost;
- algebraic interaction;
- geometric structure.

The scalar test equation showed why a stable continuous system can become numerically unstable.

The stiff witness showed why stable does not mean accurate.

The split witness showed why operator order matters.

And Strang splitting showed that composition symmetry can improve accuracy without changing the primitive operators.

These are not implementation details.

They are mathematical properties of computation.

The next Atlas chapters can now ask a sharper architectural question:

If depth, modules, and residual updates are treated as numerical machinery, which integration contract are they actually satisfying?

11.46 References used in this chapter

- (Iserles 2008)
- (Hairer and Wanner 1996)
- (McLachlan and Quispel 2002)
- (Hairer et al. 2006)

See ‘sources/source-locks/ATLAS-CH-NUMERICS-001.yaml’ for exact source roles and claim boundaries.

Chapter 12

Local-to-Global Mathematics

Epistemic status: standard sheaf/local-to-global mathematics + audited Objects prerequisite + Atlas synthesis + exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-LOCALGLOBAL-001.md

Formal packet: mathematics/derivations/ATLAS-CH-LOCALGLOBAL-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-LOCALGLOBAL-001.md

Source lock: sources/source-locks/ATLAS-CH-LOCALGLOBAL-001.yaml

Many systems are easier to understand locally than globally.

A sensor sees one region.

A module exposes one interface.

A worker holds one piece of state.

A chart describes one patch.

A subsystem enforces one constraint.

The difficult question is not whether each piece makes sense by itself.

It is:

do the pieces agree where they meet, and if they do, is there one coherent global object that realizes them?

That is the local-to-global problem.

This chapter introduces only the amount of graph and sheaf language needed to make that question precise.

12.1 Local validity is not global coherence

Suppose two teams each provide a locally valid configuration.

One says the shared boundary value is 2.

The other says it is 3.

Nothing is wrong inside either local object.

The failure appears only when the objects are compared on their common interface.

This is the central structural lesson:

consistency lives on overlaps.

12.2 Start with incidence

Before assigning data, we need to know which pieces touch.

A graph:

$G = (V, E)$

can record pairwise incidence.

A cover:

$U = \bigcup_i U_i$

can record which local regions together describe a larger region.

These structures answer:

- what is local?
- which pieces overlap?
- which interfaces exist?

They do not yet say what data live there.

12.3 A graph is not a sheaf

A graph can tell us that vertex A is adjacent to vertex B.

It cannot, by itself, tell us:

- what values A and B carry;
- what part of those values is visible at the edge;

- what equality or transformation the edge requires.

A sheaf-like object adds data spaces and maps over the incidence structure.

This is why graph topology and carried algebra must remain distinct.

12.4 Local sections

For a region U , let:

$F(U)$

denote the allowable data on that region.

An element:

s in $F(U)$

is called a local section.

The word **section** is useful because it emphasizes that the data belong to a particular region.

The same numerical vector can be a section on one region and not even be well-typed on another.

12.5 Restriction maps

If:

$V \subseteq U$,

a restriction map:

$\rho^U_V: F(U) \rightarrow F(V)$

asks:

what does the data on U say when viewed only on V ?

Restriction must be coherent.

Restricting to the same region does nothing:

$\rho^U_U = \text{id}$.

Restricting in stages must agree with restricting directly:

$\rho^V_W \circ \rho^U_V = \rho^U_W$

for:

$W \subseteq V \subseteq U$.

12.6 Why restriction is an interface map

OBJECTS-001 introduced interfaces as the obligations composition may assume across a boundary.

Restriction makes that idea mathematical.

Two larger local objects do not need to be identical.

They need to induce compatible data on what they share.

The shared region is the comparison interface.

12.7 Matching families

Let:

$\{U_i\}$

cover U .

Choose one local section:

s_i in $F(U_i)$

for each region.

The family is matching when every pair agrees after restriction to its overlap:

$\rho^{\{U_i\}}_{\{U_i \text{ intersect } U_j\}}(s_i) = \rho^{\{U_j\}}_{\{U_i \text{ intersect } U_j\}}(s_j).$

This is exact compatibility.

12.8 Compatibility is typed

The two local sections may live in different spaces:

$F(U_i)$

and:

$F(U_j).$

They become comparable only after both are mapped into:

$F(U_i \text{ intersect } U_j).$

This prevents a common category error:

compare whole local states when only boundary data are supposed to agree.

12.9 A presheaf gives the restriction machinery

A presheaf supplies:

- local data spaces;
- restriction maps;
- identity and composition laws.

That is already useful.

It gives a disciplined way to talk about local views.

But it does not yet guarantee that compatible local views come from one global object.

12.10 A sheaf adds unique gluing

A sheaf adds the local-to-global property.

For every matching family over a cover, there must exist a unique:

s in $F(U)$

whose restriction to each U_i is s_i .

The word **unique** matters.

The condition has two parts:

1. existence;
2. uniqueness.

12.11 Existence and uniqueness answer different questions

Existence asks:

can these local pieces be assembled at all?

Uniqueness asks:

if they can, do they determine one global object or several?

A system can fail either way.

Keeping these clauses separate is useful far beyond sheaf theory.

12.12 The equalizer picture

For a finite cover, gather all local restrictions into a map:

$R: F(U) \rightarrow \prod_i F(U_i).$

Gather all overlap discrepancies into a map:

$\Delta: \prod_i F(U_i) \rightarrow \prod_{i < j} F(U_i \cap U_j)$.

Matching families satisfy:

$\Delta = 0$.

The sheaf condition says, informally:

the local families that actually come from global sections are exactly the matching families, and the global source is unique.

For vector-space-valued examples:

$\text{im}(R) = \ker(\Delta)$

with R injective.

12.13 Why this is a useful computational form

The equalizer picture turns local-to-global reasoning into explicit operations:

- restrict;
- subtract on overlaps;
- test a kernel condition;
- reconstruct.

That is exactly the amount of sheaf theory needed for many finite engineering examples.

The deeper theory remains available later if needed.

12.14 Exact witness: three points

Let:

$X = \{a, b, c\}$.

Use the cover:

$U = \{a, b\}$

and:

$V = \{b, c\}$.

Their overlap is:

$W = \{b\}$.

This is the smallest example that has:

- two local regions;

- one genuine overlap;
- one global union.

12.15 Sections are functions

For every subset S of X , define:

$$F(S) = \mathbb{R}^S.$$

A section is simply a real-valued function on S .

Restriction means forgetting coordinates outside the smaller subset.

Nothing abstract is hidden.

12.16 The global-to-local map

Write a global section as:

$$(x, y, z).$$

Its restrictions are:

$$(x, y)$$

on U and:

$$(y, z)$$

on V .

So:

$$R(x, y, z) = (x, y, y, z).$$

12.17 Uniqueness is injectivity

Suppose two global sections have the same restrictions to U and V .

They agree at:

- a because U contains a ;
- b because both regions contain b ;
- c because V contains c .

Therefore the global sections are equal.

In matrix language, R has rank 3.

The local restrictions determine at most one global function.

12.18 The overlap-discrepancy map

A general pair of local sections is:

$$((u_a, u_b), (v_b, v_c)).$$

Define:

$$\Delta(u_a, u_b, v_b, v_c) = u_b - v_b.$$

The pair is compatible exactly when:

$$\Delta = 0.$$

The discrepancy lives precisely on the overlap coordinate b .

12.19 Image equals kernel

The image of R consists of vectors:

$$(x, y, y, z).$$

The kernel of Δ consists of vectors satisfying:

$$u_b = v_b.$$

These are the same set.

Therefore:

$$\text{im}(R) = \ker(\Delta).$$

This is the exact gluing statement in the finite witness.

12.20 A compatible family

Choose:

$$s_U = (1, 2)$$

and:

$$s_V = (2, 4).$$

The overlap values both equal 2.

So:

$$\Delta = 0.$$

The unique global glue is:

$$s = (1, 2, 4).$$

12.21 An incompatible family

Now choose:

$q_U = (1, 2)$

and:

$q_V = (3, 4)$.

The overlap discrepancy is:

$\Delta = 2 - 3 = -1$.

It is nonzero.

Therefore the pair is not in the image of R .

No global function can restrict to both local sections.

12.22 The failure is localized

The value at a is not a problem.

The value at c is not a problem.

The incompatibility is entirely at b .

This is one practical virtue of explicit overlap maps:

they tell us where coherence fails.

12.23 Gluing is stronger than aggregation

Suppose we simply concatenate the two local records:

$(1, 2, 3, 4)$.

We have stored all the data.

We have not resolved the fact that the two copies of b disagree.

Aggregation collects.

Gluing enforces compatibility.

12.24 The graph view of the witness

Represent U and V as two vertices.

Connect them by an edge because their overlap is nonempty.

That graph records interaction.

But the numerical obstruction -1 does not come from the graph alone.

It comes from:

- data spaces;
- restrictions;
- the two local sections.

The carried algebra matters.

12.25 Cellular sheaves make graph-local algebra explicit

Cellular sheaves assign data spaces to cells and maps along incidences under a declared convention.

This creates a finite algebraic object over a graph or cell complex.

Curry develops cellular sheaves as a computable combinatorial sheaf framework, and Hansen–Ghrist develop further graph/cell-complex structure.

The Atlas needs only the elementary idea here.

12.26 We do not need cohomology yet

Sheaf cohomology is a powerful obstruction and structural tool.

It is not required to understand the basic gluing mechanism.

For this chapter, a nonzero overlap discrepancy already provides an exact finite obstruction.

The more advanced machinery would distract from the foundational idea.

12.27 Pairwise compatibility needs the sheaf context

A common slogan says:

pairwise agreement does not imply global agreement.

That slogan is too crude here.

For an actual sheaf over an open cover, agreement on every pairwise overlap is precisely the matching-family condition, and the sheaf axiom supplies a unique global glue.

The correct warning is different:

pairwise plausibility in an arbitrary constraint system is not enough unless the restriction structure and gluing property are specified.

12.28 Local-to-global claims are model-relative

A global section is global relative to the chosen sheaf.

Change:

- the regions;
- the data spaces;
- the restriction maps;
- the overlap semantics;

and the set of global sections can change.

Therefore a successful glue is evidence about one declared local-to-global model.

It is not universal coherence.

12.29 Exact consistency is brittle by design

If one local sensor reports 2.000 and another reports 2.001, exact equality fails.

That is not a defect in the sheaf condition.

It means the mathematical claim being tested is exact.

If the application tolerates small discrepancies, it needs a different object:

- a metric;
- a tolerance;
- a likelihood;
- a loss;
- an optimization problem.

12.30 Approximate consistency is a second layer

Robinson's sensor-integration work makes this distinction operational.

Local measurements can be compared through a sheaf model even when they are not exactly consistent.

One can then define an approximate-section or fusion problem.

The Atlas keeps the layers separate:

1. exact structural compatibility;
2. approximate repair/fusion.

12.31 An approximate glue is not a sheaf-theoretic equality

Suppose the overlap values are:

2

and:

2.1.

A least-squares procedure might replace both with:

2.05.

That can be sensible.

But it changes the data.

The original family still did not satisfy exact compatibility.

The optimization result and the sheaf-gluing result are different claims.

12.32 Local truth is not required

A section can be mathematically valid while factually wrong.

Suppose two sensors both report 100 when the true value is 0.

Their data can glue perfectly.

The sheaf has checked consistency, not truth.

This distinction will matter whenever sheaf language is used for evidence fusion.

12.33 Global consistency is not semantic adequacy

A software system can satisfy all declared interface equalities and still implement the wrong specification.

A scientific model can integrate all measurements consistently while omitting a latent variable.

A local-to-global formalism checks the contracts it was given.

It does not prove the contracts are complete.

12.34 Local-to-global structure can expose hidden assumptions

The advantage is not only successful reconstruction.

Writing explicit overlaps forces questions such as:

- what must two modules agree on?
- which values are shared?
- which transformations compare them?
- where can inconsistency appear?

That makes assumptions inspectable.

12.35 Graphs provide the skeleton

A graph or hypergraph can organize:

- subsystems;
- agents;
- sensors;
- patches;
- data sources;
- interfaces.

It gives the combinatorial skeleton.

A sheaf-like structure then says what algebra or information moves across that skeleton.

Keeping those layers distinct makes models easier to audit.

12.36 Covers provide another skeleton

Sometimes the natural structure is spatial or semantic rather than graph-first.

A collection:

$\{U_i\}$

covers a larger domain.

Overlaps:

$U_i \text{ intersect } U_j$

become the places where local descriptions must agree.

This view is natural for:

- coordinate charts;
- distributed observations;

- overlapping datasets;
- decomposed physical domains.

12.37 Interface data can be lower-dimensional

Two large subsystems may share only a small boundary.

They need not expose all internal state.

Restriction maps formalize this compression:

$F(U_i) \rightarrow F(U_i \cap U_j)$.

The local-to-global problem can therefore be much smaller than comparing whole subsystem states.

This anticipates later boundary-contract and separator ideas without importing them as prerequisites.

12.38 Compatibility can be transformed rather than literal identity

In the function witness, both sides restrict to the same scalar and equality is literal.

More general sheaves can use maps that translate local coordinates into a common comparison space.

Thus the real principle is not:

local arrays must be numerically equal.

It is:

their induced boundary data must agree after the declared restriction maps.

12.39 Coordinate changes fit naturally

Two charts can describe the same underlying object with different coordinates.

The overlap maps translate each local representation before comparison.

This is one reason local-to-global mathematics appears throughout geometry.

The Atlas will use the same structural idea later in more computational settings.

12.40 Composition requires more than adjacency

Connecting modules with arrows is not enough.

A meaningful composition needs:

- compatible types;
- compatible boundary semantics;
- agreement on shared quantities;
- a rule for assembling the composite.

Sheaf gluing provides one mathematically clean archetype for such assembly.

It is not the only possible compositional formalism.

12.41 Obstruction certificates are valuable outputs

When gluing fails, the answer need not be only:

no.

A useful system should expose:

- which overlap failed;
- by how much;
- under which restriction maps;
- which local sections were involved.

In the witness, the certificate is:

$\Delta = -1$

at b .

That is already actionable.

12.42 A larger graph produces a discrepancy vector

For many overlaps, collect all boundary disagreements into:

$\Delta(s)$.

Then:

$\Delta(s) = 0$

means exact compatibility.

Nonzero coordinates localize violated interfaces.

This is a finite linear-algebra pattern that later chapters can reuse.

12.43 Zero discrepancy can have several meanings

A zero discrepancy might arise because:

- the data are genuinely consistent;
- the restrictions are too weak to detect a conflict;
- the comparison space has discarded important information;
- two wrong local models happen to agree.

Therefore the quality of the restriction maps matters.

An easy compatibility test can be weak evidence.

12.44 Richer interfaces detect more and cost more

A boundary representation that exposes more information can detect more incompatibilities.

It can also increase:

- communication;
- storage;
- computation;
- coupling.

This creates a general design tradeoff:

expose enough boundary structure to protect composition, but not so much that modularity disappears.

The chapter does not solve this tradeoff.

It only makes it visible.

12.45 Sheaf language should earn its keep

The word **sheaf** is useful when the problem genuinely has:

- local data;
- overlaps/incidences;
- restriction maps;
- a local-to-global question.

If those objects are absent, sheaf language can become decorative.

The Atlas therefore applies a discipline:

name the section spaces and restriction maps before claiming a sheaf viewpoint.

12.46 A practical local-to-global ledger

Before accepting a gluing claim, record:

Field	Question
domain	What is the global object/space?
local regions	Which pieces cover or compose it?
incidence	Which pieces overlap or meet?
local data	What is $F(U_i)$?
overlap data	What is $F(U_i \text{ intersect } U_j)$?
restrictions	How is local data mapped to the overlap?
compatibility	Which equalities must hold?
existence	Does every matching family considered have a glue?
uniqueness	Can two global objects have the same local restrictions?
obstruction	What records a failure to match?
approximation	If disagreement is tolerated, which metric/loss?
interpretation	What does a global section mean in the application?
boundary	What does consistency not prove?

This ledger prevents local-to-global language from becoming metaphor.

12.47 Failure modes

12.47.1 Graph-equals-sheaf

Incidence structure is confused with the data and maps carried over it.

12.47.2 Local-equals-global

Locally valid pieces are assumed to assemble without overlap checks.

12.47.3 Pairwise-plausible-equals-matching

Informal compatibility substitutes for exact restriction equations.

12.47.4 Matching-equals-truth

A global section is treated as factual correctness.

12.47.5 Approximate-equals-exact

A low-loss fusion is described as exact gluing.

12.47.6 Zero-obstruction-equals-complete-model

The declared restrictions agree, so omitted constraints are assumed not to exist.

12.47.7 Sheaf-as-decoration

The terminology is used without explicit section spaces or restriction maps.

12.48 What the exact witness establishes

The companion witness proves:

- the global-to-local map $R:R^3 \rightarrow R^4$ is injective;
- the overlap discrepancy is $\Delta(u_a, u_b, v_b, v_c) = u_b - v_b$;
- $\text{im}(R) = \ker(\Delta)$;
- (1,2) on U and (2,4) on V glue uniquely to (1,2,4);
- (1,2) on U and (3,4) on V have discrepancy -1 and cannot glue;
- the obstruction is localized to the shared coordinate b.

Nothing about noisy fusion, semantic truth, or global system stability is inferred.

12.49 Downstream handoff

Later chapters may now assume the following local-to-global vocabulary:

- graph/cover incidence;
- local section;
- restriction map;
- matching family;
- gluing existence;
- gluing uniqueness;
- discrepancy/obstruction;
- exact versus approximate compatibility.

They must independently define their domain-specific section spaces, restriction maps, and interpretation.

The local-to-global lesson is simple:

make the interfaces explicit, compare what each local piece says there, and do not call the whole coherent until the gluing condition is actually satisfied.

12.50 References used in this chapter

- Justin Michael Curry, *Sheaves, Cosheaves and Applications*, University of Pennsylvania doctoral thesis, 2014, arXiv:1303.3255.
- Michael Robinson, *Sheaves are the canonical data structure for sensor integration*, Information Fusion 36 (2017), 208–224, DOI 10.1016/j.inffus.2016.12.002.
- Jakob Hansen and Robert Ghrist, *Toward a spectral theory of cellular sheaves*, Journal of Applied and Computational Topology 3 (2019), 315–358, DOI 10.1007/s41468-019-00038-7.

Exact source identities and authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-LOCALGLOBAL-001.yaml`

Chapter 13

Representations and Invariants

Epistemic status: Established Theory + Atlas Synthesis + Computational Witness

Specification: manuscript/specifications/ATLAS-CH-REP-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-REP-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-REP-001.md

Source lock: sources/source-locks/ATLAS-CH-REP-001.yaml

13.1 A representation is not merely a vector

A learned representation is often described as a vector

$$z \in \mathbb{R}^d.$$

That description tells us where the coordinates live.

It does not tell us:

- what transformations preserve meaning;
- which coordinates are arbitrary;
- which symmetries are respected;
- what downstream tasks can recover;
- which latent factors are identifiable;
- whether sparse features exist;
- whether multiple features share dimensions.

The Atlas therefore uses a stronger working definition.

A representation is an information-bearing encoding whose geometry, invariances, equivalences, and downstream transformation laws matter.

This framing is consistent with the broad representation-learning literature (Bengio et al. 2013), but the exact organization used here is Atlas synthesis.

13.2 Languages that preserve the same message

Consider two languages.

Their words look different.

Their grammar may differ.

Yet a translation can preserve the distinctions needed by a reader.

This is a useful analogy for representation equivalence.

Two encodings may differ coordinate-by-coordinate while preserving the structure needed by downstream computation.

The analogy fails if taken literally.

Natural-language translation is approximate, context-dependent, and not generally invertible.

Representation equivalence must be defined mathematically for the property we care about.

13.3 Representation map

Let

$$r : \mathcal{X} \rightarrow \mathcal{Z}$$

map inputs into a representation space.

A representation is useful only relative to some downstream purpose.

That purpose might involve:

- classification;
- control;
- reconstruction;
- retrieval;
- prediction;
- planning;
- communication.

There is therefore no universal scalar called “representation quality.”

Different tasks can prefer different structures.

13.4 Coordinates versus represented structure

Suppose

$$z = r(x).$$

For a finite-dimensional real representation space $\mathcal{Z} = \mathbb{R}^d$, apply an invertible linear transformation

$$T : \mathbb{R}^d \rightarrow \mathbb{R}^d$$

and define

$$\tilde{z} = Tz.$$

The coordinates changed.

Did the representation change?

The answer depends on the downstream computation.

If a linear readout is

$$y = w^\top z,$$

then choosing

$$\tilde{w} = T^{-\top} w$$

gives

$$\tilde{w}^\top \tilde{z} = w^\top z.$$

For this readout class, the recoding is behavior-preserving.

This motivates an important distinction:

coordinate identity $\neq$ functional equivalence.

13.5 Exact recoding witness

Take

$$T = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix},$$

$$z = (2, 3),$$

and

$$w = (4, -1).$$

The original readout is

$$w^\top z = 5.$$

The transformed representation is

$$\tilde{z} = Tz = (5, 3).$$

The transformed readout is

$$\tilde{w} = T^{-\top} w = (4, -5).$$

Then

$$\tilde{w}^\top \tilde{z} = 5.$$

Nothing in this example says the two coordinate systems are semantically identical for every possible downstream task.

It establishes one exact equivalence relation for a declared readout class.

13.6 Invariance

Suppose a group G acts on inputs.

A representation is invariant when

$$\boxed{r(g \cdot x) = r(x)}$$

for the transformations under consideration.

Invariance intentionally removes distinctions.

That can be useful.

If image classification should not change under a small translation, a translation-invariant feature can reduce unnecessary variability.

But invariance has a cost.

If the downstream task requires the transformed quantity, the information has been erased.

A position-invariant representation is poor for a task that must report position.

13.7 Equivariance

Equivariance preserves transformation structure rather than erasing it.

Let

$$\rho(g)$$

be the action of group element g in representation space.

Then equivariance means

$$\boxed{r(g \cdot x) = \rho(g)r(x).}$$

The input transforms.

The representation transforms predictably.

Group-equivariant neural architectures provide concrete examples of this principle (Cohen and Welling 2016).

13.8 Reflection witness

Let

$$G = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$$

reflect the first coordinate.

For

$$x = (2, 3),$$

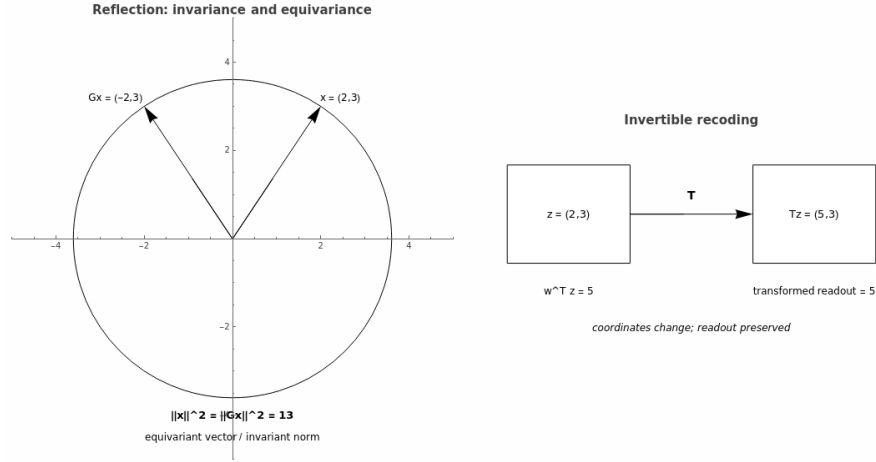


Figure 13.1: A reflection example showing an equivariant vector and invariant norm beside an invertible coordinate recoding that preserves a linear readout.

$$Gx = (-2, 3).$$

The identity representation

$$r(x) = x$$

is equivariant:

$$r(Gx) = Gr(x).$$

The scalar representation

$$r_{\text{inv}}(x) = \|x\|_2^2$$

is invariant:

$$r_{\text{inv}}(Gx) = r_{\text{inv}}(x) = 13.$$

These are different design choices.

One preserves the transformation.

The other removes it.

13.9 Invariance and equivariance are not synonyms

The difference matters.

Invariance says:

the representation should not change.

Equivariance says:

the representation should change in a predictable corresponding way.

A system that confuses these can destroy useful structure.

The right choice depends on the task.

13.10 Geometry enters representation

The Geometry chapter established that admissible states can live on constrained spaces.

Representation learning inherits that fact.

Examples include:

- unit-normalized embeddings;
- orthogonal frames;
- quotient representations;
- spherical positions;
- subspace-valued features.

A representation is therefore not always best modeled as an unconstrained point in $\mathbb{R}^d$.

Its geometry can carry semantics.

This is the direct handoff from the Geometry keystone.

13.11 Information enters representation

The previous chapter supplied probability and information structure.

Representation learning often asks whether an encoding preserves or discards statistically relevant information.

But an information measure does not completely determine representation quality.

Two encodings can have the same mutual information with a target and still differ in:

- geometry;
- robustness;
- linear accessibility;
- sparsity;
- causal role;
- compositional compatibility.

Information is one axis.

Representation structure is richer.

13.12 Identifiability

Suppose latent variables z generate observations through a decoder

$$x = f(z).$$

Now choose an invertible transformation T and define

$$\tilde{z} = Tz.$$

The decoder can be rewritten as

$$\tilde{f}(\tilde{z}) = f(T^{-1}\tilde{z}).$$

The same observations can therefore be explained through different latent coordinates.

Without additional assumptions, there may be no unique latent representation.

This is an identifiability problem.

13.13 Disentanglement needs assumptions

A common hope is that unsupervised learning will recover independent or semantically meaningful generative factors automatically.

That hope requires care.

Locatello et al. show that unsupervised disentanglement is not identifiable without inductive biases under the setting they analyze (Locatello et al. 2019).

The Atlas uses that result as a boundary.

It does not conclude that disentanglement is useless.

It concludes that claims of uniquely recovered semantic factors require assumptions, supervision, or other structure.

13.14 Sparse representations

A representation is sparse when relatively few coefficients are active.

One classical model writes

$$x \approx D\alpha,$$

where D is a dictionary and α is sparse.

Olshausen and Field gave a historically influential example in which sparse coding of natural images produced localized oriented receptive-field-like structures (Olshausen and Field 1996).

This is evidence for one representation regime.

It is not proof that neural representations are universally sparse.

13.15 Dictionaries and features

A dictionary representation introduces another distinction.

The coordinates α_i are meaningful only relative to dictionary elements d_i .

Changing the dictionary changes the coordinate interpretation.

The Atlas will later use this observation when discussing:

- feature dictionaries;
- sparse autoencoders;
- superposition;
- reconstruction;
- transferable residual structure.

Again, coordinates and represented structure are not identical objects.

13.16 Superposition

If the number of potentially useful features exceeds the number of representational dimensions, a system may encode multiple features non-orthogonally.

Toy Models of Superposition studies this phenomenon in deliberately simplified systems and provides geometric and empirical toy-model evidence (Elhage et al. 2022).

The source is useful.

Its scope is narrow.

The Atlas does not infer:

all frontier neural networks store features exactly according to the toy model.

The chapter uses superposition as a research concept, not a universal established law.

13.17 Representation equivalence

What does it mean for two representations to be “the same”?

There is no single answer.

Possible equivalence relations include:

- exact coordinate equality;
- invertible linear recoding;
- invertible nonlinear recoding;
- equality up to group action;
- equality of pairwise distances;
- equality of downstream predictions;
- equality of recoverable sufficient statistics.

Different scientific questions require different equivalence relations.

The Atlas therefore treats “representation equivalence” as a declaration, not a default.

13.18 Quotient thinking

If multiple coordinates represent the same object, the mathematically natural object may be an equivalence class.

That leads toward quotient spaces.

For example, the Geometry chapter distinguished orthonormal frames from the subspaces they span.

Different frames can represent the same subspace.

The later Quotient Representations chapter generalizes this lesson.

13.19 Representation collapse

A representation can discard too much.

If

$$r(x) = c$$

for every input, the representation is perfectly invariant to everything.

It is also useless for tasks that need to distinguish inputs.

This is the simplest counterexample to the idea that more invariance is always better.

The useful question is:

invariant to what, while preserving what?

13.20 Identical task performance does not imply identical mechanism

Two representations can support the same benchmark score.

That does not mean they encode the same internal structure.

One may use:

- sparse features;
- distributed features;
- different symmetries;
- different causal pathways;
- different robustness margins.

Behavioral equivalence on one task is therefore a weak equivalence relation.

Mechanistic claims require stronger evidence.

13.21 The Residual question

The Atlas later asks a more ambitious question:

What is the minimal transferable structure that survives relevant representational changes?

That programme uses the provisional term **Residual**.

This chapter does not answer the question.

It prepares the mathematics required to ask it correctly.

If representations can differ under recoding while preserving capability, then the transferable object may not be any one coordinate system.

13.22 Five failure modes

13.22.1 Coordinates mistaken for meaning

A basis component is treated as a semantic primitive without an identifiability argument.

13.22.2 Invariance mistaken for universal virtue

Task-relevant variation is erased.

13.22.3 Equivariance claimed without group action

The transformation law is not specified.

13.22.4 Disentanglement claimed without assumptions

Non-identifiability is ignored.

13.22.5 Toy superposition promoted to universal mechanism

A bounded research model is treated as established empirical prevalence.

13.23 Atlas connections

Normalized representations.

Once geometry matters, unit-norm and hyperspherical representations become natural special cases.

Quotient representations.

Equivalence under transformations suggests quotient structure.

Attention and position.

Representations are acted on by state-dependent and position-dependent operators.

Memory.

External stores may preserve semantic objects under different internal encodings.

Diagnostics.

Interpretability must distinguish coordinate features from invariant or causally relevant structure.

Residual programme.

Transfer under representation change becomes a first-class question.

13.24 Closing view

A representation is not just where a vector lives.

It is a contract between an input, an encoding, a transformation law, and a downstream use.

Its coordinates can change while capability survives.

Its invariances can help or destroy.

Its latent factors may not be identifiable.

Its features may be sparse, distributed, or superposed.

The Atlas therefore asks a stronger question than:

What is the representation?

It asks:

Which properties of the representation survive the transformations
that should not matter, and which distinctions remain available to
the computations that do?

That question opens the path to normalized geometry, quotient spaces, and the search for transferable residual structure.

13.25 References used in this chapter

- (Bengio et al. 2013)
- (Cohen and Welling 2016)
- (Locatello et al. 2019)
- (Olshausen and Field 1996)
- (Elhage et al. 2022)

Chapter 14

Normalized and Hyperspherical Representations

Epistemic status: Established Theory + Atlas Derivation + Source-Scoped Research Evidence

Specification: manuscript/specifications/ATLAS-CH-NORMREP-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-NORMREP-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-NORMREP-001.md

Source lock: sources/source-locks/ATLAS-CH-NORMREP-001.yaml

14.1 Normalization is a modeling decision

A vector

$$x \in \mathbb{R}^d$$

contains at least two kinds of Euclidean information:

- direction;
- norm.

The normalization map

$$N(x) = \frac{x}{\|x\|_2}$$

keeps the first and removes the second.

This is often useful.

It is never neutral.

If two vectors lie on the same positive ray,

$$x_2 = cx_1, \quad c > 0,$$

then

$$N(x_2) = N(x_1).$$

After normalization, positive radial scale has become an invariance.

The central question of this chapter is therefore not:

Why is normalization good?

It is:

What mathematical structure do we create when we deliberately remove radial freedom?

14.2 Direction without volume

The useful allegory is an arrow whose length has been erased.

Before normalization, the arrow has:

- a direction;
- a magnitude.

After normalization, only direction remains.

The structural correspondence is:

- direction $\leftrightarrow$ angular information;
- arrow length $\leftrightarrow$ radial information;
- tangent step $\leftrightarrow$ change of direction;
- renormalization $\leftrightarrow$ return to the sphere.

The limit is essential.

Length is not noise by definition.

A task may depend on:

- magnitude;
- confidence encoded in norm;
- energy;

- count;
- scale;
- uncertainty.

Normalization should therefore be understood as a declared invariance, not an automatic improvement.

14.3 The unit hypersphere

For unit vectors,

$$u \in S^{d-1} = \{u \in \mathbb{R}^d : \|u\|_2 = 1\}.$$

The Geometry chapter established the tangent space

$$T_u S^{d-1} = \{\xi : u^\top \xi = 0\}.$$

Once representations are constrained to the sphere, legal first-order motion is tangential.

This changes the meaning of an update.

An unconstrained Euclidean step can point partly:

- along the sphere;
- away from the sphere.

Only the first component changes direction to first order.

14.4 The normalization map

For

$$x \neq 0,$$

define

$$u = N(x) = \frac{x}{\|x\|_2}.$$

Let

$$r = \|x\|_2.$$

Then

$$N(x) = r^{-1}x.$$

Differentiating gives

$$J_N(x) = \frac{1}{\|x\|_2} (I - uu^\top).$$

The matrix

$$I - uu^\top$$

is the Euclidean projector onto the tangent space at u .

Normalization therefore has a clean local interpretation:

project away the radial differential component, then scale the tangent component by $1/\|x\|_2$.

14.5 Radial and tangential decomposition

Any perturbation

$$\delta \in \mathbb{R}^d$$

can be decomposed as

$$\delta = (uu^\top)\delta + (I - uu^\top)\delta.$$

The first term is radial.

The second is tangential.

Applying the normalization Jacobian gives

$$J_N(x)\delta = \frac{1}{\|x\|_2} (I - uu^\top)\delta.$$

The radial term vanishes.

This is not metaphorical.

It is the differential of the normalization map.

14.6 Exact witness at $(3, 4)$

Take

$$x = (3, 4).$$

Then

$$\|x\|_2 = 5,$$

and

$$u = (3/5, 4/5).$$

The normalization Jacobian is

$$J_N(x) = \begin{pmatrix} 16/125 & -12/125 \\ -12/125 & 9/125 \end{pmatrix}.$$

The exact computational witness verifies

$$J_N(x)x = 0.$$

The radial direction disappears to first order.

Now take

$$t = (-4, 3).$$

Since

$$x^\top t = 0,$$

this is tangent to the radius through x .

The witness verifies

$$J_N(x)t = (-4/5, 3/5).$$

The direction survives.

Its scale changes by $1/5$.

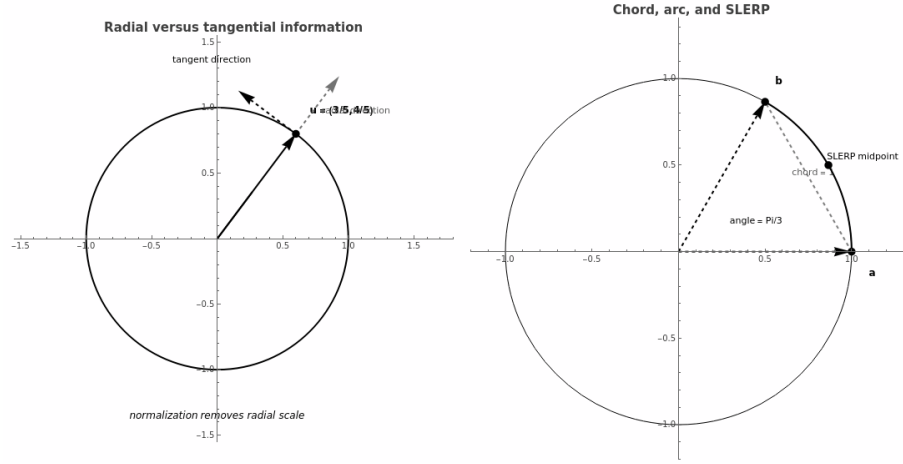


Figure 14.1: A unit-circle figure showing the normalized direction $(3/5, 4/5)$, a radial direction removed by normalization, an orthogonal tangent direction, and a second panel comparing the chord, great-circle arc, and SLERP midpoint for two unit vectors separated by π over three.

14.7 A singularity at zero

Normalization is undefined at

$$x = 0.$$

This is not a minor implementation detail.

The map

$$x \mapsto \frac{x}{\|x\|_2}$$

has no direction to preserve at the origin.

Practical systems may:

- add an epsilon;
- avoid zero states by construction;
- use guarded normalization;
- switch to another chart or convention.

Each choice changes the exact mathematical object.

The Atlas will therefore distinguish idealized normalization from epsilon-regularized implementations where the difference matters.

14.8 Norm erasure is information erasure

Take

$$x_1 = (1, 0), \quad x_2 = (2, 0).$$

Then

$$N(x_1) = N(x_2) = (1, 0).$$

Suppose the target is

$$y = \|x\|_2.$$

Before normalization,

$$y_1 = 1, \quad y_2 = 2.$$

After normalization, the two states are identical.

No downstream function of the normalized representation alone can recover which norm was present.

This is the smallest counterexample to the idea that norm is always redundant.

14.9 Angular similarity

For unit vectors u, v ,

$$u^\top v = \cos \theta,$$

where θ is the angle between them.

Thus cosine similarity reduces to the dot product.

This is one reason normalized representations are attractive:

scale no longer contaminates angular comparison.

But the statement should be read literally.

Normalization makes the comparison scale-invariant because scale has been removed.

It does not prove that scale was irrelevant to the original task.

14.10 Chord distance and angle

For unit vectors,

$$\begin{aligned}\|u - v\|_2^2 &= (u - v)^\top (u - v) \\ &= \|u\|_2^2 + \|v\|_2^2 - 2u^\top v \\ &= 2 - 2\cos\theta.\end{aligned}$$

Therefore

$$\boxed{\|u - v\|_2^2 = 2 - 2\cos\theta.}$$

Chord distance and angular distance are monotonically related for

$$0 \leq \theta \leq \pi.$$

They remain different metrics.

The Geometry chapter's geodesic distance on the unit sphere is

$$d_{\text{geo}}(u, v) = \theta.$$

The ambient chord distance is

$$d_{\text{chord}}(u, v) = \sqrt{2 - 2\cos\theta}.$$

14.11 Sixty-degree witness

Take

$$a = (1, 0),$$

and

$$b = \left(\frac{1}{2}, \frac{\sqrt{3}}{2}\right).$$

Then

$$a^\top b = \frac{1}{2},$$

so

$$\theta = \frac{\pi}{3}.$$

The chord length is

$$\|a - b\|_2 = 1.$$

The geodesic distance is

$$\frac{\pi}{3}.$$

One pair of unit vectors therefore carries at least two natural distances.

Which one matters depends on the problem.

14.12 SLERP

Let a, b be unit vectors with angle

$$\theta = \arccos(a^\top b).$$

For $0 < \theta < \pi$, spherical linear interpolation is

$$\text{SLERP}(a, b; t) = \frac{\sin((1-t)\theta)}{\sin \theta} a + \frac{\sin(t\theta)}{\sin \theta} b.$$

For coincident endpoints $a = b$, the displayed quotient is $0/0$, so define $\text{SLERP}(a, a; t) = a$ by continuous extension. At antipodal endpoints $a = -b$, the shortest interpolation path is not unique without an additional directional choice.

For the sixty-degree witness at

$$t = \frac{1}{2},$$

the exact midpoint is

$$\left(\frac{\sqrt{3}}{2}, \frac{1}{2} \right).$$

It remains unit norm.

It also lies halfway along the great-circle arc.

Straight interpolation in ambient coordinates does not generally do that.

14.13 Retraction

The Geometry chapter introduced the normalized sphere retraction

$$R_u(\xi) = \frac{u + \xi}{\|u + \xi\|_2},$$

for tangent

$$u^\top \xi = 0.$$

This provides a simple update pattern:

1. compute a tangent direction;
2. take an ambient step;
3. renormalize.

The result stays on the sphere.

A retraction approximates the manifold exponential map locally.

It is not generally identical to the exponential map.

14.14 Angular learning rate

Once a state lives on a sphere, raw Euclidean step norm is no longer the only natural update magnitude.

A more intrinsic quantity is angular displacement.

If unit states u and u' satisfy

$$u^\top u' = \cos \theta,$$

then

$$\theta$$

directly measures rotational change.

This makes “learning rate” interpretable in angular units when the update rule is explicitly designed that way.

The public GCL MODULUS Hyperball module implements this kind of machinery:

- optional tangent projection;
- target angular/update scaling;
- normalization-based retraction;
- norm control.

Its source explicitly describes the module as designed for “nGPT / MODULUS-style training.”

That is a real public implementation connection.

It is not evidence of a separate public GCL nGPT repository.

14.15 Hyperspherical contrastive representations

Normalized features also appear outside Transformer architectures.

Wang and Isola analyze contrastive representations in terms of:

- alignment of positive pairs;
- uniformity of normalized features on the hypersphere (Wang and Isola 2020).

This provides a peer-reviewed example in which hyperspherical geometry is not decorative.

The distribution of normalized features on the sphere becomes part of the analysis.

The result is still setting-specific.

It does not establish that every useful representation should be uniformly distributed on a hypersphere.

14.16 nGPT

The nGPT paper proposes a normalized Transformer with representation learning on the hypersphere (Loshchilov et al. 2024).

The paper’s central architectural move is to constrain major representation and parameter vectors to unit-norm geometry and interpret layerwise changes as movement on the hypersphere.

A later preprint develops a practical training recipe and reports scaling experiments on larger hybrid architectures (Loshchilov and Ginsburg 2026).

These are important examples because they make the geometry operational.

The Atlas keeps three levels separate:

1. sphere geometry is established mathematics;
2. nGPT is a specific research architecture and empirical programme;
3. broader claims that normalized Transformers are universally superior remain unsupported.

14.17 Geometry does not choose the task for us

Suppose two hidden states differ only in norm.

On the unit sphere they are identified.

This can be desirable when the system should care only about direction.

It can be harmful when norm encodes:

- evidence strength;
- uncertainty;
- frequency;
- value magnitude;
- resource level.

The right representation geometry therefore depends on what distinctions the downstream task should preserve.

14.18 Normalization can relocate information

Removing norm from one representation does not necessarily remove magnitude from the whole system.

A model may reintroduce scale through:

- separate scalar channels;
- learned temperatures;
- biases;
- logits;
- gating coefficients;
- residual amplitudes;
- external metadata.

This leads to a more useful question:

Where should radial information live?

A normalized architecture may be best understood not as destroying all scale, but as assigning scale to explicit channels rather than letting it remain entangled with direction.

That is an architectural hypothesis.

It must be tested system by system.

14.19 Collapse on the sphere

Unit norm does not prevent representational collapse.

Every input can map to the same unit vector.

Then

$$\|r(x)\|_2 = 1$$

for all x , yet the representation preserves no input distinctions.

Normalization controls radius.

It does not guarantee:

- diversity;
- separability;
- identifiability;
- useful semantics.

This is why hyperspherical representation objectives often care about angular distribution as well as norm.

14.20 Antipodal ambiguity

SLERP has a special case when

$$a = -b.$$

Then

$$\theta = \pi,$$

and the shortest interpolation path is not unique. On S^1 there are two shortest semicircular arcs; on S^{d-1} for $d \geq 3$ there are infinitely many great-circle semicircles joining the endpoints.

This is another reminder that normalization simplifies one degree of freedom without making all geometry trivial.

14.21 Five failure modes

14.21.1 Radial information assumed irrelevant

Normalization removes a quantity the task needed.

14.21.2 Unit norm mistaken for non-collapse

All states can collapse to one point on the sphere.

14.21.3 Cosine similarity mistaken for complete similarity

Angular proximity can ignore task-relevant scale or other structure.

14.21.4 Retraction mistaken for exponential map

The normalized update is a local manifold-compatible device, not exact geodesic flow in general.

14.21.5 nGPT evidence overgeneralized

Paper-scoped empirical results are promoted into a universal architectural law.

14.22 Atlas connections

Representation.

Normalization declares positive radial scaling an invariance.

Geometry.

The sphere supplies tangent spaces, geodesics, retractions, and SLERP.

Optimization.

Updates can be measured by angular displacement and projected tangentially.

Attention.

Normalized queries, keys, values, or parameter vectors alter the scale/angular decomposition of attention computations.

Position.

Spherical geometry provides a natural substrate for directional or harmonic positional constructions.

Quotient thinking.

Removing radial scale can be understood as identifying states along positive rays, though the precise quotient and sphere cross-section should not be conflated automatically.

14.23 Closing view

Normalization changes the question a representation can answer.

Before normalization, a vector can say:

where am I pointing, and how large am I?

After normalization, it can say only:

where am I pointing?

That trade is sometimes exactly what we want.

It exposes clean angular geometry.

It makes tangent motion and spherical interpolation natural.

It can stabilize or clarify specific architectures.

But it is still a trade.

The correct mathematical stance is not to celebrate norm removal.

It is to account for what has been removed, what structure has been gained, and where the missing information must go if the system still needs it.

14.24 References used in this chapter

- (Lee 2018)
- (Absil et al. 2008)
- (Wang and Isola 2020)
- (Loshchilov et al. 2024)
- (Loshchilov and Ginsburg 2026)

See ‘sources/source-locks/ATLAS-CH-NORMREP-001.yaml’ for exact source roles, the public GCL MODULUS implementation-context object, and claim scope.

Chapter 15

Equivalence and Quotient Geometry

Epistemic status: Established Theory + Atlas Derivation + Open Research Question

Specification: [manuscript/specifications/ATLAS-CH-QUOTIENT-001.md](#)

Derivation packet: [mathematics/derivations/ATLAS-CH-QUOTIENT-001-DERIVATIONS.md](#)

Computational witness: [mathematics/computational-witnesses/ATLAS-CW-QUOTIENT-001.md](#)

Source lock: [sources/source-locks/ATLAS-CH-QUOTIENT-001.yaml](#)

15.1 Different parameters can mean the same thing

Suppose two neural networks have different parameter arrays.

Are they different models?

At the level of stored bytes, yes.

At the level of the represented function, perhaps not.

A hidden-unit permutation can reorder internal coordinates without changing the input-output map.

A positive rescaling in a ReLU network can change incoming and outgoing weights while preserving the function exactly.

A basis change can alter a frame without changing the subspace it spans.

These are not numerical coincidences.

They are redundancies in representation.

Quotient geometry asks us to stop treating redundant representatives as distinct objects when the scientific question concerns the underlying equivalence class.

15.2 Many addresses, one place

Imagine one physical location with several valid addresses.

Different naming conventions can point to the same place.

The structural correspondence is:

- address $\leftrightarrow$ parameter representative;
- place $\leftrightarrow$ represented function or equivalence class;
- address conversion $\leftrightarrow$ symmetry action;
- quotient map $\leftrightarrow$ forgetting redundant labels.

The analogy is useful because it separates identity of coordinates from identity of the object.

It also has a strict limit.

Neural parameter spaces can contain:

- fixed points;
- singular strata;
- hidden symmetries;
- changing stabilizers;
- regions where a smooth quotient description fails.

The quotient is not automatically one smooth map of a tidy city.

15.3 Equivalence relations

A relation

$$\sim$$

on a set X is an equivalence relation if it is:

- reflexive;
- symmetric;
- transitive.

The equivalence class of x is

$$[x] = \{y \in X : y \sim x\}.$$

The quotient set is

$$X/\sim = \{[x] : x \in X\}.$$

The quotient keeps the distinctions that survive the chosen equivalence relation and removes the distinctions declared redundant.

Everything depends on that declaration.

Choose an equivalence relation that is too coarse and functionally different objects can be collapsed together.

15.4 Group actions and orbits

Let a group G act on X .

The orbit of x is

$$G \cdot x = \{g \cdot x : g \in G\}.$$

A natural equivalence relation is

$$x \sim y \iff y = g \cdot x \text{ for some } g \in G.$$

Then equivalence classes are group orbits.

This construction is central to symmetry reduction.

It also immediately raises a geometric question:

Does the orbit space inherit a well-behaved smooth structure?

Sometimes yes.

Sometimes only on a regular subset.

Sometimes singularities remain.

15.5 Grassmann: the canonical example

The Geometry chapter distinguished:

$$\text{St}(n, p),$$

the space of ordered orthonormal p -frames, from

$$\text{Gr}(n, p),$$

the space of p -dimensional subspaces.

If

$$X \in \text{St}(n, p),$$

and

$$Q \in O(p),$$

then

$$XQ$$

is another orthonormal frame spanning the same subspace.

Therefore

$$\boxed{\text{Gr}(n, p) \cong \text{St}(n, p)/O(p).}$$

The quotient removes basis choice.

The subspace remains.

This is not merely philosophical.

It changes which directions are treated as genuine motion.

15.6 Parameter equality versus functional equality

Let

$$f_\theta$$

denote the function represented by parameters θ .

Parameter equality is

$$\theta = \theta'.$$

Functional equality is

$$f_\theta(x) = f_{\theta'}(x)$$

for every admissible input x .

The second is weaker.

A parameterization can therefore be many-to-one:

$$\theta \mapsto f_\theta.$$

When that happens, Euclidean geometry in parameter space contains directions that may change coordinates without changing the represented function.

This is the opening for quotient analysis.

15.7 Hidden-unit permutation symmetry

Consider

$$f(x) = W_2 \sigma(W_1 x),$$

with elementwise activation σ .

Let P permute hidden units.

Define

$$W'_1 = P W_1,$$

and

$$W'_2 = W_2 P^{-1}.$$

Because elementwise activation commutes with permutation,

$$\sigma(Pz) = P\sigma(z).$$

Therefore

$$\begin{aligned} f'(x) &= W'_2 P^{-1} \sigma(P W'_1 x) \\ &= W_2 P^{-1} P \sigma(W_1 x) \\ &= f(x). \end{aligned}$$

The hidden units were relabeled.

The function did not change.

15.8 Exact two-unit example

Take scalar input and

$$W_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 3 & 4 \end{pmatrix}.$$

With ReLU activation,

$$f(x) = 3 \operatorname{ReLU}(x) + 4 \operatorname{ReLU}(2x).$$

For positive x ,

$$f(x) = 11x.$$

For nonpositive x ,

$$f(x) = 0.$$

Swap the hidden units:

$$W'_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad W'_2 = \begin{pmatrix} 4 & 3 \end{pmatrix}.$$

The parameter arrays change.

The function remains

$$f(x) = 11 \max(0, x).$$

15.9 Positive-rescaling symmetry

ReLU is positively homogeneous:

$$\operatorname{ReLU}(cz) = c \operatorname{ReLU}(z), \quad c > 0.$$

For positive diagonal D ,

$$\operatorname{ReLU}(Dz) = D \operatorname{ReLU}(z).$$

Define

$$W_1'' = DW_1,$$

and

$$W_2'' = W_2 D^{-1}.$$

Then

$$f''(x) = f(x).$$

For

$$D = \text{diag}(2, 1/3),$$

the exact witness becomes

$$W_1'' = \begin{pmatrix} 2 \\ 2/3 \end{pmatrix},$$

$$W_2'' = \begin{pmatrix} 3/2 & 12 \end{pmatrix}.$$

These parameters look very different.

They still compute the same function.

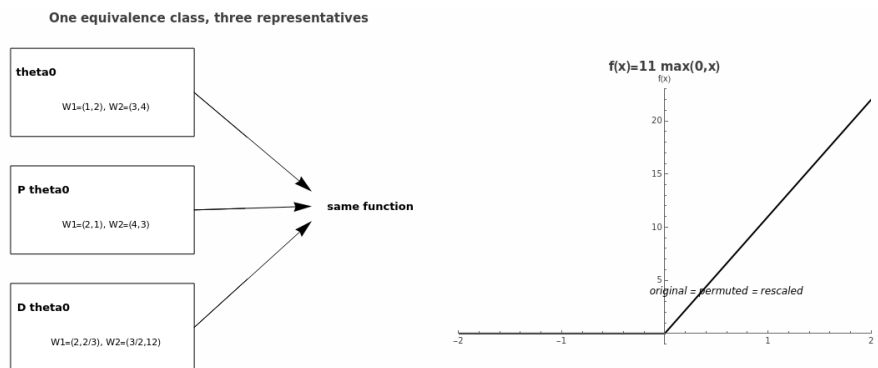


Figure 15.1: Three distinct parameter representatives, original, hidden-unit permuted, and positively rescaled, all mapping to the exact same ReLU function $f(x) = 11 \max(0, x)$.

15.10 Exact replay

The computational witness evaluates all three parameterizations on

$$\{-2, -3/2, -1, -1/2, 0, 1/2, 1, 3/2, 2\}.$$

Every representative gives

$$\{0, 0, 0, 0, 0, 11/2, 11, 33/2, 22\}.$$

That finite replay is a check.

The algebra above is the reason equality holds for every real input.

This distinction mirrors the Atlas evidence doctrine:

computation and proof can support one another without becoming the same object.

15.11 Why quotienting is tempting

Suppose an optimizer moves through parameter space.

If many parameter directions correspond only to symmetry motion, then ambient Euclidean distance can overstate meaningful change.

Two far-separated parameter vectors may compute the same function.

This suggests a cleaner object:

$$[\theta] \in \Theta/G.$$

Instead of asking how far parameters moved, ask how far the equivalence class moved.

This can remove redundancy from the description.

It does not automatically remove difficulty from the optimization problem.

15.12 Vertical and horizontal directions

On a regular quotient manifold, tangent directions can be separated conceptually into:

- **vertical directions**, tangent to the symmetry orbit;
- **horizontal directions**, transverse directions that represent quotient motion.

The terminology is standard in quotient geometry (Absil et al. 2008).

A vertical move changes the representative.

A horizontal move changes the equivalence class.

This distinction is especially useful when the scientific object is the represented function rather than the raw parameter vector.

15.13 Local null directions

Suppose

$$F(\theta)$$

is constant along a smooth symmetry orbit

$$\theta(s).$$

Then

$$F(\theta(s)) = F(\theta(0)).$$

Differentiating at zero gives

$$J_F(\theta)\dot{\theta}(0) = 0.$$

The orbit tangent lies in the local null space of the function map.

If a loss depends only on $F(\theta)$, its directional derivative also vanishes along the orbit.

An actual Hessian null vector requires a stronger statement than “one path has constant loss.”

Assume a smooth one-parameter group action g_s is an exact symmetry of the loss in a neighborhood:

$$L(g_s\theta) = L(\theta).$$

If θ_* is a smooth critical point, symmetry maps it to other critical points.

For the orbit tangent

$$v = \left. \frac{d}{ds} g_s \theta_* \right|_{s=0},$$

differentiating

$$\nabla L(g_s \theta_\star) = 0$$

gives

$$\boxed{H_L(\theta_\star)v = 0.}$$

A minimal exact example is

$$L(a, b) = \frac{1}{2}(ab - y)^2.$$

Positive rescaling

$$(a, b) \mapsto (a/c, cb)$$

preserves the model product.

On the critical manifold $y = ab$,

$$H = \begin{pmatrix} b^2 & ab \\ ab & a^2 \end{pmatrix},$$

and the scaling-orbit tangent

$$v = (-a, b)$$

satisfies

$$Hv = 0.$$

This is a precise source of Hessian degeneracy.

It does not mean every flat direction is a symmetry direction.

15.14 Hidden symmetries

Permutation and positive scaling are not necessarily the whole story.

Grigsby, Lindsey, and Rolnick study hidden symmetries in ReLU networks and show that the redundancy structure can be richer and parameter-dependent (Grigsby et al. 2023).

This matters for quotient reasoning.

If the symmetry group changes across parameter space, a single globally regular manifold model may fail.

The orbit space can have different local structure in different regions.

15.15 Symmetry can affect learning dynamics

Symmetry is not only a bookkeeping problem.

It can constrain optimization and interact with regularization and noise.

Ziyin studies consequences of loss symmetries for learned parameter structure under explicit assumptions (Ziyin 2024).

The Atlas uses this literature to support a limited point:

symmetry can influence learning dynamics.

It does not infer that one universal quotient geometry determines all neural training.

15.16 Quotienting removes declared redundancy

Suppose the group G captures an exact functional symmetry.

Then moving within one orbit does not change the function.

Passing to

$$\Theta/G$$

removes those orbit directions from the represented object.

This is a genuine simplification of description.

It can reduce:

- redundant coordinates;
- gauge-like degrees of freedom;
- ambiguity in comparing models.

That is already useful.

A stronger claim would be:

the quotient objective is easier to optimize.

That does not follow automatically.

15.17 Redundancy removed does not mean convexity

A quotient can still contain:

- multiple functionally distinct minima;
- saddles;
- barriers;
- poor conditioning;
- bifurcations;
- singular regions.

Therefore

symmetry removed $\nRightarrow$ convex landscape.

The quotient can be the more intrinsic object while remaining mathematically difficult.

15.18 Singular quotients

A group action is easiest to quotient when it acts freely and regularly.

Neural networks can violate that simplicity.

Examples include:

- a hidden unit with zero incoming or outgoing weights;
- two hidden units that become identical;
- parameter points with extra stabilizer symmetries;
- architecture-dependent hidden symmetries.

At such points, orbit dimension can change.

The quotient can develop singular structure.

This is why the chapter refuses to speak of “the neural-network quotient manifold” without qualifications.

15.19 Function space is an even stronger quotient

One possible equivalence relation is

$$\theta \sim \theta' \iff f_\theta = f_{\theta'}.$$

This identifies every parameterization of the same function.

Conceptually, that is attractive.

Practically, it may be difficult to characterize.

The exact function-equivalence relation can include more than the obvious architecture symmetries.

The quotient can also lose useful information about optimization trajectories, regularization, or implementation cost.

The most intrinsic representation is not automatically the most useful operational representation.

15.20 Quotienting by the wrong relation

Suppose we declare two models equivalent because they agree on a finite training set.

That relation can merge functions that differ elsewhere.

If the intended object is global predictor behavior, the equivalence is too coarse.

A quotient is only as meaningful as its equivalence relation.

This is the quotient analogue of the boundary-contract question:

sufficient for what?

15.21 Gauge language

Physics often uses the word **gauge** for representational freedom that does not change observable quantities.

That language can be useful in machine learning when parameter transformations leave the represented function invariant.

But it should be used carefully.

Not every neural symmetry has the full mathematical structure of a gauge theory.

The safe statement is:

some neural parameter redundancies are gauge-like in the limited sense that multiple representatives encode the same functional object.

The exact group action should be stated.

15.22 Representation equivalence revisited

The Representation chapter introduced invertible recoding:

$$\tilde{z} = Tz,$$

with compatible downstream transformation.

Quotient thinking generalizes the lesson.

A representation is often meaningful only up to some family of transformations.

Possible equivalences include:

- orthogonal basis change;
- invertible linear recoding;
- permutation;
- positive scaling;
- group action;
- exact functional equivalence.

Different questions require different quotients.

There is no universal “correct quotient” for every analysis.

15.23 The benign-nonconvexity question

The Atlas now reaches a frontier question:

What portion of neural nonconvexity is merely redundancy induced by representation or parameter symmetries, and what portion remains intrinsic after those redundancies are removed?

This is a central motivation for the constructive benign-nonconvexity programme.

The hope is not that quotienting magically makes every problem convex.

The more modest possibility is:

- identify symmetry-induced degeneracy;
- remove it explicitly;
- measure the remaining landscape;
- separate representational difficulty from intrinsic optimization difficulty.

That programme remains open.

15.24 A reconstruction test

Suppose two parameter vectors are declared equivalent.

A useful test is:

Can every downstream property we intend to preserve be reconstructed from the equivalence class?

If not, the quotient discarded too much.

This reconstruction test connects quotient geometry to the Atlas's broader Residual programme:

find the smallest structure that can change representation while preserving the capability of interest.

15.25 Six failure modes

15.25.1 Parameter difference mistaken for functional difference

Symmetry-related representatives are counted as distinct models.

15.25.2 Quotient relation too coarse

Functionally relevant distinctions are erased.

15.25.3 Quotient relation too narrow

Redundant coordinates remain and contaminate geometry.

15.25.4 Regular quotient assumed globally

Singular orbit structure is ignored.

15.25.5 Flat direction assumed to be symmetry

Other sources of degeneracy are overlooked.

15.25.6 Quotienting assumed to solve optimization

Redundancy removal is promoted into an unsupported global simplicity theorem.

15.26 Atlas connections

Geometry.

Grassmann is the canonical quotient example.

Representation.

Functional or task-relative equivalence can survive coordinate change.

Normalized representations.

Normalization removes one radial degree of freedom, but a sphere cross-section and a quotient by positive scaling should be distinguished mathematically.

Optimization.

Symmetry directions can create redundant ambient motion and local degeneracy.

Diagnostics.

Comparing models may require symmetry alignment before interpreting parameter distance.

Residual.

The next chapter asks what transferable structure survives representation and parameter changes.

Benign nonconvexity.

Quotient structure provides a constructive route for separating redundant from intrinsic landscape complexity.

15.27 Closing view

A parameter vector is often an address, not the place itself.

Two different addresses can point to one function.

Quotient geometry gives us a disciplined way to say which distinctions are representational and which remain intrinsic.

That discipline is already valuable.

It prevents parameter distance from being mistaken for functional distance.

It exposes exact symmetry directions.

It clarifies why some flatness is coordinate-dependent.

But the deeper question remains open.

After we remove the redundancy we understand, what difficulty is left?

That remainder is where the Atlas turns next.

15.28 References used in this chapter

- (Lee 2018)
- (Absil et al. 2008)
- (Edelman et al. 1998)
- (Grigsby et al. 2023)
- (Ziyin 2024)

See ‘sources/source-locks/ATLAS-CH-QUOTIENT-001.yaml’ for exact source roles and claim scope.

Chapter 16

The Residual

Epistemic status: Atlas/GCL Research Synthesis + Established Supporting Theory + Exact Toy Witness

Specification: manuscript/specifications/ATLAS-CH-RESIDUAL-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-RESIDUAL-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-RESIDUAL-001.md

Source lock: sources/source-locks/ATLAS-CH-RESIDUAL-001.yaml

16.1 What survives when representation changes?

Suppose two systems implement the same useful capability through very different internal coordinates.

One uses dense vectors.

Another uses sparse features.

A third uses an operator basis.

A fourth has been permuted, rescaled, normalized, compressed, or otherwise reparameterized.

What, if anything, must survive?

The Atlas uses the provisional name **Residual** for the answer we are trying to find:

the smallest transferable structure sufficient to reconstruct a declared capability after admissible changes of representation.

The word is intentionally singular.

The theory is not.

There may be no unique universal Residual.

There may be no finite-dimensional one.

There may be no efficiently computable one.

The object can only be discussed after the capability, transformations, and admissible descriptor class have been declared.

16.2 The thing that cannot be transformed away

A useful GCL phrase is:

The Residual is the thing that cannot be transformed away.

Taken literally, that phrase is too strong.

Many objects can survive a transformation while being irrelevant.

A constant survives almost everything.

What matters is not survival alone.

The proposed object must satisfy at least three demands:

1. it survives the transformations we have declared irrelevant;
2. it still contains enough structure to reconstruct the capability we care about;
3. among admissible such descriptions, it retains no unnecessary distinctions.

So the stronger working phrase is:

The Residual is the least admissible structure that survives the declared representational changes while remaining sufficient to reconstruct the declared capability.

That is the object this chapter formalizes.

16.3 The message that survives translation

The pedagogical allegory is a message translated between languages.

The surface symbols change.

Word order may change.

The alphabet may disappear entirely.

Yet a receiver can still recover what is needed to act.

The structural correspondence is:

- language $\leftrightarrow$ representation;
- translation $\leftrightarrow$ admissible representation change;
- task-relevant meaning $\leftrightarrow$ declared capability;
- compact adequate message $\leftrightarrow$ Residual.

The limit is important.

Human meaning is not a formal mathematical map.

Translation is rarely invertible.

“Shortest message” depends on the coding language and admissible operations.

The allegory teaches one point only:

what survives is meaningful here only because it remains reconstructively sufficient for something declared in advance.

16.4 From representation equivalence to reconstruction

The Representation chapter established that coordinate identity is stronger than functional equivalence.

The Quotient chapter established that multiple parameter representatives can encode one function.

Those results remove a common mistake:

different coordinates $\nRightarrow$ different capability.

The Residual question goes further.

It asks:

Once representation-specific redundancy is removed, what portable structure is still enough to rebuild the capability?

This is not merely a quotient question.

A quotient tells us which distinctions we are willing to ignore.

A Residual must also pass a reconstruction test.

16.5 Declaring the problem

Let

$$X$$

be a representation or system-state space.

Let

$$G$$

be a declared family of admissible transformations acting on X .

Let

$$\mathcal{Q}$$

be a declared family of capability probes, contexts, or interventions.

Let

$$B : X \times \mathcal{Q} \rightarrow \mathcal{Y}$$

be the declared behavior profile.

For each state x , write

$$B_x(q) = B(x, q).$$

Finally, let

$$\mathfrak{D}$$

be a declared class of admissible portable descriptors.

This last object matters more than it first appears.

Without it, the problem can collapse into a tautology.

16.6 Transformation invariance

A descriptor

$$R : X \rightarrow \mathcal{Z}, \quad R \in \mathfrak{D},$$

is invariant to the declared representation changes when

$$\boxed{R(g \cdot x) = R(x)}$$

for every admissible $g \in G$.

Invariance is necessary for the intended notion of transfer.

It is not sufficient.

The constant descriptor

$$R_0(x) = 0$$

is invariant under every transformation.

It usually preserves no useful capability.

16.7 Capability sufficiency

A descriptor R is sufficient for the declared behavior profile when there exists a reconstructor

$$D : \mathcal{Z} \times \mathcal{Q} \rightarrow \mathcal{Y}$$

such that

$$\boxed{B(x, q) = D(R(x), q)}$$

for every declared state/probe pair.

This is capability reconstruction.

It is weaker than full-state reconstruction.

The Residual is allowed to forget anything the declared capability does not need.

That forgetting is not a bug.

It is the point.

16.8 The factorization preorder

Suppose we have two descriptors,

$$R_1 : X \rightarrow Z_1,$$

and

$$R_2 : X \rightarrow Z_2.$$

Write

$$R_1 \preceq R_2$$

when there exists an admissible post-processing map ϕ from a declared class $\mathfrak{M}$, with

$$\phi : Z_2 \rightarrow Z_1$$

such that

$$\boxed{R_1 = \phi \circ R_2.}$$

Interpretation:

R_1 contains no more distinctions than R_2 , because everything in R_1 can be recovered from R_2 .

This is a preorder when the declared post-processing class contains identities and is closed under composition.

If $\mathfrak{M}$ is the class of all set-theoretic maps, the preorder is purely semantic.

If $\mathfrak{M}$ is restricted to efficiently computable, bounded-cost, differentiable, or otherwise admissible maps, the preorder becomes operationally stronger.

The leastness argument below uses unrestricted set-theoretic post-processing. If $\mathfrak{M}$ is restricted, an abstract decoder witnessing sufficiency need not belong to $\mathfrak{M}$; it cannot be used to certify operational leastness without that additional check.

Two different codings can factor through one another.

That is exactly what we want: coordinate choice should not automatically create a new intrinsic object.

16.9 Least, not merely small

Let

$$\mathcal{S}_{B,G,\mathfrak{D}}$$

be the admissible descriptors that are both:

- invariant under G ;

- sufficient for B .

A provisional Residual is a **least element** of this class under $\preceq$.

Thus R must satisfy

$$R \preceq S$$

for every

$$S \in \mathcal{S}_{B,G,\mathfrak{D}}.$$

This is stronger than saying R is “small.”

It is also stronger than saying no obviously redundant field remains.

Every other admissible invariant sufficient descriptor must be able to produce R .

16.10 Why leastness is useful

Suppose R and R' are both least.

Then

$$R = \phi \circ R',$$

and

$$R' = \psi \circ R$$

for some maps ϕ, ψ .

On the realized images,

$$\psi \circ \phi = \text{id},$$

and

$$\phi \circ \psi = \text{id}.$$

So two least Residuals are equivalent up to invertible recoding on what the system actually realizes.

This is the kind of non-uniqueness the Atlas can tolerate.

Different coordinates need not mean different Residual structure.

16.11 The behavior-table trap

There is an immediate problem.

If $\mathfrak{D}$ allows arbitrary descriptors, define

$$R_B(x) = B_x.$$

That is, store the complete behavior profile itself.

Then a decoder can simply evaluate

$$R_B(x)(q).$$

If B_x is invariant under the declared representation changes, R_B is an invariant sufficient descriptor.

And because every sufficient descriptor must be able to reconstruct B_x , the full behavior profile is already least under factorization.

Mathematically, this is fine.

Scientifically, it can be useless.

We wanted transferable mechanism.

We got a behavior table.

16.12 Descriptor admissibility is part of the theorem statement

A nontrivial Residual programme therefore has to declare what counts as an admissible portable descriptor.

Possible restrictions include:

- bounded description length;
- finite-dimensionality;
- efficient computability;
- architectural independence;
- intervention stability;
- bounded reconstruction cost;
- portability across model families;
- composability with a fixed interpreter.

The choice is not housekeeping.

It changes the object.

The Residual is therefore relative to

$$(B, G, \mathfrak{D}, \mathcal{Q}).$$

Suppressing those arguments may be convenient later.

It is dangerous at the beginning.

16.13 Classical sufficiency as a model

Statistics gives a mature example of this kind of reasoning.

A sufficient statistic preserves all sample information needed for inference about a declared parameter under a declared statistical model.

Minimal sufficiency then asks for a statistic that is no more informative than any other sufficient statistic, in the appropriate factorization sense (Lehmann and Casella 1998).

The Residual borrows this pattern:

- declare what must be preserved;
- identify nuisance variation;
- compress without losing what matters;
- compare candidate descriptions by factorization.

The Residual is broader.

Therefore classical sufficiency theorems do not automatically apply.

16.14 Minimal sufficient representations

Achille and Soatto study task-relative representations in terms of:

- sufficiency;
- minimality;
- invariance to nuisance factors (Achille and Soatto 2018).

This is close in spirit to the Residual programme.

It supports a key lesson:

minimality is not meaningful without a declared task and nuisance structure.

The Atlas extends the question beyond one probabilistic representation-learning setup toward transferable computational structure.

That extension is ours.

It is not a theorem imported from the paper.

16.15 Information bottleneck

The information bottleneck programme asks for compressed encodings that preserve information relevant to a declared target (Tishby et al. 2000).

Again, the structural analogy is strong:

compress while preserving what matters.

But the Residual is not defined here as an information-bottleneck optimum.

A Residual may be:

- symbolic;
- algorithmic;
- operator-valued;
- graph-structured;
- programmatic;
- hybrid.

Mutual information is one possible formalization, not the definition.

16.16 A calibration toy model

Let

$$X = \mathbb{R}^2,$$

with state

$$x = (s, n).$$

Interpret:

- s : task-relevant signal;
- n : nuisance.

Let admissible transformations translate only the nuisance coordinate:

$$g_a(s, n) = (s, n + a).$$

Use a singleton probe family for this calibration example and define capability

$$B(s, n) = s.$$

Now define

$$R(s, n) = s.$$

This is deliberately simple.

It is a calibration case for the definitions, not evidence for a frontier-model Residual.

16.17 Invariance of the toy Residual

For any a ,

$$R(g_a(s, n)) = R(s, n + a) = s.$$

Therefore

$$R(g_a x) = R(x).$$

The Residual is constant along vertical nuisance orbits.

16.18 Sufficiency of the toy Residual

Choose decoder

$$D(r) = r.$$

Then

$$D(R(s, n)) = s = B(s, n).$$

The Residual reconstructs the declared capability exactly.

It does not reconstruct n .

It does not need to.

16.19 Leastness in the toy model

Let S be any sufficient descriptor.

Then by sufficiency there exists a decoder D_S such that

$$B = D_S \circ S.$$

But in this toy model,

$$R = B.$$

Therefore

$$R = D_S \circ S.$$

Hence, for unrestricted semantic post-processing,

$$R \preceq S.$$

So $R = s$ is least among the declared sufficient descriptors in the semantic preorder. For an operational preorder, this argument additionally requires $D_S \in \mathfrak{M}$ for every compared sufficient descriptor S .

The proof is almost embarrassingly simple.

That is useful.

It shows exactly what the formal definition does before we ask it to carry frontier complexity.

16.20 Invariance without sufficiency

Now define

$$S_0(s, n) = 0.$$

This descriptor is invariant.

Take

$$x_1 = (2, 0), \quad x_2 = (3, 0).$$

Then

$$S_0(x_1) = S_0(x_2) = 0.$$

But

$$B(x_1) = 2,$$

and

$$B(x_2) = 3.$$

No deterministic decoder from the constant descriptor can produce both capability values.

This is the simplest failure mode:

invariant $\nrightarrow$ sufficient.

16.21 Sufficiency without minimality

Consider the full-state descriptor

$$S_{\text{full}}(s, n) = (s, n).$$

It is sufficient for $B = s$.

A decoder can discard n .

But it stores more than the declared capability needs.

Indeed,

$$R = \pi_1 \circ S_{\text{full}}.$$

Full-state reconstruction is stronger than capability reconstruction.

The Residual programme explicitly does not demand the stronger object unless the capability itself requires it.

16.22 Representation change

Now change coordinates.

Let

$$T = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Define

$$z = Tx.$$

If

$$x = (s, n),$$

then

$$z_1 = s + n,$$

and

$$z_2 = s - n.$$

The original signal is no longer stored in one coordinate.

But it remains reconstructable:

$$s = \frac{z_1 + z_2}{2}.$$

This is the first genuinely Residual-like lesson of the toy model.

The transferable object is not a coordinate slot.

It is reconstructable structure.

16.23 Exact recoding witness

Take

$$x = (2, 5).$$

Then

$$z = Tx = (7, -3).$$

Reconstruct:

$$\frac{7 + (-3)}{2} = 2.$$

The exact computational witness also moves

$$(2, 5)$$

along its nuisance orbit to

$(2, 2),$

while preserving the Residual value 2.

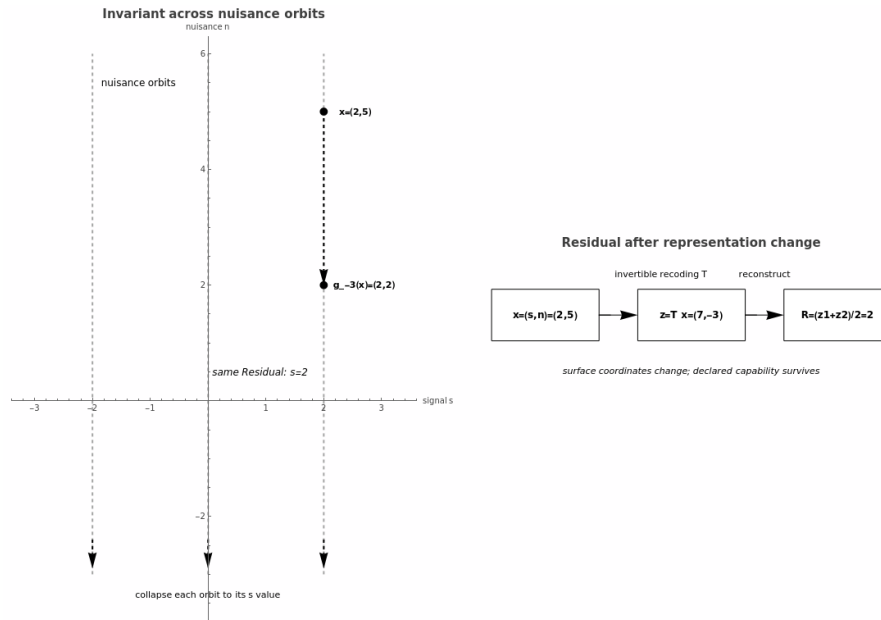


Figure 16.1: Vertical nuisance orbits in signal- nuisance space collapse to their signal coordinate, while an invertible recoding sends $x = (2, 5)$ to $z = (7, -3)$ and the Residual is reconstructed as $(z_1 + z_2) / 2 = 2$.

The surface representation changed.

The declared capability did not.

16.24 Task dependence

Now change the capability.

Suppose

$$B'(s, n) = (s, n).$$

Then

$$R(s, n) = s$$

is no longer sufficient.

The nuisance has stopped being nuisance.

This is why the Residual cannot be declared before the capability is declared.

A structure can be residual for one task and inadequate for another.

16.25 Transformation dependence

The same issue applies to G .

Suppose we enlarge the admissible transformation family to permit changing s .

Then $R = s$ is no longer invariant.

If the transformation family identifies capability-distinct states, the problem itself becomes inconsistent:

no descriptor can be both invariant to that transformation and exactly sufficient for the capability.

This gives a useful compatibility condition:

admissible transformations must preserve the declared capability if
an exact invariant sufficient Residual is to exist.

16.26 Residual as equivalence class

Sometimes the natural Residual may be an equivalence class rather than a chosen coordinate.

If

$$x \sim x'$$

whenever every declared capability probe gives the same result, then one can consider the class

$$[x].$$

That quotient is canonical relative to the behavior relation.

But it can be enormous.

It may be impossible to compute.

It may still be a behavior-table object in disguise.

The research challenge is to find a smaller structural representative that supports reconstruction.

16.27 Residual as operator structure

The transferable object need not be a vector.

Suppose capability depends on a transformation rule rather than on a stored state.

Then what survives may be:

- a linear operator;
- an algebra of operators;
- a transition kernel;
- a control law;
- a program;
- a graph of conditional dependencies.

This is why the Atlas calls the target **computational structure** rather than “the minimal vector.”

16.28 Residual as program

Consider two neural systems that implement the same algorithm with different hidden bases.

A basis-independent program or operator composition may transfer more naturally than any activation vector.

This possibility connects the Residual to the Atlas shift:

vectors $\rightarrow$ operators.

It also raises a harder question:

What is the right language in which minimality should be measured?

Description length depends on representation language.

There is no free universal metric.

16.29 Exact versus approximate Residuals

Frontier systems rarely invite exact equivalence.

A practical Residual may need an error tolerance.

One can imagine requiring

$$d_Y(B(x, q), D(R(x), q)) \leq \varepsilon$$

for all probes in a declared test family, or in expectation under a declared distribution.

Then minimality becomes approximate.

Tradeoffs appear among:

- descriptor size;
- reconstruction error;
- computation cost;
- robustness;
- transfer range.

The exact theory in this chapter is the zero-error baseline.

The approximate theory is a later research problem.

16.30 Reconstruction is stronger than correlation

Suppose a candidate feature correlates strongly with capability.

That is useful evidence.

It is not yet a Residual.

The candidate must support reconstruction under the declared changes of representation.

A feature that predicts capability in one model but disappears under a basis change fails transferability.

A feature that survives transformations but loses task-critical distinctions fails sufficiency.

The Residual standard is deliberately stricter.

16.31 Intervention strengthens the test

A compelling Residual candidate should survive more than passive measurement.

Where possible:

1. isolate the candidate structure;
2. alter non-Residual structure;
3. reconstruct or transplant the candidate;
4. test whether capability returns or remains;
5. repeat across admissible representations.

This turns the question from:

Can we read the capability from this feature?

into:

Is this structure enough to rebuild the capability under the transformations we claim should not matter?

That is closer to the intended programme.

16.32 Finite replay can fool us

Suppose a descriptor reproduces outputs on a finite benchmark.

That does not prove it reconstructs the capability.

It may simply encode the benchmark.

The declared probe family $\mathcal{Q}$ therefore matters.

A serious reconstruction test must include enough variation to distinguish:

- memorized outputs;
- interpolated behavior;
- genuine transferable mechanism.

The Replayable Evidence chapter tells us how to preserve the test.

It does not tell us that the test family was sufficient.

16.33 Minimal curriculum

The Residual idea connects directly to the minimal-curriculum programme.

If later capability can be efficiently reconstructed from a small early-learned computational basis, then that basis is a candidate Residual across training trajectories.

The question becomes:

Which early structures survive retraining, recoding, or architectural change while remaining sufficient to rebuild later capability?

That is a reconstruction problem.

16.34 Minimal reasoning basis

The same pattern appears in mathematical reasoning.

Perhaps many surface reasoning traces can be generated from a smaller set of transferable operations.

Then the target is not a canonical chain of thought.

It is a basis of operations sufficient to reconstruct broad reasoning capability.

Again:

- invariance to surface trace;
- sufficiency for capability;
- leastness within an admissible operation language.

The Residual formalism gives the question a shape.

It does not answer it.

16.35 Model interoperability

Suppose two models use incompatible hidden coordinates.

Directly exchanging activations may fail.

A transferable Residual could act as a lingua franca if both systems can:

- compile their internal state into the Residual;
- reconstruct the required capability from it.

This is one possible route to model interoperability that does not require one universal latent coordinate system.

16.36 Memory

The Residual also changes how we think about memory.

A memory store need not preserve every internal activation that produced a result.

It may preserve only the structure required to reconstruct the capability later.

This suggests a compression principle:

store what future reconstruction needs, not necessarily the historical representation that happened to carry it.

Whether such a sufficient Residual can be identified is the hard part.

16.37 Benign nonconvexity

The Quotient chapter asked which nonconvexity disappears after symmetry reduction.

The Residual asks an adjacent question:

Which computational distinctions survive after all representation-specific redundancy is removed?

If a capability can be reconstructed from a small quotient-stable object, then large regions of parameter variation may be irrelevant to that capability.

This could help separate:

- representational complexity;
- intrinsic computational complexity.

That remains a research programme.

16.38 Governed adaptation

A self-changing system needs to know what it must preserve.

The Residual offers one possible object for that preservation contract.

If a system can modify:

- architecture;
- parameterization;
- memory layout;
- routing;
- tool topology;

while retaining a certified Residual sufficient for declared capabilities, then adaptation may become safer to reason about.

This is an aspiration.

No such universal certificate is established here.

16.39 What would falsify a proposed Residual?

A candidate fails if any of the following occurs.

16.39.1 Invariance failure

An admissible representation change alters the descriptor.

16.39.2 Sufficiency failure

The declared capability cannot be reconstructed.

16.39.3 Leastness failure

A strictly coarser admissible invariant descriptor remains sufficient.

16.39.4 Transfer failure

The descriptor works only in one coordinate system or architecture despite the claimed transfer range.

16.39.5 Intervention failure

Capability disappears when supposedly nonessential structure changes.

16.39.6 Probe failure

The descriptor passes a narrow benchmark but fails on the declared capability family.

These are operational failure modes.

They make the programme falsifiable.

16.40 What the Residual is not

The Residual is not automatically:

- a hidden neuron;
- a feature direction;
- a latent vector;
- a quotient coordinate;
- a sufficient statistic in the classical sense;
- an information-bottleneck optimum;
- a compressed checkpoint;
- a behavior table;
- a universal invariant of intelligence.

Any of these could instantiate the role in a particular setting.

None is the definition.

16.41 Atlas connections

Representation.

The Residual is defined only after representation equivalence and task-relative structure are explicit.

Quotient geometry.

Symmetry reduction removes representational redundancy but does not by itself identify sufficient reconstructive structure.

Information.

Sufficiency and bottleneck ideas provide formal analogies and possible special cases.

Memory.

A Residual could be what a system stores when historical coordinates are disposable.

Interoperability.

A shared Residual could support translation without a shared latent basis.

Minimal curriculum / reasoning basis.

The target becomes a minimal transferable basis from which capability can be reconstructed.

Governed adaptation.

The Residual may eventually become a preservation contract across self-modification.

16.42 Closing view

The Residual programme begins with a refusal.

We refuse to identify capability with the coordinates that happen to implement it today.

We also refuse the opposite mistake: calling anything invariant “the essence.”

The target has to survive.

It has to reconstruct.

And, within a declared admissible language, it has to be no richer than necessary.

In the toy model, the answer is trivial:

$$R(s, n) = s.$$

In frontier systems, we do not know the answer.

That ignorance is the point of naming the problem.

The Residual is not yet a discovered object.

It is a reconstruction test for finding what, after all admissible transformations have done their work, still has to remain.

16.43 References used in this chapter

- (Lehmann and Casella 1998)
- (Achille and Soatto 2018)
- (Tishby et al. 2000)

See ‘sources/source-locks/ATLAS-CH-RESIDUAL-001.yaml’ for exact source roles, audited Atlas dependencies, and claim boundaries.

Chapter 17

Reconstruction and Transfer

Epistemic status: Established Transfer Framework + Atlas Synthesis + Exact Toy Witness

Specification: manuscript/specifications/ATLAS-CH-TRANSFER-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-TRANSFER-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-TRANSFER-001.md

Source lock: sources/source-locks/ATLAS-CH-TRANSFER-001.yaml

17.1 Transfer is not reuse

A model begins on one task.

Some of what it learned is reused somewhere else.

That story is often called transfer learning.

But several different phenomena can hide inside the word **transfer**:

- reusing parameters as initialization;
- freezing a feature extractor;
- fine-tuning a whole model;
- adapting to a new domain;
- adapting to a new task;
- reusing architecture but not weights;
- compiling one representation into another;
- reconstructing a capability through a shared latent object.

These are not the same operation.

The Atlas therefore asks a narrower question before discussing methods:

What structure must remain recoverable for a declared target capability to survive a change of representation, task, or domain?

This chapter connects that question to the Residual.

17.2 Moving a machine through a narrow doorway

A useful allegory is shipping a machine through a doorway.

One option is to push the whole assembled machine through.

Another is to:

1. disassemble it;
2. keep a portable core;
3. carry that core across;
4. rebuild what is needed on the other side.

The structural correspondence is:

- source representation $\leftrightarrow$ original assembly;
- compiler $\leftrightarrow$ disassembly map;
- Residual $\leftrightarrow$ portable core;
- target decoder $\leftrightarrow$ reconstruction procedure;
- bottleneck $\leftrightarrow$ doorway width.

The limit matters.

Real transfer learning also involves:

- finite data;
- optimization;
- distribution shift;
- task mismatch;
- decoder restrictions;
- stochastic training.

Exact deterministic reconstruction is only one clean special case.

17.3 Standard source and target framing

A common transfer-learning formulation distinguishes a **domain** from a **task** (Pan and Yang 2010).

A domain can be written as

$$\mathcal{D} = (\mathcal{X}, P(X)),$$

where $\mathcal{X}$ is an input space and $P(X)$ a distribution.

A task can be written as

$$\mathcal{T} = (\mathcal{Y}, f),$$

where $\mathcal{Y}$ is an output space and f a predictive object.

A transfer setting then relates:

$$(\mathcal{D}_s, \mathcal{T}_s)$$

to

$$(\mathcal{D}_t, \mathcal{T}_t).$$

The source and target may differ in:

- input distribution;
- feature space;
- output space;
- predictive task;
- available labels;
- data volume.

The important point is relational:

transfer is always transfer **from something to something**.

17.4 Transfer needs a target criterion

Suppose we reuse a source model on a target task.

Did transfer succeed?

That question is meaningless without a target criterion.

We need at least:

- a target metric;
- a target data budget;
- an adaptation procedure;
- a baseline.

For a higher-is-better score J , transfer improves the target when

$$J(M_{\text{transfer}}) > J(M_{\text{target-only}}),$$

under a matched comparison.

If

$$J(M_{\text{transfer}}) < J(M_{\text{target-only}}),$$

the result is negative transfer.

The sign depends on the metric convention.

The concept depends on the baseline.

17.5 Negative transfer is not paradoxical

Why can previously learned structure hurt?

Because reusable structure can be mismatched.

Possible causes include:

- source-specific features;
- wrong invariances;
- optimization bias;
- insufficient adaptation;
- domain shift;
- label mismatch;
- representational bottlenecks;
- co-adaptation among features.

Pan and Yang discuss negative transfer as a central issue in transfer learning (Pan and Yang 2010).

The Atlas will use the term operationally, not metaphysically.

A source model does not carry a universal transferable essence merely because it learned something useful once.

17.6 Source accuracy is not transferability

A source model can perform extremely well on its original task while encoding features that transfer poorly.

Yosinski and colleagues experimentally studied how feature transferability changes across network depth and task distance (Yosinski et al. 2014).

Their results show at least two distinct limitations:

- higher-layer specialization can reduce transferability;
- splitting co-adapted representations can create optimization difficulties.

Kornblith, Shlens, and Le later showed that source benchmark quality and transferable-feature quality are not interchangeable, and that training choices can substantially affect transfer behavior (Kornblith et al. 2019).

The bounded conclusion is:

source performance $\nrightarrow$ universal transfer quality.

17.7 Parameter reuse is weaker than capability transfer

Suppose we copy source weights into a target model.

That establishes one fact:

the target optimization began from source parameters.

It does not establish:

- which source capabilities survived;
- which features were reused;
- whether the target behavior improved;
- whether the target could have learned the same behavior from scratch;
- whether the transferred parameters were actually necessary.

Parameter reuse is a mechanism.

Transfer is an outcome relative to a declared target criterion.

17.8 Representation transfer

A common strategy is to reuse an intermediate representation

$$z = E_s(x)$$

and learn a new target readout.

This works only if the representation retains distinctions the target task needs.

A feature extractor can be highly sufficient for the source task and still erase information needed by the target.

This is the direct bridge to the Residual formalism.

The target asks:

Is the transferred representation still sufficient for the capability I now want?

17.9 The common-Residual reconstruction certificate

Let

$$E_s : X \rightarrow Z_s$$

be a source representation.

Let

$$E_t : X \rightarrow Z_t$$

be a target representation.

Let

$$R : X \rightarrow \mathcal{R}$$

be a declared admissible Residual.

Suppose there exist compilers

$$C_s : Z_s \rightarrow \mathcal{R},$$

and

$$C_t : Z_t \rightarrow \mathcal{R},$$

such that

$$\boxed{C_s(E_s(x)) = C_t(E_t(x)) = R(x)}$$

for every declared state x .

Now let the target capability family be indexed by $q \in \mathcal{Q}_t$.

Suppose each target behavior factors through R :

$$\boxed{B_t(x, q) = D_q(R(x))}.$$

Then the target capability can be reconstructed from either representation:

$$B_t(x, q) = D_q(C_s(E_s(x))),$$

and

$$B_t(x, q) = D_q(C_t(E_t(x))).$$

This is the **common-Residual reconstruction certificate**.

17.10 What the certificate says

The certificate says:

source and target coordinates may differ, but if both compile to the same capability-sufficient structure, the declared capability survives.

This is a sufficient condition.

It is not a characterization of all transfer.

A direct adapter

$$A : Z_s \rightarrow Z_t$$

might exist even when we have not identified a shared Residual.

The common Residual is useful because it separates:

- representation-specific coordinates;
- transferable capability structure.

17.11 Exact linear witness

Let the transferable structure be

$$r = \begin{pmatrix} a \\ b \end{pmatrix}.$$

The source representation is

$$z_s = A_s r,$$

with

$$A_s = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

The target representation is

$$z_t = A_t r,$$

with

$$A_t = \begin{pmatrix} 2 & 0 \\ 0 & 1/2 \end{pmatrix}.$$

Both matrices are invertible.

Therefore define

$$C_s = A_s^{-1},$$

and

$$C_t = A_t^{-1}.$$

Both recover the same r .

17.12 Exact numeric reconstruction

Take

$$r = (3, 1).$$

Then

$$z_s = A_s r = (4, 2).$$

And

$$z_t = A_t r = (6, 1/2).$$

The exact witness verifies:

$$C_s z_s = (3, 1),$$

and

$$C_t z_t = (3, 1).$$

Different coordinates.

Same reconstructive core.

17.13 Two target tasks

Now define two target tasks:

$$D_1(a, b) = a + b,$$

and

$$D_2(a, b) = 2a - b.$$

For

$$r = (3, 1),$$

we obtain

$$D_1 = 4,$$

and

$$D_2 = 5.$$

Both source and target representations reconstruct both outputs exactly through the same r .

17.14 Exact recoding is the easy case

In the witness above, both representations are invertible transforms of the Residual.

No information is lost.

This is the easiest kind of transfer.

It isolates one concept:

coordinate change $\neq$ capability loss.

But exact invertibility is not required in general.

A target representation can discard information as long as it preserves everything needed for the declared target task family.

That brings us to bottlenecks.

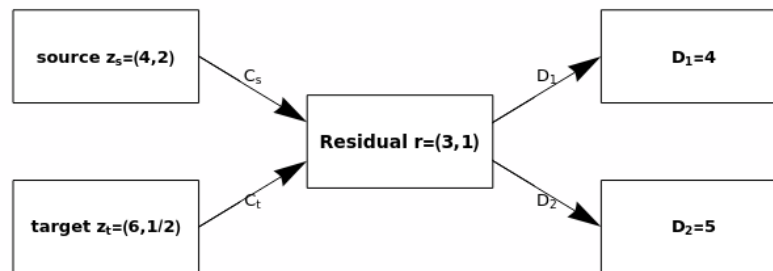
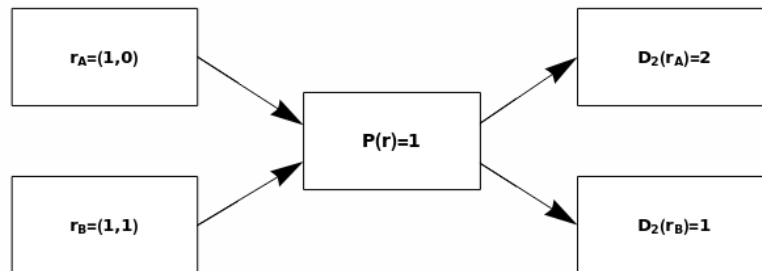
Common-Residual reconstruction certificate*different coordinates, same reconstructive core***Lossy bottleneck destroys task sufficiency***same bottleneck**one decoder cannot output both*

Figure 17.1: Two different representations compile to the same exact Residual $r=(3,1)$ and reconstruct target outputs 4 and 5; a second panel shows a lossy bottleneck $P(r)=1$ merging two states that require different D_2 outputs.

17.15 A lossy bottleneck

Define

$$P(a, b) = a.$$

The second coordinate is discarded.

This can still be sufficient for tasks that depend only on a .

For example,

$$D_0(a, b) = 3a$$

can be reconstructed from P .

But the same bottleneck can fail for another task.

17.16 Exact bottleneck failure

Take

$$r_A = (1, 0),$$

and

$$r_B = (1, 1).$$

Then

$$P(r_A) = 1,$$

and

$$P(r_B) = 1.$$

The bottleneck makes the states identical.

Now evaluate

$$D_2(a, b) = 2a - b.$$

We get

$$D_2(r_A) = 2,$$

and

$$D_2(r_B) = 1.$$

One bottleneck value would have to decode to two different outputs. That is impossible for a deterministic decoder.

17.17 The fiber criterion

The bottleneck example is a special case of a general exact criterion. Let

$$Z : X \rightarrow \mathcal{Z}$$

be a representation.

Let

$$B : X \rightarrow \mathcal{Y}$$

be a deterministic capability map.

A deterministic decoder

$$D : \mathcal{Z} \rightarrow \mathcal{Y}$$

with

$$B = D \circ Z$$

exists if and only if B is constant on every fiber of Z .

That is:

$$\boxed{Z(x_1) = Z(x_2) \Rightarrow B(x_1) = B(x_2).}$$

This is the exact reconstruction test.

17.18 Why the fiber criterion is necessary

Suppose

$$B = D \circ Z.$$

If

$$Z(x_1) = Z(x_2),$$

then

$$B(x_1) = D(Z(x_1)) = D(Z(x_2)) = B(x_2).$$

So any representation that merges capability-distinct states cannot support exact deterministic reconstruction.

This is information loss in the most concrete possible sense.

17.19 Why the fiber criterion is sufficient

Now suppose B is constant on each realized fiber of Z .

For each realized representation value z , choose any state x with

$$Z(x) = z.$$

Define

$$D(z) = B(x).$$

This is well-defined because every state in the same fiber has the same B -value.

Therefore

$$B = D \circ Z.$$

This is a semantic existence result.

It does not guarantee that the decoder is simple or efficiently learnable.

17.20 One-task sufficiency is not task-family sufficiency

A representation may preserve exactly the information needed for one task and destroy information needed by another.

The bottleneck

$$P(a, b) = a$$

is sufficient for any target depending only on a .

It is insufficient for

$$D_2(a, b) = 2a - b.$$

Therefore transferability should often be indexed by a task family:

$$\mathcal{Q}_t.$$

A representation that transfers to task q_1 need not transfer to q_2 .

There is no contradiction.

The target changed.

17.21 Universal representation quality is under-specified

People often ask whether a representation is “good.”

Good for what?

Possible criteria include:

- linear separability;
- reconstruction;
- invariance;
- robustness;
- compression;
- adaptation speed;
- transfer across tasks;
- transfer across domains.

These objectives can disagree.

The Transfer chapter therefore refuses to assign one universal scalar quality to a representation without a declared target family and adaptation setting.

17.22 Information bottleneck connection

The Information Bottleneck programme asks for compression while preserving information relevant to a declared target (Tishby et al. 2000).

That is structurally close to transfer through a bottleneck.

A representation can discard source variation while retaining target-relevant structure.

But mutual information is not the definition of transfer.

Transfer also depends on:

- decoder class;
- optimization;
- sample size;
- shift;
- task geometry;
- adaptation cost.

Information preservation is one piece.

17.23 Minimal sufficient representation connection

Achille and Soatto frame representations through task sufficiency, minimality, and nuisance invariance (Achille and Soatto 2018).

The Transfer chapter inherits a compatible lesson:

compressing away nuisance can help only if the target does not later require what was removed.

The target task determines which variation is nuisance.

A source-task nuisance can become a target-task signal.

17.24 Transfer across domains

Suppose source and target use the same task but different input distributions:

$$P_s(X) \neq P_t(X).$$

A source representation can preserve all distinctions needed on the source support while behaving poorly on target regions.

The exact fiber criterion must therefore be evaluated over the declared target domain.

A certificate over the wrong support is not a reconstruction certificate.

This is one reason domain shift matters.

17.25 Transfer across tasks

Suppose source and target share the same domain but differ in target function.

A source representation can be sufficient for

$$f_s$$

but insufficient for

$$f_t.$$

This is exactly what the bottleneck witness demonstrates in miniature.

Task change can turn discarded variation into required information.

17.26 Transfer across representation systems

The Residual framing is most distinctive when the source and target do not share coordinates.

One model may encode a capability in:

- a dense hidden state;
- a sparse dictionary;
- an operator basis;
- a normalized manifold state;
- external memory.

If both can compile to the same admissible Residual, transfer does not require one universal latent basis.

This is the bridge to model interoperability.

17.27 Exact information preservation does not imply easy learning

Suppose source and target representations are related by an invertible matrix.

No information is lost.

But the adapter may still be hard to learn because of:

- poor conditioning;
- limited samples;
- nonlinear parameterization;
- restricted decoder class;
- optimization failure.

Therefore:

semantic transfer path $\neq$ guaranteed learnable adapter.

The Atlas keeps semantic sufficiency and computational attainability separate.

17.28 Transfer can fail at the interface

Suppose a source representation contains the required information.

Suppose the target decoder class could in principle use it.

Transfer can still fail if the interface contract is wrong.

Examples include:

- unit mismatch;
- scaling mismatch;
- permutation mismatch;
- unsupported uncertainty semantics;
- precision loss;
- stale memory schema.

Transfer is therefore also a composition problem.

The Boundary Contracts chapter will later make this more explicit.

17.29 Architecture transfer versus feature transfer

Kornblith and colleagues provide a useful empirical warning (Kornblith et al. 2019).

An architecture can transfer well even when a particular set of source-trained features is less transferable than source performance might suggest.

This separates two objects:

- reusable architecture;
- reusable representation state.

The Atlas treats them differently.

Architecture is a transformation/composition template.

Features are instantiated states.

Transfer can occur through either.

17.30 Fine-tuning changes the object

When a transferred representation is fine-tuned, it does not simply “remain transferable.”

It becomes a new target-adapted state.

The causal story now includes:

- initialization;
- optimization path;
- target data;
- regularization;
- adaptation schedule.

A post-fine-tuning representation cannot be treated as unchanged source structure.

Transfer attribution requires care.

17.31 A reconstruction certificate is not a training algorithm

The common-Residual certificate proves that exact target reconstruction is possible under declared maps.

It does not tell us how to learn:

$$C_s, \quad C_t, \quad D_q.$$

Those may be known analytically in a toy setting and unknown in practice.

A future Atlas programme can distinguish:

- **existence of transfer path;**
- **identifiability of transfer path;**
- **learnability of transfer path;**
- **efficiency of transfer path.**

These are different questions.

17.32 Reconstruction tests

A practical transfer claim can be stress-tested with reconstruction.

Given source and target representations:

1. declare the target capability family;
2. declare admissible compilers/adapters;
3. attempt to reconstruct capability from each representation;
4. search for target states collapsed by the representation;
5. compare transfer against a matched target-only baseline;
6. intervene on supposedly irrelevant source structure;
7. repeat under domain and representation changes.

Failure at different stages means different things.

17.33 What would falsify a transfer claim?

17.33.1 Sufficiency failure

Two target states share the same transferred representation but require different outputs.

17.33.2 Domain failure

A certificate holds on source support but not target support.

17.33.3 Adapter failure

An admissible adapter class cannot recover the required object.

17.33.4 Baseline failure

Transfer underperforms a matched target-only baseline.

17.33.5 Robustness failure

The transferred capability disappears under allowed representation changes.

17.33.6 Attribution failure

Target performance comes from relearning rather than transferred structure.

These failure modes should not be collapsed into one number.

17.34 Transferable Residual versus universal Residual

The Residual chapter was task-relative.

Transfer does not remove that relativity.

A shared Residual sufficient for one target family may fail for another.

The useful object is therefore:

$$R_{\mathcal{Q}}$$

for a declared family of capability probes $\mathcal{Q}$, not an assumed universal essence of the model.

The search for broader Residuals is an empirical and mathematical programme.

It is not a free theorem.

17.35 Minimal curricula

Transfer connects directly to curriculum design.

If an early mechanism can reconstruct many later capabilities with limited adaptation, it has high transfer value.

A minimal curriculum therefore asks:

Which learned structures create the broadest reconstructive reuse across later tasks?

This is stronger than asking which examples reduce next-step training loss fastest.

The later Minimal Curricula chapter will combine this transfer criterion with learning-progress search.

17.36 Minimal reasoning basis

A reasoning operation is transferable if it can be reused across problem families without carrying problem-specific surface structure.

The Residual/Transfer pair suggests a test:

- identify a candidate reasoning operation;
- recode the problem;
- change surface notation;
- change task family;
- test whether the operation still supports reconstruction of successful reasoning behavior.

This is how “canonical reasoning transformations” can become falsifiable rather than rhetorical.

17.37 Model interoperability

Two models need not share hidden coordinates if both can compile to a common transferable object.

This suggests an interoperability architecture:

$$\text{Model A state} \rightarrow R \rightarrow \text{Model B capability.}$$

The hard questions are:

- what is R ?
- which tasks does it preserve?
- how costly are the compilers?
- what semantics must the interface expose?
- how do we certify reconstruction?

Transfer becomes a systems problem.

17.38 Memory and transfer

Persistent memory is another transfer surface.

A model may write an object today and another model may consume it later.

If memory stores representation-specific coordinates, the second model may not understand them.

If memory stores a transferable Residual or sufficient evidence object, reconstruction can survive model change.

This is one reason external memory should preserve semantics and provenance rather than raw activations alone.

17.39 Compression as a transfer probe

A bottleneck tests what can be discarded.

If a compressed object still transfers across many target tasks, it may have isolated reusable structure.

If transfer collapses after one additional compression step, that step removed capability-relevant information.

Compression can therefore become an empirical probe for transfer structure.

This connects directly to the later Compression as Discovery chapter.

17.40 Closing view

Transfer learning is often described as reuse.

The Atlas asks a stricter question.

What, exactly, survived?

A copied parameter is not yet evidence.

A reused architecture is not the same thing as a reused feature.

A high source score is not a transfer guarantee.

A common Residual gives one clean sufficient answer:

different representation $\rightarrow$ same reconstructive core $\rightarrow$ target capability.

And the fiber criterion gives the corresponding impossibility test:

if a representation merges states the target must distinguish, exact reconstruction is gone.

Transfer therefore sits at the intersection of:

- representation;
- information;
- reconstruction;
- optimization;
- composition.

The practical question is not whether knowledge was reused in some vague sense.

It is whether the structure that crossed the boundary was sufficient for what the target still needed to do.

17.41 References used in this chapter

- (Pan and Yang 2010)
- (Yosinski et al. 2014)
- (Kornblith et al. 2019)
- (Achille and Soatto 2018)
- (Tishby et al. 2000)

See ‘sources/source-locks/ATLAS-CH-TRANSFER-001.yaml’ for exact source roles, Atlas dependencies, and claim boundaries.

Chapter 18

From Layered Networks to Residual Systems

Epistemic status: Primary Architecture History + Established Structural Mathematics + Atlas Synthesis

Specification: manuscript/specifications/ATLAS-CH-ARCHHIST-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-ARCHHIST-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-ARCHHIST-001.md

Source lock: sources/source-locks/ATLAS-CH-ARCHHIST-001.yaml

18.1 Architecture is a choice of mathematical object

A neural architecture is not merely a list of layers.

It declares:

- what counts as state;
- which transformations are shared;
- what information persists;
- where bottlenecks live;
- how depth transports information;
- which symmetries are built in.

The useful historical question is therefore not:

Which architecture came next?

It is:

What new mathematical object became worth analyzing?

This chapter follows a selective lineage:

composition $\rightarrow$ structured operators $\rightarrow$ persistent state $\rightarrow$ interfaces $\rightarrow$ gated carry $\rightarrow$ residual transport

That sequence is pedagogical.

It is not a claim that the actual history of neural networks was linear.

18.2 From assembly line to evolving state

A useful allegory is an assembly line.

A feed-forward network moves an object through a sequence of stations.

A convolutional network reuses the same local tool at many spatial locations.

A recurrent network gives the object a notebook that persists through computational time.

An encoder–decoder system forces two subsystems to communicate through an interface.

A highway layer adds learned gates deciding how much to transform and how much to carry.

A residual block carries the current state forward and adds a correction.

A continuous-depth model replaces the discrete sequence with an explicitly parameterized evolution law.

The limit of the allegory is important.

Neural networks are not factories.

The value of the metaphor is only that it makes the change in mathematical object visible.

18.3 Layered composition

A feed-forward network can be written as

$$x_{k+1} = F_k(x_k).$$

After L layers,

$$x_L = F_{L-1} \circ \cdots \circ F_0(x_0).$$

The basic object is therefore a composition of learned maps.

This viewpoint is already enough to introduce:

- hidden representations;
- depth;
- nonlinear composition;
- back-propagated derivatives.

Rumelhart, Hinton, and Williams gave one influential formulation of efficient error back-propagation through multilayer networks (Rumelhart et al. 1986).

The historical claim here is deliberately narrow.

The chapter does not assign unique invention of multilayer learning to one paper.

18.4 Composition creates a product of Jacobians

If each F_k is differentiable, the chain rule gives

$$J_{\text{total}} = J_{F_{L-1}}(x_{L-1}) \cdots J_{F_0}(x_0).$$

This simple product has enormous consequences.

Signals and gradients can be:

- amplified;
- attenuated;
- rotated;
- collapsed;
- made ill-conditioned.

The architecture therefore determines not only which functions can be represented.

It also shapes how derivatives are transported through depth.

18.5 The early object: a hierarchy of representations

In a layered network, each hidden state

$$x_k$$

is both:

- the output of earlier computation;
- the input to later computation.

This creates a hierarchy of internal representations.

The Representation chapter taught us not to assume that a hidden coordinate is intrinsically meaningful.

Architecture adds a further question:

Which transformations are imposed before that representation even has a chance to form?

Convolution provides the first major structural answer in this lineage.

18.6 Convolution replaces arbitrary connectivity with a structured operator

A dense linear map can connect every input coordinate to every output coordinate.

A convolutional map ties weights across locations.

For a periodic one-dimensional signal, define

$$(C_k x)_j = \sum_r k_r x_{j-r}.$$

The same kernel coefficients

$$k_r$$

are reused across positions.

LeCun and colleagues used local receptive fields and weight sharing in a highly influential convolutional architecture for document recognition (LeCun et al. 1998).

The structural lesson is broader than the application:

architecture can encode a symmetry assumption through operator structure.

18.7 Exact translation equivariance in the declared model

Let cyclic shift S_m act by

$$(S_m x)_j = x_{j-m}.$$

Then

$$\begin{aligned}(C_k S_m x)_j &= \sum_r k_r (S_m x)_{j-r} \\ &= \sum_r k_r x_{j-r-m} \\ &= (S_m C_k x)_j.\end{aligned}$$

Therefore

$$\boxed{C_k S_m = S_m C_k.}$$

This is exact translation equivariance for the declared circular-convolution model.

The architecture is not learning this symmetry from scratch.

The operator class already contains it.

18.8 Exact convolution witness

Use

$$S = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

and

$$C = I + 2S.$$

Because C is a polynomial in S ,

$$CS = SC.$$

For

$$x = (1, 2, 3, 4),$$

the exact computational witness gives

$$CSx = SCx = (10, 9, 4, 7).$$

The symmetry is algebraic in this toy model.

It is not an empirical approximation.

18.9 Practical CNNs can break exact equivariance

The exact proof above is intentionally narrow.

Real pipelines may include:

- zero padding;
- asymmetric boundaries;
- striding;
- pooling;
- resizing;
- nonlinear preprocessing;
- position-dependent operations.

Any of these can alter exact equivariance.

So the safe statement is:

convolution supplies a translation-structured operator, while full-pipeline equivariance must be checked operation by operation.

This distinction matters later when positional information is added deliberately.

18.10 Recurrent systems change the object again

A feed-forward layer consumes a state and passes a new state onward.

A recurrent system makes state persistence explicit:

$$h_{t+1} = F(h_t, x_t; \theta).$$

The same parameter set θ can be reused across computational time.

Now the central object is no longer only a composition of distinct maps.

It is a state-transition system.

18.11 Parameter sharing across time

Unrolling a recurrent network produces a deep computation.

But the same transition law can reappear at every time step.

This creates a different kind of depth:

depth by repeated state evolution.

The distinction becomes important later for:

- recurrent depth;
- weight tying;
- latent clocks;
- iterative reasoning.

A recurrent architecture is naturally read as a discrete dynamical system.

It is not automatically a continuous flow.

18.12 Exact linear recurrence

Consider

$$h_{t+1} = ah_t + bx_t.$$

Repeated substitution gives

$$h_T = a^T h_0 + b \sum_{j=0}^{T-1} a^{T-1-j} x_j.$$

Earlier inputs are transported by powers of a .

The recurrence therefore exposes a memory law directly.

If

$$|a| < 1,$$

old contributions decay geometrically in this scalar model.

If

$$|a| > 1,$$

they can grow.

The architecture creates a temporal transport problem.

18.13 Exact recurrence witness

Take

$$a = \frac{1}{2}, \quad b = 2, \quad h_0 = 1,$$

and

$$(x_0, x_1, x_2) = (3, -1, 4).$$

The direct recurrence gives

$$(h_0, h_1, h_2, h_3) = \left(1, \frac{13}{2}, \frac{5}{4}, \frac{69}{8}\right).$$

The closed form independently gives

$$h_3 = \frac{69}{8}.$$

This is the simplest exact demonstration that recurrent state carries weighted history.

18.14 Long-term memory is not guaranteed by recurrence

A recurrent loop can preserve state in principle.

That does not mean it preserves useful information over long intervals.

Problems can arise from:

- repeated Jacobian products;
- contraction;
- expansion;
- saturation;
- interference;
- optimization difficulty.

This is one reason gated recurrent systems were developed.

18.15 LSTM: persistent state plus gates

The Long Short-Term Memory architecture uses gated recurrent machinery designed to address long-lag learning difficulties (Hochreiter and Schmidhuber 1997).

The structural shift is important:

persistence becomes an explicit controlled pathway.

The state is no longer merely whatever survives repeated application of one transition.

Gates regulate:

- what enters;
- what remains;
- what becomes externally visible.

This is an architectural move toward managed state transport.

18.16 Gates are operators on information flow

A gate can be read as a data-dependent multiplicative operator.

Schematically,

$$g_t \odot v_t.$$

The gate value decides how strongly a quantity is passed.

This is distinct from a plain additive update.

The later Highway Networks construction will reuse the same broad design idea across depth rather than only across recurrent time.

18.17 Encoder–decoder architecture introduces an explicit interface

Now separate computation into two subsystems:

$$z = E(x),$$

$$\hat{y} = D(z).$$

The encoder produces an interface object z .

The decoder is constrained to work from that object.

Sequence-to-sequence systems made this separation especially visible for variable-length input/output mappings (Sutskever et al. 2014).

The important Atlas object is the interface itself.

18.18 The interface is a contract

Once the decoder receives only z , every downstream capability must be recoverable from z .

This makes encoder–decoder architecture a concrete instance of a broader principle:

an interface is sufficient only for the distinctions downstream computation still needs.

The Representation, Residual, and Transfer chapters formalized this idea more generally.

Here it appears as architecture.

18.19 Encoder information-loss boundary

Suppose

$$E(x_1) = E(x_2) = z,$$

but the required outputs satisfy

$$y_1 \neq y_2.$$

A deterministic decoder receives the same z in both cases.

Therefore

$$D(E(x_1)) = D(E(x_2)).$$

It cannot equal both distinct targets.

So a noninjective encoder may discard information safely only relative to the downstream capability being preserved.

This is not an argument that useful encoders must be injective.

Compression is often the point.

18.20 Bottleneck is a design decision

The interface can be:

- a fixed-dimensional vector;
- a sequence;
- a memory;
- a graph;
- an operator state;
- a set of retrieved evidence.

What matters is what distinctions it preserves.

Later attention architectures loosen the fixed-vector bottleneck dramatically.

But the encoder–decoder lesson remains:

communication between subsystems creates an explicit sufficiency boundary.

18.21 Highway Networks: transform or carry

Highway Networks use learned transform and carry gates across depth (Srivastava et al. 2015).

A structural form is

$$y = T(x) \odot H(x) + C(x) \odot x.$$

A common tied-gate choice is

$$C(x) = 1 - T(x).$$

Then

$$y = T(x) \odot H(x) + (1 - T(x)) \odot x.$$

The layer can learn how much to transform and how much to carry.

18.22 Highway identity limit

In the tied-gate form, if

$$T(x) = 0,$$

then

$$C(x) = 1,$$

so

$$\boxed{y = x.}$$

The architecture therefore contains an exact carry path.

At the opposite limit,

$$T(x) = 1,$$

we obtain

$$y = H(x).$$

With the tied gate constrained coordinatewise to $([0,1])$, the layer interpolates coordinatewise between carried and transformed state.

18.23 Exact highway witness

Take

$$x = 2, \quad H(x) = 5,$$

and

$$T = \frac{1}{4}, \quad C = \frac{3}{4}.$$

Then

$$y = \frac{1}{4} \cdot 5 + \frac{3}{4} \cdot 2 = \frac{11}{4}.$$

The exact carry limit returns

2.

The exact transform limit returns

5.

This small witness shows the gate algebra without making any claim about training dynamics.

18.24 Gated carry is not residual addition

Highway and residual systems both create a direct path for state transport.

But they do so differently.

A highway layer uses multiplicative gates:

$$T \odot H + C \odot x.$$

A canonical residual block uses additive identity transport:

$$x + F(x).$$

These are related design ideas.

They are not the same algebra.

The distinction matters when analyzing Jacobians and optimization.

18.25 Residual learning

Residual networks made the identity-plus-increment form central:

$$x_{k+1} = x_k + F_k(x_k).$$

He and colleagues used residual learning to train very deep image-recognition systems and reported substantial empirical gains in that setting (He et al. 2016).

The Atlas uses the architecture for a narrower structural reason:

depth now looks like repeated state transport plus correction.

18.26 Residual Jacobian

Differentiate:

$$x_{k+1} = x_k + F_k(x_k).$$

Then

$$\boxed{J_k = I + J_{F_k}(x_k)}.$$

Across L residual blocks,

$$J_{\text{total}} = (I + J_{F_{L-1}}) \cdots (I + J_{F_0}).$$

The identity contribution is exact.

The product can still be badly conditioned.

18.27 Identity transport

If the residual branch vanishes,

$$F_k(x) = 0,$$

then

$$x_{k+1} = x_k.$$

The block becomes exactly the identity map.

That is the precise identity-transport claim.

It does not prove:

- all residual networks optimize easily;
- all gradients remain well conditioned;
- arbitrary depth is harmless.

The architecture creates an identity route.

Training still determines what happens around it.

18.28 Exact residual witness

For

$$F(x) = Ax, \quad A = \begin{pmatrix} 1 & 2 \\ -1 & 3 \end{pmatrix},$$

the residual block Jacobian is

$$I + A = \begin{pmatrix} 2 & 2 \\ -1 & 4 \end{pmatrix}.$$

If

$$A = 0,$$

the block Jacobian is exactly

$$I.$$

The computational witness checks both identities.

18.29 Structural lineage plate

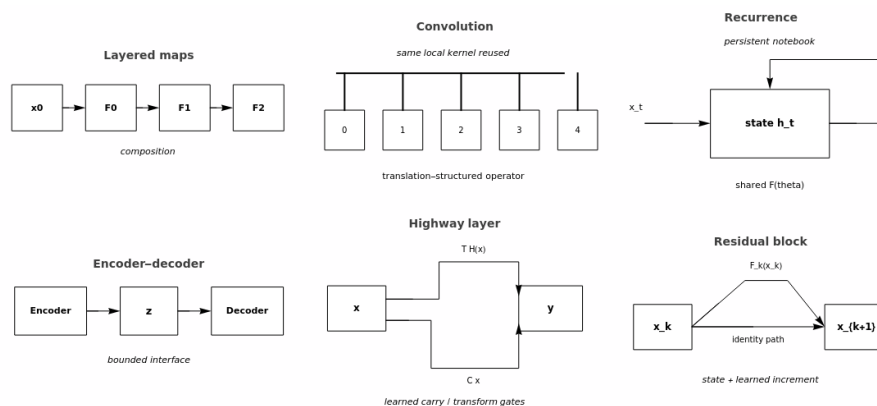


Figure 18.1: Six schematic panels showing layered composition, convolutional weight sharing, recurrent state, encoder-decoder interface, highway transform/carry gating, and residual identity bypass.

The plate is deliberately schematic.

It does not encode:

- historical priority;
- benchmark quality;
- parameter count;
- compute;
- architectural superiority.

Its only job is to show how the mathematical object changes.

18.30 Residual systems invite a dynamical lens

Rewrite the residual update as

$$x_{k+1} - x_k = F_k(x_k).$$

If

$$F_k(x) = hf_k(x),$$

then

$$x_{k+1} = x_k + hf_k(x_k).$$

This resembles an explicit Euler step.

That resemblance is useful.

It is not an identity theorem.

18.31 The ODE analogy has conditions

To interpret a residual system as a discretization of one autonomous ODE,

$$\dot{x} = f(x),$$

we would need more than the residual form.

Questions include:

- Is there one shared f ?
- Is there a meaningful small step h ?
- Does the discrete family converge as $h \rightarrow 0$?
- Are the layer operations compatible with a smooth vector field?
- Does the numerical method preserve the relevant structure?

Generic ResNets do not answer these automatically.

So the safe statement is:

residual systems admit an integrator lens.

18.32 Neural ODEs make continuous depth explicit

Neural Ordinary Differential Equations specify the evolution law directly:

$$\frac{dz}{dt} = f(z, t; \theta),$$

and obtain outputs through numerical integration (Chen et al. 2018).

Here continuous depth is part of the model definition.

The numerical solver is therefore an architectural component.

This is categorically different from retrospectively declaring every residual network to be an ODE.

18.33 Continuous depth changes where approximation lives

In a discrete network, learned layer maps are explicit model components.

In a Neural ODE, the learned object is the vector field, while finite-time transport is produced by a numerical solver.

Approximation therefore enters in two places:

- function approximation in f ;
- numerical approximation of the flow.

This creates the bridge to the Numerical Intelligence chapter.

18.34 Architecture as transport

The lineage can now be restated.

A layered network says:

apply another map.

A recurrent system says:

update persistent state.

A highway system says:

learn how much state to carry.

A residual system says:

carry state and add an increment.

A continuous-depth system says:

specify a law of motion and integrate it.

The object has shifted from static composition toward transport.

18.35 Architecture can encode invariance

Convolution encodes translation structure.

Recurrence encodes temporal parameter sharing.

Encoder–decoder systems encode a communication bottleneck.

Highway layers encode gated carry.

Residual blocks encode identity transport.

These are architectural priors.

They reduce the set of possible computations before training begins.

Architecture is therefore a form of inductive bias expressed through operators and state organization.

18.36 Architecture also creates failure modes

Every structural prior excludes something.

Convolution can mishandle boundaries or nonstationary spatial structure.

Recurrence can forget.

Bottlenecks can erase task-relevant distinctions.

Gates can saturate or route poorly.

Residual products can still become ill-conditioned.

Continuous-depth models inherit numerical-solver choices.

There is no architecture without a failure surface.

18.37 History is not a ranking

The sequence in this chapter should not be read as:

$$\text{MLP} < \text{CNN} < \text{RNN} < \text{ResNet} < \text{ODE}.$$

Different architectures solve different structural problems.

A simple MLP may be exactly right for one task.

A recurrent state may be unnecessary.

A continuous-depth solver may add cost without benefit.

The Atlas uses history only to expose reusable mathematical ideas.

18.38 What changes in the object of analysis?

The structural transitions can be summarized as follows.

18.38.1 Layered networks

Analyze:

composition.

18.38.2 CNNs

Analyze:

structured shared operators.

18.38.3 RNNs and LSTMs

Analyze:

persistent state evolution.

18.38.4 Encoder–decoders

Analyze:

interfaces and sufficiency.

18.38.5 Highway systems

Analyze:

gated transform/carry paths.

18.38.6 Residual systems

Analyze:

identity transport plus increment.

18.38.7 Neural ODEs

Analyze:

explicit continuous evolution plus numerical solution.

This is the architecture lineage the downstream Atlas needs.

18.39 Bridge to Transformers

Transformers will introduce another decisive shift.

Instead of propagating only through fixed local connectivity or recurrent state, a token representation can dynamically aggregate information from other positions through attention.

The next architecture chapter will therefore add:

content-dependent global interaction.

But it inherits everything here:

- composition;
- state;
- interfaces;
- residual paths;
- normalization;
- depth.

Transformers are not outside this lineage.

They reorganize it.

18.40 Bridge to adaptive depth

Once depth is read as accumulated state transformation, a new question becomes natural:

Why must every input receive the same number of updates?

Adaptive-depth systems make computational horizon input-dependent.

That chapter depends on the distinction established here between:

- layer count;
- state evolution;
- computational time.

18.41 Bridge to network numerics

Residual and continuous-depth systems invite numerical questions:

- What is the effective step size?
- Is the update stable?
- Which invariants drift?
- How does splitting affect the trajectory?
- Can a better integrator improve computation?

Those are not metaphorical questions once the architecture is explicitly organized as state evolution.

The Network Numerics chapter will take them literally.

18.42 What would falsify an architectural analogy?

A structural analogy should make testable predictions.

If we claim convolutional equivariance, test the commutator with translation.

If we claim recurrence stores long-term information, perturb old inputs and measure influence.

If we claim a bottleneck is sufficient, search for target-distinct states it merges.

If we claim residual transport improves conditioning, inspect the Jacobian product rather than the identity term alone.

If we claim a network approximates a continuous flow, test refinement and solver dependence.

Architecture should constrain observation.

18.43 Closing view

The important history is not a procession of brand names.

It is a progression in what computation is allowed to remember, share, preserve, and transport.

We began with

$$x_{k+1} = F_k(x_k).$$

Then architecture learned to express:

- spatial symmetry;
- temporal persistence;
- bounded interfaces;
- gated carrying;
- identity transport;
- explicit continuous evolution.

The Atlas thesis becomes clearer here.

A modern neural model is not merely a large static function.

It is increasingly useful to treat it as an organized computational system whose state is transformed under architectural rules.

The next chapters will keep asking what those rules buy us, what they destroy, and which mathematical tools can certify the difference.

18.44 References used in this chapter

- (Rumelhart et al. 1986)
- (LeCun et al. 1998)
- (Hochreiter and Schmidhuber 1997)
- (Sutskever et al. 2014)
- (Srivastava et al. 2015)
- (He et al. 2016)
- (Chen et al. 2018)

See ‘sources/source-locks/ATLAS-CH-ARCHHIST-001.yaml’ for exact source roles, internal Atlas pins, and claim boundaries.

Chapter 19

The Transformer as Baseline Object

Epistemic status: Established Architecture + Atlas Synthesis + Exact Computational Witness

Specification: `manuscript/specifications/ATLAS-CH-TRANSFORMER-001.md`

Derivation packet: `mathematics/derivations/ATLAS-CH-TRANSFORMER-001-DERIVATIONS.md`

Computational witness: `mathematics/computational-witnesses/ATLAS-CW-TRANSFORMER-001.md`

Source lock: `sources/source-locks/ATLAS-CH-TRANSFORMER-001.yaml`

19.1 We need one reference object

The Atlas will later discuss:

- normalized Transformers;
- sparse Transformers;
- mixture-of-experts;
- split-operator residual transport;
- relative-position operators;
- adaptive depth;
- mechanistic diagnostics;
- network numerics.

Those chapters can only say what they change if the baseline object is explicit.

This chapter therefore asks a deliberately conservative question:

What is the conventional Transformer object we are modifying?

The answer is not “attention.”

A Transformer is an organized state-update system built from:

- token-state representations;
- positional information;
- attention sublayers;
- position-wise feed-forward sublayers;
- residual paths;
- normalization;
- masking;
- topology;
- readout.

19.2 Attention is a component, not the architecture

The Attention-as-an-Operator chapter studies attention deeply.

Here we need less mathematics and more assembly.

One attention head computes a score-derived mixing operator and applies it to values.

A Transformer block then wraps that operation inside:

- projections;
- multi-head factorization;
- output projection;
- residual transport;
- normalization;
- a feed-forward sublayer.

So:

attention $\neq$ Transformer block $\neq$ Transformer system.

This distinction will matter repeatedly downstream.

19.3 The shared blackboard allegory

Imagine a shared blackboard.

Each token position owns one row.

Attention allows one row to read selected information from other rows.

A causal mask closes some sight lines before the read weights are normalized.

The feed-forward network then updates each row locally.

Residual paths preserve the evolving shared state while sublayers add corrections.

The analogy is useful because it separates:

- shared state;
- cross-row communication;
- local transformation;
- access restrictions.

Its limit is equally important.

The residual stream is not literally a blackboard service, and an attention weight is not automatically a semantic explanation.

19.4 Residual-stream state

Let sequence length be n and model width be d .

Write the Transformer state as

$$H \in \mathbb{R}^{n \times d}.$$

Each row represents one sequence position.

The initial state is typically formed from token embeddings plus positional information:

$$H_0 = E(\text{tokens}) + P.$$

The positional mechanism P is a separate design axis.

The chapter does not assume that sinusoidal, learned, rotary, relative, or operator-valued position is universally correct.

19.5 One attention head

For one head,

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V.$$

The score matrix is

$$S = \frac{QK^\top}{\sqrt{d_k}} + M.$$

The row-normalized mixing matrix is

$$A = \text{softmax}_{\text{row}}(S).$$

The head output is

$$O = AV.$$

This creates a useful object chain:

$$H \rightarrow (Q, K, V) \rightarrow S \rightarrow A \rightarrow O.$$

Each object answers a different question.

19.6 Scores are not weights

The score matrix S is unnormalized.

The mixing matrix A is normalized row-wise.

The values V are the content being mixed.

The output O is the result of applying the mixing operator.

Therefore:

$$\text{score} \neq \text{attention weight} \neq \text{value} \neq \text{output}.$$

This may look elementary.

Many later diagnostic errors begin by collapsing these distinctions.

19.7 The original Transformer

Vaswani and colleagues introduced the Transformer architecture built around self-attention, multi-head attention, position-wise feed-forward layers, residual connections, LayerNorm, positional encoding, and an encoder-decoder topology (Vaswani et al. 2017).

The historical claim here is source-scoped.

The Atlas uses that paper as the canonical source for the original architecture.

It does not claim every modern Transformer follows the original block ordering.

19.8 Multi-head attention

Suppose there are m heads.

Each head produces

$$O_h.$$

The outputs are concatenated:

$$C = \text{Concat}(O_1, \dots, O_m).$$

The multi-head sublayer then applies an output projection:

$$\boxed{\text{MHA}(H) = CW_O.}$$

The heads therefore do not simply remain separate.

Their projected outputs are recombined.

19.9 Exact two-head witness

Take one token and two head outputs:

$$O_1 = (1, 2),$$

$$O_2 = (3, 4).$$

Concatenate:

$$C = (1, 2, 3, 4).$$

Let

$$W_O = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 2 & -1 \end{pmatrix}.$$

Then

$$\boxed{CW_O = (12, 1).}$$

This exact witness checks head concatenation and output projection only.

It says nothing about whether the heads learned distinct semantic roles.

19.10 Multi-head does not mean one monolithic attention matrix

Each head can have its own:

- query projection;
- key projection;
- value projection;
- score matrix;
- normalized mixing matrix.

The resulting head outputs are then concatenated and projected.

One can sometimes derive equivalent block representations.

But the baseline architecture should not be casually described as one giant attention matrix unless that equivalence is stated precisely.

The factorization is part of the architecture.

19.11 Causal masking is architectural access control

For left-to-right autoregressive self-attention, define the ideal causal mask:

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

After adding the mask to scores, future positions receive ideal score

$$-\infty.$$

Therefore their exponentiated contribution is zero.

After softmax,

$$\boxed{A_{ij} = 0 \quad (j > i).}$$

The mask changes which dependencies are permitted before learned weights are normalized.

It is not a learned preference.

19.12 Mathematical mask versus implementation mask

The exact mathematical notation uses

$$-\infty.$$

Real systems usually implement this with:

- masked-softmax logic;
- a representable very negative number;
- fused attention kernels with explicit mask semantics.

That distinction matters.

The theorem is about zero support on forbidden positions.

The implementation is about realizing that support reliably in finite arithmetic.

19.13 Exact three-token causal witness

Take zero unmasked scores.

Then each row distributes weight uniformly over its allowed prefix:

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1/2 & 0 \\ 1/3 & 1/3 & 1/3 \end{pmatrix}.$$

Let

$$V = \begin{pmatrix} 2 \\ 5 \\ 11 \end{pmatrix}.$$

Then

$$AV = \begin{pmatrix} 2 \\ 7/2 \\ 6 \end{pmatrix}.$$

Now replace the third value only:

$$V' = \begin{pmatrix} 2 \\ 5 \\ 101 \end{pmatrix}.$$

Then

$$AV' = \begin{pmatrix} 2 \\ 7/2 \\ 36 \end{pmatrix}.$$

The first two outputs remain exactly unchanged.

The future value cannot affect earlier rows through this masked attention operation.

19.14 Causal mask does not imply causal understanding

The dependency graph is causal in the narrow left-to-right architectural sense: future token states are unavailable.

This does not imply that:

- the model has discovered causal structure in the scientific sense;
- attention weights identify causal explanations;
- generated text is causally grounded.

“Causal mask” describes permitted sequence dependence.

Nothing more follows automatically.

19.15 Position-wise feed-forward network

After attention, the Transformer applies a feed-forward network independently at each position.

For token state h ,

$$\boxed{\text{FFN}(h) = \phi(hW_1 + b_1)W_2 + b_2.}$$

The same FFN parameters are reused across sequence positions within the layer.

This is an important contrast:

- attention mixes across positions;
- FFN transforms each position locally in state space.

19.16 Different kinds of sharing

The architecture now contains several kinds of repeated structure.

Convolution shares a local kernel across spatial positions.

Recurrence shares one transition across time.

Transformer FFNs share one position-wise map across sequence positions.

Attention shares projection matrices across positions while the mixing pattern is content-dependent.

Parameter sharing is therefore not one phenomenon.

Its geometry depends on the operator.

19.17 Residual transport

A Transformer sublayer usually sits inside a residual connection.

Schematically:

$$R(H) = H + S(H).$$

If

$$S(H) = 0,$$

then

$$\boxed{R(H) = H.}$$

The exact identity path from the Architecture History chapter survives here.

The sublayer adds a correction to the evolving residual-stream state.

19.18 Exact residual identity witness

Take

$$H = (2, -1, 3).$$

Let

$$S(H) = (0, 0, 0).$$

Then

$$H + S(H) = (2, -1, 3).$$

This is the precise identity-path claim.

It does not establish that the total Transformer Jacobian remains well conditioned.

19.19 Layer Normalization

Layer Normalization normalizes features within a state vector (Ba et al. 2016).

For

$$x \in \mathbb{R}^d,$$

define

$$\mu(x) = \frac{1}{d} \sum_i x_i,$$

and

$$\sigma^2(x) = \frac{1}{d} \sum_i (x_i - \mu)^2.$$

Then

$$\boxed{\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta.}$$

The stabilizing constant

$$\varepsilon > 0$$

belongs in the definition.

19.20 LayerNorm is not bookkeeping

LayerNorm changes the representation.

Before the learned affine parameters are applied, it centers the declared feature vector by its sample mean and rescales it by the regularized sample standard deviation:

$$x \mapsto \frac{x - \mu(x)}{\sqrt{\sigma^2(x) + \varepsilon}}.$$

It then applies learned coordinatewise scale and shift through γ and β .

This operation changes:

- geometry;
- scale;
- derivative structure;
- how residual and sublayer signals interact.

It should not be summarized as an unconditional exact invariance to arbitrary affine rescaling. In particular, the explicit $\varepsilon > 0$ term breaks exact scale invariance in general, and learned γ, β restore trainable coordinatewise scale and offset after normalization.

Later normalized-geometry chapters will ask whether some normalization machinery can be replaced by more intrinsic constraints.

For now, LayerNorm is part of the baseline object.

19.21 Post-LN ordering

The original Transformer applies residual addition and then LayerNorm around a sublayer.

Write attention sublayer $\mathcal{A}$.

Then

$$Y = \text{LN}(H + \mathcal{A}(H)).$$

For feed-forward sublayer $\mathcal{F}$,

$$H' = \text{LN}(Y + \mathcal{F}(Y)).$$

This is a post-LN arrangement.

19.22 Pre-LN ordering

A common later design moves normalization before the residual branch:

$$Y = H + \mathcal{A}(\text{LN}(H)),$$

$$H' = Y + \mathcal{F}(\text{LN}(Y)).$$

Xiong and colleagues analyze pre-LN versus post-LN Transformer behavior and optimization differences (Xiong et al. 2020).

The Atlas uses that work only to support the architectural distinction and its training relevance.

It does not declare one ordering universally superior.

19.23 Pre-LN and post-LN are not notation variants

Compare:

$$\text{LN}(H + \mathcal{A}(H))$$

with

$$H + \mathcal{A}(\text{LN}(H)).$$

Normalization and nonlinear sublayer evaluation occur in different places.

The residual-stream state therefore evolves differently.

A chapter that says only “Transformer with LayerNorm” is underspecified.

19.24 Baseline pre-LN block

For later Atlas work, it is useful to display one common modern baseline:

$$Y = H + \text{MHA}(\text{LN}(H)),$$

$$H' = Y + \text{FFN}(\text{LN}(Y)).$$

This chapter uses that form for its main visual plate.

The choice is descriptive.

The original Transformer remains post-LN.

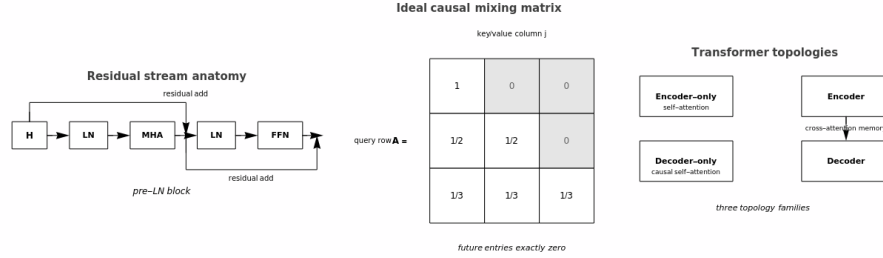


Figure 19.1: A three-panel baseline Transformer plate showing a pre-LN residual-stream block, the exact 3x3 causal mixing matrix, and encoder-only/decoder-only/encoder-decoder topology families.

19.25 Positional information is a separate axis

Attention without positional information is permutation-sensitive only through content, not sequence order in the intended way.

The original Transformer adds positional encodings (Vaswani et al. 2017).

Modern systems may instead use:

- learned absolute position;
- rotary position;
- relative bias;
- multidimensional schemes;
- operator-valued position.

The baseline object only requires that the positional mechanism be stated explicitly.

The Geometry of Position chapter develops this axis separately.

19.26 Encoder-only topology

An encoder-only Transformer applies self-attention and FFN blocks to an input sequence.

The attention pattern is not autoregressively causal by architectural necessity.

BERT is a representative encoder-only family (Devlin et al. 2019).

Its pretraining objectives and masking procedure are training/task choices layered on top of the architecture.

Encoder-only therefore does not mean:

every token always sees every other token under every possible masking policy.

Topology and task masking are separate.

19.27 Decoder-only topology

A decoder-only autoregressive Transformer uses causally masked self-attention.

Each position can depend on:

- itself;
- previous positions;
- not future positions under the ideal mask.

GPT-2 is a representative decoder-only autoregressive family (Radford et al. 2019).

Again, the source is used only to document topology.

Decoder-only is not a synonym for every language model.

19.28 Encoder–decoder topology

The original Transformer uses two stacks.

The encoder produces memory:

$$E = \text{Encoder}(X).$$

The decoder first uses masked self-attention over its generated prefix.

It then cross-attends to encoder memory.

For cross-attention:

$$Q = H_{\text{dec}}W_Q,$$

while

$$K = EW_K,$$

and

$$V = EW_V.$$

Decoder queries therefore read from encoder-derived keys and values.

19.29 Cross-attention is an interface

Encoder-decoder architecture creates an explicit computational boundary:

encoder state $\rightarrow$ decoder access.

Cross-attention is the read mechanism.

The interface can be interpreted using the same sufficiency questions developed earlier:

Did the encoder memory preserve what the decoder still needs?

This becomes especially important in multimodal and modular systems.

19.30 Three topologies, three dependency graphs

The topology families differ in which state can directly interact.

19.30.1 Encoder-only

Self-attention within one encoded sequence.

19.30.2 Decoder-only

Causally restricted self-attention over a prefix.

19.30.3 Encoder–decoder

Encoder self-attention, decoder causal self-attention, and decoder-to-encoder cross-attention.

These are graph differences.

They should not be collapsed into one generic phrase such as “a Transformer.”

19.31 Architecture versus training objective

A decoder-only Transformer can be trained with different objectives.

An encoder-only Transformer can be trained with different corruption/masking schemes.

An encoder–decoder Transformer can be used for different conditional tasks.

Therefore:

architecture $\neq$ training objective.

The distinction is necessary for interpreting experimental claims.

19.32 Architecture versus optimizer

The block equations do not determine:

- SGD versus Adam;
- learning rate;
- warmup;
- clipping;
- weight decay;
- batch size;
- precision;
- initialization.

These choices alter training behavior without changing the nominal architecture.

The First-Order Optimization chapter treats that machinery separately.

19.33 Architecture versus scale

Two systems can share architecture and differ dramatically in:

- depth;
- width;
- head count;
- context length;
- vocabulary;
- parameter count;
- training compute.

Calling both “Transformers” can be correct and still conceal most of the engineering difference.

The Atlas therefore uses architecture labels carefully.

19.34 Architecture versus tokenizer

A Transformer consumes token states.

How text becomes tokens is external to the Transformer block.

Different tokenizers can change:

- sequence length;
- morphology;
- multilingual fertility;
- information boundaries;
- effective context usage.

The Tokenization chapter will treat that interface separately.

19.35 The residual stream as the persistent state

The Architecture History chapter moved from static composition toward state transport.

The Transformer makes that transport explicit through repeated additive residual updates.

One useful lens is:

$$H_0 \rightarrow H_1 \rightarrow \cdots \rightarrow H_L.$$

Attention and FFN sublayers write corrections into this evolving state.

This residual-stream viewpoint will matter in:

- mechanistic analysis;
- split operators;
- routing;
- normalization geometry;
- adaptive depth.

19.36 But “residual stream” is not an ontological primitive

The residual stream is an architectural coordinate system.

Another architecture may store equivalent information through:

- recurrent state;
- external memory;
- operator state;
- sparse experts;
- multiple coupled streams.

The Atlas uses the residual stream as a baseline object, not as the unique natural representation of computation.

19.37 Attention as communication, FFN as local transformation

A useful first approximation is:

- attention: position-to-position communication;
- FFN: position-local nonlinear transformation.

This is not a complete mechanistic theory.

Attention changes state content.

FFNs can implement complex feature transformations.

Residual mixing across layers entangles the two.

Still, the decomposition is operationally useful.

19.38 Multi-head factorization creates multiple projected interaction spaces

Each head receives its own projected Q/K/V spaces.

Thus head h operates on

$$Q_h, K_h, V_h.$$

The outputs are later recombined.

This creates a structured factorization of the communication mechanism.

It does not guarantee that every head learns a cleanly separable function.

Later mechanistic chapters will test specialization rather than assume it.

19.39 Masking changes the operator support

A causal mask sets certain attention entries to zero after normalization.

A sparse mask can remove other edges.

A local-window mask restricts interaction distance.

Therefore masking changes the support of the mixing operator.

This will connect directly to sparse computation and structured attention.

19.40 Normalization placement changes the update law

In post-LN:

$$H \mapsto \text{LN}(H + S(H)).$$

In pre-LN:

$$H \mapsto H + S(\text{LN}(H)).$$

The residual state is therefore normalized at different points in the computation.

This becomes crucial when asking whether normalized geometry should be enforced intrinsically rather than repeatedly imposed.

19.41 Residual identity path is necessary context for depth

Because each block contains an additive identity path, a deep Transformer is not simply a composition of unrelated maps.

It is a sequence of corrections around a transported state.

This is one reason depth admits a dynamical interpretation.

But the earlier warning remains:

$$\text{identity path} \not\Rightarrow \text{automatic stability}.$$

The product of block Jacobians still matters.

19.42 The baseline object in one equation

A simplified pre-LN decoder-only block can be written:

$$Y_l = H_l + \text{MHA}_l \left(\text{LN}_l^{(1)}(H_l); M_{\text{causal}} \right),$$

$$H_{l+1} = Y_l + \text{FFN}_l \left(\text{LN}_l^{(2)}(Y_l) \right).$$

This is the reference object many later Atlas chapters will modify.

They may change:

- normalization;
- position;
- attention approximation;
- sparsity;
- expert routing;
- depth;
- residual transport;
- memory.

19.43 What the baseline does not contain

The equation above does not specify:

- tokenizer;
- embedding tying;
- bias conventions;
- dropout;
- activation function;
- attention kernel implementation;
- KV cache;
- sequence packing;
- distributed parallelism;
- optimizer;
- objective;
- data mixture.

Those are real system choices.

They are simply outside this baseline architectural abstraction.

19.44 Failure modes of vague Transformer language

19.44.1 “Attention model”

Too coarse.

It hides residual and FFN structure.

19.44.2 “Causal Transformer”

Still underspecified.

Normalization placement and topology remain open.

19.44.3 “Standard Transformer”

Often ambiguous between the 2017 post-LN encoder-decoder and later pre-LN decoder-only systems.

19.44.4 “Transformer layer”

Can refer to different arrangements of attention, cross-attention, FFN, normalization, and residuals.

The cure is equations.

19.45 Atlas connections

Attention as an Operator.

Supplies the deeper score/operator/value analysis.

Geometry of Position.

Treats position as a mathematical object rather than an embedding afterthought.

Normalized representations.

Questions whether LayerNorm-like machinery should be replaced by intrinsic constraints.

Split operators.

Treats attention and FFN/residual updates as composable evolution operators.

Mixture-of-experts.

Replaces or augments the FFN sublayer with routed conditional computation.

Mechanistic diagnostics.

Uses the residual stream, heads, keys, queries, values, and MLP channels as intervention sites.

Adaptive depth.

Questions whether every token/input requires the same number of block updates.

19.46 What would falsify a baseline claim?

A baseline statement should be architectural and checkable.

If we claim a mask is causal, inspect support.

If we claim a block is pre-LN, inspect equations.

If we claim an architecture is decoder-only, inspect whether encoder memory/cross-attention exists.

If we claim identity transport, zero the sublayer and verify state preservation.

If we claim position-wise FFN sharing, inspect parameter reuse across positions.

If the implementation violates the equations, the label is wrong.

19.47 Closing view

The Transformer is not one idea.

It is an assembly of:

- projected communication;
- normalized mixing;
- position-local nonlinear transformation;
- residual transport;
- normalization;
- masking;
- topology.

That assembly became a baseline because it is modular enough to vary and stable enough to serve as a reference point.

The Atlas now has a precise object to depart from.

From here, later chapters can ask sharper questions:

- What if normalization becomes geometry?
- What if position becomes an operator?
- What if depth becomes adaptive time?
- What if FFNs become routed experts?
- What if attention is approximated or split?
- What if persistent memory moves outside the weights?

Those questions only become meaningful once “the Transformer” stops being a vague noun and becomes an explicit computational object.

19.48 References used in this chapter

- (Vaswani et al. 2017)
- (Ba et al. 2016)
- (Xiong et al. 2020)
- (Devlin et al. 2019)
- (Radford et al. 2019)

See ‘sources/source-locks/ATLAS-CH-TRANSFORMER-001.yaml’ for exact source roles, internal Atlas provenance, and claim boundaries.

Chapter 20

Depth as Computational Time

Epistemic status: established adaptive-depth mechanisms plus Atlas synthesis and exact finite derivation.

Specification: `manuscript/specifications/ATLAS-CH-DEPTH-001.md`

Formal packet: `mathematics/derivations/ATLAS-CH-DEPTH-001-DERIVATIONS.md`

Computational witness: `mathematics/computational-witnesses/ATLAS-CW-DEPTH-001.md`

Source lock: `sources/source-locks/ATLAS-CH-DEPTH-001.yaml`

A network layer can be read as a transformation.

A stack of layers can also be read as a history.

That second reading changes what depth means.

Instead of asking only how many transformations the architecture contains, we can ask how much computation a representation has undergone before the system decides it is finished.

This is the computational-time view of depth.

It is useful precisely because it is weaker than a claim about physical time.

Depth need not be seconds.

It need not be the time variable of one hidden differential equation.

It is an ordered coordinate of computation.

Once that coordinate is explicit, fixed-depth networks, recurrent-depth networks, adaptive halting, equilibrium models, and conditional execution can be compared without pretending they are the same architecture.

20.1 Five notions of time

Several indices are routinely called “time” in machine learning.

They should not be merged.

Physical time measures duration in the external world.

Sequence time indexes positions or events in data.

Optimization time indexes parameter updates.

Network depth indexes transformations applied within one forward computation.

Computational time counts or weighs internal state transitions used to produce one result.

A recurrent model may have sequence time t and computational-depth index k at the same moment.

A training run adds optimization step n .

A deployed system adds wall-clock duration.

These coordinates interact, but they answer different questions.

The purpose of this chapter is to make the depth coordinate explicit enough that later chapters can reason about allocating it.

20.2 Fixed depth

Write a depth- L computation as

$$x_{k+1} = F_k(x_k), \quad k = 0, \dots, L-1.$$

Then

$$x_L = (F_{L-1} \circ \dots \circ F_0)(x_0).$$

The architecture commits in advance to L sequential transformations.

Different inputs may produce different activations, but the nominal transformation count is fixed.

This is the familiar layer-stack view.

The computational-time view does not reject it.

It notices that L is also a budget decision.

Every input is given the same number of depth transitions, whether it needs all of them or not.

That uniformity can be desirable.

It simplifies batching, latency prediction, and implementation.

It is also only one possible allocation rule.

20.3 Depth is not automatically physical time

The Numerics chapter gave us a disciplined way to interpret repeated updates.

A residual update can resemble a numerical step.

A tied recurrence can resemble repeated evolution under one operator.

Those analogies are useful.

But the index k does not become physical time merely because the equations resemble an integrator.

A network may change its operator with k .

Its step size may have no declared physical units.

Training may learn transformations with no claim of approximating a continuous process.

The Atlas therefore uses a careful phrase:

depth can serve as computational time.

That is an architectural interpretation.

A stronger claim that a particular network discretizes a particular differential equation requires additional evidence.

20.4 Recurrent depth

Now tie the transformation:

$$x_{k+1} = F(x_k).$$

One operator is reused across computational depth.

This separates parameter count from execution count.

Running F ten times does not require ten independently parameterized layers.

It also introduces a new question:

what happens if we keep going?

Nothing in weight tying answers that.

The sequence may converge.

It may oscillate.

It may diverge.

It may enter a cycle.

It may converge in some regions of state space and fail in others.

Recurrent depth therefore turns “more layers” into an iterative-dynamics question.

Universal Transformer is a representative example in which a transformation is applied recurrently across depth rather than assigning a wholly independent block to every depth position.

The important mechanism is repeated computation, not the product name.

20.5 A computation can stop itself

Fixed depth chooses L before inference.

Adaptive depth lets the execution decide when to stop.

Let h_k be a declared halting signal.

Let ϵ be a threshold.

Let $K_{\max}$ be the largest permitted depth under the current compute budget.

Define the bounded stopping depth

$$\tau = \min(\{k \in \{0, \dots, K_{\max}\} : h_k \leq \epsilon\} \cup \{K_{\max}\}).$$

The realized output is x_τ . The termination status must also be retained:

- `criterion_met` if $h_\tau \leq \epsilon$;
- `budget_exhausted` otherwise.

This equation separates two ideas.

The recurrence determines how state changes.

The stopping rule determines how long the recurrence is allowed to run.

That separation is essential.

A poor halting rule can stop a good recurrence too early.

A good halting rule cannot rescue a recurrence that moves in the wrong direction.

And a halting score is not automatically an error estimate merely because training learns it.

Graves's Adaptive Computation Time is a concrete mechanism for learning variable numbers of recurrent steps.

Universal Transformer adds a dynamic per-position halting mechanism to recurrent depth.

These sources show that learned variable computation is an implementable mechanism.

They do not show that learned halting always tracks task difficulty correctly.

20.6 An exact depth-as-error witness

Consider the scalar recurrence

$$x_{k+1} = \frac{x_k + 2}{2}, \quad x_0 = 0.$$

The fixed point is 2.

Subtract it:

$$x_{k+1} - 2 = \frac{1}{2}(x_k - 2).$$

Hence

$$x_k = 2(1 - 2^{-k}), \quad |x_k - 2| = 2^{1-k}.$$

Depth now has a literal approximation meaning for this one system.

At depth 1, the error is 1.

At depth 2, it is 1/2.

At depth 3, it is 1/4.

At depth 5, it is 1/16.

For tolerance epsilon in (0,2), the minimum depth satisfying the tolerance is

$$\tau(\varepsilon) = \left\lceil \log_2 \left(\frac{2}{\varepsilon} \right) \right\rceil.$$

The transformation did not change.

The requested accuracy changed how long we computed.

This is the simplest useful model of adaptive computational time.

It is also deliberately narrow.

Most learned networks do not hand us an exact error formula.

The later Adaptive Depth as Error Control chapter exists precisely because learned stopping becomes much more interesting when h_k must stand in for unknown task error.

20.7 Compute is more than step count

Suppose two systems both take four depth transitions.

They need not cost the same amount.

One may use a small operator at every step.

Another may invoke attention over a long sequence.

One may reuse cached state.

Another may move large tensors through memory.

One may parallelize internal work.

Another may be sequential.

So define realized computational cost more carefully.

For one declared resource unit,

$$C(x_0) = \sum_{k=0}^{\tau-1} c_k(x_k),$$

where c_k records the cost of the executed transition in that unit.

If several heterogeneous resources are tracked, use a vector such as

$$C_{\text{vec}} = (C_{\text{FLOPs}}, C_{\text{memory}}, C_{\text{energy}}, C_{\text{latency}}).$$

rather than adding unlike units without a declared scalarization.

The integer τ tells us sequential depth.

It does not by itself tell us wall-clock latency.

This becomes especially important for conditional computation.

Skipping half the mathematical blocks is not useful as a speed claim unless the implementation actually avoids the relevant work and the hardware can exploit the irregular execution.

20.8 Equilibrium depth

Recurrent depth suggests taking more and more iterations.

Equilibrium models make a different move.

Instead of declaring an explicit number of layers, define the representation by a fixed-point equation:

$$x_{\star} = F(x_{\star}).$$

The computational problem becomes:

find x^* .

Deep Equilibrium Models are a representative neural architecture built around this idea.

The phrase “infinite depth” is evocative but must be handled carefully.

The model does not need to store an actually infinite sequence of layer activations.

It solves a fixed-point/root-finding problem.

And the equilibrium equation does not prescribe one solver.

Naive fixed-point iteration

$$x_{k+1} = F(x_k)$$

is one possibility.

Other root-finding methods can reach the same equation through different computational trajectories.

This distinction matters because an equilibrium can exist while naive iteration fails to converge.

Definition of a fixed point and convergence of a solver are different claims.

20.9 Convergence is an architectural question only after assumptions

For the exact witness, F is a contraction.

Every step halves the distance to the fixed point.

That gives a clean convergence story.

A learned recurrent transformation need not have that property.

The local Jacobian may expand in some directions.

The operator may have multiple fixed points.

A solver may depend on initialization.

A stopping test may declare convergence while task-relevant quantities remain wrong.

Equilibrium depth therefore does not abolish numerical analysis.

It makes numerical analysis part of the architecture.

The solver, tolerance, maximum iterations, residual test, initialization, and failure handling are all computational choices.

20.10 Conditional depth

Adaptive halting asks when to stop a sequential computation.

Conditional execution asks which blocks to run at all.

For a residual-style block, write

$$x_{k+1} = x_k + g_k(x_k)\Delta_k(x_k).$$

With a hard gate

$$g_k \in \{0, 1\},$$

the block is either applied or skipped.

If $g_k=0$ and the implementation respects that decision before evaluating Δ_k , the block's expensive transform can be avoided.

SkipNet is a representative architecture of this kind: an input-dependent routing policy skips residual blocks.

This creates input-dependent effective depth.

Two examples entering the same nominal network can traverse different numbers of transformations.

20.11 Soft gates are not free skips

During training, it is often convenient to relax a binary gate:

$$g_k \in [0, 1].$$

Then the update can interpolate continuously between carry and transform.

But the computational claim changes.

If the system must compute $\Delta_k(x_k)$ before multiplying it by a small g_k , then the expensive operation was not skipped.

A soft coefficient near zero is mathematical sparsity of influence.

It is not necessarily computational sparsity of execution.

This distinction is easy to lose when a paper reports sparse weights, sparse gates, or small activation coefficients.

The hardware question is concrete:

which operations were never executed?

That boundary is handed directly to the later Conditional Computation chapter.

20.12 Effective depth is input-dependent execution history

For hard gating, define the active block set

$$\mathcal{A}(x_0) = \{k : g_k(x_k) = 1\}.$$

A simple realized block cost is

$$C_{\text{block}}(x_0) = \sum_{k \in \mathcal{A}(x_0)} c_k,$$

plus the overhead required to make the routing decisions.

The path is a function of the evolving state.

This means conditional depth can create system effects absent from a static layer count.

Different examples may have different latency.

A batch may wait for its slowest member.

Branch divergence may reduce hardware utilization.

Routing decisions themselves cost compute.

The mean number of active blocks is therefore not the same quantity as end-to-end throughput.

20.13 Parameter depth and execution depth

A weight-tied recurrent system may have few unique parameters and many execution steps.

A large feed-forward system may have many unique parameters and one pass through each.

An equilibrium model may have a compact parameterization but a variable number of solver iterations.

A conditional network may contain many blocks but execute only some of them for one input.

This suggests at least three separate depth coordinates:

architectural depth: how many transformation slots are represented in the nominal graph;

parameter depth: how many distinct parameterized transformations occur along the path;

execution depth: how many sequential transformations are actually evaluated for this input.

They coincide in a plain feed-forward stack.

They separate in recurrent, equilibrium, and conditional systems.

20.14 Halting is a decision under uncertainty

An adaptive system rarely knows the true remaining task error.

It sees proxies.

A confidence score.

A residual norm.

A change in representation.

A learned halting logit.

A budget counter.

A task-specific certificate.

The stop decision therefore has two possible errors.

Stop too early and computation is insufficient.

Continue too long and resources are wasted.

This is the computational analogue of a sequential decision problem.

But this chapter does not yet equate adaptive halting with numerical error control.

A learned halting logit may correlate with difficulty without estimating a rigorous truncation error.

The next numerical chapter will ask what additional structure is needed before a stopping signal can be interpreted as an error-control mechanism.

20.15 More depth can make things worse

The computational-time picture can tempt us into a monotone story:

more time, better answer.

That is false in general.

An unstable recurrence can amplify error.

Repeated application can oversmooth or collapse distinctions.

A learned iterative process can move away from a useful intermediate state.

A conditional controller can route difficult inputs into pathological loops.

An equilibrium solver can fail its tolerance or reach an unintended root.

More computation is only useful when the dynamics and stopping semantics support it.

Depth is a resource coordinate.

It is not a quality score.

20.16 Budget exhaustion is a first-class outcome

Suppose execution terminates at the finite budget cap $K_{\max}$ before the declared criterion $h_k \leq \epsilon$ is met.

The termination status is then `budget_exhausted` rather than `criterion_met`.

If the budget limit is reached first, the system has not necessarily converged.

It has terminated because resources ended.

That outcome should be distinguishable from successful halting.

A production system may still return its best current state.

A scientific report should not label that return “converged” unless the declared criterion was met.

This sounds obvious.

It becomes less obvious when adaptive computation is buried inside a large model and the maximum-step cap is treated as an implementation detail.

The cap is part of the semantics.

20.17 Training and inference can disagree

Dynamic-depth mechanisms often use relaxations, penalties, or stochastic estimators during training.

Deployment may use hard thresholds.

A training objective may reward expected compute.

Hardware may care about tail latency.

A controller may learn under one maximum depth and be deployed under another.

Thus there are two distinct questions:

did the learning procedure optimize the intended depth policy?

does the deployed execution realize the intended compute behavior?

The first is an optimization question.

The second is a systems question.

A complete conditional-compute story requires both.

20.18 What recurrent, adaptive, equilibrium, and conditional depth share

These mechanisms look different.

Their common object is a computational path whose length or structure is no longer just the static count of named layers.

Recurrent depth asks how many times to reuse an operator.

Adaptive depth asks when to stop.

Equilibrium depth asks what state satisfies a terminal fixed-point condition and how to solve for it.

Conditional depth asks which transformations to execute.

All four make computation allocation explicit.

That is why they belong in one chapter.

20.19 What the next chapters may assume

ATLAS-CH-ADAPTDEPTH-001 — Adaptive Depth as Error Control may assume:

- depth as computational time;
- fixed versus adaptive stopping;
- the stopping variable τ ;
- exact distinction between a stopping signal and true error;
- fixed-point/equilibrium semantics;
- resource-aware compute accounting.

Its task is to add numerical error estimation and adaptive-step logic.

ATLAS-CH-SPARSE-001 — Conditional Computation may assume:

- hard and relaxed gates;
- active execution sets;
- realized path cost;
- input-dependent effective depth;
- the distinction between small soft weights and actually skipped operations.

Its task is to extend this into sparse activation, token selection, routing, and hardware-aware conditional execution.

20.20 The boundary to remember

Depth is often drawn vertically.

The Atlas needs a second picture.

Imagine depth horizontally as a computational trajectory.

A representation enters at $k=0$.

Each transition changes it.

A fixed-depth system stops because the architecture says L .

An adaptive system stops because a rule says enough.

An equilibrium system stops because a solver says the fixed-point criterion is met.

A conditional system changes which transitions exist on the realized path.

This picture does not prove that neural computation is continuous time.

It does something more practical.

It turns depth from a static architectural count into a resource that can be allocated, reused, skipped, or terminated under explicit rules.

Once depth is treated that way, adaptive computation becomes a question of dynamics, numerics, and systems design rather than a synonym for “deeper network.”

20.21 References used in this chapter

The exact source identities and authority scopes are pinned in:

`sources/source-locks/ATLAS-CH-DEPTH-001.yaml`

The external mechanism basis includes Graves’s Adaptive Computation Time, Universal Transformers, Deep Equilibrium Models, and SkipNet. The numerical and architectural interpretation is grounded in the audited Atlas Numerics and Architecture History prerequisites.

Chapter 21

Split-Operator Networks

Epistemic status: established operator-splitting theory + Atlas architectural synthesis.

A neural block can contain the same named ingredients and still compute a different map when their order changes.

That sounds obvious when stated in ordinary language. It becomes more consequential when the block is described as if attention, an MLP, normalization, gating, transport, or another update were merely ingredients in a bag. They are not. They are transformations applied to state. A later transformation sees the state produced by the earlier one.

The mathematics of operator splitting gives us a disciplined way to talk about this fact.

It also gives us a trap.

Classical splitting theory studies compositions of flows or operators under explicit assumptions. A Transformer sublayer is not automatically one of those exact flows. If we borrow the notation without preserving the assumptions, an architectural metaphor quietly turns into a false numerical theorem.

This chapter keeps the useful part and blocks the overclaim.

The central lesson is:

When subcomputations do not commute, ordering is part of the architecture. Classical splitting theory can explain that structure exactly in declared reference systems, but learned neural maps inherit its convergence orders only when the required flow and regularity assumptions are actually established.

The numerical substrate comes from the audited Numerics chapter and standard splitting references (McLachlan and Quispel 2002; Hairer et al. 2006). The baseline neural anatomy comes from the audited Transformer chapter and the original Transformer architecture (Vaswani et al. 2017).

21.1 The smallest place order can matter

Suppose a state x is acted on by two transformations.

Call them A and B .

There are at least three different things one might mean by “use both.”

One is an additive update:

$$x^+ = x + F_A(x) + F_B(x).$$

Another is sequential composition:

$$x_1 = \Psi_A(x), \quad x^+ = \Psi_B(x_1).$$

A third is the reversed sequence:

$$\tilde{x}_1 = \Psi_B(x), \quad \tilde{x}^+ = \Psi_A(\tilde{x}_1).$$

These are different constructions.

Even when the two transformations are small residual increments, the sequential map contains cross terms that the additive map does not.

Take the linear residual increments

$$F_A(x) = hAx, \quad F_B(x) = hBx.$$

The additive update is

$$x^+ = (I + h(A + B))x.$$

If A acts first and B acts on the resulting state,

$$x^+ = (I + hB)(I + hA)x,$$

so

$$x^+ = (I + h(A + B) + h^2 BA) x.$$

The $h^2 BA$ term is not decoration. It records that B saw a state already changed by A .

Reverse the order and the cross term becomes $h^2 AB$.

This is the first split-operator fact worth carrying into neural architecture design:

Sequential composition creates interaction terms that depend on order.

21.2 A telescope with two lenses

A useful picture is a telescope with two different lenses.

Passing light through lens A and then lens B need not produce the same transformation as B then A. If the lenses implement transformations that commute, the order does not matter. If they do not commute, the ordering is part of the instrument.

The analogy maps cleanly onto operator composition:

- the incoming light corresponds to the current state;
- each lens corresponds to one state transformation;
- lens order corresponds to composition order;
- order independence corresponds to commutation.

The analogy stops there.

A neural attention sublayer is not literally an optical lens, and this picture does not prove numerical convergence, reversibility, or stability. Its job is only to make one structural point visible before we formalize it.

21.3 The exact reference problem

Start with the declared linear system

$$\dot{x} = (A + B)x,$$

where A and B are constant matrices.

The exact combined flow over a step h is

$$e^{h(A+B)}.$$

If we can solve the two parts separately, we can form the Lie-Trotter compositions

$$S_{AB}(h) = e^{hA}e^{hB},$$

and

$$S_{BA}(h) = e^{hB}e^{hA}.$$

The first means one exact B -subflow and one exact A -subflow in the composition order encoded by the matrix product. The second reverses them.

Expanding both exponentials gives

$$e^{hA}e^{hB} = e^{h(A+B)} + \frac{h^2}{2}[A, B] + O(h^3),$$

where

$$[A, B] = AB - BA.$$

The reversed order gives

$$e^{hB}e^{hA} = e^{h(A+B)} - \frac{h^2}{2}[A, B] + O(h^3).$$

So the first commutator is the leading object that records noncommutation.

If

$$[A, B] = 0,$$

the two constant matrix exponentials commute and

$$e^{hA}e^{hB} = e^{hB}e^{hA} = e^{h(A+B)}$$

exactly.

This is not a vague statement that “order sometimes matters.” It tells us what object measures the leading order difference in the declared smooth finite-dimensional setting.

21.4 The exact two-by-two witness

Use

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}.$$

Both are nilpotent:

$$A^2 = B^2 = 0.$$

Therefore

$$e^{hA} = I + hA, \quad e^{hB} = I + hB.$$

Their products are

$$AB = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad BA = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix},$$

so

$$[A, B] = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

The two Lie steps are exactly

$$S_{AB}(h) = \begin{pmatrix} 1 + h^2 & h \\ h & 1 \end{pmatrix},$$

and

$$S_{BA}(h) = \begin{pmatrix} 1 & h \\ h & 1 + h^2 \end{pmatrix}.$$

Subtract:

$$S_{AB}(h) - S_{BA}(h) = h^2[A, B].$$

At $h = 1/2$,

$$S_{AB} = \begin{pmatrix} 5/4 & 1/2 \\ 1/2 & 1 \end{pmatrix},$$

while

$$S_{BA} = \begin{pmatrix} 1 & 1/2 \\ 1/2 & 5/4 \end{pmatrix}.$$

The two computations contain the same two exact subflows. They differ only in order. Yet the result is different.

That is the architectural fact we want.

21.5 The combined flow is not either Lie step

For this witness,

$$A + B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix},$$

and

$$(A + B)^2 = I.$$

Hence

$$e^{h(A+B)} = \begin{pmatrix} \cosh h & \sinh h \\ \sinh h & \cosh h \end{pmatrix}.$$

At $h = 1/2$, neither Lie ordering equals this exact combined flow.

Their Frobenius errors are equal in this symmetric witness, approximately

$$0.1793148494.$$

The equality of those two error norms is incidental to this example. The important structural result is the signed leading commutator term.

This distinction matters when translating the picture into neural networks. The fact that an ordered block is a useful decomposition does not imply that it is the exact flow of some sum operator.

21.6 Symmetry cancels the quadratic defect

A standard symmetric composition is Strang splitting:

$$S_{ABA}(h) = e^{hA/2} e^{hB} e^{hA/2}.$$

For sufficiently regular problems, Strang splitting has a local defect of order $O(h^3)$, corresponding to second-order global accuracy under the usual numerical-analysis assumptions (McLachlan and Quispel 2002; Hairer et al. 2006).

Our witness shows the cancellation directly.

Because $A^2 = B^2 = 0$,

$$S_{ABA}(h) = \begin{pmatrix} 1 + h^2/2 & h + h^3/4 \\ h & 1 + h^2/2 \end{pmatrix}.$$

The exact combined-flow series is

$$e^{h(A+B)} = \begin{pmatrix} 1 + h^2/2 + h^4/24 + O(h^6) & h + h^3/6 + O(h^5) \\ h + h^3/6 + O(h^5) & 1 + h^2/2 + h^4/24 + O(h^6) \end{pmatrix}.$$

The quadratic mismatch has disappeared.

At $h = 1/2$,

$$S_{ABA} = \begin{pmatrix} 9/8 & 17/32 \\ 1/2 & 9/8 \end{pmatrix},$$

whose Frobenius error against the exact flow is approximately

$$0.02370487547.$$

This one number is not the reason Strang is second order. The series derivation is the reason. The numeric checkpoint only witnesses the declared finite case.

21.7 A commuting control

Now choose

$$A_c = \text{diag}(1, 2), \quad B_c = \text{diag}(3, 4).$$

Then

$$[A_c, B_c] = 0.$$

Because the matrices commute,

$$e^{hA_c}e^{hB_c} = e^{hB_c}e^{hA_c} = e^{h(A_c+B_c)}$$

for every h .

This control is important.

Without it, the chapter could leave the impression that every composition order must differ. The correct statement is narrower:

Noncommutation creates order dependence. Commutation removes it in the exact setting.

The question for a learned architecture is therefore not “does order matter in principle?” but “what are the actual submaps, and how strongly do they fail to commute on the states that matter?”

That later empirical question is not answered by this chapter.

21.8 From exact flows to neural sublayers

Return to the Transformer residual stream.

Let H be the current residual-stream state. A simplified pre-normalized pair of learned submaps might be written as

$$\Psi_A(H) = H + F_A(N_A(H)),$$

$$\Psi_B(H) = H + F_B(N_B(H)).$$

Here F_A might stand for attention-like computation and F_B for an FFN-like computation.

The ordered block is then

$$H' = (\Psi_B \circ \Psi_A)(H).$$

The reversed block is

$$\tilde{H}' = (\Psi_A \circ \Psi_B)(H).$$

In general,

$$H' \neq \tilde{H}'.$$

Why?

Because the second submap receives a different input state.

This remains true even before we invoke any continuous-time interpretation.

The split-operator lens is therefore useful at two levels.

At the exact mathematical level, it gives the established theory of flows, commutators, Lie products, symmetric compositions, and splitting error.

At the architectural level, it gives a disciplined language for asking how named subcomputations are isolated and ordered.

Those two levels must not be collapsed.

21.9 Normalization belongs inside the submap

A common source of false simplification is to write “attention operator A ” and “MLP operator B ” while quietly omitting normalization.

In a pre-LN block, the actual maps look more like

$$H_1 = H + \text{Attn}(\text{LN}(H)),$$

$$H_2 = H_1 + \text{FFN}(\text{LN}(H_1)).$$

If we call these two stages Ψ_A and Ψ_B , then LayerNorm is part of the maps.

It affects the state dependence.

The same rule applies to:

- masking;
- gating;
- clipping;
- routing;
- residual scaling;
- stochastic dropout when active;
- state-dependent normalization;
- data-dependent control flow.

A commutator or order argument that drops these operations is analyzing a different system unless an explicit approximation justifies the omission.

21.10 Symmetry is not exact reversibility

The word “symmetric” is dangerous because it has several meanings.

Strang composition is symmetric in the step sequence:

$$A/2 \rightarrow B \rightarrow A/2.$$

For exact flows, this time-symmetric composition has important numerical consequences.

To even write a neural analogue of that sequence, one must first define a step-parameterized family of learned maps

$$\Psi_A(h), \quad \Psi_B(h),$$

with a meaningful half-stage $\Psi_A(h/2)$. Only then can one form

$$\Psi_A(h/2) \circ \Psi_B(h) \circ \Psi_A(h/2).$$

For a generic learned block Ψ_A with no declared step parameter, the symbol “half of A” is not defined by the architecture. Halving a residual coefficient or duplicating a block is a new design choice, not automatically the classical half-flow.

Even when a valid step-parameterized family is declared, the resulting neural palindromic block is not automatically invertible.

If Ψ_A or Ψ_B loses information, clips state, normalizes non-injectively, uses non-invertible activation, drops tokens, or otherwise fails to be bijective on the relevant domain, the palindromic ordering does not repair that.

So we must distinguish:

- symmetric ordering;
- self-adjoint numerical composition under a declared flow model;
- exact invertibility;
- practical reconstructability;
- reversible-network implementation.

These are related ideas, not synonyms.

21.11 Shared operators and changing operators

Classical notation often suggests one fixed pair A, B .

Neural networks often do not behave that way.

Layer k may use

$$A_k, \quad B_k,$$

with different learned parameters at every depth.

Then the architecture is better viewed as stage-dependent or nonautonomous.

A commutator such as

$$[A_k, B_k]$$

may still be meaningful in a local linearized model, but it does not become one global autonomous commutator for the entire network.

This distinction separates two regimes:

21.11.1 Shared operators

The same submaps recur across depth.

This is structurally closer to repeated stepping of one declared dynamical system.

21.11.2 Layer-varying operators

Each stage has different submaps.

This is closer to a time-dependent system or a general composition chain.

Both can be studied with operator language. Their mathematical contracts differ.

21.12 Splitting error is only one error

Suppose a network is motivated by splitting a reference evolution.

Even then, “the error” is not one scalar object.

At minimum distinguish:

1. **splitting/discretization error:** difference between a numerical composition and the declared reference flow;
2. **approximation error:** learned submaps may not represent the intended component dynamics exactly;
3. **estimation error:** finite data limits what can be learned;
4. **optimization error:** training may not reach the intended parameter solution;
5. **stochastic variation:** minibatching, initialization, routing, or noise may change outcomes;

6. **implementation error:** finite precision, kernels, masking details, and system behavior may differ from the mathematical model.

Classical Lie or Strang order controls the first category under its assumptions.

It does not automatically control the others.

This is the same epistemic discipline used throughout the Atlas: one support route does not acquire authority over neighboring claims merely because the notation is convenient.

21.13 Ordering as an architectural degree of freedom

With those boundaries in place, the design lesson becomes stronger, not weaker.

A network designer can ask:

- Which transformations should be isolated as distinct submaps?
- Which state does each submap receive?
- Should they be applied sequentially, additively, or in a symmetric composition?
- Are parameters shared across depth?
- Is there a declared continuous reference problem?
- Is the architecture trying to preserve a structure?
- Where can noncommutation be measured?
- Does reversing order alter useful behavior?
- Is a small local commutator associated with robustness, stability, or transfer?
- When does symmetric composition help, and when is it merely extra compute?

These are empirical and mathematical questions.

The splitting framework organizes them without answering them in advance.

21.14 A Transformer block through the split lens

A baseline Transformer already presents an ordered pair of major subcomputations: attention and position-wise feed-forward transformation (Vaswani et al. 2017).

In a pre-LN form,

$$H_1 = H + \text{Attn}(\text{LN}(H)),$$

$$H_2 = H_1 + \text{FFN}(\text{LN}(H_1)).$$

A split-operator reading does not claim that these are exponentials of two known generators.

It asks instead:

What changes if we treat these as two named state transformations whose order and composition law are explicit design choices?

That question exposes several architecture variants immediately:

- attention then FFN;
- FFN then attention;
- additive parallel updates from the same input state;
- explicitly step-parameterized palindromic attention-like / FFN-like compositions, when a meaningful half-step family is actually defined;
- shared repeated submaps;
- depth-varying submaps;
- submaps with measured or constrained local interaction.

Whether any of these is better is an experimental question.

The Atlas does not promote an architectural rearrangement from mathematical elegance alone.

21.15 What a neural commutator could mean

For linear operators, the commutator is unambiguous:

$$[A, B] = AB - BA.$$

For nonlinear maps, several related objects are possible.

One can compare compositions directly:

$$\Psi_B(\Psi_A(x)) - \Psi_A(\Psi_B(x)).$$

One can also differentiate the nonlinear composition defect. If

$$C_\Psi(x) = \Psi_B(\Psi_A(x)) - \Psi_A(\Psi_B(x)),$$

then, when the maps are differentiable,

$$DC_\Psi(x) = J_B(\Psi_A(x))J_A(x) - J_A(\Psi_B(x))J_B(x).$$

A same-state algebraic Jacobian commutator

$$J_B(x)J_A(x) - J_A(x)J_B(x)$$

can still be used as a local diagnostic, but it is generally not the derivative of C_Ψ . Intermediate-state evaluation matters.

One can work with vector-field Lie brackets when actual differentiable vector fields have been declared.

These are not interchangeable.

A future diagnostic chapter may define a specific operational noncommutation probe. SPLIT-001 only establishes why such a probe could be informative and what mathematical distinctions it must preserve.

21.16 Failure modes

21.16.1 Calling every residual block an integrator

A residual form

$$x_{k+1} = x_k + F_k(x_k)$$

resembles an Euler step.

Resemblance is not identity.

Without a declared reference vector field, step-size semantics, and refinement family, “integrator” is an interpretation rather than a convergence theorem.

21.16.2 Treating Strang order as architectural magic

A palindromic block can be aesthetically attractive.

The $O(h^3)$ local defect belongs to the classical splitting setting under the required assumptions. It does not automatically apply to arbitrary learned nonlinear sub-layers. A neural half-step must itself be defined by a declared step-parameterized family before Strang language has mathematical content.

21.16.3 Ignoring normalization

If normalization is present in the executed block, dropping it from the operator model changes the object.

21.16.4 Confusing symmetry and invertibility

A symmetric sequence of non-invertible maps is still non-invertible.

21.16.5 Measuring one local commutator and claiming global control

A small commutator at one state, one layer, or one linearization does not establish global near-commutation.

21.16.6 Folding training error into splitting error

Optimization failure is not a discretization defect.

21.16.7 Assuming shared-operator formulas for layer-varying networks

A different A_k, B_k at every layer is a different mathematical object from repeated application of fixed A, B .

21.17 What later chapters may now assume

The downstream Composition chapter may assume:

- additive and sequential composition are distinct;
- operator ordering can be architecturally load-bearing;
- the commutator is the leading finite-dimensional Lie-order object in the declared exact setting;
- symmetric composition has stronger cancellation properties under classical assumptions;
- neural submaps must include their actual normalization, routing, masking, and residual semantics.

The downstream Transport chapter may assume:

- split computation can be interpreted as staged transport only when the stage maps are explicitly declared;
- an exact-flow interpretation is stronger than a generic residual-composition interpretation;
- layer-varying stages require nonautonomous semantics.

Neither chapter may inherit a claim that Transformer sublayers are exact flows or that Strang-style neural blocks are automatically second order.

21.18 Epistemic status

Established theory:

- Lie-Trotter and Strang splitting;
- commutator-controlled order effects in the appropriate operator setting;

- composition and structure-preserving numerical methodology (McLachlan and Quispel 2002; Hairer et al. 2006);
- baseline Transformer sublayer anatomy (Vaswani et al. 2017).

Atlas-owned derivation:

- the exact nilpotent two-matrix witness;
- the additive-versus-sequential residual comparison;
- the explicit claim firewall connecting operator-splitting theory to neural architecture.

Computational witness:

- exact matrix products;
- exact commutator;
- exact Lie and Strang formulas;
- bounded $h = 1/2$ error values;
- commuting control.

Not established here:

- empirical superiority of split-operator neural architectures;
- small commutators in trained Transformers;
- exact flow semantics for attention or FFN sublayers;
- global reversibility from symmetric ordering;
- general convergence guarantees for learned split networks.

That boundary is deliberate.

The value of the split-operator lens is not that it turns a neural network into a numerical theorem.

It is that it forces us to say what the transformations are, what order they act in, what state each sees, and which mathematical claims survive the translation.

21.19 References used in this chapter

- (McLachlan and Quispel 2002) for Lie-Trotter/Strang splitting, commutators, composition, and order conditions.
- (Hairer et al. 2006) for geometric/composition-method boundaries and structure-preserving semantics.
- (Vaswani et al. 2017) for baseline Transformer attention/FFN residual-block anatomy.

Exact provenance and authority boundaries are locked in:

`sources/source-locks/ATLAS-CH-SPLIT-001.yaml`.

Chapter 22

Representation as Transport

Epistemic status: established geometry and dynamics + audited prerequisite inheritance + Atlas synthesis.

Specification: `manuscript/specifications/ATLAS-CH-TRANSPORT-001.md`

Derivation packet: `mathematics/derivations/ATLAS-CH-TRANSPORT-001-DERIVATIONS.md`

Computational witness: `mathematics/computational-witnesses/ATLAS-CW-TRANSPORT-001.md`

Source lock: `sources/source-locks/ATLAS-CH-TRANSPORT-001.yaml`

A representation does not merely exist. In a deep adaptive system it is repeatedly changed.

The same hidden state may be:

- mixed by attention;
- transformed by a feed-forward map;
- normalized;
- gated;
- routed;
- projected;
- constrained;
- carried through a residual path;
- or updated by a continuous-depth evolution law.

The useful question is therefore not only:

What is the representation?

It is also:

How is the representation moved?

This chapter develops a disciplined answer.

The word **transport** will mean the evolution of a declared representation state through declared maps.

That definition is deliberately broad enough to cover finite neural compositions and constrained updates, but deliberately narrow enough to block several false identifications.

Representation transport is not automatically:

- optimal transport of probability mass;
- parallel transport under a connection;
- vector transport of tangent data;
- geodesic motion;
- an exact ODE flow.

Those objects may enter a model when their assumptions are actually present. They are not consequences of the word “transport.”

22.1 The primary object is a sequence of maps

Let the representation state at stage k be

$$z_k \in \mathcal{Z}_k.$$

A stage map is

$$\Phi_k : \mathcal{Z}_k \rightarrow \mathcal{Z}_{k+1}.$$

The update is

$$z_{k+1} = \Phi_k(z_k).$$

After K stages,

$$z_K = (\Phi_{K-1} \circ \cdots \circ \Phi_0)(z_0).$$

This composition is exact as written.

It does not require:

- an ODE;
- an infinitesimal step;
- a manifold;
- invertibility;
- a metric;
- a probability distribution.

That is the first transport object of the Atlas:

a finite representation trajectory is the ordered composition of the maps that actually execute.

This sounds elementary. It is also where many conceptual mistakes begin.

If the executed map contains normalization, masking, routing, clipping, stochasticity, or projection, those operations belong to Φ_k . Omitting them means analyzing another system.

22.2 Residual transport

A residual stage has the familiar form

$$\Phi_k(z) = z + F_k(z).$$

The identity term gives a direct state path. The learned residual modifies it.

Residual networks made this identity-plus-increment structure an explicit architecture (He et al. 2016).

At the discrete level, the statement is exact:

$$\text{next state} = \text{current state} + \text{learned increment}.$$

The dynamical interpretation is useful, but it must come second.

If we instead write

$$\Phi_{k,h}(z) = z + hF_k(z),$$

the algebra resembles an explicit Euler step.

That resemblance becomes a numerical statement only after a reference evolution law has been declared.

The equation

$$z_{k+1} = z_k + hF_k(z_k)$$

does not by itself tell us that one unique continuous vector field generated the network.

This is inherited doctrine from the audited Split-Operator and numerical chapters.

22.3 Continuous depth is a different declaration

Neural ODEs make a stronger object explicit:

$$\frac{dz}{dt} = f(z, t; \theta).$$

The architecture defines a continuous-depth evolution law and obtains finite-time state changes through the associated ODE solution and numerical integration (Chen et al. 2018).

This is a genuine continuous-time transport object.

A residual stack and a Neural ODE can be related.

They are not identical by definition.

The safe implication is:

$$\text{explicit ODE architecture} \Rightarrow \text{declared continuous state evolution.}$$

The unsafe implication is:

$$\text{residual stack} \Rightarrow \text{unique exact ODE.}$$

Haber and Ruthotto use dynamical-systems and discrete-ODE stability ideas to motivate deep architecture design (Haber and Ruthotto 2018). That bridge is valuable precisely because it makes assumptions visible. It should not be turned into an identity between all deep networks and one continuous flow.

22.4 Stage index is not automatically physical time

The index

$$k = 0, 1, \dots, K$$

orders computation.

It need not represent physical time.

Even when one speaks of “depth as time,” several meanings are possible:

- literal continuous time in an ODE;
- numerical time in a discretization;
- recurrent iteration count;
- transformer block index;

- optimization iteration;
- adaptive computational depth.

Transport language becomes useful when these are distinguished rather than blended.

A chapter that says “the representation moves through time” must say what the time variable is.

22.5 The constrained case

Suppose representation states lie on a manifold

$$M \subseteq \mathbb{R}^n.$$

Now the update has two separate questions:

1. what direction is proposed?
2. how is a legal endpoint on M produced?

At state z_k , a tangent proposal satisfies

$$\xi_k \in T_{z_k} M.$$

A retraction maps tangent data back to the manifold:

$$z_{k+1} = R_{z_k}(\xi_k).$$

For the unit sphere,

$$S^{n-1} = \{u : \|u\|_2 = 1\},$$

the tangent condition is

$$u^\top \xi = 0.$$

A standard normalized retraction is

$$R_u(\xi) = \frac{u + \xi}{\|u + \xi\|_2}.$$

This guarantees

$$\|R_u(\xi)\|_2 = 1.$$

It guarantees feasibility.

Nothing in that equation says the update is:

- geodesic;
- optimal;
- invertible;
- information preserving;
- dynamically stable;
- task improving.

The NORMREP prerequisite established exactly this geometry and the radial-information boundary (Lee 2018; Absil et al. 2008).

22.6 Feasible motion is not geodesic motion

Take the unit circle.

Let

$$u = (1, 0), \quad \xi = (0, 1).$$

The normalized retraction gives

$$R_u(\xi) = \frac{(1, 1)}{\sqrt{2}}.$$

This point is on the circle.

Its angular displacement from u is

$$\frac{\pi}{4}.$$

Now use the sphere exponential map with the same unit tangent vector at unit time.

The endpoint is

$$\text{Exp}_u(\xi) = (\cos 1, \sin 1).$$

Its angular displacement is

$$1.$$

Since

$$\frac{\pi}{4} \neq 1,$$

the endpoints differ.

This is the smallest exact warning we need:

returning to the manifold is not the same thing as following the manifold geodesic.

A retraction is often computationally convenient because it approximates the exponential map locally (Absil et al. 2008).

Approximation is not identity.

22.7 Moving the point and moving an arrow are different operations

The phrase “transport” is overloaded in geometry.

Suppose

$$v_0 \in T_{u_0}M.$$

After the base point moves from u_0 to u_1 , the same ambient coordinate vector need not belong to

$$T_{u_1}M.$$

This matters whenever a system carries local information such as:

- momentum;
- a tangent update;
- a local search direction;
- a basis;
- a differential perturbation;
- optimizer state defined intrinsically on a manifold.

The state update answers:

Where did the point move?

A vector-transport or parallel-transport rule answers another question:

How should tangent information be represented at the new point?

Those are not the same map.

22.8 Exact tangent-ownership witness

Take

$$u_0 = (1, 0), \quad v_0 = (0, 1).$$

Initially,

$$u_0^\top v_0 = 0,$$

so

$$v_0 \in T_{u_0} S^1.$$

Move the state by normalized retraction:

$$u_1 = \frac{(1, 1)}{\sqrt{2}}.$$

Now test the old tangent vector against the new base point:

$$u_1^\top v_0 = \frac{1}{\sqrt{2}}.$$

Therefore

$$v_0 \notin T_{u_1} S^1.$$

The same ambient coordinates are no longer legal tangent data.

The orthogonal projection onto the new tangent space is

$$P_{u_1} v_0 = v_0 - (u_1^\top v_0) u_1 = \left(-\frac{1}{2}, \frac{1}{2} \right).$$

This projected vector is tangent because

$$u_1^\top P_{u_1} v_0 = 0.$$

Projection is one possible local operation.

It is not automatically parallel transport.

It is not automatically the best vector transport for a learning algorithm.

The durable rule is simply:

tangent data belongs to a base point.

22.9 The local-compass allegory

Imagine moving across the surface of a globe while carrying a small arrow that lies flat against the surface.

Two things happen:

- your location changes;
- the meaning of “this arrow lies tangent here” changes with location.

Moving the location does not automatically tell you how to carry the arrow.

That is the geometric distinction between:

- state transport;
- tangent-data transport.

The analogy has a strict limit.

A neural representation space may not be a smooth manifold at all. Even when we constrain vectors to a sphere, the complete learned computation can include discrete routing, masking, non-smooth nonlinearities, and dimension-changing interfaces.

The allegory is useful only where the declared geometry supports it.

22.10 Path order can matter even when every endpoint is feasible

The Split-Operator prerequisite established that ordered maps need not commute.

Constrained transport adds another useful witness.

Start on

$$S^2$$

at

$$u_0 = (1, 0, 0).$$

Choose

$$a = (0, 1, 0), \quad b = (0, 0, 1).$$

Both are tangent at u_0 .

Use normalized retraction

$$R_u(\xi) = \frac{u + \xi}{\|u + \xi\|}.$$

Apply a first:

$$u_a = \frac{(1, 1, 0)}{\sqrt{2}}.$$

The vector b remains tangent there, because

$$u_a^\top b = 0.$$

Apply the second stage:

$$u_{ab} = R_{u_a}(b) = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{\sqrt{2}}\right).$$

Reverse the order:

$$u_b = \frac{(1, 0, 1)}{\sqrt{2}},$$

then

$$u_{ba} = R_{u_b}(a) = \left(\frac{1}{2}, \frac{1}{\sqrt{2}}, \frac{1}{2}\right).$$

Both endpoints are feasible:

$$\|u_{ab}\|_2 = \|u_{ba}\|_2 = 1.$$

But

$$u_{ab} \neq u_{ba}.$$

The exact inner product is

$$u_{ab}^\top u_{ba} = \frac{1}{4} + \frac{1}{\sqrt{2}}.$$

Constraint preservation therefore does not erase path ordering.

This is not a universal theorem that every pair of constrained neural updates must fail to commute.

It is a finite counterexample to the opposite claim.

22.11 The executed maps determine the path

If the network executes

$$z_{k+1} = R_{z_k}(\xi_k(z_k)),$$

then both objects matter:

- the field or proposal ξ_k ;
- the endpoint map R .

Replacing the retraction changes the transport.

Replacing the order changes the transport.

Dropping normalization changes the transport.

Changing the base point before evaluating the next stage changes the transport.

This is the correct level at which architectural comparisons should begin.

22.12 State-dependent transport

Many neural maps are state-dependent.

Attention is the clearest example: the operator applied at one stage depends on the current state.

So a map such as

$$z_{k+1} = \Phi_k(z_k)$$

is generally not a fixed matrix multiplying every possible input.

Its differential at one point is

$$J_{\Phi_k}(z_k).$$

That Jacobian can describe local perturbation propagation.

It is another object again.

We must separate:

- the nonlinear state map;

- its local Jacobian;
- a tangent-space transport rule;
- a global flow, if one exists.

These objects can interact without being interchangeable.

22.13 Differential pushforward is not automatically vector transport

For a differentiable map,

$$\Phi : M \rightarrow N,$$

the differential maps tangent vectors by

$$D\Phi_z : T_z M \rightarrow T_{\Phi(z)} N.$$

This is a precise geometric construction when the required smooth manifold setting exists.

In ordinary neural calculations one often works instead with an ambient Jacobian

$$J_\Phi(z).$$

If constraints, quotient structure, projections, or non-smooth operations are present, the relation between the ambient Jacobian and an intrinsic tangent-space map must be stated.

Calling every Jacobian multiplication “vector transport” would erase these distinctions.

The Atlas will not do that.

22.14 Transport can lose information

A constrained path can remain perfectly feasible while destroying distinctions.

The simplest example is normalization:

$$N(x) = \frac{x}{\|x\|}.$$

For every

$$c > 0,$$

$$N(cx) = N(x).$$

The positive radial degree of freedom disappears.

If the original representation stored useful information in norm, the transport has erased it unless another channel carries that quantity.

Therefore:

feasible constrained transport $\nrightarrow$ information-preserving transport.

This is inherited directly from the NORMREP chapter.

22.15 Transport can also relocate information

Information removed from one coordinate channel can reappear elsewhere.

A normalized architecture may retain magnitude through:

- explicit scale parameters;
- residual amplitudes;
- gates;
- temperatures;
- separate scalar channels;
- logits;
- routing variables.

The right question is not whether normalization “destroys scale” in some absolute sense.

It is:

Which distinctions survive in the complete state carried forward?

Transport is therefore an interface-level concept.

A local geometric constraint does not describe the full system state unless the system state has actually been defined that way.

22.16 Shared and stage-dependent transport

Suppose every layer uses the same map:

$$\Phi_k = \Phi.$$

Then

$$z_K = \Phi^K(z_0).$$

This repeated-map structure supports one kind of dynamical interpretation.

Now suppose the parameters change at every layer:

$$\Phi_0, \Phi_1, \dots, \Phi_{K-1}.$$

Then the system is stage-dependent.

A continuous analogy, if one is introduced, is naturally nonautonomous:

$$\dot{z} = f(z, t).$$

The chapter does not collapse these two cases.

A single autonomous generator is a stronger statement than an ordered family of learned maps.

22.17 Stability is a separate question

Transport language does not answer whether perturbations grow.

For two nearby states,

$$z_k, \quad z_k + \delta z_k,$$

a local linearization gives

$$\delta z_{k+1} \approx J_{\Phi_k}(z_k) \delta z_k.$$

Across many stages,

$$\delta z_K \approx J_{\Phi_{K-1}}(z_{K-1}) \cdots J_{\Phi_0}(z_0) \delta z_0.$$

The product may:

- contract;
- preserve;
- amplify;
- rotate;
- shear;
- become highly non-normal.

Those are stability and sensitivity questions.

They do not follow from the fact that the base representation path remains feasible.

Haber–Ruthotto’s dynamical-systems perspective is relevant here (Haber and Ruthotto 2018), but no universal stability claim is inherited.

22.18 Invertibility is also separate

A map can transport a state forward while discarding information.

Normalization is one example.

ReLU is another familiar non-injective operation on part of its domain.

Pooling, clipping, quantization, token dropping, and many routing decisions can also be non-invertible.

Therefore

$$\text{state evolution} \not\equiv \text{reversible state evolution.}$$

A reversible architecture requires its own construction and proof obligations.

22.19 This is not optimal transport

Optimal transport studies the movement of mass or probability measures under a specified cost and admissible coupling/map structure.

Nothing in

$$z_{k+1} = \Phi_k(z_k)$$

by itself defines:

- a probability measure;
- a source and target distribution;
- a coupling;
- a transport cost;
- an optimality problem.

Representation transport may eventually be connected to optimal-transport ideas in a separately specified model.

The current chapter does not make that identification.

22.20 This is not parallel transport

Parallel transport requires geometric structure specifying how tangent vectors are carried along curves, usually through a connection.

The state path

$$z_0 \rightarrow z_1 \rightarrow \cdots \rightarrow z_K$$

does not automatically define such a rule.

Even on a sphere, “project the old tangent vector onto the new tangent plane” and “parallel transport the tangent vector along the connecting geodesic” are different constructions.

The Atlas uses the terminology literally.

22.21 This is not vector transport either

Optimization on manifolds often introduces a **vector transport** to move tangent vectors from one tangent space to another in a computationally convenient way (Absil et al. 2008).

That is closer to the machine-learning setting than abstract parallel transport, but it remains distinct from moving the representation point itself.

The notation should therefore preserve two maps when both are present:

$$z_{k+1} = R_{z_k}(\xi_k),$$

and

$$v_{k+1} = \mathcal{T}_{\xi_k}(v_k).$$

The first moves state.

The second moves tangent information.

A system that carries optimizer momentum on a manifold may need both.

22.22 Transport and split operators

Suppose one representation block contains two declared state maps,

$$\Phi_A, \quad \Phi_B.$$

Then chronological A-then-B transport is

$$\Phi_B \circ \Phi_A.$$

Chronological B-then-A transport is

$$\Phi_A \circ \Phi_B.$$

The Split-Operator chapter already established why these can differ and why execution order must be explicit.

TRANSPORT adds the constraint-aware reading:

- the first stage changes the base state seen by the second;
- tangent spaces may also change;
- a constraint-restoring map can introduce additional path dependence;
- exact-flow commutator theory must not be imported unless its assumptions hold.

This is a synthesis, not a new numerical theorem.

22.23 Transport and residual identity paths

Residual architectures are especially natural to describe as transport because the identity path makes the old state explicitly available to the next stage.

If

$$F_k(z) = 0,$$

then

$$\Phi_k(z) = z.$$

That is exact identity transport at that stage.

It does not prove:

- that information remains unchanged when $F_k \neq 0$;
- that gradients propagate perfectly;
- that the full model is invertible;
- that the representation follows a geodesic;
- that the architecture is stable.

Identity transport is one structural ingredient.

22.24 Transport as an interface contract

The most useful role of this chapter is to define what a downstream method is allowed to assume.

A downstream algorithm should be able to ask:

- What is the state space?
- What is the current base state?
- What map advances it?
- Is the map linear or nonlinear?
- Is it shared or stage-dependent?
- Is there a constraint?
- Is the update tangent?
- What returns the state to the constraint set?
- Is tangent history being carried?
- If so, by what rule?
- Is there an exact continuous flow or only a finite composition?
- Which information is intentionally invariant or discarded?

Those questions turn “representation transport” from a metaphor into an interface.

22.25 Handoff to Neural Krylov Transport

The next downstream chapter is

ATLAS-CH-NEURALKRYLOV-001.

That chapter will explore short-horizon preconditioned solves as representation computation.

TRANSPORT supplies only the representation-side interface.

For example, if a differentiable state map is

$$z_{k+1} = \Phi_k(z_k),$$

one may study its local linearization

$$J_{\Phi_k}(z_k).$$

A Krylov construction could then be posed for a declared operator built from that local object.

But classical Krylov theory requires its own assumptions:

- a specified linear operator;
- a generated Krylov subspace;

- a projection or residual criterion;
- conditioning/preconditioning semantics;
- finite-precision limits;
- convergence hypotheses.

None of those arrive for free from the transport language.

The hard boundary is:

representation transport $\nrightarrow$ Krylov convergence guarantee.

22.26 Five distinctions to keep visible

22.26.1 State transport versus tangent transport

Moving z_k is not the same problem as moving $v_k \in T_{z_k}M$.

22.26.2 Feasibility versus geodesic exactness

A retraction can keep a point on the manifold without following the exponential map.

22.26.3 Discrete composition versus continuous flow

A finite neural stack is exact as a composition of maps, not automatically as an ODE solution.

22.26.4 Constraint preservation versus information preservation

A normalized path can remain feasible while erasing radial distinctions.

22.26.5 Order versus ingredients

The same named stages applied in another order can produce another endpoint.

22.27 Failure modes

The transport language fails when it is used to smuggle in stronger mathematics than the model contains.

Common failures are:

- calling any sequence of activations an ODE trajectory;
- calling any normalized update geodesic motion;
- calling state evolution vector transport;
- calling tangent projection parallel transport;

- calling representation movement optimal transport without measures and costs;
- treating unit norm as information preservation;
- treating feasibility as stability;
- treating repeated residual form as one autonomous generator;
- treating a local Jacobian as the global nonlinear map;
- transferring Krylov convergence claims to learned nonlinear transport by analogy.

Each mistake replaces a declared object with a stronger undeclared one.

22.28 Closing view

A deep model can be read as a machine for moving representations.

That reading is useful when it remains literal.

The representation at one stage is a state.

The next stage applies a map.

Constraints may restrict where the state is allowed to live.

Retractions may restore feasibility.

Tangent data may require its own transport rule.

Order may change the path.

Continuous-depth models may provide an actual flow.

Finite residual networks may provide only a composition.

The durable Atlas synthesis is therefore:

representation transport = declared state + declared stage maps + declared constraints + declared path

Everything stronger must be earned separately.

22.29 References used in this chapter

- (Lee 2018)
- (Absil et al. 2008)
- (He et al. 2016)
- (Chen et al. 2018)
- (Haber and Ruthotto 2018)

Exact provenance and claim boundaries are locked in:

`sources/source-locks/ATLAS-CH-TRANSPORT-001.yaml`.

Chapter 23

Attention as an Operator

Epistemic status: mathematical exposition built from the standard Transformer equations, source-locked kernel interpretations, and Atlas-owned operator decompositions.

Primary figure: ‘ATLAS-FIG-ATTNOP-001’

Derivation packet: ‘mathematics/derivations/ATLAS-CH-ATTNOP-001-DERIVATIONS.md’

23.1 From an attention picture to an attention action

Attention is often introduced as a matrix of weights. The matrix is useful, but it is easy to confuse a picture of coefficients with the computation those coefficients perform.

For one attention head there are at least four distinct objects:

scores $\longrightarrow$ mixing operator $\longrightarrow$ value field $\longrightarrow$ output.

The score matrix records pairwise compatibility. Softmax turns each score row into normalized mixing coefficients. Those coefficients define an operator acting on value vectors. The full attention transformation is nonlinear because both the operator and the values depend on the input state.

The useful allegory is a **dynamic switching board**.

Incoming channels carry values. The present state determines how strongly each channel contributes to each output. The board is not fixed; queries and keys reconfigure it.

The correspondence is:

- compatibility signal $\leftrightarrow$ query-key score;
- switch configuration $\leftrightarrow$ normalized mixing operator;
- incoming channels $\leftrightarrow$ value vectors;
- outgoing channels $\leftrightarrow$ mixed outputs.

The limit matters. Softmax attention is not discrete switching. It is differentiable weighted mixing, often dense, and the weights themselves are computed from the state. The board is a mnemonic for state-dependent transformation, not a literal circuit.

23.2 Standard scaled dot-product attention

Let

$$X \in \mathbb{R}^{n \times d}$$

contain n token representations.

For one head,

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

The score matrix is

$$S(X) = \frac{QK^\top}{\sqrt{d_k}}.$$

Row-wise softmax produces

$$A(X) = \text{softmax}_{\text{row}}(S(X)),$$

and the head output is

$$Y = A(X)V.$$

This is the standard scaled dot-product attention construction (Vaswani et al. 2017).

The Atlas reserves different names for different roles:

$$S = \text{score matrix}$$

$$A = \text{normalized mixing operator}$$

$$V = \text{value field}$$

$$F(X) = A(X)V(X) = \text{full attention transformation.}$$

This distinction prevents several category errors. The score matrix is not the normalized operator. The operator is not the value field. And the fact that $Y = AV$ is linear in V for fixed A does not make F linear in X .

23.3 Why “operator” is the right word

Fix Q and K . Then A is fixed.

For two value fields V_1, V_2 and scalars α, β ,

$$A(\alpha V_1 + \beta V_2) = \alpha AV_1 + \beta AV_2.$$

So

$$V \mapsto AV$$

is an ordinary linear operator on the value field.

The operator language changes the question.

A heat-map reading asks:

Which coefficients are large?

An operator reading asks:

What transformation is being performed on the information carried by the values?

The second question naturally leads to rank, singular values, invariants, sparsity, support constraints, approximation, perturbation, and composition.

Those are properties of transformations rather than images.

23.4 Softmax gives a row-stochastic mixing stage

For finite unmasked scores,

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{\ell=1}^n e^{S_{i\ell}}}.$$

Therefore

$$A_{ij} > 0$$

and

$$\sum_{j=1}^n A_{ij} = 1.$$

Equivalently,

$$\boxed{A\mathbf{1} = \mathbf{1}.}$$

Each output row is a convex combination of value rows before any later output projection.

There is a useful but limited Markov analogy here. The rows lie on probability simplices. But an attention head is not generally a fixed Markov chain: the operator changes with the input, values are learned feature vectors rather than probabilities, and later transformations change the state again.

The algebraic fact is row-stochasticity. The Markov story is only an intuition aid.

23.5 The full self-attention map is nonlinear

Conditional linearity in V can produce the opposite mistake: calling the whole self-attention layer linear.

It is not.

Take two scalar tokens,

$$X = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad W_Q = W_K = W_V = 1, \quad d_k = 1.$$

Then

$$Q = K = V = X,$$

and

$$S = XX^\top.$$

The first output component is

$$F(X)_1 = \frac{e}{1+e}.$$

Now double the input. The scores scale quadratically, so

$$F(2X)_1 = \frac{2e^4}{1+e^4}.$$

But

$$2F(X)_1 = \frac{2e}{1+e}.$$

Therefore

$$F(2X) \neq 2F(X).$$

Numerically the first-component discrepancy has magnitude about 0.5019104228.

The exact statement is:

for fixed queries and keys, attention is linear in the value field;
ordinary self-attention is nonlinear in the hidden state because the
operator itself depends on that state.

23.6 A small operator we can inspect completely

The primary figure uses three tokens:

$$Q = K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{pmatrix}, \quad d_k = 2.$$

Then

$$S = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}.$$

Wolfram evaluation gives

$$A \approx \begin{pmatrix} 0.401112 & 0.197776 & 0.401112 \\ 0.197776 & 0.401112 & 0.401112 \\ 0.248255 & 0.248255 & 0.503490 \end{pmatrix},$$

and

$$Y = AV \approx \begin{pmatrix} 0.802224 & -0.203336 \\ 0.598888 & 0 \\ 0.751745 & -0.255235 \end{pmatrix}.$$

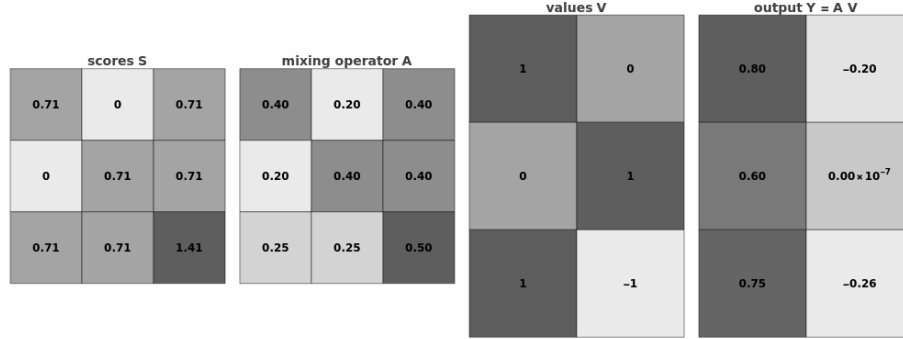


Figure 23.1: Four numeric matrix panels showing the score matrix S , row-softmax mixing operator A , value field V , and output Y equals $A V$.

The gray intensity is deliberately redundant. Every cell carries its numerical value. The plate is intended to be read as a transformation rather than interpreted as a color pattern.

The computational witness verifies two bounded claims:

$$A\mathbf{1} = \mathbf{1}$$

to numerical precision, and, with A fixed,

$$A(2V) = 2AV.$$

23.7 The operator changes when the query changes

Keep K and V fixed, but perturb the first query:

$$q_1 = (1, 0) \longrightarrow q'_1 = (1, \tfrac{1}{2}).$$

Only the first score row changes. Consequently, only the first row of the mixing operator changes.

The replay gives

$$\Delta A_{1,:} \approx (-0.08124593, 0.02683053, 0.05441540).$$

This exposes a useful local chain:

state perturbation $\longrightarrow$ operator perturbation $\longrightarrow$ changed information mixing.

Later chapters will ask whether a perturbation is causally important. Here the purpose is narrower: to make state dependence explicit.

23.8 Permutation equivariance before position enters

Suppose no positional signal or asymmetric mask is present.

Let P permute token order:

$$X' = PX.$$

Then

$$Q' = PQ, \quad K' = PK, \quad V' = PV.$$

Thus

$$S' = \frac{Q'K'^\top}{\sqrt{d_k}} = PSP^\top.$$

Row-wise softmax respects the simultaneous row/column permutation:

$$A' = PAP^\top.$$

Therefore

$$Y' = A'V' = PAP^\top PV = PY.$$

Hence

$$\boxed{F(PX) = PF(X)}.$$

This is permutation **equivariance**, not invariance. Reordering inputs reorders outputs in the same way.

The result is useful precisely because sequence models need order-sensitive structure somewhere. Positional encodings and causal masks modify the admissible operator family and break arbitrary permutation symmetry.

23.9 A mask is a constraint on the operator family

For causal attention, define

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Then

$$A = \text{softmax}_{\text{row}}(S + M)$$

satisfies

$$A_{ij} = 0 \quad \text{for } j > i.$$

The usual phrase says that a token “cannot attend to the future.”

The operator view says something slightly sharper:

causal masking restricts the admissible support pattern of the mixing operator.

This places causal masking in the same mathematical family as local windows, graph adjacency, block sparsity, and learned sparse support.

23.10 Multihead attention is a family of state-dependent operators

For head h ,

$$Q_h = XW_Q^{(h)}, \quad K_h = XW_K^{(h)}, \quad V_h = XW_V^{(h)},$$

and

$$A_h(X) = \text{softmax}_{\text{row}} \left(\frac{Q_h K_h^\top}{\sqrt{d_k}} \right).$$

Each head produces

$$Y_h = A_h V_h.$$

The layer then concatenates and projects the head outputs.

A useful abstraction is therefore

$$X \mapsto \{A_1(X), \dots, A_H(X)\} \mapsto \{A_1 V_1, \dots, A_H V_H\} \mapsto Y.$$

This is more informative than saying that a layer contains several attention maps. It contains a family of separately parameterized, state-dependent mixing operators whose outputs are recombined.

That viewpoint prepares later questions about head specialization, redundancy, substitution, spectral structure, and relative-position modes.

23.11 The kernel lens

Attention can also be written as normalized similarity-weighted smoothing. Tsai et al. develop an explicit kernel-smoother interpretation of Transformer attention (Tsai et al. 2019).

In generic form,

$$y_i = \frac{\sum_j k(q_i, k_j) v_j}{\sum_j k(q_i, k_j)}.$$

For

$$k(q, k) = \exp \left(\frac{q^\top k}{\sqrt{d_k}} \right),$$

this recovers ordinary softmax attention.

The lens is useful because it isolates the similarity function. But it does not erase the rest of the architecture. Queries and keys are learned and state-dependent; masks and position alter interactions; multihead attention constructs several such operators and recombines them.

The disciplined conclusion is:

kernel language is one productive mathematical lens on attention, not a universal identification of every attention mechanism with one fixed kernel machine.

23.12 Feature maps and linear attention

Suppose the similarity has a feature representation

$$k(q, k) = \phi(q)^\top \phi(k).$$

Then a numerator

$$\sum_j \phi(q_i)^\top \phi(k_j) v_j^\top$$

can be reassociated:

$$\phi(q_i)^\top \left(\sum_j \phi(k_j) v_j^\top \right).$$

This is the algebraic move exploited by linear-attention constructions such as Katharopoulos et al. (Katharopoulos et al. 2020).

From the Atlas viewpoint, the interesting fact is not merely lower asymptotic cost. The representation of the operator action has changed.

That prepares a later question:

when we change the factorization or approximation of attention,
which properties of the original operator survive?

23.13 Attention sensitivity has a concrete Jacobian

For one softmax row

$$a = \text{softmax}(s),$$

the Jacobian is

$$J_{\text{softmax}}(a) = \text{diag}(a) - aa^\top.$$

A query perturbation δq_i changes the score row by

$$\delta s_i = \frac{\delta q_i K^\top}{\sqrt{d_k}}.$$

Therefore, to first order,

$$\delta a_i \approx (\text{diag}(a_i) - a_i a_i^\top) \frac{\delta q_i K^\top}{\sqrt{d_k}}.$$

This is already an interface-style sensitivity calculation. A representation perturbation induces an operator perturbation with a measurable local Jacobian.

Nothing about this calculation says that a large coefficient is semantically important. Sensitivity is not explanation.

23.14 Three distinctions that must remain visible

23.14.1 Score matrix versus mixing operator

The score matrix can contain arbitrary real values.

The softmax operator is positive and row-normalized. Adding a constant to one score row changes nothing:

$$\text{softmax}(s + c\mathbf{1}) = \text{softmax}(s).$$

So even the score representation has a redundancy.

23.14.2 Mixing operator versus full head

The operator A determines how value rows mix. The value projection decides what information is available to be mixed. A visually striking A can have little downstream effect if the value/output pathway contributes little or is canceled.

23.14.3 Operator versus explanation

An observed A answers:

what mixing coefficients were produced?

It does not automatically answer:

which component caused the model's output?

Causal claims require interventions, substitutions, ablations, or comparable evidence.

23.15 Why this viewpoint matters for position

Position is often introduced as extra information attached to vectors.

The operator view suggests a broader possibility: relative displacement may change how representations are transformed.

RoPE already points in that direction because displacement appears through compositions of rotations. Later the Atlas will ask whether position can be isolated into a low-dimensional family

$$R(\Delta)$$

acting on representation space.

That question is natural once attention is already being described through operator action and composition.

23.16 What the computational witness establishes

The figure and Wolfram replay establish:

1. a concrete score matrix S ;
2. a row-softmax operator A ;
3. a concrete value field V ;
4. the operator output $Y = AV$;
5. row-stochasticity of A ;
6. linearity in V when A is fixed;
7. change in A under a query perturbation.

They do **not** establish:

- that an attention map explains a model decision;
- that a high weight is a causal feature importance;
- that the toy operator resembles a trained head;
- that kernel language is the only useful interpretation.

The witness makes the decomposition visible. It does not settle interpretation.

23.17 Atlas connections

Approximate and structured attention.

Approximation can now be evaluated in terms of operator error, output error, complexity, and preserved structure.

Position geometry.

Position can modify structured transformations rather than merely decorate vectors.

Relative-position operators.

The operator vocabulary becomes explicit: displacement may be encoded by a low-dimensional transformation family.

Retrieval.

Attention and retrieval both perform state-dependent selection and mixing, but differ in persistence, indexing, and externality.

Mechanistic diagnosis.

Operator substitution and counterfactual replacement are stronger tests than coefficient inspection.

The recurring Atlas shift is now concrete:

vectors and pairwise scores $\longrightarrow$ state-dependent operators acting on representations.

23.18 Closing view

The attention matrix is worth looking at.

Its deeper importance is that it **acts**.

A query-key state constructs a transformation. That transformation mixes information carried by values. Masks restrict its support. Position changes its structure. Heads create a family of such transformations. Approximation changes how the action is represented.

The dynamic switching board is only an allegory.

The operator is the object.

23.19 References used in this chapter

- (Vaswani et al. 2017)
- (Tsai et al. 2019)
- (Katharopoulos et al. 2020)

See ‘sources/source-locks/ATLAS-CH-ATTNOP-001.yaml’ for exact source identities and claim scope.

Chapter 24

Approximate and Structured Attention

Epistemic status: audited attention/Krylov substrate + primary Performer, Linformer, Nyströmformer, and alpha-entmax sources + Atlas synthesis.

Specification: manuscript/specifications/ATLAS-CH-ATTNAPPROX-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-ATTNAPPROX-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-ATTNAPPROX-001.md

Source lock: sources/source-locks/ATLAS-CH-ATTNAPPROX-001.yaml

Approximate attention is not one problem.

A method may approximate the exponential kernel before normalization.

Another may compress the normalized attention matrix.

Another may avoid constructing that matrix by projecting keys and values into a lower-dimensional sequence space.

Another may intentionally replace softmax with a sparse probability map.

All four can be called “efficient attention.”

Mathematically, they are different operations.

This chapter imposes one rule before any speed or accuracy claim:

name the object that changed.

The rule sounds administrative.

It is actually the shortest route to clear mathematics.

24.1 The attention stack

For one head, let

$$Q = XW_Q, \quad K_{\text{key}} = XW_K, \quad V = XW_V.$$

To avoid using K for both “keys” and “kernel,” write the score matrix as

$$S = \frac{QK_{\text{key}}^\top}{\sqrt{d_k}}.$$

Now define the positive exponential kernel matrix

$$G_{ij} = e^{S_{ij}}.$$

Its row sums are

$$z_i = \sum_j G_{ij}.$$

Let

$$D_G = \text{diag}(z_1, \dots, z_n).$$

The normalized attention operator is

$$A = D_G^{-1}G.$$

Finally,

$$Y = AV.$$

The audited Attention-as-an-Operator chapter already separated:

$$S \longrightarrow A \longrightarrow V \longrightarrow Y.$$

This chapter inserts one more object:

$$S \longrightarrow G \longrightarrow A \longrightarrow V \longrightarrow Y.$$

That extra step matters because several efficient-attention methods act on G, while others act on A, K/V sequence structure, or the normalization itself.

24.2 Five different approximation statements

Suppose a method produces hatted objects.

The following statements are not interchangeable.

24.2.1 Score approximation

$$\hat{S} \approx S.$$

This concerns pairwise compatibility before exponentiation.

Small score error can be magnified by exponentiation, especially when norms are large.

24.2.2 Kernel approximation

$$\hat{G} \approx G.$$

This is the natural target for random-feature approximations of the exponential dot-product kernel.

24.2.3 Operator approximation

$$\hat{A} \approx A.$$

This is the natural target if we care about the actual mixing coefficients.

24.2.4 Output approximation

$$\hat{Y} = \hat{A}V \approx AV.$$

This is weaker than operator approximation when tested on only one value field.

24.2.5 Alternative operator

$$\tilde{A} = T(S),$$

where T is deliberately not softmax.

Alpha-entmax belongs here.

The method can be valuable without being an estimator of softmax.

The distinction is the central organizing principle of this chapter.

24.3 Why raw kernel error can mislead

Softmax is invariant to adding a constant to a score row.

For any row i ,

$$\text{softmax}(s_i + c_i \mathbf{1}) = \text{softmax}(s_i).$$

After exponentiation, this is positive row scaling.

Let C be any positive diagonal matrix.

Then

$$D(CG)^{-1}CG = D(G)^{-1}G.$$

So replacing G by CG leaves A unchanged.

Take the simplest case:

$$\hat{G} = cG.$$

Then

$$\hat{A} = A,$$

but

$$\|\hat{G} - G\|_F = |c - 1|\|G\|_F.$$

By taking c large, the raw kernel error becomes arbitrarily large while the attention operator does not move at all.

This is not a pathology.

It is the algebraic expression of a symmetry already present in softmax.

The practical lesson is:

kernel error is meaningful only together with the normalization convention and the route by which kernel error propagates to normalized attention.

24.4 Kernel error needs denominator control

Take one positive kernel row g .

Let

$$z = \mathbf{1}^\top g.$$

Suppose

$$\hat{g} = g + e, \quad \hat{z} = z + \delta, \quad \delta = \mathbf{1}^\top e.$$

The normalized rows (assuming $\hat{z} > 0$) are

$$a = \frac{g}{z}, \quad \hat{a} = \frac{g + e}{\hat{z}}.$$

This algebra does not by itself require every entry of $\hat{g} = g + e$ to be nonnegative. If any approximate entry is negative, $\hat{a}$ still sums to one but is not a probability row; interpreting it as an attention distribution requires separate entrywise nonnegativity. Positive random-feature constructions supply that nonnegativity by design.

Direct subtraction gives

$$\hat{a} - a = \frac{e}{\hat{z}} - \frac{g\delta}{z\hat{z}}.$$

Hence

$$\|\hat{a} - a\|_1 \leq \frac{2\|e\|_1}{\hat{z}}.$$

If

$$\|e\|_1 \leq \rho z$$

with $0 \leq \rho < 1$, then

$$\hat{z} \geq (1 - \rho)z$$

and therefore

$$\|\hat{a} - a\|_1 \leq \frac{2\rho}{1 - \rho}.$$

The bound is intentionally elementary.

Its purpose is not to replace Performer’s analysis.

It isolates the missing ingredient in vague statements such as “the kernel is close, so attention is close.”

Normalization must be controlled too.

24.5 Positive random features

The exponential dot-product kernel has a useful probabilistic representation.

Let

$$\omega \sim N(0, I)$$

and define

$$\phi_\omega(x) = \exp\left(\omega^\top x - \frac{1}{2}\|x\|_2^2\right).$$

Then

$$\mathbb{E}_\omega[\phi_\omega(q)\phi_\omega(k)] = e^{q^\top k}.$$

The derivation is one line of Gaussian moment-generating algebra after expanding

$$\|q + k\|_2^2.$$

For scaled dot-product attention, choose

$$q' = \frac{q}{d_k^{1/4}}, \quad k' = \frac{k}{d_k^{1/4}},$$

so

$$q'^\top k' = \frac{q^\top k}{\sqrt{d_k}}.$$

Therefore

$$e^{q'^\top k' / \sqrt{d_k}}$$

inherits the same positive random-feature identity.

This is the clean mathematical reason random features are relevant to softmax attention.

24.6 From an expectation identity to Performer

A finite feature approximation replaces the expectation by an average:

$$\hat{G}_{ij} = \frac{1}{m} \sum_{r=1}^m \phi_{\omega_r}(q'_i) \phi_{\omega_r}(k'_j).$$

With independent Gaussian features, the kernel estimator is unbiased entrywise.

But “unbiased” does not mean “small error in one run.”

The finite estimator has variance.

The normalized operator is a ratio involving the approximate row sums.

Finite precision can matter because exponentials span large ranges.

And the cost depends explicitly on the feature count m .

Performer addresses this setting with FAVOR+, Fast Attention Via positive Orthogonal Random features (Choromanski et al. 2021).

The primary source develops positive orthogonal random features and source-specific accuracy/variance guarantees for softmax-kernel attention.

The Atlas inherits that result at its stated scope.

It does not convert “random features” in general into a guarantee.

24.7 Why feature attention can avoid the dense matrix

Write feature matrices

$$\Phi_Q \in \mathbb{R}^{n \times m}, \quad \Phi_K \in \mathbb{R}^{n \times m}.$$

Then

$$\hat{G} = \Phi_Q \Phi_K^\top.$$

The numerator can be associated as

$$\hat{G}V = \Phi_Q (\Phi_K^\top V).$$

The denominator is

$$\hat{G}\mathbf{1} = \Phi_Q (\Phi_K^\top \mathbf{1}) .$$

So the normalized output can be computed without constructing the full $n \times n$ matrix.

This is the computational move.

The cost is linear in n only after exposing what is held fixed, especially m and the feature/value widths.

The word “linear” is therefore shorthand for a scaling regime, not a property independent of approximation budget.

24.8 Low-rank attention is another idea

Linformer starts from a different structural premise (Wang et al. 2020).

Its motivating claim is that self-attention can often be represented or approximated through a low-dimensional sequence structure.

Rather than Monte Carlo features for the exponential kernel, Linformer projects key/value sequence dimensions into a smaller width.

The distinction matters.

A random-feature method asks:

can I approximate the kernel action through features?

A low-rank projection method asks:

can I compress the sequence interaction into a lower-dimensional subspace?

Both reduce cost.

They do not have the same approximation variable.

For Performer-like methods the explicit budget is feature count m .

For Linformer-like methods it is a projection/rank width k .

A theorem about one budget does not automatically say anything about the other.

24.9 Low rank is a hypothesis, not a universal property

Any matrix has a rank.

Saying that an attention operator has a useful low-rank approximation is stronger.

It says that most of the relevant action can be captured with fewer directions than the ambient sequence length.

That may occur.

It need not occur uniformly across:

- layers;
- heads;
- inputs;
- training stages;
- positional regimes;
- masks;
- distribution shifts.

The correct experimental question is not:

is attention low rank?

It is:

for this declared target, norm, data regime, and rank budget, how much error remains?

This is the same discipline that classical low-dimensional numerical methods demand.

The audited Krylov chapter supplies vocabulary for reduced action.

It does not supply a convergence theorem for learned attention.

24.10 Landmark reconstruction: Nyströmformer

Nyströmformer uses a landmark-based route (Xiong et al. 2021).

The broad Nyström idea is to reconstruct a large matrix from interactions involving a smaller landmark set.

Schematically, one encounters an approximation of the form

$$\hat{A} \approx CW^\dagger R,$$

where W is a reduced landmark block and C/R connect the full matrix to the landmarks.

Nyströmformer adapts this idea to softmax self-attention.

This introduces a third approximation budget:

$$m = \text{number of landmarks.}$$

Again, the reduced representation is not the same object as random features.

A landmark is not a Gaussian feature.

A pseudoinverse-like reduced solve is not a Monte Carlo average.

The shared idea is economy through structure, not mathematical identity.

24.11 Conditioning belongs in the landmark story

Reduced systems can be small and still be numerically awkward.

Whenever reconstruction involves an inverse or pseudoinverse-like operation, conditioning matters.

A reduced matrix that is nearly singular can amplify perturbations.

This is where the Krylov chapter contributes a useful habit:

never identify a computable residual or reduced dimension with the true target error without the operator relation that connects them.

For attention, the analogous warning is:

a small landmark system does not by itself imply a small reconstruction error.

Landmark quality, matrix structure, numerical conditioning, and the chosen norm remain part of the claim.

24.12 Sparse attention can mean a different normalization

Softmax has a rigid property.

For finite scores,

$$\text{softmax}(s)_i > 0$$

for every coordinate.

The resulting row is dense.

The alpha-entmax family changes this geometry (Peters et al. 2019).

For alpha greater than one, entmax can produce exact zeros.

At alpha equal to two, the family gives sparsemax.

This is not merely “softmax with a cheap approximation.”

It is a different score-to-simplex map.

That distinction is useful because exact sparsity may be the desired inductive bias.

It is also necessary because a sparse alternative should not be penalized as a bad softmax estimator when reproducing softmax was never its goal.

24.13 An exact sparse-support witness

Take

$$s = (2, 0, -1).$$

Softmax gives

$$\frac{1}{e^2 + 1 + e^{-1}}(e^2, 1, e^{-1}).$$

All entries are positive.

For sparsemax, use the simplex-projection threshold form

$$p_i = \max(s_i - \tau, 0).$$

With

$$\tau = 1,$$

we obtain

$$p = (1, 0, 0).$$

So

$$\text{entmax}_2(2, 0, -1) = (1, 0, 0).$$

One map mixes all three positions.

The other selects one exactly.

The difference is structural, not a rounding artifact.

24.14 Sparsity and low rank are independent

A sparse matrix can have full rank.

The identity matrix is the simplest example.

A low-rank matrix can be completely dense.

For example,

$$\frac{1}{n} \mathbf{1} \mathbf{1}^\top$$

has rank one and no zero entries.

Therefore

sparse $\nrightarrow$ low rank,

and

low rank $\nrightarrow$ sparse.

This prevents another common collapse of vocabulary.

“Structured attention” is a family name.

The structures inside the family remain different.

24.15 Exact sparsity is not yet a complexity theorem

Suppose we first form

$$S = QK_{\text{key}}^\top.$$

That step already considers all token pairs.

If a later normalization turns most coefficients into zero, the dense pairwise score cost has already been paid.

So

many zeros in A

does not by itself imply

cheap attention.

To obtain computational savings, the implementation must exploit structure earlier or later in a way that changes actual work.

Examples include:

- structured neighborhoods;
- block support;
- hashing/routing;
- feature factorization;
- low-rank projection;
- landmark reconstruction;
- hardware-efficient sparse kernels.

The exact mechanism matters.

24.16 Three error surfaces

The Atlas uses three primary numerical surfaces.

24.16.1 Kernel error

$$E_G = \|\hat{G} - G\|.$$

This is natural for random-feature methods.

But it is sensitive to row scale.

24.16.2 Operator error

$$E_A = \|\hat{A} - A\|.$$

This measures the mixing transformation itself.

It is the central comparison surface when the target is ordinary softmax attention.

24.16.3 Output error

$$E_Y = \|\hat{A}V - AV\|.$$

This measures one realized action on one value field.

For fixed V ,

$$E_Y \leq \|\hat{A} - A\|_2 \|V\|_F.$$

The reverse implication fails from one V.

A large operator discrepancy can lie in directions not excited by the chosen value field.

24.17 A small exact operator approximation

Consider

$$A = \begin{pmatrix} 1/2 & 1/3 & 1/6 \\ 1/2 & 1/3 & 1/6 \\ 1/6 & 1/3 & 1/2 \end{pmatrix}.$$

It is positive and row-stochastic.

Its rank is two.

Now replace all rows by their average:

$$\hat{A} = \begin{pmatrix} 7/18 & 1/3 & 5/18 \\ 7/18 & 1/3 & 5/18 \\ 7/18 & 1/3 & 5/18 \end{pmatrix}.$$

This approximation is rank one.

The exact error is

$$A - \hat{A} = \frac{1}{9} \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ -2 & 0 & 2 \end{pmatrix}.$$

Therefore

$$\|A - \hat{A}\|_F^2 = \frac{4}{27}$$

and

$$\|A - \hat{A}\|_2 = \frac{2\sqrt{3}}{9}.$$

The witness is not intended to imitate a trained head.

It exists so every quantity can be inspected exactly.

24.18 The same operator error can look smaller after acting on values

Take

$$V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{pmatrix}.$$

Then

$$AV = \begin{pmatrix} 2/3 & 1/6 \\ 2/3 & 1/6 \\ 2/3 & -1/6 \end{pmatrix}$$

and

$$\widehat{AV} = \begin{pmatrix} 2/3 & 1/18 \\ 2/3 & 1/18 \\ 2/3 & 1/18 \end{pmatrix}.$$

The squared output error is

$$\frac{2}{27}.$$

This is smaller than the squared Frobenius operator error,

$$\frac{4}{27}.$$

Nothing paradoxical has happened.

V does not probe every operator direction equally.

This is why output fidelity and operator fidelity must be reported separately.

24.19 Task performance is a fourth question

Suppose a model is retrained with an approximate mechanism and reaches the same benchmark score as a standard Transformer.

That is important empirical evidence.

It still does not prove

$$\hat{A} \approx A$$

for the corresponding softmax operator.

The trained model may compensate elsewhere.

Its representations may reorganize.

Its heads may specialize differently.

The approximate mechanism may be better for the task while being farther from softmax.

Thus we should ask four questions separately:

1. Is the kernel close?
2. Is the normalized operator close?
3. Is the output on the declared V close?
4. Does the trained system perform well?

The fourth does not subsume the first three.

24.20 Complexity must expose the approximation budget

Standard dense attention pays for pairwise interactions across n tokens.

Feature methods replace n -by- n interaction with a feature width m .

Low-rank methods replace sequence dimension with a projection width k .

Landmark methods replace full interaction with m landmarks.

Sparse methods introduce a support size only if the implementation actually exploits it.

Therefore every scaling statement should look like:

cost as a function of n , with m/k /support budget declared.

The phrase “linear attention” is only meaningful after that declaration.

If maintaining a fixed error tolerance requires m to grow with n , the effective complexity can change.

24.21 Accuracy-cost curves are more informative than single points

A fair comparison should sweep the approximation budget.

For random features:

$$m = 16, 32, 64, \dots$$

For low-rank projection:

$$k = 16, 32, 64, \dots$$

For landmarks:

$$m = 16, 32, 64, \dots$$

For sparse alternatives, vary alpha or the mechanism's support-control parameter when appropriate.

At each budget, measure:

- wall-clock or kernel time;
- memory;
- kernel/operator/output error where defined;
- support/rank diagnostics;
- downstream performance if training is part of the experiment.

One point can hide the shape of the tradeoff.

The curve is the object.

24.22 Masks must remain in the target definition

Causal attention is not ordinary unmasked attention followed by a cosmetic display rule.

The mask constrains support.

If

$$M_{ij} = -\infty$$

for forbidden interactions, the exact target operator has zeros there.

An approximation must state whether it:

- preserves those zeros exactly;
- approximates only the allowed region;
- uses a causal recurrence;
- leaks mass into forbidden positions.

A method that approximates unmasked attention accurately can still be invalid for a masked target if support is wrong.

24.23 Position changes what is being approximated

Suppose scores include a relative-position term:

$$S_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} + b_{ij}.$$

Then b is part of the target.

A feature map derived only for the content exponential kernel does not automatically approximate the combined positional operator.

The same warning applies to rotary structure, head-specific biases, and other positional mechanisms.

The later positional chapters can inherit the approximation vocabulary developed here.

They must restate which structure is inside the target.

24.24 Finite precision is not an afterthought

Approximation changes numerical pathways.

Random-feature exponentials can overflow or underflow.

Reduced landmark systems can be ill-conditioned.

Low-rank factorizations can amplify errors through poorly scaled factors.

Sparse transformations can change gradient behavior near support transitions.

These are not reasons to reject the methods.

They are reasons to include finite precision in the contract.

The correct question is:

what numerical operation is actually performed, and which quantity remains stable under its implementation?

24.25 A useful comparison table in words

The four representative routes can now be summarized without collapsing them.

Performer/FAVOR+.

Target: exponential softmax kernel and normalized attention action.

Structure: positive orthogonal random features.

Budget: feature count.

Characteristic risk: finite-feature variance and normalization error.

Linformer.

Target: compressed attention action through sequence projection.

Structure: low-dimensional learned projection.

Budget: projection/rank width.

Characteristic risk: low-rank premise failing on relevant states.

Nyströmformer.

Target: normalized self-attention matrix/action.

Structure: landmarks and reduced reconstruction.

Budget: landmark count.

Characteristic risk: landmark adequacy and reduced-system conditioning.

Alpha-entmax.

Target: a different score-to-simplex operator.

Structure: exact sparse support for alpha greater than one.

Budget: not naturally an approximation rank; alpha controls geometry/sparsity.

Characteristic risk: calling a changed operator a softmax approximation when it is not intended to be one.

24.26 The Krylov analogy, used carefully

The Krylov chapter taught a productive lesson:

large operator problems can sometimes be attacked through informative low-dimensional directions.

That lesson transfers.

The convergence theorems do not.

A Krylov subspace is generated by

$$r_0, Ar_0, A^2r_0, \dots$$

for a declared operator.

Random features are sampled feature directions.

Linformer projections are learned sequence reductions.

Nyström landmarks are selected representatives.

These are all reduced representations.

They are not the same reduced representation.

The safe inheritance is methodological:

- define the target;
- define the reduced object;
- define the residual or error;
- expose conditioning;
- expose the budget;
- test convergence or approximation rather than assuming it.

24.27 What not to infer

This chapter rejects several tempting shortcuts.

24.27.1 Small kernel error does not automatically give small operator error

Normalization can amplify errors if denominator control is weak.

24.27.2 Small output error on one value field does not give small operator error

The value field may fail to excite discrepant directions.

24.27.3 Low rank does not imply sparsity

Dense rank-one matrices exist.

24.27.4 Sparsity does not imply low rank

The identity matrix is sparse and full rank.

24.27.5 Exact zeros do not imply fast execution

Dense score formation may already have occurred.

24.27.6 Few random features do not imply uniform accuracy

Finite approximation quality is a statistical question.

24.27.7 Good benchmark performance does not imply softmax fidelity

Retraining can change the system.

These are not edge cases.

They are the basic claim boundaries.

24.28 A practical evaluation protocol

For a fixed trained head or controlled synthetic case:

1. freeze Q , keys, V , mask, and position;
2. compute exact dense softmax A and Y when feasible;
3. choose one approximation family;
4. state its budget m or k ;
5. compute its own target objects;
6. measure operator error to A if softmax fidelity is intended;
7. measure output error on V ;
8. record row sums, positivity, support violations, and rank/sparsity;
9. record memory and runtime;
10. sweep the budget;
11. repeat over representative inputs;
12. separate approximation fidelity from retrained task performance.

This turns “efficient attention” from a label into an experiment.

24.29 Research bridges

Several later Atlas questions become sharper after this chapter.

24.29.1 Spectral diagnostics

If A is approximated by $\hat{A}$, how do singular values, non-normality, mixing rates, or transient amplification change?

24.29.2 Context compression

Can a low-rank or landmark structure be used deliberately as a context-compression operator rather than merely as a cheaper attention surrogate?

24.29.3 Relative-position operators

Can positional structure be factored exactly or approximately without destroying support or equivariance?

24.29.4 Systems

When does theoretical sparsity or low rank translate into actual accelerator speed?

24.29.5 Adaptive approximation

Can feature count, landmark count, or rank be selected dynamically using an error indicator rather than fixed globally?

That last question connects directly to the Atlas treatment of adaptive error control.

The error indicator would need an explicit relation to the operator or output error being controlled.

24.30 What the computational witness establishes

The finite witness establishes exactly:

1. a positive row-stochastic rank-2 operator;
2. a positive row-stochastic rank-1 approximation;
3. exact Frobenius operator error $4/27$ in squared norm;
4. exact spectral operator error $2\sqrt{3}/9$;
5. exact output error $2/27$ in squared Frobenius norm for a declared V ;
6. the standard operator-to-output inequality;
7. a score row for which softmax is dense;
8. the $\alpha=2$ entmax/sparsemax result $(1, 0, 0)$;
9. row-scaling invariance showing raw kernel error and operator error can completely decouple.

It does not establish trained-model superiority or a finite-feature Performer guarantee.

24.31 Closing view

Approximate attention is best understood as a map of design choices.

One route approximates a kernel.

One route compresses an operator.

One route reconstructs from landmarks.

One route changes the normalization geometry and creates exact sparsity.

The useful unifying picture is not:

all efficient attention is the same.

It is:

target + structure + budget + error notion + implementation

.

That tuple tells us what was preserved and what was changed.

Without it, “linear,” “sparse,” “low-rank,” and “approximate” are only adjectives.

With it, they become mathematical claims.

24.32 References used in this chapter

- (Choromanski et al. 2021)
- (Wang et al. 2020)
- (Xiong et al. 2021)
- (Peters et al. 2019)

Exact source identities and claim boundaries are recorded in `sources/source-locks/ATLAS-CH-ATTNAPPROX-001.yaml`.

Chapter 25

The Geometry of Position

Epistemic status: mathematical exposition built from audited Attention and Geometry prerequisites, primary positional-encoding sources, and Atlas-owned exact witnesses.

Derivation packet: ‘mathematics/derivations/ATLAS-CH-POSGEOM-001-DERIVATIONS.md‘

Computational witness: ‘mathematics/computational-witnesses/ATLAS-CW-POSGEOM-001.md‘

25.1 Position is not content

A Transformer sees token representations.

Without an asymmetric mask or positional signal, ordinary self-attention is permutation equivariant: permuting the token order permutes the output order.

Sequence order therefore has to enter somewhere.

The first conceptual separation is:

content geometry $\neq$ position geometry.

Content determines what a token represents.

Position determines where that token sits in a declared coordinate system.

The two interact inside attention, but they are not the same object.

This chapter develops position as an explicit transformation acting on attention representations.

25.2 Additive sinusoidal position

The original Transformer adds a fixed sinusoidal vector to the token embedding (Vaswani et al. 2017).

For one angular frequency ω , consider the two-dimensional component

$$p_m(\omega) = \begin{pmatrix} \sin(m\omega) \\ \cos(m\omega) \end{pmatrix}.$$

A fixed offset δ acts linearly:

$$p_{m+\delta} = A_\delta p_m,$$

where A_δ is a rotation matrix determined by $\delta\omega$.

That fact explains the original motivation: relative displacement can be represented through a position-independent linear transform of the sinusoidal features.

But the mechanism is still additive.

The positional vector is added to the representation before later layers act.

25.3 Rotary position moves the geometry into query and key space

Rotary Position Embedding takes a different route (Su et al. 2021).

Instead of adding a positional vector, it rotates query and key coordinates as a function of position.

For one 2D block define

$$R(\phi) = \begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix}.$$

At position m , use

$$R_m(\omega) = R(m\omega).$$

Apply that rotation to the corresponding query and key block.

The crucial algebra is

$$R(\phi)^\top = R(-\phi),$$

so

$$R_m(\omega)^\top R_n(\omega) = R((n - m)\omega).$$

Therefore

$$(R_m q)^\top (R_n k) = q^\top R((n - m)\omega) k.$$

The absolute positions m and n disappear from this pairwise expression except through their difference.

This is the core relative-position identity.

25.4 What the identity actually says

The identity is exact.

It says that the contribution of one rotary block to the query-key inner product is determined by:

- the unrotated query block;
- the unrotated key block;
- the frequency;
- the relative displacement $n - m$.

It does not say that the whole model depends only on relative position.

Other components can break that symmetry:

- causal masks;
- finite sequence boundaries;
- content distributions;
- layerwise nonlinearities;
- caches;
- architecture-specific operations;
- any additional position-dependent mechanism.

The exact statement belongs to the transformed query-key score contribution.

25.5 An exact same-offset witness

Choose

$$\theta = \pi/6, \quad q = k = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Compare positions $(0, 2)$.

The relative offset is 2, so

$$(R_0 q)^\top (R_2 k) = \cos(2\theta) = \frac{1}{2}.$$

Now compare positions $(3, 5)$.

The relative offset is still 2, and

$$(R_3 q)^\top (R_5 k) = \frac{1}{2}.$$

The absolute coordinates moved.

The rotary score contribution did not.

This is a finite replay of the relative-position identity.

25.6 Why the rotation structure matters

The result is not true for arbitrary position-dependent transforms.

Define instead

$$S_m = \text{diag}(2^m, 1).$$

Using the same $q = k = (1, 0)^\top$,

$$(S_m q)^\top (S_n k) = 2^{m+n}.$$

For the same-offset pair $(0, 2)$,

$$\text{score} = 4.$$

For the same-offset pair $(3, 5)$,

$$\text{score} = 256.$$

The relative displacement is identical.

The score is not.

What changed?

The family S_m does not satisfy the rotary group relation

$$T_m^\top T_n = T_{n-m}.$$

The exact relative-position identity depends on the structure of the transform family.

25.7 Multi-frequency position

Real rotary encodings use many 2D blocks.

For even dimension $d = 2h$,

$$R_m = \text{diag}(R(m\omega_1), \dots, R(m\omega_h)).$$

Then

$$R_m^\top R_n = \text{diag}(R((n-m)\omega_1), \dots, R((n-m)\omega_h)).$$

Each block sees the same displacement through a different phase rate.

The frequency schedule therefore determines how relative displacement is represented across coordinates.

Low frequencies rotate slowly.

High frequencies rotate quickly.

The resulting positional representation is a bank of phase measurements at multiple scales.

25.8 Position as phase

The phase viewpoint is more useful than thinking of RoPE as “just another embedding.”

For one frequency,

$$m \mapsto m\omega \mapsto R(m\omega).$$

Position becomes a group parameter.

The representation at position m is related to position n by the phase difference

$$(n-m)\omega.$$

This makes several properties transparent:

- common translation cancels in pairwise phase difference;
- different frequencies resolve displacement differently;
- rescaling position changes phase;

- rescaling frequency also changes phase;
- those two interventions can look algebraically related while being operationally distinct.

25.9 Common translation

Shift every position by the same amount c .

Then

$$(n + c) - (m + c) = n - m.$$

So

$$R_{m+c}^\top R_{n+c} = R_{n-m}.$$

The rotary query-key contribution is invariant under a common shift of both position indices.

Again, this is not a theorem that the full sequence model is translation invariant.

It is an exact property of the rotary score geometry.

25.10 Multidimensional position

A sequence uses one coordinate.

Images, grids, fields, and other structured domains may need several.

Let a 2D position be

$$p = (u, v).$$

Use two independent rotary blocks:

$$R_p = \text{diag}(R(u\omega_x), R(v\omega_y)).$$

For another position

$$q = (u', v'),$$

we obtain

$$R_p^\top R_q = \text{diag}(R((u' - u)\omega_x), R((v' - v)\omega_y)).$$

The pairwise operator depends on the coordinate-wise displacement.

This is the direct-product version of the one-dimensional construction.

The important point is not that every 2D positional method must use this formula.

It is that multidimensional position needs an explicit coordinate action.

Duplicating a scalar position label does not automatically create geometry.

25.11 Position and attention support are separate

A causal mask and positional encoding do different jobs.

The mask decides whether an attention edge is allowed.

The positional transformation affects the score attached to an allowed query-key pair.

A pair may have a well-defined relative phase and still be masked out.

This distinction keeps support geometry separate from score geometry.

25.12 Exact algebra does not guarantee long-context generalization

The identity

$$R_m^\top R_n = R_{n-m}$$

continues to make algebraic sense far beyond any finite training context.

That does not mean the trained model knows how to use those phases.

A model is trained under a finite distribution of:

- absolute positions;
- relative displacements;
- phase combinations;
- sequence lengths;
- attention patterns;
- optimization conditions.

Outside that regime, exact continuation of the positional formula is only one ingredient.

The model-level question is empirical.

25.13 Direct extrapolation

The simplest long-context strategy is to evaluate the same positional rule at larger indices.

For rotary position, that means evaluating phases

$$m\omega$$

for m beyond the range used in training.

The formula is valid.

The behavior need not be.

This is the distinction between:

formula extrapolates

and

trained model generalizes.

They are not equivalent.

25.14 Position Interpolation

Position Interpolation changes the coordinate presented to the positional mechanism (Chen et al. 2023).

Suppose the original context scale is L and the desired scale is $L' > L$.

A simple interpolation map is

$$m \mapsto \tilde{m} = m \frac{L}{L'}.$$

The phase becomes

$$\tilde{m}\omega$$

instead of $m\omega$.

The paper reports that this can extend RoPE-based pretrained models with relatively limited fine-tuning and gives a theoretical comparison between interpolation and direct extrapolation under its stated analysis.

The Atlas uses that result at its actual scope.

It is not a theorem that every model, every frequency schedule, or every target length will behave well under the same rescaling.

25.15 YaRN

YaRN belongs to the same broader problem family but is not merely “Position Interpolation again” (Peng et al. 2023).

It combines positional rescaling ideas with additional treatment intended to improve efficient long-context extension.

The reported results are empirical method evidence.

The relevant Atlas lesson is structural:

Long-context extension is an intervention on the positional phase regime and often also on training.

The exact rotary identity remains true before and after such modifications.

What changes is which phases the model encounters and how well its learned computation handles them.

25.16 Frequency scaling and index scaling are different interventions

Consider a phase

$$m\omega.$$

One can change it by:

- replacing m with αm ;
- replacing ω with $\alpha\omega$;
- changing both nonuniformly across frequencies;
- fine-tuning after the change.

These may coincide algebraically in a single isolated block under special choices.

They are not operationally interchangeable in a full model.

A positional extension method should therefore state exactly what it changes.

25.17 The geometry of extrapolation

Long-context behavior is often described as if context length were one scalar knob.

Geometrically, several things can move at once:

- the range of phase angles;
- the density of sampled offsets;
- aliasing relationships across frequencies;
- the distribution of query-key content seen at those phases;
- mask/boundary effects;
- the learned response to phase combinations.

This is why the positional mechanism can remain mathematically well-defined while the trained system degrades.

The geometry survives.

The learned use of the geometry may not.

25.18 Failure modes

25.18.1 Additive equals rotary

It does not.

Both use sinusoidal structure, but one adds position features while the other rotates query/key coordinates.

25.18.2 Relative identity equals long-context guarantee

It does not.

The identity is algebraic.

Generalization is model-level.

25.18.3 Any position transform works

It does not.

The non-orthogonal control shows that same-offset dependence can fail.

25.18.4 Frequency schedule equals context policy

It does not.

The schedule defines phase rates.

A context-extension policy says how positions/frequencies/training are changed when the target range changes.

25.18.5 2D means two copies of 1D

Not automatically.

The coordinate action has to be declared.

25.18.6 Interpolation equals extrapolation

They are opposite geometric choices.

Interpolation maps a larger target range back into a smaller phase regime.

Direct extrapolation evaluates outside that regime.

25.18.7 Position explains all long-context behavior

It does not.

Attention content, optimization, retrieval, memory, architecture, and training distribution all matter.

25.19 Downstream handoff

Relative-Position Operators may assume:

- the exact block-rotation identity;
- the multi-frequency decomposition;
- common-translation invariance of the rotary score contribution;
- multidimensional direct-product position;
- the distinction between exact positional algebra and empirical context extension.

It may not assume that vector RoPE already provides:

- a low-dimensional operator decomposition;
- a DC-mode theory;
- frequency-mode specialization;
- head-specific relative-position operators.

Those are the next chapter's obligations.

25.20 References used in this chapter

- (Vaswani et al. 2017) — fixed additive sinusoidal positional encoding.
- (Su et al. 2021) — rotary position embedding and relative-position score construction.
- (Chen et al. 2023) — Position Interpolation for RoPE-based context extension.
- (Peng et al. 2023) — YaRN context-extension method.

Exact provenance and claim boundaries are locked in:

‘sources/source-locks/ATLAS-CH-POSGEOM-001.yaml’

Chapter 26

Relative-Position Operators

Epistemic status: audited Positional Geometry/Linear Algebra prerequisites + primary relative-position sources + Atlas synthesis + exact finite witness

Specification: manuscript/specifications/ATLAS-CH-RPO-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-RPO-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-RPO-001.md

Position can enter a Transformer as a vector.

It can also enter as an operator.

That shift matters because an operator can be decomposed into modes, projected, truncated, compared across heads, and composed with other structure. The chapter develops that viewpoint without pretending that every positional mechanism is the same object.

26.1 Two spaces, two frequency notions

The audited Positional Geometry chapter already gives a precise RoPE identity.

For a relative offset Δ , one can write a feature-space operator

$$\rho(\Delta) = \bigoplus_a R(\omega_a \Delta),$$

where each $R(\omega_a \Delta)$ is a 2D rotation block.

Those frequencies live in feature coordinates.

This chapter also studies translation-invariant operators over sequence index.

Those operators have Fourier modes over positions.

The two frequency notions are not automatically the same.

That distinction is the first discipline of RPO.

26.2 Relative position as an operator over sequence index

Suppose positions lie on the cyclic set

$$\mathbb{Z}_L.$$

For one attention head, define a relative-position kernel

$$\kappa_h(\Delta).$$

Then let

$$(K_h x)_i = \sum_{\Delta=0}^{L-1} \kappa_h(\Delta) x_{i+\Delta \bmod L}.$$

This is a circulant linear operator.

The periodic boundary is deliberate.

It makes the algebra exact.

It is not a claim that real language sequences wrap around from the end to the beginning.

For ordinary finite nonperiodic sequences, a relative-position matrix is generally Toeplitz-like rather than circulant.

26.3 Why the Fourier basis appears

A circulant operator commutes with cyclic shifts.

The natural basis is therefore the Fourier basis

$$\phi_k(n) = L^{-1/2} e^{2\pi i k n / L}.$$

Each mode is an eigenvector:

$$K_h \phi_k = \lambda_{h,k} \phi_k,$$

with

$$\lambda_{h,k} = \sum_{\Delta} \kappa_h(\Delta) e^{2\pi i k \Delta / L}.$$

So the operator becomes

$$K_h = \sum_k \lambda_{h,k} \phi_k \phi_k^*.$$

This is the core low-dimensional operator view.

A relative-position rule becomes a collection of labeled mode gains.

26.4 The DC mode

The zero-frequency mode is

$$\phi_0 = L^{-1/2} \mathbf{1}.$$

Its eigenvalue is

$$\lambda_{h,0} = \sum_{\Delta} \kappa_h(\Delta).$$

The corresponding operator is

$$K_h^{DC} = \frac{\lambda_{h,0}}{L} \mathbf{1} \mathbf{1}^\top.$$

It acts on the constant sequence-index mode.

This is what DC means here.

It is not an absolute-position embedding.

It contains no distinguished position label.

26.5 RoPE can have a feature-space DC block too

There is a parallel feature-space idea.

If a RoPE-style block uses

$$\omega = 0,$$

then

$$R(\omega\Delta) = I$$

for every offset.

That block is offset-invariant.

It is natural to call it a zero-frequency feature-space block.

But it is still not the same object as the sequence-index Fourier DC mode.

One lives in feature coordinates.

The other lives over positions.

26.6 Low-dimensional operator reduction

Suppose only a subset of Fourier modes is retained:

$$S \subseteq \{0, \dots, L-1\}.$$

Define

$$K_{h,S} = \sum_{k \in S} \lambda_{h,k} \phi_k \phi_k^*.$$

For a circulant operator,

$$\|K_h - K_{h,S}\|_F^2 = \sum_{k \notin S} |\lambda_{h,k}|^2,$$

and, when at least one mode is omitted,

$$\|K_h - K_{h,S}\|_2 = \max_{k \notin S} |\lambda_{h,k}|.$$

If $S = \{0, \dots, L-1\}$, no mode is omitted and $K_{h,S} = K_h$; both errors are exactly zero. This avoids interpreting a maximum over the empty set as a numerical value.

These are exact approximation statements.

They do not imply that the truncated operator preserves downstream language-model quality.

An operator norm is not a task metric.

26.7 Relative position in actual attention architectures

Shaw, Uszkoreit, and Vaswani introduced relative-position representations directly into self-attention rather than relying only on absolute input-position representations (Shaw et al. 2018).

T5 uses a simpler relative-position mechanism: a learned scalar positional bias is added to attention logits, and different heads within a layer use different learned relative-position embeddings (Raffel et al. 2020).

These architectures motivate an operator question:

what positional operator does each head implement before content-dependent attention is applied?

The Atlas answers that question mathematically without claiming the papers themselves performed the Fourier decomposition developed here.

26.8 Relative bias is not full attention

Suppose a head has relative-position logit bias

$$B_h(i, j) = b_h(j - i).$$

That positional component can be studied as an operator.

But the full attention logits are

$$L_h = Q_h K_h^\top / \sqrt{d} + B_h.$$

The attention weights are

$$A_h = \text{softmax}(L_h).$$

The content term changes from input to input.

Softmax is nonlinear.

Therefore diagonalizing the positional bias does not diagonalize the full attention mechanism in general.

This chapter diagnoses one component of attention, not the entire head.

26.9 Head-specific positional mode profiles

For a head h whose declared relative-position operator is nonzero, so that $\sum_j |\lambda_{h,j}|^2 > 0$, define

$$q_{h,k} = \frac{|\lambda_{h,k}|^2}{\sum_j |\lambda_{h,j}|^2}.$$

This is a probability distribution over labeled sequence-index modes. When $K_h = 0$, the denominator vanishes and the normalized profile is undefined; report a zero-energy operator instead of assigning it a fictitious frequency preference.

It answers a narrow question:

where does this head's relative-position operator place its spectral energy?

That is a useful notion of positional specialization.

It is not yet semantic specialization.

It does not tell us whether the head is causally necessary for a task.

26.10 Exact two-head witness

Take four cyclic positions.

Head A has kernel

$$\kappa_A = \left(1, \frac{1}{2}, 0, \frac{1}{2}\right).$$

Its mode eigenvalues are

$$\lambda_A = (2, 1, 0, 1).$$

Head B has

$$\kappa_B = \left(1, -\frac{1}{2}, 0, -\frac{1}{2}\right),$$

with

$$\lambda_B = (0, 1, 2, 1).$$

Both heads have the same unordered singular values:

$$(2, 1, 1, 0).$$

If we looked only at the singular-value multiset, they would appear identical.

But the mode labels reveal a sharp difference.

26.11 One head is DC-dominant

Head A has energy profile

$$q_A = \left(\frac{2}{3}, \frac{1}{6}, 0, \frac{1}{6} \right).$$

Two-thirds of its spectral energy lies in the DC mode.

Its DC component is

$$K_A^{DC} = \frac{1}{2} \mathbf{1} \mathbf{1}^\top.$$

A DC-only approximation has Frobenius error

$$\sqrt{2}.$$

26.12 The other head is Nyquist-dominant

Head B has

$$q_B = \left(0, \frac{1}{6}, \frac{2}{3}, \frac{1}{6} \right).$$

Its DC eigenvalue is zero.

Two-thirds of its spectral energy lies in the $k = 2$ mode, the Nyquist mode on four cyclic positions.

A DC-only approximation is therefore the zero operator and has Frobenius error

$$\sqrt{6}.$$

The same approximation strategy can be reasonable for one head and useless for another.

26.13 Singular values lose mode identity

The witness gives the chapter’s main finite warning:

same singular values $\nRightarrow$ same positional-frequency specialization

Singular values retain magnitude.

They do not retain which Fourier mode carries that magnitude.

For positional operators, the label can be the interesting part.

26.14 Head specialization needs a qualifier

T5 gives an architectural precedent for different relative-position parameters across heads.

That does not mean every head develops a meaningful positional role.

A head-specific profile can differ because of:

- initialization;
- optimization;
- architecture;
- redundancy;
- incidental parameterization;
- genuinely useful positional structure.

Therefore this chapter uses the phrase **positional-operator specialization**.

Semantic or causal specialization requires additional evidence.

26.15 The DC component is not always desirable

A large DC component means strong response to the constant sequence-index mode.

That may be useful.

It may also be an uninformative offset-like component.

The meaning depends on how the positional operator enters the model.

RPO therefore reports DC energy before interpreting it.

26.16 Boundary conditions matter

The Fourier diagonalization above is exact because the toy operator is circulant.

Real relative-position schemes may use:

- finite clipping;
- logarithmic buckets;
- causal masks;
- nonperiodic boundaries;
- asymmetric left/right treatment;
- context-length-dependent rules.

Those operations break or modify simple circulant structure.

The right generalization may involve Toeplitz analysis, block structure, finite-section operators, or numerical spectral diagnostics rather than a single exact DFT.

26.17 Bucketing is an operator approximation too

T5 maps ranges of relative offsets into shared learned buckets.

That can be viewed as reducing the degrees of freedom of the positional operator.

But bucket reduction is not the same as Fourier truncation.

One constrains the kernel in offset space.

The other constrains it in mode space.

The two approximations preserve different structures.

26.18 Frequency truncation can be head-specific

Suppose a system wants a low-dimensional positional operator.

A global rule such as “keep the DC mode first” can fail badly when different heads emphasize different modes.

The exact witness makes this concrete:

- head A: DC-only error $\sqrt{2}$;
- head B: DC-only error $\sqrt{6}$.

A head-specific reduction can therefore be mathematically better even before any task-specific evaluation is considered.

26.19 Multi-frequency RoPE and sequence modes should not be collapsed

RoPE already uses multiple frequencies.

RPO also uses multiple sequence-index Fourier modes.

The notation can look temptingly similar.

But the two decompositions diagonalize different actions.

RoPE blocks encode relative phase in feature coordinates.

Sequence Fourier modes diagonalize a translation-invariant operator acting across positions.

A model can contain both structures at once.

26.20 What a positional-operator diagnostic should record

At minimum:

- the space on which the operator acts;
- the sequence boundary convention;
- the relative-position kernel or bias rule;
- clipping or bucketing;
- whether the operator is before or after content interaction;
- the head/layer identity;
- the basis used for decomposition;
- retained mode labels, not only sorted magnitudes;
- the norm used for approximation error;
- any task metric used to interpret the approximation.

Without these fields, “low-frequency positional behavior” can refer to several different mathematical objects.

26.21 Handoff to Latent Clocks

Observed sequence position and latent time are not the same thing.

But relative-position operators supply useful structure for asking whether a model treats some offsets as equivalent, periodic, slowly varying, or high-frequency.

ATLAS-CH-LATENTTIME-001 may inherit:

- feature-space relative-offset operators;
- sequence-index relative-position kernels;
- Fourier-mode decomposition;
- DC components;
- exact low-dimensional truncation errors;
- head-specific positional-mode profiles;
- the finite example separating singular values from labeled frequencies.

LATENTTIME must add the inference problem:

when do observed sequence offsets correspond to an underlying latent temporal coordinate, and how should asynchronous or warped sequences be aligned?

26.22 Durable lesson

A positional mechanism is easier to reason about once we ask:

what operator acts on what space?

Then frequency, DC structure, truncation, and head-specific variation become typed mathematical questions instead of metaphors.

The operator view does not replace empirical evaluation.

It makes the positional object explicit enough to evaluate.

26.23 References used in this chapter

- (Shaw et al. 2018)
- (Raffel et al. 2020)

Exact source authority, prerequisite identities, and claim boundaries are locked in:

`sources/source-locks/ATLAS-CH-RPO-001.yaml`

Chapter 27

Latent Clocks

Epistemic status: audited Relative-Position Operators and Dynamics substrates + primary time-warping sources + Atlas-owned exact finite alignment witness.

Specification: manuscript/specifications/ATLAS-CH-LATENTTIME-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-LATENTTIME-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-LATENTTIME-001.md

Source lock: sources/source-locks/ATLAS-CH-LATENTTIME-001.yaml

Sequence position is observed.

Time can be inferred.

Those statements are not the same.

A token, frame, sensor sample, or event has an index in an observed stream. But two streams can progress through the same underlying phase at different rates, with pauses, duplicates, missing samples, or modality-specific distortions.

This chapter asks:

when should observed offsets be replaced by an inferred alignment coordinate?

The governing distinction is:

observed sequence index $\neq$ latent temporal alignment.

And the governing caution is:

optimal alignment $\neq$ proof of a uniquely true physical clock.

27.1 The handoff from positional operators

Relative-Position Operators already separates:

- feature-space RoPE rotations;
- sequence-index relative-position kernels;
- Fourier modes over positional operators;
- DC structure;
- low-dimensional positional approximations.

Those are positional objects.

They can tell us that a model treats certain offsets as:

- similar;
- periodic;
- slowly varying;
- high-frequency.

They do not by themselves tell us that two asynchronous streams are at the same latent phase.

LATENTTIME adds that inference problem.

27.2 The handoff from Dynamics

Dynamics already separates:

- state;
- trajectory;
- vector field;
- flow;
- discrete update.

That distinction matters because time-warping produces a discrete correspondence path.

A correspondence path is not automatically a trajectory of a continuous dynamical system.

The chapter therefore keeps:

alignment

separate from:

law of motion.

27.3 Two observed streams

Let:

$$X = (x_1, \dots, x_n),$$

$$Y = (y_1, \dots, y_m).$$

The indices:

$$i = 1, \dots, n, \quad j = 1, \dots, m$$

refer to positions in different observed sequences.

There is no reason in general to assume:

$$i = j$$

means “same latent time.”

27.4 Dynamic time warping

Dynamic time warping searches over monotone correspondences between two ordered streams.

For the finite Atlas formalism, an admissible path is:

$$P = ((i_1, j_1), \dots, (i_L, j_L))$$

with:

$$(i_1, j_1) = (1, 1),$$

$$(i_L, j_L) = (n, m),$$

and local increments:

$$(1, 0), \quad (0, 1), \quad (1, 1).$$

These steps allow one observed sample in one stream to align with more than one consecutive sample in the other.

27.5 The path is monotone

Indices never move backward.

This encodes an ordering assumption:

later aligned observations cannot be matched to earlier positions
after the path has passed them.

That is already a modeling choice.

A system with genuine temporal reversals or reordering would need a different alignment object.

27.6 A local discrepancy

For scalar witness sequences use:

$$c(i, j) = (x_i - y_j)^2.$$

The cost of a path is:

$$C(P) = \sum_{(i,j) \in P} c(i, j).$$

The optimal path solves:

$$\boxed{\text{DTW}(X, Y) = \min_{P \in \mathcal{P}_{n,m}} C(P).}$$

The optimum is relative to the declared cost and path class.

27.7 Dynamic programming

Let:

$$D(i, j)$$

be the minimum cost of an admissible path ending at (i, j) .

Then:

$$D(i, j) = c(i, j) + \min\{D(i-1, j), D(i, j-1), D(i-1, j-1)\}.$$

This is the recursive structure behind the finite alignment problem.

The terminal value:

$$D(n, m)$$

is the exact optimum.

27.8 The exact warped witness

Use:

$$X = (0, 1, 2),$$

$$Y = (0, 0, 1, 2).$$

The first stream has three observations.

The second has four.

The second stream repeats the initial value.

That duplicate can represent many things:

- slower local progress;
- repeated sampling;
- a plateau;
- asynchronous acquisition.

The witness does not choose among those explanations.

27.9 The local-cost matrix

The exact squared-distance matrix is:

$$\begin{pmatrix} 0 & 0 & 1 & 4 \\ 1 & 1 & 0 & 1 \\ 4 & 4 & 1 & 0 \end{pmatrix}.$$

The zero-cost cells are exactly:

$$(1, 1), (1, 2), (2, 3), (3, 4).$$

27.10 There are 25 admissible paths

Under the declared endpoint and step constraints, exhaustive enumeration gives:

$$\boxed{25}$$

admissible paths from:

$$(1, 1)$$

to:

$$(3, 4).$$

The path-count recurrence is:

$$N(i, j) = N(i - 1, j) + N(i, j - 1) + N(i - 1, j - 1).$$

The terminal count is:

$$N(3, 4) = 25.$$

27.11 The unique zero-cost alignment

Because every local cost is nonnegative, a zero-cost path can use only zero-cost cells.

Only one admissible chain connects all required endpoints through those cells:

$$\boxed{P^* = ((1, 1), (1, 2), (2, 3), (3, 4)).}$$

Thus:

$$\boxed{\text{DTW}(X, Y) = 0.}$$

And the optimum is unique.

27.12 Observed index has separated from alignment phase

The first two observations of Y :

$$y_1 = 0, \quad y_2 = 0$$

both align to:

$$x_1 = 0.$$

Then:

$$y_3 = 1$$

aligns to:

$$x_2 = 1,$$

and:

$$y_4 = 2$$

aligns to:

$$x_3 = 2.$$

Therefore:

$$j = 2$$

in one stream does not correspond to:

$$i = 2$$

in the other.

This is the smallest exact latent-clock lesson.

27.13 A common alignment coordinate

Index the path itself by:

$$\ell = 1, \dots, L.$$

Each latent alignment position carries a pair:

$$(i_\ell, j_\ell).$$

This gives a common ordered phase coordinate.

It is useful to call this a latent alignment coordinate.

It is not yet justified to call it physical time.

27.14 Why “clock” is a dangerous word

A clock suggests something stronger than alignment.

It can suggest:

- elapsed physical duration;
- causal progression;
- an intrinsic phase variable;
- a unique temporal parameterization.

DTW alone gives none of those automatically.

It gives an optimizer of an alignment objective.

The chapter therefore uses “latent clock” as an inference object whose epistemic status must remain explicit.

27.15 Identity/no-warp control

Now use:

$$X_0 = (0, 1, 2), \quad Y_0 = (0, 1, 2).$$

The two streams have equal length and already progress in the same observed order.

There are:

13

admissible paths under the same rule.

27.16 The diagonal is uniquely optimal

The local-cost matrix is:

$$\begin{pmatrix} 0 & 1 & 4 \\ 1 & 0 & 1 \\ 4 & 1 & 0 \end{pmatrix}.$$

The only zero-cost cells are diagonal.

Therefore the unique zero-cost path is:

$$\boxed{((1, 1), (2, 2), (3, 3)).}$$

So:

$$\text{DTW}(X_0, Y_0) = 0$$

without any warp.

27.17 The control prevents a false lesson

The warped witness alone could tempt us to think that latent-time machinery always invents a nontrivial warp.

The control shows otherwise.

When observed indices already match the phase structure, the exact optimum is the identity alignment.

Thus:

warping is inferred when supported by the declared data and cost, not imposed by default.

27.18 Constraints are part of the model

Changing the allowed local steps changes the feasible path set.

Adding a slope band or window can also change:

- the optimum;
- its cost;
- uniqueness.

Therefore path constraints are not implementation details.

They are assumptions.

Sakoe and Chiba's classical formulation makes time-normalization and slope constraints part of the alignment problem (Sakoe and Chiba 1978).

27.19 A unique optimum is still model-relative

The finite witness has a unique optimum.

But that means only:

there is one best path under this observation space, squared cost, endpoint rule, and step set.

Change any of those, and the optimum may change.

Therefore:

unique optimum $\nrightarrow$ unique true time.

27.20 Multimodal alignment changes the comparison problem

Suppose one stream is:

- video;

and another is:

- motion capture;
- audio;
- physiology;
- text events.

The raw coordinates are not generally comparable.

A reasonable framework needs representation maps:

$$\phi_X : \mathcal{X} \rightarrow \mathcal{Z},$$

$$\phi_Y : \mathcal{Y} \rightarrow \mathcal{Z}.$$

Then local cost can be computed in the comparison space:

$$d_Z(\phi_X(x_i), \phi_Y(y_j)).$$

27.21 Generalized time warping

Zhou and De la Torre explicitly treat multimodal temporal alignment and allow richer monotonic warping structure (Zhou and De la Torre 2012).

That is useful precedent for LATENTTIME because the inferred clock need not be tied to one raw sensor space.

But the Atlas does not inherit a universal multimodal representation.

The representation and discrepancy remain part of the declared model.

27.22 Asynchrony is not one phenomenon

Two streams can become misaligned because of:

- different sampling rates;
- missing observations;
- duplicated observations;
- variable execution speed;
- buffering;
- latency;
- sensor-specific preprocessing;
- genuinely different event timing.

A low-cost warp cannot by itself identify which explanation is correct.

27.23 Positional frequency is not a clock

RPO can expose a strong low-frequency positional mode.

That can mean the operator varies slowly with sequence offset.

It does not imply that the same mode tracks:

- elapsed time;
- behavioral phase;
- event age;
- causal stage.

Thus:

positional Fourier structure $\neq$ latent temporal coordinate.

27.24 RoPE frequency is not elapsed time either

A RoPE frequency describes rotation in feature coordinates as position changes.

Its phase is an algebraic function of positional offset.

That does not automatically make it a physical or semantic clock.

The same word “phase” can describe different mathematical objects.

LATENTTIME keeps them typed.

27.25 Discrete alignment is not a continuous trajectory

The DTW path is:

$$((i_1, j_1), \dots, (i_L, j_L)).$$

This is a finite combinatorial object.

A continuous trajectory is something like:

$$t \mapsto x(t)$$

generated under an explicit dynamical model.

Those objects can be related only after adding:

- interpolation;
- parameterization;
- regularity assumptions;
- possibly a law of motion.

Therefore:

$\text{DTW path} \not\approx \text{continuous-time flow.}$

27.26 Continuous-time latent clocks require more structure

If a model claims a scalar latent time:

$$\tau$$

with continuous evolution, it should specify how:

$$i \mapsto \tau_i$$

or:

$$j \mapsto \tau_j$$

is obtained.

It should also specify whether τ is:

- merely monotone;
- metrically meaningful;
- differentiable;
- dynamically generated.

A monotone ordering alone does not give a metric clock.

27.27 Alignment is not causality

Suppose audio and video align tightly under a learned warp.

That supports temporal correspondence under the model.

It does not prove:

- audio causes video;
- video causes audio;
- one latent system drives both.

Thus:

alignment $\nRightarrow$ causal direction.

27.28 Low cost is not semantic identity

Two sequences can be numerically easy to align while representing different semantics.

Conversely, semantically corresponding events can look very different in raw modalities.

So:

low alignment cost $\nRightarrow$ semantic identity.

27.29 Non-unique alignments should stay non-unique

Some data admit multiple optimal paths.

A responsible system should not silently pretend one arbitrary tie-break is the discovered clock.

It should report:

- multiplicity;
- tie-breaking;
- alignment uncertainty;
- downstream sensitivity where relevant.

The exact Atlas witness avoids this only because the optimum happens to be unique.

27.30 Partial and missing-data alignment

The core witness uses full endpoint-to-endpoint matching.

Real streams may require:

- subsequence matching;
- open-begin or open-end paths;
- partial matching;
- missing-event models.

Those are different problem definitions.

They should be declared rather than smuggled into “DTW.”

27.31 A useful latent-clock protocol

For two asynchronous streams:

1. define the observed streams;
2. define any representation maps;
3. define the local discrepancy;
4. define endpoints and path constraints;
5. compute the optimal path;
6. test an identity/no-warp control where possible;
7. inspect uniqueness or multiplicity;
8. report the induced alignment coordinate;
9. keep physical-clock claims separate;
10. test downstream usefulness independently.

27.32 When the alignment coordinate is useful

An inferred alignment coordinate can support:

- cross-modal fusion;

- event matching;
- phase-normalized comparison;
- memory alignment;
- asynchronous agent communication;
- trajectory comparison.

Its usefulness can be empirical even when it is not a uniquely true physical time variable.

27.33 Atlas non-implications

The chapter rejects:

observed position $\nRightarrow$ latent time,

unique optimal alignment $\nRightarrow$ unique true clock,

periodic positional mode $\nRightarrow$ temporal clock,

alignment $\nRightarrow$ causality,

discrete warping path $\nRightarrow$ continuous-time flow,

and:

low alignment cost $\nRightarrow$ semantic identity.

27.34 Atlas connections

Relative-Position Operators.

RPO supplies operator/frequency structure over observed position; LATENT-TIME adds alignment inference across asynchronous indices.

Dynamics.

Dynamics supplies the distinction between sampled states, trajectories, and flows; LATENTTIME keeps discrete warping separate from continuous-time laws.

Multimodal systems.

A shared alignment coordinate can organize observations from heterogeneous sensors or representations.

Agent systems.

Different agents can observe related events on different local clocks; alignment can provide a common phase coordinate without assuming synchronized wall time.

Memory.

Episodes with different sampling density can be compared after an explicit alignment transformation.

27.35 Closing view

Position is cheap to observe.

Time is often not.

A sequence index tells us where a sample sits in a record.

It does not necessarily tell us where that sample sits in a shared process.

The exact witness shows the smallest nontrivial case:

$$X = (0, 1, 2), \quad Y = (0, 0, 1, 2).$$

The second stream lingers on one phase.

The optimal correspondence therefore repeats one alignment state:

$$(1, 1) \rightarrow (1, 2) \rightarrow (2, 3) \rightarrow (3, 4).$$

The identity control shows the same machinery leaves already aligned streams unwarped.

The right conclusion is:

latent time is an inferred correspondence structure whose meaning depends on the declared alignment

It can be useful without being a metaphysical clock.

27.36 References used in this chapter

- (Sakoe and Chiba 1978)
- (Zhou and De la Torre 2012)

Exact source identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-LATENTTIME-001.yaml`

Chapter 28

First-Order Optimization

Epistemic status: Established Theory + Bounded Deep-Learning Practice + Exact Computational Witness

Specification: [manuscript/specifications/ATLAS-CH-OPTBASE-001.md](#)

Derivation packet: [mathematics/derivations/ATLAS-CH-OPTBASE-001-DERIVATIONS.md](#)

Computational witness: [mathematics/computational-witnesses/ATLAS-CW-OPTBASE-001.md](#)

Source lock: [sources/source-locks/ATLAS-CH-OPTBASE-001.yaml](#)

28.1 The gradient is not the optimizer

Under the Euclidean inner product on parameter coordinates, the gradient answers a local question:

Among unit Euclidean directions, which infinitesimal direction increases the objective fastest?

For nonzero gradient, that direction is

$$\frac{\nabla F}{\|\nabla F\|_2}.$$

Its negative is the Euclidean steepest-descent direction.

Later geometry chapters will change the metric and therefore change what “steepest” means.

An optimizer answers a different question:

Given that signal, how should the parameters actually move?

That second question introduces choices:

- step size;
- momentum;
- stochastic sampling;
- coordinatewise scaling;
- clipping;
- weight decay;
- schedules;
- initialization;
- optimizer state.

The Atlas therefore treats

$$g_k$$

and

$$\Delta\theta_k$$

as different objects.

The gradient is a signal.

The optimizer converts that signal into motion.

28.2 Steering versus changing the road

A useful allegory is steering a vehicle down a sloped landscape.

The gradient indicates local downhill direction.

The learning rate controls how far we move.

Momentum carries history.

A preconditioner changes the coordinatewise steering response.

Clipping acts like a speed limiter.

Weight decay adds shrinkage.

A schedule changes the control policy over time.

The structural correspondence is useful.

The literal-mechanics analogy is not.

Optimization state is algorithmic state, not physical momentum unless a specific mathematical correspondence is established.

28.3 Gradient descent

For a differentiable objective

$$F(\theta),$$

gradient descent uses

$$\theta_{k+1} = \theta_k - \eta \nabla F(\theta_k).$$

This is already a discrete dynamical system:

$$\theta_{k+1} = \Psi(\theta_k).$$

The learning rate η is not simply a “speed” parameter.

It changes the discrete map itself.

A step size can therefore alter stability.

28.4 Exact quadratic calibration

Take

$$F(x) = \frac{\lambda}{2} x^2, \quad \lambda > 0.$$

Then

$$\nabla F(x) = \lambda x,$$

so

$$x_{k+1} = (1 - \eta\lambda)x_k.$$

Thus

$$x_k = (1 - \eta\lambda)^k x_0.$$

The dynamics are completely determined by the scalar amplification factor

$$r = 1 - \eta\lambda.$$

28.5 Step-size stability

For asymptotic convergence to zero,

$$|r| < 1.$$

Therefore

$$|1 - \eta\lambda| < 1,$$

which gives

$$0 < \eta < \frac{2}{\lambda}.$$

At

$$\eta = 0,$$

the iterate does not move.

At

$$\eta = \frac{2}{\lambda},$$

the factor is -1 , so the iterate alternates sign without decaying.

Beyond that boundary, the discrete method diverges.

Corrected-edition figure candidate: This plate is the Wolfram Language 15.0.1 evaluator derivative in `figures/derivatives/ATLAS-FIG-OPTBASE-001-v0.1.2.yaml`, which replaces the malformed historical Wolfram `GraphicsGrid` layout without changing the toy mathematics. The earlier Matplotlib derivative remains separately attributed as a superseded editorial alternative. The original Wolfram source and master remain unchanged. Print-scale and accessibility acceptance of the derivative is still pending under #324.

28.6 Curvature sets the useful scale

The bound

$$0 < \eta < \frac{2}{\lambda}$$

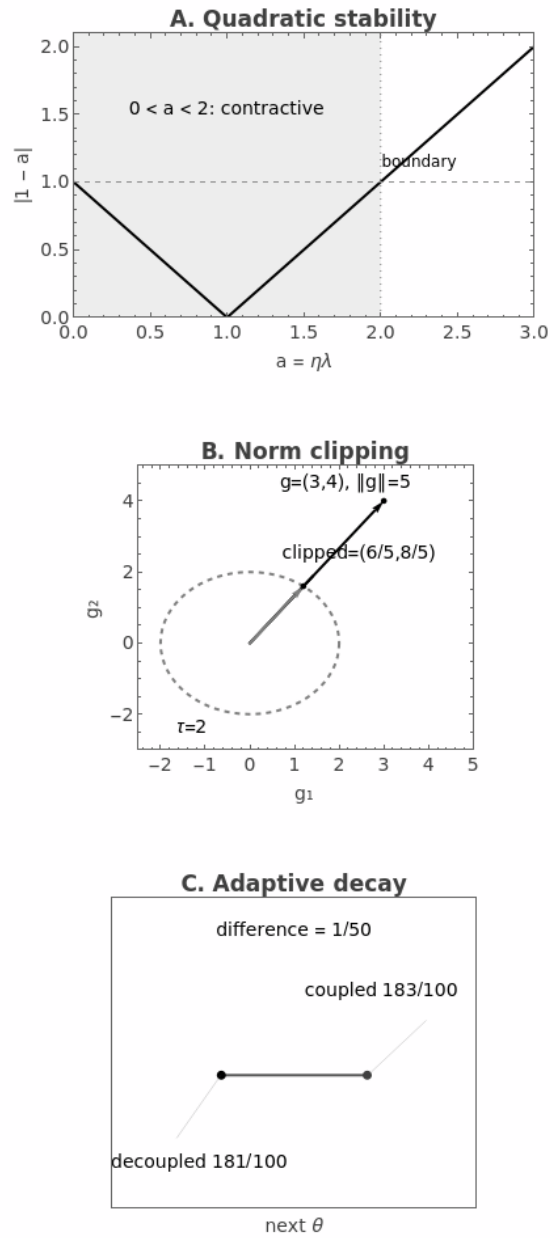


Figure 28.1: Three exact panels: scalar quadratic gradient-descent amplification and its convergence interval; norm clipping of $g=(3,4)$ to threshold 2; and the exact coupled-versus-decoupled adaptive decay next iterates $183/100$ and $181/100$.

belongs to the declared quadratic curvature λ .

Change the curvature and the stability range changes.

For a multidimensional quadratic with Hessian eigenvalues

$$\lambda_i,$$

one expects the most restrictive positive curvature scale to limit a simple constant step.

This is the first hint that optimization should be read spectrally, not only coordinatewise.

Later chapters will develop that point.

28.7 Full gradient versus stochastic gradient

Suppose

$$F(\theta) = \mathbb{E}_{\xi}[\ell(\theta; \xi)].$$

A stochastic-gradient method uses a random sample or minibatch gradient

$$g(\theta; \xi)$$

in place of the exact expectation.

The ideal unbiased relation is

$$\mathbb{E}_{\xi}[g(\theta; \xi)] = \nabla F(\theta).$$

Unbiasedness is an expectation statement.

It says nothing by itself about variance.

28.8 Exact unbiased-SGD witness

Take two sample losses:

$$\ell_1(x) = \frac{1}{2}(x - 1)^2,$$

$$\ell_2(x) = \frac{1}{2}(x + 1)^2.$$

Sample uniformly.

The finite-average objective is

$$F(x) = \frac{1}{2}(\ell_1 + \ell_2) = \frac{1}{2}x^2 + \frac{1}{2}.$$

Therefore

$$\nabla F(x) = x.$$

The sample gradients are

$$g_1 = x - 1, \quad g_2 = x + 1.$$

Their expectation is

$$\frac{1}{2}(g_1 + g_2) = x.$$

So the estimator is unbiased.

Its variance is

$$1.$$

Thus:

unbiased $\nRightarrow$ low variance.

Robbins and Monro provide the foundational stochastic-approximation setting behind this distinction (Robbins and Monro 1951).

28.9 Noise can be useful and harmful

Stochasticity can:

- reduce per-step cost;
- perturb iterates out of narrow regions;
- act as implicit regularization;
- increase estimator variance;
- destabilize large steps;
- complicate convergence diagnostics.

There is no single sign.

A gradient-noise model must be tied to:

- sampling scheme;
- batch size;
- data distribution;
- optimizer;
- current parameter state.

The Atlas will not treat “SGD noise” as one universal distribution.

28.10 Momentum adds state

One heavy-ball convention is

$$v_{k+1} = \beta v_k + g_k,$$

$$\theta_{k+1} = \theta_k - \eta v_{k+1}.$$

The optimizer state is now

$$s_k = (\theta_k, v_k).$$

This is a conceptual change.

The update no longer depends only on the current gradient.

It depends on history carried in v_k .

Polyak’s work supplies the classical acceleration context (Polyak 1964).

28.11 Momentum conventions differ

Some implementations define velocity with the opposite sign.

Some absorb η into the momentum variable.

Some write updates directly as parameter differences.

Therefore two formulas both called “momentum” need not look identical.

The Atlas uses equations, not labels, as the canonical object.

This discipline becomes essential when we later study optimizer-state Jacobians.

28.12 Nesterov-style lookahead

A common lookahead form evaluates the gradient at a predicted state:

$$g_k = \nabla F(\theta_k - \eta\beta v_k),$$

followed by

$$v_{k+1} = \beta v_k + g_k,$$

$$\theta_{k+1} = \theta_k - \eta v_{k+1}.$$

The important structural change is:

the gradient is evaluated at a lookahead point rather than the current parameter point.

Nesterov's convex-optimization theory gives the classical accelerated context (Nesterov 2004).

The Atlas does not transfer convex rate guarantees automatically to deep non-convex networks.

28.13 Deep-learning momentum

Sutskever and colleagues provide an influential deep-learning study of momentum and initialization (Sutskever et al. 2013).

That work is useful here as a bridge between classical first-order machinery and practical neural-network training.

It is not used to rank momentum universally above all alternatives.

28.14 AdaGrad

AdaGrad accumulates squared gradients coordinatewise:

$$G_{k,i} = G_{k-1,i} + g_{k,i}^2.$$

A standard coordinate update is

$$\theta_{k+1,i} = \theta_{k,i} - \eta \frac{g_{k,i}}{\sqrt{G_{k,i} + \varepsilon}}.$$

Coordinates with large historical squared gradients receive smaller effective step scales (Duchi et al. 2011).

This is more than a changing learning rate.

It is coordinate-dependent scaling.

28.15 Adaptive scaling as a diagonal geometry

Write a generic first-order update as

$$\theta_{k+1} = \theta_k - \eta P_k g_k,$$

where P_k is diagonal and positive.

When

$$P_k = I,$$

we recover ordinary gradient descent.

When P_k depends on accumulated gradient statistics, the algorithm changes the relative step size across coordinates.

This is the beginning of a geometric viewpoint.

Later Atlas chapters will replace diagonal scaling with richer matrix and manifold structure.

28.16 RMSProp

RMSProp uses an exponential moving average of squared gradients:

$$v_k = \rho v_{k-1} + (1 - \rho) g_k^2,$$

coordinatewise.

Then

$$\theta_{k+1} = \theta_k - \eta \frac{g_k}{\sqrt{v_k} + \varepsilon}.$$

The original RMSProp description is course lecture material rather than a peer-reviewed paper (Tieleman and Hinton 2012).

The Atlas marks that source status explicitly.

28.17 Adam

Adam combines two moving averages (Kingma and Ba 2015).

First moment:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k.$$

Second raw moment:

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2.$$

With zero initialization, both estimates are biased toward zero early in training.

Adam corrects that initialization bias.

28.18 Bias correction

The corrected moments are

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k},$$

and

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^k}.$$

A standard update is

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}.$$

Bias correction is an algebraic adjustment.

It is not a proof that the full optimizer is unbiased or universally convergent.

28.19 Exact Adam first step

Take

$$g_1 = 3,$$

$$\beta_1 = \frac{9}{10},$$

$$\beta_2 = \frac{99}{100},$$

with

$$m_0 = v_0 = 0.$$

Then

$$m_1 = \frac{3}{10},$$

and

$$v_1 = \frac{9}{100}.$$

After bias correction:

$$\hat{m}_1 = 3,$$

$$\hat{v}_1 = 9.$$

For the calibration

$$\varepsilon = 0,$$

the normalized direction is exactly

$$1.$$

Practical implementations use nonzero epsilon.

The zero-epsilon witness exists only to isolate the correction algebra.

28.20 Adaptive methods are not one algorithm

AdaGrad, RMSProp, and Adam all use gradient-history information.

But they differ in:

- accumulation versus exponential forgetting;
- first-moment state;
- bias correction;

- denominator construction;
- epsilon placement;
- update ordering.

Treating them as one generic “adaptive optimizer” can hide mathematically important differences.

28.21 Gradient clipping

Global norm clipping at threshold τ uses

$$g_{\text{clip}} = g \min \left(1, \frac{\tau}{\|g\|_2} \right).$$

If the norm is below threshold, nothing changes.

If the norm is above threshold, the vector is rescaled to norm τ .

Pascanu, Mikolov, and Bengio discuss clipping in the context of exploding gradients in recurrent networks (Pascanu et al. 2013).

28.22 Exact clipping geometry

Take

$$g = (3, 4).$$

Then

$$\|g\|_2 = 5.$$

With threshold

$$\tau = 2,$$

the scale factor is

$$2/5.$$

Therefore

$$g_{\text{clip}} = (6/5, 8/5).$$

Its norm is exactly

$$2.$$

The direction is unchanged in this global-norm scheme.

The magnitude is not.

28.23 Clipping changes the estimator

Clipping is nonlinear.

Therefore in general

$$\mathbb{E}[\text{clip}(g)] \neq \text{clip}(\mathbb{E}[g]).$$

Even if the original stochastic gradient is unbiased,

$$\mathbb{E}[g] = \nabla F,$$

the clipped gradient need not remain unbiased.

This is not necessarily a defect.

It is simply a different estimator/update rule.

The distinction should be explicit.

28.24 Weight decay and L2 regularization

For plain gradient descent, adding an L2 penalty

$$\frac{\lambda}{2} \|\theta\|^2$$

produces gradient contribution

$$\lambda\theta.$$

The update becomes

$$\theta^+ = \theta - \eta(g + \lambda\theta).$$

Rearranging,

$$\theta^+ = (1 - \eta\lambda)\theta - \eta g.$$

In this simple setting, L2 regularization and multiplicative weight shrinkage coincide algebraically.

That equivalence does not survive arbitrary preconditioning.

28.25 Coupled regularization under adaptive scaling

Suppose a scalar preconditioner p scales the gradient.

If L2 regularization is coupled into the gradient,

$$\theta_c^+ = \theta - \eta p(g + \lambda\theta).$$

The preconditioner acts on the shrinkage gradient too.

This changes the effective decay.

28.26 Decoupled weight decay

AdamW separates shrinkage from the adaptive gradient step (Loshchilov and Hutter 2019).

In the scalar calibration:

$$\theta_d^+ = (1 - \eta\lambda)\theta - \eta p g.$$

Subtracting coupled from decoupled gives

$$\boxed{\theta_c^+ - \theta_d^+ = \eta\lambda\theta(1 - p).}$$

They coincide only under special conditions.

28.27 Exact decay counterexample

Take

$$\theta = 2, \quad g = 3, \quad p = \frac{1}{2}, \quad \eta = \frac{1}{10}, \quad \lambda = \frac{1}{5}.$$

Coupled regularization gives

$$\theta_c^+ = \frac{183}{100}.$$

Decoupled decay gives

$$\theta_d^+ = \frac{181}{100}.$$

The exact difference is

$$\boxed{\frac{1}{50}}.$$

So under adaptive scaling:

$$\boxed{\text{coupled L2} \neq \text{decoupled weight decay}}$$

in general.

28.28 Learning-rate schedules

A constant learning rate uses

$$\eta_k = \eta.$$

A schedule makes the step scale time-dependent:

$$\eta_k.$$

Common patterns include:

- piecewise decay;
- exponential or inverse-power decay;
- cosine schedules;
- warmup plus decay.

A schedule changes the optimization dynamics over training time.

It is therefore part of the algorithm, not cosmetic metadata.

28.29 Warmup

Warmup deliberately begins with smaller step scales.

One influential example is the Transformer schedule of Vaswani et al., which combines warmup with inverse-square-root scaling (Vaswani et al. 2017).

The Atlas treats warmup as a training recipe with many possible rationales:

- unstable early statistics;
- large transient gradients;
- poorly calibrated optimizer moments;
- scale changes during early representation formation.

No universal theorem says that warmup is always necessary or optimal.

28.30 Initialization is part of optimization

Optimization starts before the first update.

The initial parameter distribution determines:

- activation scale;
- gradient scale;
- symmetry breaking;
- saturation risk;
- early Jacobian statistics.

Initialization therefore belongs in the optimization baseline.

It defines the starting state of the dynamical system.

28.31 Fan-in second moment

Consider

$$z = \sum_{i=1}^n w_i x_i.$$

Assume:

- independent zero-mean weights;
- independent zero-mean inputs;
- weight/input independence;
- common input second moment q ;
- common weight variance σ_w^2 .

Cross terms vanish.

Therefore

$$\mathbb{E}[z^2] = n\sigma_w^2 q.$$

This is the mathematical core of fan-in scaling.

28.32 Linear second-moment preservation

To keep

$$\mathbb{E}[z^2] = q,$$

choose

$$\sigma_w^2 = \frac{1}{n}.$$

Glorot and Bengio analyze related variance/saturation issues in deep feedforward networks (Glorot and Bengio 2010).

The derivation depends on its assumptions.

Real network activations are not guaranteed to remain independent or identically distributed across depth.

28.33 ReLU second moment

Suppose the preactivation distribution is symmetric around zero.

Then

$$\text{ReLU}(z)^2 = z^2 \mathbf{1}_{\{z > 0\}}.$$

By symmetry,

$$\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2} \mathbb{E}[z^2].$$

Therefore

$$\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2} n \sigma_w^2 q.$$

Choose

$$\sigma_w^2 = \frac{2}{n}$$

to preserve the second moment.

He and colleagues develop rectifier-aware initialization in this direction (He et al. 2015).

28.34 Second moment is not variance after ReLU

This wording matters.

ReLU output generally has positive mean.

Therefore

$$\text{Var}[\text{ReLU}(z)] \neq \mathbb{E}[\text{ReLU}(z)^2]$$

unless the mean term is handled.

The simple $2/n$ derivation above is an exact second-moment statement under symmetry assumptions.

It should not be inflated into an exact theorem about full deep-network activation distributions.

28.35 Optimization is a state machine

The simplest gradient method has state

$$\theta_k.$$

Momentum has

$$(\theta_k, v_k).$$

Adam has

$$(\theta_k, m_k, v_k).$$

Schedules introduce time or step counters.

Mixed precision may introduce loss-scaling state.

Gradient accumulation introduces buffering state.

So even “first-order optimization” quickly becomes a structured state machine. This prepares the later Optimizer-State Dynamics chapter.

28.36 Coordinate scaling already hints at geometry

Ordinary gradient descent treats parameter coordinates through the Euclidean metric implicit in

$$\nabla F.$$

Adaptive methods introduce coordinate-dependent scales.

That raises a question the baseline does not answer:

Which geometry should determine steepest descent?

Natural gradient, mirror descent, Shampoo, Muon, manifold methods, and MODULUS all answer that question differently.

They belong downstream.

28.37 Spectral effects are still hidden

The scalar quadratic example had one curvature scale λ .

Real parameter blocks can have many singular/eigen directions.

A diagonal adaptive method sees only coordinatewise statistics.

It does not directly diagonalize the true curvature or account for non-normal transient behavior.

That gap motivates:

- second-order methods;
- matrix preconditioning;
- spectral shaping;
- optimizer-state diagnostics.

28.38 Five failure modes

28.38.1 Large step mistaken for fast learning

Beyond the stability boundary, a larger step can destroy convergence.

28.38.2 Unbiased mistaken for low variance

The exact SGD witness has variance 1 despite being unbiased.

28.38.3 Clipping mistaken for harmless rescaling

Clipping changes the estimator whenever the threshold is active.

28.38.4 Coupled L2 mistaken for AdamW decay

Adaptive scaling breaks the simple equivalence.

28.38.5 Initialization heuristic mistaken for distribution theorem

Fan-in calculations rely on assumptions that deep networks can violate.

28.39 What would falsify an optimizer story?

A useful optimizer explanation should predict something measurable.

Examples:

- stability boundary;
- update norm;
- coordinate scale;
- momentum phase lag;
- decay behavior;
- clipping activation rate;
- variance propagation;
- sensitivity to schedule.

If the predicted mechanism is absent, the explanation is incomplete.

The Atlas will use this standard later when testing optimizer geometry and state dynamics.

28.40 Atlas connections**Dynamics.**

An optimizer defines a discrete dynamical system, often with hidden state.

Second-order optimization.

Curvature information can replace or augment first-order scaling.

Manifold optimization.

The notion of allowable motion changes when parameters live on constrained spaces.

Spectral shaping.

Matrix updates can be designed around singular/eigen structure rather than coordinates.

Muon.

Orthogonalized or spectrally shaped updates depart sharply from diagonal adaptive scaling.

CPS.

Optimizer-state Jacobians expose coupling and transient instability invisible in static update formulas.

Curriculum.

The optimization state interacts with which examples or tasks are presented.

28.41 Closing view

First-order optimization begins with a simple equation:

$$\theta_{k+1} = \theta_k - \eta g_k.$$

But practical training rapidly adds structure:

gradient $\rightarrow$ stochastic estimator $\rightarrow$ state $\rightarrow$ preconditioner $\rightarrow$ clip $\rightarrow$ decay $\rightarrow$ schedule.

Each addition changes the actual dynamical system.

That is the central lesson.

The baseline is not “gradient descent plus tricks.”

It is a family of stateful rules that translate local first-order information into parameter motion.

Later Atlas chapters will ask whether those rules should instead be derived from:

- curvature;
- geometry;
- spectra;
- manifolds;
- variational principles;
- compositional contracts.

To ask that question rigorously, we first needed to say exactly what the conventional baseline is.

28.42 References used in this chapter

- (Robbins and Monro 1951)
- (Polyak 1964)
- (Nesterov 2004)
- (Duchi et al. 2011)
- (Tieleman and Hinton 2012)
- (Kingma and Ba 2015)
- (Loshchilov and Hutter 2019)
- (Pascanu et al. 2013)
- (Glorot and Bengio 2010)
- (He et al. 2015)
- (Sutskever et al. 2013)
- (Vaswani et al. 2017)

See ‘sources/source-locks/ATLAS-CH-OPTBASE-001.yaml’ for exact source roles and claim boundaries.

Chapter 29

Curvature and Second-Order Structure

Epistemic status: mathematical exposition built from audited First-Order Optimization and Geometry prerequisites, classical second-order optimization sources, and Atlas-owned exact witnesses.

Derivation packet: ‘[mathematics/derivations/ATLAS-CH-SECOND-001-DERIVATIONS.md](#)‘

Computational witness: ‘[mathematics/computational-witnesses/ATLAS-CW-SECOND-001.md](#)‘

Source lock: ‘[sources/source-locks/ATLAS-CH-SECOND-001.yaml](#)‘

29.1 The gradient tells you a direction; curvature tells you how that direction changes

First-order optimization begins with a local slope.

For a differentiable objective f , the Euclidean gradient

$$g = \nabla f(x)$$

is the first-order signal.

It answers:

What infinitesimal direction changes the objective most rapidly under the declared Euclidean metric?

Second-order structure asks another question:

How does that first-order signal change as we move?

For twice-differentiable f , the Hessian

$$H = \nabla^2 f(x)$$

is the local linear map that differentiates the gradient.

Gradient magnitude and curvature are therefore different objects.

A small gradient can occur in a flat basin, at a sharp local extremum, or near a saddle.

Curvature helps distinguish those cases locally.

29.2 Local quadratic model

Near x , a twice-differentiable objective has the local model

$$m_x(p) = f(x) + g^\top p + \frac{1}{2} p^\top H p.$$

The first term is the current objective.

The linear term is the first-order prediction.

The quadratic term describes how the prediction bends.

If H is positive definite, the quadratic term bends upward in every nonzero direction.

If H is indefinite, some directions bend upward and others downward.

That sign structure is central.

29.3 Newton's equation

A stationary point of the local quadratic model satisfies

$$\nabla_p m_x(p) = g + H p = 0.$$

If H is nonsingular,

$$\boxed{p_N = -H^{-1}g.}$$

This is the Newton step.

It is tempting to read this as “divide the gradient by curvature.”

That phrase is useful only if we remember that curvature is a matrix, not a scalar.

The inverse Hessian rotates and rescales the gradient according to local curvature directions.

29.4 Positive-definite curvature gives a descent direction

Suppose

$$H \succ 0$$

and

$$g \neq 0.$$

Then

$$g^\top p_N = -g^\top H^{-1}g < 0.$$

So the Newton direction is locally descending.

This fact depends on positive definiteness.

It is not true for arbitrary invertible Hessians.

That boundary will matter immediately.

29.5 Exact positive-definite witness

Take

$$H = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

and

$$f(x) = \frac{1}{2}x^\top Hx - b^\top x.$$

At

$$x_0 = 0,$$

the gradient is

$$g_0 = -b = \begin{pmatrix} -1 \\ -1 \end{pmatrix}.$$

The Newton equation is

$$Hp = -g_0 = b.$$

Thus

$$p_N = H^{-1}b = \begin{pmatrix} 1 \\ 1/4 \end{pmatrix}.$$

Because the objective itself is quadratic, the local model is exact.

The step reaches the unique minimizer in one move.

The objective changes from

$$f(0) = 0$$

to

$$f(p_N) = -5/8.$$

The directional derivative is

$$g_0^\top p_N = -5/4 < 0.$$

This is the clean case.

29.6 Indefinite curvature breaks the simple Newton story

Now consider

$$f(x, y) = \frac{1}{2}(x^2 - y^2).$$

At

$$z_0 = (0, 1)^\top,$$

we have

$$g = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad H = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

The Newton equation gives

$$p_N = \begin{pmatrix} 0 \\ -1 \end{pmatrix}.$$

But

$$g^\top p_N = 1 > 0.$$

So the raw Newton direction is an ascent direction.

The full step moves to

$$(0, 0),$$

where

$$f(0, 0) = 0$$

instead of the starting value

$$f(0, 1) = -1/2.$$

The objective increases.

The Newton equation solved the stationary condition of the local quadratic model.

That stationary point was a saddle.

This is why second-order methods need more than “invert the Hessian.”

29.7 Curvature signatures

For a symmetric Hessian:

- all positive eigenvalues indicate locally positive curvature;
- all negative eigenvalues indicate locally negative curvature;
- mixed signs indicate an indefinite saddle-like quadratic model;
- zero eigenvalues indicate locally flat or degenerate directions.

This is local information.

A positive-definite Hessian at one point does not make the entire objective globally convex.

An indefinite Hessian at one point does not completely describe the global landscape.

29.8 Damped and regularized Newton systems

One response to difficult curvature is to modify the local system.

A common form is

$$(H + \lambda I)p = -g.$$

For sufficiently large positive λ , the shifted matrix may become positive definite.

This changes the local model.

It can improve stability.

But it is no longer the exact unmodified Newton step.

The Atlas keeps that distinction explicit:

$\text{exact Hessian} \neq \text{regularized Hessian.}$

29.9 Trust regions: constrain where the model is trusted

A different globalization strategy is to keep the quadratic model but limit how far we are willing to follow it.

A trust-region subproblem has the form

$$\min_p \left(g^\top p + \frac{1}{2} p^\top B p \right)$$

subject to

$$\|p\| \leq \Delta.$$

The radius Δ is not merely a clipping threshold.

It expresses a model-validity region.

The algorithm compares predicted and actual decrease and can adjust the radius accordingly (Nocedal and Wright 2006).

29.10 Exact trust-region control on the indefinite witness

Return to

$$f(x, y) = \frac{1}{2}(x^2 - y^2)$$

at

$$(0, 1).$$

Use trust-region radius

$$\Delta = 1.$$

The model increment is

$$q(p) = -p_2 + \frac{1}{2}p_1^2 - \frac{1}{2}p_2^2.$$

Over the unit disk, the exact minimizer is

$$p_{\text{TR}} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The model change is

$$q(p_{\text{TR}}) = -3/2.$$

The new point is

$$(0, 2),$$

where

$$f(0, 2) = -2.$$

The raw Newton step climbed toward the saddle.

The trust-region subproblem moved along negative curvature and decreased the objective.

This one example does not prove a universal trust-region advantage.

It shows why indefinite curvature needs different logic from positive-definite Newton.

29.11 Exact Hessians can be too large to store

For d parameters, a dense Hessian has d^2 entries.

That quickly becomes impractical.

But many second-order algorithms do not need the full matrix.

They need the action

$$Hv$$

on selected vectors.

Pearlmutter showed how such products can be computed exactly in the cited neural-computation setting without explicitly forming the dense Hessian (Pearlmutter 1994).

The fundamental identity is

$$Hv = \frac{d}{d\varepsilon} \nabla f(x + \varepsilon v) \Big|_{\varepsilon=0}.$$

This is the second-order analogue of matrix-free operator access.

29.12 Matrix-free second-order structure

Once Hessian-vector products are available, several operations become possible without storing H :

- curvature probing;
- iterative linear solves for Newton-like systems;
- extremal-eigenvalue estimation;
- conjugate-gradient methods in suitable positive-definite systems;
- trust-region subproblem methods.

This does not make second-order computation free.

The relevant costs move into repeated products, iterative solves, damping, preconditioning, and numerical stability.

29.13 Fisher information is a different matrix

Optimization discussions sometimes slide from “curvature” to “Fisher information” as if they were synonyms.

They are not.

For a probabilistic model

$$p_\theta(y),$$

the Fisher information matrix is built from score covariances, for example

$$F(\theta) = \mathbb{E} [\nabla_\theta \log p_\theta(y) \nabla_\theta \log p_\theta(y)^\top]$$

under the declared distribution.

The Hessian is a second derivative of a declared scalar objective.

These matrices can be related in important settings.

They are not generically identical.

29.14 Natural gradient

The ordinary gradient depends on the metric used to define steepest descent.

In the Euclidean metric, the local squared norm is

$$\|\delta\|_2^2 = \delta^\top \delta.$$

Under a positive-definite metric $G(\theta)$,

$$\|\delta\|_G^2 = \delta^\top G(\theta) \delta.$$

The steepest-descent direction changes to a direction proportional to

$$-G^{-1}g.$$

When G is the Fisher information metric, this gives the natural gradient (Amari 1998).

This is a geometric metric statement.

It is not automatically a Newton step.

29.15 Exact metric witness

Let

$$G = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}, \quad g = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Euclidean steepest descent uses

$$-g = \begin{pmatrix} -1 \\ -1 \end{pmatrix}.$$

Metric steepest descent uses

$$-G^{-1}g = \begin{pmatrix} -1 \\ -1/4 \end{pmatrix}.$$

The directions differ.

Changing the metric changes what “steepest” means.

The numerical coincidence that this matrix resembles the positive-definite Hessian in our earlier witness does not identify natural gradient with Newton.

The interpretation of the matrix is different.

29.16 Hessian versus Fisher

A useful discipline is to ask three separate questions:

1. What scalar objective is being differentiated?
2. What probability distribution defines the Fisher expectation?
3. What metric is being used to define steepest descent?

If those objects are not explicitly aligned, the words “curvature,” “Fisher,” and “natural gradient” should not be substituted for one another.

This prevents a common category error.

29.17 Quasi-Newton methods

Exact Hessians can be expensive.

Quasi-Newton methods instead update a matrix approximation using observed changes in gradients.

Let

$$s_k = x_{k+1} - x_k$$

and

$$y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

A Hessian approximation B_{k+1} may be required to satisfy the secant equation

$$B_{k+1}s_k = y_k.$$

This forces the approximation to match the observed curvature action along the most recent displacement.

It does not make B_{k+1} the exact Hessian in every direction.

29.18 BFGS and curvature conditions

BFGS is one of the canonical quasi-Newton updates (Nocedal and Wright 2006).

For the standard positive-definite Hessian-approximation form, the condition

$$y_k^\top s_k > 0$$

is load-bearing for preserving positive definiteness when the previous approximation is positive definite.

This matters because the sign of the approximate curvature influences whether the resulting search direction is descending.

The update formula is not magic.

Its guarantees have hypotheses.

29.19 Secant replay

For our positive-definite quadratic

$$H = \text{diag}(1, 4),$$

take

$$s = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Then the exact gradient change is

$$y = Hs = \begin{pmatrix} 1 \\ 8 \end{pmatrix}.$$

Any quasi-Newton matrix satisfying

$$Bs = y$$

matches the true curvature along that one displacement.

That single equation still leaves many degrees of freedom in B .

A secant condition is partial curvature information.

29.20 Proximal structure is not “more curvature”

Second-order optimization is often discussed alongside proximal methods because both go beyond a bare gradient step.

But proximal structure addresses a different problem.

Suppose

$$F(x) = f(x) + h(x),$$

where f is smooth and h may be nonsmooth.

The proximal operator is

$$\text{prox}_{\alpha h}(v) = \arg \min_x \left[h(x) + \frac{1}{2\alpha} \|x - v\|_2^2 \right].$$

It is an implicit local optimization primitive (Parikh and Boyd 2014).

It is not a Hessian approximation.

29.21 Soft thresholding witness

Take

$$h(x) = \lambda|x|$$

with

$$\alpha\lambda = 1.$$

Then

$$\text{prox}_{\alpha h}(v) = \text{sign}(v) \max(|v| - 1, 0).$$

Thus

$$v = 3 \mapsto 2,$$

while

$$v = 1/2 \mapsto 0.$$

The zero output comes from solving the nonsmooth proximal subproblem.

It is not ordinary gradient clipping.

29.22 Proximal gradient

For composite objectives

$$F(x) = f(x) + h(x),$$

a proximal-gradient step takes the form

$$x_{k+1} = \text{prox}_{\alpha h}(x_k - \alpha \nabla f(x_k)).$$

The step explicitly separates:

- smooth first-order information from f ;
- implicit nonsmooth structure from h .

This is a different axis from exact second derivatives.

29.23 What second-order methods actually buy

Second-order structure can provide:

- anisotropic rescaling;
- local curvature-aware directions;
- faster local convergence in favorable regimes;
- curvature diagnostics;

- principled handling of local quadratic models.

But the price can include:

- expensive curvature access;
- difficult indefinite systems;
- noisy estimates;
- storage or iterative-solve costs;
- damping/globalization logic;
- metric or model misspecification.

There is no free “use curvature” switch.

29.24 Local models and global optimization

A second-order method is still local.

The Hessian tells us how the objective bends near the current point.

A trust region limits how far we believe that local model.

A line search tests a step along a direction.

A quasi-Newton matrix infers curvature from recent changes.

A natural gradient changes the metric.

A proximal operator solves a declared local implicit subproblem.

These mechanisms solve different problems.

They should not be merged into one vague notion of sophistication.

29.25 Failure modes

29.25.1 Hessian equals Fisher

Not in general.

They arise from different constructions.

29.25.2 Natural gradient equals Newton

Not in general.

Natural gradient uses a metric; Newton uses the objective Hessian.

29.25.3 Invertible Hessian means safe Newton step

False.

The indefinite witness gives an ascent direction.

29.25.4 Trust region equals clipping

False.

A trust region solves a constrained model subproblem and reasons about model validity.

29.25.5 Quasi-Newton equals exact curvature

False.

The approximation satisfies limited secant information.

29.25.6 HVP means Hessian is cheap

False.

It avoids explicit formation but repeated products and solves still cost computation.

29.25.7 Proximal method is second-order

Not by definition.

It is an implicit optimization construction, especially useful for nonsmooth/composite structure.

29.25.8 Positive local curvature means global convexity

False.

Local Hessian information is local.

29.25.9 Exact local model means global optimum

False.

Newton solves the local quadratic stationary condition.

Global behavior requires additional assumptions and globalization logic.

29.26 Downstream handoff

Matrix-Aware Optimization may inherit:

- Hessian and curvature notation;
- positive/indefinite curvature distinctions;
- matrix-free operator action;
- natural-gradient metric language;
- quasi-Newton exact-versus-approximate discipline;

- trust-region and damping boundaries.

It may not assume that these classical objects already justify:

- Shampoo’s specific preconditioner;
- polar-factor updates;
- orthogonalized matrix steps;
- Muon-like spectral shaping;
- square-to-rectangular generalization.

Those are downstream obligations.

29.27 References used in this chapter

- (Nocedal and Wright 2006) — Newton, quasi-Newton, trust-region, and large-scale smooth optimization.
- (Amari 1998) — natural gradient and information geometry.
- (Pearlmutter 1994) — exact Hessian-vector products.
- (Parikh and Boyd 2014) — proximal operators and algorithms.

Exact provenance and claim boundaries are locked in:

‘sources/source-locks/ATLAS-CH-SECOND-001.yaml’

Chapter 30

Matrix-Aware Optimization

Epistemic status: audited Linear Algebra and Curvature/Second-Order prerequisites + primary Shampoo, polar-decomposition, norm-steepest-descent, and Muon design sources + Atlas derivation + exact rectangular witness.

Specification: manuscript/specifications/ATLAS-CH-MATRIXOPT-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-MATRIXOPT-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-MATRIXOPT-001.md

Source lock: sources/source-locks/ATLAS-CH-MATRIXOPT-001.yaml

30.1 A matrix parameter is not merely a bag of scalars

A linear layer may have a weight

$$W \in \mathbb{R}^{m \times n},$$

with gradient

$$G = \nabla_W f(W).$$

An optimizer can legitimately treat every entry of G as one scalar coordinate.

But the same matrix also has row directions, column directions, singular vectors, singular values, rank, and aspect ratio.

Matrix-aware optimization asks whether that structure should shape the update.

The claim is not that matrix-aware methods are automatically better.

The claim is that they define a different local geometry.

30.2 Three ways to change a matrix gradient

An elementwise rule may rescale entries separately.

A row/column-aware rule may transform

$$G \mapsto AGB.$$

An orthogonalizing rule may preserve singular vectors while transforming singular values.

These operations can produce different updates from the same G .

They should not share one name merely because each is called “preconditioning” informally.

30.3 Left and right multiplication

For compatible A, G, B ,

$$\text{vec}(AGB) = (B^\top \otimes A) \text{vec}(G).$$

So a left-right matrix transform is a structured operator on the vectorized gradient.

It can couple coordinates across rows and columns.

That is richer than independent scalar scaling.

It is still not automatically a full Hessian or Fisher inverse. A structured preconditioner and exact curvature are distinct unless a separate result identifies them.

30.4 Shampoo: accumulated matrix geometry

Shampoo was introduced as a preconditioned stochastic tensor optimization method (Gupta et al. 2018).

For a matrix gradient sequence, its structure can be seen through accumulated left and right statistics:

$$L_t = \sum_{s \leq t} G_s G_s^\top,$$

$$R_t = \sum_{s \leq t} G_s^\top G_s.$$

A representative damped matrix update is

$$P_t = (L_t + \varepsilon I)^{-1/4} G_t (R_t + \varepsilon I)^{-1/4}.$$

This remembers row-space and column-space gradient correlations and applies matrix inverse roots rather than only coordinatewise division.

But it deliberately uses structured factors. It is not, by definition, an arbitrary dense full-curvature inverse over every scalar parameter.

The convergence and empirical statements in the original paper remain scoped to that paper's assumptions and experiments.

30.5 Rectangular matrices change the algebra

Take a tall matrix

$$G \in \mathbb{R}^{m \times n}, \quad m > n.$$

Even if G has full column rank,

$$\text{rank}(G) = n,$$

the left Gram matrix obeys

$$\text{rank}(GG^\top) = n < m.$$

Thus $GG^\top$ is singular.

Nothing is defective. The gradient's column space simply occupies only an n -dimensional subspace of $\mathbb{R}^m$.

This is why square inverse formulas cannot be transferred mechanically to rectangular blocks.

Damping, accumulated history, pseudoinverse conventions, or support-restricted inverse powers change the actual method.

30.6 The polar factor

Let $r = \text{rank}(G)$, and take a compact SVD retaining only the r strictly positive singular values:

$$G = U_r \Sigma_r V_r^\top.$$

The support-restricted polar factor is

$$Q = U_r V_r^\top.$$

For $G = 0$, set $Q = 0$. For full-column-rank tall G , this agrees with the usual $UV^\top$.

For a tall full-column-rank matrix,

$$Q^\top Q = I_n.$$

But generally

$$QQ^\top \neq I_m.$$

In the full-column-rank case, the columns are orthonormal; the rows cannot all be orthonormal because there are more rows than columns. If G has rank $r < n$, instead $Q^\top Q = V_r V_r^\top$, a rank- r projector.

For a wide full-row-rank matrix, the roles reverse.

This is semi-orthogonality.

Higham's classical treatment supplies the numerical-analysis setting for polar decomposition and its computation (Higham 1986).

30.7 Polar orthogonalization is singular-value flattening

Using the compact rank- r SVD, the polar factor is

$$Q = U_r I_r V_r^\top.$$

Every nonzero singular value is replaced by 1.

The nonzero singular subspaces remain; zero singular directions remain zero. For example, $G = \text{diag}(1, 0)$ has compact polar factor $Q = \text{diag}(1, 0)$, not I_2 .

That is exact singular-value flattening on the nonzero support.

It is a strong and specific spectral transformation.

It does not by itself prove improved conditioning, training stability, convergence, or generalization for a learning problem.

30.8 “Steepest” depends on the norm

Locally,

$$f(W + \Delta) \approx f(W) + \langle G, \Delta \rangle_F.$$

For $G \neq 0$ and $\rho > 0$, if equally large moves are defined by

$$\|\Delta\|_F \leq \rho,$$

then a steepest direction is

$$\Delta_F^* = -\rho \frac{G}{\|G\|_F}.$$

If equally large moves are instead defined by

$$\|\Delta\|_2 \leq \rho,$$

then one steepest direction is

$$\Delta_2^* = -\rho U_r V_r^\top.$$

If $G = 0$, the linearized objective is constant on both norm balls; every feasible displacement minimizes it, and choosing $\Delta = 0$ is valid. In particular, do not divide by $\|G\|_F = 0$.

The polar factor therefore appears naturally as a spectral-norm steepest direction for nonzero G . Bernstein and Newhouse develop this norm-dependent optimizer viewpoint (Bernstein and Newhouse 2024).

The two directions do not contradict one another. They minimize the same linear functional over different feasible sets.

30.9 Why their local decreases are not optimizer scores

For the Frobenius ball,

$$\min_{\|\Delta\|_F \leq \rho} \langle G, \Delta \rangle_F = -\rho \|G\|_F.$$

For the spectral ball,

$$\min_{\|\Delta\|_2 \leq \rho} \langle G, \Delta \rangle_F = -\rho \|G\|_*.$$

A larger magnitude in the second expression does not prove a better optimizer. The constraint defining an equally large step has changed.

The norm is part of the optimization problem.

30.10 A one-step inverse-root identity

On the nonzero singular support,

$$(GG^\top)^{-1/4} G (G^\top G)^{-1/4} = UV^\top.$$

This explains an algebraic meeting point between a two-sided inverse-root transform and the polar factor.

But two warnings are essential.

First, a rectangular Gram matrix on the larger side is singular, so a full-space inverse needs damping or another declared convention.

Second, Shampoo is stateful. Its L_t and R_t accumulate earlier gradients.

The identity does not say that stateful Shampoo is “take the polar factor of the current gradient.”

30.11 Muon adds state and numerical approximation

Muon is a concrete optimizer design for matrix-shaped hidden-layer parameters (Jordan et al. 2024).

At a high level, it combines momentum with a finite Newton-Schulz-like matrix iteration intended to produce an approximately orthogonalized update.

The momentum state means the transformed matrix is not simply the raw current gradient.

And a finite polynomial iteration is not the exact SVD or polar decomposition.

A finite iteration implements a finite singular-value map.

The exact numerical realization therefore matters.

30.12 Exact versus approximate matrix functions

Exact expressions such as

$$(G^\top G)^{-1/2}$$

define mathematical targets.

Large-scale optimizers may realize related transforms through eigendecomposition, iterative matrix roots, Newton-Schulz iterations, low-precision polynomials, blocking, distribution, or delayed refresh.

The realized update can differ from the exact target.

An optimizer is therefore not specified by the ideal matrix function alone. Its state, damping, numerical method, precision, and refresh schedule are also part of the algorithm.

30.13 The exact 3×2 witness

Take

$$G = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Then

$$G^\top G = \begin{pmatrix} 5 & 0 \\ 0 & 1 \end{pmatrix},$$

so the singular values are

$$\sqrt{5}, 1.$$

The left Gram matrix is

$$GG^\top = \begin{pmatrix} 4 & 0 & 2 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \end{pmatrix}.$$

It has rank 2, not 3.

The right one-step statistic is invertible while the left one is singular.

The aspect ratio is already visible in the algebra.

30.14 Polarizing the witness

The exact polar factor is

$$Q = \begin{pmatrix} 2/\sqrt{5} & 0 \\ 0 & 1 \\ 1/\sqrt{5} & 0 \end{pmatrix}.$$

Then

$$Q^\top Q = I_2.$$

But

$$QQ^\top = \begin{pmatrix} 4/5 & 0 & 2/5 \\ 0 & 1 & 0 \\ 2/5 & 0 & 1/5 \end{pmatrix},$$

which is a rank-two projector rather than I_3 .

The witness is deliberately rectangular so “orthogonalized” cannot silently mean “square orthogonal.”

30.15 A diagonal control

Apply

$$D = \text{diag}(1/2, 1, 1/2).$$

Then

$$DG = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1/2 & 0 \end{pmatrix}.$$

Its singular values are

$$\sqrt{5}/2, 1.$$

The original singular-value ratio was $\sqrt{5}$. The diagonal control changes it to $\sqrt{5}/2$. The polar factor makes it 1.

Both operations reshape the spectrum, but they do so differently.

This control is generic. It is not presented as the exact rule used by Adam or another named elementwise optimizer.

30.16 Flattening is not the whole spectral story

The polar factor gives the simplest possible nonzero singular spectrum:

$$1, 1, \dots, 1.$$

But an optimizer might instead want to suppress only large modes, temper near-zero modes, preserve relative scale over a band, or impose another target spectrum.

Those questions belong to ATLAS-CH-SPECTRALSHAPE-001.

This chapter supplies the boundary:

singular-value flattening $\neq$ arbitrary intentional spectral shaping.

30.17 Matrix-aware does not mean manifold-constrained

An optimizer can choose a semi-orthogonal update Q_t while leaving the parameter W_t unconstrained:

$$W_{t+1} = W_t - \eta Q_t.$$

Even if

$$Q_t^\top Q_t = I,$$

there is no general implication that

$$W_{t+1}^\top W_{t+1} = I.$$

Manifold optimization instead declares a parameter constraint and uses tangent/retraction machinery to maintain it.

Orthogonalized updates and orthogonality-constrained parameters are different objects.

30.18 Matrix-aware does not mean full second order

A matrix preconditioner can encode more structure than an elementwise rule without becoming an exact Newton method.

A full Hessian for a matrix block acts on the vectorized parameter space and may couple every scalar coordinate.

Left-right factors impose a structured form.

That structure may be computationally attractive. It is still only the structure actually specified.

The Atlas therefore keeps separate:

- matrix-aware preconditioning;
- second-order curvature;
- hard manifold constraints.

30.19 Optimizer state is part of the trajectory

An instantaneous map

$$G_t \mapsto T(G_t)$$

does not describe a stateful optimizer.

A stateful optimizer has a law such as

$$S_t = \Phi(S_{t-1}, G_t),$$

$$\Delta_t = \Psi(S_t, G_t).$$

Shampoo's accumulated factors and Muon's momentum make this explicit.

Two methods can share a similar instantaneous transform and still produce different trajectories because their states differ.

30.20 What this chapter establishes

The chapter establishes a controlled vocabulary and exact finite algebra for matrix-aware updates.

It supports these claims:

- rectangular geometry matters;
- left and right Gram matrices encode different sides of a matrix block;
- a rectangular polar factor is semi-orthogonal;
- polar orthogonalization flattens nonzero singular values;
- norm choice changes the steepest direction;
- a support-restricted one-step inverse-root transform meets the polar factor algebraically;
- state, damping, and numerical realization prevent that identity from collapsing named optimizers into one another.

It does not establish that one update geometry is universally best.

30.21 Failure boundaries

Keep these separations intact:

elementwise scaling $\neq$ matrix preconditioning,

matrix preconditioning $\neq$ full Hessian inversion,

one-step inverse-root identity $\neq$ stateful Shampoo equivalence,

exact polar factor $\neq$ finite Newton-Schulz realization,

orthogonalized update $\neq$ orthogonality-constrained parameter,

singular-value flattening $\neq$ all spectral shaping,

and

instantaneous transform $\neq$ stateful optimizer dynamics.

30.22 References used in this chapter

- Shampoo: Gupta, Koren, and Singer (Gupta et al. 2018).
- Polar decomposition and numerical computation: Higham (Higham 1986).
- Norm-dependent steepest-descent interpretation: Bernstein and Newhouse (Bernstein and Newhouse 2024).
- Muon design and finite Newton-Schulz update construction: Jordan et al. (Jordan et al. 2024).
- Audited Atlas prerequisites: ATLAS-CH-LINALG-001 and ATLAS-CH-SECOND-001.

Exact bibliographic authority, prerequisite blob identities, and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-MATRIXOPT-001.yaml`

Chapter 31

Optimization on Manifolds

Epistemic status: audited Geometry + audited First-Order Optimization prerequisites + standard matrix-manifold optimization sources + Atlas derivation + exact finite witness.

Specification: [manuscript/specifications/ATLAS-CH-MANOPT-001.md](#)

Derivation packet: [mathematics/derivations/ATLAS-CH-MANOPT-001-DERIVATIONS.md](#)

Computational witness: [mathematics/computational-witnesses/ATLAS-CW-MANOPT-001.md](#)

Source lock: [sources/source-locks/ATLAS-CH-MANOPT-001.yaml](#)

31.1 Optimization changes when legal states form a curved space

Ordinary first-order optimization usually assumes parameters may move freely in Euclidean coordinates.

Many useful objects do not have that freedom.

Examples include:

- unit vectors;
- orthonormal frames;
- rotation matrices;
- fixed-rank factorizations;
- positive-definite matrices;
- subspaces.

If the parameter must satisfy

$$\|x\| = 1$$

or

$$X^\top X = I,$$

then an arbitrary Euclidean update generally leaves the admissible set.

Optimization on manifolds changes the question from:

Which ambient direction decreases the objective?

to:

Which locally legal direction decreases the objective, and how do we turn it into a legal finite move?

That separation is the chapter's organizing idea.

31.2 Tangent directions are local legal velocities

For a smooth manifold M , the tangent space

$$T_x M$$

is a linear approximation to the legal directions at x .

A tangent vector is not usually another point on the manifold.

It is a velocity.

This matters because the update

$$x + \xi$$

can leave M even when

$$\xi \in T_x M.$$

A constrained optimizer therefore needs two distinct operations:

1. choose a tangent direction;
2. map the finite displacement back to the manifold.

The first is local differential geometry.

The second is finite constrained motion.

31.3 The gradient depends on the metric

The First-Order Optimization chapter treated the Euclidean gradient as the steepest direction under the Euclidean metric.

A manifold may have a different metric.

So the Riemannian gradient is defined by

$$Df(x)[\xi] = \langle \text{grad } f(x), \xi \rangle_x$$

for every tangent vector

$$\xi \in T_x M.$$

The metric appears inside the definition.

Therefore:

$$\text{gradient} = \text{objective derivative} + \text{metric}.$$

Changing the metric can change the steepest direction even when the objective function is unchanged.

31.4 Embedded manifolds and tangent projection

For an embedded manifold with the induced Euclidean metric, the Riemannian gradient is obtained by projecting the ambient Euclidean gradient onto the tangent space (Absil et al. 2008).

Write

$$\Pi_x$$

for the orthogonal tangent projection.

Then

$$\text{grad } f(x) = \Pi_x(\nabla F(x)),$$

where F is an ambient extension of f .

This formula is powerful and easy to misuse.

It is valid for the induced metric setting being described.

It should not be promoted into a universal formula for every Riemannian metric.

31.5 Sphere geometry

For the unit sphere

$$S^{n-1} = \{x \in \mathbb{R}^n : \|x\| = 1\},$$

the tangent space is

$$T_x S^{n-1} = \{\xi : x^\top \xi = 0\}.$$

The induced-metric tangent projection is

$$\Pi_x(g) = g - (x^\top g)x.$$

The normal component lies along x .

The tangent component is what first-order constrained motion may use.

31.6 Tangent projection is not the update

Suppose

$$g = \nabla F(x).$$

A projected direction

$$-\Pi_x(g)$$

is tangent.

But the raw finite step

$$x - \eta \Pi_x(g)$$

need not satisfy the original nonlinear constraint.

The tangent plane touches the manifold.

It is not the manifold.

This gives a basic boundary:

legal infinitesimal direction $\neq$ legal finite endpoint.

31.7 Retractions

A retraction is a practical map from tangent data back to the manifold (Absil et al. 2008).

Locally it satisfies:

$$R_x(0_x) = x$$

and

$$DR_x(0_x) = \text{id}_{T_x M}.$$

So the retraction agrees with tangent motion to first order.

It does not have to trace an exact geodesic.

This matters computationally because exact exponential maps may be expensive while useful retractions can be cheaper.

31.8 Sphere normalization as a retraction

For tangent ξ on the unit sphere, define:

$$R_x(\xi) = \frac{x + \xi}{\|x + \xi\|}.$$

Because

$$x^\top \xi = 0,$$

we have

$$\|x + \xi\|^2 = 1 + \|\xi\|^2.$$

Therefore

$$\|R_x(\xi)\| = 1.$$

The normalized endpoint is exactly feasible.

The move is not generally the sphere exponential map.

31.9 Exact sphere witness

Take

$$x = (1, 0)^\top, \quad a = (1, 2)^\top, \quad f(z) = a^\top z.$$

The ambient gradient is

$$a = (1, 2)^\top.$$

Projecting onto the tangent space at x gives

$$\text{grad } f(x) = (0, 2)^\top.$$

Choose

$$\eta = \frac{1}{2}.$$

The tangent descent vector is

$$\xi = (0, -1)^\top.$$

The unconstrained Euclidean update is

$$(1/2, -1)^\top,$$

whose squared norm is

$$5/4.$$

It leaves the sphere.

The raw tangent displacement

$$x + \xi = (1, -1)^\top$$

also leaves the sphere because its squared norm is 2.

After normalization:

$$R_x(\xi) = \frac{1}{\sqrt{2}}(1, -1)^\top,$$

and the norm is exactly 1.

The witness separates three things:

- ambient Euclidean descent;
- tangent descent;
- feasible finite motion.

31.10 Retraction versus exponential map

For the same sphere witness,

$$\|\xi\| = 1.$$

The exact exponential endpoint is

$$\text{Exp}_x(\xi) = (\cos 1, -\sin 1)^\top.$$

The normalized retraction endpoint is

$$(1/\sqrt{2}, -1/\sqrt{2})^\top.$$

Both are legal.

They differ.

Therefore:

retraction $\neq$ exponential map.

A retraction earns its role through local first-order agreement and feasibility, not by being an exact geodesic solver.

31.11 The Stiefel manifold

For integers $1 \leq p \leq n$, the nonempty Stiefel manifold is

$$\text{St}(n, p) = \{X \in \mathbb{R}^{n \times p} : X^\top X = I_p\}.$$

If $p > n$, an $n \times p$ real matrix cannot have p orthonormal columns; the tangent and retraction constructions below assume $1 \leq p \leq n$.

Its columns are orthonormal.

This constraint appears throughout numerical linear algebra and optimization with orthogonality constraints (Edelman et al. 1998).

A tangent matrix Z satisfies

$$X^\top Z + Z^\top X = 0.$$

This is obtained by differentiating

$$X^\top X = I.$$

The condition is linear in Z , as a tangent-space constraint should be.

31.12 Stiefel tangent projection

Under the induced Euclidean metric, an ambient matrix G can be projected by

$$\Pi_X(G) = G - X \operatorname{sym}(X^\top G),$$

where

$$\operatorname{sym}(A) = \frac{1}{2}(A + A^\top).$$

The projected matrix satisfies

$$X^\top \Pi_X(G) + \Pi_X(G)^\top X = 0.$$

Again, this produces a legal local direction.

It does not by itself produce a legal finite orthonormal matrix after ambient addition.

31.13 Polar retraction on the Stiefel manifold

For tangent Ξ , one retraction is

$$R_X(\Xi) = (X + \Xi)(I + \Xi^\top \Xi)^{-1/2}.$$

The tangent condition implies

$$(X + \Xi)^\top (X + \Xi) = I + \Xi^\top \Xi.$$

Therefore:

$$R_X(\Xi)^\top R_X(\Xi) = I.$$

The retraction restores the orthogonality constraint exactly.

31.14 Exact 2×2 witness

Let

$$X = I_2$$

and

$$\Xi = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

Because Ξ is skew-symmetric,

$$X^\top \Xi + \Xi^\top X = 0.$$

So it is tangent.

The raw step is

$$X + \Xi = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}.$$

Its Gram matrix is

$$2I.$$

Thus the raw tangent displacement is not orthogonal.

The polar retraction is

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix},$$

whose Gram matrix is exactly I .

Constraint restoration is exact.

31.15 Orthogonal does not mean geodesically exact

For the same skew generator,

$$e^{\Xi} = \begin{pmatrix} \cos 1 & -\sin 1 \\ \sin 1 & \cos 1 \end{pmatrix}.$$

This is rotation by 1 radian.

The polar-retracted endpoint is rotation by

$$\pi/4.$$

Both are orthogonal.

They are different.

This is the matrix-manifold version of the same distinction:

constraint preservation $\nRightarrow$ exponential/geodesic motion.

31.16 A generic first-order manifold step

A simple constrained first-order method can be written:

$$\xi_k = -\eta_k \operatorname{grad} f(x_k),$$

followed by

$$x_{k+1} = R_{x_k}(\xi_k).$$

This compact notation hides important design choices:

- metric;
- gradient estimator;
- tangent projection;
- step size;
- retraction;
- momentum;
- vector transport;
- stochastic sampling.

Optimization on a manifold is not one algorithm.

It is a family of algorithms respecting a declared geometric structure.

31.17 Momentum creates a tangent-space problem

Suppose an optimizer stores momentum

$$m_k \in T_{x_k} M.$$

After moving to

$$x_{k+1},$$

the new tangent space is

$$T_{x_{k+1}} M.$$

These two vector spaces are generally different subspaces of the ambient space.

Blindly reusing m_k as if nothing changed can ignore geometry.

A vector transport supplies a declared way to move tangent information between tangent spaces (Absil et al. 2008).

This is distinct from retraction:

- retraction moves a point;
- vector transport moves tangent information.

31.18 Intrinsic and extrinsic viewpoints

A manifold can be described intrinsically through its metric and curves.

It can also be embedded in a larger Euclidean space and manipulated through ambient coordinates.

Neither viewpoint is automatically superior.

The extrinsic viewpoint can make projections and constraints concrete.

The intrinsic viewpoint helps identify coordinate-independent structure.

The Atlas uses whichever representation makes the relevant claim precise.

31.19 Constraint preservation is not optimization quality

A method can preserve the constraint perfectly and still:

- increase the objective;
- take poor step sizes;
- converge slowly;
- stagnate;
- approach a bad stationary point.

Therefore:

$$\boxed{\text{constraint preservation} \neq \text{optimizer quality.}}$$

Geometry defines legal motion.

Optimization still requires a useful search rule.

31.20 Stationarity on a manifold

A first-order stationary point satisfies

$$\text{grad } f(x) = 0.$$

For an embedded manifold under induced metric, this means the ambient gradient has no tangent component.

The ambient gradient can still have a normal component.

Thus constrained stationarity differs from unconstrained Euclidean stationarity.

And first-order stationarity still does not imply global optimality.

31.21 Sphere versus Stiefel

The sphere constrains one vector:

$$x^\top x = 1.$$

The Stiefel manifold constrains multiple columns simultaneously:

$$X^\top X = I.$$

The sphere tangent condition is scalar:

$$x^\top \xi = 0.$$

The Stiefel tangent condition is matrix-valued:

$$X^\top Z + Z^\top X = 0.$$

The two geometries are related but not interchangeable.

31.22 Stiefel versus Grassmann

A Stiefel point is an orthonormal frame.

A Grassmann point represents a subspace independent of which orthonormal basis is chosen for it.

The Geometry chapter already established this distinction.

It matters for optimization.

If the objective depends only on the spanned subspace, then treating different bases as distinct states may introduce redundant directions.

This is where quotient geometry enters.

MANOPT uses the distinction but does not redevelop quotient optimization in full.

31.23 Why parameterization matters

One can sometimes avoid an explicit constrained optimizer by parameterizing the constraint indirectly.

Examples include:

- normalized vectors;
- matrix exponentials;
- factorizations;
- Householder products;
- Cayley-like transforms.

Such choices change:

- coordinates;
- conditioning;
- computational cost;
- reachable sets;
- singularities;
- optimization dynamics.

A constraint-preserving parameterization is not automatically equivalent to direct manifold optimization.

The effective geometry can differ.

31.24 Geometry can change steepest descent

In Euclidean space, steepest descent depends on the Euclidean inner product.

On a manifold, the Riemannian metric changes the local notion of unit direction.

Therefore changing geometry can change the gradient even before any adaptive optimizer state is introduced.

This is one reason later Atlas work on geometry-aware and matrix-aware optimization must keep separate:

- the objective;
- the metric;
- the optimizer;
- the parameterization.

31.25 Retraction choice can matter

Two valid retractions agree to first order at the tangent origin.

They can differ at finite step sizes.

So a large-step optimization trajectory can depend on the retraction even when the tangent gradient is unchanged.

This is not a defect.

It is part of the algorithm.

A convergence theorem must state what retraction properties it assumes.

An experiment must state which retraction it used.

31.26 Step size still matters

Geometry does not remove ordinary optimization concerns.

A constrained method still needs to choose:

- step size;
- stochastic batch;
- line search or schedule;
- momentum state;
- clipping or trust restrictions;
- stopping rule.

A legal step can still be too large.

A legal trajectory can still be poor.

Manifold structure constrains the search space.

It does not solve the search problem by itself.

31.27 Relation to normalized models

Unit-norm states and orthogonality constraints appear naturally in normalized neural architectures.

The mathematical substrate in this chapter is directly relevant to such systems.

But the chapter stops at generic manifold optimization.

It does not establish:

- a theorem for nGPT;
- a theorem for Muon;
- a theorem for rectangular spectral updates;
- a theorem for MODULUS;
- a theorem for any specific GCL optimizer.

Those require their own objectives, metrics, updates, and evidence.

31.28 Relation to variational optimization

The direct downstream consumer is:

Variational and Divergence-Derived Optimization — ATLAS-CH-VARIOPT-001.

VARIOPT may inherit:

- tangent gradients;
- constrained finite motion;
- retractions;
- metric dependence;
- vector-transport distinctions;
- sphere/Stiefel examples.

It must independently justify any claim that an update follows from:

- a divergence;
- a discrete Lagrangian;
- a symplectic principle;
- a duality relation;
- another variational construction.

Manifold optimization says how to respect a constrained geometry.

Variational optimization asks where the motion rule itself comes from.

31.29 Durable boundaries

The chapter leaves ten distinctions:

1. Euclidean gradient is not automatically the Riemannian gradient.
2. Tangent projection is not a complete optimizer.
3. Tangent direction is not a finite feasible endpoint.
4. Retraction is not the exponential map.
5. First-order agreement is not geodesic exactness.
6. Sphere and Stiefel constraints are different.
7. Retraction and vector transport have different roles.
8. Constraint preservation is not convergence.
9. Manifold stationarity is not global optimality.
10. Generic manifold optimization is not a specific GCL optimizer theorem.

These boundaries are the reusable substrate.

31.30 References used in this chapter

- (Absil et al. 2008)
- (Edelman et al. 1998)

Exact source identities and authority scopes are pinned in:

`sources/source-locks/ATLAS-CH-MANOPT-001.yaml`

Chapter 32

Spectral Shaping

Epistemic status: audited Matrix-Aware Optimization and Non-normality substrates + Atlas-owned exact finite spectral-shaping witnesses.

Specification: manuscript/specifications/ATLAS-CH-SPECTRALSHAPE-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-SPECTRALSHAPE-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-SPECTRALSHAPE-001.md

Source lock: sources/source-locks/ATLAS-CH-SPECTRALSHAPE-001.yaml

Spectral manipulation is often discussed as though it were one operation.

It is not.

A matrix update can be:

- normalized;
- clipped;
- conditioned;
- flattened;
- or deliberately reshaped toward a non-flat target.

Those operations can have different mathematical meanings even when they all alter singular values.

The governing rule is:

declare the target spectral map before interpreting the result.

32.1 The handoff from Matrix-Aware Optimization

MATRIXOPT already gives the Atlas:

- SVD language;
- polar factors;
- rectangular semi-orthogonality;
- matrix preconditioning;
- exact-versus-approximate singular-value transformations;
- the key boundary that polar flattening is not all spectral shaping.

SPECTRALSHAPE starts where that chapter deliberately stopped.

32.2 The handoff from Non-normality

NONNORMAL adds a second constraint.

Even if eigenvalues look benign, finite-horizon behavior can be very different when the operator is non-normal.

Therefore an instantaneous update spectrum cannot be promoted directly into a dynamical guarantee.

32.3 A general shaping map

Let

$$G = U\Sigma V^\top,$$

with singular values

$$\sigma_1, \dots, \sigma_r.$$

A singular-value shaping rule has the form

$$T_f(G) = Uf(\Sigma)V^\top,$$

where f acts on the singular values. When G is rank-deficient, the map must specify its behavior on zero singular directions. Taking $f(0) = 0$ leaves those directions zero and yields an SVD-independent singular-value transform. If $f(0) \neq 0$, a rule also needs a declared pairing of the left and right nullspaces; the zero-singular-vector choices in an SVD of G are not unique.

As a counterexample, let $G = \text{diag}(1, 0)$ and $f(1) = f(0) = 1$. Two valid SVDs are $U = V = I_2$ and $U = \text{diag}(1, -1), V = I_2$, both with $\Sigma = \text{diag}(1, 0)$. They give different transformed matrices, I_2 and $\text{diag}(1, -1)$. Hence a rank-increasing rule requires extra structure. The full-rank witnesses below are unaffected.

Different well-defined f give different update geometries.

32.4 Exact witness matrix

Use

$$G = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}.$$

Its singular values are:

$$(4, 1).$$

Hence:

$$\boxed{\kappa_2(G) = 4.}$$

This one matrix is enough to distinguish four common operations.

32.5 Scalar normalization changes scale

Normalize by the operator norm:

$$G_{\text{scale}} = \frac{G}{\|G\|_2} = \begin{pmatrix} 1 & 0 \\ 0 & 1/4 \end{pmatrix}.$$

The condition number remains:

$$\boxed{\kappa_2(G_{\text{scale}}) = 4.}$$

So:

$$\boxed{\text{normalization} \not\Rightarrow \text{better conditioning.}}$$

A global scalar multiplies every singular value by the same amount.

Their ratio is unchanged.

32.6 Clipping can change conditioning

Now cap singular values at 2:

$$f_{\text{clip}}(\sigma) = \min(\sigma, 2).$$

Then:

$$G_{\text{clip}} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}.$$

Thus:

$$\boxed{\kappa_2(G_{\text{clip}}) = 2.}$$

The spectrum is still non-flat.

Only the large mode was clipped.

32.7 Polar flattening is stronger

For a full-rank square positive diagonal matrix, the polar factor replaces every nonzero singular value by one.

Thus:

$$G_{\text{polar}} = I_2.$$

Hence:

$$\boxed{\kappa_2(G_{\text{polar}}) = 1.}$$

This is complete nonzero singular-value flattening.

32.8 A non-flat target is another operation

Suppose we deliberately choose target singular values:

$$(3, 2).$$

Then:

$$G_\tau = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}.$$

Its condition number is:

$$\kappa_2(G_\tau) = \frac{3}{2}.$$

This is neither normalization, clipping, nor flattening.

It is explicit spectral design.

32.9 The four maps are not synonyms

For the same input G , the shaped spectra are:

$$(1, 1/4),$$

$$(2, 1),$$

$$(1, 1),$$

and

$$(3, 2).$$

Therefore:

$$\text{scale control} \neq \text{clipping} \neq \text{flattening} \neq \text{target shaping}.$$

32.10 Better condition number is not automatically better optimization

On this witness, the condition numbers are:

$$4, \quad 4, \quad 2, \quad 1, \quad 3/2.$$

That is an exact matrix statement.

It is not a universal optimizer ranking.

A nonlinear optimizer also depends on:

- objective geometry;
- step size;
- optimizer state;
- stochasticity;
- parameterization;
- update interaction across steps.

32.11 Flat is not automatically best

The polar spectrum:

$$(1, 1)$$

is maximally flat.

But there may be reasons to preserve anisotropy.

A target:

$$(3, 2)$$

retains directional preference.

The Atlas therefore treats the target spectrum as a design object rather than an article of faith.

32.12 Conditioning is only one objective

Possible spectral goals include:

- cap large modes;
- raise small modes;
- reduce condition number;
- preserve relative scale in a band;
- remove scale entirely;
- impose a desired anisotropy.

These objectives need not agree.

32.13 Rank matters

If a singular value is zero, the standard full-space condition number is infinite.

Changing a zero singular value to a positive one changes rank.

That is no longer mere rescaling.

It is a structural intervention and must be declared as such.

32.14 Singular vectors matter too

A map

$$Uf(\Sigma)V^\top$$

changes singular values while retaining the singular-vector frames.

A more general transform can also rotate U or V .

Two matrices with the same singular values can therefore still behave differently relative to a particular interface.

Spectral shape is not the whole operator.

32.15 Non-normality supplies the dynamical warning

Consider:

$$A = \begin{pmatrix} 1/2 & 2 \\ 0 & 1/2 \end{pmatrix}, \quad N = \frac{1}{2}I.$$

Both have eigenvalues:

$$\{1/2, 1/2\}.$$

If eigenvalues were the whole story, their short-time behavior would look alike.

It does not.

32.16 Same eigenvalues, different one-step response

Probe both matrices with:

$$e_2 = (0, 1)^\top.$$

Then:

$$Ae_2 = (2, 1/2)^\top,$$

so:

$$\|Ae_2\|_2^2 = \frac{17}{4}.$$

But:

$$Ne_2 = (0, 1/2)^\top,$$

so:

$$\|Ne_2\|_2^2 = \frac{1}{4}.$$

Equal eigenvalues do not imply equal interface response.

32.17 Pseudospectra separate them further

For the upper-triangular non-normal family inherited from NONNORMAL, the exact closed 2-norm ε -pseudospectral radius around the repeated eigenvalue is:

$$r_A = \sqrt{\varepsilon(\varepsilon + K)}.$$

Set:

$$K = 2, \quad \varepsilon = \frac{1}{4}.$$

Then:

$$r_A = \frac{3}{4}.$$

For the normal scalar matrix:

$$r_N = \frac{1}{4}.$$

Matched eigenvalues coexist with a threefold difference in this pseudospectral radius.

32.18 Why that matters for optimizer updates

Suppose an optimizer shapes the singular spectrum of an instantaneous update matrix:

$$\Delta_t.$$

That says something about Δ_t .

It does not automatically say anything about the spectrum or pseudospectrum of the optimizer's joint state transition.

Those are different operators.

32.19 Stateful optimizers enlarge the object

A stateful optimizer has:

$$S_{t+1} = \Phi(S_t, G_t),$$

and perhaps:

$$W_{t+1} = W_t + \Psi(S_{t+1}, G_t).$$

The relevant local dynamical operator is then a Jacobian on the coupled state:

$$(W, S).$$

The singular values of one instantaneous parameter update are not that Jacobian.

32.20 Spectral shaping is not optimizer-state dynamics

Therefore:

instantaneous update spectrum $\nRightarrow$ optimizer-state dynamics.

This boundary is mandatory.

It prevents a useful matrix diagnostic from becoming an unsupported dynamical theorem.

32.21 Better conditioning is not convergence

Even if a spectral map reduces:

$$\kappa_2(G),$$

the nonlinear training trajectory may improve, worsen, or remain unchanged.

A convergence theorem needs the actual update law and assumptions.

So:

better update condition number $\nRightarrow$ faster nonlinear convergence.

32.22 Task quality is another layer

A shaped update can have:

- a lower condition number;
- a flatter spectrum;
- smaller operator norm;

and still harm the downstream objective.

Conversely, task quality can improve without a flatter update spectrum.

Thus:

spectral-shape quality $\nRightarrow$ task quality.

32.23 A practical reporting protocol

For a spectral-shaping method, report separately:

1. the matrix/operator being shaped;
2. the SVD or spectral object being used;
3. the shaping function f ;
4. the target spectrum or clipping rule;
5. condition number before/after;
6. rank changes, if any;
7. singular-vector changes, if any;
8. optimizer-state semantics;
9. dynamical diagnostics;
10. downstream task metrics.

This prevents one spectral statistic from impersonating the entire optimization process.

32.24 What a flat spectrum actually certifies

For a full-rank square matrix, flattening the nonzero singular values to one certifies:

$$\kappa_2 = 1.$$

It does not certify:

- minimum training loss;
- good generalization;
- stable coupled dynamics;
- absence of non-normality elsewhere;
- correct task behavior.

The certificate is exactly the matrix property it proves.

32.25 What a non-flat target actually certifies

A target such as:

$$(3, 2)$$

certifies only that the chosen singular-value profile has been imposed.

Whether this geometry is useful is an empirical or theoretical question about the downstream problem.

32.26 Atlas non-implications

SPECTRALSHAPE rejects:

normalization $\nRightarrow$ better conditioning,

conditioning $\nRightarrow$ flattening,

flat spectrum $\nRightarrow$ universal optimizer quality,

better update conditioning $\nRightarrow$ faster nonlinear convergence,

instantaneous update spectrum $\nRightarrow$ optimizer-state dynamics,

eigenvalue shape $\nRightarrow$ non-normal transient control,

and:

spectral-shape quality $\nRightarrow$ task quality.

32.27 Atlas connections

Matrix-Aware Optimization.

Supplies SVD/polar update geometry and the exact flattening boundary.

Normality, Pseudospectra, and Transient Growth.

Supplies the warning that spectral eigenvalue information may be insufficient for finite-horizon dynamics.

Optimizer-State Dynamics.

Owns the coupled model–optimizer state and its Jacobian.

Spectral and Operator Diagnostics.

Can diagnose operator behavior but does not convert a spectral signature into mechanistic evidence.

32.28 Closing view

Spectral shaping is best understood as an explicit design map:

$$\boxed{G \mapsto Uf(\Sigma)V^\top.}$$

The important question is not:

did the spectrum change?

It is:

which spectrum, by what map, toward what target, for what objective,
and with what separate evidence about dynamics and task behavior?

The exact witness gives four different answers from one input matrix.

That is the point.

32.29 References used in this chapter

No new external academic authority is added.

Matrix-update and SVD/polar authority is inherited through audited ATLAS-CH-MATRIXOPT-001.

Non-normal transient-growth and pseudospectral authority is inherited through audited ATLAS-CH-NONNORMAL-001.

Exact prerequisite identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-SPECTRALSHAPE-001.yaml`

Chapter 33

Optimizer-State Dynamics

Epistemic status: mathematical exposition built from canonical optimizer formulations, control-theoretic optimization analysis, the preceding Non-normality chapter, and Atlas-owned augmented-state derivations.

Primary figure: ‘ATLAS-FIG-OPTDYN-001‘

Derivation packet: ‘mathematics/derivations/ATLAS-CH-OPTDYN-001-DERIVATIONS.md‘

33.1 The optimizer is part of the state

A common picture of training writes

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t).$$

That picture is exact for plain gradient descent.

For momentum, Adam, and many modern optimizers it is incomplete.

The update at time $t + 1$ depends not only on the current parameters but also on internal state accumulated from previous gradients. Momentum carries velocity-like state. Adam carries first- and second-moment estimates (Kingma and Ba 2015). Other methods maintain matrices, preconditioners, or moving statistics.

The mathematical object is therefore not only θ_t .

It is an augmented state,

$$z_t = (\theta_t, s_t),$$

with update

$$z_{t+1} = F_t(z_t, \xi_t),$$

where s_t is optimizer state and ξ_t represents data or stochastic forcing.

This chapter asks:

what dynamical behavior becomes visible only when the optimizer and model are analyzed as one coupled system?

The useful allegory is a **vehicle with a flywheel**.

Position corresponds to model parameters. The flywheel corresponds to stored optimizer state. A steering command changes position, but the stored motion changes how the same command behaves one step later.

The limit of the allegory is strict. Optimizer state is not physical momentum, adaptive methods have coordinatewise and state-dependent statistics, and no conservation law is implied.

The point is only that memory in the update rule changes the state space.

33.2 Heavy-ball momentum as a state-space map

Polyak's multistep acceleration is a canonical source for momentum-like iteration (Polyak 1964), and momentum later became central to deep-network optimization (Sutskever et al. 2013).

Consider the scalar quadratic

$$L(\theta) = \frac{h}{2}\theta^2, \quad h > 0.$$

Then

$$g(\theta) = h\theta.$$

Use the recurrence

$$v_{t+1} = \beta v_t + h\theta_t,$$

followed by

$$\theta_{t+1} = \theta_t - \eta v_{t+1}.$$

Substitution gives

$$\theta_{t+1} = (1 - \eta h)\theta_t - \eta\beta v_t.$$

Define

$$z_t = \begin{pmatrix} \theta_t \\ v_t \end{pmatrix}.$$

Then

$$z_{t+1} = Jz_t,$$

with

$$J = \begin{pmatrix} 1 - \eta h & -\eta\beta \\ h & \beta \end{pmatrix}.$$

Nothing has been approximated. The optimizer-model pair is literally a two-dimensional linear dynamical system for this quadratic example.

33.3 The characteristic equation belongs to the coupled system

The trace is

$$\text{tr } J = 1 - \eta h + \beta,$$

and

$$\det J = \beta.$$

Therefore

$$p(\lambda) = \lambda^2 - (1 - \eta h + \beta)\lambda + \beta.$$

The stability boundary depends jointly on

$$\eta, \quad \beta, \quad h.$$

This already changes the conceptual story.

Curvature h matters, but curvature alone does not determine the update. Learning rate matters, but learning rate alone is not the dynamical system. Momentum matters, but it is not a scalar correction applied outside the dynamics.

The boundary is a property of the coupled recurrence.

33.4 A stable example with transient amplification

Choose

$$h = 1, \quad \eta = \frac{1}{10}, \quad \beta = \frac{9}{10}.$$

Then

$$J = \begin{pmatrix} 0.9 & -0.09 \\ 1 & 0.9 \end{pmatrix}.$$

Its eigenvalues are

$$\lambda_{\pm} = 0.9 \pm 0.3i.$$

Thus

$$|\lambda_{\pm}| = \sqrt{0.9} \approx 0.9486832981 < 1.$$

The fixed point is asymptotically stable.

If eigenvalues were the whole story, one might expect every perturbation simply to decay.

They are not the whole story.

The matrix is non-normal.

Indeed,

$$J^{\top} J = \begin{pmatrix} 1.81 & 0.819 \\ 0.819 & 0.8181 \end{pmatrix},$$

while

$$J J^{\top} = \begin{pmatrix} 0.8181 & 0.819 \\ 0.819 & 1.81 \end{pmatrix}.$$

Therefore

$$J^\top J \neq JJ^\top.$$

The preceding Non-normality chapter prepared exactly this situation: asymptotic spectral stability can coexist with finite-horizon amplification.

33.5 Four stable steps can amplify by more than $2.6 \times$

For the same matrix,

$$J^4 = \begin{pmatrix} \frac{567}{2500} & -\frac{729}{3125} \\ \frac{324}{125} & \frac{567}{2500} \end{pmatrix}.$$

The largest singular value is

$$\|J^4\|_2 \approx 2.610090585959495.$$

So there exists a unit perturbation δz_0 such that

$$\|J^4 \delta z_0\|_2 \approx 2.61.$$

The same system eventually decays because its eigenvalues are inside the unit disk.

This is the mechanism to remember:

asymptotically stable $\nrightarrow$ monotone finite-horizon decay.

The claim is exact for this linear example.

It is not yet a claim about a neural training run.

33.6 The Wolfram plate

The primary figure puts three views of the same toy system beside one another.

The left panel shows trajectories in the augmented (θ, v) state plane.

The middle panel shows both eigenvalues strictly inside the unit circle.

The right panel shows

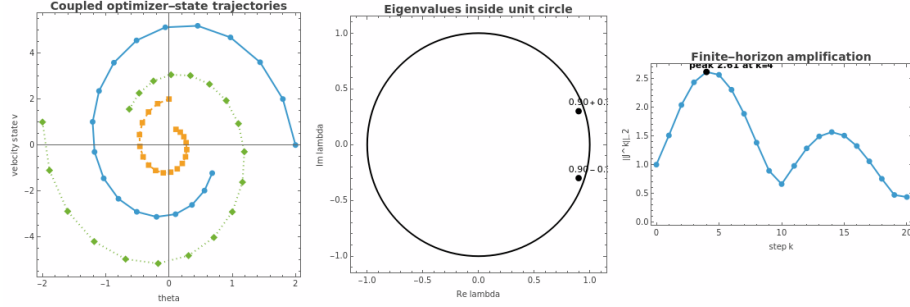


Figure 33.1: Phase trajectories, eigenvalues relative to the unit circle, and finite-horizon spectral norm gain for the exact momentum-state matrix.

$$\|J^n\|_2$$

for $n = 0, \dots, 20$, with a peak of approximately

$$2.61009$$

at

$$n = 4.$$

The three panels are not three different stories. They are three descriptions of one matrix.

The figure is therefore a compact warning against using one spectral summary as if it exhausted the dynamics.

33.7 Why the augmented state matters

If one looks only at θ_t , the optimizer's memory appears indirectly.

In augmented state, it is explicit.

This matters because the local Jacobian of the full update contains couplings between parameter and optimizer coordinates.

For a general optimizer state s_t ,

$$z_t = \begin{pmatrix} \theta_t \\ s_t \end{pmatrix},$$

and a local Jacobian has block structure

$$J_t = \begin{pmatrix} \partial_\theta F_\theta & \partial_s F_\theta \\ \partial_\theta F_s & \partial_s F_s \end{pmatrix}.$$

The off-diagonal blocks are not decoration. They encode how optimizer memory changes parameter motion and how parameter-space gradients update optimizer memory.

A parameter-only Hessian does not contain all of this information.

33.8 Curvature is not the same as optimizer dynamics

On the quadratic example, curvature appears through h .

But the state Jacobian is

$$J(\eta, \beta, h) = \begin{pmatrix} 1 - \eta h & -\eta \beta \\ h & \beta \end{pmatrix}.$$

The same h with different η or β produces a different dynamical system.

Likewise, a large Hessian eigenvalue can matter without being a complete explanation of instability.

The disciplined statement is:

the loss landscape contributes to the update dynamics, but the update dynamics belong to the combined model-optimizer recurrence.

This is the level at which learning-rate boundaries should be analyzed.

33.9 From an autonomous toy system to time-varying training

The quadratic example has one fixed matrix J .

Training in a neural network is generally nonautonomous.

Write

$$z_{t+1} = F_t(z_t, \xi_t).$$

Linearizing along a trajectory gives

$$\delta z_{t+1} \approx J_t \delta z_t, \quad J_t = \frac{\partial F_t}{\partial z}(z_t, \xi_t).$$

After k steps,

$$\delta z_{t+k} \approx J_{t+k-1} \cdots J_t \delta z_t.$$

A single-step eigenvalue calculation cannot in general summarize this product.

The relevant object is the finite-horizon propagator.

This is one reason non-normal and transient-growth thinking is useful: it asks what perturbations can do over a finite horizon, not only what one frozen matrix does asymptotically.

33.10 Adam makes the state richer

Adam maintains first- and second-moment estimates (Kingma and Ba 2015).

Schematically,

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) g_t,$$

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) g_t^2,$$

followed by a parameter update using bias-corrected moment estimates.

The exact augmented state is therefore larger:

$$z_t = (\theta_t, m_t, v_t, \dots).$$

The present chapter does not derive a universal Adam stability theory. That would obscure the mechanism we are trying to expose.

Momentum is used as the exact microscope because its state is small enough that every coupling can be seen.

Adam is included to show that the augmented-state viewpoint becomes more, not less, relevant as optimizers accumulate richer memory.

33.11 Optimization as a dynamical and control system

There is established precedent for analyzing iterative optimization as a feedback/dynamical system. Lessard, Recht, and Packard use integral quadratic constraints from robust control to analyze methods including gradient descent, heavy-ball momentum, and accelerated schemes (Lessard et al. 2016).

The Atlas does not reproduce that full framework here.

The connection is conceptual and mathematical:

- an optimizer is an iterative dynamical system;
- the objective/gradient map enters the loop;
- stability and gain are properties of the interconnection;
- finite-horizon behavior can matter even when asymptotic convergence conditions hold.

The Atlas adds a particular emphasis: in learned systems, optimizer state should be treated as part of the object being diagnosed.

33.12 What a local Jacobian can and cannot tell us

A local augmented-state Jacobian can reveal:

- instantaneous coupling;
- singular amplification directions;
- local spectral structure;
- departure from normality;
- sensitivity to hyperparameters;
- finite-horizon products along a recorded trajectory.

It cannot, by itself, establish:

- the global nonlinear behavior of training;
- the cause of an observed loss spike;
- that one local mode persists over many steps;
- that a toy quadratic captures a frontier model;
- that non-normality is the dominant mechanism in every unstable run.

The distinction between mechanism and prevalence is essential.

A toy example can prove that a mechanism exists.

It cannot prove how often that mechanism matters in practice.

33.13 The flywheel allegory, and where it fails

The flywheel picture is useful because it blocks one mistake: pretending that only current position matters.

The correspondence is:

- position $\leftrightarrow$ model parameters;
- stored rotational state $\leftrightarrow$ optimizer memory;
- control gain $\leftrightarrow$ learning rate;
- coupled overshoot $\leftrightarrow$ transient amplification.

But the optimizer is not a physical rigid body.

Adam's state has no literal angular momentum. Stochastic gradients are not deterministic forces. Coordinatewise normalization has no simple mechanical analogue.

The allegory has done its job once the reader accepts that optimizer memory enlarges the state space.

The mathematics should then take over.

33.14 A handoff to Coupling-Phase Spectroscopy

Suppose a training phase transition is genuinely a property of the augmented state.

Then a parameter-only diagnostic may miss part of the mechanism.

This motivates a later instrumentation question:

can we probe the Jacobian of the coupled model-optimizer state
strongly enough to detect changes in dynamical regime?

The Atlas names the downstream GCL programme **Coupling-Phase Spectroscopy (CPS)**.

At present, this chapter makes no project-specific empirical claim because no exact public source lock for CPS results is bound here. The source-lock manifest records that boundary explicitly.

The chapter therefore contributes the mathematical object and the diagnostic question, not an unverified result.

33.15 Five mistakes to avoid

33.15.1 Mistake 1: “The Hessian is the optimizer dynamics.”

No. Curvature enters the recurrence, but optimizer memory changes the state and the update Jacobian.

33.15.2 Mistake 2: “Eigenvalues inside the unit disk mean every perturbation shrinks every step.”

No. The worked non-normal example has spectral radius below one and peak finite-horizon gain above 2.6.

33.15.3 Mistake 3: “A transient spike proves divergence.”

No. Transient amplification and asymptotic divergence are different.

33.15.4 Mistake 4: “A local linearization predicts the whole training run.”

No. It is local, and realistic training is time-varying and nonlinear.

33.15.5 Mistake 5: “Optimizer state is implementation detail.”

No. If the next update depends on it, it is part of the dynamical state.

33.16 Atlas connections

Non-normality.

This chapter turns transient-growth mathematics into an optimizer-state mechanism.

Coupling-Phase Spectroscopy.

The augmented-state Jacobian becomes a candidate probe for phase changes.

Router dynamics.

Routing systems with memory can be analyzed through similar coupled-state ideas.

Spectral diagnostics.

Eigenvalues, singular values, pseudospectra, and finite-horizon gains become complementary diagnostic objects.

Variational optimization.

Later chapters will ask whether better update rules can be derived from geometry and discrete variational structure rather than tuned only through heuristics.

The recurring Atlas shift is:

optimizer as update rule $\longrightarrow$ optimizer-model pair as dynamical system.

33.17 Closing view

A modern optimizer remembers.

Once it remembers, its memory becomes part of the state.

Once the optimizer state is part of the state, the natural object is no longer only a loss surface or a gradient. It is a coupled recurrence with its own Jacobian, stability region, transient behavior, and finite-horizon propagator.

The simple quadratic example is small enough to solve exactly and rich enough to expose the point.

Stable eigenvalues do not forbid transient amplification.

The flywheel is only an allegory.

The augmented state is the object.

33.18 References used in this chapter

- (Polyak 1964)
- (Sutskever et al. 2013)
- (Kingma and Ba 2015)
- (Lessard et al. 2016)

See ‘sources/source-locks/ATLAS-CH-OPTDYN-001.yaml’ for exact source identities and claim scope.

Chapter 34

Coupling-Phase Spectroscopy

Epistemic status: audited Optimizer-State Dynamics + current public GSD-001 transition/artifact evidence + Atlas-owned exact spectroscopy and prediction protocol.

Specification: manuscript/specifications/ATLAS-CH-CPS-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-CPS-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-CPS-001.md

Source lock: sources/source-locks/ATLAS-CH-CPS-001.yaml

A training run can change behavior before we know what changed inside the optimization dynamics.

A benchmark state can flip.

A representation can drift.

An optimizer can carry momentum, moments, scheduler state, or update geometry that is invisible in a parameter-only snapshot.

Coupling-Phase Spectroscopy asks a narrow question:

can local structure in the augmented model-optimizer state diagnose
or predict a behavioral transition before the transition is observed?

The chapter begins with a strict warning.

diagnosis $\neq$ prediction $\neq$ causal mechanism.

All three can be scientifically useful.

They require different evidence.

34.1 The object is the augmented training state

Optimizer-State Dynamics established that optimizer memory belongs in the dynamical state whenever the next update depends on it.

Write:

$$z_{t+1} = F_t(z_t, \xi_t).$$

The local Jacobian is:

$$J_t = \frac{\partial F_t}{\partial z}(z_t, \xi_t).$$

A short-horizon perturbation is governed locally by:

$$J_{t+k-1} \cdots J_t.$$

CPS treats these local maps and products as instrumentation surfaces.

It does not claim that one frozen Jacobian globally describes training.

34.2 What is spectroscopy here?

The word spectroscopy is used by analogy with measuring several signatures of a state rather than collapsing everything into one scalar.

A CPS diagnostic vector can contain:

$$\mathcal{S}_t = \left(\text{tr } J_t, \det J_t, \rho(J_t), \hat{\sigma}_{\max}(J_t), \mathcal{N}(J_t), \hat{G}_{t,k}, \mathcal{A}_t \right).$$

Possible coordinates include:

- trace;
- determinant;
- spectral radius;
- dominant singular gain;
- non-normality proxy;
- finite-horizon gain;
- directional alignment.

No coordinate is promoted to a universal health score.

34.3 The behavioral transition is defined elsewhere

CPS does not get to create its own target after looking at optimizer diagnostics.

The behavioral transition must be frozen independently.

For a checkpoint t , define:

$$Y_t = \mathbf{1}\{\text{declared behavioral transition occurs in } (t, t + \Delta]\}.$$

CPS receives Y_t as a target variable.

It does not redefine Y_t .

34.4 Current GCL state changed since the first Atlas source lock

The original Optimizer-State Dynamics source lock was written on October 2, 2026.

At that time it recorded no separate public GCL source for a CPS result.

That statement is no longer a complete description of current public project state.

A fresh organization search on October 6 finds a public GSD programme with:

- GSD-WP04 — CPS transition-local dynamics;
- one confirmed behavioral transition;
- an explicit optimizer-artifact availability receipt.

The Atlas therefore replaces the old current-state gap with exact newer project evidence.

34.5 What GSD has actually established

The bound GSD-WP03R receipt establishes one robust OLMo2-1B transition for:

truthy_answer/surprising_truth.

The frozen GSD protocol reports:

- step 2000: GENERALIZING;
- step 3000: PATTERN_MATCHING;
- step 4000: PATTERN_MATCHING;
- transition localized between steps 2000 and 3000;
- stable controls passed;

- independent replay preserved the transition;
- prompt variants preserved the confirmed direction.

This is a behavioral result.

The receipt explicitly does not establish:

- mechanism;
- optimizer cause;
- persistence state;
- capacity-allocation explanation;
- architecture-level conclusion.

34.6 What GSD-WP04 asks

The public GSD research programme defines GSD-WP04 as:

CPS transition-local dynamics.

Its question is:

does optimizer-state structure predict or explain confirmed transitions?

Its declared outputs include:

- optimizer-state extraction;
- augmented-state probes;
- transition-local traces;
- heldout predictive testing.

Its scientific exit grammar is:

- PREDICTIVE_SIGNAL;
- DESCRIPTIVE_ONLY;
- NO_SIGNAL.

That is a useful experimental contract.

34.7 Why there is no WP04 signal result yet

The exact public transition-local revisions inspected are:

- stage1-step2000-tokens5B;
- stage1-step3000-tokens7B;
- stage1-step4000-tokens9B.

The public revisions expose:

- configuration;
- tokenizer;

- model weight shards;
- weight index.

They do not expose:

- optimizer state;
- trainer state;
- scheduler state;
- gradient artifacts;
- update-direction artifacts.

Therefore the exact GSD-WP04 receipt says:

BLOCKED_EXTERNAL_ARTIFACT_ABSENT.

34.8 Blocked is not negative

This distinction matters.

A negative CPS result would require the selected CPS probe to be computed and then fail.

That did not happen.

The needed artifacts are missing.

Therefore:

missing artifacts $\nRightarrow$ no signal.

The scientific question remains open.

34.9 The Atlas can still formalize the protocol

The empirical lane is blocked.

The mathematical lane is not.

The Atlas can still define:

- what a CPS probe is;
- how discovery differs from heldout testing;
- what counts as prediction;
- what counts only as description;
- what evidence would be needed for causal promotion.

That is the purpose of the exact finite witness.

34.10 Return to the exact momentum Jacobian

For scalar quadratic curvature h , fix:

$$\eta = \frac{1}{10}, \quad \beta = \frac{9}{10}.$$

The augmented momentum Jacobian is:

$$J(h) = \begin{pmatrix} 1 - \frac{h}{10} & -\frac{9}{100} \\ h & \frac{9}{10} \end{pmatrix}.$$

This is inherited directly from the audited OPTDYN derivation.

34.11 One invariant is constant

For every h :

$$\det J(h) = \frac{9}{10}.$$

The trace is:

$$\mathrm{tr} J(h) = \frac{19}{10} - \frac{h}{10}.$$

Define a toy matrix-derived coordinate:

$$\kappa(J) = 1 + \det J - \mathrm{tr} J.$$

Then:

$$\boxed{\kappa(J(h)) = \frac{h}{10}.$$

This coordinate is chosen for the witness.

It is not proposed as the CPS statistic for real training.

34.12 Four synthetic windows

Create two discovery windows:

$$d_0 : \quad h = 2, \quad \kappa = \frac{1}{5}, \quad Y = 0,$$

$$d_1 : \quad h = 8, \quad \kappa = \frac{4}{5}, \quad Y = 1.$$

Create two heldout windows:

$$h_0 : \quad h = 3, \quad \kappa = \frac{3}{10}, \quad Y = 0,$$

$$h_1 : \quad h = 7, \quad \kappa = \frac{7}{10}, \quad Y = 1.$$

The transition labels are synthetic.

They exist only to demonstrate protocol discipline.

34.13 Discovery comes before heldout evaluation

Using the discovery windows only:

$$\frac{1}{5} < \frac{1}{2} < \frac{4}{5}.$$

Freeze threshold:

$$\tau = \frac{1}{2}.$$

Prediction rule:

$$\hat{Y} = \mathbf{1}\{\kappa > \tau\}.$$

Now the rule is frozen.

The heldout labels cannot be used to change:

- statistic;
- threshold;
- state coordinates;
- horizon;
- normalization.

34.14 Exact heldout replay

For h_0 :

$$\frac{3}{10} < \frac{1}{2},$$

so:

$$\hat{Y} = 0.$$

For h_1 :

$$\frac{7}{10} > \frac{1}{2},$$

so:

$$\hat{Y} = 1.$$

Therefore:

$$\boxed{(\hat{Y}_{h_0}, \hat{Y}_{h_1}) = (0, 1)}.$$

The heldout synthetic labels are recovered exactly.

This proves only that the protocol mechanics are coherent.

34.15 Why one spectral scalar is not enough

All four toy Jacobians have:

$$\det J = \frac{9}{10}.$$

Their characteristic discriminants are all negative.

Therefore each has a complex-conjugate eigenvalue pair.

For each pair:

$$|\lambda|^2 = \det J = \frac{9}{10}.$$

So:

$$\rho(J) = \sqrt{\frac{9}{10}}$$

for all four toy windows.

The toy labels differ.

The spectral radius does not.

34.16 Same spectral radius, different local maps

For $h = 2$:

$$J(2) = \begin{pmatrix} \frac{4}{5} & -\frac{9}{100} \\ 2 & \frac{9}{10} \end{pmatrix}.$$

For $h = 8$:

$$J(8) = \begin{pmatrix} \frac{1}{5} & -\frac{9}{100} \\ 8 & \frac{9}{10} \end{pmatrix}.$$

The matrices are not the same.

Their trace differs.

Their coupling coordinate differs.

Their action on perturbation directions differs.

Thus:

same spectral radius $\nRightarrow$ same local augmented-state dynamics.

34.17 This is why CPS is multi-coordinate

The point is not that κ is better than spectral radius.

The point is that different matrix summaries answer different questions.

A real CPS probe may need:

- singular gain;
- finite-horizon gain;
- non-normality;
- update-direction alignment;
- layer-local block structure;
- time-varying products.

The exact toy merely proves that one scalar can be blind.

34.18 Local Jacobians remain local

Suppose a Jacobian changes sharply near a transition.

That supports a local dynamical observation.

It does not prove:

global phase change.

It does not prove:

optimizer state caused behavioral change.

It does not prove the same signature persists across checkpoints, tasks, or scales.

The local/global firewall from OPTDYN remains active.

34.19 Retrospective description is the weakest positive CPS result

Imagine scanning all layers, horizons, and statistics after the transition is known.

Suppose one statistic moves dramatically at the transition.

That is interesting.

It is also selected after seeing the answer.

The correct label is:

DESCRIPTIVE_ONLY.

It can motivate a new preregistered test.

It is not prediction.

34.20 Prediction requires temporal and selection discipline

A predictive test needs a probe frozen before heldout evaluation.

For checkpoint t , compute:

$$X_t = \text{Probe}_{\Pi_{\text{CPS}}}(\text{artifacts available at or before } t).$$

Then predict:

$$Y_t$$

for the future window:

$$(t, t + \Delta].$$

Information from $t + \Delta$ cannot enter X_t .

Otherwise the future has leaked into the predictor.

34.21 Discovery must be separated from confirmation

The discovery set can choose:

- layer;
- state block;
- horizon;
- statistic;
- threshold;
- normalization;
- estimator.

Then those choices must freeze.

The heldout set tests the frozen object.

This is the difference between:

I found a curve that moves near transitions.

and:

I can identify transition risk before the heldout transition occurs.

34.22 A real CPS heldout protocol

For future artifact-complete transition windows:

1. freeze the behavioral transition definition;
2. partition discovery and heldout windows;
3. choose CPS features on discovery only;
4. freeze checkpoint lead Δ ;

5. compute pre-transition features only;
6. compare against non-transition windows;
7. include shuffled-label controls;
8. include simple training-time baselines;
9. test the frozen predictor on heldout transitions;
10. retain negative results.

This structure is already compatible with GSD-WP04’s intended exit grammar.

34.23 What counts as PREDICTIVE_SIGNAL?

Use:

PREDICTIVE_SIGNAL

only when:

- exact required artifacts exist;
- the probe is frozen before heldout testing;
- the heldout metric clears its preregistered criterion;
- relevant baselines/controls do not explain the result.

This is predictive association.

It is not causality.

34.24 What counts as DESCRIPTIVE_ONLY?

Use:

DESCRIPTIVE_ONLY

when:

- a transition-local signature exists;
- but it was selected retrospectively;
- or it fails heldout prediction;
- or heldout confirmation was never preregistered.

Descriptive signals are legitimate research outputs.

They should not be relabeled as forecasts.

34.25 What counts as NO_SIGNAL?

Use:

NO_SIGNAL

when:

- the required artifacts exist;
- the declared probe family is executable;
- the preregistered test is actually run;
- the signal fails the declared criteria.

A negative result can save large downstream compute.

34.26 What counts as an artifact block?

Use:

BLOCKED_EXTERNAL_ARTIFACT_ABSENT

when the selected test cannot be executed because required exact artifacts are unavailable.

That is the current public GSD-WP04 state.

It is an evidence-availability disposition.

It is not a scientific result about CPS.

34.27 Prediction is not cause

Suppose a CPS feature predicts a transition.

The feature can still be:

- a correlate;
- a downstream symptom;
- a common-cause indicator;
- a proxy for training time;
- a proxy for curvature or data composition.

Therefore:

heldout prediction $\nrightarrow$ causal optimizer mechanism.

34.28 Causal promotion needs intervention

A stronger test would manipulate the optimizer-side state or update while matching relevant controls.

Examples can include:

- controlled optimizer-state reset;
- controlled moment rescaling;
- update-direction substitution;
- matched optimizer-rule intervention;
- state patching where semantics are well-defined.

The intervention must be specified before the outcome is interpreted.

Only then can the evidence move toward a causal claim.

34.29 The causal hierarchy

Use five levels.

34.29.1 C0 — behavioral transition

A transition exists independently of CPS.

34.29.2 C1 — local diagnostic movement

A CPS feature changes near the transition.

34.29.3 C2 — heldout prediction

A frozen pre-transition feature predicts heldout transition windows.

34.29.4 C3 — intervention sensitivity

Changing optimizer-side state/update changes transition behavior under matched controls.

34.29.5 C4 — mechanism-specific support

Intervention, reconstruction, and alternative-hypothesis attacks support a declared mechanism class.

Each level needs new evidence.

34.30 Current GSD is at C0 for CPS

The public source establishes one confirmed behavioral transition.

For CPS specifically, the optimizer artifacts are unavailable.

Therefore current public GSD supports:

C0

as a target transition.

It does not support:

C1, C2, C3, C4

from optimizer-state evidence.

This is not a criticism.

It is the exact evidence boundary.

34.31 One transition cannot validate a general predictor

The current GSD source has one confirmed transition.

A general predictor needs:

- discovery examples;
- heldout examples.

One event can support:

- artifact-readiness work;
- a descriptive case study if artifacts exist.

It cannot establish broad heldout predictive value by itself.

34.32 Small model does not silently become large model

The confirmed public transition is OLMo2 1B.

Even a successful CPS result there would not automatically transfer to:

- OLMo3 7B;
- OLMo3 32B;
- other model families;

- other optimizers;
- post-training settings.

Scale transfer requires its own heldout evidence.

34.33 Artifact completeness is part of the experiment

A CPS packet should identify:

- model revision;
- optimizer state;
- trainer/global step;
- scheduler;
- gradient or update direction if required;
- evaluator/tokenizer;
- code;
- hardware;
- transition label provenance.

If one required artifact is absent, the test is not silently weakened.

The block should be explicit.

34.34 Why exact identity matters

A transition-local probe can be invalidated by stale identity.

Optimizer state from one checkpoint cannot be paired casually with model parameters from another.

Scheduler state from the wrong run is not equivalent.

A CPS result must bind to:

(model, optimizer, step, run, probe code).

This is scientific provenance, not bookkeeping.

34.35 Time-varying products matter

A real training trajectory is not generally autonomous.

The local perturbation product is:

$$J_{t+k-1} \cdots J_t.$$

Therefore finite-horizon CPS may need to estimate a product or its action.

One-step eigenvalues can miss transient amplification.

OPTDYN already proved that lesson in a fixed toy.

CPS turns it into an instrumentation question.

34.36 JVP/VJP sketches can make the probe tractable

A full frontier-scale Jacobian is too large to materialize naively.

The GSD programme therefore contemplates randomized JVP/VJP estimates.

These can target:

- dominant singular directions;
- local gains;
- selected subspaces;
- layer/block couplings.

The result remains an estimate of a declared local object.

It does not become a global training theorem because the computation is scalable.

34.37 Training time is an important baseline

A CPS statistic may drift smoothly with training step.

Behavioral transitions also occur at particular steps.

A naive association can therefore be explained by time itself.

A heldout protocol should compare against:

- checkpoint index;
- tokens seen;
- learning rate;
- update norm;
- other simple progress variables.

A sophisticated probe must outperform the relevant simple baseline to justify its complexity.

34.38 Parameter-only baselines matter

If a parameter-only diagnostic predicts the transition just as well as the optimizer-state probe, then the added optimizer state may not provide incremental value for that prediction task.

This does not make optimizer state irrelevant to training.

It narrows the claim.

CPS should report incremental predictive value when that is the question.

34.39 Stable capability controls matter too

A transition-specific predictor can accidentally be a generic degradation detector.

GSD already protects against this with stable conventional-capability controls.

CPS should inherit that discipline.

A broad collapse is different from a selective generalization-state transition.

34.40 Negative CPS evidence is useful

Suppose optimizer artifacts become available and a carefully preregistered probe returns:

NO_SIGNAL.

That is informative.

It can redirect effort toward:

- representation/operator diagnostics;
- accessibility/control mechanisms;
- data-window attribution;
- other causal routes.

The Atlas should preserve such negative results.

34.41 What the current chapter establishes

Mathematically, the chapter establishes:

$$\det J(h) = \frac{9}{10},$$

$$\kappa(J(h)) = \frac{h}{10},$$

the exact synthetic discovery/heldout threshold result, and identical spectral radius across all four toy windows.

Scientifically, it establishes the current source boundary:

- one public confirmed GSD behavioral transition exists;
- the public CPS work package exists;
- the exact transition-local public revisions lack optimizer artifacts;
- therefore no public GSD CPS signal disposition is currently authorized.

34.42 Non-implications

The chapter rejects:

behavioral transition $\nRightarrow$ optimizer-state transition,

local Jacobian change $\nRightarrow$ global training phase change,

retrospective alignment $\nRightarrow$ prediction,

heldout prediction $\nRightarrow$ causal mechanism,

same spectral radius $\nRightarrow$ same augmented-state dynamics,

missing optimizer artifacts $\nRightarrow$ no CPS signal,

and:

one 1B transition $\nRightarrow$ general predictor across scale.

34.43 Atlas connections

Optimizer-State Dynamics.

CPS operationalizes the augmented-state Jacobian as an empirical diagnostic object.

Generalization-State Dynamics.

GSD supplies behavioral transition targets and the heldout predictive question.

Spectral Diagnostics.

CPS supplies a transition-local application of spectral, singular-gain, and non-normality probes.

Experiments as Arguments.

Discovery/freeze/holdout separation is part of the claim structure, not merely experimental hygiene.

Governed research systems.

Artifact identity and explicit block states prevent missing evidence from being silently converted into negative findings.

34.44 Closing view

Coupling-Phase Spectroscopy is not a claim that optimizer state explains every transition.

It is a way to ask the question correctly.

First establish the behavioral transition.

Then obtain the exact optimizer-state artifacts.

Then define the local augmented-state probe.

Then separate exploration from heldout prediction.

Then separate prediction from causality.

The current public GSD programme has reached the first step and specified the second.

The required optimizer artifacts are not publicly present at the exact transition checkpoints.

So the present result is neither success nor failure of CPS.

It is a precise boundary:

transition confirmed; optimizer-state spectroscopy currently artifact-blocked.

That is the right place to continue from when the missing evidence becomes available.

34.45 References used in this chapter

External mathematical authority is inherited through audited ATLAS-CH-OPTDYN-001.

Current GCL project evidence is bound to exact public grandchallenge/GSD files at commit:

ebd4681e1815a3d7f0285cc4ce2bc090b38fae8c

See `sources/source-locks/ATLAS-CH-CPS-001.yaml` for exact identities and claim boundaries.

Chapter 35

Variational and Divergence-Derived Optimization

Epistemic status: audited Numerics and Manifold Optimization prerequisites + primary Bregman/variational sources + exact GCL MODULUS programme evidence + Atlas finite derivations.

Specification: manuscript/specifications/ATLAS-CH-VARIOPT-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-VARIOPT-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-VARIOPT-001.md

Source lock: sources/source-locks/ATLAS-CH-VARIOPT-001.yaml

Optimization rules can be written down directly.

They can also be derived from a declared geometry, a divergence, an action, or a constraint.

Those derivations are valuable because they expose which structure the update is intended to respect.

But the derivation does not grant every desirable property at once.

The governing rule is:

derivation structure $\neq$ performance guarantee.

This chapter develops that distinction through Bregman geometry, variational mechanics, symplectic updates, and an exact GCL MODULUS programme

bridge.

35.1 Two inherited firewalls

NUMERICS-001 established that:

exact flow $\neq$ numerical update,

and that:

- stability is not accuracy;
- structure preservation is not exact energy conservation;
- order claims require assumptions;
- a computational analogy is not automatically a theorem.

MANOPT-001 established that:

- gradients depend on metric;
- tangent directions are local legal velocities;
- retractions return tangent data to the constraint set;
- retractions need not equal exponential maps;
- constraint preservation does not imply descent;
- stationarity does not imply global optimality.

VARIOPT inherits both.

35.2 Why divergences enter optimization

Euclidean distance is only one way to measure a local change.

A convex generator can define a directed discrepancy that adapts to the geometry of the domain.

Let:

$$\phi : \Omega \rightarrow \mathbb{R}$$

be differentiable and strictly convex.

The Bregman divergence is:

$$D_\phi(y, x) = \phi(y) - \phi(x) - \langle \nabla \phi(x), y - x \rangle.$$

Geometrically, it measures how far the graph of ϕ at y sits above the tangent hyperplane to ϕ at x .

35.3 Divergence is not distance

The ordering matters.

Take:

$$\phi(x) = e^x.$$

Then:

$$D_\phi(1, 0) = e - 2,$$

but:

$$D_\phi(0, 1) = 1.$$

Thus:

$$D_\phi(1, 0) \neq D_\phi(0, 1).$$

A Bregman divergence is therefore not generally a metric.

That remains true even though it can generate useful local geometry.

35.4 Local geometry from a Hessian

Assume ϕ has a locally Lipschitz Hessian near x , for example bounded third derivatives. Then for a small displacement δ :

$$D_\phi(x + \delta, x) = \frac{1}{2} \delta^\top \nabla^2 \phi(x) \delta + O(\|\delta\|^3).$$

The Hessian acts as a local quadratic form. Under merely continuous second differentiability, the same expansion holds with the weaker remainder $o(\|\delta\|^2)$; the displayed cubic-order remainder requires the stated stronger regularity.

For:

$$\phi(x) = e^x,$$

at $x = 0$:

$$D_\phi(\delta, 0) = e^\delta - 1 - \delta = \frac{1}{2} \delta^2 + \frac{1}{6} \delta^3 + \cdots.$$

The local quadratic geometry can be symmetric even when the full divergence is not.

35.5 Geometry can define an update

Suppose the objective supplies a local covector g_k .

Instead of taking a Euclidean step, choose the next state by minimizing:

$$\eta \langle g_k, x \rangle + D_\phi(x, x_k).$$

The first term asks for progress under the local linearized objective.

The second penalizes movement according to the chosen geometry.

At an interior optimum:

$$\nabla \phi(x_{k+1}) = \nabla \phi(x_k) - \eta g_k.$$

The step is simple in the dual coordinate $\nabla \phi$.

35.6 Euclidean descent is one special case

For:

$$\phi(x) = \frac{1}{2} \|x\|^2,$$

we have:

$$\nabla \phi(x) = x.$$

So:

$$x_{k+1} = x_k - \eta g_k.$$

Ordinary gradient descent therefore sits inside the broader divergence-derived construction.

The broader construction should not be reduced back to Euclidean distance by terminology.

35.7 An entropic coordinate system

For positive scalar x , choose:

$$\phi(x) = x \log x - x.$$

Then:

$$\nabla \phi(x) = \log x.$$

The update becomes:

$$\log x_{k+1} = \log x_k - \eta g_k.$$

Hence:

$$x_{k+1} = x_k e^{-\eta g_k}.$$

For:

$$x_k = 1, \quad \eta = 1, \quad g_k = \log 2,$$

we obtain exactly:

$$x_{k+1} = \frac{1}{2}.$$

A change that is additive in dual coordinates becomes multiplicative in primal coordinates.

35.8 Divergence-derived does not mean descending

Consider:

$$f(x) = \frac{1}{2}(x-2)^2.$$

At $x_0 = 0$:

$$g_0 = -2.$$

With the Euclidean Bregman generator and step size:

$$\eta = 3,$$

the update is:

$$x_1 = 6.$$

Yet:

$$f(x_0) = 2, \quad f(x_1) = 8.$$

The objective increased.

Thus:

derived from a convex divergence $\nRightarrow$ automatic descent.

Step-size, smoothness, convexity, and algorithm-specific conditions still matter.

35.9 From geometry to dynamics

A first-order update uses a local objective model plus a movement penalty.

A variational formulation can instead define a whole trajectory through an action.

For path $q(t)$:

$$\mathcal{A}[q] = \int L(q, \dot{q}, t) dt.$$

Stationarity gives the Euler-Lagrange equation:

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{q}} - \frac{\partial L}{\partial q} = 0.$$

The object being made stationary is the action.

It need not be the optimization objective itself.

35.10 Bregman Lagrangian

Wibisono, Wilson, and Jordan introduced a Bregman Lagrangian:

$$\mathcal{L}(X, V, t) = e^{\alpha_t + \gamma t} [D_h(X + e^{-\alpha_t} V, X) - e^{\beta t} f(X)] .$$

Under their declared ideal-scaling conditions and regularity assumptions, its Euler-Lagrange dynamics organize broad families of accelerated continuous-time optimization flows.

This is a powerful bridge:

$$\text{divergence geometry} + \text{variational dynamics} \rightarrow \text{optimization flow} .$$

But the source does not license every possible discretization of that flow.

35.11 Continuous flow still is not the algorithm

The audited Numerics boundary remains active.

Even when a continuous optimization flow has a proved rate:

$$\text{continuous theorem} \not\Rightarrow \text{arbitrary discrete theorem} .$$

A discrete method must be analyzed as its own map.

That map can have different:

- stability;
- error;
- invariants;
- energy behavior;
- descent behavior.

35.12 Discrete action

Variational mechanics offers a different route.

Instead of discretizing the differential equation directly, discretize the action.

Let:

$$L_d(q_k, q_{k+1}; h)$$

represent one discrete action contribution.

Then:

$$\mathcal{A}_d = \sum_k L_d(q_k, q_{k+1}; h).$$

Stationarity with respect to an interior point q_k gives:

$$D_2 L_d(q_{k-1}, q_k) + D_1 L_d(q_k, q_{k+1}) = 0.$$

This is the discrete Euler-Lagrange equation.

35.13 Why derive the discrete rule this way?

A discrete variational principle can preserve geometric structure inherited from the action formulation.

Marsden and West develop this framework systematically and show how regular discrete variational mechanics generates symplectic schemes.

The important word is **structure**.

The theorem is not:

$$\text{variational integrator} = \text{exact trajectory}.$$

Nor is it:

$$\text{symplectic integrator} = \text{monotonically improving objective}.$$

35.14 Exact oscillator construction

Take the harmonic-oscillator Lagrangian:

$$L(q, \dot{q}) = \frac{1}{2} \dot{q}^2 - \frac{1}{2} q^2.$$

Use the left-endpoint discrete Lagrangian:

$$L_d(q_k, q_{k+1}; h) = \frac{(q_{k+1} - q_k)^2}{2h} - \frac{h}{2} q_k^2.$$

This is now a finite algebraic object.

35.15 Discrete Euler-Lagrange equation

The two derivatives are:

$$D_2 L_d(q_{k-1}, q_k) = \frac{q_k - q_{k-1}}{h},$$

and:

$$D_1 L_d(q_k, q_{k+1}) = -\frac{q_{k+1} - q_k}{h} - h q_k.$$

Set their sum to zero:

$$\frac{q_k - q_{k-1}}{h} - \frac{q_{k+1} - q_k}{h} - h q_k = 0.$$

Rearrange:

$$q_{k+1} = (2 - h^2)q_k - q_{k-1}.$$

The update has been derived from a discrete stationarity condition rather than guessed from the ODE.

35.16 Momentum lift

Define:

$$p_k = \frac{q_k - q_{k-1}}{h}.$$

Then the recurrence becomes:

$$p_{k+1} = p_k - h q_k,$$

followed by:

$$q_{k+1} = q_k + h p_{k+1}.$$

This is one ordering of symplectic Euler.

35.17 Exact map

Write:

$$z_k = \begin{pmatrix} q_k \\ p_k \end{pmatrix}.$$

Then:

$$z_{k+1} = M_h z_k,$$

with:

$$M_h = \begin{pmatrix} 1 - h^2 & h \\ -h & 1 \end{pmatrix}.$$

Its determinant is exactly:

$$1.$$

More strongly:

$$M_h^\top J M_h = J$$

for the canonical symplectic matrix:

$$J = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}.$$

35.18 Symplectic is a precise property

Symplecticity means the map preserves the canonical symplectic form.

It is not a synonym for:

- stable;
- accurate;
- energy conserving;
- reversible;
- objective descending;
- globally optimal.

These properties require their own evidence.

35.19 Exact energy counterexample

Use:

$$h = \frac{1}{2}, \quad (q_0, p_0) = (0, 1).$$

The update gives:

$$p_1 = 1,$$

$$q_1 = \frac{1}{2}.$$

For:

$$H(q, p) = \frac{1}{2}(q^2 + p^2),$$

the initial energy is:

$$H_0 = \frac{1}{2}.$$

After one exactly symplectic step:

$$H_1 = \frac{5}{8}.$$

So energy increased by:

$$\frac{1}{8}.$$

The symplectic identity remains exact.

35.20 Exact objective counterexample

Define a toy objective:

$$F(q) = \frac{1}{2}q^2.$$

At the same states:

$$F(q_0) = 0,$$

$$F(q_1) = \frac{1}{8}.$$

Therefore the symplectic map moved uphill under this objective.

This is not a defect in the integrator.

The map was derived to preserve variational/symplectic structure, not to minimize F at every step.

35.21 Optimization and mechanics use different objectives

This distinction is easy to blur.

In mechanics:

- the action determines equations of motion;
- the Hamiltonian can represent energy;
- a symplectic map preserves phase-space geometry.

In optimization:

- an objective is to be minimized;
- a step may be designed for descent, convergence, acceleration, constraint preservation, or another criterion.

A variational optimizer can connect these worlds.

It must still state which mathematical object is doing which job.

35.22 Manifold geometry re-enters

MANOPT-001 established another route to geometry-derived updates.

Given a constrained state:

1. compute an ambient or Riemannian gradient;
2. project or represent it in the tangent space;
3. choose a tangent step;
4. retract back to the manifold.

This pipeline resembles variational construction in one broad sense:

the update is constrained by declared geometry.

But a retraction-based update need not arise from an action principle.

35.23 MODULUS as programme evidence

At protected commit:

7ca4ffcdace32d5ff79c27ad557bfb690fac0af3

the GCL MODULUS project describes itself as an optimization research toolkit that treats geometry as a first-class optimizer control surface.

Its Hyperball implementation wraps a base optimizer.

The base optimizer proposes an update.

Hyperball can then:

- project it into a tangent space;
- control its norm;
- apply an approximate angular-step policy;
- retract the result to a sphere;
- or apply ball-style norm control.

This is a concrete programme example of deriving an update from explicit geometric constraints.

35.24 The exact MODULUS pipeline

For parameter group w and base proposal u , the source implements:

$$u_{\parallel} = \frac{\langle u, w \rangle}{\|w\|^2} w,$$

$$u_{\perp} = u - u_{\parallel}.$$

A target-angle branch sets desired tangent-update norm approximately to:

$$\alpha \|w\|.$$

The finite point is then retracted.

This separates:

- base optimizer policy;
- tangent geometry;
- step-control policy;
- finite constraint restoration.

35.25 Exact programme witness

Take:

$$w = (1, 0), \quad u = (1, 1).$$

Then:

$$u_{\parallel} = (1, 0),$$

and:

$$u_{\perp} = (0, 1).$$

With programme parameter:

$$\alpha = 1$$

and unit radius, the pre-retraction tangent update has norm 1.

The retracted point is:

$$w_+ = \frac{(1, 1)}{\sqrt{2}}.$$

35.26 The target-angle subtlety

The exact angle from w to w_+ is:

$$\frac{\pi}{4}.$$

That is not 1 radian.

For unit tangent direction v , the general finite update:

$$w_+ = \frac{w + \alpha v}{\sqrt{1 + \alpha^2}}$$

has:

$$\theta = \arctan \alpha.$$

For small α :

$$\theta = \alpha - \frac{\alpha^3}{3} + O(\alpha^5).$$

So the implementation's target-angle parameter acts as a tangent-norm angular proxy.

It agrees to first order.

It is not an exact exponential-map/geodesic-angle control at finite step.

35.27 Why this distinction matters

If a parameter is called an angle, users may mentally identify it with a geodesic angle.

The exact construction shows the actual semantics.

This is the same discipline applied throughout the Atlas:

- name the representation;
- name the update;
- derive what it actually preserves;
- do not import stronger semantics from a convenient label.

35.28 MODULUS is not being promoted to general theory

The GCL repository profile for MODULUS is explicit.

Its authority covers:

- versioned implementation;
- benchmarks;
- empirical evidence.

It has no claim-promotion or certification authority.

Accordingly, VARIOPT uses MODULUS to show what one governed GCL geometry-first update pipeline actually does.

Established mathematical claims continue to come from established sources or Atlas proofs.

35.29 Geometry-derived is broader than variational

There are now at least three distinct mechanisms in view.

35.29.1 Divergence-derived

Choose a convex generator and derive a local proximal/mirror step.

35.29.2 Variational

Choose an action or Lagrangian and derive stationarity equations.

35.29.3 Constraint-geometric

Choose a legal state geometry and transform an update through projection/retraction.

All three can produce structured updates.

They are not interchangeable derivations.

35.30 A variational derivation is not a convergence certificate

Suppose a discrete rule comes from a stationary discrete action.

That tells us where the rule came from.

Convergence of an optimization objective can still require:

- convexity;
- smoothness;
- coercivity;
- step restrictions;
- damping;
- Lyapunov arguments;
- stochastic assumptions;
- constraint regularity.

The derivation and the convergence proof are separate artifacts.

35.31 A divergence is not automatically the right geometry

Bregman divergences are generated by a chosen convex function.

Different generators define different local Hessians and different dual coordinates.

The right generator can depend on:

- domain;
- constraints;
- scale;

- sparsity;
- probability structure;
- desired invariances.

No source in this chapter proves one universal generator.

35.32 A symplectic optimizer would still need an optimization story

If an optimization method is designed to preserve a symplectic form in an enlarged state containing momentum, that can be a meaningful structural property.

But one must still answer:

- what objective is minimized?
- what damping is present?
- what convergence notion is used?
- how is the discrete step chosen?
- what happens to energy-like quantities?
- what does the structure buy empirically?

Symplecticity alone does not answer them.

35.33 Bregman-Lagrangian acceleration remains source-scoped

The Bregman-Lagrangian framework demonstrates that acceleration can be organized through continuous-time variational structure under specific scaling conditions.

That is stronger than an analogy.

But it remains a theorem about a declared framework.

The Atlas does not infer:

has a divergence $\Rightarrow$ accelerated,

nor:

has a Lagrangian $\Rightarrow$ optimal convergence.

35.34 The final architecture lesson

The useful abstraction is not “physics-inspired optimizer.”

It is more precise:

choose structure $\rightarrow$ derive legal dynamics $\rightarrow$ discretize explicitly $\rightarrow$ prove or measure the property actu

Each arrow can fail if left implicit.

35.35 Evidence classes

A VARIOPT claim should identify whether it is:

- a standard theorem from the mathematical literature;
- an Atlas exact finite derivation;
- a numerical model;
- a GCL programme implementation fact;
- an empirical benchmark result;
- an architectural proposal.

This chapter deliberately contains all but the empirical-superiority class.

They remain separate.

35.36 Durable non-implications

The chapter freezes:

divergence $\nRightarrow$ metric,

divergence-derived update $\nRightarrow$ descent,

variational derivation $\nRightarrow$ accuracy or stability,

symplecticity $\nRightarrow$ energy conservation,

symplecticity $\nRightarrow$ objective descent,

constraint preservation $\nRightarrow$ optimizer superiority,

and:

GCL programme mechanism $\nRightarrow$ established general theory.

35.37 Closing the architecture layer

VARIOPT is the last chapter still in architecture state at this transaction baseline.

Its role is therefore not to add another optimizer catalog.

It closes an architectural question that runs through the Atlas:

how should adaptive systems derive updates when the state space,
numerical structure, or optimization geometry matters?

The answer is not one universal rule.

It is a disciplined methodology:

1. declare the geometry or action;
2. derive the update;
3. identify the preserved structure;
4. distinguish the discrete algorithm from its continuous source;
5. test descent, stability, accuracy, and empirical utility separately;
6. bind programme-specific mechanisms to their actual authority.

35.38 References used in this chapter

- L. M. Bregman (1967), *The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming*, DOI 10.1016/0041-5553(67)90040-7.
- J. E. Marsden and M. West (2001), *Discrete mechanics and variational integrators*, Acta Numerica 10:357–514, DOI 10.1017/S096249290100006X.
- Andre Wibisono, Ashia C. Wilson, and Michael I. Jordan (2016), *A variational perspective on accelerated methods in optimization*, PNAS 113(47):E7351–E7358, DOI 10.1073/pnas.1614734113.
- protected GCL MODULUS implementation/programme evidence at commit 7ca4ffcdace32d5ff79c27ad557bfb690fac0af3.

Exact authority and claim boundaries are recorded in:

sources/source-locks/ATLAS-CH-VARIOPT-001.yaml

Chapter 36

Networks as Numerical Schemes

Epistemic status: established numerical analysis + primary residual/continuous-depth architecture sources + Atlas synthesis + exact scalar witness.

Specification: manuscript/specifications/ATLAS-CH-NETNUM-001.md

Formal packet: mathematics/derivations/ATLAS-CH-NETNUM-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-NETNUM-001.md

Source lock: sources/source-locks/ATLAS-CH-NETNUM-001.yaml

A residual block looks like Euler’s method.

That resemblance is useful.

It is also one of the easiest analogies in modern deep learning to overstate.

Write a residual layer as

$$x_{k+1} = x_k + F_k(x_k).$$

Write one explicit-Euler step for

$$\frac{dx}{dt} = f(t, x).$$

as

$$x_{k+1} = x_k + h_k f(t_k, x_k).$$

The shapes match.

But numerical analysis does not begin with a shape match.

It begins with a reference problem.

To speak meaningfully about consistency, local truncation error, global error, stability, step size, stiffness, or reversibility, we must know what continuous evolution the discrete map is supposed to approximate, how one layer corresponds to one step, what changes when the mesh is refined, and which norm or state metric is being used.

The governing rule of this chapter is therefore:

residual form opens the numerical lens; it does not, by itself, complete the numerical interpretation.

That distinction lets us use numerical analysis rigorously rather than decoratively.

36.1 From architecture to update map

The Architecture History chapter established the residual form

$$x_{k+1} = x_k + F_k(x_k).$$

as a structural shift.

The identity path transports the current state forward.

The residual branch computes a change.

This makes depth look less like repeated replacement and more like repeated state update.

The Numerical Methods chapter supplied the corresponding mathematical object:

$$x_{n+1} = \Psi_h(t_n, x_n).$$

A one-step method takes the current numerical state and advances it by a discrete rule.

These objects align naturally.

The question is not whether they look similar.

The question is what additional assumptions make the alignment mathematically substantive.

36.2 The bridge must be declared

Suppose the intended continuous system is

$$\frac{dx}{dt} = f(t, x).$$

Explicit Euler gives

$$x_{k+1} = x_k + h_k f(t_k, x_k).$$

A residual layer becomes Euler-compatible after the declaration

$$F_k(x) = h_k f(t_k, x).$$

This line performs the bridge.

It says:

- which continuous field is being sampled;
- which layer corresponds to which time;
- which scale is the step size.

Without that bridge, F_k is simply a learned residual map.

There is nothing wrong with that.

But numerical-analysis terminology must remain attached to the object it actually describes.

36.3 Residual architecture is not a unique ODE

Could any residual stack be rewritten as some continuous system?

In a loose interpolation sense, often yes.

But that is not the same as saying the network has a unique or canonical underlying ODE.

A finite sequence of maps

$\Psi_0, \Psi_1, \dots, \Psi_{N-1}$

can be embedded or interpolated in many ways.

Different continuous fields may agree on the sampled layer states.

Different time parameterizations may produce the same discrete sequence.

Different modified equations may explain the same discrete update to a given asymptotic order.

Therefore the ODE is not generally recovered uniquely from residual syntax.

The Architecture History audit already fixed the safe statement:

residual systems admit an integrator lens.

This chapter sharpens the conditions under which that lens supports numerical claims.

36.4 Autonomous versus nonautonomous depth

Weight sharing is sometimes treated as the dividing line between dynamical and nondynamical networks.

That is too crude.

If

$$F_k(x) = hf(x).$$

for a common $\mathbf{f}$, then the blocks have the direct form of fixed-step Euler applied to an autonomous ODE.

But a continuous system can be explicitly time-dependent:

$$\frac{dx}{dt} = f(t, x).$$

Then layer-varying residuals can correspond to

$$F_k(x) = h_k f(t_k, x).$$

So untied weights do not rule out an ODE interpretation.

They may correspond to a nonautonomous field.

The real issue is whether there is a declared continuous object and a meaningful sampling/refinement relation.

36.5 A layer is not yet a time step

Suppose we introduce a residual scale:

$$x_{k+1} = x_k + \alpha_k G_k(x_k).$$

It is tempting to call α_k the step size.

Sometimes that is exactly right.

If G_k samples a declared vector field, then α_k can play the role of h_k .

But if changing α_k is accompanied by retraining G_k , changing normalization, changing feature geometry, or otherwise changing the update field, then the comparison is not simply “same ODE, smaller step.”

A step size has meaning only relative to an underlying evolution.

A residual scale always changes the network map.

It does not automatically refine a fixed continuous problem.

36.6 Local truncation error requires an exact flow

The Numerics chapter defined the exact flow over one step as

Φ_h

and the numerical map as Ψ_h .

The local defect is computed by starting the numerical step from the exact state:

$$\delta_{k+1} = \Phi_{h_k}(t_k, x(t_k)) - \Psi_k(x(t_k)).$$

For explicit Euler, under the usual smoothness assumptions,

$$\delta_{k+1} = O(h_k^2).$$

That statement contains more structure than it first appears.

It assumes:

- a continuous trajectory exists;
- an exact flow is defined;
- the layer is tied to a numerical step;
- h_k is a meaningful small parameter.

If a residual block is merely a learned map with no declared continuous reference, then saying it has “small local truncation error” is not merely unproven.

The quantity has not been defined.

36.7 Small residual does not mean small truncation error

Suppose

$$\|F_k(x)\|$$

is small.

Does that imply the corresponding Euler defect is small?

No.

A small update norm can arise because:

- the vector field itself is small;
- the step is small;
- the representation is scaled;
- normalization suppresses magnitude;
- the network has learned a near-identity map;
- the local coordinates make the update appear small.

Local truncation error compares a numerical step to the exact flow of a specified continuous problem.

The norm of the residual branch alone does not perform that comparison.

This is a recurring category error:

small update is not the same statement as accurate discretization.

36.8 Global error is not layerwise error added up

Define the global numerical error as

$$e_k = x(t_k) - x_k.$$

This error is not simply the sum of local defects.

Each old error is transported through later steps.

For a one-step method with the declared Lipschitz error-propagation bound, the error recursion has the schematic form

$$\|e_{k+1}\| \leq (1 + Ch_k)\|e_k\| + \|\delta_{k+1}\|.$$

Here C and the norm depend on the fixed reference problem and interval; this is not an unconditional guarantee for a learned residual network.

The factor multiplying e_k matters.

Repeated amplification can dominate small local defects.

That is why numerical analysis separates:

- consistency;
- stability;

- convergence.

A network analogy that mentions only one-step approximation misses the accumulated-dynamics problem.

36.9 More layers do not automatically mean finer discretization

Take a 12-layer residual network and a 24-layer residual network.

Can we say the second uses half the step size?

Only if the two belong to a declared refinement family.

For a numerical refinement, we normally hold the continuous problem fixed while changing the mesh.

A deeper neural network may instead change:

- all learned parameters;
- residual scaling;
- width;
- normalization;
- activation structure;
- feature coordinates;
- training objective;
- optimizer.

That is a different model, not necessarily a finer mesh.

So the statement

more layers give smaller discretization error

needs an actual refinement construction.

Depth alone does not supply it.

36.10 The scalar test equation enters neural depth

The Numerical Methods chapter used the scalar test equation

$$\dot{x} = \lambda x.$$

This small system exposes stability cleanly.

For explicit Euler,

$$x_{k+1} = (1 + h\lambda)x_k.$$

Define

$$z = h\lambda.$$

The amplification factor is

$$R(z) = 1 + z.$$

The standard non-growth stability condition is

$$|1 + z| \leq 1.$$

This exact calculation transfers directly to a linear residual block when the block has been declared to represent the Euler step.

Now numerical stability has architectural meaning:

small perturbations in that scalar mode are multiplied by the same amplification factor across depth.

36.11 A stable continuous mode can become an unstable network step

Set

$$\lambda = -1.$$

The continuous solution decays:

$$x(t) = e^{-t}x(0).$$

Euler gives

$$x_{k+1} = (1 - h)x_k.$$

The non-growth condition is

$$|1 - h| \leq 1.$$

For real nonnegative h , the non-growth set is

$$0 \leq h \leq 2.$$

Strict asymptotic decay requires

$$0 < h < 2.$$

At $h=2$, the factor is -1 : bounded but non-decaying.

Now choose

$$h=3.$$

The exact continuous factor over one step is

$$e^{-3}.$$

The Euler factor is

-2 .

The continuous mode decays strongly.

The discrete mode doubles in magnitude and flips sign.

Nothing happened to the ODE.

The discretization changed the dynamics.

This is one of the most useful lessons numerical analysis contributes to neural architecture:

the update rule has its own dynamics.

36.12 Numerical stability is not training stability

The previous example concerns forward propagation of state perturbations.

That is not the same thing as optimization stability.

Training can be unstable because of:

- exploding gradients;
- vanishing gradients;
- learning-rate choice;
- optimizer state;
- minibatch noise;
- curvature;
- non-normal transients;
- parameterization.

Forward numerical stability and training stability can interact.

They can share Jacobian structure.

But they are not interchangeable.

Haber and Ruthotto explicitly use dynamical-systems and discrete-ODE stability ideas to motivate stable deep architectures (Haber and Ruthotto 2018).

That bridge is useful precisely because the notions are related.

It should not be read as an identity between all numerical and optimization stability concepts.

36.13 The residual Jacobian is a perturbation propagator

For

$$\Psi_k(x) = x + F_k(x),$$

the Jacobian is

$$J_{\{\Psi_k\}}(x) = I + J_{\{F_k\}}(x).$$

For a small perturbation δx_k ,

$$\delta x_{k+1} \text{ approximately } J_{\{\Psi_k\}}(x_k) \delta x_k.$$

Across many layers,

$$\delta x_N \text{ approximately } J_{\{\Psi_{N-1}\}} \dots J_{\{\Psi_0\}} \delta x_0.$$

This is a discrete tangent evolution.

It is one reason residual systems are naturally analyzed dynamically.

But again the claim must remain scoped.

The Jacobian product describes local first-order perturbation propagation along the realized discrete trajectory.

It is not, by itself, a theorem about generalization or optimization success.

36.14 Exact finite witness: same flow, different discretization behavior

Consider

$$x' = -x, \quad x(0) = 1.$$

The exact solution at time 1 is

$$e^{-1}.$$

Use explicit Euler with N equal steps:

$$h = 1/N.$$

Then

$$x_N = (1 - 1/N)^N.$$

For $N=2$:

$$x_2 = 1/4.$$

For $N=4$:

$$x_4 = 81/256.$$

For $N=8$:

$$x_8 = 5764801/16777216.$$

These values move toward

e^{-1}

over the declared finite sequence.

The exact witness records both the rational values and their error against the exact flow.

But the finite table is not the convergence theorem.

The theorem that Euler is globally first order under standard assumptions comes from numerical analysis (Iserles 2008).

The witness only makes the mechanism visible.

36.15 Refinement is a family, not a single network

The exact witness contains something ordinary neural-network comparisons often lack.

It contains a controlled family:

- same ODE;
- same initial state;
- same final time;
- same numerical method;
- only the mesh changes.

That is what gives the phrase **refinement** mathematical force.

If a neural architecture study wants to make an analogous convergence claim, it needs a corresponding family.

For example, it might declare:

- a continuous target field;
- a sequence of grids;
- parameter interpolation across grids;
- a residual scaling rule;
- a common output time;
- a norm for comparing trajectories.

Without such a construction, comparing depths is architecture comparison, not numerical convergence analysis.

36.16 A modified-equation viewpoint

Numerical analysts often ask a different question:

which nearby continuous equation is the discrete method solving more accurately?

This leads to modified-equation reasoning.

Lu, Zhong, Li, and Dong explicitly connect deep architectures with numerical differential-equation discretizations and use numerical ideas to motivate architecture design (Lu et al. 2018).

This perspective can be more realistic than insisting on one exact underlying ODE.

A trained network may be profitably studied through a nearby effective flow.

But the same boundary remains:

the modified equation is an analytical construction under declared assumptions.

It is not automatically the network's unique hidden physical law.

36.17 Residual networks and Neural ODEs are related but different

The Architecture History chapter source-locks both residual networks and Neural ODEs (He et al. 2016; Chen et al. 2018).

A residual network directly defines finitely many maps.

A Neural ODE directly defines a continuous vector field and asks a numerical solver to construct the trajectory.

Those directions of definition differ.

For a ResNet:

`discrete architecture -> possible continuous interpretation.`

For a Neural ODE:

`continuous dynamics -> chosen numerical discretization at execution.`

The two meet through numerical analysis.

They should not be collapsed.

36.18 The discretization can become part of the model

In a classical numerical simulation, changing the solver while refining the mesh is often intended to preserve the same underlying physical model.

In learned systems, the situation can be more entangled.

The architecture, solver, parameterization, and training process may adapt to one another.

A change of discretization may change:

- optimization geometry;
- parameter sharing;
- expressive bias;
- compute cost;
- numerical error;
- learned representation.

So the phrase **discretization error** should be reserved for error relative to a declared continuous reference.

Performance change after changing architecture is a broader empirical phenomenon.

36.19 Reversibility has three meanings

The word **reversible** causes another common collision.

We must distinguish:

36.19.1 Map invertibility

A layer map Ψ is one-to-one and has an inverse.

36.19.2 Computational reversibility

Earlier states can be reconstructed from later states, perhaps to reduce activation storage.

36.19.3 Time reversibility of a numerical method

A one-step method is symmetric if

$$\Psi_{-h} = \Psi_h^{-1}.$$

These properties can overlap.

They are not identical.

A layer can be invertible without being a symmetric integrator.

36.20 Explicit Euler is invertible and not time reversible

Return to

$$x' = -x.$$

Euler gives

$$\text{Psi}_h(x) = (1-h)x.$$

For

$$h \neq 1,$$

the map is invertible.

Its inverse is

$$\text{Psi}_h^{-1}(x) = x/(1-h).$$

Now apply Euler with negative step:

$$\text{Psi}_{-h}(x) = (1+h)x.$$

Compose:

$$\text{Psi}_{-h}(\text{Psi}_h(x)) = (1-h^2)x.$$

Except at $h=0$, this is not the identity.

So the same update formula run with $-h$ does not undo the forward step.

The method is not time reversible.

36.21 Exact reversibility witness

Choose

$$h=1/2.$$

The forward step multiplies by

$$1/2.$$

Its true inverse multiplies by

$$2.$$

The negative-step Euler map multiplies by

$$3/2.$$

Forward followed by negative-step Euler gives

$$3/4.$$

That one line separates the concepts:

- invertible map: yes;
- same method with negative step is inverse: no;
- time-symmetric method: no.

The exact flow behaves differently:

$$e^{\{h\}}e^{\{-h\}}=1.$$

36.22 Why this matters for reversible neural networks

A reversible neural architecture may guarantee that activations can be reconstructed.

That is valuable for memory.

But it does not follow that the architecture is a reversible numerical integrator for an underlying differential equation.

Conversely, a symmetric numerical integrator has a time-reversal property tied to its step map.

Its software implementation may still use memory in an ordinary irreversible way.

The adjective is the same.

The mathematical objects are different.

36.23 Stability also has several meanings

The same linguistic problem appears with **stability**.

In this chapter we need at least:

- absolute stability of a numerical method;
- perturbation stability of a discrete map;
- Lyapunov stability of a dynamical system;
- conditioning/sensitivity;
- optimization stability;
- robustness under data or distribution perturbation.

The Numerics chapter already warned that the scalar stability region is not a universal nonlinear certificate.

NETNUM adds a second warning:

the word “stable” should not move between numerical analysis and training dynamics without an explicit bridge.

This is especially important because deep-learning papers often use one term for several phenomena.

36.24 Stiffness in networks needs a declared dynamical problem

Can a neural network be stiff?

Sometimes the numerical analogy is useful.

If a declared continuous model has widely separated decay scales, an explicit discretization may face severe step restrictions.

But “the network has large Jacobian eigenvalues” is not, by itself, a complete stiffness statement.

Stiffness is partly problem-relative and method-relative (Hairer and Wanner 1996).

To use the term rigorously we should identify:

- the continuous or effective dynamical problem;
- the numerical method;
- the scale separation;
- the step restriction or solver difficulty it induces.

Otherwise **stiff** risks becoming a synonym for “hard to train.”

36.25 The numerical lens suggests architecture questions

Once the bridge is explicit, numerical analysis supplies useful design questions.

For a residual architecture, ask:

- Is the forward map in a stable regime?
- Does the update approximate a declared flow?
- What changes under depth refinement?
- Is the residual scale acting like a step size?
- Would an implicit update enlarge the stable region?
- Would a symmetric or structure-preserving composition protect an invariant?
- Does operator splitting isolate meaningful subdynamics?
- Does adaptive depth correspond to local error control?

These questions do not answer themselves.

They create disciplined hypotheses.

That is more valuable than merely calling a network an ODE.

36.26 What Haber–Ruthotto adds

Haber and Ruthotto develop deep architectures from the perspective of nonlinear dynamical systems and discrete ODE stability (Haber and Ruthotto 2018).

The important Atlas lesson is methodological.

Numerical analysis can be used not just to interpret an existing network after the fact, but to constrain architecture design.

If a discrete forward model has undesirable stability properties, modify the architecture.

This turns the numerical lens into engineering.

Still, source scope matters.

The paper motivates particular stable constructions.

It does not establish that every network instability is a numerical ODE instability.

36.27 What Lu et al. adds

Lu et al. connect several deep architectures to numerical differential-equation schemes and propose architectures inspired by numerical methods (Lu et al. 2018).

This widens the design vocabulary.

A network block need not be compared only with forward Euler.

One can ask about analogues of:

- multistep methods;
- higher-order schemes;
- implicit methods;
- splitting methods;
- modified equations.

The Atlas uses this as a bridge, not as a license to label any multi-branch network with a numerical-method name.

Structural correspondence must be shown.

36.28 Architecture analogy versus numerical theorem

This distinction deserves a compact table.

Statement	Status
$x_{k+1} = x_k + F_k(x_k)$ resembles Euler form	algebraic observation
$F_k = h_k \mathbf{f}(t_k, \cdot)$ for declared $\mathbf{f}, h_k$	modeling declaration
local defect is $O(h^2)$	numerical theorem under smoothness assumptions
global error is $O(h)$	numerical theorem under stability/regularity assumptions
deeper network has smaller error	not implied without refinement semantics
residual Jacobian is $\mathbf{I} + \mathbf{J}_F$	exact differential identity
stable scalar Euler factor	exact for declared linear mode
training is stable	separate empirical/mathematical claim

The point is not to weaken the analogy.

It is to give each statement the correct support route.

36.29 A practical numerical-network contract

Before using numerical language for a network, record:

Field	Question
reference flow	What continuous evolution is being approximated?
step map	What network block corresponds to one numerical step?
mesh	What are t_k and h_k ?
refinement	How is depth increased while preserving the same reference problem?
error	What norm and exact reference define local/global error?
stability	Which perturbation and which stability notion are intended?
reversibility	Invertibility, reconstruction, or numerical symmetry?
scope	Which claims are theorem, interpretation, or empirical observation?

This contract prevents the numerical lens from silently changing categories.

36.30 Failure modes

36.30.1 Residual-equals-Euler

A skip connection is treated as proof that the network is Euler discretization.

Failure: no continuous reference problem is specified.

36.30.2 Depth-equals-time

Layer index is silently relabeled time.

Failure: no mesh or refinement relation exists.

36.30.3 Small-residual-equals-small-error

A small residual update is treated as a small truncation defect.

Failure: no exact flow comparison is made.

36.30.4 Stable-forward-equals-stable-training

A forward amplification argument is promoted into an optimizer theorem.

Failure: the objects differ.

36.30.5 Invertible-equals-reversible-integrator

An invertible layer is called time reversible.

Failure: $\Psi_{-h} = \Psi_h^{-1}$ was never checked.

36.30.6 More-layers-equals-convergence

A deeper model is assumed closer to a continuous limit.

Failure: the target model and discretization family changed together.

36.31 Downstream handoff: Adaptive Depth

Adaptive Depth as Error Control — ATLAS-CH-ADAPTDEPTH-001

may now assume:

- exact-flow versus step-map distinction;
- local versus global error;
- refinement-family discipline;
- step-size versus residual-scale distinction;

- scalar stability-function reasoning.

That chapter can ask whether learned stopping or variable compute can be interpreted as adaptive time stepping or local error control.

It must establish that bridge itself.

A stopping score is not automatically a local truncation-error estimator.

36.32 Downstream handoff: Boundary Probes

Boundary Probes — **ATLAS-CH-BOUNDARYPROBE-001** may now assume:

- one-step map Psi_k ;
- local Jacobian $J_{\{\text{Psi}_k\}}$;
- first-order perturbation propagation;
- accumulated Jacobian products;
- the distinction between local sensitivity and global dynamics.

This prepares the language needed for JVPs, VJPs, power iteration, and interface sensitivity.

36.33 What this chapter establishes

The numerical interpretation of neural depth is strongest when it is conditional and explicit.

A residual architecture can be read as a numerical scheme when the continuous problem, step semantics, and refinement family are supplied.

Then numerical analysis contributes exact distinctions:

- local defect versus global error;
- consistency versus stability versus convergence;
- residual scale versus step size;
- forward stability versus optimization stability;
- invertibility versus time reversibility.

Without those declarations, the same vocabulary remains metaphor.

The goal is not to ban the metaphor.

It is to know when the metaphor has become mathematics.

36.34 References used in this chapter

- Iserles, *A First Course in the Numerical Analysis of Differential Equations* (Iserles 2008).

- Hairer and Wanner, *Solving Ordinary Differential Equations II* (Hairer and Wanner 1996).
- He et al., *Deep Residual Learning for Image Recognition* (He et al. 2016).
- Chen et al., *Neural Ordinary Differential Equations* (Chen et al. 2018).
- Haber and Ruthotto, *Stable Architectures for Deep Neural Networks* (Haber and Ruthotto 2018).
- Lu et al., *Beyond Finite Layer Neural Networks: Bridging Deep Architectures and Numerical Differential Equations* (Lu et al. 2018).

Exact prerequisite identities and source-authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-NETNUM-001.yaml`

Chapter 37

Adaptive Depth as Error Control

Epistemic status: audited numerical/adaptive-depth substrate + primary adaptive-computation sources + Atlas synthesis.

Specification: manuscript/specifications/ATLAS-CH-ADAPTDEPTH-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-ADAPTDEPTH-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-ADAPTDEPTH-001.md

Source lock: sources/source-locks/ATLAS-CH-ADAPTDEPTH-001.yaml

Adaptive computation sounds like one idea.

It is not.

A numerical solver may adapt its step size because a local error estimate is too large.

A recurrent network may run more steps because a learned halting unit has not fired.

A residual model may spend more layers on one spatial position than another.

A system may terminate because it exhausted its budget even though its quality criterion was not met.

All of these change computation.

They do not carry the same semantics.

This chapter asks when adaptive depth deserves the stronger name **error control**.

The answer is deliberately strict:

adaptive computation becomes error control only when the system names a target error, an estimator or certified relation to that error, a tolerance, and a controller whose stopping or step-size decision is justified by that relation.

Without those objects, the system may still be useful.

It is simply doing adaptive computation rather than certified error control.

37.1 The four control surfaces

The audited Depth chapter separated computational depth from physical time.

The audited Networks-as-Numerical-Schemes chapter separated residual updates from literal numerical discretizations.

This chapter combines those boundaries.

Four objects recur.

37.1.1 Execution depth

How many transformations are actually executed?

$$z_0 \rightarrow z_1 \rightarrow \cdots \rightarrow z_\tau.$$

The stopping depth τ may depend on the input.

37.1.2 Numerical step size

For a declared reference evolution,

$$x_{n+1} = \Psi_{h_n}(x_n),$$

the parameter h_n controls the numerical mesh.

37.1.3 Local error estimate

A numerical controller may construct

$$\widehat{e}_n$$

to estimate a declared local truncation/error quantity.

37.1.4 Learned halting signal

A learned system may produce

$$q_k$$

and stop when q_k crosses a threshold.

The Atlas will not silently identify any pair of these objects.

37.2 What adaptive depth already gives us

Graves's Adaptive Computation Time provides a differentiable mechanism for learning how many recurrent computational steps to allocate before producing an output (Graves 2016).

Figurnov et al. extend adaptive computation into residual vision systems where different spatial positions can receive different amounts of computation (Figurnov et al. 2017).

These are important mechanisms.

They show that a network can learn to allocate compute nonuniformly.

They do not, by themselves, show that the learned halting signal estimates:

- truncation error;
- approximation error;
- task loss;
- probability of correctness;
- distance to a fixed point;
- distance to a target representation.

A learned stop signal can be useful without being an error estimate.

37.3 Numerical error control is more specific

Now consider a declared numerical problem.

Let

$$x_{n+1} = \Psi_{h_n}(x_n).$$

A numerical error controller needs more than a variable h_n .

It needs a target quantity.

For example:

e_n = local one-step error in a declared norm.

Because the exact local error is usually unavailable, the solver constructs an estimator

$$\hat{e}_n.$$

Then the controller compares it with a tolerance

$$\tau_n.$$

A basic accept/reject rule is

$$\hat{e}_n \leq \tau_n.$$

This is already much stronger than “run until a score is small.”

The score has declared semantics.

37.4 Embedded formulas

A common numerical pattern is to obtain two approximations from related stage evaluations.

Write

$$x_{n+1}^{[p]}$$

and

$$x_{n+1}^{[p+1]}.$$

Then a local estimator may use

$$\hat{e}_n = \left\| x_{n+1}^{[p+1]} - x_{n+1}^{[p]} \right\|.$$

Dormand and Prince develop a family of embedded Runge-Kutta formulas with different orders (Dormand and Prince 1980).

The important structural point for this chapter is not the exact Dormand-Prince coefficient table.

It is the control pattern:

- two approximations;
- one step;
- one difference;
- one local error estimate;
- one decision about accepting or shrinking the step.

This pattern gives the phrase “error-controlled adaptation” real mathematical content.

37.5 Estimator is not oracle

Even in numerical analysis, an embedded difference is not magic.

Its interpretation depends on:

- the declared differential equation;
- the method;
- smoothness;
- order conditions;
- the norm;
- the local asymptotic regime.

The safe statement is:

embedded difference $\rightarrow$ method-specific local error estimator.

The unsafe statement is:

embedded difference = exact total error

for every problem.

The finite witness below is chosen precisely because the equality happens to be exact there.

37.6 A one-step exact witness

Consider

$$y'(t) = t, \quad y(0) = 0.$$

The exact solution is

$$y(t) = \frac{t^2}{2}.$$

Take one step from $t = 0$ to $t = h$.

Euler gives

$$y_E = 0.$$

Why?

Because

$$f(0, 0) = 0.$$

Now use the explicit trapezoid / Heun update.

Its first slope is

$$k_1 = 0.$$

The Euler predictor is still zero.

The second slope is

$$k_2 = f(h, 0) = h.$$

Therefore

$$y_H = \frac{h}{2}(k_1 + k_2) = \frac{h^2}{2}.$$

But the exact solution at h is also

$$y(h) = \frac{h^2}{2}.$$

Thus, for this witness,

$$y_H = y(h).$$

The pair difference is

$$|y_H - y_E| = \frac{h^2}{2}.$$

And that equals the exact Euler one-step error.

This is unusually clean.

It lets us examine the controller without hiding behind asymptotic notation.

37.7 Rejecting a step

Set the tolerance to

$$\tau = \frac{1}{8}.$$

Try

$$h = 1.$$

The estimated error is

$$\hat{e} = \frac{1}{2}.$$

Therefore

$$\hat{e} > \tau.$$

The step is rejected.

Notice what was declared:

- target: Euler one-step error in absolute value;
- estimator: Euler/Heun difference;
- tolerance: $1/8$;
- decision: reject if estimator exceeds tolerance.

That is an error-control interface.

37.8 Shrinking the step

For a low-order method of order p , local truncation error scales like

$$Ch^{p+1}$$

in the asymptotic regime.

An idealized controller can use

$$h_{\text{new}} = h \left(\frac{\tau}{\hat{e}} \right)^{1/(p+1)}.$$

For Euler,

$$p = 1.$$

Using

$$h = 1, \quad \tau = \frac{1}{8}, \quad \hat{e} = \frac{1}{2},$$

we obtain

$$h_{\text{new}} = \sqrt{\frac{1/8}{1/2}} = \frac{1}{2}.$$

Retry at $h = 1/2$.

Then

$$\hat{e} = \frac{(1/2)^2}{2} = \frac{1}{8}.$$

The retry lands exactly on tolerance.

Practical controllers use safety factors and clamps.

The witness omits them because the point is the semantics, not production solver tuning.

37.9 Error control is a feedback loop

The controller has the form

estimate $\rightarrow$ compare $\rightarrow$ accept/reject $\rightarrow$ change future computation.

This is why adaptive numerical integration is conceptually useful for adaptive neural computation.

Both are feedback systems.

But analogy is not identity.

The crucial difference is what the feedback signal means.

37.10 A learned halting signal

A learned computation may evolve

$$z_{k+1} = F_k(z_k)$$

and emit a score

$$q_k.$$

Given threshold δ and budget cap $K_{\max}$,

$$\tau = \min(\{k \leq K_{\max} : q_k \leq \delta\} \cup \{K_{\max}\}).$$

This is a perfectly valid stopping rule.

It says nothing yet about whether

$$q_k$$

is:

- a probability;
- a task-loss estimate;
- a residual;
- a confidence score;
- a distance to convergence;
- a numerical local-error estimate.

The semantics must come from somewhere else.

37.11 Learned stopping is not numerical stopping

Graves ACT uses learned halting to allocate recurrent computation (Graves 2016).

Its mechanism is not “estimate a Runge-Kutta truncation error and reduce the step.”

Figurnov et al. allocate computation spatially in residual networks (Figurnov et al. 2017).

Their mechanism is not “refine a finite-element mesh where a PDE residual is large.”

These systems can inspire numerical analogies.

The Atlas preserves the direction of implication:

adaptive computation $\Rightarrow$ variable compute allocation.

It does not infer:

adaptive computation $\Rightarrow$ numerical error control.

37.12 Perfect ranking is still not calibration

A common temptation is weaker.

Suppose the learned halting score tracks difficulty very well.

Perhaps smaller score always means smaller true error.

Is that enough?

No.

Consider the audited contraction from the Depth chapter:

$$x_{k+1} = \frac{x_k + 2}{2}, \quad x_0 = 0.$$

The fixed point is 2.

Its exact error is

$$E_k = |x_k - 2| = 2^{1-k}.$$

Now define

$$q_k = 4^{-k}.$$

Then

$$q_k = \frac{E_k^2}{4}.$$

The score is a strictly monotone function of the true error.

It ranks every depth perfectly.

But it is not on the same scale.

37.13 An exact premature-halting witness

At depth

$$k = 2,$$

the score is

$$q_2 = \frac{1}{16}.$$

The true error is

$$E_2 = \frac{1}{2}.$$

Suppose we want true error at most

$$\frac{1}{16}.$$

If we mistakenly use the same numerical threshold on the learned score,

$$q_k \leq \frac{1}{16},$$

the system stops at $k = 2$.

But

$$E_2 = \frac{1}{2}$$

is eight times the requested tolerance.

This failure is not due to poor ranking.

The ranking is perfect.

The failure is magnitude calibration.

37.14 The missing map

In the witness, the relation is exactly

$$E_k = 2\sqrt{q_k}.$$

Once that map is known, the halting score can support correct thresholding.

For target error

$$\varepsilon,$$

we require

$$2\sqrt{q_k} \leq \varepsilon.$$

So the correct score threshold is

$$q_k \leq \left(\frac{\varepsilon}{2}\right)^2.$$

For

$$\varepsilon = \frac{1}{16},$$

this becomes

$$q_k \leq \frac{1}{1024}.$$

The first matching depth is

$$k = 5,$$

where

$$E_5 = \frac{1}{16}.$$

The score has become an error-control signal only because the relation to target error is declared.

37.15 Rank quality versus magnitude semantics

This distinction appears throughout machine learning.

A score can be excellent at:

- ranking examples by difficulty;
- ranking candidate states by quality;
- separating easy from hard inputs;
- predicting which examples deserve more compute.

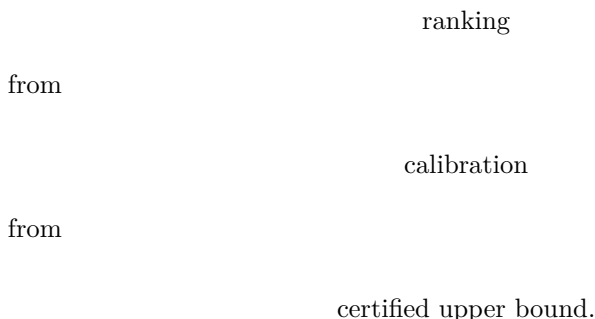
Those are useful capabilities.

They do not automatically answer:

What threshold guarantees error below epsilon?

That question requires magnitude semantics.

The Atlas therefore distinguishes:



37.16 Expected calibration may still be too weak

Suppose a learned halting score satisfies

$$\mathbb{E}[E_k \mid q_k = s] = g(s).$$

That is a meaningful probabilistic calibration statement.

It does not imply

$$E_k \leq g(s)$$

for every individual example.

If the controller requires a hard error guarantee, an expectation is not enough.

The appropriate object might instead be:

- a high-probability bound;
- a conformal upper bound;
- a deterministic residual bound;
- a certified interval;
- a worst-case estimate.

The right guarantee depends on the application.

37.17 Local error is not global error

Numerical error control has its own version of this warning.

Suppose every accepted step satisfies

$$\hat{e}_n \leq \tau_n.$$

Does that immediately prove the final trajectory is within a desired global error?

No.

Global error depends on:

- accumulation;
- stability;
- amplification;
- the mesh sequence;
- the problem dynamics;
- method order;
- regularity.

The NETNUM prerequisite already separated local and global error.

ADAPTDEPTH inherits that boundary intact.

37.18 More depth is not always refinement

Suppose a network runs twice as many layers.

That does not automatically mean the numerical step size was halved.

For a numerical refinement statement, we need a fixed reference problem.

For example, if a horizon

$$T$$

is represented by steps

$$h_0, \dots, h_{N-1},$$

then a fixed-horizon mesh satisfies

$$\sum_{n=0}^{N-1} h_n = T.$$

Increasing N can represent a finer mesh only if the h_n values are changed accordingly.

If the network simply appends more transformations, it may be extending computational time rather than refining the same interval.

This is exactly why Depth and NETNUM are both hard prerequisites.

37.19 Adaptive depth versus adaptive step size

The distinction can be written compactly.

37.19.1 Adaptive execution depth

$$z_{k+1} = F_k(z_k), \quad k < \tau(x).$$

The input chooses how many stages execute.

37.19.2 Adaptive time stepping

$$x_{n+1} = \Psi_{h_n}(x_n),$$

where an error controller changes

$$h_n$$

for a declared reference evolution.

The first changes stage count.

The second changes a numerical mesh.

A system can combine them.

It must declare that combination explicitly.

37.20 A learned error estimator

There is a legitimate bridge between the two worlds.

Suppose a learned module predicts

$$\hat{e}_\theta(z_k)$$

and training data provides a target numerical or task error.

Now the learned score is intended to estimate an error.

But intention is still not guarantee.

We should ask:

- what is the training target?
- what norm?
- what distribution?
- what calibration?
- what tail behavior?
- what happens out of distribution?
- is the estimate biased?
- is an upper bound needed?
- what does rejection do?

The bridge becomes mathematically meaningful only after these questions are answered.

37.21 Error control as an interface contract

A complete adaptive error-control interface should declare:

37.21.1 State

What state is being advanced?

$$x_n \quad \text{or} \quad z_k.$$

37.21.2 Target error

What quantity should be small?

$$E_n.$$

37.21.3 Estimator

What is actually observed?

$$\hat{e}_n.$$

37.21.4 Relation

Why does the estimator say anything about the target error?

Examples:

$$E_n \leq B(\hat{e}_n),$$

or

$$P(E_n \leq B(\hat{e}_n)) \geq 1 - \alpha.$$

37.21.5 Tolerance

What target is requested?

$$\varepsilon.$$

37.21.6 Controller

What action follows?

- stop;
- continue;
- shrink step;
- grow step;
- route elsewhere;
- spend more compute.

37.21.7 Budget

What happens if the criterion never passes?

This interface is the central durable object of the chapter.

37.22 Budget exhaustion remains a separate state

Suppose a system has maximum depth

$$K_{\max}.$$

If the error or halting criterion is not met before that cap, the output may still be returned.

But the termination reason is

`budget_exhausted`.

It is not

`criterion_met`.

This distinction was repaired explicitly in AUDIT-018.

ADAPTDEPTH preserves it.

The same rule applies to numerical solvers.

A solver that runs out of permitted work has not thereby satisfied tolerance.

37.23 Error control is not compute optimality

A step-size controller may meet an accuracy target while using more function evaluations than another method.

A halting policy may reduce FLOPs while increasing error.

A routing scheme may lower average cost while worsening tail latency.

Therefore two objectives remain separate:

error control

and

resource optimization.

They can be combined through a joint objective.

They should not be conflated.

37.24 Spatially adaptive computation

Figurnov et al. show a system in which different image regions receive different computation (Figurnov et al. 2017).

This is a useful reminder that adaptive depth need not be scalar.

We might write a local depth field

$$\tau(i, j).$$

That looks superficially like adaptive mesh refinement.

The resemblance is interesting.

It is not yet a PDE statement.

Adaptive mesh refinement requires:

- a spatial discretization;
- a reference PDE or operator;
- an error indicator/estimator;
- refinement/coarsening semantics;
- consistency across interfaces.

A spatial halting mask supplies none of these automatically.

37.25 Neural error indicators

A useful research direction is to learn an error indicator rather than merely a halting probability.

For example, a model might predict:

$$\hat{e}_k = g_\theta(z_k, z_{k+1}, r_k).$$

The controller could then continue while

$$\hat{e}_k > \varepsilon.$$

But the Atlas would still ask what supports the implication:

$$\hat{e}_k \leq \varepsilon \Rightarrow E_k \leq \varepsilon.$$

Possible answers include:

- exact residual theory;
- calibration on a declared distribution;

- conformal upper bounds;
- interval propagation;
- theorem-backed a posteriori estimates.

The learned component does not erase the proof obligation.

37.26 Residuals are promising but not universal

Numerical methods often use residuals or embedded differences because they have a known mathematical relation to error.

Neural systems also expose residual-like objects:

- state increments;
- reconstruction residuals;
- fixed-point residuals;
- gradient norms;
- disagreement scores.

These can be useful stopping signals.

But the relation

$$\text{small residual} \Rightarrow \text{small target error}$$

is problem dependent.

For an ill-conditioned or non-normal system, a small residual may coexist with a larger solution error.

The error-control interface therefore requires the relation, not just the residual.

37.27 Fixed-point stopping

Suppose an equilibrium computation seeks

$$z^* = F(z^*).$$

A natural residual is

$$r_k = \|F(z_k) - z_k\|.$$

Stopping when

$$r_k \leq \varepsilon$$

is a residual criterion.

To convert it into a state-error guarantee

$$\|z_k - z^*\| \leq C\varepsilon,$$

we need additional structure.

For a contraction with factor $L < 1$, such relations can be derived.

Without them, the residual is not the state error.

This is another instance of the same principle.

37.28 Halting as decision making

A learned halting rule can also be viewed as a decision problem.

At depth k , the system chooses between:

- stop now;
- spend another unit of compute.

The decision trades expected improvement against compute cost.

That is a valid formulation.

It is not numerical error control unless the objective or constraint explicitly references an error quantity with justified semantics.

The Atlas therefore allows adaptive depth to be interpreted in several ways without collapsing them.

37.29 A hierarchy of stopping semantics

From weakest to strongest:

37.29.1 Heuristic stop score

A learned or handcrafted number triggers stopping.

37.29.2 Empirically predictive score

The score correlates with future improvement or error on held-out data.

37.29.3 Calibrated expected error

The score predicts average error under a declared distribution.

37.29.4 High-probability error control

The score yields a probabilistic upper bound with stated coverage.

37.29.5 Deterministic certified error bound

The score or residual implies a mathematical upper bound under declared assumptions.

These levels should not be described with the same language.

37.30 Why embedded pairs matter conceptually

Embedded numerical formulas are useful beyond ODE software.

They illustrate a design pattern for adaptive intelligence:

create two computations whose discrepancy carries information about whether more computation is warranted.

The pattern might reappear as:

- shallow versus deep prediction disagreement;
- coarse versus refined representation;
- low-rank versus expanded solve;
- cheap versus expensive model;
- one-step versus multi-step estimate.

But the discrepancy becomes error control only when its relation to target error is justified.

The numerical example teaches the grammar.

It does not certify every neural analogue.

37.31 Error estimators can fail outside their regime

Numerical estimators themselves have domains of validity.

A step controller can struggle when:

- the problem is stiff;
- smoothness assumptions fail;
- the step is outside the asymptotic regime;
- the estimator suffers cancellation;
- the chosen norm misses a relevant component.

This matters for neural analogies.

Even a mathematically motivated error signal has to be checked against the regime where its guarantee was derived.

37.32 Adaptive computation and distribution shift

A learned halting model may be calibrated on one input distribution.

Deployment may shift.

Then the relation between score and error can change.

This parallels the Uncertainty chapter’s warning:

$$P\text{-calibration} \not\Rightarrow Q\text{-calibration}.$$

ADAPTDEPTH therefore treats learned error control as distribution-relative unless a stronger robust guarantee is established.

37.33 Acceptance and rejection should be explicit

A controller is incomplete if it only says “adapt.”

For each proposed computation, one should know:

- what was measured;
- whether the criterion passed;
- whether the state was accepted;
- what changed after rejection;
- whether computation was retried;
- whether the budget was consumed;
- whether the final output met tolerance.

This transaction-like view aligns numerical adaptation with the Atlas’s broader emphasis on explicit state and evidence.

37.34 A minimal controller grammar

A generic adaptive controller can be written:

1. propose an update;
2. compute an indicator;
3. map indicator to target error semantics;
4. compare against tolerance;
5. accept or reject;

6. change compute allocation;
7. stop on criterion or budget;
8. record the termination reason.

This grammar covers:

- numerical step-size control;
- recurrent halting;
- iterative solver stopping;
- conditional depth;
- learned refinement.

The semantics differ at step 3.

That is where the important mathematics lives.

37.35 Failure modes

The recurring category errors are now concrete.

37.35.1 Halting score as error

A learned number is thresholded and then described as tolerance satisfaction without calibration.

37.35.2 Rank as magnitude

A score orders easy and hard cases correctly, so its numerical value is treated as an error bound.

37.35.3 Local as global

A local estimator is small, so final accumulated error is claimed small without stability analysis.

37.35.4 Depth as mesh

More layers are described as finer time discretization without a fixed horizon and step-scale relation.

37.35.5 Budget as convergence

The system stops at $K_{\max}$ and the output is labeled converged.

37.35.6 Spatial depth as adaptive mesh refinement

A learned spatial computation mask is described with PDE refinement language without a declared discretized PDE.

37.35.7 Error control as optimal compute

Meeting tolerance is confused with using the minimum possible resources.

37.36 Research programme

The synthesis suggests several precise research targets.

37.36.1 Learned a posteriori estimators

Train a model to estimate a mathematically defined error and then test calibration and bounds.

37.36.2 Two-resolution neural computation

Construct cheap/high-resolution pairs whose discrepancy predicts the value of more computation.

37.36.3 Conformal stopping

Use distribution-free calibration machinery to turn empirical error predictors into high-probability stopping rules under exchangeability assumptions.

37.36.4 Robust halting under shift

Test whether error/halting calibration survives changed data laws.

37.36.5 Resource-aware error control

Optimize compute subject to a declared error constraint instead of mixing accuracy and cost in one opaque score.

Each target requires its own proof or evidence layer.

37.37 Closing view

Adaptive depth is most mathematically useful when it is treated as a control problem rather than a slogan.

The system evolves a state.

It observes an indicator.

The indicator has declared semantics.

A controller compares that indicator with a target.

The system spends more or less computation accordingly.

The durable synthesis is:

adaptive error control = adaptive computation + target error + estimator/error relation + tolerance + c

Remove the estimator/error relation and we still have adaptive computation.

We no longer have justified error control.

37.38 References used in this chapter

- Graves, Adaptive Computation Time for Recurrent Neural Networks (Graves 2016).
- Figurnov et al., Spatially Adaptive Computation Time for Residual Networks (Figurnov et al. 2017).
- Dormand and Prince, A family of embedded Runge-Kutta formulae (Dormand and Prince 1980).

Exact provenance and claim boundaries are locked in:

`sources/source-locks/ATLAS-CH-ADAPTDEPTH-001.yaml`

Chapter 38

Neural Krylov Transport

Epistemic status: audited classical Krylov/numerical-linear-algebra substrate + audited representation-transport substrate + Atlas-owned exact finite local-solve witnesses.

Specification: manuscript/specifications/ATLAS-CH-NEURALKRYLOV-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-NEURALKRYLOV-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-NEURALKRYLOV-001.md

Source lock: sources/source-locks/ATLAS-CH-NEURALKRYLOV-001.yaml

A deep model already transports representations through a sequence of maps.

Sometimes a useful local computation can be posed as a linear correction problem:

$$A_z \delta_z = b_z.$$

That raises a natural question:

can a model use a few operator-generated directions to compute a better representation correction without forming or solving a large dense system?

Classical Krylov methods provide one mathematical language for that question.

But the analogy is useful only if its boundaries remain visible.

The governing rule is:

declare the linear operator first; inherit no nonlinear convergence theorem by analogy.

38.1 What comes from classical Krylov theory

Krylov Subspaces and Iterative Solves already supplies:

- operator-generated trial spaces;
- Arnoldi/Lanczos vocabulary;
- projection and residual criteria;
- conditioning boundaries;
- left/right preconditioning semantics;
- matrix-free operator access;
- restart and finite-precision cautions.

Those objects remain classical linear algebra.

NEURALKRYLOV does not redefine them.

38.2 What comes from representation transport

Representation as Transport already gives a separate interface:

$$z_{k+1} = \Phi_k(z_k).$$

A representation is a state.

A stage is a declared map.

A local Jacobian can describe differential behavior near a base point.

But the Jacobian is not the global nonlinear map.

NEURALKRYLOV joins these two languages only through a declared local linear problem.

38.3 The local-solve interface

At state z , suppose we define:

$$A_z \delta_z = b_z.$$

The operator A_z might be:

- a local Jacobian-derived map;
- a normal-equation operator;
- a Hessian-like approximation;
- a learned linear surrogate;
- another explicitly declared linear operator.

Nothing in the phrase “Neural Krylov” identifies A_z automatically.

The operator must be named.

38.4 The representation update is separate

Once an approximate correction:

$$\delta_z^{(m)}$$

has been computed, a representation block might apply:

$$z^+ = \Phi(z, \delta_z^{(m)}).$$

The local solve and the representation update are different maps.

A good local solve does not automatically imply:

- a good representation;
- a stable global trajectory;
- a better task score.

Those are separate tests.

38.5 A fixed left-preconditioned exact witness

Use the local system:

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

The exact solution is:

$$x^\star = \begin{pmatrix} 1 \\ 1/4 \end{pmatrix}.$$

Introduce fixed left preconditioner:

$$M = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}.$$

Then:

$$M^{-1}A = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix},$$

and:

$$M^{-1}b = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}.$$

Write:

$$B = M^{-1}A, \quad c = M^{-1}b.$$

38.6 Why the preconditioner matters

The Krylov process now sees:

$$Bx = c.$$

That is not merely “the same system with faster convergence.”

It is a transformed system with a transformed residual.

The original residual remains:

$$r = b - Ax.$$

The transformed residual is:

$$\hat{r} = c - Bx = M^{-1}r.$$

These are different numerical objects.

38.7 One operator-generated direction

With:

$$x_0 = 0,$$

the first Krylov space is:

$$\mathcal{K}_1(B, c) = \text{span}\{c\}.$$

So every candidate has form:

$$x = \alpha c.$$

The best transformed-residual coefficient is:

$$\alpha = \frac{3}{4}.$$

Hence:

$$x_1 = \begin{pmatrix} 3/4 \\ 3/8 \end{pmatrix}.$$

38.8 The transformed residual is nonzero

For x_1 :

$$\hat{r}_1 = \begin{pmatrix} 1/4 \\ -1/4 \end{pmatrix}.$$

Thus:

$$\|\hat{r}_1\|_2^2 = \frac{1}{8}.$$

The best one-dimensional trial-space point is still not the exact solve.

38.9 The original residual is different

The original-system residual is:

$$r_1 = \begin{pmatrix} 1/4 \\ -1/2 \end{pmatrix}.$$

Thus:

$$\|r_1\|_2^2 = \frac{5}{16}.$$

The same iterate has two different residual norms depending on which system we evaluate.

This is not a contradiction.

It is preconditioning semantics.

38.10 The true error is another object

The true error is:

$$e_1 = x^\star - x_1 = \begin{pmatrix} 1/4 \\ -1/8 \end{pmatrix}.$$

Hence:

$$\|e_1\|_2^2 = \frac{5}{64}.$$

Now we already have three different quantities:

$$\frac{1}{8}, \quad \frac{5}{16}, \quad \frac{5}{64}.$$

They answer different questions.

38.11 One more direction changes everything in this toy

Compute:

$$Bc = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

The two generated directions are:

$$c = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}, \quad Bc = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Their determinant is:

$$\frac{1}{2}.$$

Therefore:

$$\mathcal{K}_2(B, c) = \mathbb{R}^2.$$

38.12 The exact solution is in the second Krylov space

Indeed:

$$x^* = \frac{3}{2}c - \frac{1}{2}Bc.$$

So an exact residual-minimizing solve in $\mathcal{K}_2$ can recover:

$$x^*.$$

Then:

$$\hat{r}_2 = 0, \quad r_2 = 0, \quad e_2 = 0.$$

38.13 What the positive witness actually shows

It shows one finite fact:

for this declared operator, right-hand side, and preconditioner, one generated direction is insufficient while two are sufficient.

That is a useful architecture motif.

It is not:

- a rate theorem;
- a dimension-independent guarantee;
- evidence that every neural representation has a two-step Krylov structure.

38.14 Short horizon can be computationally meaningful

The representation-computation idea is not necessarily to solve a large system to machine precision.

It may instead be:

- generate a few directions;
- project a local problem;
- obtain a useful correction;
- stop because compute is bounded.

This makes horizon m an architectural resource.

But “small m ” is not itself a quality certificate.

38.15 A low-dimensional failure control

Consider:

$$B_{\text{bad}} = \begin{pmatrix} 1 & 0 \\ 0 & 10 \end{pmatrix}, \quad c_{\text{bad}} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Again:

$$\mathcal{K}_1 = \text{span}\{c_{\text{bad}}\}.$$

The trial space is one-dimensional.

38.16 The best one-dimensional residual can still be poor

The exact minimum-residual scalar is:

$$\alpha_{\text{bad}} = \frac{11}{101}.$$

The residual becomes:

$$r_{\text{bad}} = \begin{pmatrix} 90/101 \\ -9/101 \end{pmatrix}.$$

Therefore:

$$\|r_{\text{bad}}\|_2^2 = \frac{81}{101}.$$

That is large.

38.17 Dimension is not convergence

Both positive and bad controls use a one-dimensional first trial space.

One gives a moderate approximation.

The other leaves a large residual.

Thus:

$$\text{small trial-space dimension} \not\Rightarrow \text{small residual}.$$

The operator geometry matters.

38.18 Conditioning and spectral structure still matter

KRYLOV already established that convergence can depend on:

- spectrum;
- field of values;
- non-normality;
- right-hand side;
- polynomial approximation;
- preconditioning;
- finite precision.

A neural implementation does not make those dependencies disappear.

It may change the operator.

Then the analysis must change with it.

38.19 Task quality can disagree with solve quality

Now define downstream readout:

$$w = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

For the exact solution:

$$w^\top x^\star = \frac{3}{2}.$$

For the non-exact one-step iterate:

$$w^\top x_1 = \frac{3}{2}.$$

So:

$$\boxed{w^\top x_1 = w^\top x^\star.}$$

38.20 Yet the one-step solve is still wrong

At the same time:

$$r_1 \neq 0,$$

and:

$$e_1 \neq 0.$$

So the downstream scalar is exact even though the local solve is not.

Therefore:

exact task readout $\nrightarrow$ exact solve.
--

This is a minimal but important metric firewall.

38.21 The converse also needs proof

A smaller residual does not automatically imply a better task metric.

A downstream task may:

- ignore the corrected direction;
- amplify a tiny error direction;
- saturate;
- threshold;
- use a nonlinear readout.

Therefore solve quality and task quality must be evaluated separately.

38.22 Representation quality is another layer

Suppose x becomes a representation correction.

One can measure:

- Euclidean correction error;
- angular error;
- constraint violation;
- retained information;
- probe separability;
- downstream task score.

None equals:

$$\|b - Ax\|$$

by definition.

A chapter that reports only residual has not yet measured representation quality.

38.23 The local Jacobian bridge

Suppose a representation map is:

$$z^+ = \Phi(z).$$

At base point z_0 , a local differential object is:

$$J_\Phi(z_0).$$

A local correction operator might be declared as:

$$A_{z_0} = I - J_\Phi(z_0).$$

Then a Krylov method can be built for A_{z_0} .

But that is a Krylov problem for a local linear operator.

It is not a Krylov method for the entire nonlinear Φ .

38.24 Local Jacobian is not global transport

If the representation moves to z_1 , then:

$$J_\Phi(z_1)$$

can differ from:

$$J_\Phi(z_0).$$

If the operator changes at each stage, a sequence of generated directions is not automatically:

$$r, Ar, A^2r, \dots$$

for one fixed A .

The classical object has changed.

38.25 Learned preconditioners make the distinction sharper

Suppose:

$$M = M_\theta(z).$$

Then the preconditioner may depend on:

- state;
- layer;
- input;
- training parameters.

A learned preconditioner can be a useful architecture.

But it is not automatically the fixed linear M of classical theory.

Any guarantee must match the actual semantics.

38.26 Fixed left versus right versus flexible

The exact witness is fixed left preconditioning:

$$M^{-1}Ax = M^{-1}b.$$

Right preconditioning would instead change variables.

Flexible preconditioning can use different preconditioners across iterations.

Nonlinear preconditioning changes the mathematical problem further.

The word “preconditioned” is insufficient without the type.

38.27 Matrix-free access fits the architecture

A neural system may never materialize:

$$A_z.$$

It may expose only:

$$v \mapsto A_z v.$$

For Jacobian-derived operators, this can be implemented by JVP/VJP-style primitives.

That is compatible with classical operator-action semantics.

But the operator is still conceptually defined.

38.28 Matrix-free does not mean operator-free

The fact that a matrix is not explicitly stored does not license an undefined operator.

For every Krylov claim, one should be able to answer:

what map is being applied to the vector?

Without that answer, the subspace claim is underspecified.

38.29 Residual and error need conditioning

For an invertible linear problem:

$$Ae = r.$$

Therefore:

$$e = A^{-1}r.$$

Bounds such as:

$$\|e\| \leq \|A^{-1}\| \|r\|$$

depend on the operator.

Residual is observable from A, b, x .

True error requires the exact solution or a certificate.

They are not interchangeable.

38.30 Preconditioned residual adds another layer

Under left preconditioning:

$$\hat{r} = M^{-1}r.$$

A solver can minimize:

$$\|\hat{r}\|$$

while the application ultimately cares about:

$$\|r\|.$$

Sometimes this is exactly the intended geometry.

Sometimes it is not.

The distinction should be explicit.

38.31 Finite precision still exists inside neural systems

If a block explicitly constructs orthogonal or nearly orthogonal operator-generated directions, finite precision matters.

Possible failures include:

- loss of orthogonality;
- direction collapse;
- residual-estimate drift;
- numerical dependence.

A learned implementation is still numerical computation.

38.32 Restarting and truncation are architectural choices

A bounded-compute Neural Krylov block may stop after m directions.

It may also restart.

This controls:

- memory;
- latency;

- operator calls.

But it can throw away useful subspace information.

The exact positive witness already shows:

$$m = 1$$

misses the exact solution, while:

$$m = 2$$

captures it.

38.33 A Neural Krylov block as an interface

A bounded block can be specified as:

1. receive z ;
2. define A_z ;
3. define b_z ;
4. define preconditioner M_z , if used;
5. generate at most m directions;
6. solve or project in the trial space;
7. return correction $\delta_z^{(m)}$;
8. update representation;
9. expose diagnostics.

This is a precise architecture description.

38.34 Diagnostics should remain plural

A serious implementation should report, where available:

- transformed residual;
- original residual;
- estimated/known solve error;
- representation change;
- constraint violation;
- downstream task metric;
- operator/preconditioner condition diagnostics.

One scalar should not impersonate all of them.

38.35 The right empirical question

The most useful empirical question is not:

does this architecture “look like Krylov”?

It is:

for the exact operator and preconditioner declared by the architecture,
what does the short generated subspace buy us relative to a matched
compute baseline?

That question can be falsified.

38.36 The right theoretical question

For a learned architecture, the theoretical burden is:

- identify the operator family;
- identify whether it is fixed or state-dependent;
- identify the projection criterion;
- identify the preconditioner class;
- control nonlinearity/state drift;
- state finite-precision assumptions;
- connect solve quality to representation/task quality only with an explicit theorem.

Without these, classical convergence is only an analogy.

38.37 Non-implications

NEURALKRYLOV rejects:

low subspace dimension $\nRightarrow$ small residual,

small residual $\nRightarrow$ small true error without conditioning information,

small solve error $\nRightarrow$ good representation/task quality,

exact task readout $\nRightarrow$ exact solve,

local Jacobian Krylov step $\nRightarrow$ global nonlinear convergence,

representation transport $\nRightarrow$ Krylov convergence theorem,

and:

learned preconditioner $\nRightarrow$ fixed-linear preconditioner guarantees.

38.38 Atlas connections

Krylov Subspaces and Iterative Solves.

Supplies the exact linear-algebra vocabulary and limits.

Representation as Transport.

Supplies the state-map/local-Jacobian interface while blocking ODE/geodesic/Krylov overclaims.

Adaptive depth.

A future controller can treat horizon m as adaptive compute, but must justify its stopping signal.

Boundary probes.

JVP/VJP and sensitivity machinery can expose matrix-free local operator actions.

Systems.

Short-horizon iterative representation computation creates explicit operator-call, memory, and latency tradeoffs.

38.39 Closing view

Neural Krylov Transport is not the claim that neural networks secretly run GMRES.

It is a disciplined architecture pattern:

declared local operator + few operator-generated directions + declared projection + declared representation update.

The finite witness shows why this can matter.

One direction leaves nonzero residual and error.

Two directions recover the exact local solution.

A separate one-dimensional control remains poor.

And an exact downstream readout can coexist with a non-exact solve.

So the durable lesson is not “Krylov always works.”

It is:

short-horizon operator-generated computation can be useful, but every guarantee is operator-, metric-,

38.40 References used in this chapter

No new external academic authority is added.

Classical numerical-linear-algebra authority is inherited through audited ATLAS-CH-KRYLOV-001.

Representation-transport authority is inherited through audited ATLAS-CH-TRANSPORT-001.

Exact prerequisite identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-NEURALKRYLOV-001.yaml`

Chapter 39

Boundary Probes

Epistemic status: established linear algebra + established algorithmic differentiation + audited numerical-network sensitivity substrate + Atlas synthesis + exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-BOUNDARYPROBE-001.md

Formal packet: mathematics/derivations/ATLAS-CH-BOUNDARYPROBE-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-BOUNDARYPROBE-001.md

Source lock: sources/source-locks/ATLAS-CH-BOUNDARYPROBE-001.yaml

A learned component can be shape-compatible with its neighbor and still compose poorly.

One component may amplify a direction the next component cannot tolerate. A downstream objective may be sensitive to one interface coordinate. A finite probe may look benign because it did not explore the dominant singular direction. A local derivative can look small while the nonlinear map changes behavior farther away.

The governing rule is:

a boundary probe is evidence about a declared local interface question,
not a global certificate of the component or system.

39.1 Boundary object

Let a component be

$F: X \rightarrow Y$.

A later component may consume it:

$G:Y \rightarrow Z$.

At the interface, several obligations can coexist:

- type and shape compatibility;
- numerical sensitivity;
- semantic compatibility;
- units and conventions;
- provenance;
- downstream task adequacy.

This chapter develops the numerical/local-sensitivity part only.

39.2 Local linearization

Let

$F:\mathbb{R}^n \rightarrow \mathbb{R}^m$

be differentiable at operating point x .

Write:

$J = J_F(x)$.

Then:

$F(x+\delta) = F(x) + J \delta + r(\delta)$

with

$\|r(\delta)\| / \|\delta\| \rightarrow 0$

as $\delta \rightarrow 0$.

LINALG-001 already established the key boundary: a Jacobian is local.

39.3 Why derivative products matter

The full Jacobian can be too large to materialize.

Many interface questions require only products:

- response to one input perturbation;
- pullback of one output objective;
- dominant local Euclidean gain;
- sensitivity of a selected interface mode.

This motivates JVPs and VJP.

39.4 JVP

For tangent direction

$\mathbf{v}$ in $\mathbb{R}^n$,

define:

$\text{JVP_F}(\mathbf{x}; \mathbf{v}) = \mathbf{J_F}(\mathbf{x}) \mathbf{v}$.

Equivalently:

$\mathbf{J_F}(\mathbf{x}) \mathbf{v} = \lim_{\epsilon \rightarrow 0} \frac{F(\mathbf{x} + \epsilon \mathbf{v}) - F(\mathbf{x})}{\epsilon}$.

A JVP pushes a local input perturbation forward.

Forward-mode algorithmic differentiation computes this product through the computational graph without requiring a dense Jacobian (Griewank and Walther 2008; Baydin et al. 2018).

39.5 JVP versus finite differences

The finite difference

$\frac{F(\mathbf{x} + \epsilon \mathbf{v}) - F(\mathbf{x})}{\epsilon}$

can approximate the JVP.

It is not the same computational object.

Finite differences depend on a chosen step and carry truncation and roundoff effects. Algorithmic differentiation propagates derivative rules through the represented computation, subject to ordinary numerical arithmetic error.

39.6 VJP

Under the declared Euclidean coordinate convention, represent the output covector by

$\mathbf{w}$ in $\mathbb{R}^m$.

Use:

$\text{VJP_F}(\mathbf{x}; \mathbf{w}) = \mathbf{J_F}(\mathbf{x})^T \mathbf{w}$.

If:

$\ell(\mathbf{x}) = \mathbf{w}^T \mathbf{F}(\mathbf{x})$,

then:

$\text{grad}_{\mathbf{x}} \ell(\mathbf{x}) = \mathbf{J_F}(\mathbf{x})^T \mathbf{w}$.

Reverse-mode algorithmic differentiation computes this pullback (Griewank and Walther 2008; Baydin et al. 2018).

39.7 Pairing identity

For compatible $\mathbf{v}$ and $\mathbf{w}$:

$$\mathbf{w}^T(\mathbf{J}\mathbf{v}) = (\mathbf{J}^T \mathbf{w})^T \mathbf{v}.$$

This identity gives a compact consistency check between independently implemented JVP and VJP routines.

Agreement is useful evidence.

It is not a proof of complete derivative correctness.

39.8 Chain rule at an interface

For:

$$\mathbf{x} \xrightarrow{\mathbf{F}} \mathbf{y} \xrightarrow{\mathbf{G}} \mathbf{z},$$

the local Jacobian is:

$$\mathbf{J}_{(\mathbf{G} \circ \mathbf{F})}(\mathbf{x}) = \mathbf{J}_{\mathbf{G}}(\mathbf{F}(\mathbf{x})) \mathbf{J}_{\mathbf{F}}(\mathbf{x}).$$

A JVP propagates forward through this product.

A VJP propagates backward through the transposed factors.

The products follow the chain rule without requiring all intermediate Jacobians to be stored explicitly.

39.9 Local Euclidean gain

LINALG-001 established:

$$\|\mathbf{J}\|_2 = \sigma_{\max}(\mathbf{J}).$$

Therefore:

$$\|\mathbf{J} \delta\|_2 \leq \sigma_{\max}(\mathbf{J}) \|\delta\|_2.$$

The largest singular value is the largest first-order Euclidean amplification factor at the operating point.

39.10 Local does not mean global

The nonlinear map obeys:

$$\mathbf{F}(\mathbf{x} + \delta) - \mathbf{F}(\mathbf{x}) = \mathbf{J} \delta + \mathbf{r}(\delta).$$

So a known local operator norm controls the linear term.

A global Lipschitz claim needs additional evidence, such as a derivative bound over a region or another nonlinear argument.

39.11 Power iteration as a probe

Define:

$$\mathbf{A} = \mathbf{J}^T \mathbf{J}.$$

Its eigenvalues are:

$$\sigma_i(\mathbf{J})^2.$$

Power iteration applies:

$$u_{k+1} = Az_k, \quad z_{k+1} = \frac{u_{k+1}}{\|u_{k+1}\|_2},$$

only when $\|Az_k\|_2 > 0$. If $Az_k = 0$, the normalized step is undefined and a probe must report that breakdown rather than silently divide by zero. For example, $A = \text{diag}(9, 0)$ and $z_0 = (0, 1)^T$ give $Az_0 = 0$, even though the true dominant singular value of the corresponding $J = \text{diag}(3, 0)$ is 3. A restart with a different direction may be needed.

For a nonzero probe direction, the Rayleigh quotient

$$\rho_k = \frac{z_k^T Az_k}{z_k^T z_k}$$

can approach the dominant eigenvalue under the stated spectral and initialization conditions.

Then:

$$\text{sqrt}(\rho_k)$$

approaches the dominant singular value.

39.12 JVP and VJP implement the power operator

One application of $\mathbf{A}$ is:

$$\mathbf{A} \mathbf{z} = \mathbf{J}^T(\mathbf{J}\mathbf{z}).$$

Thus it can be evaluated as:

1. JVP: $\mathbf{u} = \mathbf{J}\mathbf{z}$;

2. VJP: $J^T u$.

This supports local spectral probing without explicit Jacobian materialization.

39.13 Estimator and quantity are different objects

$\text{sigma_max}(J)$ is a mathematical quantity.

Power iteration is an estimator procedure.

A finite run can fail to recover the dominant singular direction.

Therefore the procedure that generated an estimate is part of the evidence.

39.14 Initialization matters

Let eigenpairs of A be:

$\lambda_1 > \lambda_2 \geq \dots$

with eigenvectors q_i .

Write:

$z_0 = \sum_i c_i q_i$.

Then:

$A^k z_0 = \sum_i c_i \lambda_i^k q_i$.

If c_1 is nonzero, the dominant component can eventually control the normalized iterate.

If $c_1=0$ exactly, ordinary power iteration cannot create the missing component.

39.15 Finite Rayleigh estimates can be low

For positive semidefinite A :

$\rho(z) = (z^T A z) / (z^T z) \leq \lambda_{\max}(A)$.

Hence:

$\sqrt{\rho(z)} \leq \text{sigma_max}(J)$.

A finite Rayleigh estimate is not automatically a certified upper bound on the true local operator norm.

39.16 Exact witness map

Use:

$$F(x_1, x_2) = (x_1^2 + x_2, x_1 + 2x_2).$$

At:

$$x_0 = (1, 1),$$

$$F(x_0) = (2, 3).$$

The Jacobian is:

$$J = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

39.17 Exact singular structure

The eigenvectors of J are:

$$q_1 = (1, 1)/\sqrt{2}$$

and

$$q_2 = (1, -1)/\sqrt{2}$$

with eigenvalues:

3 and 1.

The singular values are therefore:

3, 1.

Thus:

$$\|J\|_2 = 3.$$

39.18 Exact JVP and nonlinear remainder

Choose:

$$v = (1, 1).$$

Then:

$$Jv = (3, 3)$$

and the Euclidean gain is exactly 3.

For scalar ϵ :

$$F(x_0 + \epsilon v) - F(x_0) = \epsilon (3, 3) + (\epsilon^2, 0).$$

The JVP is the exact first-order term. The quadratic remainder makes the local/global distinction explicit.

39.19 Exact VJP

Choose:

$$\mathbf{w} = (1, 2).$$

Then:

$$\mathbf{J}^T \mathbf{w} = (4, 5).$$

For:

$$\text{ell}(\mathbf{x}) = \mathbf{w}^T \mathbf{F}(\mathbf{x}),$$

the gradient at $\mathbf{x}_0$ is exactly:

$$(4, 5).$$

The pairing check gives:

$$\mathbf{w}^T (\mathbf{J}\mathbf{v}) = 9 = (\mathbf{J}^T \mathbf{w})^T \mathbf{v}.$$

39.20 Exact power matrix

For the witness:

$$\mathbf{A} = \mathbf{J}^T \mathbf{J} = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}.$$

Its eigenvalues are:

$$9, 1.$$

The exact dominant singular target is therefore:

$$\sqrt{9} = 3.$$

39.21 A mixed start converges

Start with:

$$\mathbf{z}_0 = (1, 0).$$

One multiplication gives:

$$\mathbf{u}_1 = (5, 4).$$

The next gives:

$$\mathbf{u}_2 = (41, 40).$$

The corresponding Rayleigh quotients are:

$$\rho_1 = 365/41$$

and

$\rho_2=29525/3281$.

Both approach the exact dominant eigenvalue 9 from below.

Their square roots approach the true singular norm 3.

39.22 The convergence has a closed form

Because:

$z_0=(q_1+q_2)/\sqrt{2}$,

the unnormalized k th iterate is:

$A^k z_0 = ((9^{k+1})/2, (9^{k-1})/2)$.

Its Rayleigh quotient is:

$\rho_k = (9 \cdot 81^k + 1)/(81^k + 1)$.

Hence:

$\rho_k \rightarrow 9$.

This exact sequence makes the estimator dynamics visible without numerical ambiguity.

39.23 A different start misses the dominant direction

Now choose:

$z_{\text{bad}}=(1,-1)$.

Then:

$A z_{\text{bad}}=z_{\text{bad}}$.

Every iterate remains in the weak eigenspace.

The Rayleigh quotient is always:

1.

The inferred singular value is always:

1.

The true operator norm remains:

3.

Nothing about the component changed.

Only the probe initialization changed.

39.24 One run is not a certificate

Suppose a monitoring rule records whether the reported local gain is below 2.

The weak-eigenspace run reports 1, while the true local norm is 3.

Therefore that finite estimate alone cannot establish the stronger condition $||J||_2 \leq 2$.

The lesson is not that power iteration is unusable.

The lesson is that estimator metadata and limitations belong to the result.

39.25 Restarts and subspace methods

Random starts reduce the risk of exact orthogonality under ordinary continuous sampling assumptions.

Multiple restarts improve directional coverage.

Subspace iteration can track several modes.

These are useful engineering choices.

They do not turn a local finite estimate into a global nonlinear theorem.

39.26 Probe metadata

A reproducible spectral or sensitivity probe should record at least:

- exact component identity;
- operating point or sampled region;
- input/output norm convention;
- tangent or cotangent initialization;
- random seed;
- iteration count;
- restart count;
- normalization rule;
- stopping criterion;
- Rayleigh or residual trace;
- numerical precision;
- runtime/software environment.

A scalar estimate without this context is weak evidence.

39.27 Boundary-probe contract

Use:

$B=(F,X,Y,O,U,N_X,N_Y,P,E,\tau)$.

Here:

- F is the component map;
- X, Y are typed input/output spaces;
- O is the operating point or region;
- U is the perturbation or tangent class;
- N_X, N_Y are norm or inner-product conventions;
- P is the probe procedure;
- E is estimator evidence and metadata;
- τ is an acceptance threshold or policy.

This is the numerical-probe portion of a later full boundary contract.

39.28 The perturbation class matters

A full Euclidean operator norm ranges over every Euclidean direction.

A real interface may allow only structured perturbations.

Examples include:

- tangent directions on a constrained manifold;
- physically admissible perturbations;
- data-supported directions;
- parameter-induced variations;
- a full norm ball.

A sensitivity statement should name which class is being probed.

39.29 Norm choice matters

A gain measured in Euclidean norm need not have the same value in another norm.

Interfaces may care about:

- Euclidean energy;
- maximum coordinate error;
- weighted physical units;
- probability geometry;
- task-specific seminorms.

The geometry belongs to the contract.

39.30 Units precede numerical norms

If different coordinates represent incompatible physical units, a raw Euclidean norm can be meaningless.

Unit conversion or scaling conventions must be established before the numerical gain is interpreted.

This is not something the Jacobian probe can infer.

It is a semantic interface obligation.

39.31 Relation to NETNUM sensitivity

NETNUM-001 established that residual-step Jacobians propagate first-order perturbations locally.

For:

$$\mathbf{Psi}(\mathbf{x}) = \mathbf{x} + \mathbf{F}(\mathbf{x}),$$

the exact local Jacobian is:

$$\mathbf{J}_{\mathbf{Psi}} = \mathbf{I} + \mathbf{J}_{\mathbf{F}}.$$

Boundary probes turn that local derivative into an inspectable interface diagnostic.

The NETNUM boundary remains active:

local perturbation propagation is not a complete global stability theorem.

39.32 Composite interfaces

For two local Jacobians $\mathbf{J}_1$ and $\mathbf{J}_2$:

$$\mathbf{J}_{\text{composite}} = \mathbf{J}_2 \mathbf{J}_1.$$

The submultiplicative norm bound gives:

$$\|\mathbf{J}_2 \mathbf{J}_1\|_2 \leq \|\mathbf{J}_2\|_2 \|\mathbf{J}_1\|_2.$$

This can expose possible amplification chains.

The bound can be loose.

The product of individual worst-case gains need not equal the actual composite gain.

39.33 Small sensitivity is not semantic adequacy

Consider a component that maps every input to zero.

Its Jacobian norm is zero.

Numerically it is insensitive.

Semantically it may destroy all information needed downstream.

Therefore:

low sensitivity is not automatically good composition.

Numerical compatibility and semantic adequacy are separate axes.

39.34 Large sensitivity is not automatically failure

A component may intentionally amplify a weak signal.

If the downstream consumer expects that scale, a large singular value can be acceptable.

The singular value is a diagnostic quantity.

Whether it violates a contract depends on:

- perturbation class;
- units;
- downstream tolerance;
- application policy.

The threshold is not produced by the Jacobian itself.

39.35 VJP magnitude is not causal attribution

A large coordinate in:

$\mathbf{J}^T \mathbf{w}$

means the chosen scalarized output is locally sensitive to that input coordinate under the derivative model.

It does not by itself establish causal influence under interventions.

Sensitivity and causal explanation are different objects.

39.36 Probe ensembles

A richer boundary profile may contain:

- several JVP directions;
- several VJP objectives;
- one or more singular-value estimates;
- directional gains;
- local invariant directions;
- estimator residuals.

This can be more informative than a single scalar.

Each statistic still needs an explicit interpretation.

39.37 Probe freshness

Learned components change.

Fine-tuning, quantization, pruning, compilation, or kernel changes can alter boundary behavior.

A probe result should therefore bind to the exact component artifact and execution configuration.

API compatibility does not make an old probe current evidence for a changed component.

39.38 Operating-region coverage

A derivative can be exactly computed at every sampled point while the sampled region fails to represent later inputs.

This separates two questions:

- derivative/probe estimation;
- operating-region coverage.

Power iteration addresses only the first, and only locally.

39.39 Acceptance thresholds are policy

Suppose the measured gain is 1.8 and the threshold is 2.

The mathematics can establish the measurement and its procedure.

It cannot determine whether 2 is acceptable for the application.

Threshold selection depends on downstream tolerance, uncertainty margins, model role, and governance.

39.40 Numerical versus semantic conditions

A numerical boundary probe can answer questions such as:

- which tangent directions amplify?
- which input directions affect a chosen output objective?
- what local singular gain is observed?
- how stable is the spectral estimate across restarts?

It cannot by itself answer:

- do physical units agree?
- does the output mean what the consumer assumes?
- is provenance acceptable?
- are business, legal, or scientific invariants preserved?
- is the full nonlinear composition globally safe?

Those are separate contract dimensions.

39.41 Failure modes

39.41.1 Local-to-global promotion

A Jacobian norm at one point is presented as global stability.

39.41.2 Estimate-to-certificate promotion

A finite power estimate is presented as a proved upper bound.

39.41.3 Initialization erasure

A spectral estimate is reported without its start or restart procedure.

39.41.4 Norm erasure

A gain is reported without the geometry in which it was measured.

39.41.5 Semantic laundering

Low numerical sensitivity is treated as proof of interface meaning.

39.41.6 VJP-as-causality

A derivative pullback is presented as causal explanation.

39.41.7 Stale evidence

A probe from an earlier component version is reused after the component changes.

39.42 Practical probe ledger

Field	Question
component	Which exact artifact/version?
boundary	Which input and output spaces?

Field	Question
operating region	At which point(s) or data region?
perturbations	Which directions are admissible?
norms	Which input/output geometry?
JVP	Which tangent directions were pushed forward?
VJP	Which output covectors were pulled back?
spectral probe	Which power/subspace procedure?
initialization	Which seeds/vectors/restarts?
iterations	How many and with what stop rule?
estimate	Which Rayleigh/singular values were observed?
limitation	What estimator or coverage uncertainty remains?
threshold	Which policy determines acceptance?
semantic contract	Which non-numerical obligations remain separate?

This ledger converts **stable interface** into an inspectable claim.

39.43 What the witness establishes

The companion witness proves:

- exact Jacobian $[[2,1],[1,2]]$;
- exact singular values $3,1$;
- JVP $(3,3)$ for $\mathbf{v}=(1,1)$;
- VJP $(4,5)$ for $\mathbf{w}=(1,2)$;
- exact JVP/VJP pairing value 9;
- nonlinear remainder ($\epsilon^2,0$) along the chosen tangent;
- mixed-start Rayleigh values $365/41$ and $29525/3281$ approaching 9;
- weak-eigenspace initialization returning Rayleigh value 1 forever despite true singular norm 3.

No global nonlinear theorem is inferred from these finite facts.

39.44 Downstream handoff

Boundary Contracts — **ATLAS-CH-BCONTRACT-001** may now assume:

- JVP semantics;
- VJP semantics;
- local Jacobian composition;
- Euclidean singular-gain interpretation;
- power-iteration estimator behavior and initialization failure;
- probe metadata requirements;
- numerical-versus-semantic separation.

The downstream chapter must independently define:

- semantic interface conditions;
- low-order separator variables;
- admissibility obligations;
- cross-component contract composition;
- governance and evidence rules.

A boundary probe measures.

A boundary contract says what must hold.

39.45 References used in this chapter

- Griewank and Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation* (Griewank and Walther 2008).
- Baydin et al., *Automatic Differentiation in Machine Learning: a Survey* (Baydin et al. 2018).
- Trefethen and Bau, *Numerical Linear Algebra* (Trefethen and David Bau 1997).
- Golub and Van Loan, *Matrix Computations* (Golub and Loan 2013).

Exact source identities and authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-BOUNDARYPROBE-001.yaml`

Chapter 40

Boundary Contracts

Epistemic status: mathematical exposition built from standard sensitivity/conditioning and automatic-differentiation machinery, local-to-global compatibility ideas, exact toy derivations, and a source-bounded GCL project connection.

Primary figure: ‘ATLAS-FIG-BCONTRACT-001‘

Derivation packet: ‘mathematics/derivations/ATLAS-CH-BCONTRACT-001-DERIVATIONS.md‘

Computational witness: ‘mathematics/computational-witnesses/ATLAS-CW-BCONTRACT-001.md‘

40.1 Composition fails at boundaries

A complex learned system is rarely one undivided map.

It is assembled from components:

$$x \xrightarrow{f} y \xrightarrow{g} z.$$

The dimensions may match. The program may run. The result can still be wrong.

The reason is simple: a boundary carries more than shape.

It carries meaning.

It carries admissible geometry.

It carries perturbations and gradients.

It carries numerical error.

It may also carry hidden assumptions about scale, normalization, units, support, precision, or invariants.

The central question of this chapter is therefore:

What must one component expose at its boundary so that another component can use it without reopening the entire interior?

The Atlas answer is a **boundary contract**.

A boundary contract is not merely a software API and not automatically a certificate. It is a structured statement of obligations that a composition depends on.

40.2 The engineered flange

The useful allegory is an **engineered flange**.

Two physical machines may have matching bolt patterns and still be unsafe to connect. One side may expect pressure the other cannot tolerate. A load may arrive along the wrong axis. Two fluids may be chemically incompatible. The dimensions fit while the interface semantics fail.

The correspondence is:

- bolt geometry $\leftrightarrow$ shape/type compatibility;
- working medium $\leftrightarrow$ semantic interpretation;
- load direction $\leftrightarrow$ perturbation direction;
- rated load $\leftrightarrow$ sensitivity bound;
- tolerance $\leftrightarrow$ numerical/error budget;
- alignment constraint $\leftrightarrow$ geometric invariant.

The limit is important.

A learned component is not a steel flange. Its behavior can depend on state, data, adaptation, and training history. It need not obey fixed constitutive laws.

The flange is useful only for one lesson:

compatibility is a bundle of obligations, not one matching dimension.

Once that point is clear, the mathematics should take over.

40.3 A provisional contract object

Let

$$f : \mathcal{X} \rightarrow \mathcal{Y}.$$

The Atlas uses the provisional explanatory object

$$C_f = (\mathcal{X}, \mathcal{Y}, \Sigma, \mathcal{I}, \mathcal{S}, \mathcal{E}).$$

Its fields are:

$\mathcal{X}$ = admissible input domain,

$\mathcal{Y}$ = admissible output domain,

Σ = semantic declaration,

$\mathcal{I}$ = invariants and geometric obligations,

$\mathcal{S}$ = sensitivity and perturbation behavior,

$\mathcal{E}$ = numerical error and precision obligations.

This tuple is not proposed as a universal standard.

It is a disciplined way to ask what must cross a boundary besides bytes and dimensions.

40.4 Interface signature versus boundary contract

An interface signature might say:

$$f : \mathbb{R}^{768} \rightarrow \mathbb{R}^{768}.$$

That is useful.

It tells us what shapes can be connected.

It does not tell us whether the output is:

- normalized;
- centered;
- a probability vector;
- a logit vector;
- an embedding whose norm carries confidence;
- an embedding whose norm is intentionally discarded;
- an element of a quotient class;
- valid only in a bounded operating region.

A boundary contract adds these obligations explicitly.

The distinction can be written schematically as

$$\boxed{\text{signature} \subset \text{contract information.}}$$

A signature can be part of a contract.

It is not the whole contract.

40.5 Shape-compatible, semantically incompatible

Consider

$$f_{\text{dir}}(x) = \frac{x}{\|x\|_2}, \quad x \neq 0.$$

The output is a unit vector.

Suppose its intended semantics are:

direction only; amplitude has been intentionally discarded.

Now define

$$g_{\text{mag}}(y) = \|y\|_2,$$

and suppose the downstream component interprets magnitude as meaningful amplitude.

The tensor shapes compose perfectly:

$$\mathbb{R}^2 \rightarrow \mathbb{R}^2 \rightarrow \mathbb{R}.$$

Take

$$x = (3, 4).$$

Its amplitude is

$$\|x\|_2 = 5.$$

But

$$g_{\text{mag}}(f_{\text{dir}}(x)) = 1.$$

The software connection is legal.

The semantic connection is not.

This is the simplest reason a boundary contract must carry meaning, not only type.

40.6 Four obligation classes

The Atlas separates four classes because they fail in different ways.

40.6.1 Semantic obligations

A semantic obligation answers:

What does this state mean?

Examples include:

- probability versus logit;
- displacement versus absolute position;
- direction versus magnitude-bearing vector;
- uncertainty estimate versus raw score;
- local coordinate versus globally identified state.

Semantic mismatch can survive every shape check.

40.6.2 Geometric obligations

A geometric obligation answers:

What states are admissible, and what equivalences matter?

Examples include:

$$\|x\| = 1,$$

$$X^\top X = I,$$

simplex constraints,

quotient identifications,

positivity,

sparsity,

or a known gauge freedom.

The Geometry chapter showed that ambient coordinates and admissible states need not be the same object.

A boundary that forgets this distinction can hand a downstream component an invalid state even when every tensor dimension is correct.

40.6.3 Differential obligations

A differential obligation answers:

How do perturbations and gradients cross this boundary?

For a differentiable map

$$y = f(x),$$

the local derivative is

$$J_f(x).$$

Rather than materializing the full Jacobian, we often probe it through

$$J_f(x)v$$

and

$$J_f(x)^\top u.$$

These are Jacobian-vector products and vector-Jacobian products. Automatic differentiation provides efficient machinery for computing them in many practical systems (Baydin et al. 2018).

Differential obligations can include:

- directional gain;
- dominant singular value estimates;
- gradient transfer conventions;
- null directions;
- sensitivity to selected subspaces.

40.6.4 Numerical obligations

A numerical obligation answers:

What error and conditioning assumptions are being passed across the boundary?

It may include:

- dtype or precision;
- scaling convention;
- absolute or relative error budget;
- conditioning estimate;
- tolerance;
- solver residual;
- stopping criterion;
- stability region.

Numerical analysis has long distinguished problem conditioning from algorithmic stability (Higham 2002).

A composition can fail because the mathematics is badly conditioned, because an implementation is unstable, or because an interface discards the information needed to tell which happened.

40.7 The chain rule is the first composition law

Let

$$y = f(x),$$

and

$$z = g(y).$$

Then

$$z = (g \circ f)(x).$$

Locally,

$$\boxed{J_{g \circ f}(x) = J_g(f(x))J_f(x).}$$

This elementary identity is one of the most useful facts in the chapter.

It tells us that local perturbation behavior composes multiplicatively.

If

$$\delta x$$

is a small input perturbation, then

$$\delta z \approx J_g J_f \delta x.$$

Using the induced 2-norm,

$$\|J_g J_f\|_2 \leq \|J_g\|_2 \|J_f\|_2.$$

This is a local bound.

It is not automatically a global theorem.

40.8 An exact two-module witness

Take

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 1/2 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}.$$

Let

$$f(x) = Ax,$$

and

$$g(y) = By.$$

Then

$$C = BA = \begin{pmatrix} 2 & 1/2 \\ 0 & 1 \end{pmatrix}.$$

The composed local sensitivity is exactly the matrix C .

For

$$v = \begin{pmatrix} 1 \\ -1 \end{pmatrix},$$

the forward directional sensitivity is

$$Cv = \begin{pmatrix} 3/2 \\ -1 \end{pmatrix}.$$

For

$$u = \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

the reverse sensitivity is

$$C^\top u = \begin{pmatrix} 2 \\ 3/2 \end{pmatrix}.$$

The same boundary can therefore be probed in the forward direction through a JVP and in the reverse direction through a VJP.

40.9 Exact local amplification

For this C ,

$$C^\top C = \begin{pmatrix} 4 & 1 \\ 1 & 5/4 \end{pmatrix}.$$

Its eigenvalues are

$$\lambda_{\pm} = \frac{21 \pm \sqrt{185}}{8}.$$

Therefore

$$\|C\|_2 = \sqrt{\frac{21 + \sqrt{185}}{8}} \approx 2.079707626949502.$$

This number has a precise meaning.

Among unit perturbations, there is a direction whose local output perturbation is amplified by about 2.08.

It does **not** mean:

- every perturbation grows by 2.08;
- every state has the same Jacobian;

- the nonlinear system is globally 2.08-Lipschitz;
- the composition is unsafe.

A sensitivity number is useful only when its scope remains attached to it.

40.10 The boundary figure

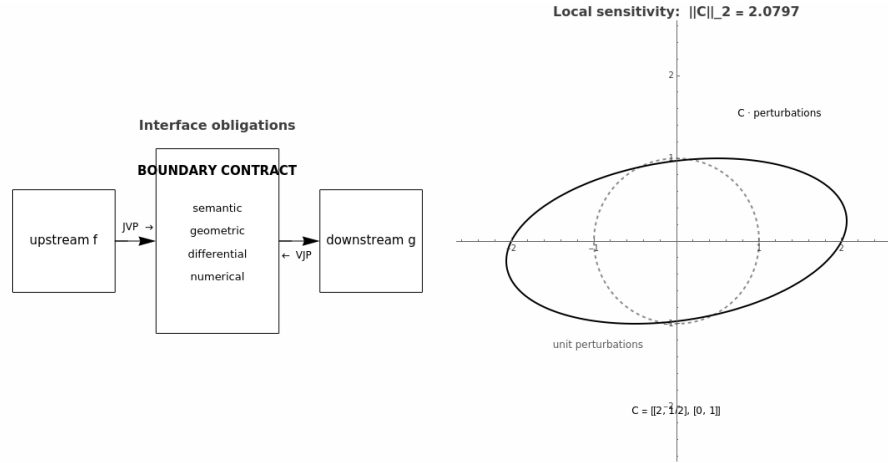


Figure 40.1: A schematic upstream-to-downstream boundary contract listing semantic, geometric, differential, and numerical obligations beside the exact image of the unit perturbation circle under C equals $[[2, 1/2], [0, 1]]$.

The left panel is schematic.

It names the obligation classes and shows forward JVP and reverse VJP channels.

The right panel is literal for the toy system.

The dashed circle is the set of unit perturbations.

The solid ellipse is its image under

$$C.$$

The longest radius is the dominant singular value,

$$2.0797 \dots$$

The figure deliberately keeps the schematic contract language beside an exact local sensitivity object.

One should not be mistaken for the other.

40.11 Power iteration as a boundary probe

If the full Jacobian is too large to form, one may estimate a dominant singular value using repeated applications of

$$J$$

and

$$J^\top.$$

For the toy map,

$$M = C^\top C.$$

Power iteration uses

$$v_{k+1} = \frac{Mv_k}{\|Mv_k\|_2}.$$

The Rayleigh quotient

$$\mu_k = \frac{v_k^\top M v_k}{v_k^\top v_k}$$

approaches the dominant eigenvalue under standard convergence conditions.

Then

$$\sqrt{\mu_k}$$

approaches the dominant singular value of C .

Starting from the normalized direction proportional to $(1, 1)$, the witness gives

$$1.9039433,$$

$$2.0701070,$$

$$2.0792647,$$

2.0796874,

2.0797067,

and then agreement with the exact value to the displayed precision.

This is a good use of a probe because the answer is known independently.

40.12 Estimation is not certification

Power iteration has assumptions.

Its convergence depends on the spectrum and the starting vector.

A finite iteration gives an estimate, not an oracle.

In a nonlinear component, the Jacobian may also change with the operating point.

Therefore a contract should not silently transform

we measured a large singular direction near this state

into

the component has a proven global gain bound.

The first can be strong local evidence.

The second requires a much stronger argument.

This is where numerical conditioning discipline matters (Higham 2002).

40.13 Separator variables

Large systems often cannot expose their entire internal state at every interface.

They summarize.

Let

$$s(y)$$

be a low-dimensional boundary summary.

This can be valuable.

It can also erase exactly the information the next component needs.

Suppose

$$s(y) = y_1.$$

Take

$$y = (0, 1), \quad y' = (0, 2).$$

Then

$$s(y) = s(y') = 0.$$

But if downstream behavior depends on

$$h(y) = y_2,$$

then

$$h(y) \neq h(y').$$

The separator has identified two states that are operationally different.

A low-order interface is therefore justified only relative to a property:

sufficient for what?

That question cannot be skipped.

40.14 Local-to-global reasoning

Boundary contracts are local objects.

They describe what neighboring components may assume about each other.

This naturally recalls local-to-global mathematics.

Sheaf theory provides a formal language for compatible local data and gluing (Curry 2013). That language is useful because it teaches the right kind of question:

if local pieces agree on overlaps, under what conditions do they assemble into a coherent global object?

The Atlas does not claim that learned-system contracts are automatically sheaves.

Nor does local compatibility automatically yield:

- global stability;
- global safety;

- global optimality;
- absence of feedback loops;
- preservation of a system-level invariant.

The sheaf connection is a lens on compatibility.

It is not a free global theorem.

40.15 Individually acceptable pieces can fail together

Suppose two components each have a finite local gain bound.

That does not settle the behavior of a large feedback system containing them.

The problem can arise from:

- gain multiplication;
- incompatible semantics;
- hidden state;
- delay;
- approximation error;
- non-normal coupling;
- feedback;
- state-dependent regime changes.

The chapter's local contract object is deliberately not named a certificate.

It is an interface statement.

Whether the interface obligations are sufficient for a stronger global property must be proved or tested separately.

40.16 Public GCL project state

The Atlas design connects this chapter to the GCL MODULUS research programme.

That connection must remain source-bounded.

The current public repository

‘grandchallenge/MODULUS‘

was inspected at commit

‘9fc42eb5f29d5fff396f13e1a6c972af8fe64b35‘.

It contains typed contract machinery, including the public file

‘modulus/online/contracts.py‘,

which defines a ‘RegretContract’ and associated geometry, feedback, comparator, loss, guarantee, constraint, and telemetry fields.

That is real public project state.

However, the remembered research names

- ‘BoundaryContract’;
- ‘SeparatorCompiler’;
- ‘Modula’

were not found in the inspected public tree or searchable commit history.

The chapter therefore does **not** claim that those named extensions are implemented in current public MODULUS.

This is an example of the source discipline the Atlas will later formalize: project memory can motivate a question without being promoted into documentary evidence.

40.17 The GCL working slogan, properly bounded

The Atlas specification preserves the working slogan:

“Modula tells the optimizer how to move. Boundary contracts tell the system how to compose.”

It is useful rhetorically because it distinguishes two scales:

- an update rule governs motion;
- an interface contract governs composition.

But the slogan is not a theorem.

The present chapter establishes only the bounded mathematical pieces developed above:

- semantic obligations;
- geometric obligations;
- JVP/VJP sensitivity;
- numerical conditioning;
- separator sufficiency questions;
- local composition bounds.

The stronger programme remains research.

40.18 Six failure modes

40.18.1 Shape without meaning

Dimensions match while semantic interpretation differs.

40.18.2 Geometry without preservation

The downstream component receives a state outside its admissible set.

40.18.3 Local sensitivity without global scope

A Jacobian norm measured at one state is treated as a global bound.

40.18.4 Compressed boundary without sufficiency

A separator erases state that changes downstream behavior.

40.18.5 Numerical tolerance without conditioning

An absolute tolerance is quoted without regard to problem scaling or conditioning.

40.18.6 Contract without support

An obligation is written down but not measured, derived, proved, or otherwise supported.

A contract is only as strong as the obligations it records and the evidence behind them.

40.19 What a mature contract might carry

A future machine-readable boundary contract could record, for example:

- object identity;
- input/output semantic types;
- shape and dtype;
- manifold or invariant constraints;
- units or scale conventions;
- local sensitivity probes;
- norm conventions;
- uncertainty or error budget;
- approximation method;
- separator variables;
- operating region;
- failure conditions;
- evidence identity;

- version identity.

The list is intentionally not canonical.

Different components need different obligations.

The purpose is to make hidden assumptions inspectable.

40.20 Atlas connections

Geometry.

A boundary can carry admissible state geometry, not merely coordinates.

Numerical intelligence.

Conditioning, residuals, and error budgets become interface obligations.

Automatic differentiation.

JVPs and VJP provide practical directional probes (Baydin et al. 2018).

Composition Without Catastrophe.

The next step is to ask when local contracts are sufficient for system-level reasoning.

Governed adaptation.

A self-changing component should not silently invalidate the contract under which other components depend on it.

The recurring Atlas shift is:

architecture as connected modules $\longrightarrow$ architecture as composition of explicit contracts.

40.21 Closing view

The most dangerous interface assumptions are often the ones nobody wrote down.

A boundary contract makes them objects.

What does the state mean?

What geometry is valid?

What perturbations are amplified?

What numerical error is being inherited?

What summary has been retained, and what has been discarded?

The flange is only an allegory.

The contract is the object.

And a local contract is not the end of the argument.

It is the beginning of a compositional one.

40.22 References used in this chapter

- (Baydin et al. 2018)
- (Higham 2002)
- (Curry 2013)

See ‘sources/source-locks/ATLAS-CH-BCONTRACT-001.yaml’ for exact source identities, public GCL project-state boundaries, and claim scope.

Chapter 41

Composition Without Catastrophe

Epistemic status: audited Boundary Contracts + audited Split-Operator Networks + Atlas-owned exact composition synthesis.

Specification: [manuscript/specifications/ATLAS-CH-COMPOSE-001.md](#)

Derivation packet: [mathematics/derivations/ATLAS-CH-COMPOSE-001-DERIVATIONS.md](#)

Computational witness: [mathematics/computational-witnesses/ATLAS-CW-COMPOSE-001.md](#)

Source lock: [sources/source-locks/ATLAS-CH-COMPOSE-001.yaml](#)

A system can be built from components that are individually reasonable and still fail as a whole.

That is not paradoxical.

The reason is that composition introduces new questions.

A component can satisfy its own input/output contract while a downstream component assumes something different.

Two maps can each have modest local sensitivity while their product violates a system budget.

The same two transformations can be safe in one order and unsafe in another.

A local certificate can be correct and still be irrelevant outside its operating region.

The governing rule of this chapter is:

component validity + component validity $\not\Rightarrow$ system validity.

The missing object is the composition argument.

41.1 Components do not compose by optimism

Suppose

$$x \xrightarrow{f} y \xrightarrow{g} z.$$

A naive implementation view asks:

do the tensor shapes match?

A stronger contract view asks:

- does y mean what g expects?
- is y inside the operating region required by g ?
- are invariants preserved?
- are perturbation and error budgets compatible?
- does ordering matter?
- what happens to upstream error after downstream amplification?

These are different questions.

Passing one does not answer the others.

The audited Boundary Contracts chapter supplied the interface language.

The audited Split-Operator Networks chapter supplied the order/noncommutativity language.

COMPOSE puts them together.

41.2 A composition has obligations of its own

Write a provisional boundary contract for f as

$$C_f = (\mathcal{X}, \mathcal{Y}, \Sigma_f, \mathcal{I}_f, \mathcal{S}_f, \mathcal{E}_f).$$

Write one for g as

$$C_g = (\mathcal{Y}, \mathcal{Z}, \Sigma_g, \mathcal{I}_g, \mathcal{S}_g, \mathcal{E}_g).$$

The shared symbol $\mathcal{Y}$ is not enough.

The connecting boundary also has to satisfy:

- semantic compatibility;
- domain compatibility;

- invariant compatibility;
- sensitivity assumptions;
- numerical/error accounting.

This chapter uses the explanatory notation

$$C_f \triangleright C_g$$

to mean that the exported guarantees of f satisfy the imported assumptions of g for the declared composition.

This is Atlas notation.

It is not proposed as a universal contract standard.

41.3 The first composition law is the chain rule

At a differentiable operating point,

$$J_{g \circ f}(x) = J_g(f(x))J_f(x).$$

Therefore

$$\|J_{g \circ f}(x)\|_2 \leq \|J_g(f(x))\|_2 \|J_f(x)\|_2.$$

This statement is elementary.

Its implications are not.

The local sensitivities multiply in an upper bound.

So a downstream stage can amplify upstream perturbation.

But the product bound can also be very loose.

The actual alignment of the singular directions matters.

This is already enough to show why component-local sensitivity numbers are not a system certificate.

41.4 The exact two-component witness

Take

$$A = \begin{pmatrix} 3/2 & 0 \\ 0 & 2/3 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 3/2 \\ 2/3 & 0 \end{pmatrix}.$$

Both components have the same operator norm:

$$\|A\|_2 = \|B\|_2 = \frac{3}{2}.$$

Suppose each component has a valid local contract allowing gain up to

$$\frac{3}{2}.$$

Now impose a system-level gain budget

$$\tau = 2.$$

Each component passes its local test.

The composition still has work to do.

41.5 Chronological order matters

For column vectors, chronological A then B is represented by

$$BA.$$

Compute:

$$BA = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

This is an orthogonal swap.

Therefore

$$\|BA\|_2 = 1.$$

The system gain budget passes:

$$1 < 2.$$

Now reverse the chronology.

Chronological B then A is

$$AB.$$

Compute:

$$AB = \begin{pmatrix} 0 & 9/4 \\ 4/9 & 0 \end{pmatrix}.$$

Its singular values are

$$\frac{9}{4}, \quad \frac{4}{9}.$$

Therefore

$$\boxed{\|AB\|_2 = \frac{9}{4}.$$

Now the system gain budget fails:

$$\frac{9}{4} > 2.$$

Same components.

Same component-local contracts.

Different order.

Different system disposition.

41.6 Product order is not chronological prose

The notation matters.

For column-vector states, matrix products act right-to-left.

So:

- chronological A then B corresponds to BA ;
- chronological B then A corresponds to AB .

This distinction was already repaired and frozen in the audited Split-Operator chapter.

COMPOSE inherits it without ambiguity.

41.7 The pair is genuinely noncommuting

The difference is

$$[A, B] = AB - BA.$$

Exactly,

$$[A, B] = \begin{pmatrix} 0 & 5/4 \\ -5/9 & 0 \end{pmatrix} \neq 0.$$

So the order effect is not a naming artifact.

The transformations genuinely do not commute.

This is the finite-dimensional version of a general warning:

when operations do not commute, ordering is part of the system.

41.8 The product norm bound can be loose

The component norms give the generic upper bound

$$\|BA\|_2 \leq \|B\|_2 \|A\|_2 = \frac{9}{4}.$$

But the exact composition is

$$\|BA\|_2 = 1.$$

The upper bound overestimates the actual gain.

For the reverse product,

$$\|AB\|_2 = \frac{9}{4},$$

so the same generic bound is attained exactly.

This gives a useful pair of facts:

component norm ceilings provide a bound, not the exact composition.

and

relative alignment can make the bound either loose or sharp.

41.9 A commuting control

Now use

$$A_c = \text{diag}(3/2, 2/3),$$

$$B_c = \text{diag}(2/3, 3/2).$$

Each still has operator norm

$$\frac{3}{2}.$$

But now

$$[A_c, B_c] = 0,$$

and

$$A_c B_c = B_c A_c = I.$$

Therefore either order has gain

$$\boxed{1}.$$

This is the matched control.

The local component norm ceilings are unchanged.

What changed is the relationship between the transformations.

41.10 The lesson of the control

The witness and control have the same individual gain ceilings.

They do not have the same composition.

Therefore a component-local statement such as

$$\|J_f\| \leq L_f$$

does not contain enough information to reconstruct the exact composed gain.

The interaction matters.

That interaction can involve:

- singular-vector alignment;
- cancellation;
- amplification;
- noncommutativity;
- invariant mismatch;
- domain escape.

The contract needs to say which of these matter for the downstream obligation.

41.11 Error budgets also compose

Gain is only one failure mode.

Approximation error is another.

Suppose $\hat{f}$ approximates f and $\hat{g}$ approximates g .

Assume:

$$\|\hat{f}(x) - f(x)\| \leq \varepsilon_f,$$

and, on the connecting region,

$$\|\hat{g}(y) - g(y)\| \leq \varepsilon_g.$$

Suppose the true downstream map has Lipschitz bound

$$\|g(y_1) - g(y_2)\| \leq L_g \|y_1 - y_2\|.$$

Then:

$$\begin{aligned} \|\hat{g}(\hat{f}(x)) - g(f(x))\| &\leq \|\hat{g}(\hat{f}(x)) - g(\hat{f}(x))\| \\ &\quad + \|g(\hat{f}(x)) - g(f(x))\| \\ &\leq \varepsilon_g + L_g \varepsilon_f. \end{aligned}$$

So

$$\boxed{E_{g \circ f} \leq \varepsilon_g + L_g \varepsilon_f.}$$

The downstream stage amplifies the upstream error.

41.12 The connecting domain is load-bearing

The previous derivation quietly used an important assumption:

$$\widehat{f}(x)$$

must remain inside the region where the downstream bounds apply.

If upstream approximation error pushes the state outside that domain, then:

- the downstream Lipschitz bound may no longer hold;
- the downstream implementation error bound may no longer hold;
- the semantic interpretation may no longer be valid;
- an invariant may be violated.

This is exactly why a boundary contract needs more than a scalar error budget.

41.13 A finite error-budget failure

Take the scalar maps

$$f(x) = x,$$

$$g(y) = \frac{3}{2}y.$$

Suppose the local implementation errors satisfy

$$|\widehat{f}(x) - f(x)| \leq \frac{1}{10},$$

and

$$|\widehat{g}(y) - g(y)| \leq \frac{1}{10}.$$

The downstream gain is

$$L_g = \frac{3}{2}.$$

The derived composition guarantee is

$$E \leq \frac{1}{10} + \frac{3}{2} \frac{1}{10} = \frac{1}{4}.$$

Suppose the system requirement is

$$E \leq \frac{1}{5}.$$

Each component-local error ceiling is only

$$\frac{1}{10}.$$

Yet the derived system guarantee is

$$\frac{1}{4} > \frac{1}{5}.$$

The local budgets pass individually. **The upper bound alone would show only that the system budget cannot yet be certified.** It would not prove that a particular implementation violates the requirement.

For this witness, we can exhibit an implementation attaining that bound. Choose

$$\widehat{f}(x) = x + \frac{1}{10}, \quad \widehat{g}(y) = \frac{3}{2}y + \frac{1}{10}.$$

Each component has actual error exactly $1/10$, on all real inputs, so the local contracts are satisfied. But

$$\begin{aligned} \widehat{g}(\widehat{f}(x)) - g(f(x)) &= \frac{3}{2} \left(x + \frac{1}{10} \right) + \frac{1}{10} - \frac{3}{2}x \\ &= \frac{1}{4} > \frac{1}{5}. \end{aligned}$$

Thus **this explicit composition actually fails** the system error requirement. The separate statement that the generic upper bound is insufficient for certification remains true when no particular implementations are supplied.

41.14 This is not the same as noncommutativity

The scalar error-budget witness has no interesting order effect.

Its failure comes from amplification and accumulation.

The matrix witness has a strong order effect.

Its failure comes from noncommutativity and alignment.

These are different failure mechanisms.

They should not be collapsed into one generic word such as instability.

41.15 Longer chains

Suppose a chain has stages

$$f_1, f_2, \dots, f_n.$$

Let local approximation error at stage k be bounded by

$$\varepsilon_k,$$

and let the downstream true map at stage k have gain bound

$$L_k$$

on the connecting region.

If

$$E_k \leq L_k E_{k-1} + \varepsilon_k,$$

with

$$E_0 = 0,$$

then induction gives

$$E_n \leq \sum_{j=1}^n \varepsilon_j \prod_{k=j+1}^n L_k.$$

The formula makes one point visible:

upstream error is weighted by every downstream gain that follows it.

41.16 Uniform local budgets can still grow badly

If every stage has

$$L_k = L$$

and

$$\varepsilon_k = \varepsilon,$$

then

$$E_n \leq \varepsilon \sum_{r=0}^{n-1} L^r.$$

If

$$L = 1,$$

this becomes

$$E_n \leq n\varepsilon.$$

If

$$L > 1,$$

the worst-case bound grows geometrically:

$$E_n \leq \varepsilon \frac{L^n - 1}{L - 1}.$$

This does not mean the worst case is attained.

It means a local budget cannot be interpreted independently of chain depth and downstream gains.

41.17 Four levels of certificate

It helps to separate four evidence objects.

41.17.1 Component certificate

A statement about one component.

For example:

$$\|J_f(x_0)\|_2 \leq L_f.$$

41.17.2 Interface compatibility certificate

A statement that the upstream output satisfies the downstream assumptions.

For example:

$$f(x_0) \in D_g.$$

41.17.3 Composition certificate

A derived statement about the combined map.

For example:

$$\|J_g(f(x_0))J_f(x_0)\| \leq L_g L_f.$$

41.17.4 System-level guarantee

A statement about the full reachable behavior of the assembled system.

For example:

every reachable state satisfies the declared safety property.

These are not the same evidence level.

41.18 Local does not mean global

A local Jacobian bound at one point,

$$\|J_f(x_0)\| \leq L,$$

does not imply that f is globally L -Lipschitz.

The derivative can be larger elsewhere.

The operating region can change.

The system can leave the region.

A global conclusion requires a bound over the relevant region and assumptions that connect those local bounds into a global result.

The audited Boundary Contracts chapter already enforced this firewall.

COMPOSE keeps it.

41.19 Interface compatibility is not system safety

Suppose every adjacent pair of modules is interface-compatible.

That can still be insufficient for a full system guarantee.

Why?

Because a global property can depend on:

- long-horizon accumulation;
- loops and feedback;
- reachable-state geometry;
- interaction between distant modules;
- stochasticity;
- adaptation;
- distribution shift;
- stateful memory.

Local compatibility is useful.

It is not magical.

41.20 Shape compatibility is the weakest case

The simplest failure is inherited from Boundary Contracts.

An upstream module can output a normalized direction.

A downstream module can interpret vector magnitude as meaningful amplitude.

The shapes match.

The semantics do not.

The software graph composes.

The intended mathematical object does not.

This is the most basic reason the contract must carry meaning.

41.21 Invariants can fail at the boundary

Suppose a downstream component assumes

$$\|y\| = 1.$$

If the upstream component guarantees this exactly, the invariant composes.

If it guarantees only

$$|||y|| - 1| \leq \delta,$$

the downstream theorem may or may not survive.

That depends on the downstream assumptions.

The phrase approximately normalized is not a complete contract.

It needs a tolerance and a theorem that knows what to do with that tolerance.

41.22 Semantic mismatch and numerical error can interact

A system can have both:

- a semantic mismatch;
- a numerical error budget.

A small numerical error does not repair a wrong semantic interpretation.

Likewise, semantically aligned modules can still violate a numerical budget.

The obligation classes remain separate because the remedies are different.

41.23 Order can change what the next module sees

In the exact matrix witness, B acts on a state already transformed by A , or vice versa.

That is the point.

Sequential composition is not a bag of independent ingredients.

The second stage sees the result of the first.

This is why order is architectural information.

The Split-Operator chapter made the same point using exact flows and commutators (McLachlan and Quispel 2002).

COMPOSE turns it into a contract-level system-budget question.

41.24 Classical splitting results stay bounded

The existence of an exact matrix witness does not make an arbitrary learned network an exact split flow.

A learned block can include:

- normalization;
- masking;
- stochasticity;
- routing;
- parameter variation;
- data-dependent control flow.

Classical Lie or Strang order only transfers when the required mathematical assumptions are actually established (McLachlan and Quispel 2002; Hairer et al. 2006).

The chapter uses the mathematics as a reference system, not as a free theorem generator.

41.25 JVP/VJP probes remain local evidence

Boundary Contracts inherited JVP and VJP machinery (Baydin et al. 2018).

These probes are useful for testing:

- directional gain;
- gradient transfer;
- dominant local sensitivity;
- interface alignment.

They still report local differential behavior.

A successful local probe does not certify a whole state space.

This is an evidence boundary, not a criticism of the probe.

41.26 Conditioning remains a separate question

A large observed error can come from:

- an ill-conditioned problem;
- an unstable implementation;
- upstream approximation error;
- downstream amplification;
- interface mismatch.

Numerical analysis keeps conditioning and algorithmic stability distinct (Higham 2002).

COMPOSE should do the same.

The goal is not merely to detect failure.

It is to identify which obligation failed.

41.27 Feedback changes the problem

So far the chapter has mostly discussed feed-forward composition:

$$x \rightarrow f(x) \rightarrow g(f(x)).$$

Now suppose the composed output is fed back:

$$x_{t+1} = (g \circ f)(x_t).$$

A one-pass composition certificate does not automatically answer:

- whether trajectories remain bounded;
- whether they converge;
- whether transient growth occurs;
- whether invariant sets survive;
- whether noise accumulates.

These are dynamical questions.

COMPOSE marks the boundary and stops.

A full closed-loop theorem would require additional dependencies and source authority.

41.28 Repeated local validity can still miss reachability

A component may be certified on an operating region

$$D.$$

If the composed system leaves D , the certificate has expired.

This creates a subtle failure mode:

1. every local theorem is correct;
2. every theorem is scoped;
3. the assembled trajectory exits the scope;
4. the system becomes uncertified.

Nothing contradictory happened.

The composition argument failed to prove reachability containment.

41.29 Budget accounting should be explicit

A system contract should say which budget is being composed.

Examples include:

- perturbation gain;
- absolute numerical error;
- relative error;
- probability mass leakage;
- constraint violation;
- latency;
- memory;
- energy;
- uncertainty;
- privacy loss.

Different budgets obey different composition laws.

COMPOSE derives only the laws it states.

It does not assume every budget adds or multiplies in the same way.

41.30 Conservative bounds are not failures

The product norm bound

$$\|J_g J_f\| \leq \|J_g\| \|J_f\|$$

can be loose.

That does not make it wrong.

A conservative bound may still be useful for screening.

But if the conservative bound violates a threshold, one should not immediately conclude that the real composition violates it.

The exact BA witness demonstrates this:

$$\|B\| \|A\| = \frac{9}{4},$$

while

$$\|BA\| = 1.$$

The right conclusion is:

the coarse certificate is inconclusive for that threshold.

41.31 Tightness can be order-dependent

The reverse product AB reaches the full product bound:

$$\|AB\| = \|A\|\|B\| = \frac{9}{4}.$$

So a conservative certificate can be loose in one ordering and exact in another.

That is a useful diagnostic fact.

It means a system designer should not treat component-local norm ceilings as a complete description of compositional geometry.

41.32 A practical composition protocol

For every boundary $f \rightarrow g$, record:

1. operating domain;
2. semantic meaning of the connecting state;
3. invariants;
4. local sensitivity evidence;
5. numerical/error budget;
6. precision/unit convention;
7. downstream assumptions.

Then for the composition:

1. verify interface compatibility;
2. derive the composition rule for the relevant budget;
3. test order if operations may not commute;
4. compare the derived bound with the system budget;
5. check whether the composed state remains in the region where the local contracts apply;
6. use an exact or higher-fidelity computation when a conservative bound is inconclusive.

41.33 A practical order test

When two modules may be reordered, compare:

$$g \circ f$$

with

$$f \circ g.$$

Possible diagnostics include:

- exact finite witness;
- output difference;
- commutator in a declared linear reference model;
- Jacobian commutator as a local diagnostic;
- task-level difference.

Do not silently identify a same-state Jacobian commutator with the derivative of the full nonlinear composition defect.

That distinction is inherited from the audited Split chapter.

41.34 A practical error test

For a two-stage approximation, report:

$$\varepsilon_f, \quad \varepsilon_g, \quad L_g,$$

and the derived composition budget:

$$\varepsilon_g + L_g \varepsilon_f.$$

Also report the connecting domain on which those quantities are valid.

Without the domain, the number is incomplete.

41.35 A practical chain test

For a long chain, report the recurrence:

$$E_k \leq L_k E_{k-1} + \varepsilon_k.$$

Do not only report the individual ε_k .

The downstream gains determine how strongly early errors matter.

This is the composition analogue of keeping provenance attached to evidence.

41.36 What counts as catastrophe here?

The title is broader than any one metric.

A catastrophe can mean:

- a violated numerical tolerance;

- a broken invariant;
- a semantic mismatch;
- a system gain over budget;
- a state leaving the certified region;
- order-dependent behavior not represented in the design;
- accumulated error beyond specification.

The chapter does not define a universal catastrophe metric.

It requires the system-level obligation to be declared.

41.37 What this chapter proves

The exact finite witness proves:

$$\|A\| = \|B\| = \frac{3}{2},$$

$$\|BA\| = 1,$$

$$\|AB\| = \frac{9}{4}.$$

It proves that the same components pass a gain budget of 2 in one order and fail it in the reverse order.

The commuting control proves:

$$A_c B_c = B_c A_c = I.$$

The error derivation proves, under its assumptions:

$$E_{g \circ f} \leq \varepsilon_g + L_g \varepsilon_f.$$

The chain derivation proves, under its assumptions:

$$E_n \leq \sum_{j=1}^n \varepsilon_j \prod_{k=j+1}^n L_k.$$

These are the load-bearing results.

41.38 What this chapter does not prove

It does not prove:

all local contracts pass $\Rightarrow$ global safety.

It does not prove that learned neural modules are exact flows.

It does not prove that a local Jacobian bound is a global Lipschitz theorem.

It does not prove that commuting linearizations imply globally commuting nonlinear maps.

It does not prove long-horizon stability.

It does not prove a universal contract calculus.

41.39 Non-implications

The chapter therefore rejects:

component contracts pass $\nRightarrow$ system contract passes,

$\|J_f(x_0)\| \leq L \nRightarrow$ global L -Lipschitz,

$\|J_g J_f\| \leq \|J_g\| \|J_f\| \nRightarrow$ tight estimate,

$[A, B] = 0$ in a reference model $\nRightarrow$ learned modules commute globally,

local error budget $\nRightarrow$ long-horizon stability,

and

interface compatibility $\nRightarrow$ closed-loop safety.

41.40 Atlas connections

Boundary Contracts.

COMPOSE consumes the obligation language and asks whether the guarantees actually survive assembly.

Split-Operator Networks.

COMPOSE consumes order/noncommutativity discipline and turns it into explicit system-budget tests.

Adaptive depth.

A variable-depth system changes the number of times local gain/error budgets compose.

Routing and sparse systems.

Different routes induce different composition chains and therefore different contract obligations.

Agents and tools.

A tool interface can be type-correct while semantically or numerically incompatible with the agent's assumptions.

Governed adaptation.

A self-changing system must know which local contract changes invalidate downstream composition arguments.

41.41 Closing view

A component can be correct.

Another component can be correct.

Their composition can still be wrong.

Sometimes the problem is semantic.

Sometimes it is numerical.

Sometimes an error budget accumulates.

Sometimes a noncommuting order turns a harmless composition into an amplifying one.

Sometimes every local statement remains true while the system simply leaves the region where those statements apply.

The correct response is not to distrust modularity.

It is to make composition explicit.

safe composition requires a proved bridge between local guarantees and the declared system obligation.

When that bridge is absent, local validity remains local.

41.42 References used in this chapter

Inherited through audited prerequisites:

- (Higham 2002)
- (Baydin et al. 2018)
- (McLachlan and Quispel 2002)
- (Hairer et al. 2006)

Exact source identities and claim boundaries are recorded in `sources/source-locks/ATLAS-CH-COMPOSE-001.yaml`.

Chapter 42

Conditional Computation

Epistemic status: established sparse/conditional-computation mechanisms + audited Depth prerequisite + Atlas synthesis + exact finite resource witness.

Specification: manuscript/specifications/ATLAS-CH-SPARSE-001.md

Formal packet: mathematics/derivations/ATLAS-CH-SPARSE-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-SPARSE-001.md

Source lock: sources/source-locks/ATLAS-CH-SPARSE-001.yaml

A sparse model is not one thing.

A matrix can contain many zeros.

A network can skip whole blocks.

A transformer can drop tokens.

A router can select only part of a module family.

An adaptive-depth system can stop early.

These mechanisms can all reduce some notion of work.

They do not reduce the same resource, and they do not imply the same hardware behavior.

The governing rule of this chapter is:

conditional computation is about which computation actually executes for a particular input or state; sparsity alone does not determine realized cost.

42.1 From computational depth to conditional paths

The Depth chapter established that execution depth need not equal architectural depth.

A network can:

- recur;
- halt early;
- solve toward an equilibrium;
- skip blocks;
- allocate different numbers of updates to different inputs.

Conditional computation broadens that idea.

Instead of asking only:

how many steps execute?

ask:

which parts of the available computation execute, for which input,
under which routing rule, and at what resource cost?

That question covers depth, width, token count, modules, and sparse parameter structure.

42.2 A typed sparsity object

Use

$S=(P,A,T,B,D,R)$.

Here:

- P is parameter sparsity;
- A is activation sparsity;
- T is token or item sparsity;
- B is block, module, or expert sparsity;
- D is conditional execution depth;
- R is the routing or selection rule.

These coordinates are not interchangeable.

A network can be highly sparse in parameters while executing the same sparse graph for every input.

Another network can have dense parameters but execute only two of eight blocks for one input and all eight for another.

The first is sparse.

The second is conditional.

A system can be both.

42.3 Static parameter sparsity

A simple static pruning model is

$$W' = M \odot W$$

with fixed binary mask M .

The zeros remain in the same positions across evaluated inputs.

Han, Mao, and Dally's Deep Compression is a representative pruning/compression lineage (Han et al. 2016b).

Static parameter sparsity can reduce:

- storage;
- nonzero arithmetic;
- memory movement;

if the implementation and hardware exploit the structure.

It does not require a per-input router.

42.4 Sparsity ratio is not execution semantics

Suppose 90 percent of matrix entries are zero.

A dense kernel may still multiply through the zeros.

In that implementation:

- the representation is sparse;
- the mathematical operator has many zeros;
- the hardware execution may remain dense.

Therefore:

stored zero structure is not enough to establish skipped physical work.

The execution path must be inspected.

42.5 Conditional computation

Conditional computation makes the active graph depend on the current input or state.

Write:

```

p(x)=Route(x)
and
y=Execute(p(x),x).

```

The path $p(x)$ may specify:

- which blocks execute;
- which tokens survive;
- which modules activate;
- how long computation continues.

The router and executor are separate objects.

That separation matters because routing costs work.

42.6 Hard block skipping

For residual-style branch F_k , write

$$x_{k+1} = x_k + g_k(x_k)F_k(x_k).$$

If

$$g_k(x_k) \text{ in } \{0,1\}$$

and the implementation avoids evaluating F_k when the gate is zero, the branch is genuinely skipped.

SkipNet gives a concrete learned example of input-dependent residual-block skipping (Wang et al. 2018).

The important mechanism is not merely that a coefficient becomes zero.

The block is bypassed.

42.7 Soft gates are not automatically sparse execution

Suppose instead

$$g_k \text{ in } [0,1].$$

If the system computes

$$F_k(x_k)$$

and only afterward multiplies by a small or zero coefficient, the expensive branch has already run.

So:

$g_k=0$

does not by itself prove:

$\text{cost}(F_k)=0$.

A differentiable training relaxation can therefore describe a sparse intent without producing sparse execution.

This boundary was already present in DEPTH-001 and remains load-bearing here.

42.8 Spatially adaptive computation

Conditional computation need not apply uniformly across one input.

Figurnov et al. allocate different computation to different spatial positions in residual networks (Figurnov et al. 2017).

This introduces a finer distinction:

- input-level conditional computation;
- region- or token-level conditional computation.

Two examples can execute different paths.

Two positions inside one example can also receive different computation.

The execution graph can vary within the input.

42.9 Token sparsity

A transformer-like system processes a set or sequence of tokens.

Let active tokens before stage k be X_k .

A selector produces

$A_k=\text{Select}_k(X_k)$.

Later computation can run only on A_k .

DynamicViT is a concrete example of progressive, input-dependent token sparsification in vision transformers (Rao et al. 2021).

The crucial implementation question is:

at what point does the dropped token stop participating in expensive computation?

42.10 Masking after dense work is not the same as removing before it

Suppose attention over n tokens constructs an $n \times n$ score matrix.

If all n^2 pair interactions are computed and some outputs are masked afterward, the score-matrix work was not saved.

If selection reduces the active token count to $m < n$ before the later attention stage, the later dense pair count becomes m^2 .

That arithmetic distinction is exact.

Whether it becomes a wall-clock speedup depends on the execution system.

42.11 Structural sparsity and dynamic sparsity

A useful taxonomy is:

42.11.1 Structural/static

The sparse pattern is known before seeing the particular input.

Examples:

- fixed pruning mask;
- fixed block-sparse matrix;
- fixed local attention window.

42.11.2 Dynamic/input-dependent

The sparse pattern depends on the current input or state.

Examples:

- block routing;
- token dropping;
- early exit;
- adaptive spatial computation.

The same mathematical zero pattern can have different engineering implications depending on when it becomes known.

42.12 Why when matters

Static structure can often be compiled, fused, packed, or scheduled ahead of time.

Dynamic structure may require:

- a router;
- synchronization;
- variable shapes;
- irregular memory access;
- dynamic batching.

So dynamic sparsity can reduce arithmetic while increasing control overhead.

Conditional computation is not free selection.

42.13 Parameter count and active parameter count

A conditionally executed model may contain many parameters while using only a subset per input.

Therefore distinguish:

- total parameter capacity;
- active parameters per input;
- arithmetic operations per input;
- memory residency;
- parameter communication.

A model can have large total capacity without evaluating all parameters on every example.

The downstream Mixture-of-Experts chapter will develop this in the expert-routing setting.

42.14 Routing is a computation

A router can be:

- a small neural network;
- a confidence threshold;
- a heuristic;
- a learned policy;
- a top-k selector.

Whatever its form, its cost belongs in the resource ledger.

Write, in one resource unit,

$$C_{\text{total}}(\mathbf{x}) = C_{\text{route}}(\mathbf{x}) + C_{\text{exec}}(\mathbf{p}(\mathbf{x}), \mathbf{x}).$$

A system should not report only the work after routing and pretend selection was free.

42.15 Resource vectors

One scalar rarely captures conditional-computation cost.

Use a resource vector such as

$$\mathbf{C}(\mathbf{x}) = (\mathbf{F}(\mathbf{x}), \mathbf{K}(\mathbf{x}), \mathbf{M}(\mathbf{x}), \mathbf{E}(\mathbf{x})),$$

with:

- arithmetic/FLOPs $\mathbf{F}$;
- launches or dispatches $\mathbf{K}$;
- memory traffic $\mathbf{M}$;
- energy $\mathbf{E}$.

Other deployments may add:

- communication volume;
- synchronization;
- sequential depth;
- accelerator occupancy.

Do not add unlike units without a declared scalarization.

42.16 Average cost

Conditional computation is often motivated by easy and hard inputs.

For input distribution $\mathbf{X}$, average cost is

$$\mathbf{E}[\mathbf{C}(\mathbf{X})].$$

On a finite benchmark it is the empirical mean.

This quantity answers:

what does a typical input cost under the evaluation distribution?

It does not answer:

how much capacity is required for the worst input?

42.17 Peak and tail cost

A service may care about:

- maximum active blocks;
- p95 latency;
- p99 communication;
- worst-case memory;
- deadline misses.

These are tail or peak questions.

A low mean can coexist with a dense worst-case path.

So reports should separate:

- average compute;
- peak compute;
- tail latency.

42.18 Exact witness: fixed baseline

Consider four sequential blocks.

Each contributes ten arithmetic units.

Assume the fixed implementation fuses them into one launch.

Its resource pair is

$C_{\text{fixed}} = (40, 1)$.

The coordinates are:

- arithmetic units;
- launches.

Every input receives the same computation.

42.19 Exact witness: conditional paths

Now use four input-dependent paths:

- $x_1: \{1, 4\}$;
- $x_2: \{1, 2, 4\}$;
- $x_3: \{1, 4\}$;
- $x_4: \{1, 2, 3, 4\}$.

Assume:

- one router launch;
- one launch for each active block.

The resource pairs are:

- $x_1: (20, 3)$;
- $x_2: (30, 4)$;
- $x_3: (20, 3)$;
- $x_4: (40, 5)$.

42.20 Average arithmetic falls

Average arithmetic is

$$(20+30+20+40)/4 = 55/2 = 27.5.$$

Compared with fixed arithmetic 40, the reduction is

$$1-(55/2)/40 = 5/16 = 31.25\%.$$

The conditional system therefore performs less arithmetic on average under this finite workload.

That claim is exact.

42.21 Peak arithmetic does not fall

The fourth input executes all four blocks.

So:

$$F_{\text{peak_cond}}=40.$$

The fixed baseline also uses 40.

Thus:

- average arithmetic drops;
- peak arithmetic is unchanged.

If hardware must provision for the worst path, the mean alone is insufficient.

42.22 Launch count gets worse

Average conditional launches are

$$(3+4+3+5)/4 = 15/4 = 3.75.$$

The fixed path uses one.

So the conditional system is better in one coordinate and worse in another:

$$(40, 1)$$

versus

$$(55/2, 15/4).$$

Neither dominates componentwise.

42.23 Hardware assumptions determine the ordering

Define a toy scalar cost

$T_{\text{alpha,beta}}(F,K) = \text{alpha } F + \text{beta } K$.

This is not claimed to be a realistic universal latency model.

It is a proof device.

If

$\text{alpha}=1, \text{beta}=0$,

then only arithmetic matters:

- fixed = 40;
- conditional = 27.5.

Conditional wins.

If

$\text{alpha}=1/10, \text{beta}=10$,

then launches are expensive:

- fixed = 14;
- conditional = 40.25.

Fixed wins.

The same execution traces support opposite latency conclusions under different hardware cost models.

42.24 FLOPs do not determine latency

The witness proves a structural point:

fewer arithmetic operations do not uniquely determine lower latency.

Real hardware adds further effects:

- memory bandwidth;
- cache behavior;
- kernel launch overhead;
- vectorization;
- occupancy;
- synchronization;
- batching;
- communication.

The Roofline model makes one part of this explicit by relating attainable performance to arithmetic intensity, memory bandwidth, and compute capability (Williams et al. 2009).

FLOPs are one coordinate.

They are not the clock.

42.25 Dynamic irregularity can cost efficiency

Hardware often prefers regular work.

A dense matrix operation can be highly optimized.

A dynamically sparse workload may introduce:

- tiny kernels;
- irregular memory reads;
- divergent branches;
- underfilled accelerators;
- fragmented batches.

Therefore a method can report strong nominal arithmetic reduction and weak realized speedup.

That is not necessarily an implementation failure.

It may be the consequence of the hardware/software cost structure.

42.26 Static sparsity can also fail to accelerate

Parameter pruning creates zeros.

But unstructured zeros only help execution if the kernels and hardware exploit them.

Gale, Elsen, and Hooker’s large-scale sparsity study reinforces the need to evaluate sparsification methods at scale rather than treating sparsity ratio itself as a complete performance metric (Gale et al. 2019).

The Atlas conclusion is broader:

sparsity percentage is not a hardware benchmark.

42.27 Conditional depth

One form of conditional computation is to vary execution depth.

Easy input:

`tau(x)=2.`

Hard input:

$\tau(x)=8$.

DEPTH-001 already supplied the stopping semantics.

SPARSE-001 adds the resource question:

- what work disappears when execution stops?
- what routing/halting overhead remains?
- what are average and tail depths?
- what hardware can exploit the variable path?

42.28 Early exit versus block skipping

Early exit stops the remaining suffix of a network.

Block skipping can omit internal blocks while later blocks still execute.

These are different path families.

For an eight-block network:

early exit might produce

$\{1,2,3\}$.

Skipping might produce

$\{1,3,4,7,8\}$.

Both are conditional.

Their scheduling and representation behavior can differ.

42.29 Token dropping versus conditional depth

Token dropping changes active width.

Conditional depth changes the number or identity of sequential transformations.

A system can do both.

For example:

- prune tokens at stage 3;
- skip block 5;
- stop at block 7.

One scalar called “sparsity” hides these distinctions.

The typed sparsity object keeps them visible.

42.30 Training-time execution can be denser

Discrete routing creates optimization difficulties.

Training may therefore use:

- continuous gates;
- straight-through estimators;
- stochastic sampling;
- auxiliary losses;
- policy gradients;
- dense masking.

Inference may use hard decisions.

The training graph and inference graph can therefore have different cost.

Do not infer inference sparsity directly from the training relaxation.

42.31 Compute and quality form a joint claim

Skipping computation changes the function executed.

So an efficiency report should pair compute with quality.

At minimum report:

(Q, C)

for task quality Q and cost C .

A method that cuts computation in half and destroys task performance is not a successful efficiency result under most objectives.

The acceptable frontier is application-dependent.

42.32 Dynamic computation can specialize effort

The attraction of conditional computation is simple:

not every input may need the same amount or kind of processing.

Easy examples may need less work.

Hard examples may need more.

Some tokens may be redundant.

Some spatial locations may be uninformative.

This creates an allocation problem.

The system spends compute where the router expects it to matter.

42.33 The router can be wrong

A conditional system introduces a new failure surface.

The router can:

- drop useful tokens;
- skip necessary blocks;
- stop too early;
- over-compute easy inputs;
- under-compute hard inputs;
- become unstable under distribution shift.

A dense baseline does not have exactly the same failure mode.

Efficiency comes with a control problem.

42.34 Routing confidence is not correctness

A router may emit a high confidence score.

That does not prove the skipped computation was unnecessary.

The same epistemic boundary from DEPTH-001 applies:

a learned halting or routing signal is not automatically a calibrated error estimator.

If one wants guaranteed error-controlled adaptation, a separate theorem or calibration layer is required.

42.35 Batching changes the economics

Suppose active-block counts in a batch are

2,3,2,4.

The mean is

11/4.

But a synchronous implementation may effectively wait for the four-block path.

Then wall-clock behavior tracks something closer to the maximum.

Dynamic batching, compaction, or regrouping can recover efficiency.

Those mechanisms add overhead of their own.

Average path length is not enough to predict batch latency.

42.36 Communication can dominate arithmetic

In distributed conditional systems, selecting a remote module can require communication.

Then total cost can contain:

- local compute;
- routing;
- all-to-all exchange;
- synchronization;
- imbalance.

A method that reduces local arithmetic can still slow down if communication grows.

This becomes central in Mixture-of-Experts systems.

SPARSE-001 establishes the accounting discipline before that chapter.

42.37 Capacity and active work can decouple

Conditional execution allows a model to possess more total components than one input uses.

This can increase representational capacity without proportional per-input arithmetic.

But total capacity still affects:

- storage;
- loading;
- communication;
- optimizer state during training;
- deployment footprint.

“Only k modules active” is not the same as “only k modules exist.”

42.38 Static compression and conditional capacity are different strategies

Static pruning asks:

which structure can be removed for all evaluated inputs?

Conditional computation asks:

which structure can be omitted for this input?

The former reduces a model globally.

The latter selects among available computation dynamically.

Combining them is possible.

Conflating them obscures the design space.

42.39 Token sparsity can alter later complexity superlinearly

If a later dense attention stage has pairwise score cost proportional to n^2 , reducing token count from n to m can reduce that score-matrix arithmetic roughly from n^2 to m^2 .

But end-to-end speedup also depends on:

- selector cost;
- projection cost;
- memory movement;
- fixed layers;
- implementation shape efficiency.

The local arithmetic saving is not the whole pipeline.

42.40 Measured speedup is contextual evidence

DynamicViT reports both FLOP reduction and measured throughput improvement on its evaluated systems (Rao et al. 2021).

That is stronger evidence than FLOPs alone for those experiments.

It remains contextual:

- device;
- software;
- batch;
- model;
- implementation.

A measured speedup should not be promoted to a hardware-independent theorem.

42.41 Conditional computation is an allocation policy

Viewed abstractly, the router allocates a compute budget.

For input x , it chooses path $p(x)$.

The execution then spends resources according to that path.

This suggests three separate questions:

1. **selection:** which path is chosen?
2. **execution:** what work does that path perform?
3. **valuation:** was the saved work worth the quality change?

These questions should be evaluated separately.

42.42 Failure modes

42.42.1 Sparsity/FLOP conflation

Treat sparse weights as proof of skipped arithmetic.

42.42.2 FLOP/latency conflation

Treat fewer operations as proof of faster execution.

42.42.3 Soft/hard conflation

Treat a relaxed gate as actual skipped hardware work.

42.42.4 Mean/peak conflation

Report low average compute while hiding dense worst-case paths.

42.42.5 Router-free accounting

Count only selected computation and omit selection overhead.

42.42.6 Training/inference conflation

Measure soft training sparsity and claim hard inference savings.

42.42.7 Capacity/active-work conflation

Treat inactive modules as if they do not consume storage or training resources.

42.42.8 Quality omission

Report compute savings without the associated task-performance change.

42.43 A practical conditional-compute ledger

Before accepting an efficiency claim, record:

Field	Question
sparse object	Parameters, activations, tokens, blocks, experts, depth?
static/dynamic	Fixed structure or input-dependent path?
selector	What chooses the active work?
hard/soft	Is work physically skipped or merely weighted?
training graph	What executes during training?
inference graph	What executes during deployment?
average work	Mean over which workload?
peak/tail work	Worst case or declared quantile?
arithmetic	How many operations are avoided?
memory	What traffic/storage changes?
launches	Does execution become more fragmented?
communication	Is data routed across devices?
hardware	Which machine/software/batch regime?
quality	What performance change accompanies the saving?

This ledger prevents “sparse” from becoming an untyped efficiency claim.

42.44 What the exact witness establishes

The companion witness establishes:

- fixed baseline resource pair (40,1);
- conditional average pair (55/2,15/4);
- exact average arithmetic reduction $5/16=31.25\%$;
- unchanged peak arithmetic 40;
- compute-dominated toy cost favors conditional execution;
- launch-dominated toy cost favors fixed execution.

The witness does not claim either toy cost model is a real accelerator.

It proves only that arithmetic count does not uniquely identify latency.

42.45 Downstream handoff

Mixture-of-Experts Systems — **ATLAS-CH-MOE-001** may now assume:

- static versus conditional sparsity;
- hard versus soft routing;

- routing versus execution cost;
- parameter capacity versus active work;
- average versus peak/tail compute;
- resource-vector accounting;
- arithmetic-versus-latency boundary;
- hardware-context dependence of measured speedup.

The downstream chapter must independently develop:

- expert routing;
- capacity factors;
- load balancing;
- specialization;
- collapse;
- expert parallelism;
- communication structure.

Conditional computation supplies the grammar.

Mixture-of-Experts will supply the system.

42.46 References used in this chapter

- Han, Mao, and Dally, *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding* (Han et al. 2016b).
- Figurnov et al., *Spatially Adaptive Computation Time for Residual Networks* (Figurnov et al. 2017).
- Wang et al., *SkipNet: Learning Dynamic Routing in Convolutional Networks* (Wang et al. 2018).
- Rao et al., *DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification* (Rao et al. 2021).
- Gale, Elsen, and Hooker, *The State of Sparsity in Deep Neural Networks* (Gale et al. 2019).
- Williams, Waterman, and Patterson, *Roofline: An Insightful Visual Performance Model for Multicore Architectures* (Williams et al. 2009).

Exact source identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-SPARSE-001.yaml`

Chapter 43

Mixture-of-Experts Systems

Epistemic status: established sparse-expert mechanisms + audited Conditional Computation and Transformer prerequisites + Atlas synthesis + exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-MOE-001.md

Formal packet: mathematics/derivations/ATLAS-CH-MOE-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-MOE-001.md

Source lock: sources/source-locks/ATLAS-CH-MOE-001.yaml

A mixture-of-experts model can contain far more parameters than it executes for one token.

That is its central architectural attraction.

It is also the source of several confusions.

A router emits probabilities.

A capacity rule accepts or rejects assignments.

A distributed runtime moves tokens between devices.

Experts receive different training signals.

An auxiliary loss pushes traffic toward a target distribution.

These are related operations.

They are not the same object.

The governing rule of this chapter is:

an MoE system is defined by router scores, discrete dispatch, capacity semantics, expert computation, and distributed execution together—not by the router probabilities alone.

43.1 From conditional computation to experts

SPARSE-001 established the broad conditional-computation picture:

- a model can expose more capacity than it executes for one input;
- hard and soft gating must be distinguished;
- routing overhead counts;
- arithmetic reduction does not automatically imply latency reduction.

Mixture-of-experts systems instantiate that idea with a family of alternative expert modules.

For each token, only a subset is selected.

43.2 Where experts enter a Transformer

TRANSFORMER-001 established the standard Transformer block:

- residual-stream state;
- attention;
- position-wise feed-forward transformation;
- residual paths;
- normalization.

A common sparse-expert Transformer replaces selected dense feed-forward sub-layers with expert families.

Attention still performs token mixing.

The expert layer performs conditionally selected token-wise transformation.

Other designs are possible, but this is the baseline used here.

43.3 Expert family

Let a token state be:

$\mathbf{h}_i \in \mathbb{R}^d$.

Let expert e be:

$E_e: \mathbb{R}^d \rightarrow \mathbb{R}^d$.

There are:

$e=1, \dots, M$

experts.

Total expert parameter capacity can grow with M even if each token visits only one or a few experts.

That decoupling between total capacity and active work is a defining sparse-expert property.

43.4 Router logits

A router maps token state to scores:

$$\mathbf{z}_i \text{ in } \mathbb{R}^M.$$

The scores can be arbitrary real values.

By themselves they do not specify which experts execute.

They are preferences or routing evidence.

43.5 Router probabilities

A common transformation is:

$$p_{\{i,e\}} = \exp(\mathbf{z}_{\{i,e\}}) / \sum_j \exp(\mathbf{z}_{\{i,j\}}).$$

Then:

$$p_{\{i,e\}} \geq 0$$

and:

$$\sum_e p_{\{i,e\}} = 1.$$

These probabilities are useful for ranking and weighting.

They are still not the executed route.

43.6 Preferred top-k route

Let:

$$P_i = \text{TopK}(p_i, k).$$

For top-1:

$$|P_i| = 1$$

before capacity handling.

For top-2 or larger k, several experts are preferred.

The word **preferred** matters.

Capacity can prevent a preferred route from executing.

43.7 Accepted dispatch

Define:

$a_{\{i,e\}}$ in $\{0,1\}$.

Here:

$a_{\{i,e\}}=1$

means token i is actually dispatched to expert e after capacity and overflow policy.

This is the first object that directly describes expert execution.

A router probability can be nonzero while:

$a_{\{i,e\}}=0$.

43.8 Gate weights

Suppose token i is accepted by expert set:

$A_i=\{e:a_{\{i,e\}}=1\}$.

A generic sparse mixture writes:

$\text{MoE}(h_i) = \sum_{(e \in A_i)} \alpha_{\{i,e\}} E_e(h_i)$.

The weights α might be:

- original router probabilities;
- probabilities renormalized over accepted experts;
- hard unit weights;
- another declared convention.

Dispatch and aggregation weighting are separate design choices.

43.9 Top-1 is simpler, not trivial

Switch Transformer popularized a particularly simple routing case: one expert per token.

This reduces some communication and combination complexity.

But even top-1 requires decisions about:

- capacity;
- overflow;
- balancing;
- dropped tokens;
- expert placement;
- batch composition.

One selected expert does not mean one simple system.

43.10 Capacity is not a probability threshold

Suppose an expert can process at most C routed tokens in the current routing group.

Capacity constrains:

$a_{\{i,e\}}$.

It does not change the fact that the router may assign high probability to more than C tokens.

Thus:

capacity acts on accepted assignments, not on the probability simplex.

This distinction is load-bearing.

43.11 Nominal capacity

A common nominal form is:

$C = \text{ceil}(c_f \cdot k \cdot N/M)$.

Here:

- N is token count in the routing group;
- M is expert count;
- k is nominal experts per token;
- c_f is a capacity factor.

The exact formula and routing group are implementation choices.

The Atlas does not promote one capacity formula into a universal definition.

43.12 Demand versus accepted load

Preferred demand for expert e is:

$d_e = \sum_i 1\{e \text{ in } P_i\}$.

Accepted token load is:

$n_e = \sum_i a_{\{i,e\}}$.

When capacity or overflow changes routing:

$d_e \neq n_e$

can occur.

This is the first reason a router histogram can differ from executed expert load.

43.13 Overflow needs policy

If:

$d_e > C$,

something must happen.

Possible policies include:

- drop overflow tokens;
- reroute them;
- send them through a fallback dense path;
- expand effective capacity;
- buffer work.

The router probabilities do not answer this question.

The overflow policy is part of the model's operational semantics.

43.14 Dropping changes the computation

If a token is dropped from the expert layer, then the executed computation differs from the nominal top-k design.

The token may:

- skip the expert transform;
- continue through a residual path;
- use some fallback;
- be handled differently by implementation.

Therefore drop rate belongs in both model-quality and systems accounting.

43.15 Rerouting changes the meaning of top-1

Suppose a token's preferred expert is full.

A reroute policy may send it to its second choice.

Then the executed expert is not the argmax expert.

This is not a contradiction.

It means:

preferred route and accepted route are different objects.

Any analysis of router behavior should state which one it studies.

43.16 Expert token load

Define normalized accepted load:

$f_e = n_e / \sum_j n_j$, **provided at least one expert assignment was accepted**, so $\sum_j n_j > 0$. If every token is dropped (or there are no dispatched tokens), the normalized accepted-load vector is undefined; record an empty-dispatch state and the absolute loads rather than divide by zero or claim balanced traffic.

If all top-1 tokens are accepted:

$\sum_e n_e = N$.

If some are dropped:

$\sum_e n_e < N$.

A load-balancing plot should therefore state its denominator.

43.17 Router probability mass

Define:

$m_e = \sum_i p_{\{i, e\}}$

and normalized mass:

$q_e = m_e / N$.

Because each probability row sums to one:

$\sum_e q_e = 1$.

But there is no general identity:

$q_e \neq f_e$.

One measures soft router preference.

The other measures accepted discrete traffic.

43.18 Balance has multiple meanings

“Balanced experts” can mean several different things:

- equal token counts;
- equal router probability mass;
- equal arithmetic work;
- equal bytes communicated;
- equal device time;
- equal training signal;
- equal utility.

These are not interchangeable.

The word **balance** is incomplete without the quantity being balanced.

43.19 Auxiliary balancing objectives

Sparse expert systems often add an auxiliary objective:

$$L_{\text{total}} = L_{\text{task}} + \lambda L_{\text{balance}}.$$

Different systems use different proxies and stabilizers.

The important structural point is:

$$L_{\text{balance}} \neq L_{\text{task}}.$$

A router can become more balanced under its proxy while the main task becomes worse.

Or the reverse.

43.20 Balance is a systems pressure

Why balance at all?

If one expert receives nearly every token while others sit idle:

- capacity is wasted;
- one device can become a straggler;
- communication patterns become uneven;
- underused experts may receive little training signal.

So load balancing often serves utilization and optimization stability.

It is not evidence that every expert should receive exactly the same semantic workload.

43.21 Specialization is a different claim

An expert is specialized when its behavior or performance differs systematically on some class of inputs.

Evidence might include:

- domain enrichment;
- conditional accuracy;
- feature selectivity;
- stable functional differences;
- systematic output differences.

Traffic imbalance alone does not prove any of these.

A popular expert may simply be a routing attractor.

43.22 Equal traffic does not prove equal usefulness

Two experts can each receive 10,000 tokens.

One can be critical.

The other can be nearly redundant.

Traffic counts measure use frequency.

They do not directly measure value.

This distinction will appear exactly in the finite witness.

43.23 Preferred-router collapse versus accepted-load concentration

Call **preferred-router collapse** a regime where router probability mass or preferred routes concentrate on a small expert subset under a declared statistic and horizon.

Accepted dispatch is a different object. Capacity, rerouting, or dropping can make accepted loads look more balanced than the underlying preferences.

Useful diagnostics therefore distinguish:

- probability-mass concentration;
- preferred-route concentration;
- accepted-load concentration;
- persistent overflow at particular experts.

“Collapse” without naming the object and statistic is too vague.

43.24 Expert underuse

An expert can receive so few tokens that it gets little training signal.

That is underuse.

It may be caused by:

- router bias;
- initialization;
- data distribution;

- capacity interactions;
- optimization dynamics.

Underuse is not the same thing as two experts learning the same function.

43.25 Expert redundancy

Two experts can both be heavily used and still implement similar functions.

A functional comparison might evaluate:

$$E[||E_e(h) - E_j(h)||^2]$$

under a declared input distribution.

Low functional distance is one possible redundancy signal.

It says something traffic counts cannot.

43.26 Capacity overload is different again

A routing system can repeatedly demand more slots than an expert is allowed to serve.

That is a capacity/overflow problem.

It may co-occur with router concentration.

But the concepts differ:

- preferred-router collapse concerns where router preferences concentrate;
- accepted-load concentration concerns executed traffic after capacity handling;
- capacity overload concerns preferred demand relative to an execution limit.

Keeping them separate helps identify the correct repair.

43.27 Expert parallelism

Large expert families are often partitioned across devices.

Let:

$\text{dev}(e)$

denote the device hosting expert e .

A routed token may originate elsewhere.

Then conditional computation becomes a distributed-systems problem.

The expert computation may be sparse while token movement is expensive.

43.28 Dispatch communication

If token i starts on:

$\text{src}(i)$

and is accepted by expert e , inter-device movement occurs when:

$\text{src}(i) \neq \text{dev}(e)$.

A simple communication-count proxy is:

$K_{\text{comm}} = \sum_e a_{\{i,e\}} 1_{\{\text{src}(i) \neq \text{dev}(e)\}}$.

This is only a count.

Actual cost depends on:

- representation size;
- collective implementation;
- batching;
- topology;
- return traffic;
- synchronization.

43.29 All-to-all is not free

Expert-parallel systems often group tokens by destination expert, exchange them, compute expert outputs, and return results.

This can require collective communication.

Therefore:

low active arithmetic can coexist with high communication cost.

GShard and Switch make this systems issue central to large sparse-expert execution.

43.30 Stragglers matter

Suppose expert computation is parallel.

Step completion can depend on the slowest device/expert path.

Even if average load is good, one overloaded expert can dominate latency.

This is another reason average token count is not enough.

Peak and tail behavior matter.

43.31 Equal token counts need not mean equal compute

If every expert has the same architecture and input shape, equal accepted counts can equalize one arithmetic proxy.

But experts may differ in:

- size;
- precision;
- hardware placement;
- kernel efficiency;
- input shape;
- cache locality.

Then:

$n_e = n_j$

does not imply:

$C_e = C_j$.

The load unit must match the cost question.

43.32 Training and serving need not use the same routing semantics

During training, the system may tolerate:

- router noise;
- auxiliary balance pressure;
- higher capacity factors;
- token dropping;
- larger batches.

Serving may prioritize:

- determinism;
- tail latency;
- replica placement;
- availability;
- bounded communication.

A routing analysis should therefore state the execution phase.

43.33 The exact witness: six tokens

Use six tokens and three experts.

The router probabilities are:

token	E1	E2	E3
t1	9/10	1/20	1/20
t2	4/5	3/20	1/20
t3	1/2	1/10	2/5
t4	1/20	9/10	1/20
t5	1/20	3/4	1/5
t6	1/20	1/20	9/10

Every row sums to one.

43.34 Naive top-1 preferences

The preferred experts are:

- t1 -> E1;
- t2 -> E1;
- t3 -> E1;
- t4 -> E2;
- t5 -> E2;
- t6 -> E3.

So preferred loads are:

(3,2,1).

Let capacity be:

C=2.

E1 is overloaded.

43.35 Keep the strongest E1 assignments

For E1:

- t1 has probability 9/10;
- t2 has probability 4/5;
- t3 has probability 1/2.

The declared overflow policy retains the two highest E1 preferences:

t1,t2.

Token t3 becomes overflow.

43.36 Reroute the overflow token

For t_3 :

- E2 probability is $1/10$;
- E3 probability is $2/5$.

E3 has one spare slot.

Therefore:

$t_3: E1 \rightarrow E3$.

Final accepted assignments are:

- E1: t_1, t_2 ;
- E2: t_4, t_5 ;
- E3: t_3, t_6 .

Accepted loads are:

$(2, 2, 2)$.

43.37 Accepted load is perfectly balanced

Accepted load fractions are:

$f = (1/3, 1/3, 1/3)$.

For the squared count-imbalance statistic:

$B_count = \sum_e (f_e - 1/3)^2$,

we get:

$B_count = 0$.

By this metric, routing is perfectly balanced.

43.38 Router probability mass is not balanced

Summing probabilities over all six tokens gives:

$m = (47/20, 2, 33/20)$.

Normalize by six:

$q = (47/120, 1/3, 11/40)$.

This is not uniform.

Therefore:

$f \neq q$.

Balanced accepted counts have not made the router's soft preference distribution uniform.

43.39 Exact probability-imbalance value

The offsets from uniform are:

$$47/120 - 1/3 = 7/120;$$

$$1/3 - 1/3 = 0;$$

$$11/40 - 1/3 = -7/120.$$

Therefore:

$$B_{\text{prob}} = 2(7/120)^2 = 49/7200.$$

So the exact witness has:

$$B_{\text{count}} = 0$$

but:

$$B_{\text{prob}} > 0.$$

The two balance notions disagree.

43.40 Equal traffic still does not imply equal usefulness

Now attach a toy post-routing utility diagnostic.

Let final assignment utilities be:

- E1 on t1: 4;
- E1 on t2: 4;
- E2 on t4: 2;
- E2 on t5: 2;
- E3 on t3: 1;
- E3 on t6: 1.

Expert totals are:

$$(8, 4, 2).$$

Traffic remains:

$$(2, 2, 2).$$

Equal counts do not imply equal evaluated utility.

43.41 The utility numbers are not router scores

This distinction matters.

The router probability says:

how strongly did the router prefer this expert?

The toy utility says:

under a separate evaluation, how much value did this accepted expert assignment contribute?

These need not match.

Indeed, one research question in MoE systems is whether the router reliably sends inputs to experts that are actually useful for them.

43.42 Drop instead of reroute

Now keep the same probability table and same capacity.

Change only the overflow policy.

Drop t3 instead of rerouting it.

Then accepted loads become:

(2,2,1).

Drop rate is:

1/6.

The router has not changed.

The executed MoE has.

43.43 Overflow policy changes arithmetic

Assume identical expert cost c per accepted token.

Under rerouting:

$C_{\text{expert}}=6c$.

Under dropping:

$C_{\text{expert}}=5c$.

Thus capacity semantics alter realized expert arithmetic even with identical router probabilities.

This is a concrete version of the SPARSE-001 distinction between routing logic and executed work.

43.44 Balanced counts do not settle communication

Suppose E1, E2, and E3 live on different devices.

Even with:

(2,2,2)

accepted token counts, the communication pattern depends on where the tokens originated.

One routing assignment can be local.

Another can require extensive cross-device exchange.

Count balance is therefore not a communication theorem.

43.45 Shazeer et al.: sparse gating and balancing

Shazeer et al. introduced a sparsely gated MoE layer at large scale, with a trainable gating network selecting a sparse combination of experts.

Their work also makes balancing pressure explicit because highly uneven expert use creates practical problems.

The Atlas uses this as a primary historical mechanism.

It does not treat the original routing loss or expert layout as universally canonical.

43.46 GShard: capacity meets distributed execution

GShard places sparse MoE computation inside a large Transformer and combines it with automatic sharding.

That makes two facts impossible to ignore:

- routing creates per-expert capacity constraints;
- experts distributed over accelerators create communication and placement problems.

The Atlas uses GShard to connect algorithmic routing to expert-parallel execution.

43.47 Switch: top-1 simplifies one axis

Switch Transformer reduces the number of selected experts per token to one.

This can simplify routing and communication relative to multi-expert mixtures.

It does not remove:

- capacity;
- balancing;
- training instability;
- distributed communication.

The important lesson is that simplification of one routing axis leaves a multi-layer system.

43.48 ST-MoE: stability and transfer remain system properties

ST-MoE studies sparse expert design with emphasis on training stability and downstream transfer.

That matters because an MoE system cannot be judged from parameter count or routing sparsity alone.

Useful questions include:

- does training remain stable?
- do experts receive enough signal?
- does specialization persist?
- does sparse pretraining transfer?

These questions belong downstream of the basic routing semantics established here.

43.49 A practical MoE ledger

Before accepting an MoE result, record:

Field	Question
tokens	What is the routing group and token count?
experts	How many, and what does each expert compute?
router	Which logits/probabilities are produced?
top-k	How many experts are preferred per token?
gate weights	How are accepted expert outputs weighted?
capacity	Which unit and formula set expert capacity?
overflow	Drop, reroute, fallback, buffer, or other?
accepted load	How many assignments actually execute per expert?
probability mass	What soft router mass goes to each expert?
auxiliary loss	Which balance/stability proxy is optimized?
drops	Which tokens receive no expert execution?
specialization	What functional evidence supports the claim?

Field	Question
redundancy	How are expert functions compared?
placement	Which device hosts each expert?
communication	Which token/activation bytes move?
phase	Training or inference/serving?
quality boundary	What does balance not prove?

This ledger turns “MoE routing” into an auditable system description.

43.50 Failure modes

43.50.1 Probability-equals-dispatch

Soft router mass is treated as executed expert traffic.

43.50.2 Top-k-equals-execution

Preferred experts are reported without capacity/overflow semantics.

43.50.3 Balanced-counts-equals-specialization

Equal traffic is used as evidence that experts learned distinct useful functions.

43.50.4 Balanced-counts-equals-cheap

Equal accepted loads are treated as proof of equal latency or communication.

43.50.5 Collapse-as-one-word

Preferred-router concentration, accepted-load concentration, underuse, redundancy, and capacity overload are conflated.

43.50.6 Auxiliary-loss-equals-task-objective

A balancing proxy is treated as if it directly measures task quality.

43.50.7 Sparse-parameters-equals-sparse-system-cost

Active expert arithmetic is counted while router, communication, padding, and synchronization are ignored.

43.51 What the exact witness establishes

The companion witness proves:

- naive top-1 demand is $(3, 2, 1)$;
- capacity is 2;
- t3 is the lowest-confidence E1 token;
- its best available alternative is E3;
- rerouting t3 yields accepted loads $(2, 2, 2)$;
- probability mass is $(47/20, 2, 33/20)$;
- normalized mass is $(47/120, 1/3, 11/40)$;
- count imbalance is 0;
- probability-mass imbalance is $49/7200$;
- toy utility totals are $(8, 4, 2)$;
- drop-on-overflow instead yields $(2, 2, 1)$ and drop rate $1/6$;
- under equal expert token cost, reroute and drop policies use expert arithmetic 6c and 5c.

No universal routing optimum is inferred.

43.52 Downstream handoff: Router Dynamics

ATLAS-CH-ROUTERDYN-001 may now assume:

- logits/probabilities versus accepted dispatch;
- capacity and overflow semantics;
- count-load versus probability-mass diagnostics;
- preferred-router concentration versus accepted-load concentration versus underuse versus redundancy;
- explicit evidence requirements for specialization.

It must independently develop temporal churn, instability, commutators, spectral diagnostics, and route evolution.

43.53 Downstream handoff: Systems

ATLAS-CH-SYSTEMS-001 may now assume:

- expert placement;
- dispatch communication;
- capacity;
- accepted per-expert work;
- active versus total expert capacity;
- straggler boundaries.

It must independently develop hardware/system cost models, communication topology, serving constraints, and measured efficiency.

The MoE lesson is:

sparse experts create conditional capacity, not free capacity. The router proposes; the capacity policy dispatches; the experts compute;

the network moves data; and each layer needs its own evidence.

43.54 References used in this chapter

- Noam Shazeer et al., *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*, 2017, arXiv:1701.06538.
- Dmitry Lepikhin et al., *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding*, 2020, arXiv:2006.16668.
- William Fedus, Barret Zoph, and Noam Shazeer, *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*, Journal of Machine Learning Research 23(120), 1–39, 2022.
- Barret Zoph et al., *ST-MoE: Designing Stable and Transferable Sparse Expert Models*, 2022, arXiv:2202.08906.

Exact source identities and authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-MOE-001.yaml`

Chapter 44

Router Dynamics and Diagnostics

Epistemic status: audited MoE/Optimizer-Dynamics prerequisites + primary router-stability/routing sources + Atlas synthesis + exact finite witnesses

Specification: manuscript/specifications/ATLAS-CH-ROUTERDYN-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-ROUTERDYN-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-ROUTERDYN-001.md

A routing system can look balanced while changing its mind about every token.

That is the central diagnostic problem.

Mixture-of-Experts systems expose several routing layers at once: probabilities, preferred routes, capacity-constrained dispatch, realized expert load, and expert behavior. Looking at only one can hide motion in the others.

44.1 Four moving objects

For N tracked tokens and E experts, let

$$P_t$$

be the router probability matrix at checkpoint t .

Let

$$r_t(i)$$

be the declared preferred top-1 expert for token i .

Let

$$A_t$$

be the accepted dispatch matrix after capacity, overflow, and execution policy.

Finally, let

$$\ell_t = A_t^\top \mathbf{1}$$

be accepted expert load.

The audited MoE chapter already established that these are not interchangeable.

ROUTERDYN adds time.

44.2 Probability drift

Define average rowwise probability drift by

$$D_P(t) = \frac{1}{2N} \sum_i \|P_t(i, :) - P_{t-1}(i, :)\|_1.$$

The factor one-half turns rowwise L1 distance between probability vectors into total variation.

This diagnostic can move even when the preferred expert does not.

That matters because routing margins can become sharper or flatter before a top-1 assignment flips.

44.3 Preferred-route churn

For top-1 routing, define

$$\chi_R(t) = \frac{1}{N} \sum_i \mathbf{1}\{r_t(i) \neq r_{t-1}(i)\}.$$

This is token-identity sensitive.

A value of zero says every tracked token kept the same preferred expert.

A value of one says every tracked token changed preferred expert.

It says nothing by itself about whether those changes helped the task.

44.4 Accepted-dispatch churn

Capacity and overflow can change what is actually executed.

For one accepted expert per token,

$$\chi_A(t) = \frac{1}{2N} \|A_t - A_{t-1}\|_{1,\text{entry}}.$$

Preferred-route churn and accepted-dispatch churn can differ when capacity handling intervenes.

That is why a router-dynamics dashboard should not report one as a synonym for the other.

44.5 Load drift

Accepted load is much coarser:

$$D_\ell(t) = \frac{1}{2N} \|\ell_t - \ell_{t-1}\|_1.$$

Load answers a systems question: how much accepted work moved among experts?

It does not preserve token identity.

That information loss creates the first exact counterexample.

44.6 Exact counterexample: perfect balance, total churn

Take four tracked tokens.

At one checkpoint,

$$r_0 = (1, 1, 2, 2).$$

At the next,

$$r_1 = (2, 2, 1, 1).$$

The load vector is unchanged:

$$\ell_0 = \ell_1 = (2, 2).$$

So

$$D_\ell = 0.$$

But every token changed expert:

$$\chi_R = \chi_A = 1.$$

Thus:

$$\boxed{\text{stable load} \not\Rightarrow \text{stable routing}}$$

in the exact finite witness.

This is why load balancing is not a temporal-stability metric.

44.7 Transition operators on expert identity

For a fixed tracked token panel, define the empirical expert-transition matrix

$$T_t(a, b) = \frac{\#\{i : r_{t-1}(i) = a, r_t(i) = b\}}{\#\{i : r_{t-1}(i) = a\}}.$$

Every row with a positive denominator is stochastic. An expert with **no previously assigned tracked token** has no empirical conditional transition row: the quotient is undefined there, and any zero/uniform fill-in would be an explicit smoothing convention rather than an observation. The following witness has both prior expert classes occupied, so both rows are defined.

The swap witness produces

$$T_{\text{swap}} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

A stable routing panel produces

$$T_{\text{stable}} = I.$$

The matrices describe very different temporal behavior.

Yet both have singular values

$$(1, 1).$$

So:

same singular spectrum $\nRightarrow$ same routing dynamics

for this pair.

Spectral diagnostics are useful only when the operator being diagnosed and the information discarded by the summary are explicit.

44.8 One-step transition is not automatically a Markov model

The matrix T_t is an empirical transition object.

Calling it a stationary Markov chain requires additional assumptions.

The data distribution may change.

The router parameters may change.

The experts themselves may change.

Capacity policy may change.

The token panel may not be representative.

ROUTERDYN therefore uses Markov-style algebra only as a diagnostic lens unless stationarity and sampling assumptions are separately justified.

44.9 Route churn can be zero while probabilities move

Now take two tokens and two experts:

$$P_0 = \begin{pmatrix} 0.6 & 0.4 \\ 0.4 & 0.6 \end{pmatrix}, \quad P_1 = \begin{pmatrix} 0.9 & 0.1 \\ 0.1 & 0.9 \end{pmatrix}.$$

The preferred experts do not change.

Therefore

$$\chi_R = 0.$$

But the average probability drift is

$$D_P = 0.3.$$

A routing system can therefore become much more confident without changing its top-1 token assignment.

The converse is also possible: a small probability change near a decision boundary can cause route churn.

44.10 Routing stability and training stability are different claims

ST-MoE studies training instabilities in large sparse models and introduces router z-loss as one stabilization technique in that setting (Zoph et al. 2022).

That is valuable evidence about sparse-model training.

But “training became more stable” is not identical to “token assignments became temporally stable.”

A loss spike, router-logit magnitude, assignment churn, load imbalance, and expert specialization are different observables.

The chapter keeps them separate.

44.11 Routing rules alter what can specialize

Expert Choice reverses a common routing perspective: instead of each token choosing a fixed number of experts, experts choose tokens subject to capacity (Zhou et al. 2022).

This reinforces a broader point.

Specialization is not produced by experts in isolation.

It depends on the routing rule that determines what experience reaches them.

A temporal specialization study must therefore record the routing policy alongside the expert profile.

44.12 Specialization needs a declared taxonomy

Suppose each tracked token has a declared category

$$c_i \in \mathcal{C}.$$

For expert e with nonzero accepted load, define

$$S_t(e, c) = \frac{\sum_i A_t(i, e) \mathbf{1}\{c_i = c\}}{\ell_t(e)}.$$

This gives the expert’s accepted-token category profile.

A simple specialization score can compare that profile with the tracked-panel category distribution.

But the category system is part of the measurement.

If the categories are language, syntax, topic, token identity, task family, or another feature, the meaning of “specialized” changes.

Therefore:

load concentration $\neq$ semantic specialization

and

specialization score $\neq$ causal importance.

44.13 Temporal specialization drift

An expert can keep the same load while the composition of that load changes.

A total-variation profile drift can be written

$$D_S(t, e) = \frac{1}{2} \sum_c |S_t(e, c) - S_{t-1}(e, c)|.$$

This lets the Atlas ask whether an expert’s observed role is stable even when its utilization is stable.

Again, the answer is relative to the declared taxonomy.

44.14 Router dynamics include optimizer memory

Routing parameters are trained parameters.

If their next update depends on momentum, adaptive moments, schedules, or other optimizer state, then the dynamical state is larger than the router weights alone.

Let

$$\xi_t$$

collect the declared router-plus-optimizer state.

For a differentiable router-plus-optimizer update map H_t , set $J_t = DH_t(\xi_t)$. Its **tangent (first-variation) dynamics** are

$$\delta\xi_{t+1} = J_t\delta\xi_t.$$

Here $\delta\xi$ denotes a tangent perturbation along a fixed reference trajectory, not an arbitrary finite difference between two nonlinear runs. For finite input displacement v , differentiability gives $H_t(\xi_t + v) - H_t(\xi_t) = J_tv + o(\|v\|)$ instead.

Along a changing trajectory,

$$\delta\xi_{t+h} = J_{t+h-1} \cdots J_t \delta\xi_t.$$

This product is exact for the tangent recursion with its declared trajectory-dependent Jacobians, not a global finite-amplitude nonlinear evolution law. This is inherited from the audited Optimizer-State Dynamics viewpoint.

44.15 Commutators detect order sensitivity

Successive local maps need not commute.

Define

$$\mathcal{C}_t = J_t J_{t-1} - J_{t-1} J_t.$$

If

$$\mathcal{C}_t \neq 0,$$

the order of the two local linear maps matters.

This is a diagnostic fact.

It is not a causal explanation of why the maps changed.

44.16 Exact commutator witness

Take

$$J_0 = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad J_1 = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Each has eigenvalues

$$(1, 1).$$

But

$$J_1 J_0 \neq J_0 J_1.$$

The commutator is

$$\begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix},$$

with Frobenius norm

$$\sqrt{2}.$$

Identical one-step eigenvalue multisets therefore do not determine the two-step order-sensitive behavior.

44.17 Spectral router diagnostics are typed

Several spectra may be interesting:

- the spectrum of an empirical expert-transition matrix;
- singular values of that transition matrix;
- eigenvalues of a local augmented router-state Jacobian;
- singular values of that Jacobian;
- finite-horizon gains of Jacobian products.

These are spectra of different operators.

They cannot be compared as though “the router spectrum” were one universal object.

44.18 Churn is not automatically bad

High churn can indicate instability.

It can also indicate:

- adaptation to a changing data distribution;
- a router still discovering useful specialization;
- exchange among functionally similar experts;
- movement caused by changing capacity semantics;
- benign margin crossing near equivalent experts.

Low churn can be bad too.

A router can be stably wrong.

ROUTERDYN therefore reports churn before interpreting churn.

44.19 Capacity can hide preference dynamics

The MoE audit established an important warning: accepted load can look balanced even when preferred routing is concentrated because capacity handling modifies execution.

The temporal version is stronger.

Accepted-load stability can arise from a capacity mechanism even while preferred token assignments or router probabilities move substantially.

A longitudinal study should therefore preserve both pre-capacity and post-capacity observables.

44.20 A practical diagnostic record

A router-dynamics measurement should declare at least:

- checkpoint/time index;
- tracked token panel or sampling distribution;
- router logits/probabilities;
- preferred-route rule and tie-breaking;
- capacity and overflow policy;
- accepted dispatch;
- load vector;
- route-churn metric;
- probability-drift metric;
- any token taxonomy used for specialization;
- any transition operator;
- any augmented-state Jacobian approximation;
- spectral statistic and operator identity.

Without these fields, two “router stability” claims may be measuring different phenomena.

44.21 Handoff to Routing as Online Decision Making

ATLAS-CH-REGRETRROUTE-001 may now assume:

- probability, preferred-route, accepted-dispatch, and load temporal separation;
- exact route-churn and probability-drift definitions;
- empirical expert-transition operators;
- taxonomy-relative specialization profiles;
- local augmented-state commutator diagnostics;
- the finite examples showing that loads and singular spectra can hide token-level motion.

REGRETROUTE must add the online-decision layer.

It must define:

- the routing action;
- comparison policy/class;
- reward or loss timing;
- regret;
- optionality;
- correction capacity;
- nonstationarity assumptions.

Temporal diagnostics are not regret theory by themselves.

44.22 Durable lesson

A router does not have one dynamics.

It has several coupled temporal surfaces:

probability $\rightarrow$ preference $\rightarrow$ dispatch $\rightarrow$ load,

plus expert behavior and optimizer memory.

The chapter's main discipline is to diagnose each surface before interpreting the system.

44.23 References used in this chapter

- (Zoph et al. 2022)
- (Zhou et al. 2022)

Exact source authority, prerequisite identities, and claim boundaries are locked in:

`sources/source-locks/ATLAS-CH-ROUTERDYN-001.yaml`

Chapter 45

Routing as Online Decision Making

Epistemic status: audited Router Dynamics substrate + primary adversarial-bandit/shifting-expert authority + Atlas-owned exact finite routing witnesses.

Specification: manuscript/specifications/ATLAS-CH-REGRETRROUTE-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-REGRETRROUTE-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-REGRETRROUTE-001.md

Source lock: sources/source-locks/ATLAS-CH-REGRETRROUTE-001.yaml

A sparse router changes over time.

That does not yet make it an online decision problem.

To speak about regret, one must say:

- what action was actually taken;
- what actions were feasible;
- when loss was incurred;
- what feedback arrived;
- what comparator is allowed;
- what kind of nonstationarity is being measured.

The governing rule is:

diagnose routing first; define regret only after the decision problem is explicit.

45.1 Router dynamics are not regret theory

Router Dynamics already distinguished:

probability $\rightarrow$ preference $\rightarrow$ dispatch $\rightarrow$ load.

It also showed that:

- stable load need not mean stable token assignment;
- zero preferred-route churn need not mean zero probability drift;
- equal singular spectra need not mean equal dynamics.

Those are temporal diagnostics.

Regret asks a different question:

how much cumulative loss did the routing decisions incur relative to
a declared comparator class?

45.2 Preferred route is not always the executed action

Let:

$$p_t$$

be the preferred route proposed by the router.

Let:

$$F_t$$

be the set of dispatch actions that remain feasible after capacity and availability constraints.

Let:

$$a_t \in F_t$$

be the accepted/executed dispatch.

When capacity binds:

$$a_t \neq p_t$$

can occur.

That distinction matters because the executed action is the one that actually consumes capacity and incurs the declared decision loss.

45.3 Capacity is part of the decision semantics

Write:

$$a_t = G_t(p_t, \kappa_t),$$

where κ_t contains the relevant capacity/overflow state.

The map G_t can:

- accept the proposal;
- reroute it;
- reject it;
- defer it;
- send it to overflow capacity.

The online problem is underspecified until this behavior is declared.

45.4 Loss happens after the accepted action

Use the ordering:

1. observe the pre-action context;
2. propose a route;
3. enforce feasibility/capacity;
4. execute accepted dispatch;
5. incur loss;
6. reveal feedback.

Denote the accepted-action loss by:

$$\ell_t(a_t).$$

If future losses are used to choose a_t , the object is an oracle or hindsight construction, not an ordinary online policy.

45.5 Feedback must be named

Under full information, after round t the learner receives:

$$\{\ell_t(a) : a \in F_t\}.$$

Under bandit feedback, it receives only:

$$\ell_t(a_t).$$

Auer, Cesa-Bianchi, Freund, and Schapire treat adversarial/nonstochastic multi-armed bandits under partial feedback (Auer, Cesa-Bianchi, Freund, et al. 2002).

That distinction changes what an online router can infer and guarantee.

45.6 Comparator comes before regret

Let:

$$\Pi$$

be a declared class of comparison policies or action sequences.

Then define:

$$R_T^\Pi(a) = \sum_{t=1}^T \ell_t(a_t) - \min_{u \in \Pi} \sum_{t=1}^T \ell_t(u_t).$$

Without Π , “regret” is incomplete.

45.7 Static regret asks one question

A static comparator uses one fixed route for every round, restricted here to routes in $\bigcap_{t=1}^T F_t$. If the intersection is empty, a feasible static comparator does not exist and a different comparator or an expressly labeled counterfactual benchmark is needed.

It asks:

how much worse was the router than the best single fixed expert in hindsight?

That is useful when one expert should remain globally best.

It is not the only plausible sparse-routing benchmark.

45.8 Shifting regret asks another question

Suppose which expert is best changes over time.

Let:

$$\Pi_S(F_{1:T}) = \left\{ u_{1:T} : u_t \in F_t \text{ for every } t, \sum_{t=2}^T \mathbf{1}[u_t \neq u_{t-1}] \leq S \right\}.$$

Now the comparator may switch at most S times **while respecting the same declared feasible accepted-dispatch sets**. This version assumes the comparator class is nonempty; otherwise the minimum in the regret definition is not defined without an additional convention. A comparator allowed to use capacity-infeasible actions is a different, explicitly counterfactual benchmark.

Herbster and Warmuth study tracking a best expert that can change over segments (Herbster and Warmuth 1998).

This is the comparator style used in the exact witness.

45.9 One common finite routing problem

Take two accepted actions:

$$A, B.$$

Use four rounds.

Both routes remain feasible every round:

$$F_t = \{A, B\}.$$

Use losses:

t	A	B
1	0	1
2	1	0
3	0	1
4	1	0

The loss-minimizing route alternates every round.

45.10 The shifting comparator

Allow up to three switches:

$$\Pi_3.$$

The unique zero-loss comparator is:

$$(A, B, A, B).$$

So the hindsight comparator loss is:

$$0.$$

45.11 A perfectly stable route can regret its stability

Consider:

$$a^{\text{stay}} = (A, A, A, A).$$

Its accepted-dispatch churn is:

$$0.$$

Its cumulative loss is:

$$0 + 1 + 0 + 1 = 2.$$

Therefore its shifting regret is:

$$\boxed{2}.$$

The route is maximally stable.

It is not regret-optimal for the shifting comparator.

45.12 A maximally churning route can be regret-optimal

Now use:

$$a^{\text{track}} = (A, B, A, B).$$

Its switch count is:

$$3.$$

Its cumulative loss is:

$$0.$$

Therefore its shifting regret is:

$$\boxed{0}.$$

The route churns every possible time.

It is nevertheless comparator-optimal.

45.13 Churn is not a regret surrogate

The two policies use the same:

- action space;
- feasible sets;
- horizon;
- losses;
- feedback;
- comparator class.

Yet:

$$0 < 3$$

for churn, while:

$$2 > 0$$

for regret.

So:

$$\boxed{\text{low churn} \not\Rightarrow \text{low regret}}$$

and:

$$\boxed{\text{high churn} \not\Rightarrow \text{high regret.}}$$

45.14 Static regret tells a different story

The fixed action A loses two.

The fixed action B also loses two.

So best static comparator loss is two.

Against that comparator:

- stay-on- A has regret zero;
- the alternating tracker has regret -2 .

The numbers changed because the question changed.

Comparator choice is part of the meaning of regret.

45.15 Negative static regret is not an error

The alternating route beats every fixed comparator.

Therefore static regret can be negative for a realized sequence under this definition.

That simply means the policy outperformed the restricted comparator class.

It does not invalidate the calculation.

45.16 Optionality is feasible choice

Define:

$$O_t = |F_t| - 1.$$

This counts feasible accepted alternatives other than the executed action.

In the exact witness:

$$O_t = 1$$

for every round.

Both routes have the same optionality.

Their regret differs.

So optionality is not regret either.

45.17 Correction capacity is ability to change course

One operational form is remaining switch budget.

Let allowed switch count be S .

Before round t , let:

$$N_t = \sum_{s=2}^{t-1} \mathbf{1}[a_s \neq a_{s-1}].$$

Define:

$$K_t = \max(0, S - N_t).$$

This says how many further route changes the controller may still make under the declared switch budget.

45.18 Optionality and correction capacity are distinct

A router can have many feasible alternatives but no remaining switching budget.

Or it can have switching budget but only one feasible action because capacity is saturated.

Thus:

available alternatives $\neq$ ability to correct.

45.19 The stay policy preserves correction capacity

In the exact witness, $S = 3$.

The stay route uses no switches.

Its remaining switch budget remains high.

Yet it accumulates regret two.

Preserving correction capacity is not automatically the same as using it well.

45.20 The tracking policy spends correction capacity

The alternating route consumes switch budget.

Its remaining correction capacity falls over time.

But it obtains zero shifting regret.

Again, correction capacity is a resource, not a performance score.

45.21 Capacity should constrain the comparator too

Suppose expert B is full at round t .

Then $B \notin F_t$.

A feasible comparator should normally obey the same restriction.

Otherwise the regret gap mixes online routing quality with access to an impossible action.

An unconstrained oracle comparator can still be informative, but it must be labeled as counterfactual.

45.22 Preferred-route regret is a separate counterfactual

Suppose the router prefers B , but capacity sends the token to A .

One can measure:

- accepted-dispatch regret;
- preferred-route counterfactual regret.

They answer different questions.

The default REGRETROUTE object is accepted-dispatch regret.

45.23 Capacity can hide or create churn

Changing capacity can make accepted dispatch move even while preference remains fixed.

It can also make accepted dispatch remain fixed while preference changes.

Therefore ROUTERDYN's typed observables are required even after regret enters the analysis.

45.24 Load balance is not online optimality

A capacity mechanism can enforce balanced load.

Balanced load can be operationally valuable.

But it does not imply that accepted decisions minimize cumulative routing loss.

So:

balanced load $\not\Rightarrow$ low regret.

45.25 Specialization is not regret

A specialization statistic asks which token classes or features tend to use an expert.

Regret asks how losses compare to a benchmark.

An expert can be highly specialized and still be wrong for the current token.

A broad expert can be the regret-minimizing action under another objective.

45.26 Probability drift is not regret

Router probabilities can drift substantially while the accepted route remains unchanged.

Then accepted-dispatch regret can remain identical.

Conversely, tiny probability movement near a routing threshold can change the accepted action and cumulative regret.

45.27 If switching is costly, put it in the loss

Suppose switching has penalty λ .

Use:

$$\tilde{\ell}_1 = \ell_1(a_1), \quad \tilde{\ell}_t = \ell_t(a_t) + \lambda \mathbf{1}[a_t \neq a_{t-1}] \quad (t = 2, \dots, T).$$

This convention charges only switches **between** the T executed actions; there is no undeclared a_0 or first-round switching penalty. A deployment that charges initialization or migration from a pre-existing route must declare that initial route and charge separately.

Now churn enters the decision objective directly.

For the alternating route, three switches contribute:

3λ .

A large enough switching cost can make stability preferable.

That is not a contradiction.

It is a different objective.

45.28 Nonstationarity must be typed

The exact witness has changing roundwise losses.

But empirical router nonstationarity can come from:

- changing contexts;
- changing expert parameters;
- changing capacity;
- optimizer-state drift;
- changing token mix;
- reward-model drift.

Those mechanisms can produce different regret problems.

45.29 Bandit feedback changes the online problem

Under bandit feedback, the router sees only the loss of the accepted action.

It does not immediately know whether another expert would have been better.

This is why exploration matters in adversarial bandits (Auer, Cesa-Bianchi, Freund, et al. 2002).

The exact witness computes realized regret after the loss table is fixed; it does not claim the online policy knows the hindsight-optimal route.

45.30 Delayed feedback weakens correction

If routing loss arrives late, the controller can spend several rounds without seeing a mistake.

That changes the operational value of correction capacity.

The exact witness uses immediate post-round feedback.

Delayed-feedback guarantees require separate analysis.

45.31 Hindsight comparator is not an executable oracle

The comparator may use the full realized sequence to define the benchmark.

The online router cannot.

Regret measures the gap to hindsight performance.

It does not prove the comparator was available as a real-time policy.

45.32 Downstream task loss is still separate

Routing loss can be defined from:

- latency;
- communication;
- expert quality;
- capacity violation;
- token-level prediction loss;
- a weighted combination.

A low routing regret under one declared loss does not automatically imply low end-to-end task loss under another metric.

An interface theorem or empirical relation is needed.

45.33 A practical routing-decision record

A regret-bearing routing experiment should record at least:

- context/token identity or sampling rule;
- router probability/logit state;
- preferred route;
- feasible action set;
- capacity/overflow state;
- accepted dispatch;
- incurred loss/reward;
- feedback revealed;
- comparator class;
- switch/churn metric;
- optionality;
- correction-capacity state;
- regret notion;
- downstream task metric separately.

Without these fields, “router regret” is underspecified.

45.34 Durable non-implications

REGRETRROUTE rejects:

low churn $\nRightarrow$ low regret,

high churn $\nRightarrow$ high regret,

balanced load $\nRightarrow$ low regret,

stable probabilities $\nRightarrow$ low regret,

high optionality $\nRightarrow$ low regret,

and:

low routing regret $\nRightarrow$ low downstream task loss without an explicit bridge.

45.35 Closing view

A router has dynamics before it has regret.

Once routing is treated as online decision making, the burden becomes explicit:

- name the executed action;
- constrain the feasible set;
- state when loss arrives;
- state what feedback is revealed;
- define the comparator;
- define optionality and correction capacity;
- state what kind of nonstationarity is allowed.

The four-round witness then gives the smallest useful lesson:

(A, A, A, A)

has zero churn but regret two, while:

(A, B, A, B)

has three switches but regret zero under the same common frame.

So the right conclusion is not “stable routing is good” or “adaptive routing is good.”

It is:

routing quality is comparator-, loss-, feedback-, feasibility-, and capacity-relative.

45.36 References used in this chapter

- (Auer, Cesa-Bianchi, Freund, et al. 2002)
- (Herbster and Warmuth 1998)

Exact source identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-REGRETROUTE-001.yaml`

Chapter 46

A Taxonomy of Machine Memory

Epistemic status: established memory mechanisms plus Atlas synthesis and exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-MEMTAX-001.md

Formal packet: mathematics/derivations/ATLAS-CH-MEMTAX-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-MEMTAX-001.md

Source lock: sources/source-locks/ATLAS-CH-MEMTAX-001.yaml

A machine does not have one thing called memory.

It has states that live in different places, survive for different lengths of time, are written by different mechanisms, and can be reached through different kinds of address.

A model's parameters can carry information across deployments.

A context can carry information across one inference.

A cache can survive seconds.

A replay buffer can preserve individual experience.

A database can survive model replacement.

A vector index can recover a record because its content resembles a query even when the caller does not know its exact key.

Calling all of these “memory” is useful only if we preserve the distinctions.

This chapter builds that vocabulary.

46.1 Memory is an interface, not merely a medium

Suppose two systems store exactly the same records.

One lets us retrieve them only by exact identifier.

The other lets us retrieve them by similarity.

The storage is the same. The memory behavior is different.

Now reverse the example.

One system stores a fact in model parameters.

Another stores the same fact in a database.

Both may answer the same query, yet their write paths, update costs, provenance, mutability, and failure modes are radically different.

So the first question should not be merely:

Where is the memory?

It should be:

What memory contract does this state satisfy?

For the Atlas, write a memory system as

$$M = (L, W, R, T, A, U, P, S).$$

Here:

- L is storage locus;
- W is the write or update path;
- R is the read or retrieval path;
- T is lifetime and retention policy;
- A is addressability;
- U is mutability and update cadence;
- P is provenance;
- S is sharing and synchronization scope.

The familiar memory labels are patterns across these coordinates.

They are not six boxes into which every implementation must fit exactly once.

46.2 Six roles, not six rungs

The chapter uses six recurring labels:

parametric memory;

working memory;
 episodic memory;
 semantic memory;
 associative memory;
 external memory.

They do not form a hierarchy.

They are not even all the same kind of category.

Parametric and external primarily describe where state lives and how it is updated.

Working primarily describes active availability and lifetime.

Episodic and semantic primarily describe how stored information relates to experience.

Associative primarily describes how memory is addressed.

A single store can therefore occupy several roles at once.

An external vector store of timestamped experiences can be external, episodic, and associative simultaneously.

That overlap is not a defect in the taxonomy.

It is the point.

46.3 Parametric memory

A trained model carries information in its parameters.

Let θ denote the parameter state.

A training or editing procedure changes it:

$$\theta' = W_{\text{param}}(\theta, \text{data}, \text{objective}, \text{update_rule}).$$

A forward pass reads from that state implicitly:

$$y = f_{\theta}(x).$$

There may be no explicit record saying that one fact is stored at one address.

Information can be distributed across many parameters and activated only by particular inputs.

This is what the chapter means by parametric memory.

Empirical work on language models has shown that factual associations can sometimes be elicited without supplying an external knowledge source at inference time.

That is evidence that parameters can function as a memory substrate.

It is not evidence that parameters behave like a clean database.

A stored association can be hard to locate.

A correction can interfere with other behavior.

Provenance may be diffuse.

Freshness depends on retraining, editing, or replacement.

And the model may generate a plausible relation that was never stored as a stable factual record.

Parametric memory is powerful partly because storage and computation are fused.

That same fusion makes explicit memory management difficult.

46.4 Working memory

Some state exists only because a computation is currently in progress.

A recurrent hidden state.

An active context.

A scratch buffer.

A temporary plan.

A cache of intermediate keys and values.

These are examples of machine working memory when they remain directly available to the current computation but are not intended to survive indefinitely.

The term has a cognitive-science lineage. Baddeley and Hitch used working memory for limited active storage and processing in human cognition.

The Atlas borrows the engineering intuition, not the biological architecture.

For machines, the operational question is:

what state is actively available now, and when will it disappear?

Working memory is often cheap to read because it is already in the active computation.

It is also easy to lose.

A context can be truncated.

A process can restart.

A cache can be evicted.

A session can end.

High immediate availability and long-term persistence are different properties.

46.5 Context is not durable memory

Modern systems often call context memory.

That can be reasonable.

But context residence should not be confused with persistence.

If a fact appears in the current prompt, the model can use it.

If the next session begins without that prompt, the fact may be gone.

The same applies to hidden state and caches.

A useful test is:

if the process is destroyed and reconstructed, does the state still exist somewhere from which it can be recovered?

If no, the state may be working memory without being durable memory.

A system that repeatedly compiles durable state into an active context therefore has at least two memory layers, not one.

46.6 Episodic memory

An episode remembers an event as an event.

A machine episodic record might contain:

event identity;

time;

task;

observation;

action;

outcome;

local context;

provenance.

Write one record as

$$e_i = (\text{id}_i, \text{time}_i, \text{context}_i, \text{payload}_i, \text{provenance}_i).$$

The key feature is event binding.

If we later ask,

what happened on attempt 17?

we are asking an episodic question.

If we ask,

what general rule did the last thousand attempts support?

we are asking for something closer to semantic consolidation.

Neural Episodic Control is a concrete machine-learning example of rapidly written experience records used in decision making.

The Atlas uses it as a mechanism example.

It does not infer that every replay buffer or event log has the same semantics.

46.7 Semantic memory

Semantic memory stores generalized content that can be used without reconstructing one originating event.

Suppose many episodes support a relation:

objects of type X usually require operation Y.

A consolidation procedure can produce a reusable knowledge object:

$$K = C(\{e_i\}_{i \in I}).$$

The new object may be more compact and easier to reuse than the raw episodes.

But consolidation can destroy information.

Which event first supported the claim?

Which examples were exceptions?

How uncertain was the conclusion?

Which source supplied the observation?

A semantic representation may preserve the relation and discard the path that produced it.

This is why semantic memory and provenance must remain separate coordinates.

The term semantic memory comes from the episodic/semantic distinction associated with Tulving.

Again, the Atlas imports a useful distinction in information organization.

It does not claim that a knowledge graph, parameter tensor, or vector store reproduces a human semantic-memory system.

46.8 Episodic and semantic are not storage media

The same physical store can contain both.

A database can hold raw event records and consolidated facts.

A parameterized model can acquire abstractions from many episodes.

An external memory can store summaries whose provenance links back to source events.

Thus episodic and semantic are not answers to:

where are the bits?

They are answers to:

what relationship does the stored object preserve to the experience that produced it?

Representation role and storage substrate should not be collapsed.

46.9 Associative memory

Exact addressing asks for a known location or key.

Associative addressing asks for a memory because the query resembles its content or pattern.

Hopfield's classic content-addressable model makes this distinction concrete.

A partial or noisy cue can drive the system toward a stored pattern.

In modern systems, association may be implemented through nearest-neighbor search, attention, learned key matching, hashing, pattern predicates, energy minimization, or hybrid symbolic/vector indices.

The Atlas therefore treats associative memory primarily as an access contract.

A memory can be both episodic and associative.

It can be both semantic and associative.

It can even be parametric and exhibit associative behavior.

The access path does not uniquely identify the storage locus.

46.10 One store, three memory behaviors

The computational witness makes this exact.

Store three immutable records.

A has identifier A, vector (1,0), time 1, and value red.

B has identifier B, vector (0,1), time 2, and value blue.

C has identifier C, vector (1,1), time 3, and value green.

Nothing about the stored records changes.

Now change only the read operator.

Ask for exact identifier B.

The answer is blue.

Ask for the record nearest to

$q=(4/5,1/5)$.

The squared distances to A, B, and C are respectively

$2/25$, $32/25$, $17/25$.

So the associative answer is A, red.

Now ask for episodes with time at least 2.

The result is B and C.

The latest is C, green.

Same store.

Three access contracts.

This is why a memory taxonomy that looks only at storage media is incomplete.

46.11 External memory

External memory places state outside the current model parameters and transient active state. Externality is a locus property; persistence is a separate lifetime choice.

That can include files, databases, journals, knowledge graphs, vector stores, tuple spaces, object stores, or differentiable memory matrices.

“External” is relative to the model/process boundary.

It does not necessarily mean remote over a network.

Memory Networks, Neural Turing Machines, and Differentiable Neural Computers all make external memory an explicit part of the learned computational architecture.

Their mechanisms differ.

The shared idea is that stored state can be read and written through an interface distinct from ordinary parameter access.

That separation opens possibilities unavailable to purely parametric memory:

rapid writes;

explicit record identity;

large persistent stores;

different retention policies;

independent indexing;

provenance;

sharing across models.

It also creates new systems problems.

46.12 External does not mean trustworthy

A database can contain false data.

A vector store can return the wrong neighbor.

A file can be stale.

A journal can contain a bad event with perfect provenance.

A knowledge graph can encode an obsolete relation.

An external memory can be unavailable during a partition.

The important gain is controllability of the memory interface.

Truth remains a separate question.

External memory is sometimes described as if retrieval itself solved hallucination or knowledge reliability.

It does not.

It can make information more inspectable, replaceable, and independently up-dateable.

Whether the information is correct still depends on evidence, provenance, validation, and retrieval quality.

46.13 Memory Networks and explicit long-term state

Memory Networks were designed around a long-term memory component used jointly with learned inference.

That is an important architectural separation.

Instead of forcing every relevant fact into parameters or current input, the system can maintain a memory object that is read and written as part of the task.

Inference asks what to do with available memory.

Memory management asks what to store and how to expose it.

These can be learned jointly while remaining conceptually distinct.

That separation becomes essential in later chapters, where retrieval quality and continual update must be analyzed independently.

46.14 Differentiable external memory

Neural Turing Machines and Differentiable Neural Computers expose explicit read/write memory to a neural controller and make addressing differentiable enough to learn from data.

This demonstrates that memory management can itself become part of learned computation.

But differentiability does not erase memory semantics.

A read head still has an address rule.

A write still changes stored state.

A usage policy still determines overwrite.

A link structure still changes what sequences can be recovered.

The memory is learnable infrastructure, not magic persistence.

46.15 Lifetime is a separate axis

Consider four pieces of information:

a token in a current context;

a record in an episodic buffer;

a fact encoded in model weights;

a row in a durable database.

Their lifetimes can differ by many orders of magnitude.

But longer is not automatically better.

A stale fact persisting forever can be worse than a fresh fact retained briefly.

A replay buffer that never evicts may become computationally unusable.

A context that keeps every previous token may exceed its budget.

Retention is therefore a policy.

The taxonomy records lifetime T separately from semantic role.

46.16 Writes have different costs

Parametric writes can require training, editing, or optimization.

Working-memory writes can happen every step.

Episodic writes may be append-like.

Semantic writes may require consolidation.

External stores may support transactional updates.

These differences determine how quickly a system can incorporate new information.

A system that knows something only after retraining has a very different adaptation loop from one that can append a validated record immediately.

This is one reason Continual Learning depends on this taxonomy.

Continual learning is not only about retaining old information.

It is about how different write mechanisms interact over time.

46.17 Provenance can be preserved or destroyed

Suppose a memory contains:

Paris is the capital of France.

The payload alone does not tell us:

who asserted it;

when;

from which source;

under which transformation;

with what evidence;

whether the source has since been superseded.

An episodic record can retain those fields.

A semantic consolidation step can drop them.

A parameterized model can make them difficult to recover.

An external governed store can preserve them explicitly.

This makes provenance a first-class memory coordinate rather than decorative metadata.

A memory system that preserves provenance supports different downstream claims from one that stores only payloads.

46.18 Shared memory introduces coordination

A memory can be private to one process.

It can be shared across agents.

It can be replicated.

It can be transactional.

It can be eventually synchronized.

These choices belong to the sharing coordinate S.

Once several actors write to one memory, the Coordination chapter's concerns return:

ordering;

atomicity;

duplicate effects;

stale views;

partial failure.

Memory architecture and coordination architecture therefore meet at shared state.

But they remain distinct.

A shared store tells us where state resides and who can access it.

Coordination semantics tell us how concurrent access behaves.

46.19 Failure by memory role

Different roles fail differently.

Parametric memory can become stale, entangled, or difficult to edit without interference.

Working memory can overflow, truncate, or vanish.

Episodic memory can duplicate events, lose identity, or grow without bound.

Semantic memory can overgeneralize or lose provenance during consolidation.

Associative memory can return false neighbors or suffer representation drift.

External memory can become unavailable, inconsistent, stale, or disconnected from the process that needs it.

These are not one failure called forgetting.

A useful memory diagnosis begins by identifying which coordinate failed.

46.20 Memory is not knowledge

A stored object may be a fact, a hypothesis, a failed attempt, a prediction, a raw observation, a proof, or an obsolete record.

The memory system does not decide which is true merely by retaining it.

The Evidence chapter taught us to separate claim from support.

The same discipline applies here.

Memory is infrastructure for persistence and access.

Knowledge is an epistemic status that requires more.

This is particularly important for agent systems that retrieve old outputs from themselves.

Self-retrieval is not self-verification.

46.21 What Retrieval may assume

ATLAS-CH-RETRIEVAL-001 can now start from a stable distinction between storage and access.

It may assume:

$M=(L,W,R,T,A,U,P,S);$

exact-key versus associative addressing;

parametric versus external locus;

working versus durable lifetime;
episodic versus semantic organization;
provenance as an independent coordinate.

Its job is to develop retrieval itself:

vector search;
symbolic lookup;
hybrid retrieval;
multi-index access;
key-value memory;
ranking and selection.

It need not rebuild the taxonomy.

46.22 What Continual Learning may assume

ATLAS-CH-CONTINUAL-001 may assume:

parametric write semantics;
episodic records;
semantic consolidation;
retention and mutability coordinates;
the distinction between preserving records and preserving behavior.

Its job is to study what happens when learning continues:

interference;
catastrophic forgetting;
replay;
consolidation;
regularization;
parameter isolation.

Again, the taxonomy becomes useful because “remembering” can now mean several different things.

46.23 The boundary to remember

The most useful question is not:

Does the system have memory?

Almost every nontrivial adaptive system does.

The useful questions are:

Where does the state live?

How is it written?

How is it read?

How long does it survive?

How is it addressed?

How can it change?

What provenance survives with it?

Who else can see or modify it?

Once those coordinates are explicit, the familiar labels become precise.

Parametric memory tells us that information is carried in learned parameters.

Working memory tells us that state is actively available for current computation.

Episodic memory tells us that event identity matters.

Semantic memory tells us that generalized content can survive without the full originating episode.

Associative memory tells us that access can be content-driven.

External memory tells us that state can live outside the current model and active context; whether it is durable is a separate retention decision.

None of these labels is the whole memory system.

Together they give us enough structure to ask the next question:

given that something has been stored, how should the system find the right thing again?

That is where retrieval begins.

46.24 References used in this chapter

Exact source identities and authority scopes are pinned in:

`sources/source-locks/ATLAS-CH-MEMTAX-001.yaml`

The external basis includes Baddeley and Hitch for working-memory terminology lineage, Tulving for episodic/semantic terminology lineage, Hopfield for content-addressable memory, Memory Networks, Neural Turing Machines, Differentiable Neural Computers, Neural Episodic Control, and empirical work on factual associations stored in language-model parameters.

Chapter 47

Retrieval and Associative Access

Epistemic status: established information-retrieval/database/vector-search foundations + audited Memory Taxonomy and Attention prerequisites + Atlas synthesis + exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-RETRIEVAL-001.md

Formal packet: mathematics/derivations/ATLAS-CH-RETRIEVAL-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-RETRIEVAL-001.md

Source lock: sources/source-locks/ATLAS-CH-RETRIEVAL-001.yaml

A memory is useful only if a system can find the right part of it.

That sounds obvious.

The difficulty begins when we ask what **right** means.

A record can be right because:

- its exact key matches;
- its metadata satisfies a predicate;
- its vector is nearest to the query;
- its lexical score is highest;
- it survives a symbolic filter;
- it wins a fused rank;
- it is recent;
- it comes from an admissible source.

These are different retrieval contracts.

The governing rule of this chapter is:

retrieval is a declared read operation over records, not a synonym for relevance, truth, provenance, or usefulness.

The same immutable records can support several retrieval behaviours.

That is not inconsistency.

It is the architecture.

47.1 Retrieval starts where the memory taxonomy stops

The Memory Taxonomy chapter wrote a memory object as

$M = (L, W, R, T, A, U, P, S)$

for storage locus, write path, read path, lifetime, addressability, mutability, provenance, and sharing.

Retrieval is chiefly about the read side:

R

and the addressability coordinate:

A.

But the other coordinates still matter.

A result may be easy to retrieve but stale.

A result may be relevant but lack provenance.

An index may be external while the record is ephemeral.

A query may be associative while the store itself is exact-key addressable.

So retrieval must not collapse the rest of the memory contract.

47.2 A retrieval contract

Let $\mathcal{D}$ be the underlying record set.

Write a retrieval contract as

$$\text{Retr} = (\mathcal{D}, \mathcal{Q}, F, s, \pi, k, \mathcal{O}).$$

Here:

- $\mathcal{Q}$ is the query space;
- $F(q, \mathcal{D})$ is an optional admissibility/filter operator;
- $s(q, d)$ is a score, distance, or match relation;

- π is a ranking/selection rule;
- k is a truncation or budget;
- $\mathcal{O}$ is the returned projection or set of fields.

This representation makes one fact explicit:

a retrieval result is defined by more than a score.

The candidate set matters.

The filter matters.

The tie rule matters.

The top-k budget matters.

The fields returned matter.

Two systems can use the same embeddings and still implement different retrieval semantics.

47.3 Exact-key lookup

The simplest access contract is exact addressing.

Let every record carry a unique key:

`key(d) in K.`

Then exact-key retrieval asks for the unique record satisfying

`key(d)=u.`

This is not approximate.

It is not semantic.

It does not ask whether the key's record resembles the query.

A key is an address.

Dynamo is a large-scale systems example of key-value access in which a caller supplies a key and the system manages distributed storage and availability around that interface (DeCandia et al. 2007).

This chapter uses only the access distinction.

Replication, versioning, failure recovery, and consistency policies are separate systems concerns.

47.4 Key-value retrieval is not semantic retrieval

Suppose key

invoice-2026-1042

maps to a record.

The lookup can be exact even if:

- the record is obsolete;
- the payload is wrong;
- another record would be semantically more useful;
- the record has poor provenance.

Exactness refers to the address relation.

It does not certify the payload.

This distinction will recur throughout the chapter.

47.5 Symbolic retrieval

A symbolic query specifies conditions over structured fields.

Let

$P_q(d)$

be a predicate compiled from query q .

Then symbolic retrieval returns

$\{d \text{ in } D : P_q(d)\}$.

Examples include:

- `year >= 2024`;
- `type = "paper"`;
- `source_id in approved_sources`;
- `status = "audited"`;
- conjunctions and disjunctions of structured conditions.

Codd's relational model provides the foundational idea that data can be addressed through declarative relations and operations rather than by exposing physical storage layout (Codd 1970).

The Atlas takes a narrow lesson:

symbolic admissibility can be specified independently of physical storage.

47.6 A predicate is not a ranking

The result of a predicate can be a set.

If a caller needs an ordered list, another rule is required.

For example:

- sort by date;
- sort by score;
- sort by source priority;
- rank by a learned model;
- apply a deterministic record-ID tie break.

So a system that says

retrieve all approved papers

has specified admissibility.

It has not yet specified which approved paper should come first.

Selection and ranking are distinct operators.

47.7 Vector retrieval

Vector retrieval maps queries and records into a shared coordinate space.

Let

$$\phi_Q(q) \in \mathbb{R}^m$$

and

$$\phi_D(d) \in \mathbb{R}^m.$$

Then a distance-based retrieval rule can be written as

$$d^* \in \arg \min_{d \in \mathcal{D}} \delta(\phi_Q(q), \phi_D(d)).$$

Or a similarity rule can maximize a score.

Salton, Wong, and Yang's vector-space model is a foundational information-retrieval construction in which stored objects and queries are compared through vector representations (Salton et al. 1975).

Modern learned embeddings change how the vectors are obtained.

They do not change the core fact that ranking is relative to a representation and a similarity geometry.

47.8 Similarity is representation-relative

Suppose two embedding models map the same records to different coordinates.

The nearest record can change.

Suppose the embeddings remain fixed but cosine similarity is replaced by Euclidean distance.

The ranking can change again.

Suppose vectors are normalized before scoring.

The ranking may change again.

Therefore the statement

record A is nearest

is incomplete without:

- query embedding;
- record embedding;
- normalization rule;
- distance or similarity;
- candidate set.

Nearest is never representation-free.

47.9 A similarity score is not automatically relevance probability

A dense retriever can emit a score:

$s(q, d)$.

That score can be useful for ordering candidates.

But unless a separate calibration argument is established, it does not mean

$P(\text{relevant} \mid q, d)$.

Scores may be:

- inner products;
- cosine similarities;
- negative distances;
- logits;
- learned ranking outputs.

Their numerical scale can be arbitrary.

A score of 0.9 need not mean 90 percent relevant.

Ranking and probability calibration are different tasks.

47.10 Top-k is a budgeted truncation operator

Suppose candidate order is

$d_{(1)}, \dots, d_{(n)}$.

Top-k returns the ordered list

$(d_{(1)}, \dots, d_{(k)})$.

This is a computational and interface decision.

It does not prove:

- the omitted records are irrelevant;
- the returned records are all relevant;
- the k-th score marks a natural semantic boundary.

Top-k converts an ordering into a bounded ordered working list; its underlying selected set is a separate view when set semantics are needed.

That is useful precisely because downstream systems cannot consume everything.

47.11 Tie rules matter for exact replay

Two records can have the same score.

If the system needs reproducible ordered output, it must declare a tie rule.

Examples include:

- stable record ID;
- insertion order;
- timestamp;
- secondary score.

Without such a rule, two conforming implementations can return different top-k lists while agreeing on all primary scores.

This is not always a problem.

But it matters when exact replay is part of the evidence contract.

47.12 Exact nearest neighbour

Exact nearest-neighbor search asks for the global optimum under the declared metric and candidate set.

For query q :

$\text{NN}(q, D) = \operatorname{argmin}_{\{d \in D\}} \delta(q, d).$

If the store is small, exhaustive comparison may be acceptable.

At large scale, exhaustive search can become too expensive.

That motivates indexes and approximation.

47.13 Approximate nearest neighbour

Approximate nearest-neighbor search changes the search procedure.

The target remains similarity search.

The algorithm accepts some risk of missing the exact global nearest candidate in exchange for improvements such as:

- lower latency;
- lower query compute;
- better scalability;
- bounded memory structures;
- practical billion-scale search.

Malkov and Yashunin’s HNSW algorithm is one graph-based approximate nearest-neighbor construction (Malkov and Yashunin 2020).

Its layered navigable graph is a search structure.

The crucial Atlas boundary is:

an ANN result is not silently identical to the exact nearest-neighbor result.

47.14 Search approximation and representation error are different

Imagine the truly task-relevant record is semantically important but badly embedded.

Exact nearest-neighbor search can retrieve the wrong semantic record perfectly.

That is not an index failure.

The index exactly solved the wrong geometric proxy.

Now reverse the case.

Suppose the embedding geometry puts the relevant record closest to the query, but an approximate index misses it.

That is a search/index approximation failure.

So separate:

1. representation error;
2. search/index error;
3. downstream relevance error.

Without this separation, retrieval diagnostics become confused.

47.15 Sparse and lexical retrieval

Not every retrieval system needs dense embeddings.

Classical text retrieval can score overlap and term evidence through sparse representations.

The probabilistic relevance framework and BM25 family are an important lineage for such ranking (Robertson and Zaragoza 2009).

Sparse lexical systems can be attractive because their matching evidence is often easier to inspect:

- term occurrence;
- term frequency;
- document frequency;
- field structure.

Dense systems can recover conceptual proximity that exact lexical overlap misses.

Neither representation dominates universally.

47.16 Symbolic, sparse, and dense are different axes

It is tempting to place all retrieval into two boxes:

- lexical;
- vector.

That misses structured predicates.

A symbolic filter such as

```
source="primary" AND year>=2024
```

does not need to behave like BM25 or embedding similarity.

It declares admissibility.

A complete retrieval architecture may therefore combine:

- symbolic filtering;
- sparse lexical scoring;

- dense similarity;
- recency weighting;
- source constraints.

The composition rule is the system.

47.17 One store can have many indices

A record can be reachable through:

- primary key;
- metadata index;
- inverted lexical index;
- vector index;
- temporal index;
- provenance index.

These do not necessarily duplicate the logical record.

They are access structures.

This gives a useful invariant:

`record_id != index_entry_id.`

One logical record may have several index entries.

Changing or deleting the record can therefore require coordinated index maintenance.

47.18 Multi-index retrieval

The phrase **multi-index** can mean more than one thing.

At the broad architecture level, the Atlas uses it for multiple access structures over one record set.

At the algorithm level, Babenko and Lempitsky's inverted multi-index is a specific similarity-search construction based on product quantization (Babenko and Lempitsky 2012).

These meanings should not be conflated.

The paper establishes one concrete structured vector-search index.

The Atlas generalization is an architectural synthesis:

one record identity can participate in several independent retrieval access paths.

47.19 Hybrid retrieval requires composition semantics

The word **hybrid** is often too vague.

A hybrid retriever might mean:

- symbolic filter, then vector rank;
- lexical candidate generation, then dense reranking;
- dense and sparse score fusion;
- vector candidates intersected with policy constraints;
- several retrievers feeding a learned reranker.

These pipelines are not equivalent.

A serious description should state the operator order.

47.20 Exact witness store

Use three immutable records.

A:

- $\text{id} = A$;
- $\text{kind} = \text{paper}$;
- $\text{year} = 2024$;
- $\text{vector} = (1, 0)$;
- $\text{value} = \text{alpha}$.

B:

- $\text{id} = B$;
- $\text{kind} = \text{paper}$;
- $\text{year} = 2022$;
- $\text{vector} = (4/5, 3/5)$;
- $\text{value} = \text{beta}$.

C:

- $\text{id} = C$;
- $\text{kind} = \text{note}$;
- $\text{year} = 2025$;
- $\text{vector} = (24/25, 1/25)$;
- $\text{value} = \text{gamma}$.

Query vector:

$\mathbf{q} = (19/20, 1/20)$.

Use squared Euclidean distance.

Nothing about the records changes across the following reads.

47.21 Exact vector distances

To A:

$$d(q, A)^2 = 1/200.$$

To B:

$$d(q, B)^2 = 13/40.$$

To C:

$$d(q, C)^2 = 1/5000.$$

Therefore:

C, A, B

is the exact vector ranking.

Unfiltered top-1 is C.

47.22 Exact-key read of the same store

Ask for:

id=B.

The answer is B.

Vector proximity is irrelevant.

The store did not change.

The access contract did.

This is the same structural lesson established abstractly by MEMTAX-001, now made specific to retrieval.

47.23 Symbolic read of the same store

Ask for:

year>=2024.

A qualifies.

B does not.

C qualifies.

So the result set is:

{A, C}.

This query defines admissibility but not an order.

A secondary ranking clause would be required to choose which record comes first.

47.24 Hybrid retrieval does not generally commute

Now require:

`kind=paper`

and also rank by vector distance.

There are at least two plausible pipelines.

47.24.1 Filter then rank

First keep papers:

`{A,B}`.

Then rank by distance.

A is closer.

Top-1 result:

A.

47.24.2 Rank then filter

First rank the full store.

Top-1 is C.

Then apply the paper predicate.

C fails.

The result is empty.

Thus:

`FilterThenRank != RankThenFilter`.

The difference arises only from operator order and top-k truncation.

47.25 Why filter-then-rank often has cleaner semantics

If a symbolic predicate means

only these records are admissible,

then applying it before ranking ensures the ranking operates inside the permitted set.

This is particularly important for constraints such as:

- approved sources;
- tenant boundaries;
- date windows;
- document status;
- legal access permissions.

But the Atlas does not declare filter-first universally superior.

Other pipelines can be legitimate.

The requirement is to state what the operators mean.

47.26 Score fusion is another hybrid family

Suppose a document has sparse score

`s_sparse(q,d)`

and dense score

`s_dense(q,d)`.

A fused ranker may use:

`s_hybrid = lambda s_sparse + (1-lambda)s_dense`

after suitable normalization.

This immediately creates new questions:

- are the score scales commensurate?
- how is `lambda` chosen?
- what happens when one retriever has no score?
- are ranks fused instead of raw scores?
- is the rule query-dependent?

Calling the system hybrid does not answer them.

47.27 Retrieval can be staged

A large corpus may use a cheap first stage to generate candidates and a more expensive second stage to rerank them.

Write:

`D -> C_k(q) -> Rank_expensive(C_k(q)).`

This reduces cost.

It can also introduce a hard ceiling:

if the relevant record never enters the candidate set, the reranker cannot recover it.

Candidate recall and reranker quality are therefore separate quantities.

47.28 Attention and retrieval overlap at the operator level

ATTNOP-001 showed attention as a state-dependent operator built from query-key scores and values.

Retrieval can share the pattern:

`query -> candidate scores -> selection -> returned values.`

This resemblance is real.

It does not make attention and retrieval architecturally identical.

47.29 Retrieval can exceed the active field

Attention normally operates over the keys and values supplied to the active computation.

A retriever can search:

- a database;
- a document collection;
- a vector index;
- a file store;
- a shared organizational memory.

The persistent corpus can be much larger than the active context.

Retrieval therefore often acts as the bridge between long-lived memory and bounded working context.

47.30 Retrieval can use hard identity

Attention weights usually mix supplied values.

Retrieval can instead return explicit record identities.

That creates capabilities such as:

- source citation;
- stable replacement;
- deletion;

- access control;
- provenance lookup;
- deduplication.

The record can remain independently addressable after retrieval.

This becomes central in the External-Memory chapter.

47.31 Retrieval is not causal explanation

A high retrieval score says:

this candidate ranked highly under this retrieval contract.

It does not say:

this record caused the final output.

A downstream model may:

- ignore it;
- contradict it;
- combine it with other records;
- distort it;
- fail for unrelated reasons.

ATTN0P-001 already enforced the analogous boundary for attention coefficients.

The same discipline applies here.

47.32 Retrieval and provenance

Suppose record A ranks first.

The score does not tell us:

- where A came from;
- who authored it;
- whether it has been superseded;
- which transformation produced its embedding;
- whether it was audited.

Those belong to the record/provenance system.

A governed retriever should preserve a path from result to stable record identity and source metadata when downstream claims depend on provenance.

47.33 Rank does not create authority

A record can rank first because its vector is close.

That does not promote it to:

- verified;
- approved;
- authoritative;
- current.

A retrieval index is not an adjudicator.

The Evidence and Replay chapters remain relevant whenever retrieved material is used to support claims.

47.34 Retrieval correctness, relevance, and usefulness

Three distinct questions are:

1. **contract correctness:** did the retriever return what its algorithm specifies?
2. **task relevance:** does the record bear on the user's task?
3. **downstream usefulness:** does giving the record to the consumer improve the objective?

A result can be correct under the retrieval contract and irrelevant to the task.

A relevant record can be too long for the available context.

A useful summary can rank below a less useful full document under raw similarity.

Evaluation should therefore name which layer it measures.

47.35 Recall and precision require a relevance judgment

Information retrieval metrics such as precision and recall need a declared notion of relevance.

For retrieved set S and relevant ground-truth set G :

$$\text{precision} = \frac{|S \cap G|}{|S|}$$

and

$$\text{recall} = \frac{|S \cap G|}{|G|}$$

when denominators are nonzero.

The formulas are simple.

The hard question is how $\mathbf{G}$ is defined.

Human labels, task outcomes, source policies, and benchmark annotations can produce different relevance notions.

Metrics inherit that choice.

47.36 ANN recall is not semantic recall

Approximate-nearest-neighbor evaluation often asks whether the ANN method recovered exact nearest neighbors under a metric.

That is an index-quality question.

Semantic retrieval recall asks whether task-relevant records were found.

These should not be conflated.

An ANN system can have perfect recall relative to exact vector nearest neighbors while the vector model itself has poor semantic recall.

Again:

representation and search are separate layers.

47.37 Retrieval-augmented generation is downstream composition

Lewis et al. combine a parametric generator with a dense vector index used as non-parametric memory (Lewis et al. 2020).

This is a useful architecture example because it makes retrieval a distinct stage in a larger model.

The retrieval stage can be inspected separately from generation.

But retrieval augmentation does not create a theorem that generated answers are true.

The pipeline can fail at:

- query representation;
- index search;
- relevance;
- context assembly;
- generation;
- interpretation.

Each layer deserves its own evidence.

47.38 Retrieval is not memory writing

A read can be excellent while the memory is stale.

A memory can be correct while the retriever is weak.

A retriever can repeatedly surface one record without changing the store.

This preserves the MEMTAX distinction:

- write path;
- read path;
- lifetime;
- mutability.

Retrieval is one side of the memory interface.

47.39 Retrieval under updates

When records change, indexes may need maintenance.

A system should be able to state whether updates are:

- synchronous;
- asynchronous;
- transactional;
- eventually indexed;
- versioned.

A record can exist in the source store before it becomes visible to one index.

That creates a freshness gap.

The chapter records the issue but does not develop distributed consistency.

That belongs to systems/coordination layers.

47.40 Deletion is also a retrieval property

If a record is deleted from the canonical store but remains in an index, retrieval can surface a ghost.

If an index deletes the vector but the canonical record remains, one access path disappears while others survive.

So deletion semantics require:

- canonical record identity;
- index maintenance;
- visibility rules.

This is another reason record and index identity must remain separate.

47.41 Multi-index consistency

Suppose one record has:

- primary-key entry;
- lexical postings;
- vector node;
- time index.

An update to the record can leave these access structures temporarily inconsistent.

The retrieval layer should therefore distinguish:

- logical record version;
- index version;
- query-time snapshot.

The chapter does not prescribe a transaction protocol.

It makes the state distinction visible.

47.42 Retrieval budgets shape cognition

A downstream model rarely receives the full store.

It receives a working set.

The retrieval budget k therefore shapes what information becomes available for reasoning.

A small k can exclude complementary evidence.

A large k can consume context and introduce distractors.

So retrieval is not only a search problem.

It is a compilation boundary between persistent memory and active computation.

The later Context Compilation chapter will build on this.

47.43 The nearest record need not be the best context item

Suppose one highly similar document repeats the query without adding evidence.

A slightly less similar record may contain the decisive fact.

Vector similarity can therefore be a useful proxy without being the downstream utility function.

Possible downstream reranking signals include:

- novelty;

- coverage;
- provenance quality;
- contradiction;
- diversity;
- recency;
- task-specific expected utility.

The Atlas keeps these as separate design layers.

47.44 Diversification changes the objective

Top-k nearest-neighbor retrieval can return near-duplicates.

A diversified retriever may intentionally sacrifice some score to cover different aspects of a query.

Then its objective is no longer:

return the k highest individual scores.

It becomes a set-selection problem.

The distinction matters when evaluating retrieval quality.

Individual rank and set utility are different objects.

47.45 Retrieval over governed records

A governed store can expose symbolic fields such as:

- epistemic class;
- review state;
- source identity;
- supersession state;
- authority;
- validity interval.

These fields can participate in filters.

That makes retrieval policy-aware.

But the filter must not invent those statuses.

It can only consume statuses established elsewhere.

Retrieval applies governance metadata.

It does not create governance authority.

47.46 Failure modes

47.46.1 Address conflation

Treat exact key match as semantic relevance.

47.46.2 Similarity laundering

Treat nearest vector as correct answer.

47.46.3 Score calibration fiction

Interpret arbitrary ranking scores as probabilities.

47.46.4 Top-k completeness fiction

Treat omitted records as irrelevant.

47.46.5 ANN/exact conflation

Describe approximate search output as the exact global nearest neighbour without verification.

47.46.6 Hybrid ambiguity

Say “hybrid retrieval” without specifying operator order or fusion.

47.46.7 Index/record conflation

Treat multiple index entries as multiple independent records.

47.46.8 Provenance laundering

Infer source authority from retrieval rank.

47.46.9 Retrieval/generation conflation

Treat a retrieval-augmented answer as correct because retrieval occurred.

47.46.10 Read/write conflation

Treat a retrieval improvement as a memory freshness improvement.

47.47 A practical retrieval ledger

Before evaluating a retriever, record:

Field	Question
corpus	What records are eligible in principle?
record identity	What is the stable logical record ID?
query	How is the query represented?
filter	What makes a record admissible?
index	Which access structure is searched?
representation	Sparse, dense, symbolic, mixed?
score	What quantity orders candidates?
approximation	Exact or approximate search?
budget	What is k or the candidate limit?
tie rule	How are equal scores ordered?
projection	Which fields are returned?
provenance	How is source identity preserved?
relevance target	What labels or task define relevance?
downstream target	What consumer/usefulness metric matters?

This is the minimum grammar needed to interpret a retrieval result.

47.48 What the exact witness establishes

The companion witness proves four facts about one immutable finite store.

First:

`id=B`

returns B by exact key.

Second:

`year>=2024`

returns the symbolic set

`{A,C}`.

Third, vector ranking under the declared squared Euclidean geometry is:

`C,A,B`.

Fourth, with the paper predicate:

- filter-then-rank top-1 returns A;
- rank-top-1-then-filter returns nothing.

Therefore retrieval semantics depend on the read contract and operator order even before learned embeddings, ANN approximations, or large-scale infrastructure are introduced.

47.49 Downstream handoff

The External-Memory Thesis — ATLAS-CH-EXTMEM-001 may now assume:

- exact-key versus associative access;
- symbolic predicate retrieval;
- vector similarity retrieval;
- top-k semantics;
- ANN versus exact-NN distinction;
- multiple indices over one record identity;
- explicit hybrid-composition rules;
- provenance preservation;
- retrieval correctness versus relevance versus downstream usefulness.

The downstream chapter must independently argue:

- which information should leave model parameters;
- which memory loci should persist;
- how shared memory changes system architecture;
- what update/provenance/governance contract makes external memory preferable.

Retrieval provides access.

It does not by itself establish the case for externalization.

47.50 References used in this chapter

- Salton, Wong, and Yang, *A Vector Space Model for Automatic Indexing* (Salton et al. 1975).
- Codd, *A Relational Model of Data for Large Shared Data Banks* (Codd 1970).
- Robertson and Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond* (Robertson and Zaragoza 2009).
- Malkov and Yashunin, *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs* (Malkov and Yashunin 2020).
- Babenko and Lempitsky, *The Inverted Multi-Index* (Babenko and Lempitsky 2012).
- DeCandia et al., *Dynamo: Amazon’s Highly Available Key-value Store* (DeCandia et al. 2007).

- Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (Lewis et al. 2020).

Exact source identities and authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-RETRIEVAL-001.yaml`

Chapter 48

Continual Learning and Forgetting

Epistemic status: established continual-learning mechanisms + audited memory/optimization prerequisites + Atlas synthesis + exact scalar witness.

Specification: manuscript/specifications/ATLAS-CH-CONTINUAL-001.md

Formal packet: mathematics/derivations/ATLAS-CH-CONTINUAL-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-CONTINUAL-001.md

Source lock: sources/source-locks/ATLAS-CH-CONTINUAL-001.yaml

A learner can become better at what it sees now and worse at what it knew before.

That sentence contains the central difficulty of continual learning.

Ordinary optimization asks how to improve an objective.

Continual learning asks what happens when the objective, data distribution, label space, task context, or available evidence changes over time while the same learning system must preserve useful earlier capability.

The problem is not simply that parameters move.

Parameters are supposed to move.

The problem is that later updates can alter earlier behavior in ways the learner no longer has enough evidence, capacity, or structural protection to repair.

This chapter develops continual learning as a problem of **controlled interference under sequential updates**.

Its central rule is:

forgetting is a behavioral statement under an evaluation protocol,
not a synonym for parameter change.

That rule keeps the discussion grounded.

48.1 Sequential learning creates a history

Suppose contexts arrive in order

$1, 2, \dots, T$.

After training through context i , evaluate the current model on every context seen so far.

Write

$R_{\{i,j\}}$

for the score on context j after training through context i .

Then the basic object is not one number.

For retention and forgetting, the mandatory record is a lower-triangular performance-through-time matrix over contexts already encountered.

For three contexts it has the form

$R_{\{1,1\}}$

$R_{\{2,1\}}, R_{\{2,2\}}$

$R_{\{3,1\}}, R_{\{3,2\}}, R_{\{3,3\}}$.

If a protocol also evaluates not-yet-trained contexts, those $j > i$ entries can complete a full matrix for forward-transfer analysis. Forward transfer additionally needs a declared untrained or reference baseline for the future context; it cannot be inferred from the lower triangle above.

The performance record can reveal things a final average hides.

A later task may damage one earlier task and improve another.

A model may preserve average accuracy while destroying a rare context.

A method may learn new tasks slowly but retain old ones almost perfectly.

Another may adapt rapidly and forget sharply.

All of those behaviors can collapse to similar final averages.

So continual learning should begin from the trajectory of performance, not only its endpoint.

48.2 An Atlas forgetting summary

For a sequence with $T \geq 2$, and for an earlier context $j < T$, define its best prior score

$$B_j = \max_{k=j, \dots, T-1} R_{k,j}.$$

Define endpoint forgetting

$$F_j = \max(0, B_j - R_{T,j}).$$

Then define

$$F_{\text{avg}} = \frac{1}{T-1} \sum_{j < T} F_j$$

and

$$F_{\text{max}} = \max_{j < T} F_j.$$

These are Atlas summaries.

They are intentionally modest.

Cross-context aggregation is meaningful only when the context scores share a commensurate scale or a declared normalization. If different contexts use incomparable metrics or ranges, retain the per-context values instead of averaging or taking a cross-context maximum.

They do not claim to be the one canonical continual-learning metric.

Lopez-Paz and Ranzato emphasize that continual-learning evaluation should include transfer behavior, not only final task accuracy (Lopez-Paz and Ranzato 2017).

That matters because the nonnegative F_j definition deliberately clips improvement.

If later learning improves an earlier task, the signed change should remain available elsewhere in the evaluation record.

48.3 Parameter movement is not forgetting

Suppose a model moves from parameters

`theta_old`

to

`theta_new`.

The norm

`||theta_new-theta_old||`

does not tell us how much was forgotten.

A large movement may occur along directions that barely matter for an earlier task.

A small movement may cross a sensitive decision boundary and cause a large behavioral change.

Forgetting is therefore evaluated through retained performance, loss, calibration, reward, or another declared behavioral quantity.

Parameter-space movement becomes meaningful only after it is connected to that behavior.

This distinction is crucial for understanding consolidation methods.

They act in parameter space.

Their purpose is behavioral retention.

The two levels are related but not identical.

48.4 Why the word catastrophic exists

McCloskey and Cohen studied what they called catastrophic interference in connectionist networks under sequential learning (McCloskey and Cohen 1989).

The important phenomenon is not merely that an old score decreases.

It is that new learning can produce severe loss of earlier capability when the same distributed parameters are reused.

Modern deep-learning literature often uses **catastrophic forgetting** for the same broad family of failures.

The Atlas uses the word carefully.

A small noisy decline is not automatically catastrophic.

A report should state:

- what was learned first;
- what was learned later;
- how old performance changed;
- over what training interval;
- under what evaluation protocol.

Severity belongs in the evidence.

It should not be smuggled in by the adjective.

48.5 Continual learning is not one benchmark regime

A major source of confusion is that different continual-learning experiments provide different information to the learner at evaluation.

Van de Ven, Tuytelaars, and Tolias distinguish task-incremental, domain-incremental, and class-incremental scenarios (Ven et al. 2022).

48.5.1 Task-incremental learning

The system knows, or can unambiguously determine, which task is being evaluated.

Task-specific heads, masks, or modules can therefore be selected.

48.5.2 Domain-incremental learning

The input distribution or context changes, but the output semantics remain shared.

Explicit task identity need not be supplied at test time.

48.5.3 Class-incremental learning

The output problem grows.

The model must discriminate across a global set of classes accumulated over time.

These regimes differ materially.

A method that relies on a task selector has an easier inference contract than one that must infer among all accumulated classes.

So a continual-learning result should always say what context information is available.

48.6 Memory enters continual learning in different roles

The Memory Taxonomy chapter established that memory should be described by locus, write path, read path, lifetime, addressability, mutability, provenance, and sharing.

That discipline prevents an immediate mistake.

Replay is not a memory type.

Replay is a use of remembered information during learning.

The remembered material might live in:

- a local buffer;
- an external store;
- a generated model;
- a compressed exemplar set;
- another durable or ephemeral locus.

The continual-learning mechanism is the reintroduction of earlier evidence.

The memory architecture is a separate design decision.

48.7 Replay: put old evidence back into the objective

Let

$L_{\text{cur}}(\theta)$

be the current-context loss.

Let a memory M support an earlier-data loss

$L_M(\theta)$.

A simple replay objective is

$L_{\text{replay}}(\theta) = L_{\text{cur}}(\theta) + \alpha L_M(\theta)$.

The logic is direct.

If later optimization sees only the new context, it may move into a region that is excellent for the new objective and poor for the old one.

Replay makes earlier evidence visible again.

Rebuffi et al.'s iCaRL uses a bounded exemplar set in a class-incremental learning system (Rebuffi et al. 2017).

Gradient Episodic Memory also retains episodic examples, although it uses them differently (Lopez-Paz and Ranzato 2017).

Replay is powerful because it attacks one root cause of forgetting:

the optimizer otherwise cannot optimize against evidence it no longer sees.

48.8 Replay does not guarantee retention

Reintroducing old evidence is not the same as freezing old capability.

A replay buffer may be incomplete.

Its sampling distribution may be distorted.

Its weight in the loss may be too small.

Optimization may stop early.

The model may lack enough capacity to satisfy old and new objectives simultaneously.

The remembered examples may not represent the full earlier distribution.

Therefore replay changes the optimization problem in a retention-friendly direction.

It does not create an automatic theorem of zero forgetting.

48.9 Consolidation: preserve important parameter directions

A different strategy is to remember not the old examples themselves but information about which parameters were important for the old task.

Elastic Weight Consolidation is a canonical example.

After learning task A, let

θ_A^*

be the selected parameter point.

When learning B, EWC uses

$$L_{\text{EWC}}(\theta) = L_B(\theta) + (\lambda/2) \sum_i F_i (\theta_i - \theta_{A,i}^*)^2.$$

Kirkpatrick et al. motivate the weights F_i through a diagonal Fisher-information approximation to parameter importance (Kirkpatrick et al. 2017).

The geometry is intuitive.

Moving a parameter with large F_i is expensive.

Moving one with small F_i is cheaper.

The old solution behaves like an anisotropic spring anchor.

48.10 EWC is an approximation, not an oracle

The EWC penalty is useful precisely because storing and replaying all old data may be impossible or undesirable.

But compression has a price.

The old task is represented through:

- an anchor point;
- a local quadratic approximation;
- diagonal importance estimates.

That is far less information than the full old data distribution and full old loss surface.

The original EWC paper explicitly frames the diagonal Fisher construction as a tractable approximation (Kirkpatrick et al. 2017).

So the correct claim is not:

important parameters can never move.

It is:

estimated important directions are penalized more strongly.

That can reduce interference.

It does not guarantee exact behavioral preservation.

48.11 The stability-plasticity tension

If every old parameter were frozen forever, old behavior could be protected locally.

But new learning would eventually run out of freedom.

If every parameter remained fully plastic, adaptation would be easy but interference could be severe.

Continual learning therefore repeatedly faces a stability-plasticity tension.

How much should the system preserve?

How much should it change?

The answer depends on:

- capacity;
- task compatibility;
- available memory;
- context information;
- horizon;

- evaluation priorities.

The exact witness makes this tension visible with no stochastic noise.

48.12 Exact witness: two tasks, one scalar

Use one parameter

w .

Task A has loss

$$L_A(w) = (1/2)(w+1)^2.$$

Task B has loss

$$L_B(w) = (1/2)(w-1)^2.$$

Task A wants

$$w = -1.$$

Task B wants

$$w = 1.$$

There is no single scalar that makes both losses zero.

This is not an optimization failure.

It is a representational conflict.

The model has one degree of freedom and two incompatible optima.

48.13 Unprotected sequential learning

Train Task A first.

The optimum is

$$w = -1.$$

Then

$$L_A = 0.$$

Now train Task B to its own optimum.

The parameter moves to

$$w = 1.$$

Then

$$L_B = 0$$

but

$L_A=2$.

The old-task loss increased exactly by

2.

That is forgetting in the declared loss metric.

No randomness is involved.

No optimizer pathology is required.

Sequential interference follows from incompatible objectives sharing one parameter.

48.14 Exact quadratic consolidation

Set the scalar importance estimate to $F = 1$.

Use the retention objective

$$J_\lambda(w) = L_B(w) + \frac{\lambda}{2}(w+1)^2.$$

The exact minimizer is

$$w_\lambda = \frac{1-\lambda}{1+\lambda}.$$

At that point,

$$L_A(w_\lambda) = \frac{2}{(1+\lambda)^2}$$

and

$$L_B(w_\lambda) = \frac{2\lambda^2}{(1+\lambda)^2}.$$

For

$\lambda=0$,

we recover full adaptation to B:

$w=1, L_A=2, L_B=0$.

For

$\lambda=1$,

the compromise is

$w=0$, $L_A=1/2$, $L_B=1/2$.

For very large λ ,

w

approaches the old optimum and Task B becomes poorly fit.

The penalty therefore does exactly what a consolidation penalty should do in this toy:

it trades plasticity for retention.

48.15 Replay and EWC coincide here for a special reason

Now imagine exact replay with equal weight on both complete task objectives:

$L_{\text{joint}}=L_A+L_B$.

In this witness,

$L_{\text{joint}}(w)=w^2+1$.

The minimizer is again

$w=0$.

So equal replay and the $\lambda=1$, $F=1$ quadratic retention penalty produce the same solution.

This is not a general equivalence.

It happens because Task A's loss is itself exactly the quadratic used as the retention penalty.

In a real network:

- replay reevaluates remembered data under the current model;
- EWC uses a local parameter-space approximation around an earlier solution.

The exact coincidence is useful precisely because it shows both the connection and the boundary.

48.16 Gradient constraints: protect old losses at update time

Gradient Episodic Memory takes another route (Lopez-Paz and Ranzato 2017).

Let

$\mathbf{g}$

be the current gradient.

Let

$\mathbf{g}_j$

be a gradient computed from episodic memory for old context j .

Suppose the update uses

$\boldsymbol{\theta}' = \boldsymbol{\theta} - \eta \tilde{\mathbf{g}}$.

To first order,

$L_j(\boldsymbol{\theta}') - L_j(\boldsymbol{\theta})$ approximately $-\eta \mathbf{g}_j^T \tilde{\mathbf{g}}$.

If

$\mathbf{g}_j^T \tilde{\mathbf{g}} \geq 0$,

the linearized old-context loss does not increase.

GEM therefore turns remembered examples into constraints on the update direction.

This is neither ordinary replay nor EWC.

Replay changes the objective.

EWC adds a parameter penalty.

GEM changes the feasible gradient direction.

48.17 First-order protection has a scope

A gradient constraint is local.

It reasons about the immediate linearized change around the current parameters.

Finite learning rates introduce higher-order effects.

A bounded episodic memory may not represent the full old distribution.

So even a constrained update must retain its claim boundary.

This is a recurring Atlas theme:

a local protective condition is not automatically a global retention guarantee.

The correct object and scale matter.

48.18 Parameter isolation: stop sharing the contested parameters

There is a more structural response to interference.

Do not let later tasks overwrite the same parameters.

PackNet uses iterative pruning and parameter allocation to add tasks while preserving previously assigned weights (Mallya and Lazebnik 2018).

In the scalar witness, parameter isolation means replacing the one shared variable with two task-selected variables:

$w_A = -1$

and

$w_B = 1$.

Task A uses w_A .

Task B uses w_B .

Both losses are zero.

Interference disappears because the conflict has been removed from the shared parameter.

48.19 Isolation pays with capacity and routing

The previous result sounds perfect until we count resources.

The original model had one scalar.

The isolated model has two.

It also needs a selector telling it which scalar to use.

This distinction matters in the continual-learning scenarios.

If task identity is supplied at test time, task-specific masks or heads are natural.

If the model must classify across all accumulated classes without task identity, routing is part of the problem.

Parameter isolation therefore trades one form of difficulty for another:

- less overwrite;

- more capacity accounting;
- more routing structure.

48.20 Replay, consolidation, constraints, and isolation are different levers

The four mechanism families act at different places.

48.20.1 Replay

Change what data or evidence is present during optimization.

48.20.2 Consolidation

Change the cost of moving through parameter space.

48.20.3 Gradient constraints

Change which local update directions are allowed.

48.20.4 Parameter isolation

Change which parameters can be written by later contexts.

These can be combined.

They can also fail for different reasons.

A replay method may fail because the memory is unrepresentative.

A consolidation method may fail because its importance approximation is poor.

A gradient constraint may fail outside its local approximation or memory support.

An isolation method may exhaust capacity.

Calling all four “memory methods” hides the mechanism.

48.21 Capacity must be reported

Continual learning is easy if we permit unlimited duplication.

For every new task, create a fresh model.

Old performance remains untouched.

But the system grows without bound and may require explicit task routing.

That is why memory and capacity accounting are part of the scientific claim.

Relevant resources can include:

- stored examples;
- stored activations or transitions;
- generated replay models;
- Fisher or importance statistics;
- task-specific heads;
- masks;
- reserved parameter blocks;
- additional networks.

A method that retains better by spending more memory may still be the right engineering choice.

The cost must simply remain visible.

48.22 Average forgetting can hide a damaged context

Suppose two earlier contexts have

$F_1=0$

and

$F_2=1$.

Then

$F_{\text{avg}}=1/2$.

But

$F_{\text{max}}=1$.

Another learner might have

$F_1=1/2$

and

$F_2=1/2$.

The average is identical.

The risk profile is not.

This is why the Atlas keeps both average and worst-context summaries.

A safety-critical application may care about the worst retained capability.

A broad benchmark may care more about average retained performance.

Neither scalar subsumes the other.

48.23 Positive transfer should not be clipped out of existence

Forgetting metrics often focus on decline.

But later learning can improve earlier tasks.

If context B teaches a reusable representation, then

$R_{\{T,A\}} > R_{\{A,A\}}$

may occur.

A nonnegative forgetting metric reports zero in that case.

That is correct for the narrow question “how much was lost?”

It is incomplete for the broader question “what effect did later learning have?”

So continual-learning reports should preserve signed transfer information where it matters.

Lopez-Paz and Ranzato make transfer an explicit part of continual-learning evaluation (Lopez-Paz and Ranzato 2017).

48.24 Final accuracy alone is also insufficient

Suppose one learner forgets A badly while learning B, then relearns A later.

Its final score may look good.

Another learner preserves A continuously.

If the application requires always-available capability, those histories are different.

Similarly, a learner may retain old tasks perfectly but fail to learn new ones.

That is not good continual learning merely because forgetting is zero.

The performance-through-time matrix makes both failures visible:

- instability of old capability;
- failure of new acquisition.

48.25 Forgetting and transfer can be asymmetric

Task A may help Task B.

Task B may harm Task A.

Task C may improve both.

Sequential order therefore matters.

A continual-learning benchmark is not fully described by its set of tasks.

It also has an ordering, data schedule, context-transition structure, and evaluation schedule.

Changing the order can change the interference pattern.

That makes continual learning a dynamical training process, not merely multi-task learning with a shuffled dataset.

48.26 Multi-task learning is the useful counterfactual

If all tasks and all data are jointly available from the start, we can optimize a joint objective.

That is not the continual-learning information pattern.

It is still a valuable reference.

The gap between:

- joint training;
- sequential training;
- sequential training with replay;
- sequential training with consolidation;

helps separate capacity conflict from information loss.

If joint training cannot fit all tasks, the model has a compatibility or capacity problem.

If joint training succeeds but sequential training forgets, the schedule and information constraints are implicated more directly.

48.27 Replay versus joint training

Replay approximates some aspects of joint access.

But unless the replay store contains all old data with the same weighting and sampling, it is not identical to joint training.

A bounded exemplar buffer changes the old-data distribution.

Generated replay adds modeling error.

Reservoir sampling, class balancing, and prioritization change which old evidence remains visible.

So “replay” should always be accompanied by a memory contract:

what is stored, how much, how selected, and how sampled.

48.28 EWC versus ordinary L2 regularization

An ordinary quadratic penalty might constrain

$$||\theta||^2$$

or distance from a generic reference.

EWC instead anchors parameters to an earlier task solution with coordinate-specific importance weights.

The source emphasizes this selective plasticity (Kirkpatrick et al. 2017).

The distinction matters.

Uniform resistance can protect unimportant coordinates too strongly and important coordinates too weakly.

EWC attempts to allocate plasticity where the previous task is estimated to tolerate it.

Again, the quality of that estimate is part of the method’s empirical burden.

48.29 Consolidation and semantic memory are not synonyms

The Memory Taxonomy chapter uses **semantic** as a role describing generalized knowledge rather than event-specific records.

EWC is called a consolidation method because it stabilizes parameters.

That does not mean the resulting parameters form a clean semantic store.

Likewise, episodic replay examples can contribute to generalized semantic behavior during training.

Memory role and anti-forgetting mechanism answer different questions.

This distinction becomes important in the next chapter family.

48.30 Catastrophic forgetting is partly an access problem

Why does replay help?

Because the learner regains access to evidence that the current stream no longer supplies.

Why does external memory become relevant downstream?

Because parameters are a lossy, interference-prone place to preserve some kinds of information.

But this chapter stops short of the stronger claim.

It does not conclude that knowledge should generally leave the weights.

It establishes the ingredients needed to ask that question:

- what was retained;
- what was forgotten;
- what evidence had to be stored;
- what capacity was consumed;
- what mechanism prevented interference.

The External-Memory chapter will make the architectural argument.

48.31 Failure modes

48.31.1 Forgetting-equals-parameter-drift

Measure only parameter distance.

Failure: behavior is not evaluated.

48.31.2 Zero-forgetting-by-zero-learning

Freeze everything.

Failure: old capability is retained because new capability is never acquired.

48.31.3 Replay-equals-memory-architecture

Call every replay buffer an external persistent memory system.

Failure: training use is confused with storage locus and lifetime.

48.31.4 EWC-equals-guarantee

Treat a Fisher-weighted local quadratic penalty as exact preservation.

Failure: approximation becomes certification.

48.31.5 Isolation-equals-free retention

Allocate fresh parameters indefinitely.

Failure: capacity and routing costs disappear from the comparison.

48.31.6 Average-only reporting

Report one mean retention score.

Failure: severe failure on one context can be hidden.

48.31.7 Regime mixing

Compare task-incremental and class-incremental results as if task identity were equally available.

Failure: the inference contracts differ.

48.32 A practical continual-learning ledger

Before interpreting a continual-learning result, record:

Field	Question
stream	What changes over time?
context	Is task/context identity available during training and evaluation?
write target	Which parameters or memory objects are updated?
old evidence	What earlier information remains accessible?
protection	Replay, consolidation, gradient constraint, isolation, or combination?
capacity	What grows with the number of contexts?
evaluation	What is the full performance-through-time matrix?
forgetting	Average, worst-context, or another declared summary?
transfer	Are signed forward/backward effects retained?
baseline	Joint training, separate models, naive sequential training, or another comparator?

This ledger prevents an anti-forgetting mechanism from being evaluated without its information and capacity contract.

48.33 What the exact witness establishes

The scalar witness establishes a narrow but useful result.

Two tasks with incompatible optima share one parameter.

Sequentially optimizing the second task exactly raises the first task's loss from

0

to

2.

A unit quadratic retention penalty changes the exact compromise to

$w=0$

with losses

$1/2$

and

$1/2$.

Equal-weight replay happens to produce the same point because the old task loss is exactly the same quadratic as the chosen penalty.

Two isolated task-selected parameters achieve zero loss on both tasks by spending extra capacity and routing.

Nothing stronger is implied.

The witness is not a benchmark result.

It is a controlled model of interference.

48.34 Atlas handoff

The External-Memory Thesis — ATLAS-CH-EXTMEM-001.

The downstream chapter may now assume:

- forgetting is behavioral, not parameter-distance alone;
- replay restores earlier evidence to training;
- consolidation protects selected parameter directions without replaying the full old distribution;
- gradient constraints and parameter isolation are distinct mechanisms;
- memory and capacity costs belong in the comparison;
- task identity materially changes the difficulty of retention.

That chapter must independently answer the architectural question:

which knowledge should remain in parameters, and which knowledge should move into persistent shared memory?

The dependency direction matters.

Continual-learning difficulty motivates that question.

It does not answer it in advance.

48.35 References used in this chapter

- McCloskey and Cohen, *Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem* (McCloskey and Cohen 1989).
- Kirkpatrick et al., *Overcoming catastrophic forgetting in neural networks* (Kirkpatrick et al. 2017).
- Rebuffi et al., *iCaRL: Incremental Classifier and Representation Learning* (Rebuffi et al. 2017).
- Lopez-Paz and Ranzato, *Gradient Episodic Memory for Continual Learning* (Lopez-Paz and Ranzato 2017).
- Mallya and Lazebnik, *PackNet: Adding Multiple Tasks to a Single Network by Iterative Pruning* (Mallya and Lazebnik 2018).
- van de Ven, Tuytelaars, and Tolias, *Three types of incremental learning* (Ven et al. 2022).

Exact prerequisite identities and source-authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-CONTINUAL-001.yaml`

Chapter 49

The External-Memory Thesis

Epistemic status: audited retrieval + audited continual-learning mechanisms + primary retrieval-augmented/model-editing examples + Atlas synthesis + exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-EXTMEM-001.md

Formal packet: mathematics/derivations/ATLAS-CH-EXTMEM-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-EXTMEM-001.md

Source lock: sources/source-locks/ATLAS-CH-EXTMEM-001.yaml

A model can know something in at least two radically different ways.

It can carry the information implicitly in its parameters.

Or it can retrieve an explicit record when the information is needed.

Those two forms of knowing have different update semantics, provenance semantics, sharing semantics, latency costs, and failure modes.

The governing thesis of this chapter is:

an intelligent system should not force every useful fact into its weights when some information is better represented as explicit, versioned, retrievable state.

The thesis has an equally important converse:

external memory is not a universal substitute for learning.

Stable structure, reusable procedures, compressed regularities, and learned representations may belong in parameters.

The architecture is usually hybrid.

49.1 The wrong question is parameters versus memory

A simple binary question asks:

should knowledge live in parameters or external memory?

That framing is too coarse.

The same system can use:

- parameters for syntax, features, procedures, and compressed regularities;
- an external store for mutable facts, records, evidence, and shared organizational state;
- a working context for the subset needed now.

Placement is an architectural decision about which representation best serves which kind of information.

49.2 What is parametric knowledge?

Call knowledge **parametric** when its use is mediated by learned model state.

For parameters `theta`, behavior is:

`f_theta(q)`.

The knowledge may not correspond to a single readable parameter.

It can be distributed across many coordinates and interactions.

That distribution is often the point.

It enables compression and generalization.

49.3 What is external memory?

External memory means explicit state outside the model parameters.

A record can carry:

- key;
- value/content;
- source;
- timestamp;
- version;
- type;
- access policy;

49.4. RETRIEVAL IS THE READ PATH, NOT THE MEMORY ITSELF⁸³⁵

- status;
- supersession relation.

The important property is not merely that bytes are stored elsewhere.

It is that the system can address and update an explicit memory object under a declared read/write contract.

49.4 Retrieval is the read path, not the memory itself

RETRIEVAL-001 established:

retrieval is a declared read operation over records, not a synonym for relevance, truth, provenance, or usefulness.

That distinction remains active here.

A perfectly maintained memory can still fail if retrieval:

- misses the record;
- applies the wrong filter;
- ranks badly;
- uses a stale index;
- returns an unauthorized version.

External storage and successful memory use are separate layers.

49.5 Continual learning is the parameter-update side

CONTINUAL-001 established that sequential parameter learning can alter earlier behavior.

It also separated:

- replay;
- consolidation/EWC;
- episodic gradient constraints;
- parameter isolation.

Those mechanisms matter because moving knowledge into parameters creates update obligations.

The external-memory thesis asks when some of those obligations can be avoided by leaving selected knowledge explicit.

49.6 The placement descriptor

For knowledge item or class k , use:

$$\text{Place}(k) = (V, P, S, D, R, L, H, A, G).$$

The coordinates are:

- volatility V ;
- provenance/audit need P ;
- sharing scope S ;
- deletion/supersession need D ;
- retrievability/addressability R ;
- latency constraint L ;
- availability/failure-tolerance requirement H ;
- access-control/privacy requirement A ;
- value of parametric generalization/compression G .

The chapter defines no universal scalar score over these dimensions. External locus and persistence also remain distinct: this thesis focuses on persistent external records when that lifetime is deliberately chosen; it does not redefine every external memory object as persistent.

49.7 Why no scalar placement score?

Suppose one fact is highly volatile and provenance-critical.

That points toward external memory.

But suppose the same fact is required on every token with a hard microsecond latency budget.

That points toward parameterization or aggressive caching.

Neither fact cancels the other.

A placement decision must expose the tradeoff rather than hide it in an unexplained number.

49.8 Volatility favors explicit state

Some information changes often:

- current prices;
- active policies;
- account permissions;
- schedules;
- inventory;

- organizational decisions;
- scientific records under revision.

Repeatedly changing model weights for every update may be operationally awkward or behaviorally risky.

A versioned external record has a direct update operation.

That makes volatility one pressure toward externalization.

49.9 Provenance favors explicit state

Some claims must answer:

- where did this come from?
- which version?
- who authorized it?
- what replaced it?
- when was it observed?

A record can carry those fields directly.

A bare parameter vector does not, by itself, expose a per-fact source record.

This is a representation distinction, not a claim that parametric systems can never maintain provenance.

They can maintain auxiliary provenance externally.

49.10 Deletion and supersession favor explicit records

An explicit record can be marked:

- current;
- superseded;
- revoked;
- quarantined;
- deleted.

That does not make erasure trivial in replicated infrastructure.

But it gives the semantic operation a named target.

By contrast, removing one learned association from model behavior is a model-editing problem.

The two deletion semantics are not equivalent.

49.11 Sharing can favor external memory

Suppose ten agents need the same mutable operational fact.

One authoritative record can, in principle, be updated once.

Every agent can then retrieve the same current version.

But this advantage exists only if synchronization, access control, and freshness are well-defined.

A shared store with divergent caches can create multiple effective memories.

49.12 Addressability matters

Some knowledge naturally has a key:

- customer ID;
- document ID;
- theorem ID;
- experiment run;
- date;
- entity-relation pair;
- configuration name.

Such information is easy to imagine as an explicit memory record.

Other knowledge is diffuse.

There may be no sensible lookup key for:

- fluency;
- visual invariance;
- a learned optimization heuristic;
- a broad concept manifold.

Addressability is therefore a real placement coordinate.

49.13 Parametric compression has value

Parameters can compress many examples into reusable behavior.

The system does not need to retrieve every training example whenever it applies a learned rule.

This is a major advantage.

A database of examples is not the same thing as learning a representation or procedure from them.

External memory should not become an excuse to avoid learning stable structure.

49.14 Low-latency ubiquitous structure can favor parameters

Some knowledge is required constantly.

An explicit retrieval on every use may add:

- index lookup;
- memory traffic;
- network latency;
- synchronization;
- failure modes.

Parametric behavior is already present in the model’s forward computation.

This can favor parameterization for stable, ubiquitous structure.

No universal latency ordering is claimed.

49.15 RAG makes the hybrid idea concrete

Lewis et al. combine a parametric sequence model with a dense non-parametric Wikipedia index (Lewis et al. 2020).

Their formulation is useful here because it makes the two loci explicit:

- a learned parametric model;
- a retrievable external corpus.

The source also explicitly motivates retrieval in part through provenance and world-knowledge update limitations of parameter-only systems.

The Atlas uses this as one primary example, not as proof that RAG is the universal architecture.

49.16 kNN-LM shows datastore substitution

Khandelwal et al. augment a pretrained language model with a nearest-neighbor datastore (Khandelwal et al. 2020).

In their reported experiments, changing the datastore supports domain adaptation without further model training.

The bounded architectural lesson is:

some behavior can be changed by changing explicit memory while freezing the model.

That is exactly the update-separation property the external-memory thesis cares about.

49.17 RETRO shows scale, not universal superiority

Borgeaud et al. condition language modeling on chunks retrieved from a very large text database (Borgeaud et al. 2022).

Their result demonstrates that explicit retrieved memory can complement model parameters at large scale.

It does not prove that every knowledge item belongs in retrieval.

Scale is evidence that the architecture is viable, not a universal placement rule.

49.18 ROME gives the necessary counterexample

Meng et al. show that specific factual associations can be edited directly in model parameters in their reported setting (Meng et al. 2022).

That matters because it blocks an overstrong thesis.

It would be false to say:

mutable facts must always be external.

Parametric editing is possible.

The real question is which update semantics and system guarantees are desirable.

49.19 The exact parametric witness

Use two queries:

A and B .

Let:

$$f_{\theta}(A) = \theta_1 + \theta_2$$

and:

$$f_{\theta}(B) = \theta_1 - \theta_2.$$

Initial target facts are:

$$f_{\theta}(A) = 2, \quad f_{\theta}(B) = 0.$$

The unique parameter vector is:

$$\theta = (1, 1).$$

49.20 Updating one fact requires a coordinated edit here

Now change only the target for A:

$$A : 2 \rightarrow 4$$

while preserving:

$$f_{\theta}(B) = 0.$$

The new unique parameter vector is:

$$\theta' = (2, 2).$$

Therefore:

$$\Delta\theta = (1, 1).$$

Both coordinates change.

This is a property of the chosen representation.

It is not a theorem that factual parameter edits always require many changed weights.

49.21 A naive coordinate-local edit interferes

Change only:

`theta_1:1 -> 2`

and leave:

`theta_2=1.`

Then:

`f(A)=3`

and:

$f(B)=1$.

The edit fails twice:

- A does not reach 4;
- B no longer remains 0.

This is the smallest exact example of representation-induced edit coupling.

49.22 The external-store version

Represent the same two facts as records.

Initial state:

A $\rightarrow$ (2, s_{A1} , $v1$)

B $\rightarrow$ (0, s_{B1} , $v1$).

Update A by making the version transition explicit:

A, $v1 \rightarrow$ (2, s_{A1} , $v1$, $status=superseded$)

A, $v2 \rightarrow$ (4, s_{A2} , $v2$, $status=current, supersedes=v1$).

B is unchanged and current.

A latest-version exact-key read now returns:

A=4

B=0.

Only logical key A changed.

49.23 Record-locality is not compute optimality

The external witness changes one logical key.

That is a statement about state mutation locality.

It does not prove that the external update uses fewer machine operations than a model edit.

The database may replicate records, rebuild indexes, invalidate caches, or synchronize across regions.

Representation-locality and systems cost are different claims.

49.24 Provenance is explicit in the store

The external records retain:

`s_A1`

for version 1 and:

`s_A2`

for version 2.

The supersession edge preserves the update history.

The bare parameter vector:

`theta=(1,1)`

does not itself expose which source supported fact A.

Again, the correct conclusion is not:

parameters cannot have provenance.

The correct conclusion is:

provenance is not encoded as an explicit per-fact record in the bare parameter representation.

49.25 The stale-read counterexample

External memory creates a new failure mode.

Suppose one consumer still reads snapshot `M_1` after the authoritative store advances to `M_2`.

The authoritative latest value is:

`A=4`.

The stale consumer still observes:

`A=2`.

Externalization has made updating the authoritative record easy.

It has not made every read fresh.

49.26 Freshness needs a contract

Let:

`v_star(k)`

be the authoritative current version for key `k`.

Let:

`v_C(k)`

be the version observed by consumer C .

A strict freshness rule might require:

$v_C(k) = v_{\text{star}}(k)$.

A bounded-staleness policy might permit some lag.

The point is architectural:

freshness must be specified.

It is not implied by the word **memory**.

49.27 Caching creates local memory again

A system may externalize knowledge into a shared store and then cache retrieved results near each model instance.

That is often sensible for latency.

It also means the system now has several memory loci:

- authoritative store;
- retrieval index;
- cache;
- working context;
- parameters.

The memory taxonomy has returned.

Externalization does not make the architecture simple.

It makes some state explicit.

49.28 Write correctness and read correctness differ

A memory update can be correct while a later answer is wrong.

Possible causes include:

- stale cache;
- stale index;
- wrong query;
- wrong filter;
- wrong rank;
- context omission;
- downstream reasoning error.

This separation matters operationally.

It lets the system ask:

did we store the right fact?

before asking:

did the model use it correctly?

49.29 Retrieval failure is not memory-write failure

Suppose version 2 of A is present and current.

A vector retriever fails to return it.

That is a read-path failure.

Rewriting the memory record will not necessarily fix it.

The correct repair may be in:

- indexing;
- query formulation;
- ranking;
- filters;
- context compilation.

Typed failure boundaries prevent useless updates.

49.30 Poisoning is a memory-layer risk

Explicit memory can be modified.

That is an advantage for legitimate updates.

It is also an attack surface.

A malicious or low-quality record can be:

- inserted;
- ranked highly;
- given false metadata;
- propagated to many consumers.

External memory therefore needs evidence and authority controls, not only retrieval quality.

49.31 Access control becomes first-class

A shared store may contain facts that not every agent may read.

Let:

`allow(agent, key, operation)`

be an authorization predicate.

A successful lookup should not bypass this predicate merely because the record is relevant.

This creates another separation:

retrievable does not mean authorized.

The downstream Polity chapter will need this distinction.

49.32 Sharing is not global visibility

A memory can be shared among:

- one team;
- one model family;
- one tenant;
- one organization;
- one scientific programme.

Shared does not mean public.

Scope is part of the memory contract.

This is especially important when memory contains private, licensed, embargoed, or role-restricted information.

49.33 Deletion is not trivial even externally

An explicit record can be marked deleted or revoked.

But copies may remain in:

- caches;
- replicas;
- logs;
- backups;
- derived indexes;
- compiled contexts.

Therefore external memory gives deletion a clear target but not necessarily immediate total erasure.

A serious system must define deletion propagation.

49.34 Parametric erasure is a different problem

If information has been learned into parameters, deleting the original training record does not imply the behavior disappears.

The two operations target different objects.

This distinction is especially important for systems with both:

- learned parametric state;
- retained external records.

Deletion policy should say which loci are in scope.

49.35 Consolidation moves in the other direction

The external-memory thesis is not a one-way migration out of parameters.

Sometimes repeated external evidence becomes stable enough that learning it into parameters is useful.

Call this **consolidation**.

Examples might include:

- repeated tool-use patterns;
- stable schema structure;
- durable domain regularities;
- procedural skills.

Consolidation is a parameter-learning operation.

It is distinct from memory write.

49.36 Externalization can precede consolidation

A system can first store new evidence externally because it is:

- recent;
- uncertain;
- volatile;
- provenance-sensitive.

Later, after enough evidence accumulates, some stable regularity can be learned parametrically.

This creates a useful architecture:

explicit first, consolidate later.

The chapter presents this as a design pattern, not a theorem of optimal learning.

49.37 Parameter isolation and external memory solve different problems

CONTINUAL-001 showed that parameter isolation can avoid direct overwrite by adding capacity or routing.

External memory avoids some parameter updates by moving selected state elsewhere.

Those are not the same strategy.

Parameter isolation still places knowledge in learned capacity.

Externalization places it in explicit records.

The costs differ.

49.38 Replay uses external evidence without making it the final answer source

Replay stores examples or exemplars externally.

During learning, those records return to the optimizer.

This is different from runtime retrieval of a fact for direct use.

So even within one system, external records can have multiple roles:

- training evidence;
- episodic replay;
- semantic knowledge;
- runtime retrieval;
- audit/provenance archive.

Role should not be inferred from locus alone.

49.39 A record is not automatically knowledge

An external store can contain:

- false statements;
- conflicting statements;
- stale statements;
- unsupported statements;
- low-authority sources.

The system still needs:

- provenance;
- ranking;
- authority;
- reconciliation;
- uncertainty handling.

External memory makes records inspectable.

It does not make them true.

49.40 Multiple records can legitimately conflict

Suppose two sources disagree.

Parameter-only consolidation may blur the conflict into one behavior.

An external store can preserve both records explicitly.

That can be useful when the disagreement itself matters.

But then the reader must decide:

- which source is authoritative;
- whether both are shown;
- whether uncertainty is retained.

Preserving conflict is not the same as resolving it.

49.41 Update frequency is only one axis

A highly volatile fact often favors external memory.

But a rarely changing fact can also be external if:

- provenance is critical;
- deletion must be possible;
- sharing is required;
- access policy is complex.

Conversely, some moderately changing information may still be parameterized if retrieval is too costly.

Placement is multi-dimensional.

49.42 The “world in weights” failure mode

A system that tries to carry every changing fact in parameters inherits several burdens:

- weight updates for factual changes;

- unclear per-fact provenance;
- difficult selective deletion;
- duplicated updates across model copies;
- interference risk;
- stale deployed checkpoints.

This is the core motivation for the external-memory thesis.

But the remedy must not become a slogan.

49.43 The “database as intelligence” failure mode

The opposite mistake is to externalize everything.

A datastore does not replace:

- abstraction;
- reasoning;
- representation learning;
- compression;
- procedure learning;
- transfer.

If every answer requires finding a memorized record, the system may fail whenever exact evidence is absent.

External memory should complement learned capability.

49.44 The hybrid architecture

A mature system may use four layers:

1. **parameters** for compressed reusable structure;
2. **persistent external memory** for explicit versioned records;
3. **retrieval** for selecting candidate evidence;
4. **working context** for the task-specific subset currently active.

The next chapter, Context Compilation, will develop layer 4.

This chapter stops before that step.

49.45 A practical placement ledger

Before moving a knowledge class into or out of parameters, record:

Field	Question
item/class	What knowledge is being placed?
volatility	How often does it change?
provenance	Must source/version be visible per item?
sharing	Which agents/models need one common state?
supersession	Must updates, revocation, or deletion be explicit?
retrievability	Is there a meaningful key/query/index?
latency	Can retrieval cost be tolerated?
availability	What happens if the store is unavailable?
access control	Who may read/write it?
privacy	Does explicit storage create new exposure?
generalization	Is compression into reusable behavior valuable?
consolidation	Should stable evidence later move into parameters?
freshness	Which version guarantees are required?
fallback	What should happen on retrieval failure?

This ledger turns “put it in memory” into an auditable design decision.

49.46 Failure modes

49.46.1 External-equals-fresh

The record is updated but consumers use stale snapshots.

49.46.2 External-equals-correct

A stored record is treated as true merely because it is explicit.

49.46.3 Retrieved-equals-authorized

A relevant record bypasses access policy.

49.46.4 Parameter-change-equals-forgetting

Any weight movement is called forgetting without behavioral evaluation.

49.46.5 External-equals-cheap

Record-local updates are assumed to imply lower wall-clock cost.

49.46.6 Parametric-equals-uneditable

Direct model editing is ignored despite evidence that some factual associations can be changed parametrically.

49.46.7 Shared-equals-consistent

Multiple agents point to one store, so consistency is assumed without version/synchronization semantics.

49.46.8 Externalize-everything

Explicit memory is treated as a replacement for learning compressed structure.

49.47 What the exact witness establishes

The companion witness proves only that, for the chosen two-fact representations:

- initial parameter state is (1,1);
- updated parameter state is (2,2);
- both parameter coordinates change under the exact preserving update;
- the naive one-coordinate edit produces A=3,B=1;
- the external latest-version update changes only logical key A;
- external provenance retains old/new source/version information;
- a stale snapshot still returns A=2 after authoritative A becomes 4.

It does not prove universal systems superiority.

49.48 Downstream handoff: Context Compilation

ATLAS-CH-CONTEXTCOMP-001 may now assume:

- hybrid parametric/external placement;
- versioned external records;
- freshness as an explicit policy;
- retrieval success as distinct from storage correctness;
- provenance-preserving record semantics.

It must independently define how retrieved records become the bounded working context of a model invocation.

49.49 Downstream handoff: Polity

ATLAS-CH-POLITY-001 may now assume:

- shared memory is an explicit architectural locus;
- shared visibility requires synchronization/version semantics;
- access control is distinct from relevance;
- common memory can coexist with private parametric and working state.

It must independently define multi-agent governance, coordination, authority, and shared-state policy.

49.50 References used in this chapter

- Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (Lewis et al. 2020).
- Khandelwal et al., *Generalization through Memorization: Nearest Neighbor Language Models* (Khandelwal et al. 2020).
- Borgeaud et al., *Improving language models by retrieving from trillions of tokens* (Borgeaud et al. 2022).
- Meng et al., *Locating and Editing Factual Associations in GPT* (Meng et al. 2022).

Exact source identities and authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-EXTMEM-001.yaml`

Chapter 50

Context Compilation

Epistemic status: audited External-Memory Thesis + Atlas-owned finite context-compilation formalism and exact witness.

Specification: manuscript/specifications/ATLAS-CH-CONTEXTCOMP-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-CONTEXTCOMP-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-CONTEXTCOMP-001.md

Source lock: sources/source-locks/ATLAS-CH-CONTEXTCOMP-001.yaml

A memory system can contain the right information and still hand the model the wrong working context.

The record can exist.

Retrieval can find it.

The ranker can score it highly.

The final prompt can still be stale, unauthorized, over budget, missing a mandatory record, stripped of provenance, or badly assembled.

This chapter treats that assembly step as a first-class transformation.

The governing claim is:

a model context is a compiled working set, not merely a prefix of retrieved text.

The word compiled matters because context construction has inputs, constraints, policy, failure modes, and a replayable output.

50.1 The external-memory handoff

The External-Memory Thesis established a hybrid architecture:

- some structure remains parametric;
- some information remains explicit and retrievable;
- external records can carry source, version, status, and supersession meta-data;
- freshness is a policy question;
- correct storage is distinct from successful retrieval and use.

CONTEXTCOMP starts after retrieval has already produced candidates.

Its question is narrower:

which candidate records should actually become the bounded working context of this invocation, and in what serialized form?

50.2 The pipeline

Write:

$$M \xrightarrow{\text{retrieve}} R_q \xrightarrow{\text{compile}_{\Pi, B}} K_q \xrightarrow{\text{invoke}} y.$$

Here:

- M is external memory;
- R_q is the retrieved candidate set for query/invocation q ;
- Π is compilation policy;
- B is context budget;
- K_q is the compiled ordered working context;
- y is downstream model behavior.

Every arrow has different obligations.

50.3 Storage is not retrieval

A correct current record can be present in memory and never retrieved.

That is a retrieval failure.

It is not a compilation failure.

The compiler cannot select a candidate it never receives.

This distinction is inherited from EXTMEM-001 and its Retrieval prerequisite.

50.4 Retrieval is not compilation

A retriever may return:

- stale versions;
- duplicate keys;
- unauthorized records;
- many more records than the model budget permits;
- records whose metadata makes them mandatory;
- records useful for ranking but invalid under the current invocation policy.

The candidate set is therefore an input to compilation.

It is not yet the context.

50.5 Ranking is one signal, not the whole contract

A retrieval rank can express one notion of relevance or similarity.

It does not automatically encode:

- current version;
- authorization;
- mandatory policy;
- provenance requirements;
- atomic serialization cost;
- conflict state.

This chapter's exact witness makes that distinction literal.

The top-ranked policy record is stale.

The current mandatory policy ranks third.

A rank-only prefix fails.

50.6 Candidate records

Represent a candidate record as

$$r_i = (k_i, v_i, \sigma_i, p_i, a_i, m_i, \rho_i, c_i, u_i, t_i).$$

The fields are:

- logical key;
- version;
- status;
- provenance/source;

- authorization;
- mandatory flag;
- retrieval rank;
- compiled cost;
- declared compiler utility;
- type.

This is a chapter-local formal object.

It is not proposed as a universal context format.

50.7 Compiled cost includes metadata

Suppose the payload itself costs

$$d_i$$

units.

Suppose required metadata costs

$$h_i.$$

Then the actual compiled cost is

$$c_i = d_i + h_i.$$

If the system budgets only the payload while still requiring provenance in the prompt, its budget calculation is wrong.

The resource accounting must describe the object actually serialized.

50.8 Provenance is not decoration

The External-Memory Thesis made source/version identity explicit.

Context compilation should not throw that information away casually.

For the finite formalism, serialize each included record with an envelope:

$$E(r_i) = [\text{id}, \text{key}, \text{version}, \text{source}, \text{status}, \text{payload}].$$

The envelope lets the working context say not only what was included, but which version and source were included.

That does not prove the source is true.

It preserves identity.

50.9 Authorization comes before optimization

Suppose a retrieved record is highly ranked and appears highly useful.

If the invocation is not authorized to use it, the compiler must exclude it.

Authorization is therefore modeled as a hard admissibility filter:

$$a_i = 1.$$

It is not a utility bonus.

The compiler cannot trade privacy or authority against a few extra utility points.

50.10 Freshness comes before optional utility

Suppose two records share logical key policy:

- version 1 is superseded;
- version 2 is current.

If the declared policy is latest-current, version 1 is removed before optional selection.

This remains true even if version 1 has:

- better retrieval rank;
- higher declared utility;
- lower compiled cost.

A validity constraint is not a soft preference.

50.11 Unresolved conflict is a failure state

Now suppose two authorized records for the same logical key are both marked current.

If the version policy does not say which wins, the compiler should not silently guess.

It should return:

VERSION_CONFLICT.

Choosing the higher retrieval rank would be a new policy.

If that policy is desired, it must be declared.

50.12 Mandatory records

Some records are not optional under a particular invocation contract.

Examples can include:

- current governing policy;
- task instruction;
- safety constraint;
- required schema;
- experiment identity.

Let

$$M$$

be the mandatory set.

Its total cost is

$$C_M = \sum_{r_i \in M} c_i.$$

This cost is paid before optional context is selected.

50.13 Mandatory overflow

If

$$C_M > B,$$

no atomic context can both:

- contain every mandatory record;
- fit the budget.

The correct output is explicit failure:

MANDATORY_OVERFLOW.

The compiler should not make a mandatory rule half-visible merely to keep the invocation running.

50.14 Optional selection

After authorization, freshness, conflict resolution, and mandatory inclusion, let O be the optional records.

Residual budget is:

$$B' = B - C_M.$$

This chapter uses the finite objective:

$$\max_{S \subseteq O} \sum_{r_i \in S} u_i$$

subject to:

$$\sum_{r_i \in S} c_i \leq B'.$$

This is a declared compiler objective.

It is not a definition of true context usefulness.

50.15 Utility is policy input

The score

$$u_i$$

can come from:

- a heuristic;
- a learned scorer;
- a human rule;
- a task-specific policy.

The finite chapter does not care where it came from.

It only proves which subset is optimal relative to those declared scores.

Therefore:

utility-optimal under $\Pi \not\Rightarrow$ universally optimal context.

50.16 Determinism

If several feasible subsets have equal utility, the compiler needs a tie-break if exact replay matters.

This chapter uses:

1. lower total cost;
2. lexicographically smaller stable record-ID list.

The choice is arbitrary but explicit.

Determinism turns the same inputs into the same compiled result.

50.17 Serialization is part of compilation

A selected set is unordered.

A model context is ordered.

Therefore compilation is not finished when subset selection ends.

The finite policy uses:

$$\text{policy} \prec \text{evidence} \prec \text{detail} \prec \text{background}.$$

Within a type, stable record IDs break ties.

No empirical claim is made that this order is universally best for language models.

It is simply part of the declared context object.

50.18 The exact budget-8 witness

Use context budget:

$$B = 8.$$

The retrieved candidates are:

ID	key	status/version	rank	cost	utility	mandatory
r_{p1}	policy	superseded v1	1	3	9	no
r_f	task_fact	current v1	2	4	7	no
r_{p2}	policy	current v2	3	4	8	yes
r_b	background	current v1	4	2	3	no
r_d	detail	current v1	5	3	5	no

All records are authorized.

The current policy v2 supersedes v1.

50.19 Freshness removes the rank-1 policy

The latest-current policy excludes:

$$r_{p1}.$$

That record had retrieval rank 1.

It also had declared utility 9.

Neither fact can override its superseded status under this policy.

Remaining records:

$$r_f, r_{p2}, r_b, r_d.$$

50.20 Mandatory policy consumes half the budget

The current policy v2 is mandatory.

Its cost is:

$$4.$$

Residual budget:

$$8 - 4 = 4.$$

The optional candidates are:

$$r_f : (4, 7),$$

$$r_b : (2, 3),$$

$$r_d : (3, 5).$$

Each pair denotes:

(cost, utility).

50.21 Exhaustive finite selection

Under residual budget 4, feasible optional subsets are:

$$\emptyset,$$

$$\{r_f\},$$

$$\{r_b\},$$

$$\{r_d\}.$$

Their utilities are:

$$0, 7, 3, 5.$$

Every two-record optional subset costs more than 4.

Therefore the unique optimum is:

$$S^* = \{r_f\}.$$

50.22 The compiled set

Add the mandatory current policy:

$$S_{\text{final}} = \{r_{p2}, r_f\}.$$

Total cost:

$$4 + 4 = 8.$$

Total declared utility:

$$8 + 7 = 15.$$

The serialized context places:

1. current policy v2;
2. task evidence.

Both retain provenance metadata.

50.23 What the witness actually proves

It proves an exact finite statement about a declared compiler.

It proves:

- which candidate is stale;
- which record is mandatory;
- which optional subsets fit;
- which feasible set maximizes declared utility;
- which records are serialized;
- exact cost;
- exact declared utility.

It does not prove that a language model will answer correctly.

50.24 The naive rank-prefix control

Now ignore freshness and mandatory semantics.

Take candidates by retrieval rank until the next record does not fit.

First:

$$r_{p1}$$

costs 3.

Then:

$$r_f$$

costs 4.

Cumulative cost:

$$7.$$

The next candidate is current mandatory policy v2 with cost 4.

It cannot fit.

The rank-prefix result is:

$$\boxed{\{r_{p1}, r_f\}.$$

50.25 Why the rank prefix is invalid

It contains:

$$r_{p1},$$

which is superseded.

It omits:

$$r_{p2},$$

which is current and mandatory.

Therefore the result fails the declared compiler contract twice.

This is an exact counterexample to:

$$\boxed{\text{high retrieval rank} \Rightarrow \text{valid working context.}}$$

50.26 Truncating the policy is not a repair

After the rank prefix, one budget unit remains.

The current policy record costs 4.

If the compiler cuts the record into a one-unit fragment, it no longer has the declared atomic object.

The fragment can lose:

- record ID;
- version;
- source;
- status;
- payload.

A valid compiler must revise selection or fail.

It must not pretend a record fragment is the full provenance-aware record.

50.27 Atomicity is policy-relative

This chapter treats records as atomic.

That is a modeling choice.

A different system could define safe sub-record segmentation.

If it does, the segments need their own:

- identity;
- provenance;
- ordering;
- truncation semantics;
- validity rules.

The chapter's non-implication is therefore bounded:

arbitrary token-level truncation is not authorized by this atomic-record formalism.

It does not prove that no principled segmentation scheme can exist.

50.28 Compiler validity

Define:

$$\text{Valid}_{\Pi}(K; R, B).$$

The predicate requires:

1. authorization;
2. freshness/version validity;
3. mandatory inclusion;
4. budget compliance;
5. atomicity;
6. required provenance;
7. declared serialization order.

The witness context satisfies all seven.

The rank-prefix control does not.

50.29 Valid context is not correct answer

A model can receive a compiler-valid context and still fail.

It can:

- ignore the evidence;

- misread a version;
- invent unsupported content;
- follow the wrong record;
- fail for reasons unrelated to memory.

Thus:

$$\boxed{\text{Valid}_{\Pi}(K) \not\Rightarrow \text{CorrectAnswer.}}$$

The compiler certifies its transformation.

It does not certify the model.

50.30 Stored is not retrieved

This chapter keeps several non-implications visible:

$$\text{stored} \not\Rightarrow \text{retrieved.}$$

A record can exist but never enter the candidate set.

50.31 Retrieved is not included

Likewise:

$$\text{retrieved} \not\Rightarrow \text{included.}$$

A record can be excluded because it is:

- stale;
- unauthorized;
- optional but over budget;
- lower utility under the declared policy;
- in conflict.

50.32 Included is not used

And:

$$\text{included} \not\Rightarrow \text{used by model.}$$

The context can contain the right evidence while the model ignores it.

This stage separation prevents one success metric from being misreported as another.

50.33 Budget is an abstract resource

This chapter writes:

$$B$$

as a compiled-context budget.

It can represent:

- tokens;
- bytes;
- slots;
- another declared resource.

The accounting must be explicit.

The chapter does not assume all model APIs count context identically.

50.34 Prompt construction becomes replayable

A compiled context should have a receipt.

Record:

- query/invoke ID;
- compiler policy version;
- context budget;
- candidates;
- versions/statuses;
- authorization decisions;
- freshness exclusions;
- mandatory records;
- selected optional records;
- total cost;
- serialization order;
- source identities;
- explicit failure code.

Now context construction can be replayed.

That is stronger than saying:

we put the most relevant things in the prompt.

50.35 Replay does not validate retrieval quality

Suppose the compiler receipt replays perfectly.

The retriever may still have omitted the most useful record.

Replayability tells us what the compiler did with its inputs.

It does not prove the inputs were ideal.

That requires separate retrieval evidence.

50.36 Freshness and ranking answer different questions

Freshness asks:

is this the version allowed by the invocation policy?

Ranking asks:

how does the retrieval system order candidates under its scoring rule?

Those questions can disagree.

The exact witness deliberately makes them disagree.

Rank 1 is stale.

Rank 3 is current and mandatory.

The compiler resolves the conflict through policy, not wishful interpretation of the rank.

50.37 Provenance and relevance answer different questions

A record can be highly relevant and poorly sourced.

A record can be authoritative and irrelevant to the current task.

Compilation may need both dimensions.

This chapter preserves provenance metadata.

It does not define a universal method for scoring source quality.

50.38 Shared memory needs invocation-local compilation

External memory can be shared across agents or model instances.

Working context is typically invocation-local.

Therefore the same shared store can compile differently for:

- different tasks;
- different authorization scopes;
- different budgets;
- different freshness policies;
- different mandatory constraints.

Context is a view over memory, not memory itself.

50.39 Context compilation is not memory deletion

Excluding a record from one invocation does not remove it from external memory.

The compiler chooses a working set.

It does not mutate the source store unless a separate operation explicitly does so.

This is another reason memory and context need separate objects.

50.40 Context compilation is not model editing

A compiled prompt can change behavior without changing parameters.

That does not make prompt construction equivalent to parameter editing.

The state loci and update semantics differ.

EXTMEM-001 supplied that distinction.

CONTEXTCOMP keeps it visible.

50.41 Practical compiler protocol

For an invocation:

1. retrieve candidates;
2. record candidate identities and ranks;
3. apply authorization;
4. apply freshness/version rules;
5. detect unresolved conflicts;
6. identify mandatory records;
7. check mandatory feasibility;
8. compute complete compiled costs;
9. select optional records;
10. serialize deterministically;
11. emit context plus compilation receipt;
12. invoke the model.

If any hard constraint fails, emit an explicit compiler failure.

50.42 What this chapter does not claim

It does not claim that:

retrieval rank = truth.

It does not claim that:

declared utility = true model usefulness.

It does not claim that:

more context = better answer.

It does not claim that:

compiler-valid = answer-correct.

It does not claim that one universal record ordering is optimal.

50.43 Non-implications

The chapter rejects:

stored $\nRightarrow$ retrieved,

retrieved $\nRightarrow$ included,

high retrieval rank $\nRightarrow$ fresh/current,

compiled $\nRightarrow$ used by model,

compiler-valid $\nRightarrow$ answer-correct,

utility-optimal under Π $\nRightarrow$ universally optimal context,

and

fits budget $\nrightarrow$ provenance or authorization valid.

50.44 Atlas connections

Retrieval.

Retrieval supplies candidates; compilation determines the invocation working set.

External Memory.

Context compilation consumes versioned provenance-aware records without collapsing memory into prompt text.

Agents.

An agent can compile different working sets for different subgoals while sharing one persistent store.

Evidence systems.

Compilation receipts can preserve which evidence objects were actually placed before a model.

Context and token compression.

Compression can become one optional selection mechanism, but it remains constrained by freshness, authority, provenance, and mandatory content.

Governed adaptation.

Changing compiler policy changes which external state enters model computation and therefore deserves versioned governance.

50.45 Closing view

The model context is small.

The memory can be large.

The job between them is not merely retrieval.

It is compilation.

A serious compiler knows:

- which records are current;
- which records are authorized;
- which records are mandatory;
- what provenance must survive;
- what the budget actually counts;
- which optional records fit;
- how ties are resolved;

- how the final sequence is serialized.

The exact witness is small because the principle is structural.

The highest-ranked record can be stale.

The mandatory record can rank lower.

A naive prefix can fit the budget and still be invalid.

The correct conclusion is:

context construction is a governed bounded transformation from candidate memory records to a replay

That transformation deserves its own mathematics, provenance, and failure semantics.

50.46 References used in this chapter

External authority is inherited through the audited External-Memory Thesis, ATLAS-CH-EXTMEM-001 / AUDIT-031.

Exact source identities and authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-CONTEXTCOMP-001.yaml`

Chapter 51

Tokenization and Representation Boundaries

Epistemic status: audited Information/Representation prerequisites + primary tokenizer sources + Atlas synthesis + exact finite witnesses

Specification: manuscript/specifications/ATLAS-CH-TOKEN-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-TOKEN-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-TOKEN-001.md

A model never receives “text” in the abstract.

It receives a representation of text produced by a boundary convention.

That convention decides where symbols begin and end, which units receive IDs, how long the model’s input sequence becomes, and which distinctions are preserved before the first learned layer acts.

Tokenization is therefore part of the representation interface.

51.1 A ladder of boundaries

Several units are routinely called characters or tokens even though they are mathematically different:

1. bytes;
2. Unicode codepoints;
3. grapheme-like visible characters;
4. words under a declared word-boundary convention;
5. subword pieces;
6. token IDs;

7. learned token embeddings.

A count at one level is not a count at another.

That sounds elementary, but many comparisons silently move between these levels.

51.2 Tokenization as a map

For normalized text space $\mathcal{X}$ and vocabulary V , write a deterministic tokenizer as

$$\tau : \mathcal{X} \rightarrow V^*.$$

A stochastic tokenizer instead defines a conditional distribution

$$q_\tau(s \mid x)$$

over valid segmentations s of x .

Detokenization is another map:

$$\delta : V^* \rightarrow \mathcal{X}'.$$

Whether

$$\delta(\tau(x)) = x$$

holds exactly depends on normalization, whitespace handling, byte conventions, special tokens, and the tokenizer design.

Lossless detokenization is an engineering property, not an automatic fact about all tokenizers.

51.3 Bytes and codepoints are not the same unit

Take the visible string **é** in NFC form.

It is one Unicode codepoint, U+00E9, but two UTF-8 bytes.

Normalize it into the decomposed form **e** plus U+0301.

Now it is two codepoints and three UTF-8 bytes.

Thus:

byte length $\neq$ codepoint length,

and Unicode normalization can change both.

This also warns against treating codepoint count as identical to user-perceived character count.

51.4 Why subwords became useful

Sennrich, Haddow, and Birch introduced a practical BPE-inspired subword strategy for neural machine translation to reduce the rare-word/open-vocabulary problem (Sennrich et al. 2016).

The important conceptual move is between whole-word and character extremes.

A recurring string can become one reusable unit while rarer forms remain decomposable.

That gives a finite vocabulary a route to representing strings it did not store as whole words.

The source establishes this method in its NMT setting.

It does not prove that BPE is universally optimal.

51.5 BPE is an algorithm, not merely a vocabulary

A BPE-style encoder starts from a base symbolization and applies learned pair merges in a declared order or ranking.

The procedure matters.

A list of piece strings is not always enough to reproduce segmentation.

A reproducible tokenizer therefore needs more than “vocabulary size = 32k.”

It should bind, at minimum:

- normalization;
- base symbols or pretokenization;
- merge ranks or equivalent encoder state;
- special-token rules;
- vocabulary/ID mapping.

This is a recurring Atlas lesson: an interface is more than a tensor shape or a list of names.

51.6 Unigram tokenization treats segmentation differently

Kudo’s unigram method assigns scores to candidate pieces and evaluates whole segmentations under a unigram-language-model assumption (Kudo 2018).

That makes segmentation ambiguity explicit.

The same vocabulary can support several legal decompositions of the same string.

One can select the highest-scoring segmentation, or sample alternatives during training.

Those are different inference rules over the same segmentation space.

51.7 Exact ambiguity witness

Let

$$V = \{a, b, ab\}$$

with

$$p(ab) = \frac{1}{2}, \quad p(a) = p(b) = \frac{1}{4}.$$

For **abab**, consider:

$$[ab, ab],$$

$$[a, b, ab],$$

$$[ab, a, b],$$

and

$$[a, b, a, b].$$

Their unigram product weights are

$$\frac{1}{4}, \quad \frac{1}{32}, \quad \frac{1}{32}, \quad \frac{1}{256}.$$

After normalization, the probabilities are

$$\left(\frac{64}{81}, \frac{8}{81}, \frac{8}{81}, \frac{1}{81}\right).$$

The first segmentation is the mode.

It is not the only segmentation admitted by the model.

51.8 SentencePiece moves the boundary again

SentencePiece was designed to train subword models directly from raw sentences rather than requiring conventional pretokenized word sequences (Kudo and Richardson 2018).

This matters because word pretokenization is itself a language-dependent representation choice.

Removing that requirement does not remove every preprocessing decision.

Normalization, vocabulary construction, segmentation model, special-token policy, and byte/codepoint handling still remain part of the interface.

51.9 Token count is a useful but narrow quantity

Define token length

$$L_{\tau}(x) = |\tau(x)|.$$

It directly controls how many discrete input positions a token-level Transformer sees.

But it is not entropy.

It is not automatically code length.

It is not a complete compute model.

Attention, embedding lookup, softmax vocabulary cost, architecture, batching, sequence packing, and hardware all matter downstream.

51.10 Token count is not description length

Under a declared token probability model p , idealized code length is

$$\ell_p(s) = -\sum_i \log_2 p(v_i).$$

Take a two-token sequence in which each token has probability 1/16.

Its idealized length is 8 bits.

Now take a three-token sequence in which each token has probability $1/2$.

Its idealized length is 3 bits.

Therefore:

$$|s_A| < |s_B| \not\Rightarrow \ell_p(s_A) < \ell_p(s_B).$$

This preserves the Information chapter's firewall: description length is defined relative to a probability/coding model.

51.11 Fertility measures segmentation expansion

For corpus C with declared reference-word count $W(C)$, define

$$F_\tau(C) = \frac{\sum_{x \in C} |\tau(x)|}{W(C)}.$$

A fertility of 1 means one tokenizer unit per reference word on average.

A fertility of 2.5 means two and a half units per reference word on average.

The number is useful precisely because it is simple.

It is also easy to overinterpret.

Fertility depends on the corpus, normalization, reference-word convention, and tokenizer.

It is not a fixed property of a language.

51.12 Exact fertility witness

Take the two-word toy corpus:

`unbelievable cats.`

Tokenizer A returns

`un | believ | able | cat | s.`

Five tokens over two reference words gives

$$F_A = \frac{5}{2}.$$

Tokenizer B returns

unbelievable | cats,

so

$$F_B = 1.$$

Tokenizer B produces the shorter model sequence.

That alone does not tell us which tokenizer gives better morphology, likelihood, transfer, robustness, compute efficiency, or downstream quality.

51.13 Morphology gives a different diagnostic

Suppose a declared linguistic analysis places internal boundaries in a word.

Let $B_M(x)$ be those boundaries and $B_\tau(x)$ the tokenizer’s internal boundaries. Both sets must be measured as offsets in the **same declared canonical form of x** and the **same unit** (for example, positions between Unicode codepoints). Byte offsets from one representation cannot be compared directly with codepoint offsets from another.

One can define boundary precision and recall:

$$P_M = \frac{|B_M \cap B_\tau|}{|B_\tau|},$$

$$R_M = \frac{|B_M \cap B_\tau|}{|B_M|}.$$

These ratios are defined only when their respective denominators are nonzero: P_M requires $|B_\tau| > 0$ and R_M requires $|B_M| > 0$. When a boundary set is empty, report the affected quantity as **undefined/not applicable** rather than silently replacing 0/0 with zero or one. A particular evaluation protocol may separately declare a different empty-set convention.

For the toy analysis

un | believ | able,

an identical tokenization scores perfectly.

A segmentation `unbel | ievable` need not hit either declared morphology boundary.

This is a legitimate diagnostic.

It is not a theorem that morphology-aligned tokenization is always superior for neural language modeling.

51.14 Morphology and subword frequency optimize different objects

BPE primarily responds to frequent adjacent symbol patterns under its training procedure.

Unigram tokenization chooses a probabilistic piece inventory and segmentation model.

A linguistic morphology may instead aim to identify stems, affixes, roots, or other language-specific units.

These objectives can agree.

They can also disagree.

A frequent string is not necessarily a morpheme.

A morpheme is not necessarily frequent enough to earn its own vocabulary item.

51.15 Byte-level models remove one tokenizer bottleneck

ByT5 studies models that operate directly on UTF-8 bytes and reports robustness and modeling trade-offs relative to token-level counterparts (Xue et al. 2022).

A byte alphabet can cover arbitrary UTF-8 text without requiring a learned word/subword vocabulary.

That removes one kind of out-of-vocabulary boundary.

It does not make representation free.

Byte sequences are often longer.

Longer sequences alter compute and memory costs.

The empirical trade-off belongs to a declared model and workload.

51.16 Byte-level does not mean text has no structure

A byte-level model still receives an ordered symbol sequence.

UTF-8 itself is an encoding convention.

Unicode normalization still matters.

The model must learn useful regularities across multibyte sequences and longer spans.

“Token-free” therefore means free of a learned word/subword tokenizer in this context, not free of discrete representation choices altogether.

51.17 Multilingual tokenization creates allocation problems

A shared vocabulary has finite capacity.

Across many languages, that capacity must cover different scripts, frequency distributions, morphology, and corpus sizes.

Rust et al. conduct controlled comparisons showing that tokenizer adequacy can materially affect monolingual downstream performance inside multilingual models (Rust et al. 2021).

Their result motivates measurement.

It does not justify reducing multilingual performance to one tokenizer statistic.

Training-data scale, architecture, pretraining objective, vocabulary allocation, and downstream task all remain potential factors.

51.18 A multilingual tokenizer report should be vector-valued

For a language/corpus C_ℓ , useful quantities can include:

- fertility;
- unknown-token rate, if the tokenizer has unknowns;
- byte-per-token or codepoint-per-token expansion;
- vocabulary coverage;
- morphology-boundary alignment;
- sequence-length tails;
- downstream performance under controlled model/data comparisons.

These quantities do not share units.

A single scalar “tokenizer quality” score requires an explicit aggregation rule.

51.19 Token IDs are coordinates, not meaning

After segmentation, each piece receives an ID.

That ID indexes a learned embedding vector.

The Representation chapter’s boundary applies directly:

coordinate $\neq$ semantics.

Changing the ID assignment while consistently permuting the embedding table leaves the representational interface equivalent.

Changing the segmentation itself changes sequence length and decomposition, which is a different intervention.

51.20 Tokenization changes the learning problem

Two tokenizers can detokenize to the same text while presenting the model with different:

- sequence lengths;
- boundary locations;
- frequency distributions;
- embedding lookup events;
- context allocation;
- prediction targets;
- softmax classes.

Thus tokenizer replacement is not a neutral input formatting change.

It changes the model’s representation boundary and often its training objective at the discrete-symbol level.

51.21 Comparing tokenizers requires controlled scope

A credible tokenizer comparison should state:

- normalization;
- tokenizer algorithm and version/state;
- vocabulary size;
- training corpus;
- language mixture;
- special tokens;
- byte/codepoint convention;
- sequence truncation policy;
- model architecture;
- parameter-count accounting;
- training token/byte/character budget;
- evaluation corpus and downstream metric.

Without these fields, “tokenizer A is more efficient” can conceal several different interventions.

51.22 What this chapter does not claim

TOKEN does not claim:

- BPE is universally superior to unigram tokenization;
- unigram sampling is always beneficial;
- byte models are universally more robust or efficient;
- low fertility is always good;
- high morphology alignment is always good;
- multilingual vocabulary sharing is always harmful or beneficial;
- token count alone determines compute;
- tokenizer choice alone determines multilingual performance.

The chapter supplies typed objects and diagnostics, not a universal ranking.

51.23 Handoff to Tokenization as Compression and Interface

ATLAS-CH-TOKENCOMP-001 may now assume:

- tokenization as an explicit representation boundary;
- byte/codepoint/subword separation;
- fertility and relative fertility;
- probability-model code length distinct from token count;
- BPE and unigram algorithms as different tokenizer families;
- morphology-boundary diagnostics;
- multilingual measurement as a vector rather than a single score;
- the exact normalization, segmentation-ambiguity, code-length, and fertility witnesses.

TOKENCOMP must add the harder systems question:

how should vocabulary design trade description length, model compute, embedding/softmax cost, interoperability, and cross-model tokenizer interfaces?

51.24 Durable lesson

Tokenization chooses the discrete units on which the rest of the model operates.

That makes it neither mere preprocessing nor hidden semantics.

It is a representation boundary with measurable costs, ambiguities, and language-dependent effects.

51.25 References used in this chapter

- (Sennrich et al. 2016)
- (Kudo 2018)
- (Kudo and Richardson 2018)
- (Xue et al. 2022)
- (Rust et al. 2021)

Exact source authority, prerequisite identities, and claim boundaries are locked in:

`sources/source-locks/ATLAS-CH-TOKEN-001.yaml`

Chapter 52

Tokenization as Compression and Interface

Epistemic status: audited Tokenization prerequisite + Atlas-owned exact compression/compute/interoperability constructions.

Specification: manuscript/specifications/ATLAS-CH-TOKENCOMP-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-TOKENCOMP-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-TOKENCOMP-001.md

Source lock: sources/source-locks/ATLAS-CH-TOKENCOMP-001.yaml

TOKEN-001 treated tokenization as a representation boundary.

This chapter asks what happens when that boundary is also treated as a compression and systems interface.

The immediate temptation is to rank tokenizers by one number:

fewer tokens is better.

That rule is too coarse.

Vocabulary size changes how many distinct IDs must be represented.

Segmentation changes sequence length.

A model may pay costs that grow differently with sequence length and vocabulary size.

Two systems may need to exchange text while using incompatible token IDs.

The governing rule is:

tokenizer design is a multi-objective representation/interface choice, not a token-count scalar.

52.1 The inherited boundary

TOKEN-001 already established:

$$\tau : \mathcal{X} \rightarrow V^*$$

for a deterministic tokenizer, together with a separate detokenizer:

$$\delta : V^* \rightarrow \mathcal{X}'.$$

It also established that:

- byte length is not codepoint length;
- token count is not description length;
- fertility is corpus-relative;
- vocabulary is not the whole segmentation algorithm;
- token IDs and embeddings are representations, not semantics.

TOKENCOMP keeps all of those firewalls.

52.2 Compression requires a code

A token sequence is not yet a bit string.

To ask how many bits it occupies, we need a code.

One simple toy choice is a fixed-width code over token IDs.

Let:

$$m = |V| \geq 2.$$

Then a fixed-width ID needs:

$$w = \lceil \log_2 m \rceil$$

bits.

For a token sequence of length n :

$$D_{\text{fw}} = nw.$$

This is a code definition.

It is not an intrinsic property of the text.

52.3 Vocabulary size enters the accounting

Suppose one tokenizer halves the number of tokens but increases vocabulary size enough to require more bits per ID.

The two effects compete.

So:

shorter token sequence $\nrightarrow$ shorter fixed-width ID stream.

This is the first compression/interface obstruction.

52.4 Fixed-width length is not entropy coding

TOKEN-001 inherited model-based idealized code length:

$$\ell_p(s) = - \sum_i \log_2 p(v_i).$$

That quantity depends on a declared probability model.

Fixed-width length instead depends only on sequence length and vocabulary cardinality under the declared code.

They can disagree.

A complete comparison must state which description-length convention is being used.

52.5 A tokenizer is part of the codebook

The token-ID stream can be decoded only if the recipient already knows the tokenizer specification.

That can include:

- normalization;
- base alphabet;
- vocabulary;
- token-to-ID mapping;
- merge ranks or unigram scores;
- special-token rules;

- decoder behavior;
- version identity.

If that state is not shared, its description or transmission cost belongs somewhere in the protocol.

Therefore:

$$\boxed{\text{ID-stream length} \neq \text{complete compressor size.}}$$

52.6 Compression ratio needs a baseline

A ratio is meaningful only relative to something.

For byte string x with byte length $B(x) > 0$, define one toy ratio:

$$R_{\text{fw}} = \frac{D_{\text{fw}}}{8B(x)}.$$

This compares a token-ID stream to an 8-bit-per-byte baseline.

It does not count tokenizer state.

It does not claim optimal compression.

52.7 Token count is not compute

A model may perform several kinds of work.

Some costs can grow strongly with sequence length.

Others can grow with vocabulary size.

Still others depend on hidden dimension, sparsity, batching, memory, hardware, or implementation.

So the chapter refuses the shortcut:

$$\text{token count} = \text{compute}.$$

Instead it declares toy cost functions explicitly.

52.8 Two exact compute proxies

For token length n , define:

$$C_{\text{pair}}(n) = n^2.$$

This counts all ordered token-position pairs in a toy pair-interaction model.

For vocabulary size m , define:

$$C_{\text{vocab}}(n, m) = nm.$$

This counts one toy dense token-by-vocabulary scoring grid.

These are not universal model FLOP equations.

They are exact counters inside the declared finite model.

52.9 The finite witness

Take:

$$x = \text{abababab}.$$

It is eight ASCII bytes.

Thus the byte baseline contains:

$$64$$

bits.

We compare two lossless tokenizers.

52.10 Small-vocabulary tokenizer

Let:

$$V_S = \{a, b\}.$$

The tokenizer emits:

$$[a, b, a, b, a, b, a, b].$$

So:

$$n_S = 8, \quad m_S = 2.$$

One bit identifies each of two token IDs.

Hence:

$$D_S = 8 \cdot 1 = 8 \text{ bits.}$$

The toy compute vector is:

$$K_S = (8^2, 8 \cdot 2) = (64, 16).$$

52.11 Piece tokenizer

Let:

$$V_P = \{a, b, ab, x, y, z, w, q\}.$$

The witness text is segmented as:

$$[ab, ab, ab, ab].$$

Thus:

$$n_P = 4, \quad m_P = 8.$$

Three bits are required for one of eight fixed-width IDs.

Therefore:

$$D_P = 4 \cdot 3 = 12 \text{ bits.}$$

The toy compute vector is:

$$K_P = (4^2, 4 \cdot 8) = (16, 32).$$

52.12 The orderings conflict

The piece tokenizer uses fewer tokens:

$$4 < 8.$$

Its pair proxy is smaller:

$$16 < 64.$$

But its fixed-width ID stream is larger:

$$12 > 8.$$

And its dense-vocabulary proxy is larger:

$$32 > 16.$$

Thus the same design change improves one declared cost while worsening others.

52.13 There is no scalar winner without an objective

Write:

$$J_{\tau} = (D_{\text{fw}}, C_{\text{pair}}, C_{\text{vocab}}).$$

Then:

$$J_S = (8, 64, 16),$$

$$J_P = (12, 16, 32).$$

Neither is componentwise smaller.

So neither Pareto-dominates the other in this toy objective space.

To choose a winner we must add:

- weights;
- constraints;
- workload frequencies;
- a model architecture;
- or another explicit objective.

The tokenizer alone does not supply those choices.

52.14 Shorter can still matter

The counterexample does not say token length is irrelevant.

A shorter sequence can reduce any declared cost that increases with sequence length while other variables remain fixed.

The point is narrower:

sequence length is one coordinate, not the whole resource model.

52.15 Vocabulary can buy reusable pieces

A larger vocabulary can encode recurring substrings as single tokens.

That can reduce sequence length.

But the vocabulary also creates state:

- more token identities;
- a wider fixed-width ID alphabet in the toy code;
- potentially more tokenizer specification;
- a different representation boundary.

The useful question is not “large or small vocabulary?”

It is:

which objective vector is being optimized for which corpus and system?

52.16 Corpus distribution matters

Our witness contains only repeated “ab”.

A different corpus can reverse which pieces are useful.

For corpus C , one might compare:

$$\mathbb{E}_{x \sim C}[L_{\tau}(x)]$$

or:

$$\mathbb{E}_{x \sim C}[D_{\text{fw}}(x; \tau)].$$

Those expectations depend on the corpus distribution.

No finite example creates a language-independent ranking.

52.17 Tokenizer interoperability

Now consider two systems that use different tokenizers.

A direct exchange of token IDs is unsafe unless the two sides share exactly the relevant tokenizer state.

Integer ID 17 in one vocabulary need not correspond to ID 17 in another.

Even identical token strings can have different IDs.

The interface must name what crosses the boundary.

52.18 Canonical transport domain

Let:

$$\mathcal{X}_c$$

be a declared domain of canonical byte strings.

Tokenizer A has:

$$\tau_A : \mathcal{X}_c \rightarrow V_A^*,$$

with decoder:

$$\delta_A : V_A^* \rightarrow \mathcal{X}_c.$$

Tokenizer B similarly has:

$$\tau_B, \delta_B.$$

Assume both round trip exactly on the declared domain:

$$\delta_A(\tau_A(x)) = x,$$

$$\delta_B(\tau_B(x)) = x.$$

52.19 Lossless translation theorem

Define:

$$T_{A \rightarrow B} = \tau_B \circ \delta_A.$$

Then:

$$T_{A \rightarrow B}(\tau_A(x)) = \tau_B(x).$$

Applying B's decoder:

$$\delta_B(T_{A \rightarrow B}(\tau_A(x))) = x.$$

So the canonical bytes survive translation exactly.

This is an interface theorem.

It is not a model-behavior theorem.

52.20 The witness translates exactly

For the finite example:

Tokenizer S emits:

$$[a, b, a, b, a, b, a, b].$$

Its decoder concatenates these pieces back to:

$$\mathbf{abababab}.$$

Tokenizer P then emits:

$$[ab, ab, ab, ab].$$

The reverse process recovers the S sequence.

The token identities and sequence lengths change.

The canonical byte string does not.

52.21 Lossless does not mean representation-equivalent

Suppose system A looks up embeddings using table E_A and B uses E_B .

Their internal sequences can be:

$$E_A(\tau_A(x))$$

and:

$$E_B(\tau_B(x)).$$

Canonical byte equality supplies no equality between these vector sequences.

Therefore:

lossless transport $\nrightarrow$ equal internal representation.

52.22 Nor does lossless mean semantic equivalence

Canonical byte equality is a syntactic identity under the declared interface.

It does not prove:

- equal human interpretation under every context;
- equal model probability;
- equal model activation;
- equal downstream answer;
- equal learned semantics.

The representation layer has been translated.

Nothing stronger follows automatically.

52.23 Direct ID interoperability is stronger

A direct token-ID mapping would avoid decode and retokenize.

But that requires more structure.

A simple permutation works only when the token alphabets and segmentation semantics align appropriately.

In the witness, one P token “ab” corresponds to two S tokens.

So no one-to-one token permutation can implement the translation.

Canonical translation is more general.

52.24 Special tokens require protocol semantics

Real token streams may contain control tokens not meant to decode as ordinary text.

Examples can include boundary, role, padding, or control symbols.

Such objects fall outside the simple canonical-byte theorem unless their transport semantics are explicitly defined.

A tokenizer interface therefore needs both:

- text/byte semantics;

- control-token semantics.

The chapter proves only the first finite case.

52.25 Version identity matters

A tokenizer is versioned protocol state.

Changing any of these can change the meaning of an ID stream:

- normalization;
- vocabulary;
- merge order;
- token scores;
- ID assignment;
- special tokens;
- decoder.

So “uses BPE” is not a sufficient interoperability identifier.

Neither is a vocabulary size.

52.26 Three meanings of lingua franca

The phrase **tokenizer lingua franca** can refer to different architectures.

52.26.1 Canonical transport

Systems exchange normalized bytes/text and retokenize locally.

52.26.2 Shared tokenizer

Systems adopt one tokenization specification and exchange its IDs.

52.26.3 Translation layer

Systems keep local tokenizers and use explicit adapters.

These are distinct designs.

52.27 Canonical bytes as a bounded lingua franca

A canonical byte domain has one useful property:

different lossless tokenizers can translate through it without requiring matching vocabularies.

That makes it a candidate **transport interface**.

But this chapter does not claim:

- that every model should use bytes internally;
- that byte transport is always efficient;
- that a particular normalization should be universal;
- that all control tokens can be represented this way;
- that industry has adopted one standard.

“Candidate lingua franca” is therefore an architectural idea, not an adoption fact.

52.28 Shared tokenizers trade interoperability for constraints

A shared tokenizer can remove translation ambiguity at the token-ID layer.

It also couples systems to:

- one vocabulary allocation;
- one normalization;
- one version lifecycle;
- one special-token namespace.

That can be desirable in one architecture and restrictive in another.

No universal choice follows.

52.29 Translation layers preserve local freedom

A translation layer lets each subsystem retain its own tokenizer.

That preserves local representation choices.

But it adds:

- encode/decode work;
- protocol state;
- version management;
- possible normalization hazards;
- control-token mapping obligations.

Again, interoperability has costs.

52.30 Compression and interoperability are different objectives

Tokenizer S compresses the witness’s fixed-width ID stream better than P.

That tells us nothing by itself about which is easier to standardize across systems.

Conversely, a widely shared tokenizer could improve ID-level interoperability without minimizing description length on a particular corpus.

Thus:

$$\boxed{\text{compression quality} \neq \text{interoperability quality.}}$$

52.31 Compute and interoperability are different objectives

Tokenizer P reduces the witness's pair proxy.

That does not make its token IDs meaningful to another system.

A compute optimization inside one model can worsen a protocol boundary if it increases specialization or version coupling.

System objectives must remain typed.

52.32 A practical comparison vector

A tokenizer/interface comparison might track:

$$M_\tau(C) = (L, D, F, C_{\text{pair}}, C_{\text{vocab}}, I_{\text{roundtrip}}, I_{\text{control}}, Q),$$

where the coordinates could denote:

- token length;
- declared description length;
- fertility;
- compute proxies or measured costs;
- round-trip interoperability;
- control-token interoperability;
- downstream quality.

These coordinates do not share one unit.

A scalar score requires an explicit objective.

52.33 Multilingual scope remains explicit

TOKEN-001 already required language/corpus-specific measurement.

TOKENCOMP preserves that rule.

A shared vocabulary may allocate capacity differently across scripts and languages.

A canonical byte interface may be lossless while token lengths vary substantially across local tokenizers.

Neither fact creates a universal multilingual optimum.

52.34 Modeled compute versus measured compute

The exact witness uses:

$$n^2$$

and:

$$n|V|$$

as toy counters.

A real system can have:

- different algorithms;
- sparsity;
- caching;
- batching;
- hardware kernels;
- memory effects;
- parallelism;
- compiler transformations.

Therefore the toy counters must not be relabeled as measured runtime.

A runtime claim requires a runtime experiment.

52.35 What would an empirical comparison require?

At minimum:

- exact tokenizer versions;
- normalization;
- corpus/language distribution;
- vocabulary sizes;
- sequence-length distribution;

- code-length convention;
- model architecture;
- model parameterization affected by vocabulary size;
- training or serving workload;
- hardware/software stack for runtime;
- downstream quality metrics.

Without these, “tokenizer A is more efficient” is underspecified.

52.36 Durable non-implications

The chapter preserves:

fewer tokens $\nRightarrow$ fewer encoded bits,

fewer tokens $\nRightarrow$ lower cost under every compute model,

lossless translation $\nRightarrow$ equal tokenization,

equal decoded bytes $\nRightarrow$ equal embeddings or behavior,

shared interface $\nRightarrow$ universal standard.

52.37 Tokenization as interface design

The central systems lesson is not that one tokenizer should dominate.

It is that a tokenizer boundary carries several contracts at once:

- representation;
- compression;
- compute shape;
- serialization;
- versioning;
- interoperability.

Those contracts can favor different designs.

A mature tokenizer choice therefore begins by naming the objective vector rather than by minimizing token count in isolation.

52.38 References used in this chapter

No new external academic authority is added in TOKENCOMP-001.

Tokenizer mechanisms, probability-model code length, multilingual boundaries, and representation-interface authority are inherited through the audited TOKEN-001 source lock. TOKENCOMP's new compression/compute/interoperability results are exact Atlas-owned constructions.

Exact source identity and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-TOKENCOMP-001.yaml`

Chapter 53

Data Quality, Mixtures, and Contamination

Epistemic status: established empirical dataset results + audited Evidence prerequisite + Atlas synthesis + exact finite corpus witness.

Specification: manuscript/specifications/ATLAS-CH-DATA-001.md

Formal packet: mathematics/derivations/ATLAS-CH-DATA-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/DATA-001.md

Source lock: sources/source-locks/ATLAS-CH-DATA-001.yaml

A dataset is not a bag of text.

It is a governed sampling object with lineage.

Two collections can contain the same visible records and still train different models because they differ in:

- duplication;
- filters;
- provenance;
- partition boundaries;
- sampling weights;
- synthetic-data lineage;
- evaluation overlap.

The governing rule of this chapter is:

data quality is not one scalar property, and contamination is not one predicate.

Every empirical claim must retain the exact dataset construction and evaluation threat model that support it.

53.1 Start with the data object

Use:

$D = (R, S, P, \Phi, \Delta, \mu, E)$.

Here:

- R is the logical record set;
- S contains source and domain labels;
- P records provenance and lineage;
- Φ is the transformation/filtering pipeline;
- Δ contains duplicate and overlap relations;
- μ is the training sampling measure;
- E is the evaluation boundary.

This object is richer than a directory of files.

53.2 Why lineage matters

Suppose two records contain identical text.

One may be copied from the other.

Both may descend from the same upstream source.

One may be human-authored and the other generated.

One may have entered through a benchmark-derived instructional set.

The bytes alone do not recover this history.

Provenance is therefore part of the data object.

This follows directly from the Evidence chapter's rule that support and source identity remain attached to claims.

53.3 Exact duplicates

Let $\text{canon}(r)$ be a declared canonicalization.

Define:

$\text{delta_exact}(a, b) = 1_{\{\text{canon}(a) = \text{canon}(b)\}}$.

Even “exact” requires a method.

Do we normalize whitespace?

Unicode?

HTML?

Case?

Boilerplate?

A deduplication claim without canonicalization semantics is incomplete.

53.4 Near duplicates

Exact equality misses modified copies.

Let $T(r)$ be a token-set representation.

A simple near-duplicate score is token Jaccard:

$$J(a,b) = |T(a) \cap T(b)| / |T(a) \cup T(b)|.$$

At threshold τ :

$$\text{delta_near_tau}(a,b) = 1\{J(a,b) \geq \tau\}.$$

The result depends on both representation and threshold.

Lee et al. demonstrate why exact and near-duplicate structure both matter in language-model corpora.

53.5 Semantic redundancy is different again

Two records can have low lexical overlap while conveying nearly the same information.

Conversely, boilerplate can create high lexical overlap without carrying the same substantive content.

So we need at least three distinct relations:

- exact duplicate;
- near duplicate under a declared representation;
- semantic redundancy under a declared semantic method.

No one relation silently substitutes for the others.

53.6 Deduplication changes the training measure

Suppose a record appears ten times.

Uniform record sampling gives the underlying content roughly ten times the probability mass of a singleton.

Remove nine copies and the empirical training distribution changes.

Therefore:

deduplication is not merely storage compression.

It changes the effective sampling measure unless the sampler explicitly compensates.

This is one reason deduplication can alter model behavior.

53.7 Deduplication itself has policy

A near-duplicate relation can induce clusters.

Then someone must decide:

- which representative survives;
- whether source metadata are merged;
- whether domain labels are preserved;
- whether all cluster members remain addressable;
- whether a canonical source is preferred.

The clustering threshold and representative policy can change corpus composition.

53.8 Cross-domain duplicates are especially revealing

Suppose one article appears on the open web and in a curated book corpus.

If deduplication keeps only the web copy, the text survives but the book-domain count falls.

If mixture weights are later computed from retained counts, the deduplication policy has affected the mixture.

This is not a bug in arithmetic.

It is a consequence of how lineage, clustering, and sampling interact.

53.9 “Clean” is not a mathematical primitive

Dataset pipelines often use words such as:

- clean;
- filtered;
- high quality;
- safe;
- curated.

These words can hide several separate criteria.

Dodge et al.’s analysis of C4 is important because it shows that web-corpus filtering choices change both content and population composition, and that large corpora can contain unexpected material and benchmark examples.

The Atlas therefore treats “clean” as a pipeline label, not a theorem.

53.10 Quality is multidimensional

For application **A**, write a quality feature vector:

$Q_A(r) = (q_{\text{prov}}, q_{\text{valid}}, q_{\text{relevance}}, q_{\text{fresh}}, q_{\text{label}}, \dots)$.

Possible dimensions include:

- provenance completeness;
- parse/format validity;
- domain relevance;
- freshness;
- label reliability;
- duplication burden;
- policy admissibility.

A scalar score may be constructed for a particular application.

The chapter assumes no universal weighting.

53.11 Filters can trade one quality for another

A strict filter can remove malformed text.

It can also remove rare dialects, minority language, domain-specific notation, or valuable edge cases.

A permissive filter can preserve breadth.

It can also preserve noise.

The correct filter depends on the objective and the evidence.

A change in retained distribution should be measured, not inferred from the word “quality.”

53.12 Raw corpus proportions

First declare the counting unit. In this simple discussion use retained records. Suppose domain **d** has n_d retained records.

Raw corpus proportion is:

$$p_d = n_d / \sum_j n_j.$$

This describes the retained corpus under the counting unit being used.

It does not yet say how often domain **d** will appear during training.

53.13 Training mixture weights

A training sampler can declare separate weights:

$w_d \geq 0$

with:

$\sum_d w_d = 1$.

A two-stage sampler may:

1. sample domain d with probability w_d ;
2. sample a record inside that domain.

Then record probability depends on both the domain weight and the number of retained records in that domain.

DoReMi makes mixture proportions an explicit optimizable training variable in its reported experiments.

53.14 Corpus composition and training composition differ

Three distributions should be kept separate:

- raw source/corpus proportions p ;
- post-filter/dedup proportions p' ;
- training sampling weights w .

In general:

$p \neq p' \neq w$.

The difference is often intentional.

The important thing is to document it.

53.15 Mixture weights are an intervention

Changing w changes the examples a model expects to see.

That makes mixture weighting part of the training intervention.

A downstream performance claim should therefore retain:

- domain definitions;
- weights;
- sampler;
- number of draws/tokens;
- any dynamic weight schedule.

A dataset name alone is not enough.

53.16 Synthetic provenance

Add a lineage field:

```
origin(r) in {human,synthetic,mixed,unknown}.
```

This says something about where the record came from.

It does not determine whether the record is correct, useful, novel, or harmful.

Synthetic data should not be collapsed into a scalar quality judgment.

53.17 Recursive generated-data failure mode

Shumailov et al. study recursive training on generated data and report model-collapse behavior in the investigated settings.

The bounded lesson is:

recursive generated-data pipelines can create distributional failure modes that provenance-aware dataset accounting should expose.

The chapter does not infer:

```
synthetic => bad.
```

53.18 Synthetic data need lineage depth

For generated records, useful provenance may include:

- generator model/version;
- prompt or conditioning source;
- decoding policy;
- upstream human or synthetic inputs;
- generation date;
- filtering;
- whether the generator was trained on earlier synthetic generations.

A single Boolean “synthetic” can be too weak to diagnose recursive lineage.

53.19 Train/evaluation contamination

Let **T** be training data and **V** evaluation data.

For overlap predicate **delta**, define:

```
C_delta(e;T)=1
```

when some training record overlaps evaluation item e under δ .

Unweighted item contamination rate is:

$$CR_{\delta}(T,V) = (1/|V|) \sum_{(e \text{ in } V)} C_{\delta}(e;T).$$

If evaluation items or tasks carry unequal weights, those weights define a different measure and must be stated. This is not one universal number.

It is relative to the predicate and partitions.

53.20 Exact overlap is one threat model

An exact-match threat model asks whether the evaluation item itself, under a declared canonicalization, occurs in training.

This is precise.

It is also narrow.

Paraphrased, templated, or answer-bearing variants can evade it.

53.21 Near overlap is a different threat model

A near-duplicate threat model may detect modified copies.

But it introduces:

- representation choice;
- threshold choice;
- false positives;
- false negatives.

So a near-contamination rate should always travel with the detector definition.

53.22 Benchmark examples in web corpora

Dodge et al. found examples from evaluation datasets inside C4.

Lee et al. also measured train-test overlap and showed that deduplication can reduce such overlap in the studied datasets.

These findings justify treating benchmark overlap as an empirical threat worth measuring.

They do not make every lexical overlap equally damaging.

53.23 Exposure is not memorization

Suppose an evaluation item overlaps training.

This establishes an opportunity for exposure.

It does not automatically establish that the trained model memorized the item.

Memorization is a model-behavior claim.

It needs its own evidence.

53.24 Memorization is not causal score inflation

Even demonstrated memorization does not by itself prove that a benchmark score was inflated by that memorization.

A causal claim would ask something like:

what would the model's score have been if the overlapping material
had not been available?

That counterfactual generally requires a stronger experimental design or other causal evidence.

53.25 Benchmark pathology is broader than exact leakage

Potential pathologies include:

- exact item reuse;
- near-duplicate item reuse;
- answer leakage;
- template leakage;
- benchmark-derived instruction examples;
- public benchmark discussions repeated in web data;
- tuning on evaluation feedback.

Each has a different detection problem.

“Contaminated” should therefore name the threat model.

53.26 Benchmark version matters

A benchmark name can hide version changes.

Items may be corrected, expanded, translated, reordered, or reformatted.

A contamination claim should therefore bind to:

- exact benchmark version;
- exact item set;
- canonicalization;
- detector.

Otherwise two analyses can report different numbers while appearing to discuss the same object.

53.27 Exact finite witness

Use five training records:

- t1: web, {alpha,beta,gamma};
- t2: web, exact duplicate of t1;
- t3: books, {alpha,beta,gamma,delta};
- t4: code, {lambda,mu,nu};
- t5: synthetic, {theta,iota,kappa}.

Use three evaluation records:

- e1: {alpha,beta,gamma};
- e2: {alpha,beta,gamma,epsilon};
- e3: {omega,psi,chi}.

The records never change across the following calculations.

53.28 Exact-match contamination in the witness

Under exact token-sequence equality:

- e1 overlaps t1 and t2;
- e2 does not exactly match any training record;
- e3 does not match any training record.

Therefore:

$$CR_{\text{exact}}=1/3.$$

This is an exact statement under one overlap predicate.

53.29 Near-overlap contamination in the witness

Use token-set Jaccard with threshold:

$$\tau=3/4.$$

For e2 and t1:

$$J(e2,t1) = 3/4.$$

Therefore e2 now counts as overlapping.

e1 remains overlapping.

e3 remains non-overlapping.

Hence:

$CR_{near}=2/3$.

The data did not change.

The detection rule did.

53.30 Why this does not make either rate false

The rates answer different questions.

1/3 means:

one third of evaluation items have an exact match under this canonicalization.

2/3 means:

two thirds have a training neighbor at or above this Jaccard threshold.

Neither statement should be reported without its predicate.

53.31 Exact deduplication changes the corpus

Training contains five records.

t1 and t2 are exact duplicates.

After exact deduplication there are four representatives.

The raw domain histogram changes from:

(2,1,1,1)

for web, books, code, synthetic

to:

(1,1,1,1).

The corresponding proportions change from:

(2/5,1/5,1/5,1/5)

to:

(1/4,1/4,1/4,1/4).

53.32 Near-duplicate clustering changes it again

At threshold $3/4$:

$J(t_1, t_3) = 3/4$.

So a graph-clustering policy over pairwise near-duplicate edges places t_1 , t_2 , and t_3 in one connected cluster.

Together with t_4 and t_5 , there are three clusters.

If the cluster representative is chosen from the web copy, the books-domain representative disappears.

This is not a theorem that such a representative policy is good.

It is a demonstration that deduplication policy can alter domain balance.

53.33 Pairwise near-duplication need not be transitive

Suppose A is near B and B is near C .

It need not follow that A is near C .

Therefore a dataset pipeline must distinguish:

- pairwise duplicate edges;
- connected-component clustering;
- complete-link clustering;
- representative-based clustering;
- other grouping policies.

For an exact nontransitivity example at the same $3/4$ threshold, let $A = \{a, b, c\}$, $B = \{a, b, c, d\}$, and $C = \{b, c, d\}$. Token-set Jaccard gives

$$J(A, B) = J(B, C) = \frac{3}{4}, \quad J(A, C) = \frac{1}{2}.$$

Thus A is near B , and B is near C , but A is not near C . Connected-component clustering puts all three in one cluster; a complete-link rule need not. This is a separate toy counterexample, not an alteration of the five-record contamination witness.

The cluster definition is part of the data method.

53.34 Mixture weights in the witness

The raw proportions are:

`p_raw=(2/5,1/5,1/5,1/5).`

The exact-deduplicated proportions are:

`p_dedup=(1/4,1/4,1/4,1/4).`

Now declare training weights:

`w=(1/2,1/4,1/8,1/8).`

These are different from both corpus distributions.

For eight expected domain draws:

`8w=(4,2,1,1).`

The sampling plan is a training decision, not a passive reflection of file counts.

53.35 Why mixture records should be first-class

If a training run reports only:

trained on corpus X

we may still not know:

- domain weights;
- temperature sampling;
- oversampling;
- epoch boundaries;
- token caps;
- curriculum changes;
- dynamic reweighting.

A reproducible run should treat mixture configuration as a first-class artifact.

53.36 Data quality and model quality are not identical

A more carefully curated corpus can still produce a worse model under a different architecture, optimizer, budget, or objective.

Conversely, a model may perform well despite obvious corpus defects.

Therefore:

dataset quality is not defined by one downstream score alone.

The data object and the training system should be evaluated separately and jointly.

53.37 Deduplication precision and recall

A deduplication detector can make two kinds of error.

False merge:

- distinct useful records are treated as duplicates.

Missed duplicate:

- redundant records survive.

Therefore deduplication itself has precision/recall-like tradeoffs relative to a declared duplicate gold standard or evaluation protocol.

One threshold cannot be interpreted without this tradeoff.

53.38 Contamination detection also has precision and recall

A contamination detector can:

- miss paraphrased benchmark content;
- flag legitimate domain overlap;
- confuse generic templates with copied items.

So the detector's own error profile matters.

A high measured contamination rate can reflect a broad detector.

A low rate can reflect a weak detector.

The measurement process itself needs validation.

53.39 Source-level versus span-level overlap

A source document can overlap a benchmark without containing the exact benchmark answer.

A small span can exactly reproduce an answer while the source documents are otherwise unrelated.

Useful overlap units include:

- source;
- document;
- paragraph;
- span;
- item;
- template;
- semantic neighborhood.

The unit should match the evaluation threat model.

53.40 Time can be part of contamination

If an evaluation set was published before the training crawl, exposure may be possible.

A publication date after the final training cutoff does not by itself prove non-exposure: the same item or its source may have existed earlier. Ruling out exposure requires evidence that the relevant item/content was unavailable to the training pipeline before the cutoff.

Therefore useful lineage includes:

- source timestamps;
- crawl dates;
- benchmark release date;
- preprocessing date;
- training cutoff.

Temporal reasoning can eliminate some contamination hypotheses before expensive similarity analysis.

53.41 Public benchmarks create a moving target

Widely discussed benchmarks can appear in:

- tutorials;
- blog posts;
- solution repositories;
- papers;
- forum discussions;
- synthetic instructional data.

Over time, the benchmark becomes part of the public data ecosystem.

That does not make evaluation impossible.

It means the validity claim must become more explicit.

53.42 Synthetic data can carry contamination too

A generator trained on benchmark material can emit benchmark-derived examples.

A synthetic record therefore has at least two relevant lineage questions:

1. who or what generated it?
2. what data influenced the generator?

Synthetic generation does not erase upstream provenance concerns.

It can make them harder to trace.

53.43 Recursive lineage is a graph

For synthetic and transformed data, lineage is naturally a directed graph.

Nodes can include:

- source documents;
- filtering stages;
- model versions;
- prompts;
- generated records;
- downstream datasets.

Edges encode derivation.

A flat source label can lose this structure.

The chapter does not require every corpus to maintain a perfect lineage graph.

It establishes why one may be needed for strong provenance claims.

53.44 Data mixtures can evolve during training

A fixed vector $\mathbf{w}$ is the simplest case.

A curriculum or adaptive sampler can use:

$\mathbf{w}_t$.

Then the effective training distribution changes with training time.

A later Curriculum chapter will study ordering and adaptive selection.

DATA-001 supplies the prerequisite distinction between corpus contents and sampling policy.

53.45 “More data” is underspecified

Adding records can change:

- corpus size;
- domain balance;
- duplication;
- freshness;

- synthetic fraction;
- overlap with evaluation;
- number of unique sources;
- effective sampling weights.

So a scaling statement such as:

performance improved with more data

should identify what actually changed.

53.46 A practical dataset ledger

Before accepting a dataset-dependent empirical claim, record:

Field	Question
records	What is the logical record unit?
sources	Which domains and source IDs contribute?
provenance	What lineage is retained?
processing	Which filters and transformations ran?
duplicate rule	Exact, near, semantic? Which threshold?
clustering	How are duplicate edges grouped?
representative policy	Which record survives a cluster?
raw proportions	What is the corpus histogram?
post-filter proportions	How did processing alter it?
sampling weights	What distribution does training actually draw from?
synthetic lineage	Which records/models are generated?
train/eval boundary	Which exact partitions?
overlap detector	Which unit, representation, threshold?
benchmark version	Which exact item set?
timestamps	Which release/crawl/training dates matter?
claim boundary	Exposure, memorization, or causal score effect?

This ledger makes “the data” inspectable.

53.47 Failure modes

53.47.1 Exact-equals-near

Exact duplicates are treated as the whole redundancy problem.

53.47.2 Near-equals-semantic

A lexical threshold is treated as semantic equivalence.

53.47.3 Dedup-equals-storage

Deduplication is described as compression while its sampling effect is ignored.

53.47.4 Corpus-proportion-equals-mixture

Raw record counts are treated as training probabilities.

53.47.5 Synthetic-equals-bad

Generated provenance is turned into a universal quality judgment.

53.47.6 Overlap-equals-memorization

Detected training/evaluation overlap is described as model memorization.

53.47.7 Memorization-equals-score-inflation

Behavioral recall is treated as proof of a causal benchmark effect.

53.47.8 Clean-equals-quality

A pipeline label substitutes for measured quality dimensions.

53.48 What the exact witness establishes

The companion witness proves:

- exact contamination rate $1/3$;
- near-overlap contamination rate $2/3$ under token-Jaccard threshold $3/4$;
- exact deduplication changes five records to four representatives;
- near-duplicate graph clustering yields three clusters;
- raw domain proportions are $(2/5, 1/5, 1/5, 1/5)$;
- exact-dedup proportions are $(1/4, 1/4, 1/4, 1/4)$;
- declared training weights are $(1/2, 1/4, 1/8, 1/8)$;
- eight expected domain draws under those weights are $(4, 2, 1, 1)$.

These facts require no learned model.

They expose the data semantics directly.

53.49 Downstream handoff

Curriculum Learning — ATLAS-CH-CURRICULUM-001 may now assume:

- corpus lineage;
- exact/near duplicate distinctions;
- overlap-predicate semantics;
- raw versus effective data distributions;
- explicit mixture weights;
- synthetic lineage;
- scoped contamination evidence.

The downstream chapter must independently define:

- ordering;
- difficulty;
- competence;
- pacing;
- automatic curriculum policies.

DATA-001 defines what is being sampled.

CURRICULUM-001 will define how sampling changes over learning time.

53.50 References used in this chapter

- Lee et al., *Deduplicating Training Data Makes Language Models Better*.
- Dodge et al., *Documenting Large Webtext Corpora: A Case Study on the Colossal Clean Crawled Corpus*.
- Xie et al., *DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining*.
- Shumailov et al., *AI models collapse when trained on recursively generated data*.

Exact source identities and authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-DATA-001.yaml`

Chapter 54

Curriculum Learning

Epistemic status: established curriculum-learning literature + audited Data/Optimization prerequisites + Atlas state-aware synthesis.

A curriculum is often summarized as “show easy examples first, then harder ones.”

That description captures one historical intuition, but it is too narrow for a mathematical atlas.

The deeper object is a controller over training experience.

It decides, explicitly or implicitly, what the learner sees next.

Sometimes the controller is fixed before training begins. Sometimes it reacts to measurements of the learner. Sometimes it changes only the probability of sampling different regions of the same corpus. Sometimes it removes examples from the eligible set. Sometimes it uses a nominal difficulty score. Sometimes it uses a signal called learning progress.

These mechanisms are not equivalent.

The central lesson of this chapter is:

Curriculum learning is a control problem over the effective training distribution. A useful curriculum must be defined by its available experiences, observations, state, actions, and update rule. Static difficulty order is one special case, not the definition.

The source tradition begins with curriculum learning as an organized presentation of examples (Bengio et al. 2009), extends to automatic syllabus selection driven by learning-progress signals (Graves et al. 2017), and includes competence-based scheduling that couples example difficulty to a model-training schedule (Platanios et al. 2019).

The audited Data chapter supplies the data-distribution and evidence semantics. The audited First-Order Optimization chapter supplies the optimizer-state and training-recipe boundary.

54.1 The question is not merely “easy or hard?”

Suppose a dataset contains two kinds of examples.

One kind has a lower declared difficulty score than the other.

A fixed easy-to-hard syllabus can sort them and move through that ordering.

But what if the learner has already exhausted what the easy region can teach?

What if the learner is not yet ready for the nominally hard region?

What if two learners at the same training step occupy different states because of initialization, stochastic gradients, routing, or earlier examples?

Then “difficulty” alone is not enough to specify the useful next experience.

A curriculum must answer a stronger question:

Given the learner information that the controller is allowed to observe,
what training experience distribution should be used next?

That is a policy question.

54.2 Three states, not one

At training step t , keep at least three categories separate.

54.2.1 Model state

Let

$$\theta_t$$

denote model parameters or another declared model state.

54.2.2 Optimizer state

Let

$$u_t$$

denote optimizer state: momentum, adaptive moments, schedules, or other quantities required by the optimizer.

The audited Optimization chapter already established that optimizer motion is not determined by the raw gradient alone.

54.2.3 Curriculum state

Let

$$c_t$$

denote controller state.

It can contain:

- syllabus position;
- current competence threshold;
- sampling weights;
- region statistics;
- progress estimates;
- counters;
- eligibility masks;
- controller-bandit state.

These coordinates are not model parameters merely because they evolve during training.

The complete controller need not observe all of θ_t or u_t .

Instead declare an observation

$$z_t = \Omega(\theta_t, u_t, c_t, H_t),$$

where H_t is whatever training/evaluation history is available to the controller.

The observation function matters.

A controller that sees only aggregate loss has a different information contract from one that sees per-domain validation, gradients, probes, or held-out competence tests.

54.3 Curriculum as a policy

Let $\mathcal{X}$ be the available experience space.

A curriculum action a_t can mean many things:

- choose one example;
- choose a bucket;
- choose a domain;
- choose a task;

- set sampling weights;
- set an eligibility threshold;
- select a replay source;
- choose a stochastic syllabus arm.

Write the curriculum policy as

$$\pi_t(a \mid z_t, c_t).$$

Then a sampling kernel produces a training experience

$$x_t \sim Q(\cdot \mid a_t).$$

The learner updates through

$$(\theta_{t+1}, u_{t+1}) = \mathcal{T}_{\text{train}}(\theta_t, u_t, x_t).$$

The controller may update separately. Let f_t denote the declared post-update feedback visible to the controller, such as a loss or evaluation score. Then

$$c_{t+1} = \mathcal{T}_{\text{curr}}(c_t, z_t, a_t, x_t, f_t).$$

This decomposition matters because a curriculum is not the optimizer.

It changes which training evidence reaches the optimizer.

54.4 Open loop and closed loop

The simplest curriculum is open loop.

A deterministic syllabus is

$$a_t = \sigma(t).$$

It can be sophisticated.

It can increase difficulty gradually, cycle domains, interleave tasks, or follow a handcrafted sequence.

It is still open loop if the next action does not depend on current learner observations.

A stochastic open-loop curriculum can use

$$a_t \sim \pi_t(a).$$

Randomness does not make a curriculum adaptive.

A closed-loop curriculum uses feedback:

$$a_t \sim \pi_t(a \mid z_t, c_t).$$

Now the same training step t can produce different curriculum actions for two learners with different observed states.

This distinction is more fundamental than “easy first.”

54.5 Difficulty is a score, not an essence

Let

$$d : \mathcal{X} \rightarrow \mathbb{R}$$

be a declared difficulty score.

Examples of possible scoring rules include:

- sequence length;
- rarity;
- number of compositional steps;
- reference-model loss;
- teacher-model confidence;
- structural depth;
- human difficulty labels;
- corruption level.

Each score embeds assumptions.

A long example can be easy for one model and hard for another.

A sample with high loss can be informative, mislabeled, out-of-distribution, adversarial, or merely underfit.

Therefore:

declared difficulty score $\neq$ intrinsic universal difficulty

Bengio et al. introduced curriculum learning around meaningful organization of examples and explored a continuation-method interpretation (Bengio et al. 2009).

The Atlas keeps that historical role while refusing to turn “easy” into an observer-independent property.

54.6 Competence schedules

A competence-based curriculum can couple the difficulty score to a threshold.

Let

$$q_t$$

be a schedule and define the eligible set

$$\mathcal{X}_t = \{x \in \mathcal{X} : d(x) \leq q_t\}.$$

As q_t grows, more examples become eligible.

Platanios et al. use estimated example difficulty and model competence in a curriculum framework for neural machine translation (Platanios et al. 2019).

The Atlas uses that work as a representative mechanism.

It does not infer:

$$q_t = \text{true competence.}$$

The controller variable is a scheduling construct.

Any stronger claim about actual competence needs its own measurement contract.

54.7 Automated curricula and learning progress

Graves et al. show one way to automate curriculum construction: use signals derived from learning progress as reward for a nonstationary bandit that selects a stochastic syllabus (Graves et al. 2017).

This introduces an important architectural shift.

The curriculum is no longer merely an ordered list.

It becomes a controller that receives a reward-like signal and reallocates training attention.

A generic finite-difference progress signal for region r must also state which metric direction counts as improvement. Let $\eta_r \in \{+1, -1\}$. Define

$$LP_t(r) = \eta_r [m_t(r) - m_{t-w}(r)],$$

where $m_t(r)$ is a declared metric, w is a declared lag, $\eta_r = +1$ for higher-is-better metrics, and $\eta_r = -1$ for lower-is-better metrics.

This signal has a measurement contract.

It depends on:

- the metric;
- the window;
- evaluation noise;
- sampling frequency;
- smoothing;
- region definition;
- whether higher or lower is better.

A different window can reverse which region appears to have the highest current progress.

54.8 Learning progress is not a mechanism read-out

Suppose validation accuracy rises quickly on one region.

That is evidence about the declared behavioral metric.

It does not by itself prove:

- which internal feature was acquired;
- whether the capability will persist;
- whether it is accessible under distribution shift;
- whether the model used the intended mechanism;
- whether later training will erase or mask it.

For curriculum design, the practical consequence is simple.

A progress metric can be useful as a controller signal without being a complete scientific explanation.

That distinction lets us use feedback without overclaiming what feedback means.

54.9 Selection is not the same as reweighting

The audited Data chapter separates the retained corpus from the effective training measure.

Curriculum learning operates on that interface.

Suppose the corpus is $\mathcal{D}$ with base sampling measure μ .

A reweighting curriculum can define

$$\mu'_t(x) = \frac{w_t(x)\mu(x)}{\sum_{x'} w_t(x')\mu(x')}.$$

If every weight stays positive, the support is unchanged even though probability mass moves.

A selector can instead choose

$$S_t \subseteq \mathcal{D}$$

and sample only from S_t .

Now support changes.

This distinction matters for:

- exposure;
- contamination accounting;
- rare domains;
- replay;
- fairness of comparisons;
- target-distribution interpretation.

“Curriculum changed the data” is too vague.

We should say how.

54.10 A finite state-aware separation

The smallest useful witness needs no neural network.

Let the learner state be

$$s = (e, h) \in \{0, 1, 2\}^2.$$

Interpret the coordinates only as toy learning levels.

For this exact witness, the curriculum controller observes the entire toy state:

$$z = s.$$

This full-observability assumption is deliberately stronger than the partial measurements available in most real training systems. The actions E and H denote repeatable experience types, so the same type may be selected on successive updates.

There are two actions:

$$E = \text{nominally easy}, \quad H = \text{nominally hard}.$$

Static difficulty is

$$d(E) = 1, \quad d(H) = 2.$$

The easy update is

$$T_E(e, h) = (\min(2, e + 1), h).$$

The hard update is

$$T_H(e, h) = \begin{cases} (e, \min(2, h + 1)), & e \geq 1, \\ (e, h), & e = 0. \end{cases}$$

The hard coordinate has one toy prerequisite: $e \geq 1$.

Define

$$M(e, h) = e + h$$

and one-step progress

$$R(s, a) = M(T_a(s)) - M(s).$$

Now compare two learner states.

54.10.1 Learner A

At

$$s_A = (0, 0),$$

easy gives

$$R(s_A, E) = 1.$$

Hard gives

$$R(s_A, H) = 0.$$

Easy is uniquely progress-maximizing.

54.10.2 Learner B

At

$$s_B = (2, 0),$$

the easy coordinate is saturated.

Therefore

$$R(s_B, E) = 0.$$

Hard is available and gives

$$R(s_B, H) = 1.$$

Hard is uniquely progress-maximizing.

The nominal ranking

$$d(E) < d(H)$$

is identical for both learners.

The useful next action is not.

Therefore:

static difficulty order $\nRightarrow$ state-uniform next-action optimality

in this finite system.

That is the chapter's exact theorem-grade witness.

54.11 What the witness does not say

The witness does not say that real training contains an “easy coordinate” and a “hard coordinate.”

It does not say that hard examples require a simple prerequisite.

It does not say that immediate progress is the correct objective.

It does not say that a neural model at high performance on easy data is permanently saturated.

It establishes one logical point only:

If training utility depends on learner state, a state-independent ranking can fail to choose the same useful next experience for all learner states.

That is enough to justify treating state-aware curriculum control as a distinct object.

54.12 Equal-budget two-step replay

Start from

$$s_B = (2, 0).$$

Give both curricula exactly two learner transitions.

A fixed easy-then-hard syllabus does:

$$(2, 0) \xrightarrow{E} (2, 0) \xrightarrow{H} (2, 1).$$

Toy progress is

1.

A state-aware immediate-progress policy does:

$$(2, 0) \xrightarrow{H} (2, 1) \xrightarrow{H} (2, 2).$$

Toy progress is

2.

The comparison is intentionally bounded.

Both execute two learner updates.

The witness does not account for controller computation, evaluation overhead, measurement latency, or partial-state estimation. Its repeated H, H action is legal only because the witness explicitly defines H as a repeatable experience type.

54.13 Why immediate progress is not enough

A tempting rule is:

Always train on the region with the largest current learning progress.

That rule can be useful.

It is not automatically optimal.

An action with small immediate gain may unlock a later region.

A region with zero observed progress may already be mastered and need occasional rehearsal.

A region with high progress may be noisy and temporarily flattering the metric.

A region may need exploration before its learning potential can be estimated.

These long-horizon questions belong partly to the later Learning Progress as a Search Operator chapter.

CURRICULUM-001 defines the controller interface that chapter will consume.

It does not pre-solve search.

54.14 Curriculum versus training dose

Suppose two experiments end at the same wall-clock time, but one sees more tokens.

Or they see the same number of examples, but one repeats easy examples more often.

Or they see the same corpus, but one uses a different learning-rate schedule.

Then “curriculum effect” is not yet isolated.

A credible comparison should declare or control, as appropriate:

- optimizer-update count;
- examples or tokens processed;
- repeated exposure;
- batch size;
- optimizer;
- learning-rate schedule;
- model initialization;
- seeds;
- controller evaluation cost;
- data support;
- sampling weights;
- stopping rule;
- tuning procedure.

The required controls depend on the claim.

The principle is inherited from the Atlas evidence discipline:

attribute only what the experiment isolates.

54.15 Curriculum and mixture optimization

Curriculum learning can look like time-dependent mixture optimization.

The overlap is real.

If a curriculum chooses domain weights w_t , it changes the effective sampling distribution.

But the concepts are not identical.

Mixture optimization can seek a fixed or slowly varying distribution without an easy-to-hard semantics.

Curriculum learning can change support, ordering, eligibility, or task structure.

The useful abstraction is not a label.

It is the explicit policy over experiences.

54.16 Stochastic syllabi

A curriculum can be stochastic.

For example,

$$a_t \sim \pi_t(a \mid z_t, c_t).$$

Stochasticity can support:

- exploration;
- robustness to noisy progress estimates;
- coverage;
- softened eligibility;
- bandit-style allocation.

But stochasticity itself proves nothing about exploration quality.

A random controller can be worse than a deterministic one.

The exploration objective and feedback model must be declared.

54.17 When loss is a bad difficulty proxy

Current loss is attractive because it is already available.

But high loss can mean many things.

An example may be:

- genuinely complex;
- mislabeled;
- out-of-distribution;
- rare;
- adversarial;
- corrupted;
- already memorized but evaluated under noise;
- incompatible with the current objective.

Likewise low loss can mean:

- mastered;
- trivial;
- duplicated;
- leaked;
- overrepresented.

Therefore a loss-based curriculum must say what interpretation it assumes.

The Data chapter’s lineage and duplication semantics are directly relevant here.

54.18 The fixed lesson plan and the teacher reading the room

The chapter’s pedagogical analogy is a teacher with two possible modes.

The fixed lesson plan is open loop.

It says:

Lesson 1, then lesson 2, then lesson 3.

The adaptive teacher reads declared evidence about the class and may alter the next exercise.

That is closed loop.

The analogy is useful because it exposes the information path.

It fails if we pretend the teacher sees “understanding” directly.

A machine-learning controller sees measurements.

The quality of those measurements is part of the system.

54.19 Failure modes

54.19.1 Easy-to-hard as dogma

A curriculum can be useful without monotone difficulty.

A learner may need rehearsal, interleaving, or revisiting prerequisite regions.

54.19.2 Competence by naming

A variable called competence is still a variable.

The name does not prove that it measures actual competence.

54.19.3 Progress as mechanism

A rising metric is not a direct scan of internal learning.

54.19.4 Greedy progress as global policy

One-step gain need not maximize long-horizon capability.

54.19.5 Hidden dose changes

If one curriculum trains longer or sees more useful examples, ordering may not be the only causal difference.

54.19.6 Controller overfitting

A closed-loop curriculum can optimize its own progress metric rather than the true downstream objective.

54.19.7 Support drift

Aggressive selection can stop exposing the learner to important regions.

54.19.8 Noise chasing

A controller can repeatedly select regions whose progress estimates are noisy rather than genuinely useful.

54.19.9 Endpoint-only interpretation

An endpoint improvement does not explain when, how, or why the capability emerged.

54.19.10 Ignoring controller cost

A curriculum can reduce learner updates while increasing measurement, scheduling, or data-pipeline cost.

54.20 What later chapters may assume

Learning Progress as a Search Operator may assume:

- a curriculum action space exists;
- open-loop and closed-loop controllers are distinct;
- progress signals have explicit measurement contracts;
- data selection changes the effective training measure;
- learner state and curriculum state are separate;
- static difficulty order need not be state-uniformly optimal;
- curriculum feedback is evidence for control, not automatically evidence for mechanism.

It must independently add:

- experience-space search;
- exploration;
- long-horizon objectives;
- credit assignment;
- uncertainty over progress estimates;
- search stopping rules.

Minimal Curricula and Reasoning Bases remains further downstream.

It may not inherit a claim that the smallest curriculum is the one with the highest short-term progress.

54.21 Epistemic status

Established external literature:

- curriculum learning as organized training-example presentation and continuation-style motivation (Bengio et al. 2009);
- automated stochastic syllabus selection using learning-progress signals in the reported settings (Graves et al. 2017);
- difficulty/competence-based scheduling for NMT in the reported setting (Platanios et al. 2019).

Audited Atlas prerequisites:

- data lineage, sampling measure, mixture semantics, contamination boundaries;
- optimizer and schedule mechanics.

Atlas-owned synthesis:

- curriculum as an explicit control object over the effective training distribution;
- the separation among model, optimizer, curriculum state, and curriculum observation;
- the exact finite state-dependent next-action witness;
- the claim firewall between progress signals and mechanism claims.

Computational witness:

- exact nine-state transition/progress table;
- unique action reversal between $(0, 0)$ and $(2, 0)$;
- equal-two-transition comparison from $(2, 0)$.

Not established here:

- universal curriculum gains;
- universal easy-to-hard optimality;
- direct observability of competence;
- long-horizon optimality of greedy progress;
- mechanism identification from behavioral metrics;
- compute savings in deployed training systems.

54.22 References used in this chapter

- (Bengio et al. 2009) — foundational curriculum-learning formulation and continuation-method interpretation.
- (Graves et al. 2017) — automated curriculum/syllabus selection from learning-progress signals.
- (Platanios et al. 2019) — competence-based curriculum scheduling for neural machine translation.

Exact provenance and authority boundaries are locked in:

`sources/source-locks/ATLAS-CH-CURRICULUM-001.yaml`

Chapter 55

Learning Progress as a Search Operator

Epistemic status: audited Curriculum prerequisite + primary learning-progress/search sources + Atlas synthesis + exact finite witnesses

Specification: manuscript/specifications/ATLAS-CH-PROGRESSEARCH-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-PROGRESSEARCH-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-PROGRESSEARCH-001.md

A curriculum controller asks what the learner should see next.

A search controller asks a stronger question:

Which part of experience space should be investigated next, including regions we have not yet characterized?

That extra phrase changes the problem.

Learning progress can rank known regions. Search must also decide when to leave them.

55.1 The inherited progress signal

The audited Curriculum chapter already defines an oriented progress signal

$$LP_t(r) = \eta_r[m_t(r) - m_{t-w}(r)],$$

with an explicit metric, lag, orientation, noise boundary, and observation scope.

PROGRESSSEARCH keeps all of those restrictions.

A large positive value means only that the declared metric moved in the declared favorable direction over the chosen window.

It does not prove a mechanism was acquired.

55.2 From curriculum control to search

Curriculum control can operate over a known menu of examples, buckets, domains, or distributions.

Search becomes explicit when the controller must also decide which regions deserve measurement, refinement, or generation.

Let the experience space be $\mathcal{E}$. It may be finite, partitioned, or continuously parameterized.

Define the search state

$$\mathfrak{S}_t = (\mathcal{E}, z_t, h_t, \widehat{LP}_t, u_t, g_t, \Phi_t, H, B_t).$$

The new pieces are not cosmetic:

- h_t : what the search process has already tried;
- u_t : coverage or uncertainty;
- g_t : declared generalization-state evidence;
- Φ_t : the search objective or ordering rule;
- H : the horizon used for value/credit;
- B_t : remaining budget.

The search operator is

$$r_t \sim \mathcal{S}_t(\cdot \mid \mathfrak{S}_t).$$

55.3 Why progress is attractive

Progress is different from raw performance.

A region with high performance but no remaining improvement may be saturated.

A region with low performance and no improvement may be presently unlearnable, badly measured, or simply unexplored.

A region with moderate performance and rapid improvement may be a productive frontier.

This intuition appears in progress-sensitive intrinsic-motivation systems and competence-progress goal exploration (Oudeyer et al. 2007) (Baranes and Oudeyer 2013).

Portelas et al. make the search interpretation explicit in a continuously parameterized environment space, using absolute learning progress inside a surrogate teacher/bandit problem (Portelas et al. 2020).

The Atlas adopts the structural idea without importing a universal optimality claim.

55.4 Search needs exploration

If the controller only chooses the largest progress estimate among regions it has already measured, an unseen region can remain unseen forever.

That is exploitation, not complete search.

Search therefore needs a declared rule for unobserved or uncertain regions.

Possible rules include:

- forced coverage;
- random exploration;
- optimism/uncertainty bonuses;
- posterior sampling;
- novelty or diversity constraints;
- hierarchical refinement of promising regions.

No one rule is privileged by definition.

55.5 Exact exploration witness

Use two regions A and B.

A has been measured and gives progress 1.

B is unobserved but, when sampled in the toy system, gives progress 3.

An exploit-only policy restricted to observed regions chooses A twice:

$$1 + 1 = 2.$$

A coverage-first policy samples B once, observes 3, then selects B again:

$$3 + 3 = 6.$$

Therefore:

greedy over observed regions $\nrightarrow$ discovery of a better unobserved region .

The result is finite and exact.

It does not say that forced coverage is always best.

55.6 Generalization-state evidence is not one scalar

The GSD boundary distinguishes evidence about:

- acquisition;
- persistence;
- accessibility;
- behavioral expression.

Write

$$g_t(r) = (g_t^{acq}, g_t^{pers}, g_t^{access}, g_t^{expr})(r).$$

These coordinates can disagree.

A capability can appear acquired but fail persistence checks.

It can persist but become inaccessible to one readout.

It can be accessible but not behaviorally expressed under the tested distribution.

Search may use these observations, but it must not silently collapse them into a mechanism label.

55.7 Heterogeneous evidence needs an ordering rule

Progress, uncertainty, persistence evidence, cost, and novelty do not necessarily share units.

A controller may use a scalarization, lexicographic rule, constrained optimization, or multi-objective policy.

But that rule is part of the algorithm.

Writing

$$\Phi_t(\widehat{LP}, u, g, c, B, H)$$

does not itself justify adding the arguments numerically.

55.8 Immediate progress can be the wrong horizon

Learning progress is often measured locally in time.

Search value can be delayed.

Consider a two-step toy system.

Action G gives immediate progress 2 but moves to a state with no second-step gain.

Action I gives immediate progress 0 but unlocks a second-step action X worth 5.

Immediate greed chooses G and obtains total return 2.

The path I,X obtains 5.

Thus:

$$\boxed{\text{max immediate progress} \not\Rightarrow \text{max finite-horizon return}}.$$

55.9 Credit assignment is separate from search

Suppose an experience chosen now is followed by several updates before a later validation gain appears.

Which earlier experience gets credit?

A declared H-step return can be written

$$G_t^{(H)} = \sum_{k=0}^{H-1} \gamma^k r_{t+k}.$$

But that is only a temporal aggregation rule.

It does not prove causal attribution.

If several data regions, optimizer updates, or interventions occur in between, the attribution method must be stated separately.

55.10 Absolute learning progress needs care

Some systems use

$$|LP_t(r)|$$

rather than signed progress.

This can be useful when any strong change signals a region worth revisiting.

But magnitude is not direction.

A large absolute change may reflect improvement, forgetting, degradation, noise, or nonstationarity.

Therefore:

$$|LP| \neq \text{beneficial learning.}$$

55.11 Search changes the training intervention

A search controller can change:

- which regions are sampled;
- how often they are sampled;
- which regions are never sampled;
- how much controller computation is spent;
- the order and dependence structure of training experience.

That means search can change the effective training distribution.

A fair experiment must still inherit the Curriculum comparison obligations for training dose, optimizer schedule, exposure, stopping rule, and controller overhead.

55.12 Search can overfit its metric

A controller may repeatedly select regions that look productive only because the progress estimate is noisy or easy to game.

A search policy can therefore overfit its own feedback channel.

Useful checks include:

- held-out evaluation;
- alternate windows;
- persistence checks;
- delayed reevaluation;
- cross-region validation;
- robustness to progress-estimator noise.

These checks reduce ambiguity but do not turn the signal into a mechanism oracle.

55.13 Search does not mean permanent novelty

Exploration is not synonymous with choosing something new at every step.

Once a productive region is found, exploitation can be rational.

Likewise, revisiting an old region can be rational when forgetting or transfer changes its value.

The correct balance depends on the declared objective and horizon.

55.14 Search over generated experience

The experience space need not be a fixed dataset.

A region can parameterize:

- a simulator;
- a synthetic-data generator;
- a task template;
- a goal distribution;
- a transformation family;
- a problem generator.

In that case, the search operator chooses conditions under which new experience is produced.

This makes the support itself adaptive.

The provenance of generated experience must therefore remain explicit.

55.15 What search has not yet solved

Even a good progress-search operator does not answer:

- what the smallest sufficient curriculum is;
- which learned mechanisms are transferable;
- which reasoning operations form a reconstructive basis;
- whether the same basis works across model scales;
- whether early mechanisms remain latent after later training.

Those are downstream questions.

55.16 Handoff to Minimal Curricula and Reasoning Bases

ATLAS-CH-MINCURR-001 may now assume:

- explicit experience-space search;

- progress estimates with their observation contract;
- an exploration/coverage state;
- delayed-credit and finite-horizon semantics;
- generalization-state evidence as a non-oracular vector;
- exact examples where exploit-only and immediate-greedy choice fail.

MINCURR must add a criterion of minimality and reconstructability.

Finding a high-progress region is not the same as finding a minimal transferable basis.

55.17 Durable lesson

Learning progress can do more than order a syllabus.

It can become feedback for where to look next.

But once progress becomes a search signal, the missing pieces become unavoidable:

search = progress signal+exploration+credit+horizon+budget+declared evidence semantics.

The point is not to worship learning progress.

The point is to turn it into a governed search signal whose limits are visible.

55.18 References used in this chapter

- (Oudeyer et al. 2007)
- (Baranes and Oudeyer 2013)
- (Portelas et al. 2020)

Exact source authority, prerequisite identities, and claim boundaries are locked in:

`sources/source-locks/ATLAS-CH-PROGRESSSEARCH-001.yaml`

Chapter 56

Minimal Curricula and Reasoning Bases

Epistemic status: audited Progress Search and Residual substrates + classical teaching/machine-teaching authority + Atlas-owned exact finite minimality witness.

Specification: manuscript/specifications/ATLAS-CH-MINCURR-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-MINCURR-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-MINCURR-001.md

Source lock: sources/source-locks/ATLAS-CH-MINCURR-001.yaml

A curriculum can be useful without being minimal.

An experience can produce rapid learning progress without belonging to the smallest basis needed for a declared capability.

A later capability can be expressed without proving that an early mechanism survived.

Those are three different problems.

This chapter separates them.

The central question is:

what is the smallest admissible early basis from which the capability we care about can be reconstructed?

The answer is always relative to a declaration.

56.1 Progress search is not yet minimality

Learning Progress as a Search Operator supplied a controller for where to look next.

It can use:

- learning-progress estimates;
- exploration;
- coverage state;
- delayed credit;
- finite horizons;
- generalization-state evidence.

That controller can find useful experience.

But it does not answer:

which experiences are strictly necessary and jointly sufficient for the target capability?

That is the new question.

56.2 The Residual supplies the right discipline

The Residual already established that minimality is not meaningful in the abstract.

One must declare:

- the capability;
- admissible transformations;
- admissible descriptor class;
- reconstruction class.

MINCURR applies the same discipline to early experience and reasoning bases.

A “minimal curriculum” means minimal relative to a target and a reconstruction contract.

56.3 The smallest exact witness

Let:

$$Q = \{q_1, q_2\}.$$

Consider four possible capability hypotheses:

$$H = \{h_{00}, h_{01}, h_{10}, h_{11}\}.$$

Each hypothesis is completely described by two outputs.

For example:

$$h_{10}(q_1) = 1, \quad h_{10}(q_2) = 0.$$

Choose target:

$$h^* = h_{11}.$$

The target capability therefore answers both probes with one.

56.4 A declared reconstruction rule

Use target-consistent labeled experiences:

$$e_1 = (q_1, 1),$$

$$e_2 = (q_2, 1).$$

For a finite set T of labeled experiences, first restrict it to the target probe family:

$$T_Q = \{(q, y) \in T : q \in Q\}.$$

Define the **target-relative** version space by

$$V_H(T) = \{h \in H : \forall (q, y) \in T_Q, h(q) = y\}.$$

An auxiliary experience with probe outside Q imposes no constraint on this reconstructor. This is a declared target-identification convention, not a claim that the experience is useless for learning other capabilities.

The reconstructor succeeds only when:

$$|V_H(T)| = 1.$$

This is a very simple learner.

That simplicity is useful because the minimality proof can be exact.

56.5 Two examples are sufficient

Take:

$$T^* = \{e_1, e_2\}.$$

The first example says:

$$q_1 \mapsto 1.$$

That removes:

$$h_{00}, h_{01}.$$

The second says:

$$q_2 \mapsto 1.$$

Together the two examples leave only:

$$\boxed{V_H(T^*) = \{h_{11}\}}.$$

The target is reconstructed exactly.

56.6 No strict smaller set works

With no examples:

$$|V_H(\emptyset)| = 4.$$

With only e_1 :

$$V_H(\{e_1\}) = \{h_{10}, h_{11}\}.$$

With only e_2 :

$$V_H(\{e_2\}) = \{h_{01}, h_{11}\}.$$

Neither singleton identifies the target.

Therefore:

$$\boxed{|T^*| = 2}$$

is the exact minimum for this declared problem.

56.7 This is teaching-set minimality

Goldman and Kearns formalized teaching complexity through the size of sets that uniquely identify a target concept within a concept class (Goldman and Kearns 1995).

The Atlas witness is a tiny exact instance of that idea.

The point is not that every curriculum problem is a teaching-dimension problem.

The point is that “minimal examples” becomes mathematically meaningful only after the target class and learner semantics are fixed.

56.8 Machine teaching makes the optimization direction explicit

Ordinary learning asks what the learner can infer from given data.

Machine teaching reverses the direction:

given the learner and target, what training set should a teacher provide?

Zhu frames machine teaching as finding an optimal training set for a specified learning algorithm and target (Zhu 2015).

MINCURR uses that inverse-design perspective without claiming that real neural learning is identical to the finite witness.

56.9 Minimality is conditional

Suppose we already knew the target belonged to:

$$\{h_{10}, h_{11}\}.$$

Then one example:

$$(q_2, 1)$$

would suffice.

The minimum has changed because the side information changed.

So:

minimality is relative to the declared hypothesis/capability problem.

56.10 Capability identity can be weaker than model identity

The finite witness uniquely identifies one hypothesis.

Real systems often do not require that.

Suppose two internal systems behave identically on the entire declared probe family.

Then capability reconstruction can succeed even if internal states differ.

This is the Residual connection.

A curriculum can be minimal for:

capability equivalence

without being sufficient to reconstruct the whole model.

56.11 Now add a high-progress distractor

Introduce an auxiliary experience:

$$e_3 = (z, 1),$$

where:

$$z \notin Q.$$

This auxiliary region is outside the declared target capability family.

Freeze progress scores:

$$p(e_1) = 1,$$

$$p(e_2) = 1,$$

$$p(e_3) = 5.$$

So e_3 is the most attractive region under a pure current-progress score.

56.12 But it tells us nothing about the target class

Because z is outside the target probe family:

$$V_H(\{e_3\}) = H.$$

Therefore:

$$|V_H(\{e_3\})| = 4.$$

The auxiliary experience leaves target ambiguity unchanged.

By contrast, either e_1 or e_2 cuts the target version space from four hypotheses to two.

56.13 Search value and basis value can disagree

We therefore have:

$$p(e_3) > p(e_1), p(e_2),$$

but:

$$e_3 \notin T^*.$$

So:

highest current learning progress $\nrightarrow$ membership in a minimal reconstructive basis.

This does not make progress useless.

It says the two objectives are different.

56.14 Search asks where to learn next

A progress-search controller can ask:

where is the learner currently changing fastest?

A minimal-basis analysis asks:

what information or operation is necessary to reconstruct the declared target capability?

These questions can agree.

They do not have to.

56.15 Minimal set is not optimal sequence

The witness has one two-example set:

$$\{e_1, e_2\}.$$

Under the declared order-insensitive reconstructor, both orders:

$$(e_1, e_2)$$

and:

$$(e_2, e_1)$$

work.

Thus:

minimal set $\nRightarrow$ unique optimal sequence.

Sequence optimization requires learner dynamics.

56.16 Why reasoning bases are harder than teaching sets

A teaching set is observable.

A reasoning basis is partly mechanistic.

We might want to say that broad capability was built from a small early collection of:

- operations;
- circuits;
- heuristics;
- primitives;
- subroutines.

But that claim requires more than showing that a small dataset was sufficient.

It requires evidence about what was acquired and what survived.

56.17 Four distinct questions

For a candidate early mechanism m , ask separately:

1. was it acquired?
2. did it persist?
3. is it still accessible?
4. is it behaviorally expressed?

Those are different propositions.

56.18 Acquisition

Acquisition means we have evidence that mechanism m existed or was used during an early learning interval.

Possible evidence can include:

- a local mechanistic probe;
- a causal intervention;
- a decodable representation;
- a training-local behavior.

This is time-local evidence.

It does not tell us what happens later.

56.19 Persistence

Persistence means the same declared mechanism remains later.

That immediately raises an identity problem:

what counts as “the same” mechanism?

Possible identity criteria include:

- matched functional signature;
- aligned mechanistic subspace;
- matched causal intervention effect;

- reconstructive equivalence under a declared map.

The identity criterion must be declared.

56.20 Acquisition does not prove persistence

A mechanism can be:

- overwritten;
- transformed;
- replaced;
- absorbed into another computation.

Therefore:

acquisition $\nrightarrow$ persistence.

Longitudinal evidence is required.

56.21 Accessibility

A mechanism can persist without being reachable through the current interface.

It may be inaccessible to:

- the current prompt;
- routing;
- the active policy;
- the current readout;
- the current controller.

Therefore:

persistence $\nrightarrow$ accessibility.

Accessibility is interface-relative.

56.22 Expression

A mechanism can be accessible but not normally expressed.

Another route may dominate.

The context may suppress it.

A controller may avoid it.

So:

accessibility $\nRightarrow$ expression in every context.

56.23 Later expression does not prove early persistence

Suppose a later model succeeds on the same behavior.

That could happen because:

- the original mechanism persisted;
- it was relearned;
- a different mechanism replaced it;
- a compensating route emerged.

Thus:

later behavioral success $\nRightarrow$ persistence of a particular early mechanism.

56.24 This matters for developmental narratives

It is tempting to tell a story:

the model learned primitive A early, primitive B later, and broad reasoning emerged by composing them.

That story may be correct.

But behavior alone does not establish it.

A mechanistic developmental claim needs evidence at each link.

56.25 A minimal reasoning basis must specify composition

Suppose candidate operations are:

$$r_1, \dots, r_k.$$

A claim that subset:

$$R^*$$

is sufficient requires an admissible composition class:

ℳ.

Otherwise “reconstructable from the basis” is underspecified.

This is exactly the lesson inherited from the Residual.

56.26 Strict-smaller failure is essential

A set can be sufficient without being minimal.

To establish relative minimality, one must attack strict smaller candidates.

The finite witness does this exhaustively.

In a larger system, exact enumeration may be impossible.

Then the burden shifts to:

- lower bounds;
- adversarial ablations;
- impossibility arguments;
- certified search.

56.27 High progress can be auxiliary

The e_3 control illustrates a practical danger.

An auxiliary task can be:

- rapidly improving;
- informative for representation learning;
- useful for exploration;

yet not belong to the minimal basis for the declared target capability.

The correct conclusion is not that e_3 is worthless.

It is that usefulness is objective-relative.

56.28 Low progress can still be necessary

The converse can also happen.

An experience region can show little current progress because the learner already handles it well.

Yet it may still encode a constraint that is necessary to identify or preserve the target capability.

Therefore progress magnitude alone is not a necessity test.

56.29 Minimal basis can change with capability scope

A basis sufficient for:

$$\mathcal{Q} = \{q_1, q_2\}$$

may fail when the target probe family expands.

If we add:

$$q_3,$$

new examples or operations may become necessary.

Therefore:

minimal for one probe family $\nrightarrow$ minimal for a broader capability.

56.30 Minimal does not mean unique

Many systems can admit several distinct minimal bases of equal size.

A curriculum study should distinguish:

- minimum size;
- one minimizer;
- all minimizers;
- equivalence classes of minimizers.

The Atlas witness happens to be unique only because its declared target example language is tiny.

56.31 Operational versus semantic reconstruction

A basis may semantically contain enough information to reconstruct a capability.

That does not mean an allowed learner can efficiently use it.

So MINCURR inherits the RESIDUAL distinction:

$$\text{semantic sufficiency} \neq \text{operational accessibility}.$$

A claimed basis should say what reconstruction class is allowed.

56.32 A general machine-teaching formulation

Let:

- L : declared learner;
- θ^* : target;
- $\mathcal{E}$: admissible experiences;
- $c(T)$: curriculum cost.

Then a teaching problem can be written:

$$\min_{T \subseteq \mathcal{E}} c(T)$$

subject to:

$$L(T) = \theta^*.$$

For capability-relative reconstruction, replace exact parameter identity by:

$$B(L(T), q) = B^*(q) \quad \forall q \in \mathcal{Q}.$$

The objective must say which notion is intended.

56.33 Early mechanisms require a longitudinal protocol

To claim an early reasoning basis persists, a study should freeze:

- early acquisition criterion;
- mechanism identity criterion;
- later persistence probe;
- accessibility interface;
- behavioral probe family;
- causal test where mechanism claims are made.

Then it should evaluate each separately.

56.34 A useful empirical disposition grammar

A future MINCURR experiment can distinguish:

56.34.1 MINIMAL_RECONSTRUCTIVE_BASIS

A candidate basis is sufficient and every admissible strict smaller candidate is ruled out.

56.34.2 SUFFICIENT_NOT_MINIMAL

The candidate reconstructs the target but minimality has not been shown or a smaller basis exists.

56.34.3 SEARCH_RELEVANT_NOT_BASIS

An experience region is useful/high-progress but not required for the declared reconstruction target.

56.34.4 PERSISTENT_INACCESSIBLE

A mechanism satisfies the declared persistence criterion but fails the current accessibility criterion.

56.34.5 BEHAVIOR_ONLY

Later behavior is present without evidence identifying a persistent early mechanism.

This grammar prevents several common promotions.

56.35 The exact witness in one line

For the finite target:

$$h^* = (1, 1),$$

we have:

$$V_H(\{e_1, e_2\}) = \{h^*\},$$

but:

$$|V_H(T)| > 1$$

for every strict smaller:

$$T \subsetneq \{e_1, e_2\}.$$

Meanwhile:

$$p(e_3) = 5$$

is maximal while:

$$V_H(\{e_3\}) = H.$$

That is the chapter's core separation.

56.36 Non-implications

MINCURR rejects:

highest progress $\nrightarrow$ minimal-basis membership,

minimal for one target $\nrightarrow$ universal minimality,

minimal set $\nrightarrow$ optimal sequence,

acquisition $\nrightarrow$ persistence,

persistence $\nrightarrow$ accessibility,

later behavior $\nrightarrow$ early-mechanism persistence,

and:

behavioral equivalence $\nrightarrow$ mechanistic identity.

56.37 Atlas connections

Learning Progress as a Search Operator.

Progress can guide experience search without solving minimality.

The Residual.

Task-relative sufficiency and declared reconstruction classes provide the minimality language.

Mechanistic diagnostics.

Mechanism persistence needs longitudinal identity and causal evidence.

Generalization-state dynamics.

Later behavioral state is evidence about behavior, not automatic evidence about which early mechanism survived.

Compression and intelligence probes.

A minimal reasoning basis is related to compact reconstructive structure but is not identical to compression ratio.

56.38 Closing view

The smallest useful curriculum is not simply the region where learning is happening fastest.

And the smallest early basis is not automatically the mechanism that later behavior uses.

Minimality is a reconstruction claim.

Persistence is a longitudinal claim.

Accessibility is an interface claim.

Behavior is an expression claim.

The exact finite witness makes the first distinction formal:

$$T^{\star} = \{(q_1, 1), (q_2, 1)\}$$

is minimal for the declared target, while the highest-progress auxiliary experience contributes nothing to target identification.

The broader research programme begins when the same discipline is applied to learned operations:

identify the smallest reconstructive basis, then separately test whether it was acquired, whether it persisted, whether it remains accessible, and whether later capability actually uses it.

56.39 References used in this chapter

- (Goldman and Kearns 1995)
- (Zhu 2015)

Exact source identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-MINCURR-001.yaml`

Chapter 57

Reinforcement Learning and Control

Epistemic status: established reinforcement-learning/control foundations plus Atlas synthesis and exact finite derivation.

Specification: manuscript/specifications/ATLAS-CH-RLBASE-001.md

Formal packet: mathematics/derivations/ATLAS-CH-RLBASE-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-RLBASE-001.md

Source lock: sources/source-locks/ATLAS-CH-RLBASE-001.yaml

A dynamical system tells us how state changes.

A control problem adds a question:

which change should we choose?

Reinforcement learning enters when the consequences of those choices are uncertain, delayed, only partly modeled, or learned from interaction.

The central object is therefore not “reward” by itself.

It is a coupled system containing state, action, dynamics, reward, policy, and future consequences.

The discipline of reinforcement learning begins by keeping those objects separate.

57.1 A controlled stochastic dynamical system

For this chapter, write a discounted Markov decision process as

$M = (S, A, P, r, \gamma, \rho_0)$.

Here S is the state space.

$A(s)$ is the set of admissible actions at state s .

$P(s'|s,a)$ is the transition kernel.

$r(s,a,s')$ is the one-step reward associated with the transition.

γ in $[0,1)$ is the discount factor.

ρ_0 is the initial-state distribution.

A policy is another object:

$\pi(a|s)$.

The policy says how actions are selected.

The transition kernel says what the environment does after an action.

These should not be fused.

A poor policy can act in a perfectly specified environment.

A good policy can fail in a misspecified environment.

And a learned policy may change while the environment dynamics remain fixed.

57.2 The Markov boundary

The word Markov does real work.

The MDP state is assumed to contain enough information that, conditional on the current state and action, the next-state distribution does not depend on the earlier history.

Informally:

the present state is sufficient for predicting the controlled future.

This is a modeling claim.

It can fail.

If two histories are mapped to the same state representation but require different predictions or decisions, the chosen state is not Markov for the purpose at hand.

That failure cannot be repaired merely by applying a more sophisticated Bellman update.

Sometimes the right repair is a richer state.

Sometimes it is memory.

Sometimes the problem is genuinely partially observable.

57.3 Reward is not value

Reward is local.

Value is prospective.

Assume throughout the finite-MDP discussion that one-step rewards are finite; because S and A are finite, the reward table is then bounded.

Let

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

be the discounted return. With $\gamma < 1$ and bounded one-step reward, this series is absolutely bounded.

For a fixed policy π , define the state value

$$V^{\pi}(s) = E_{\pi}[G_t \mid S_t = s].$$

Define the action value

$$Q^{\pi}(s, a) = E_{\pi}[G_t \mid S_t = s, A_t = a].$$

These quantities summarize expected future reward under declared conditions.

They are not the same thing as uncertainty.

A value estimate of 10 does not tell us how uncertain that estimate is.

Nor does reward automatically mean “good” in a broader human sense.

Reward is part of the formal problem specification.

If the reward function encodes the wrong objective, an optimizer can solve the mathematical problem and still produce an outcome the designer did not want.

57.4 Bellman’s recursion

Sequential decision problems become tractable when the future can be folded back into the present.

Condition on the first transition.

Under policy π ,

$$V^{\pi}(s) =$$

```

sum_a pi(a|s)
sum_{s'} P(s'|s,a)
[
r(s,a,s') + gamma V^pi(s')
].

```

This is the Bellman expectation equation.

It says that value equals expected immediate reward plus discounted expected continuation value.

The equation is exact for the declared MDP and policy.

The approximation begins later, when we estimate the quantities with samples, restricted representations, learned models, or finite computation.

That distinction matters.

An approximate neural critic does not make the Bellman identity approximate.

It makes our representation or solution of the identity approximate.

57.5 A two-state control problem

Consider the exact witness used by this chapter.

There are two states.

At s1, the only action gives reward 2 and returns to s1.

At s0, action a gives reward 0 and moves to s1.

Action b gives reward 2 and returns to s0.

Let

$\gamma = 1/2$.

Start with a policy that chooses a at s0.

Then

$$\begin{aligned}
 &V^{\pi}(s_1) \\
 &= \\
 &2 + (1/2)V^{\pi}(s_1),
 \end{aligned}$$

so

$$V^{\pi}(s_1) = 4.$$

At s0,

$$\begin{aligned}
 &V^{\pi}(s_0) \\
 &= \\
 &0 + (1/2)4
 \end{aligned}$$

=

2.

Now ask what would happen if, only at s_0 , we took action b once and then followed the old policy.

Its action value is

$$Q^{\pi}(s_0, b)$$

=

$$2 + (1/2)V^{\pi}(s_0)$$

=

3.

Three is greater than two.

So the old policy contains a locally improvable decision.

Switch s_0 to action b .

The environment has not changed.

The reward function has not changed.

The transition law has not changed.

Only the policy has changed.

Under the new policy,

$$V(s_0)$$

=

$$2 + (1/2)V(s_0)$$

=

4.

The witness therefore gives the cleanest possible policy-improvement story:
same world, different controller, larger value.

57.6 Policy evaluation and policy improvement

This pattern generalizes.

Policy evaluation asks:

What is V^{π} for the current policy?

Policy improvement asks:

Can we choose actions greedily with respect to Q^{π} and obtain a policy no worse than π ?

In the exact finite discounted setting, the policy-improvement theorem answers yes.

Alternating exact evaluation and greedy improvement gives policy iteration.

Applying the optimal Bellman operator directly gives value iteration.

The important point is structural.

Evaluation and improvement are different operations.

One estimates the consequences of a controller.

The other changes the controller.

Blurring them makes it harder to identify where an error entered.

57.7 Optimality is relative

Define the Bellman optimality operator

$$\begin{aligned} & (TV)(s) \\ &= \\ & \max_a \\ & \sum_{s'} P(s' | s, a) \\ & [\\ & r(s, a, s') + \gamma V(s') \\ &]. \end{aligned}$$

The optimal value V^* satisfies

$$V^* = T V^*.$$

An optimal policy chooses maximizing actions.

But the star has a scope.

Optimal with respect to which state representation?

Which admissible actions?

Which transition model?

Which reward?

Which discount?

Which horizon?

Which observation model?

An optimal policy for the wrong MDP is still optimal for the wrong MDP.

This is why later chapters will add uncertainty, optionality, correction capacity, and information value rather than treating a single scalar Bellman optimum as the entire decision problem.

57.8 When the model is known

If P and r are known, we can reason with the model directly.

Dynamic programming applies Bellman operators using expected transitions.

Policy evaluation repeatedly applies the policy Bellman operator.

Value iteration repeatedly applies the optimal Bellman operator.

Policy iteration alternates evaluation and improvement.

This is planning in a known controlled stochastic system.

No interaction sample is needed to define the recursion.

The computational burden may still be large.

But the uncertainty about transition law is conceptually separate.

57.9 Learning from sampled transitions

Reinforcement learning often begins when the model is not used directly.

Instead we observe transitions:

$(S_t, A_t, R_{t+1}, S_{t+1})$.

A temporal-difference update for a state-value estimate has the form

$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$.

The bracketed term compares the old estimate with a one-step bootstrapped target.

This is where dynamics, information, and stochastic approximation meet.

The Bellman equation supplies the fixed-point structure.

The sampled transition supplies local evidence.

The learning rule decides how quickly to move.

These are three distinct objects.

57.10 Q-learning and off-policy control

Q-learning replaces the next-state policy average with a maximum:

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$.

This lets the behavior that generated data differ from the greedy policy defined by the current Q values.

That separation is powerful.

It also creates a distribution question.

Which state-action pairs are actually sampled?

The classical Watkins-Dayman convergence result retains strong conditions.

It does not say that arbitrary nonlinear approximation, replay, bootstrapping, nonstationarity, and off-policy data will converge merely because the update still resembles Q -learning.

The equation can survive while the theorem does not.

57.11 Policy gradients

Bellman-style control is not the only route.

Suppose a stochastic policy π_θ is differentiable in parameters θ .

Define an expected-return objective $J(\theta)$.

Policy-gradient methods estimate

$\nabla_\theta J(\theta)$

and update the policy parameters directly.

For a sampled trajectory, REINFORCE uses the score-function term $\nabla_\theta \log \pi_\theta(A_t|S_t)$ multiplied by an appropriate sampled return. Under the stated sampling assumptions, its expectation recovers the policy-objective gradient.

Williams's REINFORCE family is a foundational example.

This viewpoint is useful because the object being optimized is the policy distribution itself.

Modern actor-critic methods combine policy-gradient and value-estimation roles.

Again, the distinction matters.

The actor changes the policy.

The critic estimates a value-related quantity.

Neither label guarantees that the learned approximation is correct.

57.12 Model-based reinforcement learning

A model can itself be learned.

Then the system may use real experience to improve an estimated transition/reward model and use that model to generate planning updates.

Dyna is a representative architecture for this integration.

The same experience can support at least three activities:

direct value or policy learning;

model learning;

planning through the learned model.

This can improve data efficiency because one real transition can influence many simulated updates.

It also creates model bias.

Planning is only as faithful as the learned model in the states and actions where planning relies on it.

A planner can be internally exact and externally wrong because the model is wrong.

57.13 Partial observability

The MDP assumes the controller has access to the state used by the transition model.

Many real systems do not.

In a partially observable Markov decision process, hidden state S_t generates observations rather than being directly exposed.

The controller acts on history or on an information state derived from history.

A standard choice is the belief state

$$\begin{aligned} b_t(s) \\ = \\ P(S_t=s \mid \text{history}_t). \end{aligned}$$

The belief is a probability distribution over hidden states.

Under the standard POMDP formulation, with the transition/observation model fixed and the posterior updated from the complete available history, the exact belief state can recover a Markov control problem in information space.

That is an elegant transformation.

It is not free.

Exact beliefs may be expensive.

The observation model may be wrong.

Approximate memory may discard information.

And two histories that induce the same approximate representation may still require different decisions.

57.14 What value leaves out

Value is useful precisely because it compresses a future distribution into an expectation.

Compression loses distinctions.

Two actions can have the same expected return and very different downside tails.

Two policies can have similar value and different information gain.

One policy can preserve many future corrective actions while another reaches the same expected return through an irreversible commitment.

A value function does not automatically record these differences.

That is not a defect in the definition.

It is a boundary.

The later decision chapters exist because the Atlas does not want one scalar expectation to silently absorb every notion of good decision making.

57.15 Model-free is not assumption-free

The phrase model-free can sound stronger than it is.

A model-free algorithm may avoid explicitly representing P .

It still relies on assumptions about the data-generating process, reward signal, state representation, stationarity, coverage, function class, optimizer, or learning-rate schedule.

The environment model can be absent from the learned representation while remaining present in the mathematics of why an estimator should work.

This is another recurring Atlas lesson:

not representing an object explicitly does not make the object irrelevant.

57.16 Control and uncertainty are different layers

A value estimate can be wrong.

A transition model can be wrong.

A reward model can be wrong.

An observation model can be wrong.

Future value estimates can be wrong.

These uncertainties interact.

RLBASE-001 does not solve that joint propagation problem.

It establishes the controlled stochastic substrate on which the later uncertainty chapter can operate.

This is why ATLAS-CH-JOINTUNC-001 depends on both this chapter and the Information substrate.

The goal there will not be to add several independent error bars mechanically.

It will be to reason about uncertainty that propagates through one coupled decision system.

57.17 What exploration adds

The Bellman optimality equation tells us what would be optimal if the relevant model or values were known.

Learning has another problem.

Which actions should be tried in order to learn what is not yet known?

An action can have two kinds of consequence:

it can produce reward;

it can produce information.

Those are not the same object.

A low-reward action now may improve future decisions by revealing structure.

A high-reward familiar action may teach us almost nothing.

ATLAS-CH-EXPLORE-001 begins from that gap.

It may assume everything established here:

states, actions, transitions, rewards, policies, returns, values, Bellman operators, and model-based/model-free distinctions.

Its job is to add the value of information and the exploration-exploitation problem.

57.18 The boundary to remember

Reinforcement learning is often summarized as learning to maximize reward.

That slogan is too compressed for the Atlas.

A more useful statement is:

reinforcement learning studies how a controller can improve sequential decisions from interaction when consequences are distributed through time and the relevant dynamics or values are not simply handed to it.

The objects remain distinct.

Dynamics describe what follows an action.

Reward scores a local transition according to the model.

Return accumulates those scores through time.

Value takes an expectation under a policy.

Bellman recursion connects present and future.

Learning estimates or improves the relevant objects from data.

Planning uses a model to reason ahead.

Partial observability moves control into information state.

And optimality remains relative to the problem we chose to write down.

Once those boundaries are explicit, later chapters can ask harder questions without overloading the word value.

57.19 References used in this chapter

The exact source identities and authority scopes are pinned in:

`sources/source-locks/ATLAS-CH-RLBASE-001.yaml`

The external basis includes Bellman's Markovian decision-process work, Sutton and Barto's standard reinforcement-learning treatment, Watkins and Dayan on Q-learning, Williams on REINFORCE, Sutton on Dyna, and Kaelbling, Littman, and Cassandra on POMDPs.

Chapter 58

Exploration and Information Value

Epistemic status: established bandit/exploration foundations plus Atlas synthesis and one exact finite Bayesian decision witness.

Specification: manuscript/specifications/ATLAS-CH-EXPLORE-001.md

Formal packet: mathematics/derivations/ATLAS-CH-EXPLORE-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-EXPLORE-001.md

Source lock: sources/source-locks/ATLAS-CH-EXPLORE-001.yaml

A rational action can look worse now and still be better overall.

That is the core difficulty of exploration.

If every consequence of every action were already known, a controller could simply choose the action with the highest value under its declared objective.

Exploration appears when actions do two things at once:

1. they change the world and produce reward;
2. they change what the decision-maker knows.

The second effect can alter future choices.

That makes information part of the control problem.

A random action is not automatically exploratory.

An uncertain action is not automatically informative.

An informative action is not automatically useful.

And useful information is not automatically worth its acquisition cost.

This chapter develops the mathematical distinctions required to say exactly when exploration has decision value.

58.1 From control to learning while acting

The Reinforcement Learning and Control chapter introduced a controlled stochastic system

$M = (S, A, P, r, \gamma, \rho_0)$

and separated environment dynamics, policy, reward, return, value, model, uncertainty, and observability.

That substrate answers a question of the form:

given a declared control problem, how should future consequences enter present decisions?

Exploration adds another question:

what should we do when some decision-relevant part of the control problem is not yet known, and our actions determine what evidence we will receive?

The difference is subtle.

In ordinary planning with a known model, an action matters because of its reward and state transition.

Under epistemic uncertainty, an action can also matter because of the observation it produces.

An exploratory action is therefore a **dual-purpose action**:

- it acts;
- it probes.

This dual role is what creates the exploration–exploitation tension.

58.2 A minimal Bayesian exploration object

Let **theta** denote an unknown latent parameter.

It might encode:

- an arm mean;
- a transition probability;
- a reward parameter;
- a hidden environment mode;
- the identity of a better controller;
- another bounded source of model uncertainty.

At time t , let

$b_t(\theta)$

be the current belief over θ .

For action a , let

$p(y|\theta, a)$

be the observation law.

After observing y , the belief updates by Bayes' rule:

$b_{t+1}(\theta)$ proportional to $b_t(\theta)p(y|\theta, a)$.

The important feature is not the formula itself.

It is the direction of dependence:

action $\rightarrow$ observation $\rightarrow$ belief $\rightarrow$ future action.

This is the exploration channel.

If an action changes reward but not the future information state, its value is purely instrumental.

If it changes the information state, the continuation value may differ even when its immediate reward is lower.

58.3 Bayes value makes exploration explicit

For a finite horizon, write the value of belief state b at time t as

$V_t(b)$.

Let $C_t(b)$ be the set of actions currently admissible under any declared constraints.

Then the Bayes recursion is

$$V_t(b) = \max_{\{a \in C_t(b)\}} E[r(\theta, a, Y) + V_{t+1}(B(b, a, Y))].$$

The first term is immediate reward.

The second term is continuation value after the observation has changed the belief.

This is where exploration lives mathematically.

An action can lose on the first term and win on the second.

A purely myopic controller ignores that possibility.

A finite-horizon exploratory controller does not.

58.4 The smallest useful example

Consider a two-step decision.

There are two actions.

A known action **S** always pays

$1/2$.

An unknown action **U** pays

theta,

where

theta in $\{0,1\}$.

The prior is

$P(\text{theta}=1)=2/5$, $P(\text{theta}=0)=3/5$.

The immediate expected reward of **U** is

$2/5$.

So at the first step,

$2/5 < 1/2$.

If only the current reward mattered, **S** would be the rational choice.

But pulling **U** reveals **theta** exactly.

That changes the second decision.

If the first action is **S**, no information is gained.

The second action is again **S**.

Total expected reward:

$1/2 + 1/2 = 1$.

If the first action is **U**, the first expected reward is $2/5$.

Then:

- with probability $2/5$, we learn **theta**=1 and choose **U** next for reward 1;
- with probability $3/5$, we learn **theta**=0 and choose **S** next for reward $1/2$.

So the expected second reward is

$(2/5)(1) + (3/5)(1/2) = 7/10$.

The total becomes

$2/5 + 7/10 = 11/10$.

Exploration wins:

$11/10 > 1$.

The lower immediate-reward action is the better two-step action.

That is the entire exploration problem in miniature.

58.5 Exploration cost and future information value

The example can be decomposed exactly.

The immediate cost of exploring rather than choosing the known action is

$1/2 - 2/5 = 1/10$.

Without new information, the best second-step value is

$1/2$.

With the observation from U, expected second-step value becomes

$7/10$.

So the future value of the information is

$7/10 - 1/2 = 1/5$.

The net advantage is therefore

$1/5 - 1/10 = 1/10$.

This decomposition is more useful than the slogan “exploration can pay off.”

It exposes the accounting:

**net exploration value = future decision value of information -
immediate exploration cost**

in this particular two-step construction.

A different horizon, observation model, reward model, or constraint set can reverse the result.

58.6 Information gain is not value of information

Pulling U reveals **theta** exactly.

Before the pull, the entropy of **theta** is

$h_2(2/5)$

bits under base-2 logarithms.

After the pull, the entropy is zero.

So the information gain is positive.

Pulling **S** tells us nothing about **theta**, so its information gain is zero.

It is tempting to conclude:

more information is always more valuable.

That is false.

Information gain measures uncertainty reduction.

Decision value measures how the information changes attainable utility or reward.

The two quantities even have different units.

In the witness:

- information gain is measured in bits;
- value of information is 1/5 reward units.

Howard's decision-theoretic treatment of information value emphasizes exactly this dependence on consequences: uncertainty reduction matters economically only through the decisions it changes (Howard 1966).

An observation can be highly informative about something irrelevant.

Another observation can carry only a small amount of information but resolve the one ambiguity that changes the optimal action.

So the correct question is not merely:

how much uncertainty did we remove?

It is:

which future decision did that information change, and by how much?

58.7 Epistemic uncertainty versus randomness

Suppose an observed reward can be written schematically as

$Y = \mu_{\text{theta}}(a) + \epsilon$.

There are two conceptually different uncertainties.

58.7.1 Epistemic uncertainty

We do not know **theta**.

Additional data may change our belief about it.

58.7.2 Outcome stochasticity

Even if `theta` were known, `epsilon` could remain random.

More data may help estimate the distribution of `epsilon`, but it does not make the underlying process deterministic.

This distinction matters because exploration targets epistemic uncertainty.

An action can have high variance and almost no learning value.

An action can have low variance and be extremely informative.

“Choose the uncertain action” is therefore incomplete advice until we say what kind of uncertainty the action resolves.

58.8 Bandits: the cleanest exploration laboratory

The stochastic multi-armed bandit strips the problem down.

There are K arms with unknown reward laws

$\mu_1, \dots, \mu_K$.

At time t , choose one arm

A_t

and observe only

$Y_t \sim \mu_{A_t}$.

The unchosen rewards are hidden.

There is no nontrivial controlled state transition in the basic model.

That makes the bandit a clean laboratory for the exploration–exploitation problem.

If we repeatedly pull only the arm that currently looks best, we may never discover that another arm is better.

If we explore forever, we continue paying opportunity cost after uncertainty has become small.

The design problem is to trade those pressures over time.

Auer, Cesa-Bianchi, and Fischer gave finite-time analysis for upper-confidence-bound style allocation in stochastic bandits (Auer, Cesa-Bianchi, and Fischer 2002).

The chapter uses that work as a representative optimism mechanism, not as a claim that one UCB formula is universally optimal.

58.9 Optimism: act as if plausible good outcomes matter

A confidence-based exploration rule often has the schematic form

$a_t \text{ in } \operatorname{argmax}_i [\mu_{\hat{i}}(t) + \beta_i(t)]$.

The estimate

$\mu_{\hat{i}}(t)$

summarizes observed performance.

The bonus

$\beta_i(t)$

represents uncertainty or confidence width.

An uncertain arm can therefore be chosen because its plausible upper value remains attractive.

This is **optimism under uncertainty**.

The logic is:

if an action could still plausibly be much better than its current estimate, sample it enough to resolve that possibility.

The uncertainty bonus is not free reward.

It is a device for action selection under incomplete knowledge.

And a confidence interval is not the same mathematical object as a Bayesian posterior.

58.10 Posterior sampling: act according to posterior possibility

Thompson's 1933 paper proposed allocating observations according to the probability that one unknown probability exceeds another (Thompson 1933).

In modern Bayesian language, posterior sampling is often described as:

1. sample a candidate environment parameter from the posterior;
2. choose the action optimal under that sampled parameter;
3. observe the result;
4. update the posterior.

The resulting policy explores because different posterior samples make different actions look optimal.

This differs from UCB.

UCB deliberately inflates uncertain actions through an upper confidence quantity.

Posterior sampling randomizes through the posterior itself.

Both can balance exploration and exploitation.

They do so by different mathematical mechanisms.

58.11 Information-directed exploration

Russo and Van Roy proposed information-directed sampling for online optimization under partial feedback (Russo and Roy 2014).

Its distinctive idea is to compare expected short-term opportunity cost with information gained about the optimal action.

The Atlas does not need the full theory here.

The conceptual contribution is enough:

not all information is equally decision-relevant.

Suppose one action reveals a great deal about nuisance variables but almost nothing about which action is best.

Another reveals only one bit, but that bit identifies the best action.

Raw uncertainty reduction can favor the first.

Decision-directed information can favor the second.

This is why exploration mechanisms should not be collapsed into one slogan such as “seek uncertainty.”

58.12 Information is useful only through a future decision

Define the expected one-step payoff

$$\bar{r}(\theta, a) = E_{Y \sim p(\cdot | \theta, a)}[r(\theta, a, Y)].$$

The no-new-information future value is

$$J_0(b) = \max_{a'} E_b[\bar{r}(\theta, a')].$$

Now imagine taking action a , observing Y , updating the posterior, and then making one more decision.

The expected informed future value is

$$J_1(b, a) = E_Y[\max_{a'} E[\bar{r}(\theta, a') | Y, a]].$$

The one-step value of information is

$$\text{VoI}_b(a) = J_1(b, a) - J_0(b).$$

Under the idealized model, this quantity is nonnegative.

Why?

Because after receiving information, we could always ignore it.

The observation cannot reduce the feasible future value unless acquiring it also changes costs, action availability, timing, or other problem structure.

Those qualifications matter.

Real exploration can be costly.

It can consume time.

It can alter the state.

It can use finite resources.

It can violate constraints.

So the clean nonnegative value-of-information theorem belongs to the idealized information channel, not automatically to every physical probing action.

58.13 Exploration in an MDP is harder than exploration in a bandit

In a bandit, an exploratory pull mainly changes what we know about arms.

In an MDP, an exploratory action can also change where we are.

That creates path dependence.

An action may reveal useful information but move the system into a region from which recovery is difficult.

Another action may preserve future choices but reveal less.

A third may temporarily reduce reward while making an important transition model identifiable.

This coupling between learning and state evolution is one reason exploration in reinforcement learning is more difficult than bandit allocation.

The action is simultaneously:

- an intervention;
- an observation request;
- a state transition;
- a resource expenditure.

The RLBASE distinction between environment, policy, reward, value, model, and observability therefore remains essential.

58.14 Exploration is not randomization

Random action can generate diverse observations.

That does not make randomization a complete exploration strategy.

Useful exploration asks:

- what uncertainty matters?
- what observation would reduce it?
- what action produces that observation?
- what does that action cost?
- how will the observation change a later decision?

Randomness may be part of the answer.

For posterior sampling, randomness is structural.

For epsilon-greedy exploration, random action selection is explicit.

For UCB, the policy can be deterministic given the history.

For information-directed methods, the relevant object is decision-directed information.

So exploration should be defined by its **epistemic role**, not by whether the policy happens to randomize.

58.15 Safe exploration starts by naming the constraint

The phrase **safe exploration** sounds stronger than it is.

There is no useful guarantee until “safe” has mathematical content.

Different applications may mean:

- remain inside an admissible state set;
- keep the probability of a constraint violation below a threshold;
- limit cumulative cost;
- preserve a path back to a designated region;
- maintain performance above a conservative floor.

These are different constraints.

They produce different feasible policy sets.

Moldovan and Abbeel give one explicit formulation for safe exploration in unknown MDPs using an ergodicity-based safety formulation and restrictions to guaranteed-safe policy subsets (Moldovan and Abbeel 2012).

The Atlas takes a narrower general lesson from that source:

a safety requirement changes which exploratory actions are admissible.

It is not simply an extra positive bonus attached to uncertainty.

58.16 Safety can make information expensive

Suppose the most informative action is outside the admissible set.

Then the constrained agent may have to learn more slowly.

That is not necessarily a defect.

It is the mathematical consequence of placing value on something other than information and reward alone.

A safety statement should therefore expose at least:

- the constrained quantity;
- the threshold or admissible set;
- the time horizon;
- whether the guarantee is deterministic, probabilistic, or in expectation;
- the probability/confidence level when applicable;
- the model assumptions under which the guarantee is derived;
- the resulting feasible action or policy set.

Without those details, “safe” is rhetorical rather than technical.

58.17 Information acquisition can destroy optionality

An exploratory action can improve knowledge while reducing future choice.

For example, an irreversible probe may reveal the system’s type while consuming a nonrenewable resource.

A state transition may identify dynamics while entering a region from which some future actions are unavailable.

This means exploration has at least three potentially competing effects:

1. immediate reward or cost;
2. information gain;
3. change in future feasible actions.

The later Optionality chapter will study that third object directly.

This chapter does not import its theory.

It records the boundary:

information value is not the whole value of an exploratory action when the action changes the future action set.

58.18 Exploration and regret are related but distinct

Bandit literature often evaluates exploration policies through regret.

That is useful because exploration pays opportunity cost relative to a comparator.

But regret is not identical to exploration.

A policy can explore for many reasons:

- identify the best arm;
- estimate the model;
- reduce posterior uncertainty;
- protect against misspecification;
- satisfy a constraint while learning;
- gather information needed by a later controller.

The downstream Regret chapter will develop Bayesian and minimax regret and clarify what those quantities control.

This chapter supplies the exploration objects that make that analysis meaningful.

Dependency direction matters.

Regret theory does not retroactively define exploration here.

58.19 Finite horizon versus asymptotic reasoning

The exact witness is finite horizon.

The reason to explore is visible after exactly one future decision.

Bandit theory often studies much longer horizons, including asymptotic behavior.

The distinction matters.

An action worth exploring over 10,000 rounds may not be worth exploring with one round left.

Conversely, information that helps a single decisive future choice can have high finite-horizon value even if asymptotic average reward is irrelevant.

Exploration policy is therefore horizon-dependent.

Any claim that an action is “worth exploring” should make the time scale explicit.

58.20 Failure modes

58.20.1 Myopic exploitation

Choose only the action with highest current expected reward.

Failure: never collect evidence that would change the ranking.

58.20.2 Undirected exploration

Seek novelty or randomness without identifying the decision-relevant uncertainty.

Failure: collect information that does not change useful choices.

58.20.3 Uncertainty conflation

Treat high reward variance as high epistemic information.

Failure: repeatedly sample irreducible noise.

58.20.4 Information-value conflation

Maximize entropy reduction as if every bit had equal utility.

Failure: learn irrelevant facts while ignoring low-entropy decision-critical uncertainty.

58.20.5 Algorithm conflation

Treat UCB, Thompson sampling, and information-directed sampling as interchangeable implementations of one formula.

Failure: erase distinct confidence, posterior, and information-ratio semantics.

58.20.6 Safety laundering

Call exploration “safe” without naming the constraint, horizon, guarantee type, or assumptions.

Failure: a label substitutes for a theorem.

58.20.7 Horizon erasure

Evaluate exploration without saying how many future decisions can benefit from the information.

Failure: information cost and future value are compared on incompatible time scales.

58.21 A practical exploration ledger

A compact exploration analysis can be written as seven questions.

Field	Question
unknown	What decision-relevant quantity is not known?
belief/confidence	How is uncertainty represented?
probe	Which action changes the evidence state?
information	What does its observation reveal?
immediate cost	What reward/resource opportunity is sacrificed now?
future decision	Which later choice can improve because of the information?
constraints	Which probes are admissible and under what guarantee?

This ledger prevents “exploration” from becoming a synonym for trying things.

It forces an explicit link between uncertainty and a later choice.

58.22 What the exact witness establishes

The two-step witness is intentionally small.

It establishes exactly four numerical facts:

- known-first total value: 1;
- unknown-first total value: $11/10$;
- immediate exploration cost: $1/10$;
- future decision value of information: $1/5$.

And it establishes one structural fact:

the lower immediate expected reward action is optimal over the two-step horizon because the observation changes the second decision.

It does not establish that exploration always pays.

It does not establish that information gain alone is a sufficient exploration objective.

It does not establish that any specific named algorithm will behave identically in larger problems.

The witness is a microscope, not a universal theorem.

58.23 Atlas handoff

Regret — ATLAS-CH-REGRET-001.

The next chapter may assume the stochastic bandit object, the exploration–exploitation distinction, exploration cost, and the representative differences among optimism, posterior sampling, and information-directed exploration. It must independently define and analyze Bayesian and minimax regret.

The larger decision sequence is now visible:

`control -> uncertainty -> exploration -> regret -> optionality.`

At this point the Atlas can state the central lesson precisely:

information has decision value only when it changes what can rationally be done later, and that value must be weighed against immediate cost, horizon, state change, and constraints.

58.24 References used in this chapter

- Auer, Cesa-Bianchi, and Fischer, *Finite-time Analysis of the Multiarmed Bandit Problem* (Auer, Cesa-Bianchi, and Fischer 2002).
- Thompson, *On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples* (Thompson 1933).
- Russo and Van Roy, *Learning to Optimize via Information-Directed Sampling* (Russo and Roy 2014).
- Howard, *Information Value Theory* (Howard 1966).
- Moldovan and Abbeel, *Safe Exploration in Markov Decision Processes* (Moldovan and Abbeel 2012).

Exact source identities, the audited RLBASE prerequisite, and source authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-EXPLORE-001.yaml`

Chapter 59

Regret

Epistemic status: established bandit-regret theory + audited exploration prerequisite + Atlas synthesis + exact finite witnesses.

Specification: manuscript/specifications/ATLAS-CH-REGRET-001.md

Formal packet: mathematics/derivations/ATLAS-CH-REGRET-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-REGRET-001.md

Source lock: sources/source-locks/ATLAS-CH-REGRET-001.yaml

Regret measures the price of not acting like a comparator.

That sentence is useful only if the comparator is named.

A regret number does not belong to an algorithm by itself.

It belongs to a complete evaluation contract:

- environment;
- horizon;
- policy;
- reward or loss convention;
- comparator;
- expectation or prior convention.

Change the comparator and the regret can change while the learner does exactly the same thing.

Change the prior and a Bayes-optimal policy can stop being desirable.

Change cumulative interaction cost into final recommendation quality and the preferred algorithm can change again.

The governing rule of this chapter is:

regret is a relational quantity, not an intrinsic property of a trajectory.

That rule is the difference between using regret theory and merely quoting a rate.

59.1 From exploration to opportunity cost

The Exploration chapter established the central learning-while-acting tension.

An exploratory action may sacrifice immediate reward to gain information that improves later decisions.

Regret asks a complementary question:

how much reward was lost relative to a declared comparator over the interaction?

This makes regret especially natural for bandits.

If the learner samples an arm that is not truly best, it pays an opportunity cost.

If that sample reveals useful information, the cost may be worthwhile.

Regret does not say whether the information was worthwhile by itself.

It records the reward shortfall relative to the benchmark.

59.2 The stochastic-bandit object

Let environment `theta` define arm means

`mu_theta(a)`

for actions

`a` in `A`.

Let

`mu_theta^*=max_a mu_theta(a)`.

Define the gap

`Delta_theta(a)=mu_theta^*-mu_theta(a)`.

Let `Pi` be the declared admissible policy class. A policy `pi` in `Pi` chooses actions

`A_1, ..., A_T`

and observes rewards

`Y_1, ..., Y_T`.

Under the stationary stochastic-bandit convention,

$$\mathbb{E}[Y_t \mid H_{\{t-1\}}, A_t] = \mu_{\theta}(A_t).$$

This small structure is enough to expose several regret notions that are often collapsed.

59.3 Pathwise reward shortfall is random

Define the pathwise mean-benchmark regret

$$R_T^{\text{path}}(\theta) = T \mu_{\theta}^* - \sum_t Y_t.$$

The comparator term uses the expected reward of the best fixed arm.

The learner term uses realized rewards.

So R_T^{path} is random.

It can even be negative.

A learner pulling a suboptimal arm can get lucky enough that its realized reward exceeds

$$T \mu_{\theta}^*.$$

That does not make the chosen arm optimal.

It means reward noise beat the mean benchmark on that path.

59.4 Expected or pseudo-regret

A standard stochastic-bandit quantity is

$$\bar{R}_T(\pi, \theta) = \mathbb{E}_{\theta}^{\pi} \left[\sum_t \Delta_{\theta}(A_t) \right].$$

This quantity depends on which arms the learner chooses, not on favorable or unfavorable reward noise after conditioning on those choices.

Bubeck and Cesa-Bianchi develop stochastic and adversarial bandit regret in this comparator-relative framework (Bubeck and Cesa-Bianchi 2012).

Lattimore and Szepesvári give a comprehensive modern treatment across stochastic, adversarial, Bayesian, minimax, and pure-exploration settings (Lattimore and Szepesvari 2020).

Terminology varies across subliterations.

The Atlas therefore writes the formula before attaching the name.

59.5 Why expected pathwise regret equals pseudo-regret here

Under the declared stochastic-bandit model,

$$E[Y_t] = E[\mu_{\theta}(A_t)].$$

Therefore

$$\begin{aligned} E[R_T^{\text{path}(\theta)}] &= T \mu_{\theta}^* - \sum_t E[Y_t] \\ &= E[\sum_t (\mu_{\theta}^* - \mu_{\theta}(A_t))] \\ &= \bar{R}_T(\pi, \theta). \end{aligned}$$

The equality is useful.

The objects remain different.

One is a random pathwise shortfall.

The other is its expected gap accumulation under the declared model.

59.6 Pull counts expose the cost of exploration

Let

$$N_a(T)$$

be the number of times arm a is selected by time T .

Then

$$\bar{R}_T(\pi, \theta) = \sum_a \Delta_{\theta}(a) E[N_a(T)].$$

This formula turns regret into an accounting identity.

Each suboptimal arm contributes:

gap $\times$ expected number of pulls.

A learning policy must sample uncertain arms enough to discover which are suboptimal.

Regret theory asks how efficiently that information cost can be paid.

59.7 Exact witness: same policy, different pathwise regret

Take two Bernoulli arms:

$$\mu(A)=1/4, \mu(B)=3/4.$$

Suppose the learner always pulls A for two rounds.

The pseudo-regret is exactly

$$2(3/4 - 1/4) = 1.$$

But the pathwise mean-benchmark regret is

$$3/2 - (Y_1 + Y_2).$$

If both rewards are zero, regret is

$$3/2.$$

If exactly one is one, regret is

$$1/2.$$

If both rewards are one, regret is

$$-1/2.$$

The same policy in the same environment can therefore produce different pathwise regret values.

The exact expectation is still

$$1.$$

59.8 Negative pathwise regret is not paradoxical

The best arm is best in expectation.

It is not guaranteed to dominate every realized sample path.

This is the same reason a fair or unfavorable gamble can win on one trial.

So a negative pathwise value does not refute the comparator.

It exposes the difference between:

- stochastic performance on one path;
- expected performance under the environment.

A report should say which one it is using.

59.9 Bayesian regret adds a prior

Let ν be a prior over environments.

Define Bayesian regret:

$$BR_T(\pi, \nu) = E_{\{\theta \sim \nu\}}[\bar{R}_T(\pi, \theta)].$$

Now the evaluation weights environments by prior probability.

A policy can be excellent under the prior while weak on rare environments.

That may be rational if the prior is part of the intended decision problem.

It is not a worst-case statement.

The prior belongs in the claim.

59.10 Worst-case regret removes prior weighting

For environment class Θ , define

$$W_T(\pi, \Theta) = \sup_{\{\theta \in \Theta\}} \bar{R}_T(\pi, \theta).$$

This asks:

how bad can the policy be over the declared class?

Rare environments do not receive less weight.

They receive the same worst-case attention as common ones.

The environment class is therefore just as important as the prior was in the Bayesian case.

A worst-case guarantee without the class is incomplete.

59.11 Minimax regret chooses the best worst-case policy

Define

$$R_T^*(\Theta, \Pi) = \inf_{\{\pi \in \Pi\}} \sup_{\{\theta \in \Theta\}} \bar{R}_T(\pi, \theta).$$

The policy is chosen to minimize its worst environment-specific expected regret.

This is a different optimization problem from minimizing

$$BR_T(\pi, \nu).$$

The distinction is not philosophical.

It changes optimal policies in the smallest possible examples.

59.12 Exact Bayes/minimax witness

Consider one decision and two possible environments.

Under environment +:

- A pays 1;
- B pays 0.

Under environment -:

- A pays 0;
- B pays 1.

Let the policy choose A with probability q .

Then

$$R(q, +) = 1 - q$$

and

$$R(q, -) = q.$$

Worst-case regret is

$$\max(1 - q, q).$$

The minimax choice is

$$q = 1/2$$

with regret

$$1/2.$$

Now put prior mass

$$9/10$$

on +.

Bayesian regret is

$$9/10(1 - q) + 1/10 q.$$

It is minimized at

$$q = 1.$$

The Bayes-optimal regret is

$$1/10.$$

But that policy has worst-case regret

$$1.$$

The criterion changed.

So did the optimal policy.

59.13 Bayes performance is bounded by worst-case performance

For a fixed policy and a prior supported on Θ ,

$$BR_T(\pi, \nu) \leq W_T(\pi, \Theta).$$

An average over a set cannot exceed the set's supremum.

Taking the best policy over the same declared class Π on each side gives

$$\inf_{\pi \in \Pi} \text{BR}_T(\pi, \nu) \leq R_T^*(\Theta, \Pi).$$

This inequality does not say that the optimizing policies coincide.

The exact witness shows that they need not.

59.14 Minimax is not prior-free truth

A minimax policy protects against the worst environment in its declared class.

That does not make the class itself uniquely correct.

A class can be:

- too broad;
- too narrow;
- misspecified;
- computationally convenient rather than realistic.

Minimax analysis removes a prior.

It does not remove modeling assumptions.

59.15 Comparator class is part of regret

Regret can be defined against many comparators.

Examples include:

- best fixed action;
- best stationary policy;
- best policy in a hypothesis class;
- best action sequence with at most S switches;
- best dynamic action sequence;
- clairvoyant oracle subject to constraints.

A stronger comparator generally makes the benchmark harder to match.

Two regret numbers against different comparator classes are not the same quantity.

59.16 Exact comparator witness

Consider reward table:

59.17. THE COMPARATOR CAN BE TOO STRONG TO BE USEFUL1005

round	A	B
1	1	0
2	0	1

The learner chooses A twice and earns 1.

Against the best fixed arm:

- A earns 1;
- B earns 1.

Regret is

0.

Against an unrestricted per-round comparator:

- choose A on round 1;
- choose B on round 2.

Comparator reward is 2.

Regret is

1.

Same learner.

Same reward table.

Different comparator.

Different regret.

59.17 The comparator can be too strong to be useful

Suppose a learner must act causally, without future observations.

Now compare it with a clairvoyant sequence that sees all future rewards.

The resulting regret may measure a genuine gap.

But the comparator has access to information the learner never could.

That may be appropriate for one theorem and misleading for another engineering claim.

A regret analysis should state what information the comparator is allowed to use.

59.18 Cumulative regret measures interaction cost

Cumulative regret asks how much opportunity cost was paid throughout learning.

That is natural when rewards during learning matter.

If every exploratory action costs money, time, safety margin, or service quality, the cost accumulates.

Bandit regret theory often focuses on this online objective.

A learner should learn while performing well.

59.19 Simple regret measures final recommendation quality

Pure-exploration problems often care about a different object.

After T samples, the learner recommends an arm

$\hat{a}_T$.

Define simple regret:

$$SR_T = \mu^* - \mu(\hat{a}_T).$$

This ignores how much reward was lost while gathering information.

It asks only:

how good is the final recommendation?

Lattimore and Szepesvári treat pure exploration as a distinct objective family from cumulative-regret minimization (Lattimore and Szepesvari 2020).

59.20 Exact cumulative/simple witness

Suppose rewards are deterministic:

- A gives 1;
- B gives 0.

A learner:

1. pulls B once;
2. pulls A once;
3. recommends A.

Cumulative regret is

- 1.

The B pull paid one unit of opportunity cost.

Final simple regret is

0.

The recommendation is perfect.

So cumulative regret and final recommendation error are different objectives even in a trivial problem.

59.21 Exploration can raise cumulative regret and lower simple regret

This is not a contradiction.

Sampling a doubtful arm can be costly during interaction.

The information can improve the final choice.

That is exactly why pure exploration and online reward maximization require different algorithms and analyses.

The objective must be fixed before the experiment.

Otherwise a method can be declared good after the fact by selecting whichever regret notion flatters it.

59.22 Sublinear regret is an average guarantee

A common goal is

$$\bar{R}_T = o(T).$$

This means

$$\bar{R}_T/T \rightarrow 0.$$

Average regret per round vanishes.

Cumulative regret need not vanish.

It need not even stay bounded.

For example,

$$\bar{R}_T = \log T$$

diverges as T grows while

$$\log T/T \rightarrow 0.$$

So “sublinear regret” does not mean “no regret.”

It means the cumulative cost grows more slowly than the horizon.

59.23 A good asymptotic rate can hide finite-horizon cost

Asymptotics answer what happens as the horizon becomes large.

Real systems operate at finite horizons.

An algorithm with excellent limiting behavior may have poor constants or long transients.

Another may be better for the actual deployment horizon and worse asymptotically.

A regret statement should therefore say whether it is:

- finite time;
- asymptotic order;
- asymptotic constant;
- lower bound;
- upper bound.

These are different claim classes.

59.24 Lai–Robbins: exploration has an asymptotic information price

Lai and Robbins established classical asymptotic efficiency results for adaptive allocation under regular parametric stochastic-bandit assumptions (Lai and Robbins 1985).

The broad lesson is important:

a uniformly good learner cannot simply stop sampling every suboptimal arm immediately.

It must collect enough evidence to distinguish the true environment from alternatives in which that arm would be optimal.

That information requirement produces logarithmic-scale sampling costs in the classical setting.

The exact constants and regularity assumptions belong to that model class.

They are not universal bandit laws.

59.25 Finite-time analysis answers another question

Auer, Cesa-Bianchi, and Fischer gave finite-time stochastic-bandit analysis for upper-confidence methods (Auer, Cesa-Bianchi, and Fischer 2002).

The important Atlas distinction is between:

- asymptotic efficiency;
- explicit finite-T control.

A theorem can be strong in one sense and weak in the other.

Do not silently replace one with the other.

59.26 Regret does not explain the policy mechanism

Two policies can have the same regret for different reasons.

One may explore deliberately.

Another may make estimation mistakes.

A third may be constrained from selecting the best action.

A fourth may be randomized for robustness or fairness.

Regret measures comparative reward performance.

It is not a mechanistic explanation of why actions occurred.

The Exploration chapter supplies those mechanism distinctions.

59.27 Regret and information are different units

Russo and Van Roy's information-directed sampling explicitly couples expected single-period regret with information acquisition (Russo and Roy 2014).

That is useful precisely because the quantities differ.

Regret is measured in reward or loss units.

Information gain is measured in information units.

An information-ratio construction relates them.

It does not identify them.

59.28 Low regret does not imply safety

Imagine a thousand-round policy with excellent cumulative reward that takes one action capable of catastrophic failure.

If the reward objective does not encode the catastrophe at the right scale, low regret can coexist with unacceptable safety.

A safety guarantee requires:

- a constrained set;
- chance constraint;
- risk measure;
- reachability condition;
- conservative baseline;
- or another explicit safety object.

Regret alone does not create it.

59.29 Low regret does not imply calibration

A decision-maker can choose high-reward actions while maintaining badly calibrated probabilities.

If only the selected actions matter to reward, prediction quality away from those decisions may not affect regret.

Calibration therefore requires its own evaluation.

The same is true for uncertainty quality and causal identification.

59.30 Low regret does not imply fairness

A reward comparator may average across users or groups.

Low aggregate regret can coexist with severe distributional disparities.

Unless the reward or constraints encode the relevant fairness property, regret does not measure it.

This is not an objection to regret theory.

It is a reminder that objectives define what is optimized.

59.31 Low regret does not imply robustness

A policy can have low regret on the declared environment class and fail badly outside it.

Worst-case regret is only worst case over

Theta.

If distribution shift moves the world outside **Theta**, the theorem no longer applies automatically.

Robustness requires an uncertainty set, perturbation model, or other shift semantics.

59.32 Low regret does not imply recoverability

A policy can obtain nearly comparator-level reward while entering a state from which future correction is difficult.

If the comparator does not price that loss of future action freedom, regret will not record it.

This is the boundary needed by the next chapter.

Optionality is not a synonym for low regret.

59.33 Expected regret can ignore tail structure

A policy with rare enormous losses can have acceptable expected regret if those events are sufficiently rare or insufficiently penalized.

Applications sensitive to tails may need high-probability or risk-sensitive guarantees.

Expected regret should not be promoted into a tail-risk guarantee.

59.34 Bayesian regret can hide rare-environment failure

The exact Bayes/minimax witness already demonstrates this.

The prior assigns probability 1/10 to the environment where action A is wrong.

Bayes optimization chooses A deterministically.

That is optimal for the declared prior.

Its worst-case regret is still maximal.

Bayesian performance and robustness answer different questions.

59.35 Minimax regret can be conservative under a strong prior

The reverse tradeoff also exists.

If the prior is reliable and highly concentrated, a minimax policy may spend performance protecting against environments believed to be very unlikely.

Whether that is desirable depends on the application.

The mathematics cannot choose the governance stance.

It can expose the consequence of the stance.

59.36 A practical regret ledger

Before reading or reporting a regret result, record:

Field	Question
horizon	Over how many decisions?
environment	Which fixed environment or environment class?
policy	Which policy class may the learner use?
reward/loss	What is accumulated?
comparator	Best fixed action, stationary policy, switching policy, oracle, or something else?
randomness	Pathwise, expected over rewards, or expected over policy randomization?
prior	Is there a Bayesian prior, and what is it?
criterion	Bayesian, worst-case, minimax, high-probability, or another notion?
terminal objective	Is simple/final recommendation regret also relevant?
constraints	What important properties are outside the regret objective?

This ledger prevents rate notation from outrunning semantics.

59.37 What the exact witnesses establish

The companion witness establishes four finite facts.

First, in a two-round Bernoulli problem:

- pseudo-regret is 1;
- pathwise regret can be $3/2$, $1/2$, or $-1/2$;
- expected pathwise regret is exactly 1.

Second, in a one-step two-environment problem:

- minimax policy chooses A with probability $1/2$;
- minimax regret is $1/2$;
- under prior $9/10, 1/10$, Bayes-optimal policy chooses A with probability 1;
- Bayes regret is $1/10$;
- its worst-case regret is 1.

Third, one realized trajectory has regret 0 against the best fixed action and 1 against a dynamic per-round comparator.

Fourth, an exploration sequence has cumulative regret 1 and simple regret 0.

No asymptotic theorem is inferred from these finite calculations.

59.38 Downstream handoff

Optionality and Correction Capacity — ATLAS-CH-OPTIONALITY-001 may now assume:

- environment-specific expected/pseudo-regret;
- pathwise versus expected-regret distinction;
- Bayesian regret;
- worst-case and minimax regret;
- comparator-class dependence;
- cumulative versus simple regret;
- sublinear-regret semantics;
- the negative result that low reward regret does not automatically preserve recoverability or future action freedom.

The downstream chapter must independently define optionality, viable future actions, correction capacity, recoverability, and uncertainty over future opportunities.

It may use regret as one evaluation coordinate.

It may not redefine regret to contain those properties by fiat.

59.39 References used in this chapter

- Lai and Robbins, *Asymptotically Efficient Adaptive Allocation Rules* (Lai and Robbins 1985).
- Bubeck and Cesa-Bianchi, *Regret Analysis of Stochastic and Nonstochastic Multi-Armed Bandit Problems* (Bubeck and Cesa-Bianchi 2012).

- Lattimore and Szepesvári, *Bandit Algorithms* (Lattimore and Szepesvari 2020).
- Auer, Cesa-Bianchi, and Fischer, *Finite-time Analysis of the Multiarmed Bandit Problem* (Auer, Cesa-Bianchi, and Fischer 2002).
- Russo and Van Roy, *Learning to Optimize via Information-Directed Sampling* (Russo and Roy 2014).

Exact source identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-REGRET-001.yaml`

Chapter 60

Optionality and Correction Capacity

Epistemic status: audited Regret prerequisite + established irreversibility/viability/empowerment precedents + Atlas synthesis + exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-OPTIONALITY-001.md

Formal packet: mathematics/derivations/ATLAS-CH-OPTIONALITY-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-OPTIONALITY-001.md

Source lock: sources/source-locks/ATLAS-CH-OPTIONALITY-001.yaml

A system can know exactly what went wrong and still be unable to correct it.

That is the problem this chapter isolates.

The previous Regret chapter made one boundary explicit: low reward regret does not automatically imply recoverability or preserved optionality.

Here we make those missing objects visible.

The central distinction is:

information tells an agent what correction is needed; optionality determines whether a correction remains available.

Those two quantities can move together.

They need not.

60.1 The door that information cannot reopen

Imagine two stage-0 decisions.

One decision preserves both exits until new evidence arrives, but keeping both exits available has a cost.

The other commits to the left exit immediately and costs nothing.

Later, the world reveals whether left or right was needed.

If both policies had different expected values, ordinary value comparison might settle the choice.

The more interesting case is when the preservation cost is tuned so that the two policies have exactly the same prior expected return.

Even then, they do not leave the same future decision problem.

One leaves two corrections.

The other leaves one.

That residue is optionality.

The door metaphor is deliberately limited.

Real control systems rarely have literal doors, and “reopening” may be possible at finite cost rather than impossible.

The mathematics therefore needs to represent both hard loss of feasibility and softer increases in correction cost.

60.2 **Optionality needs a contract**

“Keep options open” is incomplete advice.

Options relative to what?

A usable optionality claim must name at least:

- the state abstraction;
- the future horizon;
- admissible controls;
- viability or safety constraints;
- the uncertainty that may later resolve;
- the outcomes that count as distinct;
- the post-evidence targets;
- the cost allowed for correction.

We package these as

`D_opt=(Theta,b,S,A,T,K,h,G,U,c_corr).`

Here:

- `Theta` is the environment class;
- `b` is a belief over environments when one is used;

- $\mathbf{S}$ is the state space;
- $\mathbf{A}$ is the action correspondence;
- $\mathbf{T}$ is the transition model;
- $\mathbf{K}$ is the declared viability constraint;
- $\mathbf{h}$ is the remaining horizon;
- $\mathbf{G}$ identifies functionally distinct consequences;
- $\mathbf{U}$ describes post-evidence targets or utilities;
- $\mathbf{c_corr}$ measures correction cost.

Without these declarations, “optionality” tends to collapse into metaphor.

60.3 Viable continuations come before valuable continuations

Let

$\mathbf{Pi_h(s)}$

be the admissible continuation policies from state $\mathbf{s}$ for the remaining horizon.

Define the viable set

$\mathbf{V_h(s)}$

as those continuations that satisfy the declared constraints under the declared uncertainty semantics.

This is a feasibility object.

It does not say that every viable continuation is good.

That distinction matters because optionality is not simply a larger value function.

Viability theory provides a mature mathematical language for controlled evolution under state constraints (Aubin 1991).

The Atlas uses that precedent conservatively.

The finite definitions here are ours, but they inherit one discipline from viability theory:

a continuation should not be counted as available when the governing constraints make it inadmissible.

60.4 Why counting buttons fails

Suppose a console has one button.

Now duplicate the button label ten times without changing anything the system can actually do.

The raw action count increased by a factor of ten.

The real choice set did not.

To remove this trivial representation dependence, attach a consequence signature

$G_s(\pi)$

to each viable continuation.

Declare:

$\pi \sim_s \pi'$

when they have the same consequence signature.

Then define the functional option set:

$O_h(s) = V_h(s) / \sim_s$.

A finite count

$N_h(s) = |O_h(s)|$

now counts distinct consequence classes rather than labels.

This is still not absolute.

Change the horizon or the consequence signature and the quotient can change.

That is not a defect.

It is a reminder that optionality is always relative to what distinctions matter downstream.

60.5 Cardinality is not value

Even functional option count can mislead.

One state might offer:

- two useful recoveries.

Another might offer:

- the same two useful recoveries;
- twelve irrelevant actions;
- one catastrophic action.

A larger set is not automatically better.

If a weight function w has independent meaning, one may define:

$W_h(s) = \sum_{o \in O_h(s)} w(o)$.

But the weights then carry substantive assumptions.

The Atlas therefore treats option count as a descriptive coordinate, not a universal utility.

60.6 Irreversibility enters when uncertainty meets time

Arrow and Fisher's classic analysis made a structural point that remains useful far beyond its environmental application: when actions are irreversible and new information may arrive later, preserving the ability to respond can have decision value (Arrow and Fisher 1974).

The Atlas does not import their model wholesale.

We take only the structural lesson:

- uncertainty may resolve after action;
- an early commitment may remove later responses;
- the value of waiting or preserving flexibility depends on the cost of doing so.

This last clause is essential.

Irreversibility does not imply “never commit.”

It makes the timing and cost of commitment part of the decision.

60.7 Recoverability is relative, not mystical

Let B be a declared recovery set.

Define:

$\text{Rec}_h(s; B) = 1$

when some admissible viable continuation reaches B within horizon h .

An action is irreversible relative to B, h when the resulting state has

$\text{Rec}_h = 0$.

This definition exposes four facts that ordinary language often hides.

First, irreversibility depends on the state abstraction.

Second, it depends on the recovery target.

Third, it depends on the available controls and constraints.

Fourth, it depends on the time budget.

A cracked component may be irreversible at the material level and still be operationally recoverable by replacement.

A software deployment may be reversible in principle and effectively irreversible under a five-second deadline.

60.8 A bad outcome is not the same as irreversible action

Suppose a stochastic transition causes a temporary performance drop, but the original operating region remains reachable in one step.

That is a bad outcome.

It is not irreversible.

Conversely, a commitment may have excellent immediate reward while permanently deleting one future response.

That can be irreversible without being immediately bad.

The distinction matters because tail risk and optionality are different evaluation axes.

60.9 Correction cost

After evidence arrives, the relevant question is no longer merely “what can I do?”

It is:

can I still reach an acceptable response for the world I now believe I am in?

For environment θ , let B_θ be an acceptable target set.

After evidence resolves the environment to θ , let $V_h(s; \theta)$ be the viable continuation set under that resolved environment. Define:

$k_h(s, \theta)$

as the least declared correction cost among continuations in $V_h(s; \theta)$ that reach B_θ .

If no such continuation exists:

$k_h(s, \theta) = +\infty$.

This one convention cleanly separates two failure modes.

Soft degradation:

- the correction still exists;
- it simply costs more.

Hard feasibility loss:

- the target is no longer reachable under the declared contract.

The latter is the sharp form of lost correction capacity.

60.10 Conditional correction and ex-ante correction capacity

After evidence resolves the environment to θ , define the conditional correction indicator:

$$C_{\{h, \epsilon\}}(s, \theta) = 1_{\{k_h(s, \theta) \leq \epsilon\}}.$$

This is the post-evidence statement: in this environment, can the declared correction still be executed within tolerance?

Before the evidence arrives, define the ex-ante belief-weighted summary:

$$CC_{\{h, \epsilon\}}(s; b) = \sum_{\theta} b(\theta) C_{\{h, \epsilon\}}(s, \theta).$$

Interpretation:

CC

is the pre-evidence probability mass of environments for which a satisfactory correction will remain reachable within the declared tolerance.

The quantity lies in $[0, 1]$.

It is nondecreasing in ϵ .

It exposes hard infeasibility because an infinite correction cost never enters at finite tolerance.

This is not the only possible optionality functional.

It is useful because its semantics are explicit and testable.

60.11 The exact equal-value witness

Now instantiate the opening story exactly.

Let:

$$\Theta = \{L, R\}$$

with prior:

$$P(L) = P(R) = 1/2.$$

At stage 0 the admissible action set is $\{P, C_L, C_R\}$.

Preserve P :

- pay immediate reward $-1/2$;
- move to state $\mathbf{s}_P$;
- retain terminal actions $\{L, R\}$.

Commit-left C_L :

- pay immediate reward 0;
- move to state $\mathbf{s}_L$;
- retain only terminal action $\{L\}$.

Commit-right C_R is the symmetric admissible commitment:

- pay immediate reward 0;
- move to state $\mathbf{s}_R$;
- retain only terminal action $\{R\}$.

The learner-policy comparison remains P versus C_L . Declaring C_R makes the environment-informed comparator below a member of the declared stage-0 action set.

At stage 1, observe $\mathbf{theta}$ perfectly.

Terminal reward is:

$$u_{\mathbf{theta}}(\mathbf{a})=1$$

when the terminal action matches the environment, and zero otherwise.

The preserve policy waits for the observation and matches it.

The commit-left policy can only choose left.

60.12 Prior value does not separate the policies

For preservation:

$$G(\mathbf{pi}_P, L)=1/2;$$

$$G(\mathbf{pi}_P, R)=1/2.$$

Therefore:

$$E[G(\mathbf{pi}_P)]=1/2.$$

For commitment:

$$G(\mathbf{pi}_L, L)=1;$$

$$G(\mathbf{pi}_L, R)=0.$$

Therefore:

$$E[G(\mathbf{pi}_L)]=1/2.$$

The exact prior action-value gap is zero.

A scalar expected-value comparison is silent.

60.13 Bayesian regret can also tie

Use a clairvoyant environment-informed comparator that knows `theta` before stage 0 and chooses the declared matching commitment `C_theta`.

Its value is:

$$V_L^* = V_R^* = 1.$$

Preservation regret is:

- 1/2 in L;
- 1/2 in R.

So Bayesian regret is:

$$1/2.$$

Commit-left regret is:

- 0 in L;
- 1 in R.

Its Bayesian regret is also:

$$1/2.$$

Thus the two policies tie both in prior expected return and Bayesian regret.

This does not mean every regret criterion ties.

Worst-case regret is:

- preserve: 1/2;
- commit-left: 1.

Optionality is a separate coordinate, not an assertion that all other coordinates are blind.

60.14 The future feasible sets do separate them

At `s_P`:

$$O_1(s_P) = \{L, R\}.$$

At `s_L`:

$$O_1(s_L) = \{L\}.$$

So the functional option counts are:

- preserve: 2;

- commit-left: 1.

This difference is not created by extra labels.

The two preserved options have distinct terminal consequences.

60.15 Correction capacity separates them more sharply

Let the post-evidence target in environment **theta** be:

choose terminal action **theta**.

Give a successful matching correction cost zero.

If the matching action is unavailable, correction cost is infinity.

Then in the preserved state:

$k(s_P, L) = 0$;

$k(s_P, R) = 0$.

Therefore the post-evidence indicators satisfy:

$C_{\{1,0\}}(s_P, L) = C_{\{1,0\}}(s_P, R) = 1$,

and the ex-ante capacity is:

$CC_{\{1,0\}}(s_P; b) = 1$.

In the committed-left state:

$k(s_L, L) = 0$;

$k(s_L, R) = +\text{infinity}$.

Therefore:

$C_{\{1,0\}}(s_L, L) = 1$;

$C_{\{1,0\}}(s_L, R) = 0$;

and the ex-ante capacity is:

$CC_{\{1,0\}}(s_L; b) = 1/2$.

The prior value tie hid a real asymmetry in the ability to act on future evidence.

60.16 Information is identical

The environment is revealed perfectly in both branches.

Thus:

$H(\Theta)=1 \text{ bit};$
 $H(\Theta|Y)=0;$
 $I(\Theta;Y)=1 \text{ bit}.$

The information gain is exactly the same.

Yet correction capacity differs by a factor of two.

This is the cleanest lesson in the chapter:

learning the answer and retaining a feasible response to the answer
are different resources.

60.17 Empowerment is related, but not identical

Klyubin, Polani, and Nehaniv formalize empowerment as an information-theoretic capacity of the action-to-future-sensor channel and explicitly interpret it as keeping options open (Klyubin et al. 2008).

That is an important neighboring idea.

It is not the same object as the Atlas correction capacity.

Empowerment asks how much potential influence an agent can exert and later perceive through its sensorimotor channel.

Correction capacity here asks whether declared target-specific responses remain feasible after evidence arrives.

A state can have high influence capacity over many irrelevant dimensions and still lack the one correction required by a particular future target.

Conversely, a binary switch can have modest channel capacity yet preserve exactly the two corrections that matter.

The two concepts can be compared.

They should not be silently identified.

60.18 Policy optionality is not environmental branching

Suppose the environment can branch into a hundred exogenous states while the controller has only one admissible response.

The world has many possible futures.

The policy has little optionality.

Now suppose the environment has only two states, but the controller retains two distinct corrective actions.

Environmental uncertainty is smaller.

Policy optionality is larger.

Keeping these axes separate prevents “many possible futures” from being mistaken for “many controllable futures.”

60.19 Delay is not automatically recoverability

Waiting can preserve options.

It can also consume them.

A system that postpones maintenance may “delay commitment” while a physical degradation process steadily removes future repair paths.

A model that simply waits may cross a deadline after which rollback is impossible.

Therefore optionality is not procrastination.

The correct question is whether the viable correction set is preserved, not whether a decision was deferred.

60.20 More optionality can be too expensive

Return to the exact witness.

Replace the preservation cost $1/2$ with a parameter c .

Then:

$$Q(P)=1-c.$$

Commit-left retains:

$$Q(C_L)=1/2.$$

At:

$$c=3/4,$$

preservation has expected return:

$$1/4.$$

Commit-left has:

$$1/2.$$

Preservation still has:

- two functional options;

- correction capacity 1.

Commit-left still has:

- one functional option;
- correction capacity $1/2$.

Yet commitment now wins on expected return.

This is not a paradox.

Maintaining flexibility has a price.

Optionality becomes a rational control objective only after that price and the value of future correction are part of the contract.

60.21 More labels are not more options

A second counterexample is even simpler.

Suppose one state offers action x .

Another offers x_1 and x_2 .

If both labels have identical transitions, costs, and future consequences, the second state has twice the raw action count but exactly the same functional option set.

This is why the quotient by consequence signature is load-bearing.

Without it, optionality can be manufactured by renaming buttons.

60.22 Regret and optionality answer different questions

Regret asks:

how much reward was lost relative to this comparator?

Optionality asks:

after this action, which materially distinct viable continuations remain, and which future corrections can still be executed?

The exact witness shows that Bayesian regret can tie while correction capacity differs.

But the objects can also align.

In the witness, worst-case regret prefers preservation.

In other problems, a comparator can explicitly reward recoverability, making regret sensitive to option destruction.

That does not make optionality redundant.

It means the comparator has been changed to include it.

The Regret chapter's discipline remains intact:

regret is always relative to the declared comparator and reward contract.

60.23 Optionality can be an objective, a constraint, or a diagnostic

There is no requirement to optimize CC directly.

Three common uses are structurally distinct.

Objective:

maximize a scalarization such as expected reward plus a declared optionality term.

Constraint:

maximize reward subject to:

$CC_{\{h, \epsilon\}} \geq \alpha$.

Diagnostic:

optimize another objective, but report correction capacity to expose hidden commitment.

The third use is often valuable because it reveals opportunity destruction without pretending that all opportunity should be preserved.

60.24 Hard and soft correction failure should not be merged

Suppose two states both retain the desired correction.

From state A, correction costs 1.

From state B, correction costs 100.

Both have nonzero correction capacity at tolerance 100.

Only A has nonzero correction capacity at tolerance 1.

Now compare state C, where correction is impossible.

Its correction cost is infinity.

This is why the tolerance index is useful.

It distinguishes:

- cheap correction;
- expensive correction;
- no correction.

A single Boolean “reversible” flag often hides too much.

60.25 **Optionality does not imply safety**

A state can expose many future actions, including dangerous ones.

A high correction-capacity score relative to one target family says nothing automatically about:

- safety constraints omitted from K;
- tail risk;
- fairness;
- robustness to model error;
- calibration;
- institutional authorization.

These require separate contracts or connecting theorems.

The chapter therefore resists the phrase:

more optionality is safer.

Sometimes it is.

Sometimes it is not.

60.26 **Optionality does not imply knowledge**

A system can preserve many corrections while having no idea which one is needed.

That is the mirror image of the main witness.

The witness gives perfect knowledge but limited correction in one branch.

The opposite case gives broad feasible control but weak information.

Adaptive capability needs both when the task demands both.

60.27 **A useful evaluation vector**

For a policy π_i , one may report:

`E(pi) = ( expected return, Bayesian regret, worst-case regret, functional option count, correction capacity, correction-cost profile )`.

This is a vector, not a theorem that one coordinate should dominate.

Scalarization or Pareto preference must be declared.

This is often the cleanest way to preserve the semantics of optionality while still permitting ordinary decision analysis.

60.28 What the exact witness proves

The finite witness proves exactly:

- prior expected return can tie;
- Bayesian regret can tie;
- information gain can tie;
- functional option count can differ;
- correction capacity can differ;
- preservation cost can make the higher-optionality policy lower value.

It does not prove that correction capacity is unique.

It does not establish an optimal weighting between reward and flexibility.

It does not promote optionality into a universal safety principle.

Its role is to expose a missing state variable in some adaptive decisions.

60.29 Downstream handoff

Governed Adaptation — **ATLAS-CH-GOVADAPT-001** may, after this chapter’s audit, assume:

- viable continuation sets $V_h(s)$;
- functional option quotients $O_h(s)$;
- horizon-relative recoverability;
- extended-real correction cost $k_h(s, \theta)$;
- conditional correction indicator $C_{\{h, \epsilon\}}(s, \theta)$;
- ex-ante tolerance-indexed correction capacity $CC_{\{h, \epsilon\}}(s; b)$;
- the distinction between information acquisition and ability to exploit information;
- the exact equal-value/equal-Bayes-regret separation witness;
- the counterexample showing that preserving options can be too costly.

The downstream chapter must keep the target family, horizon, constraints, consequence equivalence, and uncertainty semantics explicit.

60.30 References used in this chapter

- Arrow and Fisher, *Environmental Preservation, Uncertainty, and Irreversibility* (Arrow and Fisher 1974).
- Aubin, *Viability Theory* (Aubin 1991).
- Klyubin, Polani, and Nehaniv, *Keep Your Options Open: An Information-Based Driving Principle for Sensorimotor Systems* (Klyubin et al. 2008).

Exact source identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-OPTIONALITY-001.yaml`

Chapter 61

Joint Uncertainty Propagation

Epistemic status: audited RL/control and information-theory substrate + Atlas-owned exact covariance propagation and finite witness.

Specification: [manuscript/specifications/ATLAS-CH-JOINTUNC-001.md](#)

Derivation packet: [mathematics/derivations/ATLAS-CH-JOINTUNC-001-DERIVATIONS.md](#)

Computational witness: [mathematics/computational-witnesses/ATLAS-CW-JOINTUNC-001.md](#)

Source lock: [sources/source-locks/ATLAS-CH-JOINTUNC-001.yaml](#)

A sequential decision can be uncertain in several places at once.

The reward can be uncertain.

The transition can be uncertain.

The observation can be uncertain.

The continuation value can be uncertain.

If those uncertainties move together, adding four separate error bars as if they were independent can be wrong in either direction.

The governing principle is:

joint uncertainty requires the joint law, not only the marginals.

This chapter formalizes that principle in the smallest exact setting that exposes the issue.

61.1 The controlled-system handoff

Reinforcement Learning and Control separated:

- state;
- action;
- transition kernel;
- reward;
- policy;
- return;
- value;
- learned model;
- observation structure.

It also stated a boundary:

transition, reward, observation, and future-value uncertainty can interact.

RLBASE did not solve that interaction.

JOINTUNC starts there.

61.2 Information-theoretic discipline

Probability, Information, and Statistical Structure supplied another discipline: uncertainty quantities are defined only relative to a declared probability model.

Dependence is also model-relative.

A covariance or mutual information does not become causal direction merely because it is nonzero.

JOINTUNC inherits that firewall.

61.3 Four local coordinates

Write a declared local error vector:

$$\varepsilon = \begin{pmatrix} \varepsilon_R \\ \varepsilon_T \\ \varepsilon_O \\ \varepsilon_V \end{pmatrix}.$$

Interpret the coordinates as local scalarized uncertainty in:

- reward;
- transition effect;
- observation or belief state;

- future value.

These are not universal latent variables.

They are coordinates chosen for a declared local decision calculation.

61.4 Marginals are not enough

Suppose we know:

$$\text{Var}(\varepsilon_R), \quad \text{Var}(\varepsilon_T), \quad \text{Var}(\varepsilon_O), \quad \text{Var}(\varepsilon_V).$$

That is not yet a full joint uncertainty description.

We also need the cross-covariances collected in:

$$\Sigma = \text{Cov}(\varepsilon).$$

The diagonal records marginal variances.

The off-diagonal entries record pairwise covariance.

61.5 A declared local decision-error functional

Use the exact affine witness:

$$\delta = \varepsilon_R + \varepsilon_T + \varepsilon_O + \frac{1}{2}\varepsilon_V.$$

The sensitivity vector is:

$$a = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1/2 \end{pmatrix}.$$

The factor 1/2 plays a discount-like role for the future-value coordinate.

The first three coefficients are normalized to one.

This is a toy local functional, not a universal Bellman uncertainty equation.

61.6 Exact affine propagation

For any finite-second-moment error vector:

$$\boxed{\text{Var}(a^\top \varepsilon) = a^\top \Sigma a.}$$

Expanded:

$$\text{Var}(a^\top \varepsilon) = \sum_i a_i^2 \text{Var}(\varepsilon_i) + 2 \sum_{i < j} a_i a_j \text{Cov}(\varepsilon_i, \varepsilon_j).$$

The covariance terms are not optional decoration.

They are part of the exact variance.

61.7 The naive diagonal calculation

Give every coordinate unit marginal variance:

$$\text{Var}(\varepsilon_i) = 1.$$

Then the diagonal-only sum is:

$$1 + 1 + 1 + \frac{1}{4} = \boxed{\frac{13}{4}}.$$

If the weighted covariance terms vanish, this is exact.

If they do not, it is not.

61.8 Positive common-shock dependence

Let U, W be independent centered Rademacher variables.

Set:

$$\varepsilon_R = U,$$

$$\varepsilon_T = U,$$

$$\varepsilon_O = U,$$

$$\varepsilon_V = W.$$

The first three uncertainties share one common shock.

Their covariance matrix is:

$$\Sigma_+ = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

61.9 Positive dependence amplifies uncertainty

Now:

$$\delta_+ = 3U + \frac{1}{2}W.$$

Since U and W are independent:

$$\text{Var}(\delta_+) = 9 + \frac{1}{4} = \boxed{\frac{37}{4}}.$$

The diagonal-only value was:

$$\frac{13}{4}.$$

Difference:

$$\boxed{6}.$$

Ignoring positive covariance has substantially understated the propagated variance.

61.10 Cancellation dependence

Now keep the same marginal variances but reverse the observation coordinate:

$$\varepsilon_R = U,$$

$$\varepsilon_T = U,$$

$$\varepsilon_O = -U,$$

$$\varepsilon_V = W.$$

Then:

$$\Sigma_- = \begin{pmatrix} 1 & 1 & -1 & 0 \\ 1 & 1 & -1 & 0 \\ -1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

61.11 Dependence can also cancel

The propagated error is now:

$$\delta_- = U + \frac{1}{2}W.$$

Therefore:

$$\text{Var}(\delta_-) = 1 + \frac{1}{4} = \boxed{\frac{5}{4}}.$$

The diagonal-only value was still:

$$\frac{13}{4}.$$

Difference:

$$\boxed{-2}.$$

Ignoring covariance has now overstated the propagated variance.

61.12 Independence control

Let:

$$U_R, U_T, U_O, U_V$$

be mutually independent centered Rademacher variables.

Set each uncertainty coordinate equal to its own independent variable.

Then:

$$\Sigma_0 = I_4.$$

Therefore:

$$\text{Var}(\delta_0) = 1 + 1 + 1 + \frac{1}{4} = \boxed{\frac{13}{4}}.$$

The diagonal calculation is exact.

61.13 One set of marginals, three different results

All three cases have the same marginal variance vector:

$$(1, 1, 1, 1).$$

Their propagated variances are:

$$\boxed{\frac{37}{4}, \quad \frac{5}{4}, \quad \frac{13}{4}}.$$

Therefore:

$$\boxed{\text{same marginal variances} \not\equiv \text{same propagated uncertainty.}}$$

61.14 The difference is dependence structure

The three experiments did not change the marginal error sizes.

They changed only how the error coordinates move together.

That is the central point.

If the uncertainty object is joint, its propagation should be joint.

61.15 Independence is stronger than required

The covariance correction vanishes when the relevant weighted cross-covariances vanish.

Full independence guarantees that.

But full independence is not necessary.

Thus:

$$\boxed{\text{uncorrelated} \not\Rightarrow \text{independent.}}$$

The variance calculation needs second moments.

A claim of independence needs more.

61.16 Propagation depends on the decision functional

Suppose we keep the same covariance matrix but change the decision quantity.

Use sensitivity vector b instead of a .

Then:

$$\text{Var}(b^\top \varepsilon) = b^\top \Sigma b.$$

Different decision functionals can therefore assign different importance to the same covariance structure.

There is no universal propagated-uncertainty scalar independent of the quantity being propagated.

61.17 Reward uncertainty remains a distinct object

Reward uncertainty can mean several things:

- random reward realization;
- measurement noise;
- learned reward-model error;
- reward misspecification.

Those are not identical.

The finite witness deliberately compresses them into one local scalar only after the modeling choice is declared.

61.18 Transition uncertainty remains distinct too

Transition uncertainty can include:

- inherent next-state randomness;
- learned transition-model uncertainty;
- local approximation error;
- uncertainty in a transition-sensitive summary.

The chapter does not silently equate those notions.

61.19 Observation uncertainty is not transition uncertainty

In a partially observed system, latent-state dynamics and observation generation are separate stochastic mechanisms.

An observation can be noisy even when the latent transition is known.

A transition can be uncertain even when observation is perfect.

They can also be dependent.

This is why the observation coordinate remains explicit.

61.20 Future-value uncertainty is downstream but not redundant

Future-value uncertainty can reflect:

- uncertain next state;
- value-function estimation error;
- function approximation;
- finite data;
- model error.

It is not automatically absorbed by the transition coordinate.

A declared calculation must say which uncertainty is represented where.

61.21 Expected value is not uncertainty

RLBASE established value as expected future return under declared conditions.

That expectation is not an uncertainty estimate.

Two actions can have equal expected value and different propagated uncertainty.

Two actions can also have equal propagated variance and different expected value.

Therefore:

$$\boxed{\text{expected value} \neq \text{uncertainty.}}$$

61.22 A risk rule must be declared

A decision system may intentionally combine expectation and uncertainty.

For example, it may penalize variance.

That is a new decision rule.

It should not be smuggled into the meaning of value.

The utility or risk criterion must be declared explicitly.

61.23 Variance is useful but incomplete

Variance is a second-moment summary.

Two distributions can share mean and variance while differing in:

- tail probability;
- skewness;
- support;
- multimodality;
- catastrophic-event probability.

Therefore:

$$\boxed{\text{same variance} \not\Rightarrow \text{same risk distribution.}}$$

61.24 Nonlinear decision maps

Real decision functionals are often nonlinear.

Let:

$$F(x)$$

be differentiable at a nominal state x .

For perturbation ε :

$$F(x + \varepsilon) - F(x) = \nabla F(x)^\top \varepsilon + R.$$

The remainder R matters.

61.25 The exact remainder identity

The variance is exactly:

$$\text{Var}(F(x + \varepsilon) - F(x)) = \nabla F(x)^\top \Sigma \nabla F(x) + \text{Var}(R) + 2 \text{Cov}(\nabla F(x)^\top \varepsilon, R).$$

Only the first term is the familiar linearized propagation term.

61.26 First-order propagation is local

If the remainder contributions are controlled, then:

$$\text{Var}(F(x + \varepsilon) - F(x)) \approx \nabla F(x)^\top \Sigma \nabla F(x).$$

This is a local approximation.

It is not a license to propagate one covariance matrix globally through an arbitrary nonlinear agent.

61.27 When the local approximation can fail

The first-order picture can fail when:

- perturbations are not small;
- curvature is large;
- thresholds or discontinuities matter;
- policies switch;
- state visitation changes;
- tail events dominate;
- the covariance itself changes across the perturbed region.

A global theorem needs additional arguments.

61.28 Sequential propagation is harder than one step

For:

$$x_{t+1} = F_t(x_t, \eta_t),$$

later uncertainty depends on:

- local sensitivities;
- cross-time dependence;
- state-dependent noise;
- policy feedback;
- changing distributions;
- changing approximators.

The exact affine witness is a building block.

It does not solve the full recursion.

61.29 Why naive independent summation is dangerous

The positive common-shock witness gave:

$$\frac{37}{4}$$

while naive summation gave:

$$\frac{13}{4}.$$

The cancellation witness gave:

$$\frac{5}{4}$$

while naive summation again gave:

$$\frac{13}{4}.$$

So the naive rule is not merely slightly noisy.

It can fail in either direction.

61.30 Covariance is not causality

Suppose reward and transition errors have positive covariance.

That says they co-vary under the declared joint law.

It does not tell us whether:

- reward error causes transition error;
- transition error causes reward error;
- a third variable drives both;
- the dependence is induced by conditioning or selection.

Therefore:

statistical dependence $\nRightarrow$ causal direction.

61.31 Conditional structure matters

The relevant covariance can change after conditioning on:

- current state;
- action;
- observation;
- belief state;
- model version;
- policy.

A joint uncertainty analysis should say what is conditioned on.

One unconditional covariance matrix may be inappropriate for every local decision context.

61.32 Functional-relative uncertainty is an interface concept

Suppose a downstream system asks:

how uncertain is this action-value comparison?

The upstream uncertainty representation should expose enough joint structure to answer that question.

A list of independent scalar error bars may not be a sufficient interface.

This is a systems consequence of the mathematics.

61.33 What JOINTUNC does not claim

It does not claim that:

variance = complete risk.

It does not claim that:

covariance = cause.

It does not claim that:

local linearization = global sequential theorem.

It does not claim that the four witness coordinates are canonical.

It does not claim real RL errors have the toy joint laws.

61.34 Non-implications

The chapter rejects:

same marginal variances $\nRightarrow$ same propagated uncertainty,

zero covariance $\nRightarrow$ independence,

statistical dependence $\nRightarrow$ causal direction,

local first-order variance $\nRightarrow$ global sequential uncertainty,

same variance $\nRightarrow$ same risk distribution,

and:

high uncertainty $\nRightarrow$ low expected value.

61.35 Practical protocol

For a real decision system:

1. define the decision functional;
2. identify distinct uncertainty sources;
3. define the conditioning context;
4. estimate or bound the joint covariance/dependence structure;
5. propagate through the declared sensitivity map;
6. test whether a local linear approximation is adequate;
7. expose remainder or nonlinear risk when it is not;
8. report expectation separately from uncertainty;
9. retain dependence/causality separation;
10. preserve the full distribution when second moments are insufficient.

61.36 Atlas connections

Reinforcement Learning and Control.

JOINTUNC extends the controlled stochastic substrate without changing Bellman semantics.

Probability and Information.

Joint distributions and statistical dependence are the mathematical foundation of the uncertainty object.

Uncertainty and Calibration.

Marginal predictive uncertainty can feed JOINTUNC, but calibration of individual predictors does not by itself specify cross-source dependence.

Distribution Shift.

A covariance structure learned in one regime may change under shift.

Agents and Systems.

Joint uncertainty becomes an interface requirement when one subsystem passes uncertainty to another.

61.37 Closing view

Uncertainty does not propagate source by source unless the joint structure permits it.

The exact witness makes that unavoidable.

With the same four marginal variances:

$$(1, 1, 1, 1),$$

we obtained:

$$\frac{37}{4}, \quad \frac{5}{4}, \quad \frac{13}{4}.$$

Nothing changed except dependence.

The correct conclusion is:

propagate the joint uncertainty of the declared decision object, not an assumed independent sum of ma

And when the decision map is nonlinear, keep the local approximation boundary visible.

61.38 References used in this chapter

No new external academic authority is added.

The controlled-decision substrate is inherited through audited ATLAS-CH-RLBASE-001.

The probability/information substrate is inherited through audited ATLAS-CH-INFO-001.

Exact prerequisite identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-JOINTUNC-001.yaml`

Chapter 62

Uncertainty and Calibration

Epistemic status: established probability/statistics + source-scoped uncertainty methods + Atlas synthesis.

Specification: manuscript/specifications/ATLAS-CH-UNCERTAINTY-001.md
Derivation packet: mathematics/derivations/ATLAS-CH-UNCERTAINTY-001-DERIVATIONS.md
Computational witness: mathematics/computational-witnesses/ATLAS-CW-UNCERTAINTY-001.md
Source lock: sources/source-locks/ATLAS-CH-UNCERTAINTY-001.yaml

A model can be wrong in more than one way.

It can be:

- uncertain because the outcome itself is noisy;
- uncertain because several models or parameter settings remain plausible;
- confident but systematically miscalibrated;
- calibrated but uninformative;
- uncertain in a Bayesian approximation;
- covered by a conformal set without assigning calibrated probabilities;
- accurate on accepted examples only because it abstains on the rest.

These are not synonyms.

This chapter develops a vocabulary for keeping them separate.

The governing principle is:

A useful uncertainty statement must name the random object, the conditioning information, the estimator or diagnostic, and the guarantee.

62.1 Probability is not yet calibration

Suppose a classifier emits a score

$$S \in [0, 1]$$

for a binary outcome

$$Y \in \{0, 1\}.$$

A probability-like number can be read as a claim.

If the model says

$$S = 0.8,$$

the calibration question is not whether the model is correct on this one example.

It is whether examples receiving score 0.8 are positive about 80 percent of the time under the relevant distribution.

Population calibration can be written as

$$E[Y \mid S] = S$$

almost surely.

At an atomic score value,

$$P(Y = 1 \mid S = s) = s.$$

This is a property of the joint law of predictions and outcomes.

It is not a property of the numerical range $[0, 1]$ alone.

62.2 Confidence is not calibration

A model can emit scores close to one and still be badly calibrated.

Likewise, a model can be calibrated while rarely producing extreme probabilities.

Guo et al. study confidence calibration in modern neural networks and evaluate post-processing methods including temperature scaling (Guo et al. 2017).

The source-scoped lesson is not that temperature scaling universally solves uncertainty.

It is that probability calibration is measurable, can fail in trained neural networks, and can be altered by a post-hoc map without changing the predicted class ordering.

62.3 Calibration is not accuracy

Accuracy asks how often a decision rule is correct.

Calibration asks whether stated probabilities match conditional frequencies.

A classifier can improve one without necessarily improving the other.

Temperature scaling is a useful example: changing logits by a positive scalar temperature does not change the argmax class, yet it can change probability calibration.

This is why uncertainty work should keep:

- decisions;
- scores;
- probability claims;
- losses

as separate objects.

62.4 Calibration is not sharpness

Consider a population with

$$P(X = a) = P(X = b) = 1/2$$

and

$$P(Y = 1 \mid X = a) = 0.9, \quad P(Y = 1 \mid X = b) = 0.1.$$

The marginal positive rate is 0.5.

A constant predictor

$$S_0 = 0.5$$

is calibrated.

So is the predictor

$$S_1(a) = 0.9, \quad S_1(b) = 0.1.$$

But they do not contain the same information.

For the constant predictor,

$$E[(Y - S_0)^2] = 0.25.$$

For the sharper conditional predictor,

$$E[(Y - S_1)^2] = 0.09.$$

Both are calibrated.

Only one distinguishes the two covariate regimes.

Therefore:

calibration $\not\Rightarrow$ sharpness or informativeness.

62.5 Proper scoring and calibration are related but distinct

Log loss and Brier score reward probabilistic predictions using the realized outcome.

They are not merely calibration errors.

A predictor can have good calibration and poor resolution.

A predictor can have better proper-score risk while both predictors are calibrated.

The previous exact witness demonstrates this with Brier risk.

The Atlas therefore uses proper scores, calibration diagnostics, and discrimination metrics as distinct evaluation axes.

62.6 Empirical calibration is an estimate

Population calibration is a property of a distribution.

A reliability diagram is a finite-sample diagnostic.

Expected calibration error is a finite-sample, bin-dependent summary.

A common form is

$$ECE = \sum_j \frac{|B_j|}{n} |acc(B_j) - conf(B_j)|.$$

Change the bins and the numerical ECE can change.

Therefore a small empirical ECE is not itself a theorem that

$$E[Y \mid S] = S$$

in the deployment population.

62.7 One word, several uncertainty objects

Suppose a model defines a predictive law

$$p(y \mid x, D).$$

Possible uncertainty summaries include:

- predictive variance;
- predictive entropy;
- posterior variance over parameters;
- disagreement among models;
- mutual information between predictions and latent model variables;
- width of a prediction interval;
- conformal set size;
- probability of abstention;
- selective risk.

These quantities answer different questions.

A chapter that simply plots an “uncertainty score” without defining the underlying object has not yet specified the claim.

62.8 Predictive entropy

For a discrete predictive law,

$$H(Y \mid x, D) = - \sum_y p(y \mid x, D) \log p(y \mid x, D).$$

Entropy summarizes uncertainty in the predictive distribution.

It does not say where that uncertainty came from.

This matters because the same predictive distribution can arise from very different latent structures.

62.9 Aleatoric and epistemic uncertainty

Kendall and Gal use a two-way distinction in Bayesian deep-learning models for computer vision (Kendall and Gal 2017):

- aleatoric uncertainty: uncertainty associated with noise inherent in observations or the conditional data model;
- epistemic uncertainty: uncertainty associated with the model and its parameters.

This distinction is useful.

It is not an exhaustive ontology of every uncertainty that can arise in an adaptive system.

Numerical approximation, distribution shift, misspecification, measurement failure, hidden interventions, and unknown unknowns can require other descriptions.

The Atlas therefore treats aleatoric/epistemic language as model-relative.

62.10 Entropy decomposition

Let Θ denote a latent model variable.

Then

$$I(Y; \Theta) = H(Y) - H(Y | \Theta).$$

Equivalently,

$$H(Y) = E_{\Theta}[H(Y | \Theta)] + I(Y; \Theta).$$

In a declared Bayesian predictive model, this can motivate a decomposition into:

- expected conditional uncertainty;
- model-dependent information gain or epistemic component.

The identity is exact.

The interpretation depends on what Θ means.

62.11 Same entropy, opposite decomposition

Consider two models.

62.11.1 Model A

Theta is fixed.

$$Y \sim \text{Bernoulli}(1/2).$$

Then

$$H(Y) = 1 \text{ bit},$$

$$E[H(Y | \Theta)] = 1,$$

$$I(Y; \Theta) = 0.$$

62.11.2 Model B

$$\Theta \sim \text{Bernoulli}(1/2),$$

and

$$Y = \Theta.$$

Marginally,

$$Y \sim \text{Bernoulli}(1/2).$$

So again,

$$H(Y) = 1 \text{ bit}.$$

But now conditioning on Theta makes Y deterministic:

$$E[H(Y | \Theta)] = 0,$$

$$I(Y; \Theta) = 1 \text{ bit}.$$

The predictive distribution is identical.

The decomposition is opposite.

Therefore:

predictive entropy alone $\nrightarrow$ epistemic uncertainty.

62.12 Reducibility is relative to the information set

The phrase “epistemic uncertainty can be reduced with more data” requires a model and a data-acquisition regime.

More observations of the same kind may reduce parameter uncertainty.

They may fail to resolve:

- structural misspecification;
- unobserved confounding;
- systematic measurement bias;
- an unidentifiable parameterization;
- a changed environment.

So reducibility is not an intrinsic label attached to a scalar variance.

It is relative to the model, observations, and interventions that are possible.

62.13 Bayesian uncertainty

A Bayesian model places a distribution over uncertain quantities and updates it using data.

For parameters Θ ,

$$p(\Theta \mid D) \propto p(D \mid \Theta)p(\Theta).$$

The posterior predictive is

$$p(y \mid x, D) = \int p(y \mid x, \Theta)p(\Theta \mid D)d\Theta.$$

This is a precise probabilistic object.

A practical neural-network approximation may only approximate it.

The distinction between target and approximation should remain visible.

62.14 Dropout as an approximate Bayesian construction

Gal and Ghahramani develop a theoretical framework relating dropout training in neural networks to approximate Bayesian inference in deep Gaussian-process models (Gal and Ghahramani 2016).

Within that framework, repeated stochastic forward passes can be used to estimate predictive quantities.

The Atlas preserves two boundaries.

First:

MC dropout $\neq$ exact posterior sampling.

Second:

dropout used in a network $\nrightarrow$ all Bayesian guarantees.

The interpretation comes from the cited approximation framework, not from randomness alone.

62.15 Deep ensembles

Lakshminarayanan, Pritzel, and Blundell study independently trained neural networks as a simple and scalable method for predictive uncertainty estimation (Lakshminarayanan et al. 2017).

Given predictive distributions

$$p_m(y \mid x), \quad m = 1, \dots, M,$$

an ensemble mixture is

$$\bar{p}(y \mid x) = \frac{1}{M} \sum_{m=1}^M p_m(y \mid x).$$

This is an exact mixture identity.

It does not imply that the members are samples from

$$p(\Theta \mid D).$$

Ensemble disagreement can be useful empirically without becoming posterior variance by definition.

62.16 Ensemble spread and epistemic uncertainty

It is tempting to call all disagreement epistemic uncertainty.

That is too fast.

Ensemble spread can depend on:

- initialization;
- optimization path;
- training-data resampling;
- architecture diversity;
- explicit priors;
- regularization;
- stochastic training;
- mode coverage.

The generated ensemble determines what its spread means.

The Atlas therefore says:

ensemble disagreement is an empirical uncertainty diagnostic whose epistemic interpretation must be justified by the ensemble construction.

62.17 Prediction sets are another object

Sometimes the output is not a probability.

It is a set

$$C(x)$$

constructed to contain the future response with a target frequency.

Conformal prediction provides a framework for this kind of set-valued prediction (Vovk et al. 2005; Lei et al. 2018).

Its signature guarantee is coverage under stated exchangeability/randomness assumptions.

That is different from probability calibration.

62.18 Split conformal rank logic

Suppose we have

$$n$$

calibration nonconformity scores and one future score.

For target miscoverage $0 < \alpha < 1$, the finite-sample rank correction uses

$$k = \lceil (n+1)(1-\alpha) \rceil.$$

The threshold is the k -th smallest calibration score **when** $1 \leq k \leq n$. If $k = n+1$, there is no such order statistic in the n calibration scores; the standard nonrandomized convention sets the threshold to $+\infty$ (and hence uses an unrestricted prediction set for that score rule). This occurs when $\alpha < 1/(n+1)$. For example, with $n = 4$ and $\alpha = 0.1$, one obtains $k = 5$, not a fifth calibration observation.

Under exchangeability, the future score has a symmetric rank among the

$$n+1$$

scores.

This rank symmetry is the core of the finite-sample marginal-coverage logic.

62.19 Exact conformal rank witness

Take

$$n = 4, \quad \alpha = 0.2.$$

Then

$$k = \lceil 5(0.8) \rceil = 4.$$

Assume no ties.

The future score's rank is uniform over

$$1, 2, 3, 4, 5.$$

The future point is covered unless its score is the unique largest score.

Hence

$$P(Y_{new} \in C(X_{new})) = \frac{4}{5} = 0.8.$$

This is an exact finite rank argument.

It is marginal.

It is not arbitrary conditional coverage.

62.20 Coverage is not calibration

Calibration concerns statements such as

$$P(Y = 1 \mid S = s) = s.$$

Conformal coverage concerns

$$P(Y \in C(X)) \geq 1 - \alpha$$

under the construction’s assumptions.

A set can have the target marginal coverage without supplying calibrated class probabilities.

A calibrated probability model does not automatically produce a conformal finite-sample coverage theorem.

These are distinct guarantees.

62.21 Distribution-free does not mean assumption-free

The phrase “distribution-free” can be misleading when detached from the sampling assumptions.

Conformal validity does not require a parametric family for the outcome distribution.

It does require the symmetry/exchangeability conditions used by the theorem.

A deployment process with:

- temporal drift;
- feedback;
- covariate shift;
- intervention;
- adaptive selection

can violate the assumptions that justified the original coverage statement.

This boundary passes directly to the downstream Shift chapter.

62.22 Conditional coverage is stronger

Marginal coverage averages over the input distribution.

A stronger statement would require coverage within every relevant subgroup or condition.

Generic finite-sample conformal validity does not automatically give arbitrary exact conditional coverage.

The Atlas will not rewrite a marginal theorem as a subgroup theorem.

62.23 Abstention is another control surface

A model need not predict on every example.

Let

$$A(X) \in \{0, 1\}$$

indicate acceptance.

Coverage is

$$c = P(A(X) = 1).$$

For loss L , selective risk is

$$R_{sel} = E[L \mid A(X) = 1].$$

Selective classification studies this risk-coverage tradeoff; Geifman and El-Yaniv develop a deep-network construction in this setting (Geifman and El-Yaniv 2017).

62.24 Exact risk-coverage witness

Suppose five equally weighted examples have error indicators

$$(0, 0, 0, 1, 1).$$

If we accept all five,

$$c = 1,$$

and

$$R_{sel} = \frac{2}{5}.$$

If we accept only the first three,

$$c = \frac{3}{5},$$

and

$$R_{sel} = 0.$$

The accepted predictions are perfect.

But forty percent of the population has been rejected.

The improvement is real and conditional on lower coverage.

62.25 Abstention is not calibration

A selective classifier can reject difficult cases using:

- confidence;
- margin;
- entropy;
- an auxiliary selector;
- a calibrated risk predictor;
- another uncertainty score.

Whatever the mechanism, low selective risk does not prove the underlying probabilities are calibrated.

Conversely, a calibrated predictor need not know which examples to reject to achieve a desired risk-coverage curve.

The objects remain separate.

62.26 Uncertainty and decisions

Uncertainty matters because decisions have asymmetric costs.

A probability estimate may feed:

- abstention;
- escalation to a human;
- additional sensing;
- active learning;
- a larger conformal set;

- a safer controller;
- deferred execution.

But the correct action depends on a loss or utility model.

High entropy does not mathematically imply “abstain.”

That policy requires a decision rule.

62.27 Calibration can be local or global in different senses

A single scalar calibration metric can hide structure.

A model may appear calibrated overall while failing on:

- a demographic subgroup;
- a rare class;
- a region of feature space;
- long-context examples;
- high-loss examples;
- a shifted domain.

The fix is not to call calibration useless.

It is to state the conditioning structure being tested.

62.28 The model can be calibrated for the wrong world

Suppose calibration is established under distribution P .

Deployment occurs under distribution Q .

Even if the scoring function is unchanged,

$$P(Y = 1 \mid S = s)$$

and

$$Q(Y = 1 \mid S = s)$$

need not agree.

Therefore:

$P\text{-calibration} \not\Rightarrow Q\text{-calibration.}$
--

The same caution applies to:

- ensemble diagnostics;
- conformal coverage;
- selective-risk estimates;
- abstention thresholds.

62.29 The uncertainty interface

A downstream component should be able to ask:

1. What is random?
2. What is conditioned on?
3. Is the output a probability, variance, entropy, set, interval, or decision?
4. Is the quantity predictive or parameter/model-level?
5. Is the method Bayesian, approximately Bayesian, ensemble-based, conformal, or purely empirical?
6. What distributional assumptions support the guarantee?
7. Is the guarantee marginal or conditional?
8. Is abstention changing the evaluated population?
9. Was calibration measured or proved?
10. Does deployment match the law under which the statement was established?

This is the chapter’s practical contract.

62.30 Handoff to Distribution Shift and Robustness

The direct consumer is

ATLAS-CH-SHIFT-001.

SHIFT may inherit:

- population calibration definitions;
- the calibration-versus-sharpness witness;
- the predictive entropy decomposition;
- the same-entropy/opposite-decomposition counterexample;
- source-scoped MC-dropout and ensemble semantics;
- conformal rank coverage and its exchangeability boundary;
- selective coverage and selective risk.

SHIFT must independently establish what survives when the data law changes.

In particular it must not assume:

i.i.d. calibration $\Rightarrow$ shifted calibration,

or

exchangeable conformal coverage $\Rightarrow$ coverage under arbitrary drift.

62.31 Failure modes

The most common category errors are:

- calling confidence calibration “uncertainty estimation” without saying what is uncertain;
- calling predictive entropy epistemic uncertainty;
- treating ensemble members as posterior samples by definition;
- treating stochastic dropout passes as exact Bayesian posterior samples;
- treating a conformal set as a Bayesian credible region;
- treating marginal coverage as arbitrary conditional coverage;
- treating low ECE as population calibration;
- treating low selective risk as unchanged-population accuracy;
- treating abstention as calibration;
- transporting an in-distribution guarantee into a shifted environment without revalidation.

62.32 Closing view

Uncertainty is not one number.

The Atlas uses a family of objects:

probability calibration,
 predictive entropy,
 model/posterior uncertainty,
 aleatoric uncertainty,
 ensemble disagreement,
 conformal coverage,
 selective risk and coverage.

Each object comes with its own assumptions and its own question.

The durable rule is:

Before using an uncertainty estimate, name what it estimates and name the guarantee it actually carries.

62.33 References used in this chapter

- (Guo et al. 2017)

- (Gal and Ghahramani 2016)
- (Kendall and Gal 2017)
- (Lakshminarayanan et al. 2017)
- (Vovk et al. 2005)
- (Lei et al. 2018)
- (Geifman and El-Yaniv 2017)

Exact provenance and claim boundaries are locked in:

`sources/source-locks/ATLAS-CH-UNCERTAINTY-001.yaml`

Chapter 63

Distribution Shift and Robustness

Epistemic status: audited Uncertainty substrate + primary covariate-shift and adversarial-robust-optimization sources + Atlas-owned exact finite witnesses.

Specification: manuscript/specifications/ATLAS-CH-SHIFT-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-SHIFT-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-SHIFT-001.md

Source lock: sources/source-locks/ATLAS-CH-SHIFT-001.yaml

A model can remain numerically unchanged while the world in which it is evaluated changes.

That forces a discipline:

name the source law and deployment law before claiming robustness under shift.

63.1 Source law is not deployment law

Let P be the law under which a guarantee, diagnostic, or empirical estimate was established. Let Q be the deployment law. For predictor f and loss L ,

$$R_P(f) = \mathbb{E}_P[L(f(X), Y)],$$

$$R_Q(f) = \mathbb{E}_Q[L(f(X), Y)].$$

These are different mathematical objects unless $P=Q$ or a bridge is proved.

The Uncertainty chapter established that calibration, conformal coverage, selective risk, predictive entropy, and other uncertainty objects are assumption-scoped. SHIFT adds the changed-law layer.

63.2 Distribution shift is not one phenomenon

63.2.1 Covariate shift

$$P_X \neq Q_X, \quad P(Y | X) = Q(Y | X).$$

Sugiyama, Krauledat, and Müller study this setting and importance-weighted correction/model-selection methods (Sugiyama et al. 2007).

63.2.2 Label/prior shift

The class prior changes while an appropriate class-conditional mechanism is held fixed.

63.2.3 Concept or conditional shift

$$P(Y | X) \neq Q(Y | X).$$

Changing only X-weights cannot generally repair this.

63.2.4 Structural shift

A mechanism, topology, measurement process, routing rule, tokenizer, feature generator, or other declared system component changes. Structural shift is not identified merely because output statistics changed.

63.3 Covariate shift permits a change of measure

If

$$P(Y | X) = Q(Y | X)$$

and

$$Q_X \ll P_X,$$

define

$$w(x) = \frac{dQ_X}{dP_X}(x).$$

Then

$$R_Q(f) = \mathbb{E}_P[w(X)L(f(X), Y)].$$

This is not extrapolation. It requires the relevant target support to lie within source support.

63.4 Target-only support is a real boundary

If some region has $P_X(x)=0$ and $Q_X(x)>0$, the ordinary importance ratio is unavailable there. No finite reweighting of observed source examples creates evidence about a target-only region. Extra modeling assumptions may permit extrapolation, but they must be declared.

63.5 Exact witness: calibration can fail under pure covariate shift

Let X have two states a and b . Outcomes are deterministic:

$$Y(a) = 1, \quad Y(b) = 0.$$

Under the source law,

$$P_X(a) = P_X(b) = 1/2.$$

Under deployment,

$$Q_X(a) = 3/4, \quad Q_X(b) = 1/4.$$

Nothing about $Y|X$ changed.

Use the same predictor before and after deployment:

$$s(a) = s(b) = 1/2.$$

Under P , the event frequency among all points receiving score $1/2$ is $1/2$, so the sole score level is perfectly calibrated.

Under Q, the event frequency is $3/4$ while the predictor still emits $1/2$. The deployment calibration gap is

$$\boxed{1/4}.$$

The model did not change. The conditional outcome mechanism did not change. Only the covariate mixture changed.

63.6 Calibration failure need not mean every risk changed

For the same predictor,

$$(1/2 - Y)^2 = 1/4$$

for either label. Thus Brier risk is

$$\boxed{1/4}$$

under both P and Q.

So calibration deterioration and generic predictive-risk deterioration are not the same statement.

63.7 Exact importance weights

For that shift,

$$w(a) = \frac{3/4}{1/2} = \frac{3}{2},$$

$$w(b) = \frac{1/4}{1/2} = \frac{1}{2}.$$

And

$$\frac{1}{2} \frac{3}{2} + \frac{1}{2} \frac{1}{2} = 1.$$

The reweighted source event rate is

$$\frac{1}{2} \frac{3}{2} = \frac{3}{4},$$

which exactly recovers the deployment event rate.

63.8 A detected marginal shift need not be a performance failure

Let X in $\{u, v\}$, with $Y(u)=0$ and $Y(v)=1$. Use exact predictor $f(u)=0$, $f(v)=1$.

Let

$$P_X = (1/2, 1/2)$$

and

$$Q_X = (9/10, 1/10).$$

The marginal laws differ substantially:

$$\text{TV}(P_X, Q_X) = 2/5.$$

Yet the predictor is right on every input under both laws. Therefore

$$\boxed{R_P^{0/1} = R_Q^{0/1} = 0}.$$

A shift detector can be correct that the input marginal changed while a task-failure alarm would be wrong.

63.9 Concept shift is different

Keep the same X -marginal, but reverse the conditional labels under deployment. Then

$$P_X = Q_X$$

while

$$P(Y \mid X) \neq Q(Y \mid X).$$

The source-perfect predictor now has target error one. The covariate density ratio is identically one, so X -reweighting does nothing.

This is why distribution shift is too coarse a diagnosis.

63.10 Calibration, coverage, and selective risk each have their own transfer problem

A guarantee under P can depend on exchangeability, score distribution, subgroup mixture, acceptance threshold, calibration level sets, or loss distribution. Changing P to Q can disturb these in different ways.

Therefore:

$$P\text{-calibration} \not\Rightarrow Q\text{-calibration},$$

$$P\text{-coverage} \not\Rightarrow Q\text{-coverage},$$

and

$$P\text{-selective-risk control} \not\Rightarrow Q\text{-selective-risk control}.$$

Each transfer needs its own assumptions.

63.11 Adversarial robustness asks a different question

Average-case target risk is

$$R_Q(f) = \mathbb{E}_Q[L(f(X), Y)].$$

Adversarial risk instead has the form

$$R_{\text{adv}}(f) = \mathbb{E}_P \left[\sup_{x' \in \Delta(X)} L(f(x'), Y) \right].$$

Madry et al. study adversarial robustness through robust optimization against a specified perturbation/adversary class (Madry et al. 2018).

The supremum is the important difference. A deployment distribution and a worst-case perturbation set are not interchangeable objects.

63.12 Exact adversarial separation

Take X in $\{-1, +1\}$, with labels matching sign, and a perfect predictor. Clean risk is zero.

Declare

$$\Delta(-1) = \Delta(+1) = \{-1, +1\}.$$

For either original example, the adversary can choose the opposite input while the original label is retained. Thus worst-case loss is one at every source point:

$$\boxed{R_{\text{adv}} = 1}.$$

So $R_P=0$ can coexist with $R_{\text{adv}}=1$.

63.13 Robustness is perturbation-set relative

If instead $\Delta(x)=\{x\}$, adversarial risk equals clean risk. The model did not change; only the allowed perturbation set changed. Therefore “robust” is incomplete without the threat/perturbation model.

63.14 Robust optimization

A robust-training objective is schematically

$$\min_{\theta} \mathbb{E}_P \left[\sup_{\delta \in \Delta(X)} L(f_{\theta}(X + \delta), Y) \right].$$

The objective is meaningful only with a declared data law, perturbation set, loss, and optimization approximation. Robustness to one perturbation family is not automatically robustness to another.

63.15 Structural sensitivity is not an adversarial synonym

Suppose a system component changes: tokenizer, routing topology, communication graph, measurement process, sensor transform, feature-generation mechanism, or intervention policy.

A structural-sensitivity analysis must name that component and compare behavior across the change. A change in predictive statistics can signal changed behavior but cannot by itself identify which mechanism changed.

63.16 Average-case shift is not worst-case shift

A law Q assigns probability mass. An adversarial set $\Delta(x)$ specifies possible worst-case alternatives around a point.

One integrates:

$$\mathbb{E}_Q[\cdot].$$

The other maximizes:

$$\sup_{\delta \in \Delta(x)} [\cdot].$$

Neither operation silently replaces the other.

63.17 Shift magnitude is metric-relative

Even “large shift” requires a discrepancy: total variation, KL divergence, Wasserstein distance, feature discrepancy, calibration change, risk change, or structural edit magnitude. These can rank changes differently.

The benign marginal-shift witness has nonzero total variation but zero task-risk change.

63.18 Support overlap determines what reweighting can say

Importance weighting is strongest when target mass is represented in source support. Poor overlap creates large weights and unstable estimation. Zero overlap creates a literal support failure for ordinary density-ratio correction.

The support condition is therefore part of the claim.

63.19 The unchanged model can still face a changed theorem

When deployment law changes, model parameters, architecture, scoring rule, threshold, and code may all remain fixed. Yet the guarantee can change because its probability law changed.

Governance must bind guarantees to laws, not only model artifacts.

63.20 Metric firewall

SHIFT keeps separate calibration, conformal/marginal coverage, selective risk, source predictive risk, deployment predictive risk, adversarial risk, and structural sensitivity. Every experiment must state which object it measured.

63.21 What SHIFT does not license

It does not license:

- shift detected implies model failed;
- source calibration implies deployment calibration;
- good target-average risk implies adversarial robustness;
- changed outputs imply an identified structural cause.

63.22 Operational checklist

For every deployment-shift claim: 1. name P; 2. name Q; 3. name the changed marginal, conditional, or mechanism; 4. name the metric; 5. check support; 6. state average-case versus worst-case semantics; 7. separate detection from performance; 8. separate predictive failure from causal diagnosis; 9. replay an exact or empirical witness; 10. state what remains unproved.

63.23 Durable rule

robustness is always relative to a law, metric, and perturbation/mechanism class.

A detected change is evidence of change. It is not by itself evidence of task failure, calibration failure, adversarial vulnerability, or structural cause.

63.24 References used in this chapter

- (Sugiyama et al. 2007)
- (Madry et al. 2018)

Chapter 64

Interlude: From Readability to Functional Evidence

Epistemic status: audited Evidence and Transformer prerequisites + Atlas-owned finite counterexamples + source-scoped interpretation.

Specification: manuscript/specifications/ATLAS-CH-MECHDIAG-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-MECHDIAG-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-MECHDIAG-001.md

Source lock: sources/source-locks/ATLAS-CH-MECHDIAG-001.yaml

A representation can contain a perfectly readable feature that the system’s output does not use.

This is a problem for a tempting inference: if a probe predicts a concept from an internal activation, it is easy to say that the model uses that concept to make its decision. The first statement is about **recovering information** from a representation; the second concerns **functional dependence** of the declared output on an internal variable.

They require different evidence. The chapter’s central question is:

Which intervention changes the behavior, according to which metric and reference comparison?

A successful local test is still not, by itself, proof of a unique global explanation.

64.1 Exact finite separation

Let an input x produce an internal representation $h(x) \in \mathcal{H}$. A declared downstream map

$$F : \mathcal{H} \rightarrow \mathcal{Y}$$

produces the output $F(h(x))$.

A probe $p : \mathcal{H} \rightarrow \mathcal{Z}$ instead predicts some target quantity $z(x)$. Its observed predictive success depends on the probe class, evaluated inputs, target definition and evaluation distribution.

A probe's accuracy asks whether z is **accessible** to that probe. It does not establish that F reads the same coordinates. In particular,

$$p(h(x)) = z(x) \not\Rightarrow F \text{ functionally depends on the probed coordinates.}$$

The right-hand statement needs a specified dependence or intervention test. Neither question can be answered by an attractive visualization of an activation alone.

Two-coordinate separation

Take the finite input domain

$$x \in \{-1, +1\}, \quad h(x) = (x, x), \quad F(h_1, h_2) = h_1.$$

Both coordinate readers

$$p_1(h) = h_1, \quad p_2(h) = h_2$$

recover x exactly on this domain. Their probe accuracies are identical and perfect. But F ignores h_2 by definition.

To see the distinction, compare the unmodified state with two zero-setting interventions:

Input x	Original $F(x, x)$	Set $h_1 = 0$: $F(0, x)$	Set $h_2 = 0$: $F(x, 0)$
-1	-1	0	-1
+1	+1	0	+1

Thus the difference from the original output is nonzero when coordinate 1 is zeroed, but exactly zero when coordinate 2 is zeroed. The conclusion is exact **for this specified map, domain and intervention**. It does not assert that every high-performing probe of a trained model is functionally unused.

Reference-state substitution control

Zero is one possible intervention value, not a neutral universal default. A complementary test replaces a coordinate with its value in a declared **clean/reference state**.

Begin at the opposite-input state

$$h(-x) = (-x, -x).$$

Patch coordinate 1 from $h(x)$, obtaining $(x, -x)$. The output becomes

$$F(x, -x) = x.$$

Patch coordinate 2 instead, obtaining $(-x, x)$. The output remains

$$F(-x, x) = -x.$$

This clean/reference substitution supports the same local separation as the zero-setting test. Crucially, the states $(x, -x)$ and $(-x, x)$ are not among the naturally observed states $h(\{-1, +1\})$. The algebra is valid because this toy F is defined on both, but a real network's off-distribution patched activations may require additional controls. A restoration effect does not automatically make the patched state natural or identify a unique circuit.

64.2 Diagnostic record

A component-level assertion should be transported with its actual test, not reduced to “feature j matters.”

Use the Atlas diagnostic record

$$D = (B, S, T, M, R),$$

where:

- B is the declared target behavior, such as the original output $F(h(x))$;
- S is the component or component set under examination;
- T is the intervention family, including the value being inserted or removed;
- M is the measured quantity, such as a score difference, error rate or exact output change;
- R specifies reference inputs, replacement states, baselines and comparison rules.

For the two-coordinate example, one measurable quantity is

$$M_T(x) = |F(h(x)) - F(T(h(x)))|.$$

At either input $x \in \{-1, +1\}$, setting $h_1 = 0$ gives $M_T(x) = 1$; setting $h_2 = 0$ gives $M_T(x) = 0$.

The notation does not create an empirical guarantee. It forces the investigator to declare what was actually varied and what counts as an effect.

64.3 Redundancy boundary

Functional tests have their own false-negative modes. Consider another finite map,

$$G(h_1, h_2) = \max(h_1, h_2), \quad (h_1, h_2) \in \{0, 1\}^2.$$

At $(1, 1)$, its output is 1. Zeroing either coordinate individually leaves the output unchanged:

$$G(0, 1) = G(1, 0) = 1.$$

Zeroing both changes it:

$$G(0, 0) = 0.$$

This is an exact redundancy counterexample. A one-coordinate ablation is null even though the pair jointly sustains the output under the declared intervention. The result licenses

null one-component effect $\nRightarrow$ absence of a functionally relevant representation.

It does **not** prove that every null result hides redundancy. The correct next step is to test justified groups or alternative interventions, with the expanded search space and limitations explicitly reported.

Three diagnostic outcomes, not two

A tested property can be:

1. **Accessible:** a specified readout class recovers the property on the evaluated distribution.
2. **Not recovered by this test:** a specified readout fails, or a specified intervention produces no measured change.
3. **Absent under a stronger declared criterion:** a separate, adequately justified search or impossibility argument rules out the property in a specified representation class.

The second category is not automatically the third. Failure to find a linear probe, for example, does not rule out a nonlinear readout. Nor does probe success guarantee downstream functional use.

This distinction prevents a test result from being promoted into a universal information or mechanism claim.

Localization, restoration and uniqueness

Locating an activation with a substantial intervention effect is one question. Showing that a chosen small component set is a **complete** explanation is another.

A local restoration can demonstrate sufficiency under the chosen patch and reference comparison. It need not demonstrate:

- necessity against all alternative routes;
- uniqueness among competing component sets;
- completeness over every relevant input;
- minimality under a declared size or cost criterion;
- invariance to a change of representation basis.

For example, if an invertible linear recoding replaces h by Ah while the downstream function is correspondingly changed to $F \circ A^{-1}$, the overall input-output map remains identical, yet the identities of individual coordinates generally change. A coordinate label is therefore not automatically a basis-independent feature.

An evidence report should separate the *tested* component set and representation from claims of globally unique mechanism.

Activation interventions are not parameter edits

Changing a hidden activation during one forward pass and editing a model's stored parameters are different operations.

An activation patch compares a behavior under a controlled internal-state substitution. A parameter edit changes the function used by subsequent inputs and can have effects outside the test case. Success on one example does not establish preservation of unrelated behavior, and it does not make the edited parameter location a uniquely identified causal component.

The external interpretability literature recorded in the source lock motivates mechanistic probing, intervention and editing questions. The bibliography keys in that source-lock file are retained as documentary identifiers, not inserted as citations until the manuscript bibliography defines them. This chapter uses those sources as context, not as an authority for a general causal theorem. Its exact proof obligation is confined to the finite constructions above.

Reading an intervention report

A reader should be able to ask, in order:

1. Which behavior B was selected, and on which input distribution?

2. Which internal representation and component set S were used?
3. Is the result an observational probe, a zero-setting ablation, a clean/reference patch, or a parameter change?
4. What precisely is measured by M , and what comparison is specified by R ?
5. Does the intervention take the system outside its observed or training-supported states?
6. Were related components, groups or alternative paths checked for redundancy?
7. Was the conclusion local to the selected inputs, or was a larger claim independently supported?

The finite examples show why changing only one of these fields can change what follows from the evidence.

What this chapter does not prove

Neither exact example is an empirical finding about a deployed neural network. The first proves that a perfectly readable coordinate may be unused by a particular output map. The second proves that null one-coordinate masking can coexist with joint support of a behavior.

The chapter does not prove that a probe identifies a causal mechanism; that successful patching establishes a unique explanation; that a given head, neuron or coordinate has basis-independent semantics; that single-feature masking is generally reliable; or that activation editing and parameter editing are interchangeable.

A spectral correlate is likewise not automatically an interventionally validated explanation.

64.4 Downstream handoff

ATLAS-CH-SPECTRALDIAG-001 may inherit four bounded objects:

- readability versus functional dependence;
- the explicit diagnostic tuple $D = (B, S, T, M, R)$;
- the two-coordinate readout/patching witness;
- the redundancy control and its null-test warning.

It must independently test any claim that a spectral signature identifies a functional mechanism. The presence of a readable or statistically distinctive spectral feature does not, on its own, satisfy that requirement.

The durable rule is:

An internal measurement becomes functional evidence only relative to a declared behavior, intervention, metric, reference rule and tested scope.

64.5 References used in this chapter

- Elazar et al. (2021), identifier `ElazarEtAl2021Amnesic`.
- Meng et al. (2022), identifier `MengEtAl2022FactualAssociations`.
- Wang et al. (2023), identifier `WangEtAl2023Interpretability`.

These source-lock identifiers still need bibliographic reconciliation before chapter-level citation signoff.

Exact source identities, prerequisite locks and finite claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-MECHDIAG-001.yaml`

Chapter 65

Spectral and Operator Diagnostics

Epistemic status: audited non-normal/operator substrate + audited mechanistic-diagnostics substrate + one primary Koopman source + Atlas-owned exact finite controls.

Specification: manuscript/specifications/ATLAS-CH-SPECTRALDIAG-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-SPECTRALDIAG-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-SPECTRALDIAG-001.md

Source lock: sources/source-locks/ATLAS-CH-SPECTRALDIAG-001.yaml

A spectrum is never just “the spectrum.”

It is the spectrum of a particular object:

- a weight matrix;
- a transition matrix;
- a Jacobian at a reference state;
- a Hessian at a parameter point;
- a Koopman operator on a declared observable space;
- a finite empirical approximation to one of these.

The governing rule is:

name the operator, reference point, and metric before interpreting the diagnostic.

65.1 Different spectral objects answer different questions

For a matrix A , eigenvalues summarize invariant linear modes in one sense.

Singular values summarize Euclidean amplification geometry.

Pseudospectra and resolvent norms expose sensitivity and non-normal behavior that eigenvalues alone may hide.

A Jacobian spectrum is local to a state.

A Hessian spectrum is local to a parameter point and objective.

A Koopman spectrum belongs to an operator acting on observables.

These are not interchangeable.

65.2 The non-normal boundary

The audited NONNORMAL chapter already establishes the crucial warning:

$$\rho(A) < 1$$

need not prevent finite-horizon amplification when A is non-normal.

Therefore an eigenvalue diagnostic cannot silently substitute for transient-response or pseudospectral diagnostics.

65.3 Exact equal-eigenvalue control

Take

$$D = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \end{pmatrix}, \quad N = \begin{pmatrix} 1/2 & 2 \\ 0 & 1/2 \end{pmatrix}.$$

Both have eigenvalue multiset

$$\{1/2, 1/2\}.$$

For

$$e_2 = (0, 1)^\top,$$

$$\|De_2\|_2^2 = 1/4,$$

while

$$\|Ne_2\|_2^2 = 17/4.$$

So equal eigenvalues coexist with sharply different finite response.

The diagnostic lesson is simple:

$$\boxed{\text{eigenvalue equality} \not\Rightarrow \text{response equality}}.$$

65.4 Singular values are different from eigenvalues

The singular values of A are the square roots of the eigenvalues of

$$A^*A.$$

They describe Euclidean input-output amplification.

For non-normal systems they can be much more directly related to one-step gain than eigenvalues are.

But singular values still discard orientation information.

65.5 Equal singular values can hide interface differences

Let

$$A = \text{diag}(2, 1/2), \quad B = \text{diag}(1/2, 2).$$

Both have the same eigenvalue multiset and the same singular-value multiset:

$$\{2, 1/2\}.$$

Yet for a fixed interface vector

$$e_1 = (1, 0)^\top,$$

$$\|Ae_1\|_2 = 2,$$

while

$$\|Be_1\|_2 = 1/2.$$

The difference is not in the global singular-value list.

It is in relative position.

65.6 Relative-position diagnostics

A system often interacts with fixed subspaces, readouts, token directions, residual channels, control inputs, or protected interfaces.

A useful diagnostic may therefore involve quantities such as

$$\|Av\|,$$

principal angles between singular subspaces and a declared interface subspace, or projected gains such as

$$\|P_Y A P_X\|.$$

These are not purely spectral invariants of A.

They are operator-plus-interface diagnostics.

65.7 Zero spectral drift need not mean zero behavioral drift

Move from A to B in the previous example.

The sorted eigenvalue multiset is unchanged.

The sorted singular-value multiset is unchanged.

So a drift score using only either multiset is zero.

But the response to e_1 changes from 2 to 1/2.

Thus

zero spectral-summary drift $\nrightarrow$ zero interface drift .

65.8 Pseudospectra remain a separate diagnostic family

Under the inherited 2-norm convention,

$$\Lambda_\varepsilon(A) = \{z : \sigma_{\min}(zI - A) \leq \varepsilon\}.$$

This captures resolvent growth and spectral sensitivity that the bare eigenvalue set may miss.

But a pseudospectrum is still descriptive evidence about a declared operator.

It does not by itself identify which component of a learned system is functionally necessary.

65.9 Jacobian spectra are local

For nonlinear map

$$z^+ = \Phi(z),$$

a Jacobian diagnostic at z_0 is based on

$$J_\Phi(z_0).$$

The reference state is part of the object.

Removing z_0 from the report removes part of the diagnostic definition.

65.10 Same local Jacobian, different nonlinear maps

Let

$$F(x) = x/2,$$

and

$$G(x) = x/2 + x^2.$$

At zero,

$$F'(0) = G'(0) = 1/2.$$

But

$$F(1/2) = 1/4,$$

while

$$G(1/2) = 1/2.$$

Therefore a local Jacobian spectrum cannot be promoted into a global nonlinear theorem.

65.11 Transition-local signatures

If a system moves through states

$$z_0, z_1, z_2, \dots,$$

then one may study

$$J_t = D\Phi(z_t).$$

A spectral sequence

$$\sigma(J_t)$$

is a transition-local diagnostic.

Its interpretation requires the state/time index, estimation window, and operator-estimation method.

65.12 Hessian spectra are also local and incomplete

For objective $L(\theta)$, the Hessian

$$H_L(\theta_0)$$

captures second-order curvature at a declared parameter point.

It does not by itself determine the gradient.

65.13 Exact Hessian control

Let

$$f(x, y) = x^2 + y^2,$$

$$g(x, y) = x^2 + y^2 + x.$$

Both have Hessian

$$2I$$

and therefore the same Hessian spectrum

$$\{2, 2\}.$$

But at the origin

$$\nabla f = (0, 0),$$

while

$$\nabla g = (1, 0).$$

The same Hessian spectrum can occur at a stationary point and a non-stationary point.

65.14 Koopman changes the object being diagonalized

For nonlinear state map T and observable h , the Koopman operator acts as

$$(Uh)(x) = h(T(x)).$$

It is linear as an operator on observables even when T is nonlinear.

Mezić develops this spectral viewpoint for dynamical systems and model reduction (Mezić 2005).

The important semantic point is:

Koopman spectrum is an observable-operator spectrum.

It is not the Jacobian spectrum of T by definition.

65.15 Exact two-state Koopman witness

Let

$$T(0) = 1, \quad T(1) = 0.$$

On indicator observables δ_0 and δ_1 ,

$$U\delta_0 = \delta_1, \quad U\delta_1 = \delta_0.$$

So

$$U = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Hence

$$U^2 = I,$$

with trace zero and determinant -1, giving eigenvalues

$$\boxed{\{1, -1\}}.$$

65.16 Finite Koopman approximations need their own error language

In practice one may estimate a finite matrix from data and call it a Koopman approximation.

That introduces choices:

- observable dictionary;
- sampling distribution;
- time window;
- truncation/rank;
- regression method;
- regularization;
- finite precision.

The spectrum of that finite matrix is exactly the spectrum of the fitted matrix.

Its relation to an underlying infinite-dimensional Koopman operator requires additional approximation theory.

65.17 Spectral drift is definition-relative

A drift diagnostic can compare

- eigenvalue sets;
- singular-value lists;
- pseudospectral contours;
- resolvent norms;
- subspace angles;
- projected gains;
- local Jacobian spectra;
- Koopman estimates.

These are different drift objects.

A single scalar named “spectral drift” is incomplete unless the underlying object and discrepancy are declared.

65.18 Descriptive quality is not functional necessity

The audited MECHDIAG packet supplies a second firewall.

A probe or diagnostic can track a property without showing that the measured feature is necessary for the system’s output.

The same applies to spectral signatures.

A signature can correlate strongly with performance and still be epiphenomenal.

65.19 Predictive utility is not mechanism

Suppose a spectral statistic predicts failure one step before failure occurs.

That is useful predictive evidence.

It does not yet show that manipulating the associated eigenspace, singular direction, or pseudospectral region will change the failure.

Prediction and mechanism are different claims.

65.20 Mechanistic significance requires intervention

To promote a spectral signature toward mechanism, pair it with a functional test such as:

- ablation of the implicated subspace;

- controlled perturbation of the operator block;
- substitution of a matched spectral control;
- recovery/rescue after restoring the implicated component;
- another explicitly justified intervention.

The intervention must target the object that the spectral interpretation claims matters.

65.21 A matched-spectrum control is especially valuable

The A/B control above has the same eigenvalue and singular-value multisets but different fixed-interface response.

This is useful because it holds the global spectral summaries fixed while changing a functional interface relation.

Matched controls can therefore falsify over-strong spectral explanations.

65.22 Finite precision and conditioning

Empirical spectra can be sensitive to:

- roundoff;
- non-normality;
- ill-conditioning;
- nearly repeated eigenvalues;
- low-rank truncation;
- finite-sample noise.

A plotted spectrum should therefore carry enough numerical context to distinguish exact structure from estimated structure.

65.23 No universal scalar diagnostic

Different systems can fail for different reasons.

A scalar that is useful for one operator family can become blind under another family or another interface.

The exact controls already show this for eigenvalue and singular-value summaries.

Thus the chapter rejects:

one spectral scalar $\equiv$ universal health metric.

65.24 Reporting protocol

For every spectral/operator diagnostic report:

1. operator/object identity;
2. reference state, parameter point, or time window;
3. spectrum/diagnostic type;
4. norm and numerical convention;
5. estimation/truncation method;
6. uncertainty/tolerance;
7. interface or relative-position object, if used;
8. descriptive versus predictive versus functional claim class;
9. intervention evidence, if mechanistic significance is claimed;
10. known non-implications.

65.25 Durable non-implications

same eigenvalues $\nRightarrow$ same transient behavior,

same singular values $\nRightarrow$ same interface behavior,

same local Jacobian spectrum $\nRightarrow$ same nonlinear map,

same Hessian spectrum $\nRightarrow$ same stationarity status,

spectral correlation $\nRightarrow$ functional necessity,

and

functional association $\nRightarrow$ causal mechanism without a suitable intervention.

65.26 Closing view

Spectral diagnostics are powerful precisely because they expose structure that scalar performance metrics can miss.

They become misleading when their object is unnamed or their epistemic role is inflated.

The durable rule is:

a spectral signature is evidence about a declared operator; mechanistic meaning requires a separate functional brid

65.27 References used in this chapter

- (Horn and Johnson 2012)
- (Trefethen and Embree 2005)
- (Mezić 2005)

Mechanistic-diagnostics sources and exact prerequisite identities are recorded in `sources/source-locks/ATLAS-CH-SPECTRALDIAG-001.yaml`.

Chapter 66

Compression and Description Length

Epistemic status: mathematical and engineering exposition built from audited Information and Representation prerequisites, classical compression sources, and Atlas-owned exact witnesses.

Derivation packet: ‘mathematics/derivations/ATLAS-CH-COMPRESS-001-DERIVATIONS.md‘

Computational witness: ‘mathematics/computational-witnesses/ATLAS-CW-COMPRESS-001.md‘

Source lock: ‘sources/source-locks/ATLAS-CH-COMPRESS-001.yaml‘

66.1 Compression begins with a code

“Smaller” can mean several different things.

A model can use:

- fewer scalar parameters;
- fewer nonzero parameters;
- fewer distinct parameter values;
- fewer bits per value;
- a lower-rank factorization;
- a shorter entropy-coded file;
- a smaller student network;
- a shorter formal description under a declared code.

These are not identical quantities.

The chapter therefore begins with a rule:

compression claim $\Rightarrow$ declare the object, code, and tolerated distortion.

Without those declarations, a compression ratio can be numerically correct and scientifically ambiguous.

66.2 Description length is not parameter count

Minimum-description-length thinking treats model selection as a coding problem (Rissanen 1978).

For a declared code C , model M , and data D , a two-part objective has the form

$$L_C(M, D) = L_C(M) + L_C(D \mid M).$$

The first term describes the model.

The second describes the data given that model.

A shorter model can be worse if it leaves a much longer residual/data description.

A larger model can be justified if it dramatically shortens the unexplained data.

This is richer than “count the parameters.”

66.3 A finite MDL witness

Suppose two hypotheses have declared model-code lengths

$$L(H_A) = 3, \quad L(H_B) = 7.$$

Suppose they encode the observed data equally well:

$$L(D \mid H_A) = L(D \mid H_B) = 5.$$

Then

$$L(H_A, D) = 8,$$

while

$$L(H_B, D) = 12.$$

Under this declared code, H_A wins.

Nothing in the arithmetic says that 3 bits and 7 bits are universally correct descriptions.

The point is that the code belongs inside the claim.

66.4 Lossless and lossy are different questions

Lossless compression preserves the coded object exactly.

Lossy compression preserves it only up to a declared distortion.

For learned systems, several distortions are possible:

- parameter error;
- operator error;
- output error;
- predictive loss;
- calibration change;
- retrieval degradation;
- latency or energy change.

Two compression methods can have the same storage ratio and very different task distortion.

So the meaningful object is not just

$$\frac{\text{original size}}{\text{compressed size}}.$$

It is the size–distortion pair.

66.5 Exact low-rank compression

Consider the 4×4 all-ones matrix

$$W = \mathbf{1}_4 \mathbf{1}_4^\top.$$

Its rank is one.

Let

$$u = v = (1, 1, 1, 1)^\top.$$

Then

$$W = uv^\top.$$

The dense matrix and the factors implement exactly the same linear map:

$$Wx = u(v^\top x)$$

for every $x \in \mathbb{R}^4$.

This is exact structural compression, not approximation.

66.6 Exact code-length comparison

Use a deliberately simple codec:

- the 4×4 shape is known;
- entries are binary;
- one format bit distinguishes dense and rank-one representations.

Dense form:

- one format bit;
- sixteen matrix-entry bits.

Total:

17 bits.

Rank-one form:

- one format bit;
- four bits for u ;
- four bits for v .

Total:

9 bits.

The same exact function has two different description lengths under the declared code.

That is the chapter's simplest exact compression witness.

66.7 Low rank can also be lossy

Now take

$$A = \text{diag}(4, 3, 1).$$

Its singular values are

4, 3, 1.

By the classical low-rank approximation result (Eckart and Young 1936), the best rank-one Frobenius approximation keeps only the largest singular value:

$$A_1 = \text{diag}(4, 0, 0).$$

The squared error is

$$\|A - A_1\|_F^2 = 3^2 + 1^2 = 10.$$

The best rank-two approximation is

$$A_2 = \text{diag}(4, 3, 0),$$

with squared error

1.

Low-rank representation is therefore not synonymous with exact compression.

It is exact only when discarded singular values are zero.

66.8 Pruning removes parameters

Pruning changes the support of the parameterization.

A simple magnitude-pruning rule removes weights whose absolute values fall below a threshold.

The appeal is obvious:

- fewer nonzero connections;
- potentially less storage;
- potentially less compute.

But magnitude is not the same thing as functional importance.

66.9 A small weight can still matter

Let

$$w = \begin{pmatrix} 1 \\ 1/8 \end{pmatrix}$$

and

$$x = \begin{pmatrix} 0 \\ 8 \end{pmatrix}.$$

Then

$$w^\top x = 1.$$

Apply a pruning threshold

$$\tau = 1/4.$$

The second weight disappears:

$$w_{\text{pruned}} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Now

$$w_{\text{pruned}}^\top x = 0.$$

The small parameter was decisive on this input.

So:

small magnitude $\not\Rightarrow$ functional irrelevance.

Neural pruning methods can work well in practice, including as part of the Deep Compression pipeline (Han et al. 2016a).

Their success is empirical and distribution-dependent.

66.10 Quantization changes precision

Quantization answers a different question.

The connections can remain present while each stored value uses fewer bits or is mapped to a smaller set of representable levels.

Take

$$w = (1/4, 1/2, 1/2, 1/4).$$

Use the fixed grid

$$Q = \{0, 1/4, 1/2, 3/4\}.$$

Every value of w lies exactly on the grid.

If raw values use 8 bits each, the payload is

$$4 \times 8 = 32 \text{ bits.}$$

If fixed-grid indices use 2 bits each, the payload is

$$4 \times 2 = 8 \text{ bits.}$$

Parameter distortion is zero in this witness.

The parameter count did not change.

The number of bits per parameter did.

66.11 Codebook overhead matters

The previous witness used a fixed grid known by the codec.

If the quantization levels are learned, the levels themselves must be stored.

That overhead belongs in the accounting.

This is one reason “4-bit model” and “4 bits per raw weight” are not necessarily the same storage claim.

Metadata, scales, zero points, codebooks, indices, alignment, and container formats can matter.

66.12 Weight sharing is different again

The same vector contains only two distinct values:

$$1/4 \quad \text{and} \quad 1/2.$$

Suppose a codec stores:

- two 8-bit centroids;
- four one-bit centroid indices.

The payload is

$$16 + 4 = 20 \text{ bits.}$$

That is shorter than four independent 8-bit values:

$$32 \text{ bits.}$$

All four connections remain.

Pruning removed a connection.

Weight sharing did not.

66.13 Deep Compression combines mechanisms

Han, Mao, and Dally combine pruning, trained quantization/weight sharing, and Huffman coding in one pipeline (Han et al. 2016a).

Those stages attack different redundancies:

- pruning removes selected connections;
- quantization reduces the set of distinct values;
- weight sharing reuses centroids;
- entropy coding exploits frequency structure in the resulting symbols.

Calling the whole pipeline “compression” is correct.

Treating all its stages as the same mathematical operation is not.

66.14 Entropy coding acts on symbol statistics

After a representation has been chosen, symbols may occur with unequal frequencies.

A variable-length code can assign shorter codewords to more common symbols.

This connects compression back to the Information chapter.

Entropy coding does not decide which neural connections are important.

It compresses a symbol stream under a declared statistical model.

66.15 Low rank and sparsity are different structures

A matrix can be sparse and full rank.

A matrix can be dense and rank one.

These are different kinds of compressible structure.

Sparsity says many coordinates are zero.

Low rank says many rows or columns are dependent through a lower-dimensional factorization.

The hardware consequences, approximation errors, and implementation costs can differ.

66.16 Structured parameterizations

Low rank and weight sharing are examples of replacing an unconstrained parameter array with a smaller structured family.

For

$$W \in \mathbb{R}^{m \times n},$$

a dense matrix has

$$mn$$

scalar slots.

A rank- r factorization

$$W = UV^\top$$

with

$$U \in \mathbb{R}^{m \times r}, \quad V \in \mathbb{R}^{n \times r}$$

stores

$$r(m+n)$$

scalar slots before metadata.

When

$$r(m+n) < mn,$$

the factorized form uses fewer scalar slots.

That arithmetic alone does not prove lower wall-clock latency.

Kernel efficiency and memory traffic still matter.

66.17 Distillation compresses behavior, not bits

Knowledge distillation takes a different route (Hinton et al. 2014).

A teacher or ensemble produces predictive targets.

A smaller student is trained to match those targets.

At temperature T , one can form softened teacher probabilities

$$p_T(i) = \frac{\exp(z_{T,i}/T)}{\sum_j \exp(z_{T,j}/T)}$$

and analogous student probabilities.

The student is optimized to reproduce teacher behavior under a declared loss.

Nothing requires the student parameters to be a bit-level encoding of the teacher parameters.

This is behavioral compression or transfer.

66.18 Exact equality on a finite dataset is not global equality

Suppose a student matches a teacher exactly on every example in a finite training set.

That proves equality on that set.

It does not prove

$$f_{\text{student}}(x) = f_{\text{teacher}}(x)$$

for every possible input.

The relevant domain must be declared.

This is the Representation chapter's equivalence discipline applied to compression.

66.19 Compression ratio is incomplete without retained behavior

A 100x smaller artifact can be excellent or useless.

The ratio alone does not say.

A serious report pairs size with something like:

- output error;
- predictive loss;
- accuracy;
- calibration;
- robustness;
- retrieval quality;
- task-specific utility.

Compression is a tradeoff surface, not a one-number race.

66.20 Parameter distortion can disagree with task distortion

A tiny parameter perturbation can have a large functional effect in one system.

A large parameter change can preserve function through reparameterization in another.

Therefore

$$\|\theta - \hat{\theta}\|$$

and task degradation are different measurements.

The right distortion depends on the object being preserved.

66.21 Compression and representation equivalence

Two encodings can represent the same function differently.

The rank-one witness makes this exact:

$$W$$

and

$$uv^\top$$

are different stored representations of the same linear operator.

This is why compression is naturally connected to representation theory.

We are looking for shorter encodings inside an equivalence class of acceptable behavior.

The equivalence relation must be stated.

66.22 MDL is not “smallest network wins”

MDL balances model description and residual/data description.

A model that is too small can fit poorly and require a longer data description.

A larger model can be justified if it compresses the data sufficiently.

The principle is not:

always pick fewer parameters.

It is:

under a declared coding framework, minimize the relevant total description.

66.23 Compression can reveal redundancy

If a model can be compressed with little change in declared behavior, then some part of its original description was redundant relative to that behavior and codec.

That is a useful scientific observation.

But redundancy can come from many sources:

- duplicate parameters;
- low-rank structure;
- unused features;
- overprecise numerical representation;
- repeated values;
- symmetries;
- overparameterization;
- training artifacts.

Compression alone does not tell us which mechanism caused the redundancy.

66.24 Compressibility is not interpretability

A highly compressible model can still be opaque.

A concise program can implement a difficult-to-understand computation.

A sparse network can still have distributed causal structure.

A low-rank map can still be semantically uninterpretable.

Therefore:

short description $\nRightarrow$ human-understandable mechanism.

That stronger inference belongs downstream, where compression will be considered as a possible probe rather than a proof.

66.25 Compressibility is not intelligence

The same caution applies to intelligence.

Many simple objects compress extremely well.

Some useful computations require large descriptions under a given code.

A system's compressibility may provide evidence about reusable structure.

It is not a scalar definition of intelligence.

66.26 Failure modes

66.26.1 Parameter count equals description length

False.

Precision, codebook structure, sparsity, entropy coding, and metadata all matter.

66.26.2 Pruning equals quantization

False.

One changes support; the other changes value representation.

66.26.3 Quantization equals weight sharing

Not necessarily.

A fixed quantization grid and a learned shared codebook have different storage accounting.

66.26.4 Low rank equals sparsity

False.

These are different structural constraints.

66.26.5 Distillation equals source coding

False.

Distillation transfers predictive behavior under a training objective.

66.26.6 High compression ratio means good compression

Incomplete.

Retained behavior or distortion must also be reported.

66.26.7 Same outputs on one dataset means same function

False outside the declared dataset.

66.26.8 Compressible means interpretable

False without additional evidence.

66.26.9 Compressible means intelligent

False.

That claim requires an independent theory.

66.27 Downstream handoff

Compression as Discovery and Intelligence Probe may inherit:

- declared-code discipline;
- exact-versus-lossy distinction;
- mechanism separation among pruning, quantization, sharing, low rank, entropy coding, and distillation;
- the requirement to report compression jointly with retained behavior;
- the warning that compressibility is evidence of shorter representation under a declared equivalence, not proof of mechanism or intelligence.

The downstream chapter must independently justify any stronger claim about:

- reusable computation;
- mechanistic discovery;
- abstraction;
- intelligence;

- reconstruction from a compressed residual.

66.28 References used in this chapter

- (Rissanen 1978) — shortest-description model selection.
- (Eckart and Young 1936) — optimal lower-rank matrix approximation.
- (Han et al. 2016a) — pruning, trained quantization/weight sharing, and Huffman coding.
- (Hinton et al. 2014) — teacher-student knowledge distillation.

Exact provenance and claim boundaries are locked in:

‘sources/source-locks/ATLAS-CH-COMPRESS-001.yaml’

Chapter 67

Compression as Discovery and Intelligence Probe

Epistemic status: audited Compression/Residual substrate + primary MDL/probing/compression-progress sources + Atlas-owned diagnostic synthesis.

Specification: manuscript/specifications/ATLAS-CH-COMPINTEL-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-COMPINTEL-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-COMPINTEL-001.md

Source lock: sources/source-locks/ATLAS-CH-COMPINTEL-001.yaml

Compression is attractive because it turns a vague word such as structure into a number.

If a model can describe data in fewer bits, perhaps it has found a pattern.

If a representation allows a target to be transmitted with fewer bits, perhaps it contains something reusable.

If a system suddenly compresses its observations better, perhaps it has discovered a regularity.

Those are reasonable hypotheses.

They are not identities.

The central rule of this chapter is therefore:

compression is evidence only relative to a declared code, target, split, and control set.

The stronger the claim, the stronger the controls must be.

67.1 Compression is tempting for a good reason

Prediction and compression are closely related.

If a model assigns high probability to what actually occurs, an entropy code can often describe the observations with fewer bits.

If a representation makes a target easy to predict from few examples, a prequential or MDL-style probe can transmit those targets economically.

This connection appears in several traditions.

Solomonoff's formal induction framework assigns greater prior weight to sequences generated by shorter programs (Solomonoff 1964).

Minimum-description-length reasoning turns statistical learning into a coding problem.

Blier and Ollivier explicitly measured deep-learning models through description length and showed that coding methodology materially changes the resulting bound (Blier and Ollivier 2018).

Voita and Titov used minimum description length as a probe of learned representations, replacing a single final-accuracy number with a coding quantity that also reflects how hard it is to learn the target from the representation (Voita and Titov 2020).

Schmidhuber proposed compression progress as a signal for curiosity and discovery by a resource-bounded observer (Schmidhuber 2009).

The shared intuition is productive:

regularity that supports prediction can support compression.

The dangerous step is to reverse or overextend it.

67.2 The Atlas boundary

The Atlas does not define intelligence as compression.

It does not define understanding as short description.

It does not define mechanism as decodability.

It does not define scientific discovery as any decrease in codelength.

The chapter instead asks a narrower question:

when is compression a useful empirical probe for reusable predictive structure?

This turns a philosophical slogan into an experimental contract.

67.3 Begin with the code

A codelength is never just a property of the data.

It is a property of the data under a code.

Write

$$L_C(D)$$

for the length assigned to data object D by code C .

A different valid code can give a different answer:

$$L_{C_1}(D) \neq L_{C_2}(D).$$

That is not a defect.

It is the reason the code belongs inside the notation.

The audited Compression chapter already made the same point for model and data description.

COMPINTEL extends it to probing and discovery claims.

67.4 A short code can be trivial

Consider eight zeros:

$$Z = (0, 0, 0, 0, 0, 0, 0, 0).$$

Use a prefix code with two branches:

- one bit means eight zeros;
- otherwise transmit a literal eight-bit string behind a branch bit.

Then

$$L_C(Z) = 1.$$

A literal baseline requires eight bits.

The gain is seven bits.

This is strong compression.

It is not strong intelligence.

No learner was required.

No abstraction was discovered.

No generalization was tested.

No mechanism was explained.

The witness gives an exact counterexample:

$$\boxed{\text{compressibility} \not\Rightarrow \text{intelligence.}}$$

67.5 Why training compression is too weak

A sufficiently flexible system can memorize training data.

That can lower training loss.

It can also lower a training-set code.

Neither fact shows that the system found a rule that applies elsewhere.

A lookup table can describe previously seen labels perfectly.

Its useful content may vanish on unseen examples.

Therefore the first diagnostic upgrade is simple:

measure codelength on untouched data.

67.6 Held-out codelength

Let:

- R be the representation being tested;
- T be the training partition;
- H be an untouched held-out partition;
- Y_H be the held-out targets;
- C be the code family;
- $\mathcal{P}$ be the admissible probe or decoder class.

Write

$$L_C(Y_H \mid R, T)$$

for the number of bits needed to transmit held-out targets after learning only from the training partition and the declared representation.

Now introduce a baseline B .

Define

$$\Delta_C(R) = L_C(Y_H \mid B, T) - L_C(Y_H \mid R, T).$$

If

$$\Delta_C(R) > 0,$$

then R supports shorter held-out transmission than the baseline under the declared measurement system.

This is an empirical structure claim.

It is still conditional on the measurement system.

67.7 The period-two witness

Take the training prefix

$$T = (0, 1, 0, 1).$$

Suppose the held-out suffix is

$$H_s = (0, 1, 0, 1).$$

Use a conditional prefix code:

- one bit means continue the period-two motif inferred from training;
- otherwise use one branch bit plus four literal held-out bits.

The structured suffix costs

$$1$$

bit.

A four-bit literal baseline costs

$$4$$

bits.

Therefore

$$\Delta_C(H_s) = 3.$$

This is held-out compression.

The learned pattern made a correct prediction beyond the training prefix.

67.8 A matched failure control

Keep the training prefix exactly the same.

Change only the held-out suffix:

$$H_c = (0, 0, 1, 1).$$

The period-two rule fails.

The codec falls back to literal transmission:

$$1 + 4 = 5$$

bits.

Against the same four-bit baseline,

$$\boxed{\Delta_C(H_c) = -1.}$$

The comparison is useful because training exposure is identical.

One future continues the learned regularity.

The other does not.

The codelength changes on held-out data.

This is the smallest exact example of what COMPINTEL wants from a predictive-structure probe.

67.9 The result is still codec-relative

The witness does not mean that one bit is the universal complexity of the structured suffix.

Another code could assign a different length.

The scientifically relevant claim is narrower:

under a declared code whose semantics are held fixed, the learned regularity reduces held-out description length relative to a declared baseline.

A robust result should survive reasonable changes in code or come with an argument explaining why the comparison is invariant.

67.10 MDL probing

Voita and Titov make this logic operational for learned representations (Voita and Titov 2020).

A standard probe often asks:

what final accuracy can a classifier reach from this representation?

An MDL probe asks something closer to:

how economically can the target labels be communicated when this representation is available?

The difference matters.

Two representations can end at similar accuracy while requiring different amounts of data or model complexity before the target becomes easy to recover.

The codelength integrates that learning trajectory.

This makes it a useful diagnostic surface.

It does not make it a mechanism detector.

67.11 Same final accuracy can hide different learning difficulty

Imagine two probes that both reach perfect final held-out accuracy.

Probe A learns the target after a few examples.

Probe B requires almost the entire training set.

A prequential code charges predictions made along the way.

Probe A can therefore achieve a shorter cumulative description even though final accuracy is identical.

Hence

same final accuracy $\nRightarrow$ same codelength.

This is one reason codelength can reveal differences hidden by accuracy.

67.12 Same codelength can hide different behavior

The converse problem also exists.

Codelength is a scalar summary.

Two models can have equal total code length while:

- failing on different examples;
- having different calibration;
- behaving differently under distribution shift;
- using different internal mechanisms.

Therefore

same codelength $\nrightarrow$ same behavior.

Compression and behavior belong on separate axes.

67.13 Random-label controls

A powerful probe can learn surprising things.

Sometimes what it learns is the probing task itself rather than structure already present in the representation.

Random-label controls help separate these possibilities.

Replace the real target Y with matched random labels.

Then rerun the same pipeline.

If the representation gives nearly the same codelength advantage on random labels as on the real target, the intended structure claim is weak.

A useful diagnostic should show a gap:

$$\Delta_C^{\text{real}} > \Delta_C^{\text{random}}$$

by a meaningful margin.

Voita and Titov explicitly motivate MDL probing partly through such random-task distinctions (Voita and Titov 2020).

67.14 Shuffle controls

A second control breaks alignment.

Keep the representation vectors.

Keep their marginal distribution.

Shuffle which example receives which representation.

If the compression gain survives, then the signal may come from generic probe capacity or marginal statistics rather than example-specific structure.

A representation-target claim should weaken sharply when their correspondence is destroyed.

67.15 Random-representation controls

Where meaningful, compare against:

- an untrained network;
- a randomly initialized representation;
- a random projection with matched dimension.

This helps answer:

did training create an economical representation of the target, or would a generic feature map work just as well?

The answer is empirical.

The control belongs in the protocol.

67.16 Probe capacity is part of the instrument

A probe is a measuring device.

Its capacity matters.

If one representation is paired with a much stronger decoder than another, a codelength difference can be caused by the instrument.

Therefore a serious COMPINTEL experiment either:

- holds probe capacity fixed;
- sweeps capacity;
- charges model complexity through the code;
- or reports all three.

Regularization is part of the same issue.

The measurement apparatus must be visible.

67.17 Blier and Ollivier: coding method matters

Blier and Ollivier studied the description length of deep-learning models and found that different coding approaches gave very different compression bounds (Blier and Ollivier 2018).

The durable lesson for COMPINTEL is not a specific numeric result.

It is methodological:

a claim about compressibility can change when the coding method changes.

That is why codec sensitivity is a mandatory control rather than an editorial footnote.

67.18 Solomonoff: compression and prediction in an idealized theory

Solomonoff's induction theory gives shorter generating programs greater prior weight (Solomonoff 1964).

This provides a deep formal connection between economical description and prediction.

But several boundaries remain.

The ideal theory uses universal computation.

Its complexity quantities are not practical finite compressors.

A neural-network codelength is not automatically Solomonoff complexity.

A short empirical code is not automatically the shortest possible description.

The source supports the conceptual bridge.

It does not license a universal intelligence score.

67.19 Compression progress

Suppose a learner changes over time.

At stage t , define

$$\Gamma_t = L_{C,t-1}(Y_H \mid R, T) - L_{C,t}(Y_H \mid R, T).$$

Positive Γ_t means that the held-out target became cheaper to describe.

This can be interesting.

It suggests that the learner has acquired predictive structure.

Schmidhuber proposes a broad version of this idea as a principle for curiosity and discovery (Schmidhuber 2009).

The Atlas keeps the proposal explicitly hypothesis-level.

67.20 When compression progress is fake

A positive Γ_t can arise without discovery.

Examples:

- the held-out set changed;
- labels leaked into training;
- the code stopped charging side information;
- the probe class grew;
- the compressor changed;
- the model was allowed more compute;
- the target behavior was degraded;
- bookkeeping changed.

Therefore a discovery interpretation requires fixed or charged semantics.

The safest longitudinal comparison freezes:

- held-out data;
- target definition;
- code accounting;
- probe class;
- evaluation protocol.

67.21 Discovery needs controls

A compression-progress event is stronger when it satisfies all of the following:

1. the gain occurs on untouched data;
2. random-label controls do not show the same gain;
3. shuffled representations destroy the gain;
4. matched probe-capacity controls cannot explain it;
5. the result survives reasonable codec variants;
6. behavior is retained;
7. the newly compressed structure helps in another context.

Only then does the word discovery become plausible.

Even then, the statement remains empirical and scoped.

67.22 Reuse is stronger than one-task prediction

Suppose a descriptor is transmitted once and then used for several tasks.

Let

$$Y^{(1)}, \dots, Y^{(m)}$$

be task targets.

If the same descriptor supports low incremental codelength for many tasks,

$$\sum_j L_C(Y_H^{(j)} \mid R, T_j)$$

can be smaller than the cost of separate task-specific representations.

This is evidence of reuse.

It begins to approach the Atlas notion of portable structure.

But it still does not prove that the descriptor is least.

67.23 Compression and the Residual

The Residual chapter asks for a least admissible descriptor sufficient for a declared capability under a declared factorization preorder.

COMPINTEL asks a different question:

which descriptors make relevant behavior economical to recover?

The two questions are related.

They are not identical.

A shorter code does not prove factorization leastness.

If

$$L_C(R_1) < L_C(R_2),$$

we cannot conclude

$$R_1 \preceq R_2.$$

Codelength defines one ordering.

The Residual preorder defines another.

This distinction prevents compression from swallowing the entire representation theory.

67.24 Invertible recoding

Suppose

$$R_2 = g(R_1)$$

for invertible g .

Semantically, the two representations contain the same information.

If the admissible decoder class can efficiently apply g^{-1} , a well-designed probe may treat them equivalently.

But a restricted probe class may not.

For example, a linear probe can fail after a nonlinear invertible recoding even though no information was lost.

Therefore a probe score is a property of

$$(R, \mathcal{P}, C),$$

not of R alone.

This is one of the most important interfaces between COMPINTEL and the Residual.

67.25 Recoding stability

If a claim is intended to be representation-independent, test it under admissible recodings.

A robust result should either:

- preserve its codelength ranking;
- or have an explicit explanation for why the decoder class makes the recoding operationally harder.

A signal that disappears under a harmless coordinate change is still evidence.

But it is coordinate-sensitive evidence.

It should not be promoted to an intrinsic property.

67.26 Probe accessibility is not mechanism

Suppose a probe decodes a variable Y from representation R .

Then Y is accessible to that probe.

That does not show that the original network uses Y .

It may encode the property redundantly.

The property may be a by-product.

The network may ignore it downstream.

Therefore

probe accessibility $\nrightarrow$ causal use.

This is where compression diagnostics must hand off to mechanistic intervention.

67.27 What mechanism evidence would add

Mechanistic evidence can include:

- ablating the compressed component;
- replacing it with a counterfactual value;
- patching it from another example;
- reconstructing the behavior from the proposed descriptor;
- testing whether destroying the descriptor selectively destroys the capability.

Now the claim changes.

Instead of saying:

the property is economically recoverable,

we can begin to say:

the property participates causally in the computation.

Compression helps locate candidates.

Intervention tests them.

67.28 Codelength and task distortion

Compression can also destroy what we care about.

Suppose a lossy compressed model becomes tiny but loses predictive accuracy.

The codelength improved.

The capability did not.

Therefore the relevant object is at least two-dimensional:

(codelength, retained behavior).

Depending on the setting, retained behavior can mean:

- accuracy;
- calibration;
- reward;
- reconstruction quality;
- safety property;
- transfer performance;
- retrieval fidelity.

The distortion axis cannot be omitted.

67.29 An evidence ladder

The chapter uses five increasingly demanding levels.

67.29.1 Level 0: training compression

Evidence of fit.

Compatible with memorization.

67.29.2 Level 1: held-out compression

Evidence of predictive structure for the declared target.

67.29.3 Level 2: control-separated held-out compression

Evidence that the signal exceeds random-label, shuffle, random-representation, and probe-capacity baselines.

67.29.4 Level 3: recoding-stable compression

Evidence that the signal is not merely a coordinate accident under the declared transformation class.

67.29.5 Level 4: cross-context reuse

Evidence that one descriptor or learned rule supports several tasks with low incremental description cost.

67.29.6 Level 5: mechanistic intervention

Evidence that the compressed structure is causally involved in the capability.

No level is a definition of general intelligence.

67.30 Intelligence is broader

An intelligent system may need:

- prediction;
- adaptation;
- planning;
- abstraction;
- transfer;
- action;
- uncertainty handling;
- search;
- resource allocation;
- coordination.

Compression can interact with many of these.

It need not characterize all of them.

A highly compressible constant sequence is not intelligent.

A capable agent may face genuinely stochastic data that cannot be strongly compressed.

Therefore the safest Atlas statement is:

compression can be a probe of learned regularity and reuse, not a scalar definition of intelligence.

67.31 A practical COMPINTEL protocol

For a model checkpoint, declare:

1. representation R ;
2. target Y ;
3. train/held-out split;
4. code C ;
5. probe class $\mathcal{P}$;
6. baseline;
7. side-information accounting.

Then measure

$$L_C(Y_H \mid R, T).$$

Compute

$$\Delta_C(R).$$

Then repeat with:

- random labels;
- shuffled representation-target alignment;
- random or untrained representation;
- probe-capacity sweep;
- at least one codec/probe variant.

Finally report retained task behavior.

If the claim is representation-independent, add admissible recoding tests.

If the claim is reuse, add a second task.

If the claim is mechanism, add intervention.

67.32 Longitudinal discovery protocol

For a learner evolving through checkpoints t :

1. freeze a held-out evaluation stream;
2. freeze target semantics;
3. freeze code accounting;
4. compute $L_{C,t}$ at each checkpoint;
5. compute Γ_t ;
6. inspect whether progress coincides with learned representations or behaviors;
7. rerun random/shuffle controls;
8. test transfer to future data or another task.

A sudden positive Γ_t is then a candidate discovery event.

The word candidate matters.

67.33 Research use: finding reusable computation

A practical use of COMPINTEL is comparative.

Suppose several layers, modules, heads, memories, or latent states all support the same downstream task.

Measure the conditional codelength of the task given each candidate representation.

Then ask:

- which candidate gives the shortest held-out code?
- which result survives recoding?
- which candidate supports several tasks?

- which one survives interventions?

This does not identify mechanism automatically.

It narrows the search.

That is already valuable.

67.34 Research use: compression before interpretation

Interpretability often begins with a human-chosen feature.

COMPINTEL suggests another route.

First locate representations that economically support many predictive targets.

Then inspect what common structure they encode.

Compression becomes a screening instrument.

Interpretation comes afterward.

This ordering can reduce the temptation to overinterpret visually appealing but non-predictive features.

67.35 Research use: scientific discovery systems

A scientific agent can treat unexplained data as a coding problem.

A candidate model that predicts new observations can reduce held-out description length.

That is a useful objective.

But a scientific claim also needs:

- falsifiability;
- intervention or experimental tests;
- model comparison;
- uncertainty;
- robustness;
- external replication.

Compression progress can therefore be one component of discovery machinery.

It is not the whole scientific method.

67.36 Source boundaries

Solomonoff supports an idealized connection between short generating descriptions and induction (Solomonoff 1964).

Blier and Ollivier support empirical deep-model codelength analysis and show that coding methodology matters (Blier and Ollivier 2018).

Voita and Titov support MDL as a representation-probing method and demonstrate the value of random baselines (Voita and Titov 2020).

Schmidhuber supports the proposed compression-progress principle for curiosity and discovery (Schmidhuber 2009).

None of these sources establishes:

$$\text{compression} = \text{intelligence}.$$

The Atlas does not add that equality.

67.37 What the computational witness establishes

The finite replay establishes exactly:

1. a fixed training prefix;
2. a prefix-free conditional period-two code;
3. one-bit held-out coding for the structured continuation;
4. three bits of gain relative to a four-bit literal baseline;
5. five-bit fallback for a matched noncontinuing suffix;
6. negative one-bit gain for that control;
7. one-bit coding of an all-zero eight-bit sequence under a special run code;
8. seven bits of compression gain for that trivial sequence.

It does not establish optimality of either codec.

It does not establish intelligence.

67.38 Non-implications

The chapter rejects:

$$\text{training compression} \not\Rightarrow \text{generalization},$$

$$\text{held-out compression} \not\Rightarrow \text{mechanistic use},$$

short description $\nRightarrow$ unique explanation,

compressibility $\nRightarrow$ intelligence,

compression progress $\nRightarrow$ scientific discovery,

probe accessibility $\nRightarrow$ causal necessity,

same task accuracy $\nRightarrow$ same codelength,

and

same codelength $\nRightarrow$ same behavior.

67.39 Atlas connections

Mechanistic diagnosis.

Compression identifies economically recoverable structure; intervention tests whether the structure matters causally.

The Residual.

Codelength can compare candidate portable descriptors but does not prove leastness in the factorization preorder.

Token and context compression.

A successful compressor may expose reusable predictive summaries, provided retained behavior is measured.

Adaptive curricula and search.

Compression progress can be used as an exploration signal when held-out and control semantics are explicit.

Research strategy.

A sudden codelength reduction can flag a hypothesis worth testing, not a result worth certifying.

67.40 Closing view

Compression is powerful because it forces explanation to pay rent.

A pattern that predicts new data can shorten a code.

A representation that exposes useful structure can reduce the data needed to learn a target.

A reusable abstraction can amortize its description across tasks.

These are real advantages.

They are not magic.

A code can be badly chosen.

A probe can be too strong.

A system can memorize.

A trivial sequence can compress beautifully.

A causal mechanism can remain hidden behind an accessible correlate.

The disciplined conclusion is:

compression is a probe of predictive structure when the code, split, controls, and retained behavior are explicit.

Discovery and intelligence require more.

67.41 References used in this chapter

- (Solomonoff 1964)
- (Blier and Ollivier 2018)
- (Voita and Titov 2020)
- (Schmidhuber 2009)

Exact source identities and claim boundaries are recorded in `sources/source-locks/ATLAS-CH-COMPINTEL-001.yaml`.

Chapter 68

From Models to Agents

Epistemic status: established representative mechanisms plus Atlas synthesis and derivation.

Specification: manuscript/specifications/ATLAS-CH-AGENTS-001.md

Formal packet: mathematics/derivations/ATLAS-CH-AGENTS-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-AGENTS-001.md

Source lock: sources/source-locks/ATLAS-CH-AGENTS-001.yaml

A model can answer a question without being able to change what question comes next.

That is the threshold this chapter cares about.

A language model presented with a fixed context computes a continuation. The continuation may contain a plan, a command, a proof sketch, a search query, or a vivid description of an action. But text that describes an action is not yet an action. Nothing outside the model changes merely because the model wrote a verb.

An agent appears when selected outputs are allowed to participate in a loop.

An observation enters. A model proposes. A controller chooses among admissible operations. An interface executes one of them. The environment or tool returns a consequence. Some part of that consequence is retained. The next decision is made in light of the changed state.

The important transition is therefore not from “less intelligent” to “more intelligent.”

It is from an open-loop mapping to a closed interaction architecture.

That distinction is modest enough to be useful.

It does not require us to decide whether a system has intentions, beliefs, or autonomy in a human sense. It asks a narrower systems question:

Can consequences of selected operations change the state from which later operations are chosen?

If yes, then we have acquired dynamics that cannot be understood by inspecting one model call in isolation.

68.1 The model is not the loop

The Adaptive-System Thesis established an explanatory boundary for the Atlas.

A learned model may be one component of a larger computational system that also contains memory, tools, interfaces, evidence, and governance.

Agents make that boundary concrete.

Consider a model

$$M(c) = y,$$

where c is supplied context and y is an output.

This object can be extraordinarily capable. It can summarize, prove, translate, classify, generate code, or propose a sequence of actions.

Yet the equation says nothing about whether any proposed action is executed.

It says nothing about who may execute it.

It says nothing about what the execution changes.

It says nothing about whether the result is observed.

It says nothing about whether that result changes the next model call.

Those missing arrows are where agent structure lives.

This chapter therefore uses “model” and “agent” as structural roles, not prestige labels.

A model can be a component of an agent.

The same model can also be used without an agent loop.

And a system can become more agentic in its interaction structure without changing model weights at all.

68.2 A minimal agent object

For the purposes of the Atlas, write an agent as

$$A = (M, C, X, O, \Gamma, U, B, \sigma).$$

The components are:

- M — a model or proposal operator;
- C — a controller that selects the next admissible operation;
- X — persistent internal state;
- O — an observation constructor;
- Γ — an execution interface connecting actions to an external state;
- U — an update operator for internal state;
- B — the current budget-and-authority contract;
- σ — a stopping rule.

This is not proposed as the one true definition of an agent.

It is a map legend for the mechanisms this and later chapters need to discuss.

At step t , let e_t denote external state, x_t internal state, and g the current goal.

The loop is:

$$\begin{aligned} o_t &= O(e_t), \\ p_t &= M(x_t, o_t, g), \\ a_t &= C(x_t, o_t, p_t; B_t), \\ (e_{t+1}, r_t) &= \Gamma(e_t, a_t), \\ x_{t+1} &= U(x_t, o_t, a_t, r_t). \end{aligned}$$

The budget and authority state are then updated, and σ decides whether another transition is permitted or required.

Every symbol matters.

The model proposal p_t need not be the executed action a_t .

The observation o_t need not expose the full environment state e_t .

The returned result r_t need not be trusted.

The update U may discard information, summarize it, corrupt it, or preserve it exactly.

The authority contract B_t may prohibit an operation even if the model strongly proposes it.

And the stop rule may terminate a competent loop or fail to terminate an incompetent one.

An agent is therefore not a model plus a longer prompt.

It is a model situated inside a transition system.

68.3 Why interaction changes the problem

The difference becomes sharp when useful information does not exist in the original context.

Suppose an environment contains one hidden bit s in $\{0,1\}$.

Before interaction, both possible worlds emit the same observation: UNKNOWN.

A deterministic one-shot model must therefore produce the same answer in both worlds.

It cannot guarantee correctness on both because the correct answers differ.

Now add one authorized tool: QUERY.

QUERY returns the hidden bit.

Nothing about the downstream answer rule has changed. The same model can simply report an observed bit if one exists.

But the system can now:

1. call QUERY;
2. receive the consequence;
3. store the returned bit;
4. invoke the answer rule on the changed state.

The exact witness in ATLAS-CW-AGENTS-001 checks both possible hidden states.

Without the query transition, the fixed answer rule succeeds on one of two states.

With one authorized query and budget one, it succeeds on two of two.

Remove the authority or remove the budget, and performance returns to one of two.

The point is not that querying is sophisticated.

The point is that capability can depend on the transition structure around a fixed model.

A system can know more at step $t+1$ because it did something at step t .

That sentence is the seed from which tool use, search, planning, memory, and coordination grow.

68.4 Tools are transition channels

A tool is often described by its name: calculator, browser, compiler, database, theorem prover, shell, simulator, calendar.

For systems analysis, the name is less important than the interface.

For a tool u with argument z , write

$$\Gamma_u(e, z) = (e', r).$$

The call may leave the external state unchanged and merely return information.

Or it may change the world.

A search query and a bank transfer are both “tool calls” in loose language. Their transition semantics and authority consequences are radically different.

This is why the Atlas separates three events:

the model proposes a call;

the controller determines whether the call is admissible;

the execution interface performs the call and returns a result.

Collapsing these steps creates a dangerous fiction in which generated text appears to carry its own authority.

It does not.

Authority comes from the execution boundary.

The ReAct architecture provides a clear representative mechanism for interleaving reasoning traces, task actions, and returned observations.

Toolformer provides another representative mechanism: a model can learn when to call particular APIs, what arguments to supply, and how to incorporate returned information into later prediction.

These are important constructions.

They should not be inflated into the claim that giving a model more tools necessarily makes the resulting system better.

A larger action set creates opportunities and failure modes at the same time.

68.5 Planning is computation before commitment

Not every proposed action should be executed immediately.

Planning inserts additional computation between the current state and external commitment.

Instead of following one candidate continuation, the system may construct several.

It may estimate consequences.

It may score alternatives.

It may prune some branches.

It may backtrack.

A generic search layer can be written as

$$H_{k+1} = \text{Select}(\text{Score}(\text{Expand}(H_k))),$$

where H_k is a set of candidate partial histories at search depth k .

Nothing in this notation requires the candidates to be natural language.

They could be symbolic states, trajectories, programs, proof states, schedules, or mixed representations.

For language models, Tree of Thoughts is a useful representative example because it makes the branching structure explicit. Intermediate text states are generated, evaluated, explored, and sometimes abandoned.

The important abstraction is not the word “thought.”

It is the separation between proposing one continuation and allocating computation over several possible continuations before acting.

Planning can improve a decision while still being wrong.

The generator can miss the useful branch.

The evaluator can rank candidates badly.

The search can prune the correct path.

The lookahead model can mispredict the environment.

More search is therefore not identical to more truth.

Planning changes how inference is allocated.

It does not repeal the need for evidence about the quality of the generator, evaluator, model of consequences, and stopping rule.

68.6 Reflection is state update, not self-transcendence

Agent systems are often said to “reflect.”

The word invites more mystery than necessary.

Suppose an attempt produces feedback f_t .

A reflective mechanism computes an update

$$x_{t+1} = U_reflect(x_t, f_t).$$

The next attempt is conditioned on x_{t+1} rather than x_t .

That is enough structure to study.

Reflexion is a representative mechanism of this type. Feedback is converted into verbal material retained in episodic memory and used in subsequent trials.

The empirical question is whether the update improves later decisions.

The system may reflect accurately.

It may also construct a persuasive explanation of the wrong failure, preserve it, and become more consistently wrong.

Fluency does not settle that question.

A reflection should therefore be treated as a candidate state update whose downstream effect can be measured, ablated, substituted, or contradicted.

This returns us to one of the Atlas's recurring themes:

description is weaker than intervention.

If reflection is claimed to matter, compare the system with and without the reflective state, or with controlled alternatives, while preserving the rest of the loop as closely as possible.

68.7 Decomposition requires interfaces

Large goals are often decomposed into smaller tasks.

That sounds simple until two subtasks disagree about what passes between them.

Let a task decomposition be a directed graph

$$G_T = (V, E).$$

Each node v has at least:

- a local goal g_v ;
- admissible inputs I_v ;
- a required return contract Q_v ;
- a local resource budget R_v ;
- a stop condition σ_v .

An edge $u \rightarrow v$ means that v may consume some declared output or state produced by u .

A list is not yet a robust decomposition.

A decomposition becomes operational when the interfaces between pieces are explicit enough that a downstream node knows what it may assume.

This is the same compositional pressure seen elsewhere in the Atlas.

Tensor shapes do not guarantee semantic compatibility.

Subtask completion labels do not guarantee that a returned object means what the next task needs.

The more independent the subtasks become, the more important their boundaries become.

68.8 Delegation is decomposition plus authority

Delegation adds another ingredient.

A child process is not merely asked to compute something.

It is given some capacity to act.

Represent a delegation packet as

$$D = (g', A', R', Q', \sigma').$$

Here g' is the bounded child goal, A' the authority or tool set granted to the child, R' the resource budget, Q' the required return object, and σ' the child stop rule.

Suppose the parent currently has authority set A and resource budget R .

The default Atlas rule is:

$$A' \subseteq A$$

and

$$R' \leq R,$$

with the resource inequality interpreted componentwise when R is a resource vector, unless a distinct external authority explicitly grants an enlargement.

This is not a complete security theorem.

It is a clean invariant.

Spawning a child does not create permission out of nothing.

If every delegation edge respects authority inclusion, then authority cannot expand merely by descending the delegation tree. Set inclusion is transitive.

If every parent allocates child budgets from its own finite budget, the total resource assigned to descendants can likewise be bounded by the root allocation, provided the accounting boundary is actually enforced.

This is what “bounded delegation” means in this chapter.

Not timid delegation.

Not necessarily shallow delegation.

Delegation whose authority, resources, return contract, and stopping condition remain explicit.

68.9 The return is not the truth

A delegated child returns something.

Perhaps it is an answer, a proof, a file, a tool result, a plan, a test report, a witness, or a failure record.

The parent now faces an epistemic problem as well as a control problem.

What is this object allowed to establish?

AGENTS-001 does not solve that question fully. Later chapters on coordination and evidence exchange will.

But one rule belongs here:

child completion is not automatic parent belief.

The return Q' is an input to the parent’s next state transition.

It may be checked, replayed, challenged, combined with other evidence, rejected, or escalated.

This matters even when the child and parent use the same underlying model.

A second invocation is not independent verification merely because it happened later.

68.10 Authority and capability are different coordinates

An agent may be capable of proposing an operation it is not authorized to perform.

It may be authorized to perform an operation it cannot use competently.

These are different axes.

Let $\text{Cap}(x)$ denote the set of operations the system can successfully formulate or execute from state x under ideal access.

Let $\text{Auth}(x)$ denote the operations the current contract permits.

The executable set is constrained by their intersection, together with the actual interfaces available.

A safe system design cannot infer authority from apparent competence.

Nor should it infer competence from possession of credentials.

This distinction becomes increasingly important as agent systems acquire access to software environments, communication channels, financial operations, scientific instruments, or other consequential interfaces.

The model says what it proposes.

The authority boundary says what may happen.

68.11 Budgets turn “keep trying” into mathematics

An unbounded instruction such as “continue until solved” hides several decisions.

How many model calls?

How many tool calls?

How much wall-clock time?

How many child tasks?

How deep may recursion go?

How much money, energy, memory, or external rate limit may be consumed?

Budgets make these questions explicit.

Let R_t be a vector rather than a scalar if resources are heterogeneous.

For example,

$R_t = (\text{tokens}, \text{tool_calls}, \text{wall_time}, \text{money}, \text{child_slots})$.

Each transition incurs a nonnegative cost vector c_t .

The update is

$R_{t+1} = R_t - c_t$,

subject to component-wise nonnegativity.

A stop rule can then require termination before a forbidden negative resource state is reached.

This does not guarantee useful behavior.

It converts one vague control surface into an inspectable one.

68.12 Termination is part of the agent

A loop without a meaningful stop condition is not merely inefficient.

It is incompletely specified.

Stopping may depend on goal satisfaction, proof of impossibility, resource exhaustion, repeated state, lack of progress, external cancellation, confidence or risk threshold, authority boundary, or human review.

A good stop rule can be difficult to design because the system may not know whether one more step would succeed.

But omitting the problem does not remove it.

It merely replaces an explicit stopping policy with whatever accidental limit the runtime, user, or infrastructure imposes.

For recursive delegation, termination becomes structural.

A finite, fixed acyclic task graph terminates when each activated node terminates and execution does not create new nodes outside that graph.

Cyclic or dynamically expanding delegation requires an additional argument: decreasing resource, decreasing measure, explicit depth bound, or some other well-founded condition.

“Try again” is not a termination proof.

68.13 Where agent loops fail

The formal decomposition lets us locate failures more precisely.

68.13.1 Observation failure

The useful distinction in the environment never reaches the agent.

Two states that require different actions may look identical through O.

68.13.2 Proposal failure

The necessary information is present, but M proposes a poor continuation.

68.13.3 Controller failure

A sound proposal exists, but C selects a worse operation or admits an operation that should be blocked.

68.13.4 Interface failure

The tool or environment does not behave according to the semantics the agent assumes.

A returned result may be stale, malformed, adversarial, or simply wrong.

68.13.5 State-update failure

Useful evidence arrives and is then summarized away, overwritten, misindexed, or attached to the wrong object.

68.13.6 Planning failure

Search explores the wrong branches, scores them badly, or spends its budget before reaching a useful continuation.

68.13.7 Reflection failure

The system explains its failure incorrectly and makes the mistaken explanation persistent.

68.13.8 Delegation failure

The child goal is underspecified, authority is too broad, the return contract is ambiguous, or recursion is allowed to expand without a decreasing resource.

68.13.9 Termination failure

The system stops before the necessary evidence arrives or continues after further actions have become wasteful or unsafe.

Calling all of these “model errors” would erase the architecture we need in order to repair them.

68.14 Autonomy is not one number

People often ask whether a system is autonomous as though autonomy were a scalar property.

For engineering purposes, several coordinates matter separately.

How long can the loop continue without external intervention?

Which actions may it execute?

How much state persists?

Which environments can it observe?

Can it create child tasks?

Can it alter its own plan?

Can it spend resources?

Can it change the authority of descendants?

What events force review or termination?

Two systems using the same model may therefore have radically different autonomy surfaces.

One may be allowed to search a read-only corpus for three steps.

Another may execute code, modify files, communicate externally, spawn children, and continue for hours.

Calling both “agents” does not make those differences disappear.

The useful question is not merely whether an agent exists.

It is what transition system has actually been authorized.

68.15 What the representative mechanisms establish

The sources used in this chapter demonstrate several concrete mechanisms.

ReAct demonstrates that reasoning traces and environment-facing actions can be interleaved, with observations feeding later steps.

Toolformer demonstrates a learned mechanism for deciding when and how to call external APIs and incorporate their returns.

Tree of Thoughts demonstrates explicit branching, evaluation, lookahead, and backtracking over intermediate language-model states.

Reflexion demonstrates a feedback pathway in which verbalized reflections are retained and influence later episodes.

These results justify discussing such mechanisms as concrete engineering objects.

They do not prove that every capable agent must use them, every task benefits from them, self-evaluation is reliable, tool use is safe, search always improves performance, reflection always corrects errors, or more autonomy is better.

The Atlas needs mechanisms without mythology.

68.16 From one loop to many

A single agent already contains several interacting states: model context, persistent memory, external environment, tool state, resource state, and authority state.

The next chapter introduces a different difficulty.

What happens when several such loops share information or act on common resources?

Then we acquire communication, shared state, concurrency, race conditions, rendezvous, transactions, provenance across actors, partial failure, disagreement, and coordination costs.

Those are not decorations on the single-agent loop.

They form a new systems layer.

ATLAS-CH-COORD-001 may therefore assume the agent object

$A = (M, C, X, O, \Gamma, U, B, \sigma)$,

together with typed actions, persistent state, bounded delegation, budget state, authority state, and stop rules.

It must add the mathematics and systems semantics of coordination rather than pretending that a collection of agents is just one larger prompt.

68.17 The boundary to remember

The chapter began with a small distinction.

A model can produce text that describes an action.

An agent loop can allow a selected action to change the state from which later computation proceeds.

Everything else in this chapter elaborates that closure.

Tools enlarge the transition relation.

Planning spends computation before commitment.

Reflection changes persistent state.

Decomposition creates explicit subproblems.

Delegation grants bounded capacity to act on them.

Budgets limit continuation.

Authority limits which transitions may occur.

Stopping rules close the process.

None of these mechanisms guarantees intelligence.

None guarantees truth.

None guarantees safety.

But once they are explicit, they become objects that can be analyzed rather than properties vaguely attributed to “the model.”

That is the move from models to agents that the Atlas needs.

68.18 References used in this chapter

External mechanism sources are pinned in:

`sources/source-locks/ATLAS-CH-AGENTS-001.yaml`

They include:

- ReAct: Synergizing Reasoning and Acting in Language Models, ICLR 2023, arXiv:2210.03629;
- Toolformer: Language Models Can Teach Themselves to Use Tools, NeurIPS 2023, DOI 10.52202/075280-2997;
- Tree of Thoughts: Deliberate Problem Solving with Large Language Models, NeurIPS 2023, arXiv:2305.10601;
- Reflexion: Language Agents with Verbal Reinforcement Learning, NeurIPS 2023, DOI 10.52202/075280-0377.

The Atlas formal loop, budget-and-authority contract, and delegation invariants are chapter-owned synthesis and derivation.

Chapter 69

Coordination Architectures

Epistemic status: established coordination mechanisms plus Atlas synthesis and exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-COORD-001.md

Formal packet: mathematics/derivations/ATLAS-CH-COORD-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-COORD-001.md

Source lock: sources/source-locks/ATLAS-CH-COORD-001.yaml

One agent can fail because it chose badly.

Two agents can fail even when each chooses correctly.

That is the new phenomenon.

Once several bounded interaction loops communicate or act on shared resources, correctness is no longer only a property of each local decision. It depends on what each actor can see, when events become visible, which operations may interleave, whether messages can repeat, what state is shared, and which effects are atomic.

Coordination is the mathematics and systems discipline of those relationships.

It begins where the previous chapter ended.

From Models to Agents gave us a single interaction loop with state, tools, authority, budgets, and a stop rule. We now place several such loops in one world.

The result is not simply a larger prompt.

It is a concurrent system.

69.1 From one loop to many

Let actor i carry an audited single-agent object A_i .

Each actor can observe, propose, act, update state, spend resources, and stop.

Now suppose actor P produces an output needed by actor Q .

Several questions appear immediately.

How does Q learn that the output exists?

Can Q see a partial result?

What if P changes it while Q is reading?

What if the message is delivered twice?

What if Q acts before an earlier causal dependency arrives?

What if both actors modify the same state?

What if one fails halfway through a multi-step effect?

What if both wait for one another?

None of these questions is answered by making either model more capable.

They belong to the coordination layer.

For the Atlas, write a coordination system as

$C = (I, \{A_i\}, \{X_i\}, S, K, \prec, T, F)$.

Here:

I indexes the participating actors;

A_i is actor i 's single-agent loop;

X_i is actor-local state;

S is shared coordination state or a shared substrate;

K is the set of communication and coordination operations;

$\prec$ is the causal or application-relevant event-order relation;

T is the declared transaction and atomicity semantics;

F is the failure, retry, timeout, redelivery, and deduplication policy.

This tuple is not the definition of every distributed system.

It is the minimum map legend needed for the architectures in this chapter.

69.2 Communication is not agreement

Suppose P sends Q a message:

“candidate proof complete.”

Several facts may follow, depending on the system.

The message may have been emitted.

It may have entered a queue.

It may have been delivered.

Q may have read it.

Q may have parsed it.

Q may have accepted its meaning.

Q may have changed state because of it.

The proof may even be correct.

These are different events.

A communication channel transports an object or signal.

It does not confer epistemic authority on the content.

Nor does successful delivery imply that sender and receiver now agree.

This distinction matters for multi-agent systems because a transcript can look coordinated while the actors are operating with incompatible state.

The message is part of the coordination story.

Its truth belongs to an evidence story that comes later.

69.3 Local state and shared state

Every actor has some local state X_i .

A coordination architecture may also expose shared state S .

Shared state is attractive because it creates a common surface.

Write once, and many actors may observe.

But “shared” does not mean “simultaneously identical in every observer.”

A system may cache.

It may replicate.

It may materialize views at different times.

It may delay propagation.

It may expose snapshots.

It may permit concurrent writers.

The phrase “shared memory” therefore hides a second question:

what consistency semantics govern observation of that memory?

Two agents can both have access to the same named store while seeing different versions of it.

Coordination requires the visibility rule, not merely the storage location.

69.4 Six coordination families

The architectures below solve different coordination problems.

They should not be arranged on a single ladder from primitive to advanced.

69.4.1 Direct message passing

The simplest picture is an addressed channel.

P sends m to Q.

Q receives m .

This architecture makes actor identity and directed communication explicit.

It can keep local state local.

But it immediately raises delivery questions:

Is the channel reliable?

Can messages be reordered?

Can duplicates occur?

Does a send block?

Can the receiver refuse?

Does one sender’s order survive mixing with messages from others?

The Actor tradition is one important lineage for thinking in terms of independent computational actors and messages.

The useful abstraction is not that every modern agent system is an Actor system.

It is that independently evolving components can coordinate through explicit messages rather than through one shared call stack.

69.4.2 Rendezvous

Sometimes the communication event should happen only when both sides participate.

A rendezvous couples progress.

A sender reaches a communication point.

A receiver reaches a compatible point.

The communication transition occurs when the two synchronize.

Communicating Sequential Processes made this kind of communication a first-class structuring idea for concurrent programs.

Rendezvous can simplify some ordering questions because synchronization is explicit.

It can also create blocking and deadlock surfaces.

Coordination is always a trade: making one relation explicit moves difficulty somewhere else.

69.4.3 Blackboard coordination

A blackboard replaces pairwise addressing with shared visible problem state.

One component posts a partial hypothesis.

Another notices it and adds a refinement.

A third may detect a conflict.

The blackboard is not merely storage.

It is a coordination surface around which heterogeneous problem-solving processes organize their activity.

H. Penny Nii's blackboard account is useful because it separates the blackboard model of problem solving from the many concrete systems built around it.

For agent systems, the same lesson remains valuable.

A shared workspace can decouple contributors from direct knowledge of one another.

But the blackboard does not decide which contribution is correct.

And once several writers can modify the same state, scheduling and consistency become unavoidable.

69.4.4 Tuple spaces

Linda sharpens shared coordination into a small algebra.

The shared object is a tuple space: a logically shared multiset of tuples.

A process may perform operations such as:

out(t): add tuple t;

rd(p): read a tuple matching pattern p without removing it;

in(p): take a matching tuple and remove it.

The important word is matching.

A consumer can coordinate by describing the shape of the object it needs rather than naming the producer that must send it.

This changes the coupling structure.

Producer and consumer can be separated in space, time, and identity.

The tuple space becomes the rendezvous surface.

The destructive in operation adds another feature: possession can become exclusive if the take operation is atomic.

A work token can be placed once and consumed by one worker.

That is coordination through state semantics rather than addressed messaging.

Again, nothing about a tuple's presence makes its content true.

The space coordinates availability, not epistemic status.

69.4.5 Event-driven coordination

Another architecture treats history as the primary object.

Actors append events.

Other actors subscribe, replay, materialize, or derive later state.

This design has a powerful consequence:

the path by which state was obtained can remain visible.

But event-driven systems acquire their own obligations.

What is the event identity?

Can delivery repeat?

Which order is authoritative?

Can consumers replay from a checkpoint?

Are materialized views deterministic?

What happens when old events are reinterpreted by new code?

An event log can make provenance easier to preserve while making state reconstruction an explicit part of the architecture.

The Atlas uses the public AETHER project as one bounded GCL example of this direction.

At its pinned public revision, AETHER describes an append-only causal journal, deterministic resolution over fixed journal prefixes, recursive rule execution, and derivation provenance.

That is exact project evidence.

It is not evidence that every coordination system should look like AETHER.

69.4.6 Transactions

Transactions answer a different question.

Which set of effects should appear as one unit?

Suppose an operation reads a value, computes a new value, and writes it back.

If another actor can interleave its own read and write halfway through, the two locally correct procedures may compose incorrectly.

A transaction boundary can prevent some harmful interleavings by grouping operations under declared atomicity and isolation semantics.

Jim Gray's transaction work is a foundational reference point for this view.

But "transaction" is not a magic word.

A transaction must have a boundary.

Database writes may be inside it while network calls, emails, actuator effects, or external APIs remain outside.

Atomicity is only as broad as the set of effects the transaction system actually controls.

69.5 Causality before total order

When several actors run concurrently, it is tempting to ask:

What happened first?

Sometimes the question has no application-level answer.

Leslie Lamport's happens-before relation gives a disciplined way to say what the system actually knows.

Write $a \prec b$ for the strict happens-before relation generated by three rules:

1. if a and b occur in one actor and local program order places a before b , then $a \prec b$;
2. if a is the send of a delivered message and b is the matching receive, then $a \prec b$;
3. if $a \prec b$ and $b \prec c$, then $a \prec c$.

This gives the causal partial order used in the chapter.

But two independent events may be incomparable.

Neither happened “before” the other in the causal sense relevant to the computation.

A runtime can still assign them timestamps and impose a total order.

That can be useful.

It is also extra structure.

A total order can serialize events that the application did not need to order.

The distinction matters because unnecessary ordering can create coordination cost.

If two events are genuinely independent, forcing global agreement about their order may buy no semantic value.

69.6 A small race with a complete answer

The easiest concurrency bug is already enough to justify the chapter.

Let shared state begin at

$x = 0$.

Actors P and Q both intend to increment x once.

Each performs:

read x ;

write the value it read plus one.

Individually, each program is correct.

If P runs completely and then Q runs completely, the final value is 2.

If Q runs completely and then P runs completely, the final value is also 2.

But concurrency introduces four more legal schedules.

The computational witness enumerates all six interleavings that preserve each actor’s local read-before-write order.

Only two finish at 2.

Four finish at 1.

Both actors “succeeded” locally.

The shared effect is wrong.

This is the lost-update problem in its smallest useful form.

The lesson is not merely that concurrency is dangerous.

It is more precise:

local correctness plus shared state does not imply compositional correctness under interleaving.

69.7 Atomicity changes the schedule space

Now replace each two-step increment with one atomic operation:

INC: $x \leftarrow x + 1$.

There are still two possible actor orders.

P then Q.

Q then P.

Both produce 2.

The arithmetic did not change.

The allowed interleavings changed.

That is what atomicity buys in this example.

It removes intermediate states from the schedule space visible to competing actors.

This is a recurring systems theme.

Many coordination mechanisms do not improve the intelligence of any participant.

They change which global executions are possible.

69.8 Delivery is not effect

Messages introduce a related distinction.

Suppose a worker receives:

charge account by 10.

If the transport provides at-least-once delivery, the same logical message may arrive twice.

If the worker blindly applies the command twice, the effect is 20.

The transport may be behaving exactly as designed.

The application is still wrong.

One repair is idempotence.

For state transition E and message m , idempotence means:

$$E(E(s,m),m) = E(s,m).$$

Applying the same logical operation twice has the same effect as applying it once.

Another repair is deduplication.

Give the logical operation a stable identity.

Record that identity durably when its effect commits.

Ignore a later delivery carrying an already-committed identity.

This sounds like exactly-once processing.

It is only as strong as its assumptions.

If the deduplication record commits but an external side effect does not, or the external side effect occurs but the record does not, ambiguity returns.

Exactly-once effect is therefore not obtained merely by writing “exactly once” on a queue.

It is an end-to-end claim about identity, atomicity, persistence, and side effects.

69.9 Retry is a semantic operation

Retries are often treated as infrastructure.

They are part of the program.

Suppose an actor sends an operation and sees no reply.

Three states are compatible with that observation:

the operation never arrived;

the operation arrived and failed;

the operation succeeded but the reply was lost.

Retrying is correct in the first case.

It may be correct in the second.

It can duplicate an effect in the third.

A retry policy *F* must therefore be read together with the operation semantics.

For a pure read, retry may be harmless.

For a non-idempotent side effect, retry may require an operation identity, transaction boundary, compensating action, or human review.

The absence of a reply is not the same thing as evidence that nothing happened.

69.10 Shared state is not authority

Blackboards, tuple spaces, event logs, databases, and shared memories can all become central surfaces.

That centrality creates a temptation.

If a fact is in the shared store, treat it as true.

The previous evidence chapter already taught us why that fails.

A coordination substrate answers questions such as:

Who can see this object?

Who can consume it?

Who can overwrite it?

In what order did related events occur?

Has this operation already been applied?

Those are important facts.

They do not establish the semantic truth of the payload.

A bad hypothesis can be perfectly replicated.

A false tuple can be atomically committed.

An incorrect event can have impeccable provenance.

Coordination and evidence meet later.

They must first remain distinct.

69.11 Deadlock and livelock

Synchronization can fail even when no data is corrupted.

Consider two actors.

P holds resource a and waits for b.

Q holds b and waits for a.

No actor is locally broken.

The system stops making progress.

That is deadlock.

Livelock is different.

Actors continue changing state but repeatedly interfere so that useful progress does not occur.

A pair of overly polite processes can continually yield to one another.

An agentic analogue is a pair of workers that repeatedly reassign a disputed task without resolving it.

Progress therefore needs its own semantics.

“Nothing crashed” is weaker than “the system advanced.”

69.12 Partial failure changes reasoning

In a local sequential program, a function call usually returns, raises, or the process fails.

Distributed coordination adds ambiguous partial outcomes.

One actor can remain healthy while another fails.

A network can partition.

A shared service can be unreachable from one participant and reachable from another.

A timeout can mean failure, delay, overload, or a lost response after success.

This makes absence difficult to interpret.

Silence is not a single state.

The coordination architecture must decide what evidence about failure is available and what actions are justified under uncertainty.

This is one reason distributed protocols become more elaborate than their local analogues.

They are not merely moving bytes.

They are managing incomplete knowledge about a changing joint state.

69.13 Coordination cost is real

Coordination can improve correctness and enlarge what a group can accomplish.

It also costs something.

Messages consume bandwidth and serialization work.

Synchronization introduces waiting.

Transactions constrain concurrency.

Retries consume repeated work.

Provenance and deduplication require metadata.

Global ordering can create contention.

A useful bookkeeping object is the coordination-cost vector

$c_coord = (c_comm, c_sync, c_retry, c_meta)$,

whose components track communication, synchronization, retry/recovery, and coordination-metadata costs.

The components need not share one physical unit and therefore need not be collapsed into a scalar.

The vector is a reminder that coordination is not free structure.

This leads to a design question that will recur throughout the Atlas:

What is the weakest coordination semantics that still preserve the claim or invariant we care about?

If two computations are independent, do not synchronize them merely because synchronization is available.

If a task requires exclusive consumption, a destructive tuple-space take may be more natural than global locking.

If a shared decision requires a transaction, pretending eventual convergence is sufficient may be dangerous.

Architecture begins by identifying the invariant.

69.14 The AETHER example

AETHER is useful here because it makes one particular coordination choice explicit.

At the pinned public revision, its center of gravity is an authoritative semantic kernel built around an append-only causal journal, deterministic state resolution, recursive rule evaluation, and provenance/explanation.

Its interface record separates journal, resolver, rules, runtime, and explanation roles.

It also states an invariant that fixed journal prefix plus compiled program yields deterministic results.

This is exactly the sort of claim that belongs in a coordination chapter because it exposes the chosen shared-state semantics.

AETHER does not replace the general theory.

It is one design point.

Compared with direct message passing, it makes shared history more central.

Compared with a bare blackboard, it makes derivation and replay more explicit.

Compared with a simple queue, it treats the semantic state reconstructed from history as a first-class object.

Those are architectural choices with costs and benefits.

The Atlas records them as public project evidence, not as a universal optimum.

69.15 What coordination does not solve

A coordination system can ensure that one worker takes a task token.

It cannot ensure the worker solves the task correctly.

It can serialize two writes.

It cannot ensure the serialized value is scientifically valid.

It can deliver every message.

It cannot ensure the actors interpret the messages the same way.

It can replay every event.

It cannot ensure the original event was truthful.

It can give every actor the same state.

It cannot ensure the common state encodes the right world model.

This boundary is central.

Coordination can preserve relationships among computational events.

It cannot replace the semantics of the work those events carry.

69.16 What the next chapters may assume

Two direct architectural consumers depend on this chapter.

ATLAS-CH-EVIDEX-001 — Evidence Exchange and Zero-Context Work may assume stable actor/event identity, local/shared-state separation, channel semantics, causal order, retry and deduplication policy, and transaction boundaries.

Its task is to add evidence semantics:

what a returned object establishes;

how provenance follows a handoff;

how an independent actor receives enough context to reproduce or challenge a claim.

ATLAS-CH-POLITY-001 — The Computational Polity may assume the full coordination object

$$C = (I, \{A_i\}, \{X_i\}, S, K, \prec, T, F),$$

together with the architecture families and failure surfaces developed here.

Its task is larger:

compose models, agents, memory, tools, humans, validators, and governance into a coherent computational polity.

Neither chapter needs to rediscover what a race condition or duplicate effect is.

That is the point of dependency.

69.17 The boundary to remember

The simplest distributed-system mistake is to believe that because every component is correct, the system is correct.

The lost-update witness shows otherwise.

Correct local steps can compose into a wrong global state when the coordination semantics are underspecified.

The reverse lesson is equally important.

Coordination machinery should not be worshipped for its own sake.

Messages, blackboards, tuple spaces, event logs, rendezvous, clocks, and transactions are ways of constraining or exposing possible interactions.

They are useful when those constraints match an invariant the larger system actually needs.

The governing question is therefore not:

Which coordination architecture is best?

It is:

Which relationships among actors, events, state, and effects must remain true, even when execution is concurrent and failure is partial?

Once that question is explicit, coordination stops being plumbing.

It becomes part of the mathematics of adaptive systems.

69.18 References used in this chapter

The source lock pins the mechanism references and Atlas project objects used by this draft:

`sources/source-locks/ATLAS-CH-COORD-001.yaml`

External mechanism sources include the 1973 Actor formalism, Hoare's Communicating Sequential Processes, Carriero and Gelernter's Linda in Context, Nii's blackboard-system account, Lamport's event-ordering paper, and Gray's transaction paper.

The AETHER material is exact GCL public project evidence at commit `74b2e322a4453f1665a076bc0682adcd0c5cfb44` and is used only as a bounded coordination case study.

Chapter 70

Evidence Exchange and Zero-Context Work

Epistemic status: provenance standards plus Atlas synthesis, bounded GCL project evidence, and exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-EVIDEX-001.md

Formal packet: mathematics/derivations/ATLAS-CH-EVIDEX-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-EVIDEX-001.md

Source lock: sources/source-locks/ATLAS-CH-EVIDEX-001.yaml

A result that is useful to its author can still be useless to everyone else.

The missing ingredient is often not intelligence.

It is transfer.

A proof sketch may depend on a source version that was never named.

An experiment may depend on a hidden environment choice.

A contributor may say “this works” without stating the exact claim, the domain on which it was tested, or the assumptions it used.

A second actor can receive the same words and reconstruct a different task.

At that point the problem is no longer only epistemic and no longer only coordinative.

It is an evidence-exchange problem.

The Atlas needs a way to move candidate evidence between actors while preserving enough identity, scope, provenance, and replay information that a later reader can determine what was actually done.

That is the subject of this chapter.

70.1 Evidence must survive handoff

The Evidence chapter gave us a claim-support packet

$K = (q, \tau, S, \Omega, N, D)$.

The Coordination chapter gave us actors, channels, event identities, causal order, retries, and transaction boundaries.

Neither object by itself tells us how to package a bounded scientific or mathematical task for another actor.

That requires an interface.

The interface has two directions.

A dispatch says:

this is the exact obligation;

these are the facts you may import;

these are the sources and tools you may use;

these are the failure conditions;

this is the return grammar;

this is where the result must go.

A return says:

this is the dispatch I answered;

this is my strongest exact conclusion;

this is my support;

this is my provenance;

this is how to test or falsify it;

this is what remains unresolved.

Evidence exchange begins when both directions are explicit.

70.2 The dispatch packet

Write a dispatch as

$D = (\delta, Q, B, \Sigma, C, \Lambda, \Gamma, \rho)$.

Here δ is a stable dispatch identity.

Q is the exact bounded obligation.

B is the bootstrap: definitions, protected facts, and immutable imported-source identities.

Sigma is the allowed source and tool policy.

C is the context and independence class.

Lambda is the set of resource bounds, rejection conditions, and legitimate stop conditions.

Gamma is the exact return grammar.

rho is the durable return route.

This object is intentionally stricter than an ordinary instruction.

An ordinary instruction can rely on shared context.

A dispatch intended for independent work should not.

70.3 What zero-context actually means

“Zero context” can be misunderstood.

It does not mean the worker knows no mathematics.

It does not mean the worker has never seen the topic.

It does not mean the worker is statistically independent from every other actor.

It means the task itself should not depend on hidden conversational state.

Fix a declared class of eligible actors A.

Let H be hidden session history.

Zero-context sufficiency constrains the **authorized task contract**, not the worker’s psychology.

Write

`Contract_A(D,H)`

for the obligation, premises, permissions, source policy, stop conditions, and return requirements that the protocol authorizes when D is accompanied by hidden history H.

Then require

`Contract_A(D,H1)`

=

`Contract_A(D,H2)`

for admissible hidden histories H1 and H2.

The exact obligation must be the same.

The imported facts must be the same.

The permitted sources must be the same.

The return grammar must be the same.

The stop conditions must be the same.

A worker can bring background knowledge and can still misunderstand an instruction.

What it cannot be authorized to obtain from hidden history is the missing half of the assignment.

70.4 Hidden context and explicit prerequisites

There is nothing wrong with prerequisites.

Science depends on them.

The problem is invisible prerequisites.

Compare two instructions.

First:

“Use the convergence theorem we discussed yesterday.”

Second:

“Use Theorem T from source S at immutable version h, under hypotheses H1 through H4.”

The second instruction can still be wrong.

It can name the wrong theorem.

Its hypotheses may not match.

But the dependency is inspectable.

The first instruction delegates part of the task definition to shared memory.

That works until the worker changes, the session ends, the conversation is unavailable, or two participants remember different versions.

Zero-context work therefore does not eliminate context.

It compiles relevant context into an explicit packet.

70.5 A dispatch is a boundary object

A good dispatch has to satisfy two constituencies.

The sender needs confidence that the worker is solving the intended problem.

The worker needs enough information to know what is allowed and what a valid return looks like.

This is why a dispatch should include excluded assumptions as well as imported facts.

Suppose a source proves a theorem for finite groups.

The target problem concerns compact groups.

If the dispatch lists only the finite-group theorem and says “generalize it,” an actor may quietly assume the missing extension.

A better dispatch says:

finite-group theorem available;

compact-group extension unavailable;

do not import it.

The exclusion is not negative prose.

It defines the search space.

70.6 Source policy is part of the mathematics

Two actors can receive the same mathematical question and still be solving different assignments if one is allowed to search external literature and the other is restricted to a protected packet.

This is why Sigma belongs in the dispatch.

Possible source policies include:

protected packet only;

primary sources allowed;

public sources allowed;

repository-local sources only;

no external source use;

tools allowed but internet disallowed.

These are not administrative details.

They change what evidence can be introduced.

A zero-context dispatch must therefore tell the worker not only what the problem is, but what evidence universe it is allowed to draw from.

70.7 Context class is not a certificate of independence

Suppose a dispatch says

`C = independent_blind`.

That can mean the worker is not shown selected prior returns.

This may reduce anchoring or copying.

It may improve adversarial value.

But the label is weaker than the word independent sometimes suggests.

Two workers can use the same foundation model.

They can share training data.

They can share organizational assumptions.

They can unknowingly reproduce the same error.

They can even be the same model in separate sessions.

So an independence label should be read as an information-flow constraint.

What information was withheld?

What information remained available?

What provenance does the worker actually have?

That is more useful than pretending one adjective proves methodological independence.

70.8 The return object

Now write a return as

`R = (delta, chi, K, Pi, V, Delta)`.

The first field, `delta`, links the return to the dispatch.

`chi` is the declared disposition.

`K` is the claim-support packet from the Evidence chapter.

`Pi` is provenance.

`V` contains verification and falsification hooks.

Delta is the unresolved residual or next bounded obligation.

This structure matters because a return is not merely an answer.

It is a candidate evidence object.

A phrase such as

PROVED

or

COUNTEREXAMPLE

inside chi does not make itself true.

The disposition tells the reviewer what the contributor claims to have returned.

Review remains a separate operation.

70.9 Provenance asks how an object came to exist

The W3C PROV family gives us a useful conceptual separation.

There are entities.

There are activities.

There are agents.

Entities can be generated by activities.

Activities can use entities.

Agents can bear responsibility for activities or generated objects.

Entities can derive from other entities.

This is exactly the kind of separation evidence exchange needs.

A return can therefore carry provenance like:

which input objects were used;

which actor or execution identity produced the return;

which procedure transformed the inputs;

which output entity was generated;

which derivation links connect the result to its sources.

The Atlas does not need to reproduce the entire PROV specification to benefit from the distinction.

The key lesson is that payload and lineage are different objects.

70.10 Provenance is not truth

A false result can have excellent provenance.

Imagine an arithmetic script that reads the exact intended file, runs under the exact pinned interpreter, records the exact command, and deterministically prints the wrong number because the code contains a bug.

Every provenance field can be correct.

The scientific claim can still fail.

This is an important boundary.

Provenance lets us ask:

where did this result come from?

It does not answer:

is this result true?

The second question requires verification, proof, replication, adjudication, or some other evidence operation.

Good provenance makes those operations easier.

It does not replace them.

70.11 Identity has layers

Evidence exchange often uses identifiers.

Not all identifiers answer the same question.

A filename answers:

what human-facing path or label was used?

A commit-and-path pair answers:

which repository version and file path?

A content hash answers:

which bytes?

A DOI may answer:

which scholarly object or version?

A bibliographic record answers:

which published source is intended?

These identities can reinforce one another.

They can also diverge.

Two files can have the same name and different bytes.

Two archives can contain identical bytes under different bibliographic descriptions.

A hash can identify bytes perfectly while saying nothing about what those bytes are supposed to mean.

So source locking should be designed around the failure mode.

If version drift matters, identify the version.

If bibliographic confusion matters, identify the work.

If byte identity matters, fingerprint the bytes.

70.12 A tiny ambiguity with an exact answer

The chapter's witness uses three source versions, all called `inputs.txt`.

Version one contains:

`a=2`

`b=3.`

Version two contains:

`a=2`

`b=4.`

Version three contains:

`a=1`

`b=4.`

The declared deterministic procedure adds `a` and `b`.

The returned evidence bytes say:

`result=5.`

Now consider a handoff that records only

`source_path=inputs.txt.`

There are three candidate sources.

Replay version one and we obtain 5.

Replay version two and we obtain 6.

Replay version three and we obtain 5.

The returned value rules out version two **under the declared exact procedure and complete candidate set**, but it does not distinguish versions one and three. Both have different input bytes and reproduce exactly the same returned evidence.

This is stronger than pathname ambiguity alone: even output-consistency filtering leaves two source identities.

Now add the exact SHA-256 of version one.

Within the finite candidate set, only version one matches that source fingerprint.

Replay reproduces 5. The fingerprint selects the declared source; replay checks consistency with the returned value. Neither step proves that the original producer was truthful about the source identity.

Nothing about the arithmetic became more intelligent.

The handoff became better identified.

70.13 Hashes solve one problem, not every problem

It is tempting to turn that witness into a slogan:

hash everything.

That would be too strong.

A hash is excellent for byte identity.

It does not tell us:

whether the source is authoritative;

whether the source corresponds to the bibliographic work we intended;

whether a PDF scan omitted a page;

whether an executable is safe;

whether a dataset was collected correctly;

whether a theorem's hypotheses match our use.

A strong evidence object can therefore need both content identity and semantic identity.

The source lock says what the object is.

The fingerprint says which bytes were inspected.

The authority field says what role the source is permitted to play.

These are separate claims.

70.14 FAIRness and exchangeability

The FAIR principles were written for scientific data management and stewardship, but several of their ideas apply naturally to evidence exchange.

Objects benefit from persistent identifiers.

Metadata should be rich enough to describe the object.

Metadata should explicitly identify the data or object it describes.

Objects should carry qualified references to related objects.

Reusable objects should carry detailed provenance.

The principles also explicitly extend beyond conventional data to algorithms, tools, and workflows.

This is useful for an Atlas concerned with machine-assisted research.

But FAIRness is not a truth metric.

A beautifully described, permanently identified, interoperable object can still be wrong.

FAIR improves the conditions for discovery and reuse.

Epistemic status remains separate.

70.15 Return grammars force useful omissions into view

Free-form prose is flexible.

That is both its strength and its weakness.

A contributor can return three pages of sophisticated reasoning and forget to say the exact theorem being claimed.

A return grammar reduces that ambiguity.

A useful grammar asks for:

the strongest exact statement;

the derivation or support;

assumptions beyond the bootstrap;

verification or falsification hooks;

the claim boundary;

the next residual.

These fields create friction.

That friction is useful.

It forces the contributor to distinguish what was shown from what merely seems plausible.

It also makes machine intake easier because important evidence fields have stable locations.

70.16 The return route matters

Suppose a worker produces a perfect result in a private transient chat.

A later reviewer never sees it.

Did the programme receive evidence?

Operationally, perhaps someone read it.

Durably, perhaps not.

When downstream review depends on a return, the transport should create a persistent identity.

That could be:

a versioned file;

an issue comment;

an append-only journal event;

a database record;

an object-store artifact;

another durable evidence surface.

The exact technology is secondary.

The requirement is that a later actor can identify the same return object.

Durability is therefore part of the evidence chain when later inspection matters.

70.17 Receipt is not acceptance

Evidence workflows become confused when arrival is treated as validation.

The Atlas distinguishes four states.

Receipt:

the return reached the declared surface.

Acceptance:

an intake or reviewer determined that it satisfies the structural or substantive contract required at that stage.

Promotion:

a stronger project object incorporates or relies on some claim from the return.

Certification:

a separate process confers whatever formal or institutional status certification means.

These are not synonyms.

A structurally valid counterexample can be accepted even though it defeats the hoped-for theorem.

A returned proof can be received and rejected.

An accepted proof can still require formal replay before promotion.

A promoted theorem can still lack machine-checked certification.

The state changes must remain explicit.

70.18 Structural validity is weaker than mathematical validity

A return can satisfy every field in the grammar.

The dispatch ID can match.

The provenance can be complete.

The claim boundary can be explicit.

The code can run.

The theorem can still be false.

Structural intake answers:

is this object well formed enough to review?

Mathematical or scientific adjudication answers:

does the evidence support the claim?

Those are different layers.

This distinction prevents a parser from accidentally becoming a theorem prover.

70.19 Replayability is graded

A result is not simply replayable or non-replayable.

We can pin different amounts of the execution.

Level one might identify the source and algorithm.

Level two might add exact code and parameters.

Level three might add package versions, random seeds, and expected outputs.

Level four might add environment/container identity and hardware-sensitive details.

Different claims require different levels.

A symbolic proof may care little about GPU model.

A floating-point kernel benchmark may care a great deal.

So a return should pin what is material to the claim, not maximize metadata without purpose.

The later Replayable Evidence Objects chapter develops this in more detail.

70.20 Zero-context work is compiled context

There is a useful way to think about the entire exercise.

A zero-context packet is not context-free.

It is context that has been compiled.

The sender begins with a large working context:

papers;

repository state;

conversation history;

prior failed attempts;

governance boundaries;

current theorem frontier.

Only part of that context is necessary for one bounded task.

The dispatch compiler extracts:

definitions;

protected facts;

forbidden imports;

source identities;

success criteria;

return grammar;

stop conditions.

The worker receives the minimal sufficient task interface rather than the sender's entire history.

This is an information-design problem.

Too little context produces ambiguity.

Too much context can leak prior answers, waste attention, or defeat blindness.

70.21 Why excluded assumptions are first-class

Imported facts tell the worker what is available.

Excluded assumptions tell the worker what must still be earned.

This is especially important in theorem work.

Suppose a source states convergence but leaves the topology implicit.

If a downstream theorem needs convergence in a stronger topology, the missing strength is the problem.

A dispatch that merely says

prove convergence

may invite the worker to restate the source.

A dispatch that says

source-level convergence is imported;

strong-topology convergence is not imported;

show the smallest additional estimate needed

defines a much sharper task.

Zero-context work is strongest when the boundary between granted and un-granted knowledge is explicit.

70.22 Falsification conditions make returns cheaper to judge

A work packet should not only say what success looks like.

It should say what would make a contribution unusable.

Examples:

uses a forbidden source;

requires an unstated assumption;

changes the theorem domain;

depends on an unpinned artifact;

exceeds the search budget;

returns only numerical evidence where proof is required.

These conditions improve efficiency.

They allow both worker and reviewer to stop early when the task contract has been violated.

This is the evidence-exchange analogue of an API rejecting an invalid input rather than producing an ambiguous output.

70.23 GCL as a bounded implementation example

The public MATHSOLVE repository contains a concrete implementation of this style.

Its zero-context dispatch template requires an exact bounded obligation, definitions, imported facts, excluded assumptions, permitted contribution types, falsification conditions, resource bounds, a return contract, and an authority boundary.

Its RESULT/1 grammar requires fields for the strongest exact statement, derivation, assumptions, verification or falsification hooks, claim boundary, and next residual.

A public Yang-Mills work packet marks itself

ZERO_CONTEXT

and

independent_blind,

restricts the worker to a protected packet, states explicit success criteria, and demands one structured result.

These are useful because they are not hypothetical.

They show how the abstract exchange object can be instantiated in a real research workflow.

They remain project evidence.

The Atlas does not claim that every research programme should use the same field names or protocol.

70.24 A return must point backward before synthesis can point forward

The dispatch ID delta is not administrative decoration.

It is the primary backward edge.

A return should tell us which obligation it answers.

Without that link, a result can be mathematically interesting and operationally orphaned.

The same is true of source identities.

Evidence should point backward to what generated it before a synthesis step points forward to new claims.

This creates a directed provenance structure:

sources and protected facts;

dispatch;

actor activity;

return;

review;

promotion.

Each edge can be inspected.

That is far safer than a narrative chain in which conclusions gradually detach from their inputs.

70.25 Synthesis is a new reasoning step

Suppose two independent returns arrive.

One proves a statement for domain Ω_1 .

Another proves a related statement for Ω_2 .

A controller cannot simply paste the prose together and declare a theorem on a larger domain.

For the special pointwise form

for every x in Ω_i , $q(x)$,

two valid returns do justify q on Ω_1 union Ω_2 .

That simple rule does **not** generalize automatically.

A global property, uniqueness statement, convergence mode, coupled hypothesis, or domain-dependent definition may require a new composition theorem.

Synthesis therefore has its own burden.

Are the proposition schemas identical?

Are the definitions aligned?

Do the hypotheses match?

Are the source versions compatible?

Does one argument depend on an assumption excluded by the other?

What logical rule licenses the combined domain?

The return objects make these questions visible.

They do not answer them automatically.

Synthesis is itself an evidence-producing activity.

70.26 Stronger prose requires stronger support

Evidence exchange should be monotone in a specific sense.

Passing through more hands must not strengthen a claim unless some new evidence or argument justifies the strengthening.

A contributor returns:

“I verified the property for $n \leq 100$.”

A coordinator summarizes:

“the property appears robust.”

A final report writes:

“the property holds.”

The last sentence is stronger.

No new evidence appeared.

This is a promotion failure.

The Evidence chapter already gave us N , the set of stronger conclusions not licensed.

Evidence exchange preserves that field precisely to prevent confidence from increasing merely because an object travelled through a workflow.

70.27 Conflict is an admissible output

Independent work is valuable partly because returns may disagree.

If one contributor proves q and another returns a counterexample to q under the same stated assumptions, the correct evidence state is not

two votes for the average.

It is conflict.

The next operation may be:

compare source locks;

compare hidden assumptions;

replay the counterexample;

check the proof step;

send an adversarial packet;

narrow the theorem.

A good exchange protocol preserves contradictions until they are resolved.

It does not smooth them away for the sake of a clean dashboard.

70.28 Duplicate delivery and evidence identity

Coordination systems can retry.

That means the same logical return may appear more than once.

Two comments can carry the same result.

The same payload can be reposted by different actors.

A worker can retry after an uncertain transport failure.

So evidence intake needs stable logical identity.

The dispatch identity helps.

A return identity can help further.

Payload equality alone is not enough because two independent actors can legitimately return identical text.

Conversely, small formatting changes do not necessarily create a new logical result.

This is the evidence analogue of the Coordination chapter's distinction between delivery and effect.

70.29 Evidence exchange and memory

Evidence objects often become memory.

A returned proof can enter an archive.

A source lock can enter persistent memory.

A failed attempt can become episodic memory for later search.

A promoted theorem can become semantic project memory.

This creates a new boundary.

Storing a return does not promote it.

Retrieving it later does not revalidate it.

Memory preserves access.

Evidence status must travel with the object.

That is why provenance and epistemic class are not optional metadata in long-lived research systems.

70.30 Evidence exchange and agents

Agentic systems make these issues more urgent because delegation can happen quickly.

A parent agent can spawn several workers.

Each can search, compute, or prove.

Returns can arrive concurrently.

A summarizer can merge them.

Without explicit exchange semantics, the system can lose:

which worker used which source;

which result was independent;

which assumptions were shared;

which claim was only numerical;

which return was superseded;

which failure was never resolved.

The danger is not only hallucination.

It is provenance collapse.

A coordinated system can become less epistemically legible as it becomes more capable.

The exchange layer exists to resist that collapse.

70.31 Minimal handoff, not maximal paperwork

The goal is not to wrap every thought in bureaucracy.

A tiny task can have a tiny dispatch.

What matters is sufficiency.

If the task is

compute SHA-256 of these exact bytes

then the packet can be short.

If the task is

close the missing theorem-grade convergence bridge in a source with known omitted proofs

then the packet must be richer.

The metadata burden should scale with the ambiguity and consequence of the claim.

This is another form of adaptive computation:

spend evidence-handling effort where the epistemic boundary requires it.

70.32 What Replay may assume

ATLAS-CH-REPLAY-001 may assume:

stable dispatch and return identities;

immutable source/version identities;

the distinction between provenance and truth;

verification/falsification hooks;

graded replayability;

claim-boundary preservation.

Its job is to go further:

pin environments;
capture executable artifacts;
record exact outputs;
build semantic bridges from bytes to claims;
make replay itself a governed evidence object.
It need not rediscover why a mutable pathname is insufficient.

70.33 What the Research State Machine may assume

ATLAS-CH-RESEARCHSM-001 may later assume:
bounded work packages;
durable return routes;
context and independence declarations;
receipt versus acceptance versus promotion;
explicit residuals.
Its task is to arrange those objects into programme-level state transitions.
This chapter supplies the objects that move.
The state-machine chapter supplies the lifecycle.

70.34 The boundary to remember

Evidence exchange is successful when a result can leave one actor without losing the conditions under which it means what it means.
That requires more than copying text.
The obligation must have an identity.
The imported facts must be explicit.
The source universe must be declared.
The return must point back to the dispatch.
The claim must carry its scope and non-claims.
The provenance must identify how the object came to be.
The replay hooks must say how another actor can challenge it.

And no workflow step may promote the claim merely because the object moved successfully.

A good evidence packet therefore behaves like a scientific interface.

It says what goes in.

It says what may be assumed.

It says what comes out.

It says what the output does not establish.

And it leaves enough of a trail that another actor can disagree with it for the right reasons.

That is the purpose of zero-context work.

Not to remove context.

To make the necessary context explicit, bounded, portable, and inspectable.

70.35 References used in this chapter

Exact source identities and authority scopes are pinned in:

`sources/source-locks/ATLAS-CH-EVIDEX-001.yaml`

The external basis includes W3C PROV-DM and the FAIR Guiding Principles.

The bounded implementation examples are exact public MATHSOLVE artifacts at commit `1273b75457f42da62a8af4c69493d44d293d4567`.

Chapter 71

The Computational Polity

Epistemic status: audited prerequisites + primary scholarly sources + Atlas synthesis + exact finite witness

Specification: manuscript/specifications/ATLAS-CH-POLITY-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-POLITY-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-POLITY-001.md

A capable model is not automatically a capable system.

A real adaptive system may include several models, a shared memory, tools, human participants, validators, transaction boundaries, and rules about who may change what. Once those elements matter to the result, treating the model alone as the full locus of intelligence hides the mechanism that actually produced the outcome.

This chapter calls the larger object a **computational polity**.

71.1 From agent to polity

Wooldridge and Jennings distinguish agent theory, agent architectures, and agent languages, already making clear that an intelligent agent is an architectural object rather than merely a function from prompt to answer (Wooldridge and Jennings 1995).

The Atlas goes one level outward.

A polity is not just a collection of agents. It is a collection plus the rules by which local states, shared records, tools, validators, human roles, and authority compose.

71.2 The polity object

Write

$$\Pi = (A, M, U, H, V, C, \Gamma, Q).$$

Here:

- A is the set of model or agent roles;
- M is shared external memory;
- U is the set of tools or external services;
- H is the set of human roles;
- V is the set of validator or reviewer roles;
- C is the inherited coordination object;
- Γ is the governance relation defining admissible transitions and authority;
- Q is the task or claim contract.

These coordinates need not share one data type. The polity is a systems object, not one giant hidden vector.

71.3 Local state and shared state

The Coordination chapter already separates local actor state from shared coordination state. The External Memory chapter adds version, provenance, access, synchronization, and deletion semantics for persistent records.

POLITY therefore inherits a hard boundary:

$$\text{shared address} \not\Rightarrow \text{shared belief}.$$

Two participants may refer to the same named store while seeing different versions, different permissions, different caches, or different retrieval results.

Common memory is not identical internal state.

71.4 Capability is not authority

A role may be able to produce an action without being allowed to commit it.

Define

$$\text{Can}(r, a)$$

for capability and

$$\text{May}(r, a, \Gamma)$$

for authorization under governance Γ .

In general,

$$\text{Can}(r, a) \not\Rightarrow \text{May}(r, a, \Gamma).$$

The reverse implication also fails as a correctness claim. Permission to act does not prove that the action is wise, true, or safe.

This distinction becomes load-bearing whenever a model proposes a change that a different role must validate or authorize.

71.5 Candidate is not conclusion

A representative polity path is

$q \rightarrow \text{route} \rightarrow \text{candidate} \rightarrow \text{evidence record} \rightarrow \text{validation} \rightarrow \text{authorized commit}.$

Every arrow can fail independently.

A good candidate may be routed to the wrong validator. A correct validator result may be applied to stale evidence. An authorized commit may fail at an external side effect. A message may be delivered without being accepted. A shared record may exist without being current.

The polity view makes those failure surfaces visible.

71.6 Why humans remain inside the system boundary

Human review is often drawn outside technical diagrams, as though a person enters only after the computational story is complete.

For a polity, that is a modeling error whenever the human decision changes system state.

Human roles have inputs, information limits, authority, latency, provenance, and failure modes. Klein and colleagues emphasize that automation participating in joint human-agent activity must function as a team participant rather than as an isolated device (Klein et al. 2004).

The Atlas therefore models human roles explicitly without assuming that human participation makes the system infallible.

71.7 System-level cognition is an old idea

Hutchins analyzes navigation as a cognitive and computational activity distributed across people, artifacts, procedures, and representations rather than exhausted by one person's internal cognition (Hutchins 1995).

The Computational Polity uses that precedent carefully.

It does not claim that every distributed system is intelligent. It claims only that, when the mechanism of successful computation spans several components, the correct explanatory boundary may also span those components.

71.8 Exact two-specialist witness

Let

$$Q = \{\alpha, \beta\}.$$

Define two specialists:

$$A_\alpha(\alpha) = 1, \quad A_\alpha(\beta) = 0,$$

and

$$A_\beta(\alpha) = 0, \quad A_\beta(\beta) = 1.$$

Under a uniform task distribution, each specialist alone has accuracy

$$\frac{1}{2}.$$

Now define a router

$$R(\alpha) = A_\alpha, \quad R(\beta) = A_\beta.$$

The composed polity returns

$$\Pi(q) = R(q)(q) = 1$$

for both tasks, so

$$\text{Acc}(\Pi) = 1.$$

The system-level capability is exact and easy to inspect.

71.9 The same components can still fail

Now replace the router by one that always chooses A_α .

The component set is unchanged.

The system accuracy returns to

$$\frac{1}{2}.$$

Therefore:

more components $\nrightarrow$ greater capability.

The composition rule matters.

71.10 Shared memory as a polity institution

An accepted result may be stored as

$$m = (q, y, s, v, \sigma),$$

where q is task identity, y the result, s source identity, v version, and σ status.

This record can be read by later roles without requiring every model to internalize the result in weights.

But the External Memory boundary remains:

- stored does not mean true;
- present does not mean current;
- shared does not mean universally visible;
- retrieved does not mean correctly used.

71.11 Validators are roles, not oracles

A validator can reduce a declared class of error.

It cannot be promoted into an infallible truth function merely because it has the title ‘validator’.

A polity therefore records:

- what the validator receives;
- which criterion it applies;
- which authority it has;
- what evidence it preserves;

- what happens when it abstains, fails, or disagrees.

The same discipline applies to human review.

71.12 Governance is transition structure

Governance answers questions such as:

- who may create a candidate;
- who may validate it;
- who may commit it;
- which transitions require distinct actors;
- which states are protected;
- which evidence is required before promotion.

These are transition constraints.

Governance does not manufacture evidence that is missing from the underlying claim.

Thus:

authorized $\nRightarrow$ correct.

and

correct-looking $\nRightarrow$ authorized.

71.13 Tools and external effects

A polity may call search systems, databases, compilers, laboratories, actuators, or other services.

The inherited coordination boundary matters here: an internal transaction cannot silently make an external side effect atomic.

A successful tool response and a durably committed external state change are different events unless the interface explicitly couples them.

71.14 Cost is heterogeneous

A polity consumes more than model FLOPs.

A useful bookkeeping vector is

$$c_{\Pi} = (c_{coord}, c_{mem}, c_{tool}, c_{human}, c_{val}).$$

These entries may have different units. They should not be added into one scalar unless the application declares a conversion or objective.

This prevents ‘more agents’ from being treated as free.

71.15 Roles are not identities

One physical or software participant may occupy several roles over time.

Conversely, one role may be filled by different participants.

The polity should therefore bind authority and provenance to role and identity explicitly rather than assuming that labels such as ‘agent’, ‘reviewer’, or ‘human’ carry permanent semantics.

71.16 System-level competence needs a declared contract

A polity-level claim should identify at least:

- task set or distribution;
- component roles;
- routing/composition rule;
- memory semantics;
- validation rule;
- governance/authority rule;
- success metric.

Without these, ‘the system can do X’ is too underspecified to audit.

71.17 What this chapter does not claim

The polity view does not imply:

- that distributed systems are always better than monolithic models;
- that human review is always superior to automation;
- that more validators always improve reliability;
- that shared memory is globally consistent;
- that governance guarantees correctness;
- that specialization always outweighs coordination cost;
- that system-level behavior is mysterious emergence.

The aim is the opposite: make system-level capability reconstructable from explicit composition.

71.18 Handoff to Beyond the Monolithic Model

ATLAS-CH-SYNTHESIS-001 may now assume:

- the polity object II;
- private/shared-state separation;
- the capability/authority distinction;
- candidate, validation, and commit as separate stages;
- the exact specialization witness;
- the requirement that system capability be demonstrated under declared composition.

SYNTHESIS must still connect this systems view to the rest of the Atlas: geometry, dynamics, memory, optimization, evidence, adaptation, and governance.

71.19 Durable lesson

A model can be an important component without being the whole intelligent system.

When memory, tools, humans, validators, and governance materially determine the result, the explanatory object should include them.

The computational polity is that larger object.

71.20 References used in this chapter

- (Wooldridge and Jennings 1995)
- (Hutchins 1995)
- (Klein et al. 2004)

Exact source authority, prerequisite identities, and claim boundaries are locked in:

`sources/source-locks/ATLAS-CH-POLITY-001.yaml`

Chapter 72

The Machine Under the Mathematics

Epistemic status: audited Linear Algebra prerequisite + classical Roofline performance model + current CUDA programming/best-practices documentation + Atlas synthesis + exact finite traffic witness.

Specification: manuscript/specifications/ATLAS-CH-HARDWARE-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-HARDWARE-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-HARDWARE-001.md

Source lock: sources/source-locks/ATLAS-CH-HARDWARE-001.yaml

72.1 Mathematics does not run in zero time

A mathematical chapter can write

$$C = AB.$$

The hardware must still:

- obtain the inputs;
- schedule the work;
- move values through a memory hierarchy;
- perform arithmetic in some format;
- synchronize where required;
- place the result somewhere.

Those operations consume physical resources.

The central boundary of this chapter is therefore:

mathematical work $\neq$ physical execution cost.

Two programs can implement the same map and take different time.

Two programs can execute the same nominal number of floating-point operations and move different numbers of bytes.

Two kernels can move the same bytes and expose different amounts of parallel work.

Hardware is not an implementation footnote.

It is part of the effective computational system.

72.2 The linear map survives; the implementation changes

The Linear Algebra chapter distinguished an abstract linear map from one coordinate representation.

Hardware adds another distinction.

Even after the matrix representation has been fixed, there can be many physical implementations of the same matrix operation.

For example, a matrix product can differ in:

- blocking;
- tile order;
- data layout;
- precision;
- reuse;
- kernel fusion;
- memory placement;
- scheduling.

The output may represent the same mathematical operator.

The execution path need not be the same.

This is why algorithmic complexity, FLOP count, and runtime should not be collapsed.

72.3 FLOPs are an accounting convention

A floating-point operation count is useful only after its convention is declared.

This chapter's finite witness counts:

- one scalar multiplication as one FLOP;
- one scalar addition as one FLOP.

Under that convention, a 2×2 matrix product requires:

8

multiplications and

4

additions, for a total of:

12 FLOPs.

Other performance conventions may count fused multiply-add differently.

The point is not that one convention is universally correct.

The point is that comparisons must use one declared convention consistently.

72.4 Throughput is not latency

A machine can sustain a high rate of completed operations while one individual dependency chain still takes significant time.

These are different quantities.

Latency asks:

How long does one operation or dependency chain take?

Throughput asks:

How much work can the machine complete per unit time when enough parallel work is available?

Modern accelerators are often designed to tolerate latency by maintaining many independent operations in flight.

That strategy works only when the workload exposes enough usable parallelism.

72.5 Data movement can dominate arithmetic

Suppose a kernel performs very little arithmetic on each loaded value.

The arithmetic units may spend much of their time waiting for data.

Suppose instead the same loaded values are reused many times before eviction.

Then much more arithmetic can be performed per byte transferred.

The relevant quantity is **arithmetic intensity**.

For a declared memory boundary M , let:

$$W$$

be the declared FLOP count and

$$Q_M$$

the bytes moved across that boundary.

Then:

$$I_M = \frac{W}{Q_M}.$$

The subscript matters.

The same computation can have different intensities relative to:

- a host-device link;
- device DRAM;
- an on-chip cache;
- programmer-managed shared memory.

Arithmetic intensity is therefore not a property of source code alone.

It is a property of work together with a declared data-movement boundary and implementation.

72.6 The Roofline idea

Williams, Waterman, and Patterson introduced the Roofline model as a visual upper-bound model connecting computational throughput, memory bandwidth, and arithmetic intensity. (Williams et al. 2009)

In its simplest one-level form:

$$P_{\text{attainable}} \leq \min(P_{\text{max}}, B_M I_M).$$

Here:

- P_{max} is a declared peak compute rate;
- B_M is a declared bandwidth across memory boundary M ;

- I_M is arithmetic intensity at that boundary.

The first term gives a compute roof.

The second gives a bandwidth roof.

The crossover occurs at:

$$I_M^* = \frac{P_{\max}}{B_M}.$$

Below this point, the bandwidth-side roof is lower.

Above it, the compute roof is lower.

This does not mean an implementation will reach the roof.

It means the implementation should not be expected to exceed the declared bound.

72.7 A roof is not a prediction

Real performance can fall below a Roofline bound because of:

- dependencies;
- poor memory access;
- insufficient parallel work;
- instruction mix;
- synchronization;
- divergence;
- launch overhead;
- cache effects;
- resource constraints;
- implementation quality.

So the safe statement is:

Roofline = upper-bound diagnostic,

not:

Roofline = runtime oracle.

The model helps ask whether improving arithmetic throughput is even relevant before doing so.

72.8 The memory hierarchy

The current CUDA programming model exposes several physically and logically distinct storage levels, including thread-local registers, shared memory associated with blocks, caches, and global/device memory. (NVIDIA Corporation 2026b)

The CUDA Best Practices Guide emphasizes that these memory spaces differ in scope, latency, bandwidth, and intended use. (NVIDIA Corporation 2026a)

The exact hierarchy is architecture-specific.

The general systems lesson is not:

every accelerator has this exact hierarchy.

It is:

locality has levels, and moving data between levels has different costs.

A kernel that repeatedly fetches the same value from a distant level can behave very differently from one that reuses a nearby copy.

72.9 Registers and local state

Registers are fast, thread-local storage in CUDA’s programming model. (NVIDIA Corporation 2026b)

They are finite.

A kernel that requires more per-thread state can reduce the number of simultaneously resident threads or spill values into slower storage.

So a transformation that appears to “save memory traffic” can still create another bottleneck if it requires too much local state.

Hardware optimization is usually a trade among constrained resources.

72.10 Shared memory and reuse

CUDA shared memory is an on-chip, programmer-managed storage region shared among threads in a block. (NVIDIA Corporation 2026b)

The Best Practices Guide uses shared-memory examples to show why staging data near the compute units can:

- enable coalesced access;
- reduce redundant global-memory loads;
- improve reuse.

It also documents bank conflicts as one way shared-memory access can lose effective bandwidth. (NVIDIA Corporation 2026a)

The general lesson is that “on chip” does not mean “free.”

Access pattern still matters.

72.11 Global memory and bandwidth

Device/global memory is large relative to on-chip storage and visible across the device in the CUDA model. (NVIDIA Corporation 2026b)

Its capacity is useful.

Its distance from execution units makes repeated traffic expensive relative to reusing data already nearby.

This motivates three recurring questions:

1. How many bytes must cross the boundary?
2. How often are the same values reused?
3. Can the implementation rearrange work to increase locality?

These questions can matter more than shaving a small number of arithmetic instructions.

72.12 Warps and SIMT

CUDA groups threads into warps of 32 threads in its current programming model and describes execution using SIMT semantics. (NVIDIA Corporation 2026b)

Threads execute the same kernel but may follow different control-flow paths.

Divergence can reduce useful lane utilization.

Memory access patterns across a warp also affect how efficiently global-memory transactions are formed.

These are CUDA-specific execution facts.

They should not be silently promoted into universal laws of all accelerators.

72.13 Parallelism is a resource, not a synonym for speed

A workload can contain many operations and still expose limited parallelism because of dependencies.

Another workload can expose enormous parallelism but become bandwidth-limited.

A third can expose both parallelism and reuse but be constrained by register or shared-memory requirements.

So the Atlas separates:

- work count;
- available parallelism;
- resource residency;
- achieved throughput.

These are related.

They are not identical.

72.14 Occupancy

In GPU performance discussions, occupancy usually refers to resident active warps or threads relative to a hardware limit.

It can help hide latency.

It is not itself the objective.

Higher occupancy can coexist with lower performance if, for example:

- each thread does less useful work;
- memory access worsens;
- instruction-level reuse is lost;
- more efficient kernels need more registers or shared memory.

The safe rule is:

$$\text{occupancy} \neq \text{performance.}$$

Occupancy is one diagnostic among several.

72.15 Matrix and tensor hardware

Modern NVIDIA GPUs expose warp-level matrix multiply-accumulate operations that can use Tensor Cores for supported matrix problems of the form:

$$D = AB + C.$$

The programming guide documents tile shapes, supported data types, fragment layouts, synchronization requirements, and architecture-dependent restrictions. (NVIDIA Corporation 2026b)

This creates a strong distinction between:

abstract matrix multiplication

and

matrix multiplication mapped efficiently to specialized hardware.

A matrix expression does not automatically achieve the device’s peak matrix-unit throughput.

Shapes, layouts, types, alignment, data movement, and surrounding operations matter.

72.16 Tiles connect algebra to hardware

Large matrix products are decomposed into smaller tiles.

Tiling can improve reuse because a loaded block participates in many multiply-accumulate operations before replacement.

This is the physical counterpart of a simple algebraic fact:

one matrix entry can contribute to several output entries.

The optimization opportunity comes from arranging the implementation so the hardware retains that value long enough to reuse it.

The mathematics supplies reuse potential.

The implementation determines whether the machine realizes it.

72.17 Precision is several questions

“FP16,” “BF16,” “FP32,” and other format names describe representation and arithmetic choices.

They do not by themselves answer:

- what error accumulates;
- where rounding occurs;
- what the accumulator format is;
- whether overflow or underflow occurs;
- whether the final task tolerates the resulting perturbation.

The CUDA Best Practices Guide explicitly separates numerical accuracy/precision concerns and notes that results can differ across floating-point precisions because representation and rounding differ. (NVIDIA Corporation 2026a)

So the Atlas separates:

smaller format

from

acceptable numerical error.

The first is a storage/arithmetic choice.

The second is a property that must be checked relative to the computation and task.

72.18 Mixed precision

Specialized matrix hardware often supports mixed input and accumulator formats.

This can be valuable because:

- narrower inputs reduce storage and traffic;
- wider accumulation can reduce some forms of accumulation error.

But “mixed precision” is not one universal numerical scheme.

The actual behavior depends on the supported operation and formats.

The correct question is always:

Which values are represented in which format at which stage?

72.19 Kernels are physical programs

A mathematical operator can be implemented by one kernel or several.

Kernel boundaries matter because intermediate values may need to be materialized in memory.

A fused implementation can sometimes keep intermediates closer to the arithmetic units.

That can reduce:

- launches;
- synchronization;
- intermediate stores;
- intermediate reloads.

But a fused kernel is useful only if it preserves the required semantics and numerical tolerance.

Performance transformation does not exempt the implementation from correctness.

72.20 Fusion can change the traffic graph

Consider:

$$y = f(x), \quad z = g(y).$$

An unfused implementation may write y to global memory and later read it back.

A fused implementation may retain y in registers or another nearer storage level. The mathematical composition remains:

$$z = (g \circ f)(x).$$

The physical data-movement graph changes.

This is one reason equal high-level operation graphs can produce different runtime.

72.21 Exact finite witness

Let:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Then:

$$AB = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

Under the chapter convention:

$$W = 12 \text{ FLOPs.}$$

72.21.1 No cross-output reuse

Charge:

- 16 input scalar loads;
- 4 output scalar stores.

With 4-byte FP32 scalars:

$$Q_A = 80 \text{ bytes.}$$

Therefore:

$$I_A = \frac{12}{80} = 0.15 \text{ FLOP/byte.}$$

72.21.2 Perfect full-input reuse

Charge:

- 8 total input scalar loads;
- 4 output scalar stores.

Then:

$$Q_B = 48 \text{ bytes,}$$

and:

$$I_B = \frac{12}{48} = 0.25 \text{ FLOP/byte.}$$

Same exact matrix product.

Same exact declared FLOP count.

Different charged data movement.

72.22 The bound moves even when the mathematics does not

For a hypothetical one-level machine:

$$P_{\max} = 10 \text{ TFLOP/s,} \quad B = 1 \text{ TB/s.}$$

The Roofline bounds become:

$$P_A \leq 0.15 \text{ TFLOP/s,}$$

and:

$$P_B \leq 0.25 \text{ TFLOP/s.}$$

The example does not predict a measured GPU speedup.

It proves a narrower point:

same map + same FLOPs $\nrightarrow$ same bandwidth-side performance bound.

72.23 The witness is intentionally idealized

The finite witness does not charge:

- cache-line overfetch;
- write allocate;
- instruction traffic;
- coherence;
- host-device movement;
- launch overhead;
- synchronization.

The perfect-reuse model also assumes the inputs remain available for the full tiny product.

These are not omissions to hide.

They are part of the declared witness.

A useful performance model states what it counts.

72.24 Benchmarking can create false conclusions

Hardware benchmarks are easy to misread.

A measured number depends on choices such as:

- device model;
- driver/runtime/library versions;
- data type;
- shape;
- warmup;
- synchronization;
- repetitions;
- timing region;
- transfer inclusion;
- compiler options;
- clock/power state;
- kernel selection.

A number without this context can be precise and still be uninterpretable.

72.25 Peak versus sustained performance

A vendor peak rate is a hardware specification under declared operation/type assumptions.

A measured kernel rate is an empirical observation under a workload and software stack.

An application end-to-end rate includes more than a kernel.

These three quantities should not be mixed.

The safest comparison names which one is being reported.

72.26 FLOP/s can reward doing more work

A kernel can report more FLOP/s simply because it performs extra arithmetic that the hardware handles efficiently.

That does not necessarily make the computation finish sooner.

Time-to-solution, energy, memory use, accuracy, and throughput answer different questions.

A metric is useful only when it matches the declared objective.

72.27 Hardware-aware mathematics

Hardware considerations can feed back into algorithm design.

One method may use more arithmetic but less memory traffic.

Another may trade recomputation for storage.

A third may select a structured factorization because it maps naturally to matrix units.

The correct scientific question is not:

Is hardware corrupting the mathematics?

It is:

Which mathematically acceptable representation exposes the physical resources the machine can use efficiently?

This is a design problem across abstraction levels.

72.28 What belongs downstream

This chapter is deliberately single-device and kernel-centric.

It does not develop:

- data parallelism;
- tensor parallelism;
- pipeline parallelism;
- expert parallelism;
- collective communication;
- interconnect topology;
- distributed KV caches;
- serving/batching systems;
- multi-device throughput/latency tradeoffs.

Those belong in:

Scaling, Parallelism, and Serving — ATLAS-CH-SYSTEMS-001.

Hardware-001 supplies the substrate that Systems will need:

- bandwidth;
- locality;
- arithmetic intensity;
- precision;
- resource constraints;
- benchmark discipline.

72.29 Durable boundaries

The chapter leaves the reader with nine distinctions:

1. FLOPs are not time.
2. Throughput is not latency.
3. Equal FLOPs do not imply equal bytes.
4. Arithmetic intensity needs a declared memory boundary.
5. Peak throughput is not attained throughput.
6. Occupancy is not performance.
7. Matrix algebra is not guaranteed matrix-unit efficiency.
8. Smaller precision is not guaranteed acceptable error.
9. Kernel optimization is not distributed-system optimization.

These boundaries are more useful than memorizing one accelerator’s specification sheet.

72.30 References used in this chapter

- (Williams et al. 2009)
- (NVIDIA Corporation 2026b)
- (NVIDIA Corporation 2026a)

Exact source identities and authority scopes are pinned in:

`sources/source-locks/ATLAS-CH-HARDWARE-001.yaml`

Chapter 73

Scaling, Parallelism, and Serving

Epistemic status: audited Hardware and Mixture-of-Experts prerequisites + primary distributed-training/serving systems sources + Atlas synthesis + exact finite placement/communication witness.

Specification: manuscript/specifications/ATLAS-CH-SYSTEMS-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-SYSTEMS-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-SYSTEMS-001.md

Source lock: sources/source-locks/ATLAS-CH-SYSTEMS-001.yaml

A model that fits and runs efficiently on one accelerator is not yet a distributed system.

Once computation crosses devices, several new objects appear:

- replicas and shards;
- stage boundaries;
- collectives;
- physical links;
- synchronization;
- queues;
- request schedulers;
- distributed memory state;
- stragglers;
- service-level metrics.

The governing rule is:

distributed execution cost is a property of computation, communication, placement, scheduling, and m

This chapter develops that boundary without turning one training stack or serving system into a universal law.

73.1 From hardware to systems

HARDWARE-001 established that:

- FLOPs are not time;
- bytes moved matter;
- arithmetic intensity is boundary-relative;
- peak throughput is not attained throughput;
- latency and throughput are distinct;
- benchmark scope must be declared.

Those rules continue to hold across devices.

Distributed execution adds another layer: values may have to move between devices, and the path itself can become load-bearing.

The same mathematical model can therefore differ in:

- partition;
- placement;
- collective algorithm;
- topology;
- schedule;
- memory replication;
- request batching.

Equal model arithmetic does not erase these differences.

73.2 A typed systems object

Write:

$$\mathcal{S} = (R, \Pi, \Gamma, \mathcal{C}, \mathcal{M}, \mathcal{Q}, \mathcal{P}).$$

Here:

- R : ranks/devices;
- Π : typed parallelism decomposition;
- Γ : physical communication topology;
- $\mathcal{C}$: collective/point-to-point semantics;

- $\mathcal{M}$: memory state and placement;
- $\mathcal{Q}$: scheduling/request state;
- $\mathcal{P}$: measurement protocol.

A claim about one coordinate does not automatically determine the others.

For example:

tensor-parallel degree = 4

does not tell us:

- which tensor dimension is split;
- which collective follows;
- whether links are intra-node or inter-node;
- how much overlap occurs;
- what latency is measured.

73.3 Parallelism is not one thing

Large-model systems commonly combine several different decompositions.

The Atlas keeps four separate:

1. data parallelism;
2. tensor parallelism;
3. pipeline parallelism;
4. expert parallelism.

Narayanan et al. describe composing data, tensor, and pipeline parallelism in Megatron-LM and report measured large-cluster trade-offs in that system.

The Atlas uses that paper as primary evidence that these modes can be composed and have materially different scaling behavior.

It does not treat the reported configuration as universal.

73.4 Data parallelism

In synchronous data parallelism, parameter replicas process different data shards.

Let rank r compute local gradient:

$$g_r.$$

One declared synchronization rule is:

$$\bar{g} = \frac{1}{p} \sum_{r=0}^{p-1} g_r.$$

Replicas then apply the same update from $\bar{g}$.

The communication is not optional if exact replica synchronization under this rule is required.

But data parallelism does not by itself specify:

- ring versus tree all-reduce;
- gradient bucketing;
- optimizer-state sharding;
- communication overlap;
- physical placement.

Those are additional systems choices.

73.5 Tensor parallelism

Tensor parallelism divides one operator across ranks.

For example, a matrix operation can be partitioned by rows, columns, heads, or another declared dimension.

Each rank computes a partial result.

The next layer may then require:

- all-reduce;
- all-gather;
- reduce-scatter;
- point-to-point exchange;
- no communication until a later boundary.

The exact pattern depends on the partition and surrounding operators.

So:

tensor parallelism $\neq$ one fixed communication pattern.

Megatron-LM is a primary example of practical tensor-parallel Transformer training.

73.6 Pipeline parallelism

Pipeline parallelism divides an ordered network into stage groups.

If stages are:

$$S_1, S_2, \dots, S_k,$$

a microbatch flows through them in order.

Different microbatches can occupy different stages concurrently.

GPipe is a primary example of microbatch pipeline parallelism.

The key systems quantities include:

- stage partition;
- microbatch count;
- fill time;
- drain time;
- idle bubbles;
- activation transfers;
- backward transfers during training;
- stage imbalance.

More stages can reduce per-stage model memory.

They can also create more communication boundaries and more sensitivity to imbalance.

73.7 A pipeline bubble is schedule-relative

Suppose one stage finishes while the next required dependency has not arrived.

That stage is idle.

This idle period belongs to the schedule and dependency graph.

It is not a new mathematical layer in the model.

Pipeline utilization therefore depends on how microbatches are ordered through stages.

A stage count alone cannot determine it.

73.8 Expert parallelism

MOE-001 established a chain:

router probabilities $\rightarrow$ preferred routes $\rightarrow$ capacity/overflow $\rightarrow$ accepted dispatch $\rightarrow$ expert execution.

Expert parallelism adds physical placement.

Let:

$$\text{dev}(e)$$

be the device hosting expert e .

If token i originates elsewhere, accepted dispatch induces communication.

SYSTEMS adds one further object:

$$\text{path}(\text{src}(i), \text{dev}(e); \Gamma).$$

Hence:

$$\boxed{\text{preference} \neq \text{dispatch} \neq \text{placement} \neq \text{network path}.}$$

This distinction prevents a routing diagram from pretending to be a systems benchmark.

73.9 Collective semantics

A collective says what distributed values must become.

It does not by itself say how the network achieves that state.

For rank-local values x_r :

73.9.1 All-reduce

Every rank receives the reduction:

$$y_r = \sum_j x_j.$$

73.9.2 Reduce-scatter

The values are reduced, and declared partitions of the result are distributed across ranks.

73.9.3 All-gather

Declared partitions are gathered so every rank receives the whole collection.

73.9.4 All-to-all

Each source rank sends destination-specific payloads to every destination rank.

These are semantic contracts.

Ring, tree, hierarchical, topology-aware, and implementation-specific algorithms are ways to realize them.

73.10 Logical topology and physical topology

A ring all-reduce has a logical neighbor order.

A cluster has a physical topology.

They are not the same object.

A logical neighbor pair might map to:

- two devices on one board;
- two devices in one node;
- two nodes in one rack;
- two nodes across a slower shared fabric.

Placement can therefore change which physical links carry the same logical collective.

73.11 Exact four-rank witness

Use four ranks:

$$A_0, A_1, B_0, B_1.$$

Place:

- A_0, A_1 on node A;
- B_0, B_1 on node B.

Each rank begins with an 8-element FP32 vector.

Thus:

$$S = 8 \cdot 4 = 32 \text{ bytes.}$$

The collective is sum all-reduce implemented as ring reduce-scatter plus ring all-gather.

73.12 Per-rank ring accounting

With:

$$p = 4,$$

each vector is split into four 8-byte chunks.

Reduce-scatter requires three ring steps.

All-gather requires three more.

Each rank sends one chunk per step.

Therefore each rank sends:

$$6 \cdot 8 = 48 \text{ bytes}$$

and receives 48 bytes.

During reduce-scatter, each rank reduces three 2-scalar chunks:

$$3 \cdot 2 = 6$$

scalar additions.

The two placements below have exactly the same values.

73.13 Grouped placement

Use logical ring:

$$A_0 \rightarrow A_1 \rightarrow B_0 \rightarrow B_1 \rightarrow A_0.$$

Only two directed ring edges cross between nodes:

$$A_1 \rightarrow B_0, \quad B_1 \rightarrow A_0.$$

Each ring edge carries 48 bytes over the complete collective.

Therefore:

$$Q_{\text{cut}}^G = 2 \cdot 48 = 96 \text{ bytes.}$$

73.14 Interleaved placement

Now use:

$$A_0 \rightarrow B_0 \rightarrow A_1 \rightarrow B_1 \rightarrow A_0.$$

Every ring edge crosses the node boundary.

Therefore:

$$Q_{\text{cut}}^I = 4 \cdot 48 = 192 \text{ bytes.}$$

Nothing changed in the reduction arithmetic.

Nothing changed in per-rank total communication volume.

Only logical-to-physical placement changed.

73.15 Equal arithmetic does not imply equal physical communication

Both cases have:

- 6 additions per rank;
- 24 aggregate additions;
- 48 bytes sent per rank;
- 48 bytes received per rank.

Yet:

$$Q_{\text{cut}}^I = 2Q_{\text{cut}}^G.$$

Therefore:

equal arithmetic and equal per-rank bytes $\nRightarrow$ equal cross-cut traffic.

This is the chapter's exact finite control.

73.16 A lower bound is not a latency measurement

Suppose, only for the toy model, that all cross-node traffic shares aggregate cut capacity:

$$B_{\text{cut}} = 16 \text{ bytes/time-unit.}$$

Then any execution must satisfy:

$$T \geq \frac{Q_{\text{cut}}}{B_{\text{cut}}}.$$

So:

$$T_G \geq 6, \quad T_I \geq 12.$$

This does not say a real machine takes 6 or 12 units.

Real time can include:

- protocol overhead;
- launch overhead;
- synchronization;
- bidirectional behavior;
- overlap;
- routing;
- congestion;
- software queues;
- device-kernel time.

The witness proves an accounting distinction, not a benchmark.

73.17 Communication can hide behind computation

Distributed systems often overlap communication with local arithmetic.

If a communication step lies off the critical path, some of its time can be hidden.

But hidden is not free.

The bytes still occupy links, buffers, and collective state.

A competing transfer can expose the cost again.

So a serious claim should distinguish:

- communication volume;
- communication duration;
- exposed communication time;
- total step time.

73.18 Stragglers

A synchronized step often waits for the slowest required participant.

That participant may be slow because of:

- more work;
- slower hardware;
- slower link path;
- cache/memory effects;
- queueing;
- transient interference;
- expert imbalance.

Average rank time can therefore fail to predict synchronized step time.

The MoE straggler boundary is a special case of this broader rule.

73.19 Training scaling

For a fixed declared workload and measurement protocol, define throughput at p devices:

$$\text{Th}(p).$$

A strong-scaling-style efficiency statistic is:

$$\eta_p = \frac{\text{Th}(p)}{p \text{Th}(1)}.$$

The statistic is meaningful only if the comparison states what was held fixed.

Changing:

- global batch size;
- sequence length;
- precision;
- activation checkpointing;
- optimizer semantics;
- model partition;

can change the task being compared.

73.20 More devices can reduce efficiency

Additional devices can provide:

- more memory;

- more compute;
- more concurrency.

They can also introduce:

- more synchronization;
- more communication;
- more idle imbalance;
- more failure surface.

So:

$$p \uparrow \not\Rightarrow \eta_p \uparrow .$$

Nor does low η_p alone identify the cause.

73.21 Training and serving optimize different state

Training repeatedly performs forward work, backward work, gradient synchronization, and parameter updates.

Serving receives requests from outside the system.

Autoregressive generation then creates a new state variable: each active request has a current generated prefix and must be revisited for further tokens.

The scheduler therefore becomes part of the mechanism.

73.22 Request latency

For request i :

$$L_i = t_i^{\text{complete}} - t_i^{\text{arrival}} .$$

Different serving questions may care about:

- time to first token;
- time per output token;
- end-to-end completion latency;
- p50 latency;
- p95/p99 tail latency.

One number cannot stand for all of them.

73.23 Throughput

Serving throughput can be measured as:

- requests per second;
- output tokens per second;
- total processed tokens per second.

Those are different denominators and can rank systems differently.

A throughput claim must say which one is meant.

73.24 Goodput

Suppose requests have latency objective τ .

One possible request goodput is:

$$G_\tau = \frac{|\{i : L_i \leq \tau\}|}{T}.$$

A system can increase raw throughput while violating more latency objectives.

Thus:

throughput $\neq$ SLO goodput.

73.25 Average latency and tail latency

Mean latency is:

$$\bar{L} = \frac{1}{N} \sum_i L_i.$$

A p99 statistic is an order statistic.

The mean does not determine p99.

A system can improve the average while making rare long waits worse.

Tail behavior therefore requires its own evidence.

73.26 Iteration-level scheduling

Autoregressive requests can have different remaining lengths.

A rigid batch can force completed or short requests to wait behind longer work, depending on system semantics.

Orca introduced iteration-level scheduling, with scheduling decisions at model-iteration granularity, and selective batching in its reported system.

The Atlas imports the mechanism and the measured-example status.

It does not import Orca’s reported speedup as a universal constant.

73.27 Batching is a scheduling policy

Batching can increase arithmetic efficiency and throughput by presenting more parallel work.

It can also increase waiting time before service.

The relevant trade depends on:

- arrival pattern;
- batch formation;
- request lengths;
- device occupancy;
- memory capacity;
- service objectives.

Therefore:

larger batch $\nrightarrow$ better serving under every metric.

73.28 KV-cache state

During autoregressive Transformer serving, previously computed key/value state can be retained so later token steps do not recompute the entire prefix attention state.

For request i , use generic accounting:

$$M_i = \ell_i b_i,$$

where:

- ℓ_i is retained sequence length;
- b_i is implementation-specific cache bytes per retained position.

Then:

$$M_{\text{KV}} = \sum_i M_i.$$

The value b_i depends on architecture, precision, sharding, and layout.

73.29 Cache allocation can limit concurrency

Even if model weights fit, active-request cache state can constrain how many requests remain resident.

Fragmentation and duplication can further reduce usable capacity.

Kwon et al. identify KV-cache memory-management inefficiency as a serving bottleneck and introduce PagedAttention/vLLM as one design for mitigating it.

Their measured gains are evidence about the reported implementation and workloads.

They are not a hardware-independent theorem.

73.30 Prefill and decode are not the same workload

Processing an input prompt can expose substantial parallel work over many token positions.

Autoregressive decode advances one or a small number of positions per active request per iteration.

The arithmetic shape, memory behavior, and batching opportunity can differ.

A serving benchmark that reports only one aggregate throughput number can hide this difference.

73.31 Utilization

A device can be highly utilized while doing work that is not useful under the service objective.

Examples include:

- padded work;
- work for requests that miss an SLO;
- excess recomputation;
- synchronization overhead;
- speculative work later discarded.

Utilization is a resource-activity metric.

It is not automatically useful-work efficiency.

73.32 Communication and memory interact

Tensor or expert parallelism can reduce per-device parameter memory.

That can permit a larger model or larger request batch.

But it can also increase communication.

Conversely, replication can reduce communication on one path while consuming more memory.

Systems design is therefore multi-objective.

73.33 Placement is an optimization variable

Placement decides where shards, stages, experts, replicas, and cache state live.

The exact ring witness showed that placement alone can change cross-cut bytes.

In larger systems, placement can interact with:

- heterogeneous links;
- shared switches;
- NUMA effects;
- rack boundaries;
- expert traffic;
- request locality.

The Atlas does not prescribe one placement algorithm.

It requires the placement to be part of the claim.

73.34 Measured versus modeled performance

A model may predict:

- bytes;
- message count;
- lower bounds;
- asymptotic scaling;
- critical-path structure.

A benchmark measures a concrete realization.

A mismatch is not automatically a failure of arithmetic.

It may reveal omitted effects.

The correct response is to name those effects rather than quietly relabel a model as a measurement.

73.35 Training benchmark record

A serious training record should identify:

- model/workload;
- batch semantics;
- precision;
- optimizer/update;
- hardware count/type;
- topology;
- parallelism degrees;
- sharding;
- warmup;
- measurement interval;
- throughput unit;
- scaling baseline.

Without these, “X% scaling efficiency” is underspecified.

73.36 Serving benchmark record

A serious serving record should identify:

- model and precision;
- device count/type and topology;
- request/input-length distribution;
- output-length distribution;
- concurrency or arrival process;
- scheduler/batching;
- KV-cache policy;
- latency percentiles;
- throughput/goodput definition;
- SLO if used;
- measurement duration.

Without these, “requests per second” cannot be interpreted safely.

73.37 What the sources establish

Narayanan et al. provide primary evidence for composed data/tensor/pipeline training and their measured trade-offs in Megatron-LM.

Huang et al. provide primary evidence for microbatch pipeline execution in GPipe.

Yu et al. provide primary evidence for iteration-level scheduling/selective batching in Orca.

Kwon et al. provide primary evidence for KV-cache memory-management pressure and PagedAttention/vLLM.

These sources demonstrate representative mechanisms.

They do not close the design space.

73.38 The systems layer is not one scalar

A distributed system can be described by a resource/performance vector such as:

$$K = (W, Q_{\text{local}}, Q_{\text{net}}, M, L_{50}, L_{99}, \text{Th}, G, U).$$

Here the entries might denote arithmetic work, local bytes, network bytes, memory, median/tail latency, throughput, goodput, and utilization.

The exact vector depends on the task.

Collapsing it to one score requires a declared objective.

73.39 Failure boundaries

The durable non-implications are:

more devices $\nRightarrow$ linear speedup,

equal FLOPs $\nRightarrow$ equal communication,

equal per-rank bytes $\nRightarrow$ equal cross-cut traffic,

high throughput $\nRightarrow$ low latency,

low average latency $\nRightarrow$ low tail latency,

high utilization $\nRightarrow$ high useful-work efficiency,

training efficiency $\nRightarrow$ serving efficiency.

These are not pessimistic slogans.

They are type checks for systems claims.

73.40 Systems reasoning

The chapter’s systems method is:

1. name the computation;
2. name the partition;
3. name the communication semantics;
4. name the physical placement/topology;
5. account for bytes and memory;
6. name the schedule;
7. define latency/throughput/tail metrics;
8. distinguish models from measurements;
9. preserve the workload and benchmark boundary;
10. compare alternatives only under declared objectives.

That method scales better than treating “distributed” as a performance adjective.

73.41 References used in this chapter

- Deepak Narayanan et al. (2021), *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*, arXiv:2104.04473.
- Yanping Huang et al. (2019), *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism*, arXiv:1811.06965.
- Gyeong-In Yu et al. (2022), *Orca: A Distributed Serving System for Transformer-Based Generative Models*, OSDI 22, pp. 521–538.
- Woosuk Kwon et al. (2023), *Efficient Memory Management for Large Language Model Serving with PagedAttention*, arXiv:2309.06180.

Exact source identities, inherited prerequisite authority, and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-SYSTEMS-001.yaml`

Chapter 74

Experiments as Arguments

Epistemic status: Atlas synthesis of established experimental-design, causal-inference, statistical-reporting, reproducibility, and machine-learning experimental-method sources, with one exact finite computational witness.

Specification: manuscript/specifications/ATLAS-CH-EXPERIMENT-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-EXPERIMENT-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-EXPERIMENT-001.md

Source lock: sources/source-locks/ATLAS-CH-EXPERIMENT-001.yaml

A machine-learning experiment can be expensive, carefully logged, repeated across seeds, and still fail to answer the question its authors think it answers.

That is not because experiments are useless.

It is because **a run is not yet an argument.**

A run produces observations.

An argument explains why those observations bear on a particular claim.

The distinction is easy to lose when the object on the screen is a single number: accuracy, loss, reward, latency, FLOPs, energy, perplexity, exact-match rate.

Numbers look decisive.

But before a number can carry evidentiary force, we must ask what was changed, what stayed fixed, what population or task family the comparison refers to, what randomness was sampled, what nuisance variables moved at the same time, what selection process chose the reported result, and what claim the result is actually allowed to support.

This chapter develops experiments as **structured claim-support objects**.

The governing idea is:

an experiment is a designed contrast whose evidentiary force comes from the relation among the claim, the estimand, the intervention, the comparison, the variation model, and the scope.

The chapter does not propose one universal statistical recipe.

Different scientific questions need different designs.

What must remain stable is the logical discipline connecting design to claim.

74.1 From metric to argument

Suppose two systems are evaluated and one obtains score 82 while the other obtains score 79.

What has been established?

At minimum, under the exact evaluation protocol that produced those numbers, one recorded outcome is larger than the other.

That statement may be useful.

But many stronger statements do not follow automatically.

It does not yet establish:

- that the architectural difference caused the improvement;
- that the improvement persists across datasets;
- that the improvement persists across seeds;
- that the effect is practically important;
- that a named mechanism explains the difference;
- that the result generalizes outside the benchmark;
- that the experiment would survive a fresh implementation;
- that the comparison was selected before seeing the results.

The Evidence chapter already supplied the Atlas rule that support must retain its scope.

EXPERIMENT-001 specializes that rule for empirical and computational comparisons.

We will write an experiment-argument object as

$E = (q, \text{theta}, U, A, Z, Y, g, V, \text{rho}, \text{Omega})$.

The notation is deliberately compact.

It forces ten questions into view.

- q : What bounded claim is under examination?
- theta : What quantity is the experiment meant to estimate?

- **U**: What units, tasks, datasets, or population does the design concern?
- **A**: What intervention or assignment rule creates the comparison?
- **Z**: What nuisance variables, blocks, or controlled conditions matter?
- **Y**: What outcome is measured?
- **g**: What estimator or comparison rule converts observations into a contrast?
- **V**: What sources of variation or uncertainty are represented?
- **rho**: What stopping, tuning, selection, and reporting rule generated the reported result?
- **Omega**: What scope is the conclusion allowed to inhabit?

This tuple is an Atlas synthesis.

It is not an externally standardized definition of experiment.

Its purpose is diagnostic: if a load-bearing field is missing, the argument may be weaker than the prose suggests.

74.2 An intervention is not an observation

The NIST engineering-statistics handbook begins from deliberate variation of factors and observation of responses, and treats nuisance factors, blocking, randomization, and replication as central design concerns (National Institute of Standards and Technology, n.d.).

That intervention logic matters because observational association and intervention answer different questions.

Suppose high-capacity models obtain higher scores than low-capacity models in a collection of published results.

That is an association.

Perhaps capacity helps.

Perhaps the high-capacity models were also trained on more data, tuned more aggressively, evaluated later in the field's history, or implemented by teams with more compute.

The observation does not identify which difference produced the outcome.

A designed intervention attempts to break that ambiguity.

Change the factor of interest while holding relevant alternatives fixed, balancing them, randomizing them, or modeling them explicitly.

The resulting contrast can then bear more directly on the intended claim.

Randomization is one important mechanism for doing this, but it is not magic.

It addresses assignment-related confounding under the design.

It does not automatically repair:

- bad measurement;
- coding defects;
- selective reporting;
- inappropriate outcomes;
- protocol drift;
- hidden preprocessing;
- nonrepresentative sampling;
- generalization beyond the study scope.

Experimental control is therefore specific.

One must always ask: **what source of ambiguity did this design actually remove?**

74.3 Hypothesis versus estimand

A hypothesis is often verbal:

the intervention improves performance.

The phrase “improves performance” is not yet a mathematical object.

Improve what?

Over which population?

By what aggregation?

Under which resource budget?

Relative to what alternative?

An **estimand** makes the target explicit.

In a simple finite causal setting, imagine units indexed by $i=1, \dots, n$.

Each unit has two potential outcomes:

- $Y_i(1)$: the outcome under treatment;
- $Y_i(0)$: the outcome under control.

The finite-population average treatment effect is

$$\Delta = (1/n) \sum_i [Y_i(1) - Y_i(0)].$$

Rubin’s treatment-effect framework made the benefits of randomization and carefully controlled nonrandomized comparisons explicit in this language of treatment effects and assignment (Rubin 1974).

The difficult point is immediate.

For a particular unit, we observe only the outcome corresponding to the treatment actually assigned.

The other outcome is counterfactual.

So an experiment does not merely “measure the effect.”

It constructs a design under which observed contrasts can support claims about an otherwise unobserved comparison.

This is why an estimand is not the same thing as a metric.

Accuracy can be a metric.

“The average change in accuracy caused by replacing component X with Y under a fixed training and evaluation protocol” is an estimand.

The difference is not stylistic.

It determines what the experiment is trying to learn.

74.4 Baselines are comparators; ablations are interventions

Machine-learning papers often use the words **baseline** and **ablation** as if they were interchangeable.

They are not.

A baseline is any comparator.

An ablation is a structured intervention on a component of a reference system.

If a model S contains component c , an ablation might compare S against $S_{\{-c\}}$, where the component is removed or replaced.

The intended mechanism claim is something like:

the presence of component c contributes to the observed behavior.

But the validity of that interpretation depends on what else changes.

Removing a component might also change:

- parameter count;
- compute;
- activation scale;
- optimization stability;
- training time;
- receptive field;
- memory footprint;
- initialization;
- interface shape.

If those effects are not controlled, the ablation is not a pure intervention on one semantic mechanism.

It is a compound intervention.

This does not make the experiment worthless.

It changes the claim.

A compound ablation can show that one system configuration differs from another.

It may not isolate why.

The useful rule is:

an ablation supports mechanism evidence only to the degree that the intervention isolates the candidate mechanism from relevant alternative changes.

74.5 The confounding trap

A comparison can be numerically exact and logically wrong for the intended question.

Consider two nuisance strata: easy and hard.

Every easy unit has potential outcomes

- control: 10;
- treatment: 11.

Every hard unit has

- control: 0;
- treatment: 1.

Treatment helps every unit by exactly 1.

So the exact finite-population average treatment effect is +1.

Now assign units badly.

The treatment group receives:

- 1 easy unit;
- 9 hard units.

The control group receives:

- 9 easy units;
- 1 hard unit.

The treatment mean is

$$(11 + 9 \cdot 1) / 10 = 2.$$

The control mean is

$$(9 \cdot 10 + 0) / 10 = 9.$$

The naive aggregate contrast is therefore

$$2 - 9 = -7.$$

The aggregate says treatment is worse.

But within the easy stratum,

$$11 - 10 = +1.$$

Within the hard stratum,

$$1 - 0 = +1.$$

If we compare treatment and control within strata and then standardize with equal stratum weights, the contrast is +1.

The exact computational witness for this chapter reproduces those values.

Nothing probabilistic is hiding in the example.

The sign reversal is caused entirely by imbalance in the nuisance variable.

This is a minimal model of confounding.

NIST's blocking guidance describes the corresponding design idea: nuisance factors can be held constant within blocks so that the factor of interest can be compared without mixing the block effect into the contrast (National Institute of Standards and Technology, n.d.).

The lesson is not "always stratify."

The lesson is:

aggregate differences answer aggregate questions. If treatment assignment changes the mixture of nuisance conditions, the aggregate contrast may not estimate the treatment effect you intended.

74.6 Controls do not all play the same role

The word **control** can refer to several distinct devices.

A control group may represent the no-treatment alternative.

A baseline model may represent the incumbent method.

A placebo-like condition may preserve everything except the hypothesized active mechanism.

A negative control may be chosen because no effect is expected.

A positive control may verify that the measurement system can detect a known effect.

A blocked design may control nuisance variation by comparing like with like.

These are not synonyms.

The useful question is not “does the experiment have a control?”

It is:

which alternative explanation is this control designed to eliminate?

A good control has a job.

If its job is not stated, the reader cannot tell what ambiguity it resolves.

74.7 Counterfactuals are the missing half of the comparison

Whenever we claim that an intervention **caused** an outcome, we are implicitly comparing what happened with what would have happened otherwise.

That “otherwise” is counterfactual.

In randomized experiments, the design uses other assigned units to estimate the missing alternative.

In matched or blocked observational studies, the design attempts to build a credible comparison class.

In simulations, the counterfactual may be directly computable because the same synthetic state can be rerun under two interventions.

In machine-learning ablations, the counterfactual may be “the same training/evaluation process without this component.”

But the phrase **the same** is doing work.

If the counterfactual run changes multiple factors, the claimed mechanism becomes ambiguous.

A counterfactual therefore requires an invariance contract:

which parts of the world are intended to remain fixed while the intervention changes?

The stronger the causal interpretation, the more carefully that contract must be defended.

74.8 Seeds: useful, narrow, often overinterpreted

Random seeds are important in modern machine learning because many pipelines contain stochastic elements:

- parameter initialization;
- minibatch ordering;
- dropout masks;
- augmentation;
- environment trajectories;
- randomized search;
- sampling.

Henderson et al. documented how nondeterminism, algorithmic variance, baseline choice, and reporting practice complicate interpretation in deep reinforcement learning (Henderson et al. 2018).

The general lesson is broader than RL, but the scope must remain explicit.

Suppose an outcome can be written schematically as

$$Y = F(\mathbf{a}, \mathbf{d}, \mathbf{s}, \mathbf{h}, \mathbf{e})$$

with

- algorithm $\mathbf{a}$;
- data $\mathbf{d}$;
- seed $\mathbf{s}$;
- hyperparameter/search choice $\mathbf{h}$;
- environment and implementation $\mathbf{e}$.

If we vary only $\mathbf{s}$, we learn about run-to-run variation conditional on $\mathbf{a}, \mathbf{d}, \mathbf{h}, \mathbf{e}$.

That can be valuable.

But it does not automatically estimate:

- uncertainty across datasets;
- shift in the target population;
- variability across independent implementations;
- sensitivity to preprocessing;
- uncertainty induced by hyperparameter search;
- uncertainty about the scientific mechanism.

“Five seeds” is therefore not a universal unit of experimental reliability.

It is a statement about one coordinate of variation.

The experiment should say what that coordinate represents.

74.9 Selection rules are part of the experiment

Suppose we train 100 configurations and report the best one.

The reported score is not the observation from a prespecified configuration.

It is the maximum of a search process.

If each observed score is

$$X_j = \mu_j + \epsilon_j,$$

and the selected run is

$$j^* = \operatorname{argmax}_j X_j,$$

then the reported statistic is

$$\max_j X_j.$$

That selection changes its interpretation.

The same principle applies when we choose:

- the best seed;
- the best checkpoint;
- the best prompt;
- the best benchmark subset;
- the best hyperparameter sweep;
- the best architecture after many failed variants.

Selection is not forbidden.

Hidden selection is the problem.

The Atlas therefore places stopping, tuning, selection, and reporting inside the experiment object as `rho`.

If `rho` changes after observing outcomes, the experimental argument changes too.

74.10 Effect size is not statistical significance

A statistically significant result can be tiny.

A practically important effect can fail a significance threshold in a small or noisy study.

The American Statistical Association’s statement on p-values explicitly separates p-values from effect size and practical importance, and warns against making scientific conclusions depend only on threshold crossing (Wasserstein and Lazar 2016).

For the Atlas, this yields a simple discipline.

When an experiment supports a quantitative claim, report the quantity of interest.

Do not replace it with the fact that some test crossed a threshold.

Depending on the design, useful objects may include:

- mean difference;
- relative change;
- risk difference;
- odds ratio;
- standardized difference;
- confidence or credible interval;
- predictive interval;
- distributional comparison;
- task-level effect distribution.

The correct object depends on the estimand.

The principle does not.

Magnitude and uncertainty should remain visible.

A p-value may be part of the argument.

It cannot stand in for the argument.

74.11 Benchmark score versus mechanism evidence

Suppose a new architecture beats a baseline on five benchmarks.

That is evidence about comparative performance under those benchmark protocols.

It is not automatically evidence for the proposed explanation.

A model may improve because of:

- the claimed architectural mechanism;
- increased compute;
- parameter count;
- better optimization;
- hidden regularization;
- preprocessing;
- data leakage;
- implementation quality;
- changed training duration;
- favorable hyperparameter search.

Mechanism evidence requires a design that separates the proposed mechanism from plausible alternatives.

This is why a chapter about experiments must treat ablations, controls, and counterfactuals as structural objects rather than publication rituals.

A benchmark answers:

what happened under this benchmark protocol?

A mechanism experiment asks:

what change caused or mediated the difference?

Those are related questions.

They are not the same question.

74.12 Reproducibility is valuable and not sufficient

The National Academies report adopts a useful terminology (National Academies of Sciences, Engineering, and Medicine 2019).

In that report:

- **reproducibility** means obtaining consistent computational results using the same input data, code or methods, and analysis conditions;
- **replicability** means obtaining consistent results across studies aimed at the same scientific question using newly obtained data.

The distinction matters because reproducibility is often mistaken for validity.

A perfectly reproducible pipeline can reproduce a flawed analysis exactly.

It can reproduce:

- a confounded comparison;
- a data leak;
- a coding bug;
- a biased sample;
- a misleading metric;
- a selectively chosen result.

Reproducibility answers an identity-and-reexecution question:

can the computation be reconstructed and obtain consistent results?

Validity asks whether the design and reasoning support the claim.

Replication asks whether a fresh study aimed at the same scientific question yields consistent findings, subject to its own uncertainty and design.

Generalization asks whether the result survives beyond the original context.

These are different evidentiary axes.

No one axis should silently substitute for the others.

74.13 Falsifiability belongs in the design

A claim becomes experimentally useful when the design makes clear what result would count against it.

That does not require reducing every scientific claim to one null-hypothesis test.

It does require a rejection, revision, or failure condition.

For example:

- if a claimed component matters, an isolated ablation should change the target outcome in a declared direction or range;
- if a scaling law is claimed, specified deviations should count against the proposed law;
- if a mechanism is claimed to improve stability, instability metrics should be defined before observing the candidate;
- if two methods are claimed equivalent, a tolerance and comparison domain should be stated.

A design that cannot lose is not a strong experimental argument.

A useful experiment risks changing our mind.

74.14 Repetition, replication, and robustness

Running the same code twice is not the same as repeating a randomized study.

Running more seeds is not the same as collecting new data.

Testing more benchmarks is not automatically the same as replication.

Using another implementation is not automatically the same as generalization.

The words matter because they describe different sources of variation.

A mature experimental report should say which axes were varied.

For example:

- same code, same data, same seed: deterministic replay check;
- same code and data, different seeds: stochastic execution variation;
- same method, new dataset sampled from same frame: replication-like evidence;
- new implementation, same intended method: implementation robustness;
- shifted population or domain: external generalization;
- altered nuisance conditions: robustness or stress testing.

The more precisely these are named, the less likely one form of evidence is to be mistaken for another.

74.15 The experiment table

A practical way to force the argument into view is to write the experiment as a table before running it.

Field	Question
claim q	What proposition may change status?
estimand θ	What quantity is the experiment trying to learn?
units U	What population, tasks, examples, environments, or runs are sampled?
intervention A	What is deliberately changed or assigned?
nuisance Z	What else may affect the outcome?
outcome Y	What is measured and why is it relevant?
comparison g	How are observations converted into the target contrast?
variation V	Which uncertainty sources are sampled or estimated?
selection ρ	How are stopping, tuning, and reporting determined?
scope Ω	Where may the conclusion be applied?

This table is not a bureaucracy.

It is a compression of the experimental argument.

If the table cannot be completed, that is useful information before compute is spent.

74.16 Failure modes

Experiments fail in characteristic ways.

74.16.1 Confounded comparison

The treatment groups differ in nuisance factors that also affect the outcome.

74.16.2 Baseline asymmetry

One method receives more tuning, compute, data, or engineering effort.

74.16.3 Hidden multiplicity

Many hypotheses or configurations are tried, but the report presents the selected result as if it were prespecified.

74.16.4 Metric substitution

The measured metric drifts away from the actual scientific or engineering objective.

74.16.5 Seed laundering

Seed variation is presented as if it covered data, population, or implementation uncertainty.

74.16.6 Ablation overclaim

A compound system change is interpreted as evidence for one isolated mechanism.

74.16.7 Reproducibility overclaim

Exact rerun is presented as evidence that the scientific conclusion is valid.

74.16.8 Benchmark overclaim

Performance on a fixed benchmark is promoted into a universal capability or mechanism claim.

74.16.9 Scope drift

A result established on one domain is written as if it held across a broader population.

These failures have one family resemblance:

the claim becomes stronger than the design.

74.17 Experiments in adaptive-intelligence research

Adaptive systems make experimental design unusually difficult because the object under study may itself change its behavior in response to:

- data order;
- feedback;
- memory;
- routing;

- learned state;
- tool outputs;
- environment dynamics;
- resource constraints.

A static A/B comparison may therefore hide path dependence.

For these systems, the unit of intervention may need to be:

- an episode;
- a trajectory;
- a task sequence;
- a training curriculum;
- a memory state;
- a coordination graph;
- a bounded agent policy.

The central discipline remains unchanged.

Name the unit.

Name the intervention.

Name the comparison.

Name the sources of variation.

Name the stopping rule.

Name the scope.

Then state only the claim that this design supports.

74.18 What this chapter establishes

This chapter does not certify any empirical architecture.

It establishes a reading and design grammar.

A downstream reader may now ask of any experiment:

1. What is the claim?
2. What is the estimand?
3. What creates the contrast?
4. What nuisance variables remain?
5. What kind of baseline or ablation is used?
6. Which randomness is sampled?
7. What effect magnitude is observed?
8. What uncertainty is represented?
9. How was the result selected?
10. What is reproducible?
11. What is replicated?

12. What mechanism, if any, is actually identified?
13. Where does the claim stop?

The exact confounding witness demonstrates why these questions matter.

A clean number can point in the wrong direction when the design mixes treatment with nuisance structure.

The cure is not statistical ornament.

It is experimental clarity.

74.19 Atlas handoff

Replayable Evidence Objects — ATLAS-CH-REPLAY-001.

Replay may assume that an experiment has a declared claim, estimand, intervention/comparison structure, variation model, selection rule, and scope. Replay can then bind the exact data, code, environment, parameters, seeds, and outputs needed to reconstruct that experiment.

The dependency direction matters.

Replaying an experiment does not create its causal or scientific validity.

It preserves the experiment that was actually performed.

That preservation is powerful only when the experiment's argument was explicit in the first place.

74.20 References used in this chapter

- NIST/SEMATECH e-Handbook of Statistical Methods, experimental-design sections (National Institute of Standards and Technology, n.d.).
- Rubin, *Estimating causal effects of treatments in randomized and nonrandomized studies* (Rubin 1974).
- Wasserstein and Lazar, *The ASA Statement on p-Values: Context, Process, and Purpose* (Wasserstein and Lazar 2016).
- National Academies of Sciences, Engineering, and Medicine, *Reproducibility and Replicability in Science* (National Academies of Sciences, Engineering, and Medicine 2019).
- Henderson et al., *Deep Reinforcement Learning That Matters* (Henderson et al. 2018).

Exact source identities and authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-EXPERIMENT-001.yaml`

Chapter 75

Replayable Evidence Objects

Epistemic status: methodological synthesis grounded in reproducibility and provenance literature, exact GCL lifecycle records, and an executable Atlas replay package.

Primary figure: ‘ATLAS-FIG-REPLAY-001‘

Documentary packet: ‘mathematics/derivations/ATLAS-CH-REPLAY-001-DERIVATIONS.md‘

Computational witness: ‘mathematics/computational-witnesses/ATLAS-CW-REPLAY-001.md‘

75.1 A result should be reconstructable

A scientific result is often presented as a sentence:

We found X .

But the sentence is the end of a path.

Before the claim came:

- a source;
- a definition of the object;
- a method;
- code or derivation;
- an execution environment;
- an observation;
- an interpretation;
- some form of review.

If those links are lost, the claim may remain readable while the reason for believing it becomes opaque.

This chapter treats documentary provenance as part of the scientific object.

Its central distinction is:

replayability is a property of the evidence path.

Truth, verification, certification, and authority are separate properties.

They may depend on replay.

They are not created by replay.

75.2 The chain of custody

The useful allegory is a **chain of custody**.

An exhibit is useful only if its identity and transformation history remain traceable.

The correspondence is:

- evidence label $\leftrightarrow$ stable artifact identity;
- custody record $\leftrightarrow$ provenance chain;
- laboratory procedure $\leftrightarrow$ replay method;
- analyst conclusion $\leftrightarrow$ interpretation;
- formal disposition $\leftrightarrow$ review or certification state.

The limit is essential.

Scientific truth is not determined by legal procedure.

Institutional acceptance is not mathematical truth.

A correct theorem can exist before any institution recognizes it.

A perfectly documented computation can still be wrong.

The allegory teaches traceability only.

75.3 Reproducibility, replication, and Atlas replay

Terminology matters because similar words carry different expectations.

The National Academies distinguishes **reproducibility** from **replicability**. Reproducibility concerns obtaining consistent computational results using the same data, computational steps, methods, code, and analysis conditions. Replicability

concerns obtaining consistent results in a new study with new data (National Academies of Sciences, Engineering, and Medicine 2019).

The Atlas uses **replay** more narrowly:

an identity-bound reconstruction of a specified evidence path.

Replay may be internal or external.

Replay may be deterministic or stochastic.

Replay can be exact at the byte level or approximate at a result level, provided the contract states which.

The term is local to the Atlas/GCL context.

It is not proposed as a replacement for broader scientific terminology.

75.4 A provisional evidence object

The Atlas uses

$$E = (C, S, M, A, O, I, R)$$

as an explanatory model.

The fields are:

C = claim identity and exact wording,

S = source identities,

M = method, derivation, code, or proof object,

A = execution environment and artifact identities,

O = observations or outputs,

I = interpretation and claim boundary,

R = review, replay, adjudication, or certification records.

The tuple is not a universal schema.

It is a checklist against a recurrent failure:

“We reproduced it” without saying what “it” was.

75.5 Why claim identity comes first

Suppose two files contain the same output:

86.

One claim might be:

this candidate has score 86 under evaluator E .

Another might be:

86 is globally optimal.

The bytes are identical.

The claims are not.

The first may be supported by replaying an evaluator on an exact candidate.

The second requires a different argument entirely.

Therefore the evidence object begins with

C ,

the exact claim.

A replay object without claim identity is a pile of artifacts waiting for someone to overinterpret it.

75.6 Source identity

A source should be identified strongly enough that another actor can recover the same object.

A mutable URL is often insufficient.

A branch name can move.

A “latest” tag can change.

A webpage can be edited.

A dataset can be refreshed.

Stronger identities include:

- immutable commit SHA;
- Git blob identity;
- content digest;
- DOI plus exact version where applicable;
- archived source snapshot;
- source manifest binding multiple objects.

Content addressing does not prove correctness.

It answers a narrower question:

Are these the bytes we named?

That narrow answer is foundational.

75.7 Method identity

The method field M may contain very different kinds of support:

- a hand derivation;
- executable code;
- a proof assistant term;
- an experimental protocol;
- a finite search;
- a numerical solver;
- a statistical analysis;
- a symbolic computation.

The important point is not that every method becomes code.

It is that the method remains inspectable enough for its support role.

Sandve et al. give practical rules for reproducible computational research, including recording how results were produced and preserving exact intermediate and external information where needed (Sandve et al. 2013).

The Atlas extends that documentary instinct across computational and mathematical work.

75.8 Environment identity

A program is not executed in a vacuum.

Results may depend on:

- language version;
- library versions;
- compiler;

- architecture;
- precision;
- accelerator runtime;
- operating system;
- deterministic or nondeterministic kernels;
- external services;
- data state.

The necessary level of pinning depends on the claim.

A pure exact-integer computation may need little.

A floating-point distributed training run may need much more.

The goal is not maximal metadata for its own sake.

It is sufficient identity for the evidence path being claimed.

75.9 The tiny replay package

This chapter contains an actual executable witness.

The program

‘`mathematics/computational-witnesses/replay_examples/deterministic_fraction_sum.py`’
computes

$$\frac{1}{3} + \frac{1}{6} + \frac{1}{2}$$

using exact rational arithmetic.

The expected output is

$$\boxed{1/1}.$$

The program SHA-256 is

‘`e6e841bfe975f283ab948c4f7f9bbbf5ba488d267fda36edc1b270d5dcd062c1`’.

The expected-output SHA-256 is

‘`3117b181de4d46b7ff8adb4c78adec272c019160d4adf27a901e90ec114e1845`’.

The identity manifest records:

- code path;
- code hash;
- expected-output path;
- output hash;
- invocation;

- environment class;
- arithmetic model;
- claim boundary.

Atlas CI executes the program and compares the output byte-for-byte.

The mathematics is deliberately trivial.

The evidence mechanism is the object under study.

75.10 What successful replay establishes

For the tiny witness, successful replay establishes:

1. the committed program has the declared hash;
2. the expected output has the declared hash;
3. the program executes successfully in the CI environment;
4. its standard output matches the locked expected bytes.

That is a real result.

It does **not** establish:

- that every Atlas claim is reproducible;
- that the code implements some unstated theorem;
- that the program is independently reviewed;
- that the result is formally verified;
- that the result is certified.

A replay result should say exactly what passed.

75.11 A perfectly replayable wrong program

Imagine the program:

```
print("2+2=5").
```

Its source can be hashed.

Its environment can be pinned.

Its output can replay perfectly forever.

It remains mathematically wrong.

This is the most important counterexample in the chapter.

Replay can establish faithful reconstruction of a computation.

It cannot turn a false computation into a true claim.

75.12 A correct argument that is not replayable enough

Now take the opposite case.

A mathematician writes a correct argument on paper, but the crucial source is cited only as:

“the recent lemma from that preprint.”

The reasoning may be mathematically correct.

The documentary path may still be too weak for another actor to reconstruct efficiently.

Non-replayability does not imply falsehood.

It implies a provenance deficit.

This distinction prevents the Atlas from confusing documentation quality with truth.

75.13 Mutable dependencies

Suppose a replay instruction says:

download ‘model/latest’ and run the script.

Today, ‘model/latest’ resolves to bytes *A*.

Tomorrow, it resolves to bytes *B*.

The name did not change.

The object did.

A replay instruction tied only to the mutable name has lost identity.

Pinning an immutable revision repairs the identity problem.

It still does not prove that the pinned artifact is the **right** artifact for the claim.

Identity and semantics remain separate.

75.14 Seed is not environment

For a stochastic experiment, a random seed is valuable.

It is not always sufficient.

A seeded computation can still differ because of:

- PRNG implementation;
- library changes;

- accelerator nondeterminism;
- thread scheduling;
- reduction order;
- mixed precision;
- compiler transformations;
- external data drift.

Therefore

$$\text{seed} + \text{code} \neq \text{complete replay identity}$$

in general.

The environment record should match the sensitivity of the evidence.

75.15 Byte identity is not semantic identity

Consider a program that checks a property for every integer

$$1 \leq n \leq 100.$$

Suppose:

- the code hash matches;
- the environment matches;
- the output hash matches;
- a second actor replays it successfully.

Now suppose the manuscript says:

the property holds for every positive integer.

The replay is exact.

The claim is unsupported.

The failed object is the semantic bridge between observation O and interpretation I .

That is why an evidence object must bind both.

75.16 Formal proof of the wrong statement

Formal verification is a powerful support route.

Suppose a proof assistant verifies a theorem

$$T'.$$

If the intended human claim was

$$T,$$

and the bridge

$$T \leftrightarrow T'$$

has not been established, then the formal proof does not automatically settle T .

The kernel can be perfectly correct about the formal object while the human interpretation is wrong.

Formal verification therefore strengthens one part of the evidence path.

It does not abolish semantic review.

75.17 Finite verification generalized too far

A program may exhaustively check all cases in a finite set.

That can be an exact finite result.

If prose silently extends it to an infinite domain, the result has changed class.

For example,

$$P(n) \text{ verified for } 1 \leq n \leq 10^6$$

does not entail

$$\forall n \in \mathbb{N}, P(n).$$

Exact finite verification is strong evidence for the finite statement.

It is not a continuum or universal proof unless an additional theorem supplies the bridge.

75.18 Internal replay is not independent reproduction

Suppose the same author:

- writes the code;
- runs the code;
- reruns the code;

- confirms the same output.

That is useful regression evidence.

It is not independent reproduction.

Independence changes epistemic value only when the relevant independence is real and documented.

Questions include:

- Did the second actor reuse the same implementation?
- Did they use the same hidden assumptions?
- Did they reconstruct the mathematics independently?
- Did they share the same data-generation error?
- Was the evaluation oracle independently derived?

“Independent” is itself a claim that needs provenance.

75.19 Provenance as a graph

W3C PROV provides a general model of entities, activities, agents, and derivation relations (Moreau et al. 2013).

The Atlas specializes the idea into a scientific support path:

source $\rightarrow$ method $\rightarrow$ execution $\rightarrow$ observation $\rightarrow$ interpretation $\rightarrow$ claim.

Review and authority transitions are separate.

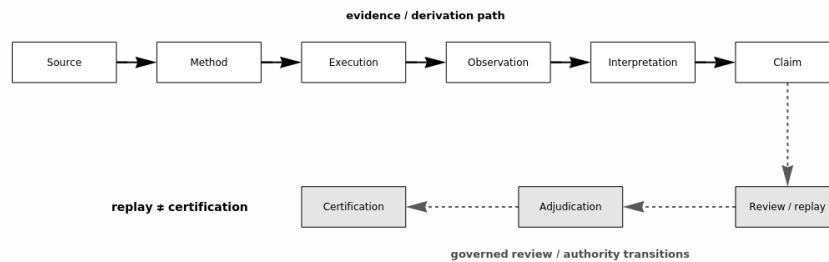


Figure 75.1: An exact declared graph with a solid evidence path from Source through Method, Execution, Observation, Interpretation, and Claim, followed by dashed separate transitions through Review or replay, Adjudication, and Certification.

Every node and directed edge in the figure is declared in the figure manifest.

The layout is presentation.

The separation of edge classes is content.

75.20 Evidence edges versus authority edges

This distinction is worth making explicit.

An evidence edge says something like:

this observation was produced by this execution.

An interpretation edge says:

this observation supports this bounded claim under this reasoning.

A review edge says:

this actor checked this aspect of the evidence path.

A certification edge says:

under this institution’s rules, this exact revision reached this authority state.

These are different relations.

Collapsing them into one arrow marked “verified” destroys information.

75.21 Current GCL technical-writing doctrine

The current GCL technical-writing reference states a programme principle:

technical communication is part of the research instrument.

Its writing invariants include:

- object identity;
- claim identity;
- bounded evidence;
- visible limitations;
- provenance;
- review specificity;
- authority separation;
- public inheritance;
- analogy discipline;
- fail-closed promotion.

The exact source is pinned in this chapter’s source lock to

‘grandchallenge/MATH-PROGRAMME@9c09521f0f7b1b7bbb2830b42f097f227f209d7d’.

The Atlas uses that doctrine as a concrete institutional example.

It does not claim that every scientific organization must use GCL's exact workflow.

75.22 Forge, Solve, Cert as role separation

GCL uses a Forge $\rightarrow$ Solve $\rightarrow$ Cert separation.

At a high level:

- **Forge** fixes source identity and problem/evidence context;
- **Solve** performs bounded mathematical or computational work and adjudication;
- **Cert** performs separate certification work under its own authority.

The value is not the names.

The value is that evidence production, problem solving, and certification are not silently treated as one operation.

This reduces one common failure mode:

the actor that produced a result implicitly promotes it to whatever authority level is convenient.

Role separation makes that promotion explicit.

75.23 The current OPENMATH lifecycle

The source-locked current OPENMATH controller records:

READY $\rightarrow$ LAUNCHED $\rightarrow$ RETURNED $\rightarrow$ CAPTURED $\rightarrow$ REPLAYED $\rightarrow$ ADJUDICATED $\rightarrow$ ADVANCED.

Its durable substrate is GitHub task and evidence state.

Its return grammar is

GCL-CONTRIBUTION-RESULT/1.

Its authority boundary is recorded as

MATHFORGE_TO_MATHSOLVE_TO_MATHCERT.

Most importantly for this chapter, the controller explicitly prohibits itself from:

- claiming mathematical correctness not established by Solve adjudication;
- performing MATHCERT certification.

That is documentary evidence of authority separation, not merely a slogan.

75.24 Replay evidence can exist while certification remains false

A current source-locked MATHCERT OPENMATH intake contains:

- source authority;
- source lock;
- statement identity;
- evaluator contract;
- proof receipt;
- search receipt;
- replay receipt;
- candidate byte identity;
- independent-executor requirements.

It also records:

`certification_effect:false.`

This is a particularly useful example because it blocks an easy conceptual collapse.

The presence of replay evidence does not itself create certification.

A certification transition is another event with another authority basis.

75.25 Fail closed

Suppose a replay requires:

- exact source identity;
- exact candidate identity;
- exact method identity;
- exact expected output.

If any of those identities disagree, the correct response is not to “approximately continue” while preserving the same status.

The support path has changed.

GCL’s current writing and OPENMATH doctrine uses a fail-closed pattern: identity or provenance mismatches block the applicable promotion.

That does not mean every scientific workflow must stop on every mismatch.

It means a workflow should not silently preserve an authority state whose prerequisites no longer hold.

75.26 Evidence object versus archive

An archive stores artifacts.

An evidence object stores relationships among artifacts and claims.

The difference is important.

A directory containing:

- ‘result.csv’;
- ‘final.py’;
- ‘notes.md’;
- ‘plot.png’

may be an archive.

It becomes stronger evidence when we can answer:

- Which code produced the CSV?
- Which source revision did the code use?
- Which rows support which claim?
- Which figure came from which data?
- What interpretation was applied?
- What review checked that interpretation?
- What version was actually certified?

Evidence is structured provenance plus bounded meaning.

75.27 The evidence object as a reconstruction test

The chapter’s acceptance test is simple:

Can another competent actor reconstruct why we believe this, and
can they see exactly what that reconstruction does not establish?

This is stronger than asking:

Are all the files present?

It asks for an intelligible support path.

For a computational result, another actor should be able to reconstruct the computation.

For a derivation, another actor should be able to reconstruct the mathematical steps.

For a literature-dependent claim, another actor should be able to recover the exact source.

For an institutional status, another actor should be able to identify the exact authority record.

75.28 Five distinctions to keep visible

75.28.1 Replay versus truth

Replay means the evidence path can be reconstructed under its stated conditions.

It does not make a false method true.

75.28.2 Replay versus replication

Replay may reuse the same source, data, and method.

Replication asks a different question and may use new data or an independently rebuilt method (National Academies of Sciences, Engineering, and Medicine 2019).

75.28.3 Replay versus formal verification

Formal verification checks a formal statement under a formal trust base.

Replay reconstructs an evidence path.

Either may support the other.

They are not synonyms.

75.28.4 Replay versus certification

Certification is a governed disposition under some authority model.

Replay can be an input to certification.

It is not certification by implication.

75.28.5 Certification versus truth

Institutional certification is a status granted under defined rules.

It does not redefine mathematical truth.

The Atlas must keep epistemic and institutional categories separate even when they interact.

75.29 What a durable evidence object should make inspectable

A mature evidence object should make it possible to inspect:

- exact claim;
- source identities;
- source revisions;
- code or derivation identity;
- environment;
- input identities;
- output identities;
- figure/data lineage;
- assumptions;
- limitations;
- negative evidence;
- replay status;
- review scope;
- independence claims;
- adjudication state;
- certification state;
- supersession history.

Not every result needs every field.

But a missing field should be a conscious absence, not an accidental one.

75.30 Atlas connections

Experiments as Arguments.

A controlled experiment becomes stronger when its exact support path can be replayed.

Formal Methods.

Formal proofs can enter the evidence graph as machine-checkable support objects without collapsing the semantic bridge.

Research as a State Machine.

Replay, adjudication, and certification become explicit state transitions.

Governed Adaptation.

A system that changes itself needs provenance strong enough to know which evidence applies to which revision.

Memory.

Institutional memory becomes more useful when it stores not only conclusions but also support paths.

The recurring Atlas shift is:

research result $\longrightarrow$ replayable evidence object.

75.31 Closing view

A claim is easier to trust when the path behind it remains visible.

But visibility is not truth.

A replayable wrong program is still wrong.

A correct theorem with poor provenance can still be correct.

A formally verified narrow statement can still be misinterpreted.

A successful replay can still await independent review.

A result can be well supported and still uncertified.

The chain-of-custody allegory is only an allegory.

The evidence object is the object.

And the final test remains:

Can another competent actor reconstruct why we believe this, and
can they see exactly what that reconstruction does not establish?

75.32 References used in this chapter

- (National Academies of Sciences, Engineering, and Medicine 2019)
- (Sandve et al. 2013)
- (Moreau et al. 2013)

See ‘sources/source-locks/ATLAS-CH-REPLAY-001.yaml’ for exact source identities, current GCL lifecycle records, and claim scope.

Chapter 76

Formal Methods and Machine-Checkable Claims

Epistemic status: established formal-methods foundations + audited replay/provenance prerequisite + Atlas synthesis + exact state-machine witness.

Specification: manuscript/specifications/ATLAS-CH-FORMAL-001.md

Formal packet: mathematics/derivations/ATLAS-CH-FORMAL-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-FORMAL-001.md

Source lock: sources/source-locks/ATLAS-CH-FORMAL-001.yaml

Formal methods are powerful because they force us to say exactly what is being claimed.

They are also easy to overstate because a machine can check the wrong formal statement perfectly.

The central problem is therefore not only:

can this theorem be checked?

It is also:

what object was formalized, what semantics does it use, what does the proof actually range over, what trusted machinery checks it, and how is the formal result connected back to the intended system?

The governing distinction of this chapter is:

machine-checked correctness is always correctness of a declared formal object under a declared trust base.

The semantic bridges around that object remain part of the scientific argument.

76.1 The formal support graph

Use the Atlas object set

$F = (R, S, M, P, K, I, W)$.

Here:

- **R**: intended requirement;
- **S**: formal specification/property;
- **M**: formal model and semantics;
- **P**: proof object, derivation, certificate, or model-checking result;
- **K**: trusted checker/kernel and surrounding trust base;
- **I**: implementation;
- **W**: deployed world or environment.

These objects should not be arranged as one untyped implication chain.

Instead, use typed obligations:

Formalizes(S,R)

asks whether the formal statement captures the intended requirement.

Interprets(M,S)

binds the statement to its formal semantics/model.

Checks(K,P,S,M)

records that checker K accepts support object P for the declared formal claim.

Conforms(I,M)

asks whether the implementation refines or conforms to the formal model strongly enough for the property being transferred.

AssumptionsHold(W,M)

asks whether the deployed world satisfies the environmental assumptions used by the model.

This graph prevents a common collapse.

A proof checker may establish **Checks**.

It does not automatically establish **Formalizes**, **Conforms**, or **AssumptionsHold**.

76.2 Formal specification is not human intent

Suppose the intended requirement is:

the controller must never issue an unsafe command.

A formal specification might encode:

`forall reachable s, command(s) in SafeSet.`

That may be precise.

But perhaps the formalization omitted:

- a sensor failure mode;
- an actuator delay;
- an environmental transition;
- a unit conversion;
- an emergency override.

The theorem can be completely correct relative to its formal world and still fail to establish the real intended requirement.

This is not a defect peculiar to theorem provers.

It is the ordinary scientific problem of model adequacy expressed sharply.

Formalization moves ambiguity into explicit definitions.

It does not make interpretation disappear.

76.3 Hoare logic: contracts as proof objects

Hoare's axiomatic account of programming introduced reasoning in the form

$\{P\} \ C \ \{Q\}$

for command C , precondition P , and postcondition Q (Hoare 1969).

Under partial-correctness semantics, the triple means:

if P holds before execution and C terminates, then Q holds afterward.

For

$x := x+2,$

a useful triple is

$\{\text{Even}(x)\} \ x:=x+2 \ \{\text{Even}(x)\}.$

If

$x=2k,$

then after the assignment

$x'=2k+2=2(k+1).$

The postcondition follows.

This small example contains the essential formal-methods pattern:

- state a contract;
- define semantics;
- prove the transition preserves the contract.

76.4 Invariants turn local preservation into global claims

Consider a transition system with:

- initial predicate $\text{Init}(x)$;
- next-state relation $\text{Next}(x, x')$;
- invariant candidate $\text{Inv}(x)$.

To prove that every reachable state satisfies Inv , establish:

1. initialization: $\text{Init}(x) \Rightarrow \text{Inv}(x)$;
2. preservation: $\text{Inv}(x) \text{ and } \text{Next}(x, x') \Rightarrow \text{Inv}(x')$.

Then induction over the reachability derivation gives the global invariant.

This is different from observing many successful executions.

The proof quantifies over every reachable state admitted by the formal transition system.

76.5 Exact witness: an infinite-state counter

Let

$x_0 = 0$

and

$x_{n+1} = x_n + 2$.

The claimed invariant is

$\text{Even}(x_n)$.

The base case is immediate:

$0 = 2 * 0$.

For the inductive step, if

$x_n = 2k$,

then

$x_{n+1} = 2k + 2 = 2(k + 1)$.

Therefore

$x_n = 2n$

for every natural number n .

Every reachable specified state is even.

This is a universal statement over an infinite family of executions/states.

76.6 Three tests are not that theorem

Now test only:

$0 \rightarrow 2, 2 \rightarrow 4, 4 \rightarrow 6$.

All pass.

These tests establish exactly those executions.

They do not, by themselves, establish the universal invariant.

The difference is logical, not rhetorical.

A finite list of successful examples has the shape:

$P(x_1), \dots, P(x_m)$.

The invariant theorem has the shape:

$\text{forall } x \text{ in Reachable, } P(x)$.

The quantifier changed.

An additional theorem is needed to bridge them.

76.7 A passing implementation can still be wrong

Define an implementation:

$\text{ImplStep}(x) = x+2$ when $x < 6$;

$\text{ImplStep}(x) = x+1$ when $x \geq 6$.

Starting from zero:

$0 \rightarrow 2 \rightarrow 4 \rightarrow 6$.

The three tests pass.

The next transition is:

$6 \rightarrow 7$.

The invariant fails.

At exactly the same state, the specification says:

6 $\rightarrow$ 8.

So three facts coexist:

1. the formal invariant proof is correct for the specification;
2. the tested implementation prefix passes;
3. the implementation does not conform to the specification.

This is the chapter's central exact counterexample.

76.8 Proof of specification is not proof of implementation

Suppose we prove:

$\text{Even}(x) \rightarrow \text{Even}(\text{SpecStep}(x))$.

That theorem says nothing directly about ImplStep .

To transfer the property, we need a bridge such as:

$\text{ImplStep}(x) = \text{SpecStep}(x)$

or a refinement relation strong enough to preserve the property.

Possible routes include:

- verified compilation;
- refinement proofs;
- equivalence proofs;
- code extraction;
- executable contracts;
- verified interpreters;
- constrained conformance testing.

Each route adds assumptions.

Formal methods do not remove bridges.

They make them inspectable.

76.9 Tests and proofs are complementary

Testing is not a weak version of proof.

It is a different support route.

Tests are excellent for:

- finding concrete counterexamples;

- exercising real implementations;
- regression checking;
- exposing integration failures;
- validating environment assumptions.

Proof is excellent for:

- universal claims over a formal domain;
- invariant preservation;
- exact algebraic properties;
- eliminating untested cases inside the formal model.

One failing test can refute a universal claim immediately.

Many successful tests usually do not establish the universal claim.

The asymmetry matters.

76.10 Exhaustive finite checking is stronger than sampling

Suppose a state space is genuinely finite.

If every reachable state is enumerated and every transition is checked, then a finite computation can establish a universal property over that finite model.

That is not the same as sampling.

The support route depends on:

- the exact finite model;
- transition semantics;
- completeness of exploration;
- property checked;
- correctness of the checker.

The word **finite** does not weaken the result.

It defines the result's domain.

76.11 Model checking

State-machine formal methods often write a system as:

- initial-state predicate;
- transition relation;
- safety properties;
- liveness/fairness assumptions where needed.

TLA+ is one mature framework for this style, and Lamport's *Specifying Systems* develops specification, safety/liveness reasoning, composition, and TLC model checking (Lamport 2002).

A model checker can explore all reachable states of a finite model.

That can be an exact result about the model.

It is still not automatically a proof about arbitrary deployed code.

The implementation/model bridge remains separate.

76.12 The model can be the wrong model

Imagine a distributed protocol model with perfect channels.

The safety theorem may hold.

The deployed system may lose, duplicate, or reorder messages in ways omitted from the formal model.

The theorem is not thereby false.

Its assumptions are simply not the deployment assumptions.

This distinction is essential:

a correct theorem with an inadequate model is still a correct theorem
about the inadequate model.

Formal verification therefore strengthens the need for explicit model identity.

76.13 Trusted kernels

Proof assistants introduce another architectural distinction.

Robin Milner's LCF work established the influential pattern of a small trusted proof core surrounded by more complex proof-producing machinery (Milner 1979).

The intuition is simple.

Tactics and automation may be sophisticated and buggy.

If they can only produce theorem objects through a small trusted interface, then bugs in automation should fail to manufacture accepted false theorems unless they compromise the trusted core or exploit assumptions already present in the logic.

This reduces the trusted computing base.

It does not eliminate trust.

76.14 Lean as a proof-assistant example

Lean is an interactive theorem prover based on dependent type theory with a small trusted kernel (Moura et al. 2015).

Its architecture separates convenient proof construction from final checking.

A user may rely on:

- notation;
- elaboration;
- type-class inference;
- tactics;
- automation;
- libraries.

The final elaborated object is checked by the kernel against the formal environment.

This is the key trust pattern:

automation may be large while the final logical checker remains small.

76.15 Small trusted core does not mean no trust

Even a small kernel has assumptions.

A formal result can depend on:

- the type theory or logic;
- explicit axioms;
- the kernel implementation;
- imported definitions;
- library theorems;
- parser/elaborator behavior if not independently reconstructed;
- compiler/runtime/hardware when executing extracted code;
- external certificates or oracles.

So “Lean checked it” is meaningful but incomplete.

A mature statement says:

- which theorem;
- under which imports/axioms;
- in which toolchain;
- checked by which kernel;
- with what semantic bridge to the intended claim.

76.16 Proof terms versus theorem meaning

In a propositions-as-types setting, a proof term inhabits the proposition representing the theorem.

Kernel acceptance therefore establishes a precise syntactic-semantic fact inside the formal environment.

But theorem meaning still depends on the chosen definitions.

If a theorem formalizes:

```
Safe := x < 100
```

while the intended policy meant:

```
Safe := x <= 10,
```

the kernel may correctly prove the wrong policy.

Machine checking is strongest when formal statement identity is preserved alongside the proof.

This directly inherits the claim-identity discipline from REPLAY-001.

76.17 Replayable proof is stronger than an unpinned proof artifact

REPLAY-001 established that source, method, environment, output, and interpretation should be identity-bound.

Formal proof benefits from exactly the same discipline.

A proof artifact can depend on:

- theorem prover version;
- library revision;
- imported modules;
- generated code;
- solver version;
- external certificate format.

If those dependencies float, another actor may not be able to reconstruct the check.

Thus:

```
formal proof
```

and

```
replayable formal proof
```

are distinct documentary states.

76.18 Replay is not proof

The converse is equally important.

A program can reproduce the same wrong output forever.

A checker can replay a finite search exactly.

A build can remain byte-for-byte stable.

None of those facts automatically imply a theorem.

REPLAY-001 already established this boundary.

FORMAL-001 adds the complementary rule:

proof and replay are orthogonal support dimensions.

A result can be:

- replayable but unproved;
- proved but poorly replayable;
- both;
- neither.

76.19 Executable checkers and proof objects

Not every machine-checkable result uses a proof assistant.

Some systems use compact certificates.

A producer may perform expensive search.

A smaller checker verifies the returned certificate.

Examples can include:

- SAT/SMT proof traces;
- primality certificates;
- optimization certificates;
- typed intermediate representations;
- proof-carrying code.

The epistemic advantage is architectural:

the producer can be less trusted if the checker fully validates the support object.

But the checker still validates only its declared relation.

76.20 Static types are formal claims with bounded scope

A type checker proves something.

What it proves depends on the type system.

A well-typed program may guarantee:

- certain operations receive values of the right form;
- some memory or effect discipline;
- absence of certain representation errors.

It does not automatically prove:

- business correctness;
- security policy;
- termination;
- liveness;
- numerical accuracy.

A formal mechanism should always be paired with its actual claim class.

76.21 Runtime contracts are path-local evidence

A runtime assertion such as

```
assert even(x)
```

can detect a violation on an executed path.

If the assertion never fires across many tests, that is useful evidence.

But unless all possible executions are covered or an additional theorem gives completeness, the assertion history remains execution-local.

This is another example of the quantifier discipline:

observed paths are not automatically all paths.

76.22 Property-based testing is still testing

Property-based testing can generate large numbers of structured inputs.

It often finds counterexamples that humans would not write manually.

This is valuable.

But if generation samples an infinite or enormous domain, passing campaigns remain sampling evidence.

If the generator is exhaustive over a finite domain, the claim class changes.

The difference should be stated, not implied.

76.23 SMT and SAT trust paths

Automated solvers are common inside formal methods.

There are several trust architectures:

- trust the solver directly;
- ask the solver to emit a proof trace;
- independently check the trace;
- reconstruct the result in a proof assistant;
- let an untrusted tactic invoke the solver and produce a kernel-checked term.

The phrase

the solver proved it

does not tell us which architecture was used.

The trust path belongs in the evidence record.

76.24 Formal proof can expose missing requirements

One practical benefit of formalization is that it forces vague language into explicit predicates and transitions.

That process often reveals that the requirement itself was incomplete.

Questions emerge:

- What exactly counts as failure?
- Which state variables matter?
- Is liveness required or only safety?
- Is termination required?
- Which environment actions are possible?
- What fairness assumption is being made?

Formal methods therefore have value even before a theorem is completed.

They refine the question.

76.25 Invariants are contracts over evolution

In adaptive systems, invariants are especially useful because the system changes over time.

Examples include:

- a resource bound is never exceeded;
- authority never escalates without a permitted transition;

- provenance is never discarded during promotion;
- a representation remains normalized;
- a safety region remains invariant;
- an evidence state cannot jump directly from unreviewed to certified.

This prepares the next chapter.

Research processes themselves can be modeled as governed transition systems.

76.26 Safety and liveness are different

A safety property says, roughly:

something bad never happens.

An invariant is a common safety-property form.

A liveness property says:

something good eventually happens.

A system can be perfectly safe by doing nothing forever.

Therefore a governance or research state machine often needs both:

- forbidden transitions must never occur;
- legitimate work must still be able to progress.

Formal verification should not hide this distinction under one word such as **correct**.

76.27 Formal certification is overloaded language

The word **certified** can mean several things:

- a theorem was machine checked;
- a proof certificate was checked;
- an implementation property was verified;
- a compliance body issued a certificate;
- an institution promoted an artifact into an authoritative state.

These are different events.

The Atlas reserves the stronger authority language for the actual governed transition.

A proof assistant accepting a theorem is not automatically institutional certification.

76.28 Machine-checked does not mean assumption-free

A theorem can use axioms.

A theorem can rely on imported propositions.

A model can assume fairness.

A verified controller can assume bounded disturbance.

A proof can be exact while its premises are conditional.

The appropriate statement is therefore:

theorem T is machine checked under assumptions A in formal environment E.

Conditional correctness is still correctness.

Its assumptions must remain visible.

76.29 The trust-base ledger

For a machine-checkable claim, record:

Field	Question
requirement	What human/system claim is intended?
formal statement	What exact proposition/property was encoded?
semantics	What model gives the symbols meaning?
proof/check	What derivation, trace, model-check run, or certificate supports it?
checker	What validates the support object?
axioms/imports	What is assumed rather than derived?
toolchain	What exact version/environment is needed to replay checking?
implementation bridge	Why does the deployed artifact satisfy the formal model?
environment bridge	Why do real conditions satisfy formal assumptions?
authority	What review/certification state was actually reached?

This ledger is the formal-methods analogue of REPLAY-001's evidence object.

76.30 Failure modes

76.30.1 Proof-of-the-wrong-specification

A theorem is valid but the formal predicate omits the real requirement.

76.30.2 Model-equals-system

A finite/abstract model is treated as identical to deployment.

76.30.3 Tests-equal-proof

A passing finite suite is promoted into a universal claim.

76.30.4 Automation-equals-trust

A tactic or solver is trusted merely because it is sophisticated.

76.30.5 Kernel-equals-no-assumptions

A small trusted kernel is treated as eliminating axioms and semantic assumptions.

76.30.6 Type-safe-equals-correct

A bounded static property is promoted into total system correctness.

76.30.7 Machine-checked-equals-certified

A proof check is treated as an institutional authority transition.

76.30.8 Formal-equals-replayable

A theorem artifact is accepted without pinning toolchain, imports, or source identity.

76.31 Why the exact witness matters

The counter witness is deliberately simple because the distinction is conceptual.

The invariant proof covers every specified reachable state.

The finite tests cover three implementation transitions.

The implementation then diverges.

Nothing subtle is happening.

The point is that all three evidence statements can be true simultaneously.

That makes the boundary hard to evade:

proof, testing, and conformance are different relations.

76.32 Downstream handoff

Research as a State Machine — ATLAS-CH-RESEARCHSM-001 may now assume:

- transition-system semantics;
- inductive invariants;
- initial/preservation proof obligations;
- safety versus liveness distinction;
- machine-checking versus semantic adequacy;
- trusted-kernel/trust-base discipline;
- replay and formal proof as complementary support routes;
- authority transitions as separate from theorem checking.

The downstream chapter must independently formalize:

- Forge -> Solve -> Cert;
- bounded work packages;
- independent actors;
- idempotence;
- promotion gates;
- failure/rollback states;
- authority transitions.

76.33 References used in this chapter

- Hoare, *An Axiomatic Basis for Computer Programming* (Hoare 1969).
- Milner, *LCF: A Way of Doing Proofs with a Machine* (Milner 1979).
- de Moura et al., *The Lean Theorem Prover (System Description)* (Moura et al. 2015).
- Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers* (Lamport 2002).

Exact source identities and authority boundaries are recorded in:

`sources/source-locks/ATLAS-CH-FORMAL-001.yaml`

Chapter 77

Research as a State Machine

Epistemic status: audited Replay/Formal prerequisites + public GCL workflow case study + Atlas synthesis + exact finite witness.

Specification: manuscript/specifications/ATLAS-CH-RESEARCHSM-001.md

Formal packet: mathematics/derivations/ATLAS-CH-RESEARCHSM-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-RESEARCHSM-001.md

Source lock: sources/source-locks/ATLAS-CH-RESEARCHSM-001.yaml

A sentence such as “the research task is done” hides too much.

A result can arrive without being preserved. A preserved artifact can remain unchecked. A successful replay can reconstruct a computation without deciding the surrounding claim. A bounded review can support a result without settling every later use of it.

This chapter replaces those ambiguities with explicit objects and transitions.

77.1 From evidence objects to research state

The Replayable Evidence chapter established that source identity, execution, observation, interpretation, review, and certification are different relations.

The Formal Methods chapter established that a transition system is useful only when its state, transition relation, invariants, and model boundary are explicit.

Research workflows need both ideas.

Evidence objects tell us what moves through the workflow.

A state machine tells us which changes are permitted, what each change means, and which facts remain unchanged when a transition fails.

The object of this chapter is therefore not a task list.

It is a typed lifecycle for evidence-bearing research objects.

77.2 A bounded work package

Let

$W = (\text{id}, Q, B, S, D, R, A, Z)$.

The coordinates record:

- stable work-package identity;
- bounded question or obligation;
- explicit bootstrap facts;
- allowed source universe;
- required deliverables;
- durable return route;
- authority granted to the worker;
- stop conditions and explicit non-authorities.

The point of this tuple is not administrative completeness.

It is semantic closure.

A competent contributor should be able to decide, from the package itself, what problem is being attempted, which premises are authorized, what a valid return must contain, and what the return is not allowed to decide.

77.3 The research state is a product

Represent the live research state as

$x = (q, E, U, J, P, C, L)$.

Here:

- q records execution or transport phase;
- E contains preserved immutable evidence identities;
- U records replay or checking state;
- J records adjudication;
- P records programme disposition;
- C records certification state;
- L is append-only history.

A representative transport path is:

READY -> LAUNCHED -> RETURNED -> CAPTURED -> REPLAYED -> ADJUDICATED.

That path is useful.

It is not the whole machine.

Programme disposition and certification are separate coordinates because neither is merely another name for successful transport.

77.4 State change requires a declared condition

For each event type, define a condition $g_e(x)$ and a partial state update $\delta_e(x)$.

The update is applied only when the declared condition holds.

A return can arrive without satisfying its schema. A schema-valid return can fail replay. A replayed result can fail adjudication. A positively adjudicated result can still await a separate programme decision.

The state machine keeps these cases separate.

77.5 Forge, Solve, and Cert are roles

The public GCL workflow uses Forge, Solve, and Cert as a separation of work.

At the level needed here:

- Forge fixes problem and source identity;
- Solve performs bounded mathematical or computational work and adjudication;
- Cert performs a separate certification function.

The names are project-specific. The structural lesson is that evidence production, adjudication, and later certification need not be one operation.

77.6 Receipt is not incorporation

Suppose a durable return reaches its declared route.

That establishes receipt.

It does not automatically establish structural validity, replay success, scientific correctness, programme incorporation, successor selection, or certification.

The earlier evidence chapters stated this boundary conceptually.

The state-machine view makes it operational: each distinction occupies a different coordinate or transition.

77.7 Idempotence belongs to canonical effect

A retryable research system needs two views of state.

The first is canonical state: which evidence objects and dispositions currently count.

The second is history: what attempts, retries, failures, and recoveries occurred.

Let $\text{Canon}(\mathbf{x})$ omit retry-log multiplicity.

For a stable event key $\mathbf{k}$, canonical idempotence means

$$\text{Canon}(\mathbf{F}_{\mathbf{k}}(\mathbf{F}_{\mathbf{k}}(\mathbf{x}))) = \text{Canon}(\mathbf{F}_{\mathbf{k}}(\mathbf{x})).$$

The history may still record both attempts.

This distinction matters.

If the entire history had to be identical after retry, the system would have to hide the fact that a retry occurred.

If canonical state were allowed to multiply with every retry, the same evidence could acquire multiple effects merely because transport repeated.

Neither behavior is desirable.

77.8 Duplicate is an identity relation

A return is not a duplicate merely because it belongs to the same work package.

Use the exact return identity

$$\mathbf{r} = (\mathbf{d}, \mathbf{h})$$

for dispatch identity $\mathbf{d}$ and immutable result identity $\mathbf{h}$.

Two returns are exact duplicates only when both coordinates agree.

If the dispatch is the same but the result identity differs, the programme has two evidence objects.

It may later decide that one supersedes, refutes, strengthens, or merely differs from the other.

But that semantic decision must not be smuggled into transport by treating every second return as “the same thing.”

77.9 Exact finite retry witness

Take dispatch

$$\mathbf{d} = 17$$

and result identity

$$\mathbf{r}_{\mathbf{A}} = (17, \mathbf{A}).$$

The first capture creates the singleton evidence set

$E_1 = \{r_A\}$.

Receive the identical result again.

Set union gives

$E_2 = E_1 \cup \{r_A\} = E_1$.

Therefore canonical evidence count remains

1.

Now introduce

$r_B = (17, B)$

with

$B \neq A$.

Then

$E_3 = \{r_A, r_B\}$

has cardinality

2.

The witness therefore separates retry from genuinely distinct evidence using only stable identity and finite set arithmetic.

77.10 Advancement needs its own guard

Receipt and replay are evidence-state transitions.

Programme advancement is a different act.

For one finite model, define five Boolean conditions:

- $I(e)$: exact identity and provenance are valid;
- $S(e)$: the structural return contract is valid;
- $R(e)$: the required replay or check has completed;
- $A(e)$: adjudication supports the bounded claim;
- $D(e)$: the programme has issued an explicit advancement disposition.

Then define:

$G_{adv}(e) = I(e) \wedge S(e) \wedge R(e) \wedge A(e) \wedge D(e)$.

This is a deliberately strict finite witness.

It is not presented as the only possible research-governance formula.

77.11 A missing programme disposition keeps the gate closed

Evaluate:

$b = (1, 1, 1, 1, 0)$.

Then:

$G_{adv}(b) = 0$.

The evidence may be well identified.

Its structure may be valid.

Replay may have succeeded.

Adjudication may even support the bounded claim.

The successor is still not authorized under this witness because programme disposition is absent.

This is exactly the distinction encoded by the public GCL Frontier Advancement Gate.

77.12 A complete witness vector opens the finite gate

Now evaluate:

$b' = (1, 1, 1, 1, 1)$.

Then:

$G_{adv}(b') = 1$.

The arithmetic is trivial.

The semantic point is not.

A transition becomes legal because every declared condition is satisfied, not because the underlying prose sounds persuasive.

77.13 A missing required check also keeps the gate closed

Take:

$b_R = (1, 1, 0, 1, 1)$.

Then:

$G_{adv}(b_R)=0$.

A strong result statement cannot substitute for a required check that the governing process explicitly declared mandatory.

This is one value of machine-readable workflow state:

missing conditions remain visible as missing conditions.

77.14 Advancement can be canonically idempotent too

Let:

K_{adv}

be the canonical set of evidence identities that have received the relevant advancement effect.

For identity m :

$K_{adv} \cup \{m\} \cup \{m\} = K_{adv} \cup \{m\}$.

So repeated exact application of the same advancement effect need not create a second canonical promotion.

The history may still record both attempts.

This matters whenever delivery, CI, merge automation, or network retries can repeat an otherwise valid event.

77.15 Idempotence is scoped, not magical

The word **idempotent** is easy to overuse.

This chapter means something narrow:

a declared canonical mutation keyed by stable identity does not multiply its canonical effect when the exact same event is replayed.

It does not mean:

- two independent reviews must reach the same judgment;
- rerunning a stochastic experiment must return identical measurements;
- policy changes have no effect;
- a new result identity should be discarded.

Idempotence is a property of a specified operation.

77.16 Different evidence under one dispatch is not a retry

Return:

`r_A=(17,A)`

and later:

`r_B=(17,B).`

Because:

`A != B,`

these are different immutable result identities.

The second object may:

- conflict with the first;
- strengthen it;
- supersede it under an explicit policy;
- address a different subclaim.

The transport layer should preserve that distinction instead of silently replacing one object by dispatch key.

77.17 Conflict is a scientific relation, not an intake shortcut

Suppose two preserved returns disagree.

The state machine should first preserve both identities.

Only then should adjudication decide the semantic relation.

This ordering matters:

`preserve -> compare -> adjudicate`

is different from:

`receive second -> overwrite first.`

The first route retains the evidence needed to understand disagreement.

77.18 Actor roles are not actor independence

Research workflows often name roles:

- contributor;
- reviewer;

- adjudicator;
- certifier.

Those labels are useful.

They do not prove independence.

Two differently named actors can share:

- information;
- incentives;
- training data;
- organizational control;
- hidden communication.

The chapter therefore treats independence as a governed relation, not a typography choice.

77.19 A narrow separation-of-duty predicate

Suppose one policy requires only that the producer and reviewer identities differ.

Define:

$\text{Sep}(a_{\text{prod}}, a_{\text{rev}}) = 1 \{a_{\text{prod}} \neq a_{\text{rev}}\}.$

Then:

$\text{Sep}(A, A) = 0$

and:

$\text{Sep}(A, B) = 1.$

This proves identity separation only.

It does not prove epistemic independence.

77.20 Same-actor review can fail a declared gate

If the governing policy requires:

$\text{Sep} = 1,$

then a review by the same actor who produced the artifact does not satisfy that particular condition.

This is a policy statement, not a universal moral claim about self-review.

Self-review can still be valuable.

It simply does not substitute for a distinct-actor condition when such a condition has been explicitly required.

77.21 Certification is another state change

Certification should not be hidden inside words such as accepted, merged, green, or adjudicated.

Let certification state be coordinate:

C.

A certification transition has its own exact target, support requirements, and authority predicate.

For example:

$G_cert(o) = T(o) \wedge S_cert(o) \wedge H(o)$,

where:

- $T(o)$ binds the exact target revision;
- $S_cert(o)$ binds the required support state;
- $H(o)$ binds the governing authority conditions.

A positive programme disposition does not automatically set certification state.

77.22 Exact target identity matters

A review of revision:

r_1

does not automatically certify later revision:

r_2 .

If r_2 changes the load-bearing artifact, the target identity has changed.

The state machine should bind evidence, replay, adjudication, and certification to the exact object they support.

This prevents stale evidence from leaking across revisions.

77.23 Forge, Solve, and Cert can be understood as authority compartments

The public GCL lifecycle controller records:

MATHFORGE_TO_MATHSOLVE_TO_MATHCERT

as an authority boundary.

At the level needed here:

- Forge fixes source/problem identity;

- Solve performs bounded work, evidence preservation, replay, and adjudication;
- Cert performs a distinct certification function.

The chapter does not claim this three-part split is universally optimal.

It uses the public workflow as one concrete example of authority compartmentalization.

77.24 Frontier advancement is not contributor authority

The public Frontier Advancement Gate makes one boundary explicit:

a contributor may return a next residual.

That residual is evidence.

It is not programme scheduling authority.

The programme separately chooses a disposition such as advance, independently verify, formalize, adversarially attack, source-block, refute, or close without successor.

This distinction prevents a useful contributor from silently becoming the scheduler of protected research state.

77.25 Independent-intelligence intake is an authority boundary

The public Controlled Epistemic Interface describes a narrow crossing:

`dispatch -> bounded contribution -> immutable intake -> separate adjudication -> protected route.`

Its compact boundary is:

`receive != believe != admit != certify.`

That is a state-machine statement.

Every verb denotes a different possible transition.

77.26 Zero-context is a work-context property

A bounded dispatch can be designed so that a contributor need not absorb the repository's local framing before attempting the task.

This can reduce one kind of institutional-context coupling.

It does not prove statistical independence, cryptographic independence, absence of shared pretraining, or absence of common mathematical culture.

The source itself makes this boundary explicit.

77.27 Fail closed means preserve protected state

Suppose a transition guard fails.

The default protected behavior in this chapter is:

do not perform the protected canonical mutation.

That does not mean nothing happens.

The system may still append a failure receipt, capture diagnostics, open a repair issue, preserve a candidate artifact, or retry an allowed check.

Fail-closed promotion and active recovery are compatible.

77.28 Safety is not liveness

This distinction is essential.

A system can preserve every safety invariant and still make no progress.

Consider a state in which evidence identities remain correct, duplicate retries do not multiply effects, invalid transitions are blocked, certification remains protected, and the advancement disposition stays false forever.

No declared safety rule is violated.

Yet the claim never advances.

Safety answers:

what bad transitions are forbidden?

Liveness answers:

what good transition eventually occurs?

They need separate arguments.

77.29 A finite safety-without-liveness witness

Let the machine be in an adjudicated state with:

$D(e)=0$.

Suppose the only future event available is an exact retry that leaves:

$D(e)=0$.

Every retry is canonically idempotent.

Every protected invariant remains true.

But:

`eventually(advanced(e))`

is false on this execution.

That is enough to show:

fail-closed safety alone does not guarantee progress.

77.30 Recovery needs typed classes

After a failure or block, “try again” is too vague.

At least five different operations should be distinguished.

77.30.1 Retry

The same event/evidence identity is attempted again.

77.30.2 New evidence

A different immutable result identity arrives.

77.30.3 Supersession

A governed relation explicitly marks one artifact as replacing another for a declared purpose.

77.30.4 Implementation repair

Tooling or workflow code changes while protected evidence identity remains intact.

77.30.5 Policy change

The governing guard or authority rule itself changes through an authorized route.

These operations have different provenance and authority meanings.

77.31 Recoverable tooling failure is not scientific refutation

A CI outage, rate limit, parser defect, or transient connector failure does not by itself falsify the research claim.

Likewise, a mathematical refutation is not a tooling failure.

The state machine should type these failures differently.

This prevents infrastructure noise from silently becoming epistemic judgment.

77.32 Protected repair requires replay

If implementation code changes after a failed run, old execution evidence should not be relabeled as evidence for the repaired implementation.

The repaired artifact needs fresh replay at its new exact identity when the governing process requires it.

This is the operational form of:

repair, then re-evaluate the repaired object.

77.33 Public lifecycle case study

The pinned GCL lifecycle controller records the concrete sequence:

READY -> LAUNCHED -> RETURNED -> CAPTURED -> REPLAYED -> ADJUDICATED
-> ADVANCED.

It also records durable GitHub evidence, protected merges, allowed and prohibited actions, fail-closed behavior on identity/provenance mismatch, and event-driven advancement plus a recovery poll.

This is evidence that one real workflow can be encoded as a state machine.

It is not proof that every research programme should use the same states.

77.34 The case study also exposes authority limits

The same controller explicitly prohibits actions such as claiming mathematical correctness not established by Solve adjudication, MATHCERT certification, topology redesign, and direct protected-main pushes.

That is important.

A controller's permissions are part of its semantics.

Automation should be described not only by what it can do, but also by what it is forbidden to do.

77.35 State machines make authority inspectable

An informal workflow often hides authority in convention.

A typed transition machine can make authority explicit through event type, actor, target, guard, and permitted mutation.

This does not make governance automatically correct.

It makes the governance claim inspectable.

77.36 The machine does not prove the science

Suppose every lifecycle transition is valid.

That establishes workflow conformance.

It does not establish mathematical truth, novelty, completeness of review, absence of omitted premises, correctness of every human judgment, or adequacy of the governance policy.

Formal workflow correctness and scientific correctness remain different claims.

77.37 Machine state should not collapse scientific uncertainty

A programme may legitimately store unresolved status, conflicting evidence, source-blocked status, conditional support, falsification under one assumption, or an open question under another.

A state machine should preserve those distinctions.

Governance is not improved by forcing every question into a binary accepted/rejected flag.

77.38 Bounded work packages protect handoff quality

A work package is strongest when a new contributor can determine the exact target, premises, allowed sources, required output, return route, authority boundary, and stop condition.

That is why the bounded package belongs inside the state-machine chapter.

It is the unit on which safe delegation operates.

77.39 Independent actors need bounded interfaces too

Actor separation is useful only if the transferred work is well defined.

Otherwise two nominally independent contributors may solve different problems while appearing to review one another.

Stable identities and bounded work packages give the separation predicate something concrete to refer to.

77.40 Idempotence enables automation under retries

Distributed automation is rarely exactly-once at the transport layer.

Events can be delivered twice, retried after timeout, replayed after a crash, or awakened by multiple triggers.

Canonical idempotence allows those operational duplicates without multiplying protected effects.

That is one reason stable identity is not administrative decoration.

77.41 History should remain append-only where possible

An append-only history preserves failed attempts, retries, repairs, policy changes, and supersession events.

Canonical state can still expose the current protected interpretation.

The two views answer different questions.

Canonical state asks:

what counts now?

History asks:

how did we get here?

77.42 A practical research-state ledger

Before accepting a protected state change, record:

Field	Question
work identity	Which exact bounded package is this?
target	Which exact artifact/revision is affected?
event type	Receipt, replay, adjudication, promotion, certification, or other?
actor	Who/what is attempting the transition?
evidence identity	Which immutable result is involved?
guard	Which conditions must be true?
authority	Who is permitted to perform this mutation?
canonical effect	What protected state changes?
history effect	What attempt/receipt is appended?
retry key	How are exact repeats recognized?
conflict rule	What happens to a different result under the same dispatch?
failure class	Tooling, evidence, policy, authority, or science?
recovery	Retry, repair, new evidence, supersession, or policy change?
certification	Is this a separate transition?
frontier effect	Who may authorize a successor?

This ledger turns vague workflow verbs into inspectable state transitions.

77.43 Failure modes

77.43.1 Done-as-one-state

Receipt, replay, adjudication, promotion, and certification are collapsed into one label.

77.43.2 Retry-multiplies-authority

The same immutable result gains multiple effects because transport repeated.

77.43.3 Dispatch-key-overwrite

A different result under the same work package silently replaces prior evidence.

77.43.4 Replay-equals-truth

A successful execution is treated as proof of the surrounding scientific claim.

77.43.5 Reviewer-label-equals-independence

Different role names are treated as evidence of independent judgment.

77.43.6 Contributor-equals-scheduler

A suggested next residual is treated as authorization to advance the research frontier.

77.43.7 Safe-equals-live

Fail-closed preservation is treated as proof the programme will eventually make progress.

77.43.8 Repair-reuses-stale-evidence

A changed implementation is promoted using evidence bound to an older revision.

77.44 What the finite witness establishes

The companion witness proves only these exact finite facts:

- first capture of $(17, A)$ creates one canonical evidence identity;
- exact retry leaves the evidence count at one;
- distinct result $(17, B)$ raises the preserved count to two;
- advancement vector $(1, 1, 1, 1, 0)$ evaluates to zero;
- advancement vector $(1, 1, 1, 1, 1)$ evaluates to one;
- missing-check vector $(1, 1, 0, 1, 1)$ evaluates to zero;
- repeated exact advancement has one canonical set-insertion effect;
- same-actor separation predicate $\text{Sep}(A, A)$ evaluates to zero;
- a fail-closed execution can preserve all declared safety invariants while never reaching advancement.

It does not prove that the five-factor gate is universal or that identity-separated actors are epistemically independent.

77.45 Downstream handoff

Governed Adaptation — **ATLAS-CH-GOVADAPT-001** may now assume bounded work packages, product research state, canonical/history separation, exact-identity idempotent retry, duplicate/conflict distinction, guarded programme advancement, separate certification state, explicit actor-separation predicates, fail-closed safety versus liveness, typed recovery operations, and exact-target evidence binding.

It must independently answer the harder question:

when a system is allowed to modify its own behavior, policy, tools, memory, or authority, which state-machine invariants and correction-capacity constraints must survive that adaptation?

77.46 References used in this chapter

- Atlas prerequisite: *Replayable Evidence Objects* (ATLAS-CH-REPLAY-001) and its audited source lock.
- Atlas prerequisite: *Formal Methods and Machine-Checkable Claims* (ATLAS-CH-FORMAL-001) and AUDIT-025.
- GCL public lifecycle controller: `grandchallenge/MATH-PROGRAMME@fdd7a3fe3df7b2d699753347080c1cbc2127governance/openmath_unattended_lifecycle_controller.json`.
- GCL public Frontier Advancement Gate at the same protected commit.
- GCL public Controlled Epistemic Interface at the same protected commit.

Exact source identities, blob hashes, and authority boundaries are recorded in:
`sources/source-locks/ATLAS-CH-RESEARCHSM-001.yaml`

Chapter 78

Governed Adaptation

Epistemic status: audited Optionality and Research-State-Machine prerequisites + bounded corrigibility/interruptibility literature + Atlas synthesis.

Adaptive systems change.

Parameters move. Policies update. Data mixtures shift. Components are replaced. New evidence arrives. Research programmes advance from one protected state to another.

The difficult question is not whether change is possible. It is:

When a system is allowed to change, what must remain true so that later correction, provenance, and bounded authority are not silently destroyed by the change itself?

This chapter calls that problem **governed adaptation**.

The central distinction is:

can change $\neq$ authorized to change $\neq$ safe to promote

The audited Optionality chapter supplies a language for correction capacity and recoverability.

The audited Research as a State Machine chapter supplies typed states, guarded transitions, exact identity, idempotence, authority separation, and fail-closed promotion.

The present chapter composes those ideas into an adaptation contract.

78.1 Adaptation as a governed transition

Let the current protected state be

$$g = (r, x, E, A, P, C, L).$$

Here:

- r is the exact protected revision identity;
- x is the operative state;
- E is canonical evidence;
- A is the authority relation;
- P is promotion/programme state;
- C is certification state;
- L is durable history.

A candidate revision is

$$q = (id, r_{parent}, \Delta, \hat{r}).$$

The candidate declares which protected parent it assumes, what bounded change it proposes, and which exact result identity it produces.

If the parent changed, the proposal may already be stale.

78.2 Authority is typed

A common governance error is to collapse every form of permission into one word.

The Atlas keeps at least five relations separate:

- proposal authority;
- execution authority;
- promotion authority;
- recovery authority;
- certification authority.

These can belong to the same actor under one policy. They are still different relations.

A contributor who may propose a change does not thereby own canonical promotion.

An executor who can stage a revision does not thereby certify it.

A recovery operator does not thereby acquire policy-writing authority.

78.3 Execution is not promotion

A candidate can be executed in a staging state:

$$g \xrightarrow{\text{execute}(q)} g_q.$$

This creates a candidate state.

Promotion is separate:

$$g_q \xrightarrow{\text{promote}} g^+.$$

This separation creates a place to ask:

- Did the exact candidate pass its required checks?
- Are those checks bound to this revision?
- Were the relevant actors authorized?
- Is a recovery path still available?
- Did a repair invalidate earlier evidence?
- Is certification required or merely promotion?

The Research-State-Machine prerequisite already established that successful execution is not sufficient for promotion.

Governed adaptation applies that rule to changing systems.

78.4 Evidence belongs to exact targets

Let an evidence object be

$$e = (id, target, claim, payload, provenance).$$

Define:

$$\text{Binds}(e, r) \iff \text{target}(e) = r.$$

Suppose revision r_1 passes validation.

A repair then produces:

$$r_2 \neq r_1.$$

The old validation remains historical evidence.

It is not exact-target evidence for r_2 .

$$\text{Binds}(e_{r_1}, r_2) = 0.$$

The practical rule is simple:

After a material revision changes exact identity, replay the load-bearing evidence on the new exact target.

This protects against validating one object and promoting another.

78.5 Canonical state and history

Adaptation can overwrite the evidence needed to understand the change.

A governed system therefore separates:

- canonical current state;
- append-only transition history;
- evidence objects;
- interpretation;
- review;
- promotion;
- certification.

A new revision should not erase the identity of its predecessor merely because the predecessor is no longer current.

History answers: how did we get here?

Canonical state answers: what is protected now?

Those are different questions.

78.6 Correction capacity enters the gate

Optionality introduced conditional correction feasibility:

$$C_{h,\epsilon}(x, \theta)$$

and ex-ante correction capacity:

$$CC_{h,\epsilon}(x; b) = \mathbb{E}_{\theta \sim b}[C_{h,\epsilon}(x, \theta)].$$

Governed adaptation can use this as one explicit admissibility condition.

For a declared floor κ , require:

$$CC_{h,\epsilon}(x_q; b) \geq \kappa.$$

This does not say that preserving more optionality is always better.

It says something narrower:

If the governing process declares preservation of specified correction paths to be a requirement, then candidate evaluation must measure those paths rather than assume they survived.

78.7 A finite admissibility gate

One possible gate is:

$$Adm(g, q) = ParentOK \cdot EvidenceOK \cdot AuthorityOK \cdot Sep \cdot Inv \cdot \mathbf{1}\{CC_{h,\epsilon} \geq \kappa\}.$$

The factors mean:

- exact parent identity matches;
- required evidence binds to the candidate;
- required authority predicates hold;
- any declared separation relation holds;
- the declared invariant holds;
- the declared correction-capacity floor holds.

The product is Boolean.

If any required factor is zero, the candidate is not admissible under this particular gate.

The Atlas does not claim this factorization is universal.

Its value is that each reason for refusal is typed.

78.8 Equal immediate value, unequal correction capacity

Consider a deliberately small witness.

Let:

$$\Theta = \{N, D\},$$

where:

- N : no later defect is discovered;
- D : a later defect is discovered.

Use symmetric prior:

$$b(N) = b(D) = 1/2.$$

Let x_0 be the common protected reference state. Candidate A leads to x_A , and candidate B leads to x_B .

Define

$$\Delta U(q) = U(x_q) - U(x_0).$$

For the two candidates,

$$\Delta U(A) = \Delta U(B) = 1.$$

Both pass the same immediate checks.

Only one preserves a governed recovery path.

78.8.1 Candidate A

If N occurs, no correction is required.

If D occurs, an authorized recovery path returns to the accepted target.

Therefore:

$$C_{h,0}(x_A, N) = 1, \quad C_{h,0}(x_A, D) = 1.$$

Hence:

$$CC_{h,0}(x_A; b) = 1.$$

78.8.2 Candidate B

If N occurs, no correction is required.

If D occurs, the declared correction target is unreachable within the authorized horizon.

Therefore:

$$C_{h,0}(x_B, N) = 1, \quad C_{h,0}(x_B, D) = 0.$$

Hence:

$$CC_{h,0}(x_B; b) = 1/2.$$

Immediate value ties.

Correction capacity does not.

78.9 Why a utility-only gate misses the difference

Define:

$$G_U(q) = \mathbf{1}\{\Delta U(q) \geq 1\}.$$

Then:

$$G_U(A) = G_U(B) = 1.$$

Now declare:

$$\kappa = 1$$

and define:

$$G_C(q) = G_U(q)\mathbf{1}\{CC_{h,0}(x_q; b) \geq 1\}.$$

Then:

$$G_C(A) = 1, \quad G_C(B) = 0.$$

The witness establishes only this finite separation.

It does not tell us that every real system should use $\kappa = 1$.

It tells us that an objective containing only immediate value can omit a property the governing process may care about.

78.10 Raw action count is not enough

Suppose both candidate states expose two commands:

- continue;
- rollback.

A superficial option count says both states have two actions.

But suppose rollback in one state:

- lacks authority;
- points to an incompatible artifact;
- cannot restore external state;
- exceeds the permitted horizon;
- or fails the declared target semantics.

Then the labels are present while the functional correction path is absent.

This is why Optionality treats functional consequences and correction feasibility separately from raw action count.

78.11 A backup is not recovery

“We have a backup” is an artifact statement.

“We can recover” is a transition claim.

Recovery can fail because:

- the backup is corrupt;
- dependencies changed;
- external data cannot be restored;
- credentials expired;
- migrations were one-way;
- the recovery actor lacks authority;
- restored bytes do not recreate the declared consequence class.

For target class $[r_\star]$, define governed recovery by:

$$RecAuth_h(g, [r_\star]) = 1$$

only when an authorized path of length at most h reaches the declared target class.

78.12 Corrigibility and intervention precedents

Corrigibility was introduced as a problem of cooperation with corrective intervention despite incentives that can make capable agents resist shutdown or preference modification (Soares et al. 2015).

Orseau and Armstrong study safe interruptibility in reinforcement learning, asking how intervention can occur without training the agent to seek or avoid interruption (Orseau and Armstrong 2016).

Hadfield-Menell and colleagues analyze an off-switch game in which an agent's incentive to preserve human intervention depends on uncertainty about the underlying objective and on treating human action as informative (Hadfield-Menell et al. 2017).

These are important precedents.

They are not interchangeable.

The Atlas keeps separate:

- corrigibility;
- interruptibility;
- human intervention-channel preservation;
- rollback;
- correction capacity;
- governance authority.

The literature motivates the intervention problem. It does not imply the Atlas gate.

78.13 Preserving correction can have cost

Optionality already established that preserving more options is not universally optimal.

The same warning applies here.

A recovery path can impose:

- storage cost;
- compatibility burden;
- slower migrations;
- operational complexity;
- delayed optimization.

Governed adaptation can therefore treat correction capacity as:

- an objective;
- a constraint;
- or a diagnostic.

It is not an unconditional command to preserve every possible past state.

78.14 Bounded authority

Authority to alter operative state x does not automatically include authority to alter the authority relation A .

If a candidate changes the governance policy itself, that is a different transition class.

The system needs whatever authority the governing process assigns to policy change.

Thus:

authority over system state $\neq$ authority over the rule that grants authority

Without this distinction, bounded delegation becomes unrestricted delegation by implication.

78.15 Safety is not liveness

A strict gate can refuse unsupported candidates.

That is a safety property.

But a gate can also refuse everything forever.

Consider:

$$g_0 \rightarrow g_0 \rightarrow g_0 \rightarrow \cdots .$$

The invariant remains true.

No candidate advances.

Therefore:

$$\text{fail-closed safety} \not\Rightarrow \text{eventual progress}.$$

A programme claiming eventual adaptation needs a separate liveness argument.

78.16 Governance conformance is not substantive truth

Suppose a candidate:

- has exact provenance;
- passes all required checks;
- has proper authority;
- preserves the declared recovery path;
- receives promotion.

What has been shown?

That the declared governance contract was satisfied.

What has not automatically been shown?

- that a scientific hypothesis is true;
- that the system is globally safer;
- that every objective improved;
- that the evidence was complete;
- that no undeclared failure mode exists.

Process integrity matters because it lets us state exactly what was established.

78.17 Repairs invalidate exact-target evidence

A common operational pattern is:

1. candidate r_1 is tested;
2. test fails;
3. candidate is repaired to r_2 ;
4. someone remembers that “validation already ran.”

That is insufficient.

The correct sequence is:

1. bind evidence to r_1 ;
2. repair to r_2 ;
3. treat r_1 ’s exact-target validation as stale for promotion of r_2 ;
4. replay on r_2 ;
5. promote only if the new exact target passes.

This closes the identity gap between evidence and mutation.

78.18 Recovery must name a target

“Return to normal” is not a formal recovery target.

A governed recovery should name:

- target revision or consequence class;
- maximum horizon;
- authority;
- acceptable cost or tolerance;
- evidence required after restoration.

A rollback can restore bytes while failing to restore behavior.

It can restore behavior while failing to restore external state.

It can restore a previous revision while invalidating later evidence.

Recovery semantics belong in the transition system.

78.19 State-relative authority

Authority can be state-relative.

Permission granted for one bounded candidate should not silently cover a materially different object.

If the candidate's parent, scope, target identity, or governing policy changes materially, the applicable authority contract should be re-evaluated.

This is the governance analogue of exact-target evidence.

78.20 Why this matters for adaptive intelligence

An adaptive system that cannot preserve correction channels can become more capable while becoming harder to steer.

A research programme that cannot preserve provenance can become more productive while becoming less auditable.

A deployment pipeline that cannot preserve exact-target evidence can become faster while losing the ability to say what was actually validated.

Governed adaptation is the attempt to preserve the infrastructure of correction while permitting change.

It is not anti-adaptation.

It is a theory of admissible change under declared constraints.

78.21 Failure modes

78.21.1 Capability as authority

A system can perform an action and therefore treats itself as permitted to perform it.

This confuses mechanism with governance.

78.21.2 Promotion by execution

A candidate runs successfully and is treated as canonical.

This skips the promotion gate.

78.21.3 Stale green evidence

A repaired candidate inherits validation from its predecessor.

This violates exact-target binding.

78.21.4 Cosmetic rollback

A rollback command exists but cannot restore the declared target.

This confuses interface with recoverability.

78.21.5 Option counting

A system counts command labels rather than functional correction paths.

78.21.6 Corrigibility by naming

An intervention channel exists and the system is therefore called corrigible.

The stronger property has not been established.

78.21.7 Safety without progress

A fail-closed system never violates the gate but also never advances.

78.21.8 Governance as truth

A compliant process outcome is treated as proof that a scientific or empirical claim is correct.

78.21.9 Unbounded delegation

Authority over operative state is interpreted as authority over the governance contract.

78.21.10 Recovery without revalidation

A prior revision is restored but its current environment and dependencies are not rechecked.

78.22 Downstream handoff

Frontier Questions of Adaptive Intelligence may assume:

- governed adaptation is a typed transition problem;
- authority is separated by role;
- candidate evidence binds to exact target identity;

- authorized recovery paths are distinct from stored backups;
- correction capacity can be an explicit gate dimension;
- immediate value can tie while correction capacity differs;
- fail-closed safety does not imply liveness;
- intervention precedents do not collapse into a universal theorem.

FRONTIER must independently determine which unresolved questions deserve programme priority.

78.23 Epistemic status

Established prerequisite layer:

- viable continuation and correction-capacity semantics;
- typed governed states and guarded transitions;
- exact identity, idempotence, recovery classes, and authority separation.

External precedent layer:

- corrective intervention/corrigibility problem (Soares et al. 2015);
- safe interruptibility in a declared RL setting (Orseau and Armstrong 2016);
- off-switch incentive analysis under objective uncertainty (Hadfield-Menell et al. 2017).

Atlas-owned synthesis:

- governed adaptive state;
- authority separation;
- exact-target adaptation gate;
- authorized recovery-path semantics;
- correction-capacity gate composition;
- equal-immediate-value finite witness.

Not established here:

- universal safety;
- universal adaptation optimality;
- complete corrigibility;
- universal rollback feasibility;
- substantive truth from governance conformance.

78.24 References used in this chapter

- (Soares et al. 2015) — corrigibility and corrective intervention.
- (Orseau and Armstrong 2016) — safe interruptibility in reinforcement learning.
- (Hadfield-Menell et al. 2017) — incentives around preserving a human off-switch.

Exact provenance and authority boundaries are locked in:
sources/source-locks/ATLAS-CH-GOVADAPT-001.yaml

Chapter 79

Frontier Questions of Adaptive Intelligence

Epistemic status: audited Atlas substrate + project-local research programme + architecture-stage questions + explicit open proof and experiment obligations.

Programme specification: ‘governance/tranches/CH171-SPEC.md’

Documentary evidence matrix: ‘governance/tranches/CH171-EVIDENCE-MATRIX.md’

Source lock: ‘sources/source-locks/ATLAS-CH-FRONTIER-001.yaml’

79.1 A frontier is not a result

The Atlas has now accumulated enough audited substrate to ask sharper research questions.

That does not mean those questions are solved.

This chapter separates audited substrate, bounded evidence, architecture-stage programmes, open proof obligations, open experiment obligations, and conjectural cross-programme connections.

The governing rule is:

research programme $\neq$ established result.

The value of a frontier chapter is that each open direction has a named next obligation.

79.2 The three governing prerequisites

Governed Adaptation contributes exact-target evidence, typed authority, execution-versus-promotion separation, recovery discipline, and correction-capacity boundaries.

The Residual contributes a reconstructive question: what is the least admissible structure that survives declared representation changes while remaining sufficient for a declared capability?

Boundary Contracts contribute the interface question: what semantic, geometric, sensitivity, and numerical information must cross a component boundary for composition to remain meaningful?

These chapters constrain how the programme is stated. They do not solve the programme.

79.3 Geometry-derived optimization

The second-order chapter is draft-v0.1 at the protected ledger snapshot and provides curvature, Hessian, Fisher, natural-gradient, trust-region, quasi-Newton, and proximal distinctions.

The live architecture-stage programmes are Matrix-Aware Optimization, Optimization on Manifolds, and Variational and Divergence-Derived Optimization.

The research question is not whether geometric language can describe an optimizer. It is whether a declared matrix, manifold, conditioning, or variational structure predicts an update rule whose behavior changes for a reason that survives comparison with matched controls.

A smallest useful result would bind a problem family, a structural assumption, an update rule, a matched baseline, a predicted consequence, and evidence that the structure caused the consequence.

A successful run is evidence. It is not yet a mechanism.

79.4 Operator-valued position

The Geometry of Position chapter already establishes exact rotary relative-phase identities and their limits.

The Relative-Position Operator programme begins beyond that substrate.

Its canonical contract asks whether positional structure is better exposed through low-dimensional operators, frequency modes, DC components, and head-specific specialization.

The smallest useful advance is not another restatement of RoPE algebra.

It is a result showing that a compact operator description supports a reproducible prediction, intervention, compression, or transfer property that ordinary positional-vector language does not already explain.

Until then:

operator notation $\neq$ operator discovery.

79.5 Adaptive depth as error control

Depth as Computational Time and Networks as Numerical Schemes are already audited.

That makes a stronger question possible:

Can adaptive compute behave like adaptive time stepping, where more computation is allocated because a declared error or quality signal says it is needed?

Variable depth alone is not error control.

A valid test needs an observable error or quality surrogate, a controller responding to that signal, a stopping or depth action, and a fixed-depth control at matched compute or matched quality.

If the error signal does not explain the depth decision, the weaker phrase “adaptive compute” is more accurate.

79.6 Persistent and shared memory

The External-Memory Thesis is already audited.

The frontier question is no longer simply whether retrieval helps.

It is:

Which information belongs in weights, which belongs in active context, and which belongs in persistent shared memory?

A shared memory layer introduces requirements for identity, indexing, provenance, updates, consistency, transactions, and access across models or agents.

The thesis does not imply that all useful knowledge should leave parameters.

It turns knowledge placement into an allocation problem under declared costs and governance.

79.7 Learning progress as a search operator

The Curriculum Learning chapter already distinguishes open-loop schedules from state-aware selection and shows that progress depends on learner state.

The next programme asks whether learning progress can become a search signal over experiences.

The key risk is myopia.

An experience can produce high immediate gain while harming later transfer, causing forgetting, consuming disproportionate compute, or steering the learner toward a narrow local mechanism.

So the frontier question is not “can progress be maximized?”

It is:

Under what conditions does progress feedback guide experience search toward the declared long-horizon objective?

That requires horizon-aware controls and separation of dose, difficulty, and compute confounds.

79.8 Minimal curricula and reasoning bases

The Minimal Curricula and Reasoning Bases chapter remains architecture-stage.

Its question is deliberately stronger than ordinary curriculum optimization:

What is the smallest early set of mechanisms or reasoning operations from which broad later capability can be reconstructed?

“Smallest” requires a declared mechanism vocabulary, capability family, reconstruction process, resource budget, and equivalence relation.

A short syllabus is not automatically a minimal curriculum.

A small dataset is not automatically a minimal reasoning basis.

The programme must also distinguish acquisition, persistence, accessibility, and behavioral expression.

79.9 The Residual remains open at frontier scale

The Residual chapter has an audited formalization.

It does not claim a universal frontier-scale Residual.

The object is relative to a declared state space, transformation family, capability probes, behavior profile, descriptor class, and admissible recovery maps.

Open questions remain: existence, useful uniqueness, finite dimensionality, efficient learnability, transfer across architecture families, and operational reconstruction at acceptable cost.

The next serious advance is a nontrivial declared family in which existence can be proved or falsified.

79.10 Benign nonconvexity

The Atlas source inventory names benign nonconvexity, landscape topology, saddle structure, conditioning, and optimizer dynamics as open research objects.

At present benign nonconvexity is a programme theme, not a theorem.

A meaningful claim needs a declared landscape class, a declared optimizer or dynamics, and a declared target property.

That property might concern reachability, absence of bad attractors, quotient simplification, correction capacity, or another explicit mechanism.

A converged training run is not sufficient.

The conjectural connection is that optimizer geometry and landscape structure may have to be analyzed together.

79.11 Boundary contracts

Boundary Contracts are already audited substrate.

The next frontier is to determine which exposed boundary variables are actually sufficient for composition.

A useful contract might expose semantics, invariants, local sensitivities, error budgets, and low-order separator state.

The audited chapter also shows that a low-dimensional separator can forget state that matters downstream.

So the open question is:

Under which assumptions do local interface obligations compose into a useful system-level guarantee?

Local compatibility alone is not a global guarantee.

79.12 Source boundaries

Research vocabulary and source evidence are different objects.

When project terminology is stronger than the exact source-locked artifact, the Atlas keeps the stronger wording at programme status and binds factual implementation claims to the inspected source.

This rule applies across the programme:

programme vocabulary $\neq$ source evidence.

79.13 Cross-programme questions

Several connections are worth testing but are not inherited results.

The evidence matrix records them explicitly:

- Residual structure may be relevant to reconstructive curriculum questions.
- Boundary contracts may carry compact state needed by downstream composition.
- Learning-progress feedback may help select experiences that expose reusable computation.
- Adaptive depth may become an error-control mechanism when a valid error signal exists.
- Shared memory may hold reconstructive state that need not be duplicated in every model.

Each remains conjectural until a declared proof or experiment supports the stronger relation.

79.14 How a programme item advances

A programme item advances when new evidence supports a more specific claim.

Governed Adaptation supplies the discipline:

- bind evidence to the exact target;
- state the claim class;
- preserve provenance;
- use the relevant comparison or proof obligation;
- replay evidence after identity-changing repair;
- separate execution from promotion.

This prevents one bounded result from silently becoming a stronger general claim.

79.15 Negative results are progress

A research programme should become smaller when it learns.

A counterexample can remove a conjecture. A failed replication can shrink a regime. A lower bound can eliminate an approach. A source audit can weaken an unsupported claim. A proof can reduce a broad question to one missing estimate.

Progress is not only accumulation. It is reduction of uncertainty in the programme graph.

79.16 Preferred research pattern

The Atlas prefers:

broad question -> bounded mechanism -> smallest named proof or experiment
-> exact evidence -> revised programme.

A frontier chapter should expose handles, not merely aspirations.

79.17 What this chapter does not claim

This chapter does not claim that the architecture-stage programmes recorded in the evidence matrix are already solved.

It does not promote bounded examples into universal results.

It does not convert programme vocabulary into public implementation evidence.

The exact open boundaries remain those recorded in the documentary matrix and source lock.

79.18 Why collect the programme now?

The substrate has matured enough to make the questions less vague.

Optimization can now be decomposed into curvature, matrix structure, manifolds, and variational derivation.

Position can now be separated from exact rotary algebra and tested as an operator hypothesis.

Adaptive compute can now be asked as an error-control question.

Minimal knowledge can now be stated in reconstructive terms.

The frontier becomes sharper as the substrate becomes less ambiguous.

79.19 Downstream handoff

ATLAS-CH-SYNTHESIS-001 may inherit:

- the programme taxonomy;
- the audited/open distinction;
- the named proof and experiment obligations;
- governed advancement rules.

It may not treat unresolved programme items as completed pillars merely because they appear here.

The final synthesis must still distinguish what is known, engineered, measured, hypothesized, and open.

79.20 References used in this chapter

This chapter introduces no new broad external literature survey.

Its authority is project-local and source-locked:

- audited Governed Adaptation;
- audited Residual formalization;
- audited Boundary Contracts;
- the Atlas Map;
- the Atlas source inventory;
- the Chapter-Family Rollout;
- the exact Chapter Ledger status snapshot.

Exact provenance and claim boundaries are locked in:

`‘sources/source-locks/ATLAS-CH-FRONTIER-001.yaml’`

Chapter 80

Beyond the Monolithic Model

Epistemic status: audited Frontier and Computational Polity prerequisites + Atlas-owned exact systems-composition witness.

Specification: manuscript/specifications/ATLAS-CH-SYNTHESIS-001.md

Derivation packet: mathematics/derivations/ATLAS-CH-SYNTHESIS-001-DERIVATIONS.md

Computational witness: mathematics/computational-witnesses/ATLAS-CW-SYNTHESIS-001.md

Source lock: sources/source-locks/ATLAS-CH-SYNTHESIS-001.yaml

The Atlas does not end by declaring one universal model architecture.

It ends by making the system boundary explicit.

A deployed capability can depend on representation, dynamics, memory, tools, co-ordination, evidence, adaptation, and governance at once. When those interfaces are part of the mechanism, they belong inside the explanatory object.

The governing rule is:

system capability is a property of declared composition, not component count alone.

80.1 A typed synthesis object

Write

$$\Sigma = (X, D, M, T, C, E, A, G, Q, K).$$

Here:

- X is model/representation and private state;
- D is dynamics and update law;
- M is persistent/shared memory;
- T is tools and external services;
- C is coordination, routing, and composition;
- E is evidence and validation;
- A is adaptation;
- G is governance and authority;
- Q is the task/success contract;
- K is the heterogeneous cost vector.

These are typed objects. They need not share a vector space, timescale, optimizer, or authority model.

80.2 Frontier status survives synthesis

FRONTIER supplies the exact programme grammar:

- AUDIT_BOUND_SUBSTRATE;
- DRAFT_SUBSTRATE;
- BOUNDED_EVIDENCE;
- ARCHITECTURE_STAGE;
- OPEN_PROOF_OBLIGATION;
- OPEN_EXPERIMENT_OBLIGATION;
- CONJECTURAL_CONNECTION.

SYNTHESIS preserves these categories. Narrative coherence does not promote an open item into established substrate.

80.3 Polity becomes a systems interface

POLITY already separates private state, shared state, candidate production, validation, authority, commit, and cost.

SYNTHESIS extends the explanatory boundary without erasing those separations.

Capability is not authority.

Evidence is not governance.

Stored state is not truth.

Component count is not composition quality.

80.4 Exact finite composition

Let

$$Q = \{\alpha, \beta\}.$$

Define specialists

$$A_\alpha(\alpha) = 1, \quad A_\alpha(\beta) = 0,$$

$$A_\beta(\alpha) = 0, \quad A_\beta(\beta) = 1.$$

Each specialist alone succeeds on one of the two tasks.

Use router

$$R(\alpha) = A_\alpha, \quad R(\beta) = A_\beta.$$

The routed system returns the correct answer 1 on both tasks.

Thus task accuracy is

$$\boxed{1}.$$

80.5 Evidence and provenance

Each candidate is

$$c = (q, y, s),$$

where s is the producing specialist.

The validator accepts only when:

1. $y = 1$;
2. s is the declared routed specialist for q .

This finite validator checks both answer and provenance. It is not promoted into a universal oracle.

80.6 Governance and commit

Governance authorizes commit only after validator acceptance.

The transition is therefore

$$\text{candidate} \rightarrow \text{validation} \rightarrow \text{authorization} \rightarrow \text{commit}.$$

These are different events.

Correct-looking output cannot silently bypass the authority boundary without changing the system.

80.7 Shared memory

Committed records are

$$m = (q, y, s, \text{validated}).$$

They persist in shared memory keyed by task.

Later recall therefore has access to both accepted answer and provenance.

For the full composition:

- task accuracy = 2/2;
- validated commit coverage = 2/2;
- persistent recall coverage = 2/2;
- unauthorized commits = 0.

80.8 Router ablation

Keep the same specialists but use

$$R'(\alpha) = A_\alpha, \quad R'(\beta) = A_\alpha.$$

Now beta receives answer 0.

The validator rejects the beta candidate.

Task accuracy and validated commit coverage each fall to

$$\boxed{\frac{1}{2}}.$$

The component set is unchanged. The composition rule changed.

80.9 Validator ablation

Restore correct routing but remove the validator while keeping governance unchanged.

The system can still produce correct transient answers on both tasks.

But governance requires positive validation evidence.

Without it, authorized commit coverage becomes

$$\boxed{0}.$$

Thus

$$\boxed{\text{answer capability} \not\Rightarrow \text{authorized durable state.}}$$

80.10 Memory ablation

Keep routing, validation, and governance.

Remove persistent shared memory.

Immediate answers can remain correct.

Later persistent recall coverage becomes

$$\boxed{0}.$$

Thus

$$\boxed{\text{immediate success} \not\Rightarrow \text{persistent shared recall.}}$$

80.11 Governance ablation

Consider

$$c_{\text{bad}} = (\beta, 1, A_\alpha).$$

The answer value looks correct but the source is wrong.

The validator rejects it.

With governance enforced, the rejected candidate cannot commit.

If the commit gate is bypassed, it can be written.

Therefore

correct-looking content $\nRightarrow$ authorized state transition.

80.12 Capability and authority

A component may be capable of producing an output it is not allowed to commit.

A role may be authorized to commit a validated result without being capable of producing that result itself.

So

Can $\neq$ May .

This distinction becomes more important as the system boundary expands.

80.13 Evidence and governance

Evidence supports a claim.

Governance constrains transitions.

Governance cannot manufacture missing evidence.

Strong evidence does not automatically grant authority.

Hence

evidence $\neq$ authorization.

80.14 Memory and truth

Shared memory can preserve accepted records.

It cannot make a record true merely by storing it.

The inherited boundary remains:

- stored does not mean true;
- present does not mean current;
- shared does not mean universally visible;
- retrieved does not mean correctly used.

80.15 Adaptation and permission

A system may be able to change weights, memory, routing, prompts, tools, or policies.

That ability belongs to A .

Whether the change is permitted belongs to G .

Thus adaptation and authority must remain separate.

80.16 Tools inside the system boundary

If a capability depends on search, compilers, proof checkers, databases, laboratories, or external services, those interfaces belong in the system-level claim.

A tool response is not automatically the same event as a durable external state change.

The transaction boundary must remain explicit.

80.17 Heterogeneous cost

Retain the polity cost vector

$$K = (c_{\text{coord}}, c_{\text{mem}}, c_{\text{tool}}, c_{\text{human}}, c_{\text{val}}).$$

A composite system can improve one capability while increasing several costs.

No scalar comparison is justified until the application declares a scalarization or objective.

80.18 Specialization does not imply net benefit

The finite specialist witness has a clear composition gain.

Real specialization can add routing latency, coordination overhead, memory contention, validation cost, and new failure modes.

Therefore

specialization benefit $\nRightarrow$ net system benefit under every cost model.

80.19 The monolithic model remains legitimate

Nothing here proves that one model cannot internalize memory-like state, routing, tool use, validation, or adaptation.

The synthesis claim is not that monoliths are obsolete.

It is that the explanatory boundary should follow the actual mechanism.

80.20 A monolith can be internally composite

One externally packaged model may contain conditional computation, recurrent state, modular subspaces, iterative computation, routing, or memory.

So “monolithic” is not a primitive mathematical category.

The relevant question is which internal or external interfaces are necessary for the declared capability.

80.21 A distributed system can still be poorly composed

Many components can wrap one weak central capability.

A collection of strong components can fail under bad routing or invalid interfaces.

Therefore

distributed $\nrightarrow$ capable.

Composition must be demonstrated.

80.22 Evidence levels remain visible

The Atlas contains exact derivations, finite computational witnesses, audited engineering contracts, bounded empirical evidence, hypotheses, open proof obligations, and open experiment obligations.

SYNTHESIS does not convert these into one evidence level.

80.23 Open research remains open

FRONTIER names research programmes and explicit obligations.

Some downstream chapters have supplied stronger local results.

Any unresolved obligation remains unresolved until separately discharged and audited.

The final chapter is not a certification shortcut.

80.24 Durable Atlas disciplines

Across the Atlas, several practices recur:

1. type the mathematical object;
2. declare the state and update law;
3. separate local and global claims;
4. expose boundary contracts;
5. separate residual, error, and task metrics;
6. distinguish memory from truth;
7. distinguish capability from authority;
8. bind evidence to exact identity;
9. replay before promotion;
10. preserve open obligations.

These are disciplines, not one prescribed architecture.

80.25 Architecture remains a design choice

Different systems may place memory, routing, tools, validation, adaptation, human review, and authority at different boundaries.

The right placement depends on tasks, failure models, costs, and governance constraints.

The Atlas does not prescribe one universal placement.

80.26 Systems claims need a contract

A serious claim should identify:

- task set/distribution;
- component roles;
- private/shared state;
- routing/composition;
- memory semantics;
- evidence rule;
- adaptation rule;
- governance rule;
- success metric;
- cost model;
- failure model.

Without these, “the system can do X” is underspecified.

80.27 Ablation is part of synthesis

If a system-level capability is claimed to depend on composition, perturb the interface.

The finite witness shows distinct failures:

- routing ablation harms task capability;
- validator ablation harms governed commit;
- memory ablation harms persistence;
- governance ablation harms authorization integrity.

Different interfaces support different properties.

80.28 Research and engineering consequence

A single benchmark score can hide mechanism.

A typed systems view can expose where information lives, where uncertainty enters, which boundary matters, which role holds authority, and which evidence supports promotion.

That makes claims more auditable even when the underlying system remains complex.

80.29 No universal architecture theorem

The Atlas does not prove

distributed > monolithic,

or

more modules > fewer modules.

Such comparisons require declared tasks, metrics, costs, and failure models.

80.30 The durable synthesis

The strongest justified statement is a scoped synthesis, not a universal theorem:

Adaptive intelligence can be analyzed as a typed system of representations, dynamics, memory, tools, coordination, evidence, adaptation, and governance.

When those interfaces are part of the mechanism, they belong inside the explanatory boundary.

The final lesson is therefore not “beyond models.”

It is beyond pretending that every consequential system property must live inside one undifferentiated model boundary.

80.31 References used in this chapter

No new external academic authority is added.

Programme-status authority is inherited through audited ATLAS-CH-FRONTIER-001.

System-composition authority is inherited through audited ATLAS-CH-POLITY-001.

Exact prerequisite identities and claim boundaries are recorded in:

`sources/source-locks/ATLAS-CH-SYNTHESIS-001.yaml`

Absil, P.-A., R. Mahony, and R. Sepulchre. 2008. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press. <https://doi.org/10.1515/9781400830244>.

Achille, Alessandro, and Stefano Soatto. 2018. “Emergence of Invariance and Disentanglement in Deep Representations.” *Journal of Machine Learning Research* 19 (50): 1–34. <https://www.jmlr.org/papers/v19/17-646.html>.

Amari, Shun-ichi. 1998. “Natural Gradient Works Efficiently in Learning.” *Neural Computation* 10 (2): 251–76. <https://doi.org/10.1162/089976698300017746>.

Arnoldi, Walter Edwin. 1951. “The Principle of Minimized Iterations in the Solution of the Matrix Eigenvalue Problem.” *Quarterly of Applied Mathematics* 9 (1): 17–29. <https://www.ams.org/qam/1951-09-01/>.

Arrow, Kenneth J., and Anthony C. Fisher. 1974. “Environmental Preservation, Uncertainty, and Irreversibility.” *The Quarterly Journal of Economics* 88 (2): 312–19. <https://doi.org/10.2307/1883074>.

Aubin, Jean-Pierre. 1991. *Viability Theory*. Birkhäuser Boston.

Auer, Peter, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. “Finite-Time Analysis of the Multiarmed Bandit Problem.” *Machine Learning* 47: 235–56. <https://doi.org/10.1023/A:1013689704352>.

- Auer, Peter, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. 2002. “The Nonstochastic Multiarmed Bandit Problem.” *SIAM Journal on Computing* 32 (1): 48–77. <https://doi.org/10.1137/S0097539701398375>.
- Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. “Layer Normalization.” *arXiv Preprint*. <https://arxiv.org/abs/1607.06450>.
- Babenko, Artem, and Victor Lempitsky. 2012. “The Inverted Multi-Index.” *2012 IEEE Conference on Computer Vision and Pattern Recognition*, 3069–76. <https://doi.org/10.1109/CVPR.2012.6248038>.
- Baranes, Adrien, and Pierre-Yves Oudeyer. 2013. “Active Learning of Inverse Models with Intrinsically Motivated Goal Exploration in Robots.” *Robotics and Autonomous Systems* 61 (1): 49–73. <https://doi.org/10.1016/j.robot.2012.05.008>.
- Baydin, Atilim Gunes, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. 2018. “Automatic Differentiation in Machine Learning: A Survey.” *Journal of Machine Learning Research* 18 (153): 1–43. <https://www.jmlr.org/papers/v18/17-468.html>.
- Bengio, Yoshua, Aaron Courville, and Pascal Vincent. 2013. “Representation Learning: A Review and New Perspectives.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35 (8): 1798–828. <https://doi.org/10.1109/TPAMI.2013.50>.
- Bengio, Yoshua, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. “Curriculum Learning.” *Proceedings of the 26th Annual International Conference on Machine Learning*, 41–48. <https://doi.org/10.1145/1553374.1553380>.
- Bernstein, Jeremy, and Laker Newhouse. 2024. *Old Optimizer, New Norm: An Anthology*. <https://arxiv.org/abs/2409.20325>.
- Blier, Léonard, and Yann Ollivier. 2018. “The Description Length of Deep Learning Models.” *Advances in Neural Information Processing Systems* 31. <https://proceedings.neurips.cc/paper/2018/hash/3b712de48137572f3849aabd5666a4e3-Abstract.html>.
- Borgeaud, Sebastian, Arthur Mensch, Jordan Hoffmann, et al. 2022. “Improving Language Models by Retrieving from Trillions of Tokens.” *International Conference on Machine Learning*, Proceedings of machine learning research, vol. 162: 2206–37. <https://proceedings.mlr.press/v162/borgeaud22a.html>.
- Bubeck, Sebastien, and Nicolo Cesa-Bianchi. 2012. “Regret Analysis of Stochastic and Nonstochastic Multi-Armed Bandit Problems.” *Foundations and Trends in Machine Learning* 5: 1–122. <https://doi.org/10.1561/22000000024>.

- Chen, Ricky T. Q., Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. 2018. “Neural Ordinary Differential Equations.” *Advances in Neural Information Processing Systems* 31.
- Chen, Shouyuan, Sherman Wong, Liangjian Chen, and Yuandong Tian. 2023. “Extending Context Window of Large Language Models via Positional Interpolation.” *arXiv Preprint arXiv:2306.15595*, ahead of print. <https://doi.org/10.48550/arXiv.2306.15595>.
- Choromanski, Krzysztof, Valerii Likhoshesterov, David Dohan, et al. 2021. “Re-thinking Attention with Performers.” *International Conference on Learning Representations*. <https://openreview.net/forum?id=Ua6zuk0WRH>.
- Codd, E. F. 1970. “A Relational Model of Data for Large Shared Data Banks.” *Communications of the ACM* 13 (6): 377–87. <https://doi.org/10.1145/362384.362685>.
- Cohen, Taco, and Max Welling. 2016. “Group Equivariant Convolutional Networks.” *Proceedings of the 33rd International Conference on Machine Learning*, Proceedings of machine learning research, vol. 48: 2990–99. <https://proceedings.mlr.press/v48/cohenc16.html>.
- Cover, Thomas M., and Joy A. Thomas. 2006. *Elements of Information Theory*. 2nd ed. John Wiley & Sons. <https://doi.org/10.1002/047174882X>.
- Curry, Justin Michael. 2013. “Sheaves, Cosheaves and Applications.” PhD thesis, University of Pennsylvania. <https://arxiv.org/abs/1303.3255>.
- DeCandia, Giuseppe, Deniz Hastorun, Madan Jampani, et al. 2007. “Dynamo: Amazon’s Highly Available Key-Value Store.” *Proceedings of the 21st ACM Symposium on Operating Systems Principles*, 205–20. <https://doi.org/10.1145/1294261.1294281>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding.” *Proceedings of NAACL-HLT 2019*, 4171–86. <https://doi.org/10.18653/v1/N19-1423>.
- Dormand, J. R., and P. J. Prince. 1980. “A Family of Embedded Runge–Kutta Formulae.” *Journal of Computational and Applied Mathematics* 6 (1): 19–26. [https://doi.org/10.1016/0771-050X\(80\)90013-3](https://doi.org/10.1016/0771-050X(80)90013-3).
- Duchi, John, Elad Hazan, and Yoram Singer. 2011. “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization.” *Journal of Machine Learning Research* 12: 2121–59. <https://www.jmlr.org/papers/v12/duchi11a.html>.

- Eckart, Carl, and Gale Young. 1936. “The Approximation of One Matrix by Another of Lower Rank.” *Psychometrika* 1 (3): 211–18. <https://doi.org/10.1007/BF02288367>.
- Edelman, Alan, Tomás A. Arias, and Steven T. Smith. 1998. “The Geometry of Algorithms with Orthogonality Constraints.” *SIAM Journal on Matrix Analysis and Applications* 20 (2): 303–53. <https://doi.org/10.1137/S0895479895290954>.
- Elhage, Nelson, Tristan Hume, Catherine Olsson, et al. 2022. “Toy Models of Superposition.” *Transformer Circuits Thread*. https://transformer-circuits.pub/2022/toy_model/index.html.
- Figurnov, Michael, Maxwell D. Collins, Yukun Zhu, et al. 2017. “Spatially Adaptive Computation Time for Residual Networks.” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- Gal, Yarin, and Zoubin Ghahramani. 2016. “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning.” *Proceedings of the 33rd International Conference on Machine Learning*, Proceedings of machine learning research, vol. 48: 1050–59. <https://proceedings.mlr.press/v48/gal16.html>.
- Gale, Trevor, Erich Elsen, and Sara Hooker. 2019. *The State of Sparsity in Deep Neural Networks*. <https://arxiv.org/abs/1902.09574>.
- Geifman, Yonatan, and Ran El-Yaniv. 2017. “Selective Classification for Deep Neural Networks.” *Advances in Neural Information Processing Systems* 30, 4878–87. https://papers.nips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cfl-Abstract.html.
- Glorot, Xavier, and Yoshua Bengio. 2010. “Understanding the Difficulty of Training Deep Feedforward Neural Networks.” *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, Proceedings of machine learning research, vol. 9: 249–56. <https://proceedings.mlr.press/v9/glorot10a.html>.
- Goldman, Sally A., and Michael J. Kearns. 1995. “On the Complexity of Teaching.” *Journal of Computer and System Sciences* 50 (1): 20–31. <https://doi.org/10.1006/jcss.1995.1003>.
- Golub, Gene H., and Charles F. Van Loan. 2013. *Matrix Computations*. 4th ed. Johns Hopkins University Press.
- Graves, Alex. 2016. *Adaptive Computation Time for Recurrent Neural Networks*. <https://doi.org/10.48550/arXiv.1603.08983>.

- Graves, Alex, Marc G. Bellemare, Jacob Menick, Rémi Munos, and Koray Kavukcuoglu. 2017. “Automated Curriculum Learning for Neural Networks.” *Proceedings of the 34th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 70: 1311–20. <https://proceedings.mlr.press/v70/graves17a.html>.
- Griewank, Andreas, and Andrea Walther. 2008. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. 2nd ed. Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9780898717761>.
- Grigsby, J. Elisenda, Kathryn Lindsey, and David Rolnick. 2023. “Hidden Symmetries of ReLU Networks.” *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 202: 11734–60. <https://proceedings.mlr.press/v202/grigsby23a.html>.
- Guo, Chuan, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. “On Calibration of Modern Neural Networks.” *Proceedings of the 34th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 70: 1321–30. <https://proceedings.mlr.press/v70/guo17a.html>.
- Gupta, Vineet, Tomer Koren, and Yoram Singer. 2018. “Shampoo: Preconditioned Stochastic Tensor Optimization.” *Proceedings of the 35th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 80: 1842–50. <https://proceedings.mlr.press/v80/gupta18a.html>.
- Haber, Eldad, and Lars Ruthotto. 2018. “Stable Architectures for Deep Neural Networks.” *Inverse Problems* 34 (1): 014004. <https://doi.org/10.1088/1361-6420/aa9a90>.
- Hadfield-Menell, Dylan, Anca Dragan, Pieter Abbeel, and Stuart Russell. 2017. “The Off-Switch Game.” *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, 220–27. <https://doi.org/10.24963/ijcai.2017/32>.
- Hairer, Ernst, Christian Lubich, and Gerhard Wanner. 2006. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*. 2nd ed. Vol. 31. Springer Series in Computational Mathematics. Springer. <https://doi.org/10.1007/3-540-30666-8>.
- Hairer, Ernst, and Gerhard Wanner. 1996. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. 2nd ed. Springer Series in Computational Mathematics. Springer. <https://doi.org/10.1007/978-3-642-05221-7>.
- Han, Song, Huizi Mao, and William J. Dally. 2016a. “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and

- Huffman Coding.” *4th International Conference on Learning Representations (ICLR 2016)*. <https://arxiv.org/abs/1510.00149>.
- Han, Song, Huizi Mao, and William J. Dally. 2016b. “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding.” *International Conference on Learning Representations*. <https://arxiv.org/abs/1510.00149>.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.” *Proceedings of the IEEE International Conference on Computer Vision*, 1026–34. <https://doi.org/10.1109/ICCV.2015.123>.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. “Deep Residual Learning for Image Recognition.” *2016 IEEE Conference on Computer Vision and Pattern Recognition*, 770–78. <https://doi.org/10.1109/CVPR.2016.90>.
- Henderson, Peter, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. “Deep Reinforcement Learning That Matters.” *Proceedings of the AAAI Conference on Artificial Intelligence* 32. <https://doi.org/10.1609/aaai.v32i1.11694>.
- Herbster, Mark, and Manfred K. Warmuth. 1998. “Tracking the Best Expert.” *Machine Learning* 32: 151–78. <https://doi.org/10.1023/A:1007424614876>.
- Higham, Nicholas J. 1986. “Computing the Polar Decomposition—with Applications.” *SIAM Journal on Scientific and Statistical Computing* 7 (4): 1160–74. <https://doi.org/10.1137/0907079>.
- Higham, Nicholas J. 2002. *Accuracy and Stability of Numerical Algorithms*. 2nd ed. SIAM. <https://doi.org/10.1137/1.9780898718027>.
- Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2014. “Distilling the Knowledge in a Neural Network.” *NIPS 2014 Deep Learning Workshop*. <https://arxiv.org/abs/1503.02531>.
- Hoare, C. A. R. 1969. “An Axiomatic Basis for Computer Programming.” *Communications of the ACM* 12 (10): 576–80. <https://doi.org/10.1145/363235.363259>.
- Hochreiter, Sepp, and Jürgen Schmidhuber. 1997. “Long Short-Term Memory.” *Neural Computation* 9 (8): 1735–80. <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Horn, Roger A., and Charles R. Johnson. 2012. *Matrix Analysis*. 2nd ed.

- Cambridge University Press. <https://doi.org/10.1017/CBO9781139020411>.
- Howard, Ronald A. 1966. “Information Value Theory.” *IEEE Transactions on Systems Science and Cybernetics* 2 (1): 22–26. <https://doi.org/10.1109/TSSC.1966.300074>.
- Hutchins, Edwin. 1995. *Cognition in the Wild*. MIT Press.
- Iserles, Arieh. 2008. *A First Course in the Numerical Analysis of Differential Equations*. 2nd ed. Vol. 44. Cambridge Texts in Applied Mathematics. Cambridge University Press. <https://doi.org/10.1017/CBO9780511995569>.
- Jordan, Keller, Yuchen Jin, Vlado Boza, et al. 2024. *Muon: An Optimizer for Hidden Layers in Neural Networks*. <https://kellerjordan.github.io/posts/muon/>.
- Katharopoulos, Angelos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. “Transformers Are RNNs: Fast Autoregressive Transformers with Linear Attention.” *Proceedings of the 37th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 119: 5156–65. <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- Kendall, Alex, and Yarin Gal. 2017. “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” *Advances in Neural Information Processing Systems* 30. https://papers.nips.cc/paper_files/paper/2017/hash/2650d6089a6d640c5e85b2b88265dc2b-Abstract.html.
- Khalil, Hassan K. 2002. *Nonlinear Systems*. 3rd ed. Prentice Hall.
- Khandelwal, Urvashi, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2020. “Generalization Through Memorization: Nearest Neighbor Language Models.” *International Conference on Learning Representations*. <https://openreview.net/forum?id=HklbjCEKDB>.
- Kingma, Diederik P., and Jimmy Ba. 2015. “Adam: A Method for Stochastic Optimization.” *3rd International Conference on Learning Representations*. <https://arxiv.org/abs/1412.6980>.
- Kirkpatrick, James, Razvan Pascanu, Neil Rabinowitz, et al. 2017. “Overcoming Catastrophic Forgetting in Neural Networks.” *Proceedings of the National Academy of Sciences* 114 (13): 3521–26. <https://doi.org/10.1073/pnas.1611835114>.
- Klein, Gary, David D. Woods, Jeffrey M. Bradshaw, Robert R. Hoffman, and Paul J. Feltovich. 2004. “Ten Challenges for Making Automation a Team Player in Joint Human-Agent Activity.” *IEEE Intelligent Systems* 19 (6):

- 91–95. <https://doi.org/10.1109/MIS.2004.74>.
- Klyubin, Alexander S., Daniel Polani, and Chrystopher L. Nehaniv. 2008. “Keep Your Options Open: An Information-Based Driving Principle for Sensorimotor Systems.” *PLoS ONE* 3 (12): e4018. <https://doi.org/10.1371/journal.pone.0004018>.
- Kornblith, Simon, Jonathon Shlens, and Quoc V. Le. 2019. “Do Better ImageNet Models Transfer Better?” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2661–71. https://openaccess.thecvf.com/content_CVPR_2019/html/Kornblith_Do_Better_ImageNet_Models_Transfer_Better_CVPR_2019_paper.html.
- Kudo, Taku. 2018. “Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates.” *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 66–75. <https://doi.org/10.18653/v1/P18-1007>.
- Kudo, Taku, and John Richardson. 2018. “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing.” *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 66–71. <https://doi.org/10.18653/v1/D18-2012>.
- Kuznetsov, Yuri A. 2004. *Elements of Applied Bifurcation Theory*. 3rd ed. Vol. 112. Applied Mathematical Sciences. Springer. <https://doi.org/10.1007/978-1-4757-3978-7>.
- Lai, T. L., and Herbert Robbins. 1985. “Asymptotically Efficient Adaptive Allocation Rules.” *Advances in Applied Mathematics* 6 (1): 4–22. [https://doi.org/10.1016/0196-8858\(85\)90002-8](https://doi.org/10.1016/0196-8858(85)90002-8).
- Lakshminarayanan, Balaji, Alexander Pritzel, and Charles Blundell. 2017. “Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles.” *Advances in Neural Information Processing Systems 30*. <https://papers.nips.cc/paper/2017/hash/9ef2ed4b7fd2c810847ffa5fa85bce38-Abstract.html>.
- Lamport, Leslie. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- Lanczos, Cornelius. 1950. “An Iteration Method for the Solution of the Eigenvalue Problem of Linear Differential and Integral Operators.” *Journal of Research of the National Bureau of Standards* 45: 255–82. <https://doi.org/10.6028/jres.045.026>.
- Lattimore, Tor, and Csaba Szepesvari. 2020. *Bandit Algorithms*. Cambridge

- University Press. <https://doi.org/10.1017/9781108571401>.
- LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. “Gradient-Based Learning Applied to Document Recognition.” *Proceedings of the IEEE* 86 (11): 2278–324. <https://doi.org/10.1109/5.726791>.
- Lee, John M. 2018. *Introduction to Riemannian Manifolds*. 2nd ed. Vol. 176. Graduate Texts in Mathematics. Springer. <https://doi.org/10.1007/978-3-319-91755-9>.
- Lehmann, E. L., and George Casella. 1998. *Theory of Point Estimation*. 2nd ed. Springer. <https://doi.org/10.1007/b98854>.
- Lei, Jing, Max G’Sell, Alessandro Rinaldo, Ryan J. Tibshirani, and Larry Wasserman. 2018. “Distribution-Free Predictive Inference for Regression.” *Journal of the American Statistical Association* 113 (523): 1094–111. <https://doi.org/10.1080/01621459.2017.1307116>.
- Lessard, Laurent, Benjamin Recht, and Andrew Packard. 2016. “Analysis and Design of Optimization Algorithms via Integral Quadratic Constraints.” *SIAM Journal on Optimization* 26 (1): 57–95. <https://doi.org/10.1137/15M1009597>.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, et al. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems* 33.
- Locatello, Francesco, Stefan Bauer, Mario Lucic, et al. 2019. “Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations.” *Proceedings of the 36th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 97: 4114–24. <https://proceedings.mlr.press/v97/locatello19a.html>.
- Lopez-Paz, David, and Marc’Aurelio Ranzato. 2017. “Gradient Episodic Memory for Continual Learning.” *Advances in Neural Information Processing Systems* 30. <https://arxiv.org/abs/1706.08840>.
- Loshchilov, Ilya, and Boris Ginsburg. 2026. “Training nGPT.” *arXiv Preprint*. <https://arxiv.org/abs/2608.01284>.
- Loshchilov, Ilya, Cheng-Ping Hsieh, Simeng Sun, and Boris Ginsburg. 2024. “nGPT: Normalized Transformer with Representation Learning on the Hypersphere.” *arXiv Preprint*. <https://arxiv.org/abs/2410.01131>.
- Loshchilov, Ilya, and Frank Hutter. 2019. “Decoupled Weight Decay Regularization.” *7th International Conference on Learning Representations*.

<https://arxiv.org/abs/1711.05101>.

Lu, Yiping, Aoxiao Zhong, Quanzheng Li, and Bin Dong. 2018. “Beyond Finite Layer Neural Networks: Bridging Deep Architectures and Numerical Differential Equations.” *Proceedings of the 35th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 80: 3276–85. <https://proceedings.mlr.press/v80/lu18d.html>.

Madry, Aleksander, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. “Towards Deep Learning Models Resistant to Adversarial Attacks.” *International Conference on Learning Representations*. <https://openreview.net/forum?id=rJzIBfZAb>.

Malkov, Yu A., and D. A. Yashunin. 2020. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824–36. <https://doi.org/10.1109/TPAMI.2018.2889473>.

Mallya, Arun, and Svetlana Lazebnik. 2018. “PackNet: Adding Multiple Tasks to a Single Network by Iterative Pruning.” *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 7765–73. <https://doi.org/10.1109/CVPR.2018.00810>.

McCloskey, Michael, and Neal J. Cohen. 1989. “Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem.” In *Psychology of Learning and Motivation*, vol. 24. Academic Press. [https://doi.org/10.1016/S0079-7421\(08\)60536-8](https://doi.org/10.1016/S0079-7421(08)60536-8).

McLachlan, Robert I., and G. Reinout W. Quispel. 2002. “Splitting Methods.” *Acta Numerica* 11: 341–434. <https://doi.org/10.1017/S0962492902000053>.

Meng, Kevin, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. “Locating and Editing Factual Associations in GPT.” *Advances in Neural Information Processing Systems* 35: 17359–72. https://proceedings.neurips.cc/paper_files/paper/2022/hash/6f1d43d5a82a37e897f454479b637600-Abstract.html.

Mezić, Igor. 2005. “Spectral Properties of Dynamical Systems, Model Reduction and Decompositions.” *Nonlinear Dynamics* 41: 309–25. <https://doi.org/10.1007/s11071-005-2824-x>.

Milner, Robin. 1979. “LCF: A Way of Doing Proofs with a Machine.” *Mathematical Foundations of Computer Science 1979*, Lecture notes in computer science, vol. 74: 146–59. https://doi.org/10.1007/3-540-09526-8_11.

Moldovan, Teodor Mihai, and Pieter Abbeel. 2012. “Safe Exploration in Markov Decision Processes.” *Proceedings of the 29th International Conference on*

Machine Learning. <https://arxiv.org/abs/1205.4810>.

Moreau, Luc, Paolo Missier, et al. 2013. *PROV-DM: The PROV Data Model*. W3C Recommendation. World Wide Web Consortium. <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>.

Moura, Leonardo de, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. “The Lean Theorem Prover (System Description).” *Automated Deduction - CADE-25*, Lecture notes in computer science, vol. 9195: 378–88. https://doi.org/10.1007/978-3-319-21401-6_26.

National Academies of Sciences, Engineering, and Medicine. 2019. *Reproducibility and Replicability in Science*. The National Academies Press. <https://doi.org/10.17226/25303>.

National Institute of Standards and Technology. n.d. *NIST/SEMATECH e-Handbook of Statistical Methods: Process Improvement and Design of Experiments*. NIST Information Technology Laboratory. <https://www.itl.nist.gov/div898/handbook/pri/pri.htm>.

Nesterov, Yurii. 2004. *Introductory Lectures on Convex Optimization: A Basic Course*. Kluwer Academic Publishers.

Nocedal, Jorge, and Stephen J. Wright. 2006. *Numerical Optimization*. 2nd ed. Springer. <https://doi.org/10.1007/978-0-387-40065-5>.

NVIDIA Corporation. 2026a. *CUDA c++ Best Practices Guide*. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>.

NVIDIA Corporation. 2026b. *CUDA Programming Guide*. <https://docs.nvidia.com/cuda/cuda-programming-guide/>.

Olshausen, Bruno A., and David J. Field. 1996. “Emergence of Simple-Cell Receptive Field Properties by Learning a Sparse Code for Natural Images.” *Nature* 381 (6583): 607–9. <https://doi.org/10.1038/381607a0>.

Orseau, Laurent, and Stuart Armstrong. 2016. “Safely Interruptible Agents.” *Proceedings of the Thirty-Second Conference on Uncertainty in Artificial Intelligence*, 557–66. <https://www.auai.org/uai2016/proceedings/papers/68.pdf>.

Oudeyer, Pierre-Yves, Frederic Kaplan, and Verena V. Hafner. 2007. “Intrinsic Motivation Systems for Autonomous Mental Development.” *IEEE Transactions on Evolutionary Computation* 11 (2): 265–86. <https://doi.org/10.1109/TEVC.2006.890271>.

- Pan, Sinno Jialin, and Qiang Yang. 2010. “A Survey on Transfer Learning.” *IEEE Transactions on Knowledge and Data Engineering* 22 (10): 1345–59. <https://doi.org/10.1109/TKDE.2009.191>.
- Parikh, Neal, and Stephen Boyd. 2014. “Proximal Algorithms.” *Foundations and Trends in Optimization* 1 (3): 127–239. <https://doi.org/10.1561/24000000003>.
- Pascanu, Razvan, Tomas Mikolov, and Yoshua Bengio. 2013. “On the Difficulty of Training Recurrent Neural Networks.” *Proceedings of the 30th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 28: 1310–18. <https://proceedings.mlr.press/v28/pascanu13.html>.
- Pearlmutter, Barak A. 1994. “Fast Exact Multiplication by the Hessian.” *Neural Computation* 6 (1): 147–60. <https://doi.org/10.1162/neco.1994.6.1.147>.
- Peng, Bowen, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023. “YaRN: Efficient Context Window Extension of Large Language Models.” *arXiv Preprint arXiv:2309.00071*, ahead of print. <https://doi.org/10.48550/arXiv.2309.00071>.
- Peters, Ben, Vlad Niculae, and André F. T. Martins. 2019. “Sparse Sequence-to-Sequence Models.” *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 1504–19. <https://doi.org/10.18653/v1/P19-1146>.
- Platanios, Emmanouil Antonios, Otilia Stretcu, Graham Neubig, Barnabas Poczos, and Tom M. Mitchell. 2019. “Competence-Based Curriculum Learning for Neural Machine Translation.” *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1*, 1162–72. <https://doi.org/10.18653/v1/N19-1119>.
- Polyak, B. T. 1964. “Some Methods of Speeding up the Convergence of Iteration Methods.” *USSR Computational Mathematics and Mathematical Physics* 4 (5): 1–17. [https://doi.org/10.1016/0041-5553\(64\)90137-5](https://doi.org/10.1016/0041-5553(64)90137-5).
- Portelas, Remy, Cedric Colas, Katja Hofmann, and Pierre-Yves Oudeyer. 2020. “Teacher Algorithms for Curriculum Learning of Deep RL in Continuously Parameterized Environments.” *Proceedings of the Conference on Robot Learning*, Proceedings of machine learning research, vol. 100: 835–53. <https://proceedings.mlr.press/v100/portelas20a.html>.
- Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. *Language Models Are Unsupervised Multitask Learners*. OpenAI.

- Raffel, Colin, Noam Shazeer, Adam Roberts, et al. 2020. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer." *Journal of Machine Learning Research* 21 (140): 1–67. <https://www.jmlr.org/papers/v21/20-074.html>.
- Rao, Yongming, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. 2021. "DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification." *Advances in Neural Information Processing Systems* 34.
- Rebuffi, Sylvestre-Alvise, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. 2017. "iCaRL: Incremental Classifier and Representation Learning." *2017 IEEE Conference on Computer Vision and Pattern Recognition*, 2001–10. <https://doi.org/10.1109/CVPR.2017.587>.
- Reddy, Satish C., Peter J. Schmid, and Dan S. Henningson. 1993. "Pseudospectra of the Orr–Sommerfeld Operator." *SIAM Journal on Applied Mathematics* 53 (1): 15–47. <https://doi.org/10.1137/0153002>.
- Rissanen, Jorma. 1978. "Modeling by Shortest Data Description." *Automatica* 14 (5): 465–71. [https://doi.org/10.1016/0005-1098\(78\)90005-5](https://doi.org/10.1016/0005-1098(78)90005-5).
- Robbins, Herbert, and Sutton Monro. 1951. "A Stochastic Approximation Method." *The Annals of Mathematical Statistics* 22 (3): 400–407. <https://doi.org/10.1214/aoms/1177729586>.
- Robertson, Stephen, and Hugo Zaragoza. 2009. "The Probabilistic Relevance Framework: BM25 and Beyond." *Foundations and Trends in Information Retrieval* 4 (1–2): 1–174. <https://doi.org/10.1561/15000000019>.
- Rubin, Donald B. 1974. "Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies." *Journal of Educational Psychology* 66 (5): 688–701. <https://doi.org/10.1037/h0037350>.
- Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. 1986. "Learning Representations by Back-Propagating Errors." *Nature* 323: 533–36. <https://doi.org/10.1038/323533a0>.
- Russo, Daniel, and Benjamin Van Roy. 2014. "Learning to Optimize via Information-Directed Sampling." *Advances in Neural Information Processing Systems* 27. <https://arxiv.org/abs/1403.5556>.
- Rust, Phillip, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. 2021. "How Good Is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models." *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International*

- Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 3118–35. <https://doi.org/10.18653/v1/2021.acl-long.243>.
- Saad, Yousef. 2003. *Iterative Methods for Sparse Linear Systems*. 2nd ed. Society for Industrial; Applied Mathematics. <https://doi.org/10.1137/1.9780898718003>.
- Sakoe, Hiroaki, and Seibi Chiba. 1978. “Dynamic Programming Algorithm Optimization for Spoken Word Recognition.” *IEEE Transactions on Acoustics, Speech, and Signal Processing* 26 (1): 43–49. <https://doi.org/10.1109/TASSP.1978.1163055>.
- Salton, Gerard, Anita Wong, and C. S. Yang. 1975. “A Vector Space Model for Automatic Indexing.” *Communications of the ACM* 18 (11): 613–20. <https://doi.org/10.1145/361219.361220>.
- Sandve, Geir Kjetil, Anton Nekrutenko, James Taylor, and Eivind Hovig. 2013. “Ten Simple Rules for Reproducible Computational Research.” *PLOS Computational Biology* 9 (10): e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>.
- Schmidhuber, Jürgen. 2009. “Driven by Compression Progress: A Simple Principle Explains Essential Aspects of Subjective Beauty, Novelty, Surprise, Interestingness, Attention, Curiosity, Creativity, Art, Science, Music, Jokes.” *Computational Creativity: An Interdisciplinary Approach*, Dagstuhl seminar proceedings, vol. 09291: 1–35. <https://doi.org/10.4230/DagSemProc.09291.14>.
- Sennrich, Rico, Barry Haddow, and Alexandra Birch. 2016. “Neural Machine Translation of Rare Words with Subword Units.” *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1715–25. <https://doi.org/10.18653/v1/P16-1162>.
- Shaw, Peter, Jakob Uszkoreit, and Ashish Vaswani. 2018. “Self-Attention with Relative Position Representations.” *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, 464–68. <https://doi.org/10.18653/v1/N18-2074>.
- Shoemake, Ken. 1985. “Animating Rotation with Quaternion Curves.” *ACM SIGGRAPH Computer Graphics* 19 (3): 245–54. <https://doi.org/10.1145/325165.325242>.
- Soares, Nate, Benja Fallenstein, Eliezer Yudkowsky, and Stuart Armstrong. 2015. “Corrigibility.” *AAAI Workshop on Artificial Intelligence and Ethics*. <https://intelligence.org/files/Corrigibility.pdf>.

- Solomonoff, Ray J. 1964. “A Formal Theory of Inductive Inference. Part i.” *Information and Control* 7 (1): 1–22. [https://doi.org/10.1016/S0019-9958\(64\)90223-2](https://doi.org/10.1016/S0019-9958(64)90223-2).
- Srivastava, Rupesh Kumar, Klaus Greff, and Jürgen Schmidhuber. 2015. “Highway Networks.” *arXiv Preprint*. <https://arxiv.org/abs/1505.00387>.
- Su, Jianlin, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. “RoFormer: Enhanced Transformer with Rotary Position Embedding.” *arXiv Preprint arXiv:2104.09864*, ahead of print. <https://doi.org/10.48550/arXiv.2104.09864>.
- Sugiyama, Masashi, Matthias Krauledat, and Klaus-Robert Müller. 2007. “Covariate Shift Adaptation by Importance Weighted Cross Validation.” *Journal of Machine Learning Research* 8: 985–1005. <https://www.jmlr.org/papers/v8/sugiyama07a.html>.
- Sutskever, Ilya, James Martens, George Dahl, and Geoffrey Hinton. 2013. “On the Importance of Initialization and Momentum in Deep Learning.” *Proceedings of the 30th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 28: 1139–47. <https://proceedings.mlr.press/v28/sutskever13.html>.
- Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. 2014. “Sequence to Sequence Learning with Neural Networks.” *Advances in Neural Information Processing Systems* 27, 3104–12.
- Thompson, William R. 1933. “On the Likelihood That One Unknown Probability Exceeds Another in View of the Evidence of Two Samples.” *Biometrika* 25 (3–4): 285–94. <https://doi.org/10.1093/biomet/25.3-4.285>.
- Tieleman, Tijmen, and Geoffrey Hinton. 2012. *Lecture 6.5—RMSPProp: Divide the Gradient by a Running Average of Its Recent Magnitude*. Coursera: Neural Networks for Machine Learning.
- Tishby, Naftali, Fernando C. Pereira, and William Bialek. 2000. “The Information Bottleneck Method.” *arXiv Preprint*. <https://arxiv.org/abs/physics/0004057>.
- Trefethen, Lloyd N., and III David Bau. 1997. *Numerical Linear Algebra*. Society for Industrial; Applied Mathematics. <https://doi.org/10.1137/1.9780898719574>.
- Trefethen, Lloyd N., and Mark Embree. 2005. *Spectra and Pseudospectra: The Behavior of Nonnormal Matrices and Operators*. Princeton University Press. <https://doi.org/10.2307/j.ctvzxx9kj>.

- Trefethen, Lloyd N., Anne E. Trefethen, Satish C. Reddy, and Tobin A. Driscoll. 1993. “Hydrodynamic Stability Without Eigenvalues.” *Science* 261 (5121): 578–84. <https://doi.org/10.1126/science.261.5121.578>.
- Tsai, Yao-Hung Hubert, Shaojie Bai, Makoto Yamada, Louis-Philippe Morency, and Ruslan Salakhutdinov. 2019. “Transformer Dissection: An Unified Understanding for Transformer’s Attention via the Lens of Kernel.” *Proceedings of EMNLP-IJCNLP*, 4344–53. <https://doi.org/10.18653/v1/D19-1443>.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30. <https://papers.nips.cc/paper/7181-attention-is-all-you-need>.
- Ven, Gido M. van de, Tinne Tuytelaars, and Andreas S. Tolias. 2022. “Three Types of Incremental Learning.” *Nature Machine Intelligence* 4: 1185–97. <https://doi.org/10.1038/s42256-022-00568-3>.
- Vershynin, Roman. 2018. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Vol. 47. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press. <https://doi.org/10.1017/9781108231596>.
- Voita, Elena, and Ivan Titov. 2020. “Information-Theoretic Probing with Minimum Description Length.” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 183–96. <https://doi.org/10.18653/v1/2020.emnlp-main.14>.
- Vovk, Vladimir, Alexander Gammerman, and Glenn Shafer. 2005. *Algorithmic Learning in a Random World*. Springer. <https://doi.org/10.1007/b106715>.
- Wainwright, Martin J., and Michael I. Jordan. 2008. “Graphical Models, Exponential Families, and Variational Inference.” *Foundations and Trends in Machine Learning* 1 (1–2): 1–305. <https://doi.org/10.1561/22000000001>.
- Wang, Sinong, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. *Linformer: Self-Attention with Linear Complexity*. <https://arxiv.org/abs/2006.04768>.
- Wang, Tongzhou, and Phillip Isola. 2020. “Understanding Contrastive Representation Learning Through Alignment and Uniformity on the Hypersphere.” *Proceedings of the 37th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 119: 9929–39. <https://proceedings.mlr.press/v119/wang20k.html>.
- Wang, Xin, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E. Gonzalez. 2018. “SkipNet: Learning Dynamic Routing in Convolutional Networks.”

- Proceedings of the European Conference on Computer Vision*, 409–24. <https://arxiv.org/abs/1711.09485>.
- Wasserstein, Ronald L., and Nicole A. Lazar. 2016. “The ASA Statement on p-Values: Context, Process, and Purpose.” *The American Statistician* 70 (2): 129–33. <https://doi.org/10.1080/00031305.2016.1154108>.
- Williams, Samuel, Andrew Waterman, and David Patterson. 2009. “Roofline: An Insightful Visual Performance Model for Multicore Architectures.” *Communications of the ACM* 52 (4): 65–76. <https://doi.org/10.1145/1498765.1498785>.
- Wooldridge, Michael, and Nicholas R. Jennings. 1995. “Intelligent Agents: Theory and Practice.” *The Knowledge Engineering Review* 10 (2): 115–52. <https://doi.org/10.1017/S0269888900008122>.
- Xiong, Ruibin, Yunchang Yang, Di He, et al. 2020. “On Layer Normalization in the Transformer Architecture.” *Proceedings of the 37th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 119: 10524–33. <https://proceedings.mlr.press/v119/xiong20b.html>.
- Xiong, Yunyang, Zhanpeng Zeng, Rudrasis Chakraborty, et al. 2021. “Nyströmformer: A Nyström-Based Algorithm for Approximating Self-Attention.” *Proceedings of the AAAI Conference on Artificial Intelligence* 35: 14138–48. <https://doi.org/10.1609/aaai.v35i16.17664>.
- Xue, Linting, Aditya Barua, Noah Constant, et al. 2022. “ByT5: Towards a Token-Free Future with Pre-Trained Byte-to-Byte Models.” *Transactions of the Association for Computational Linguistics* 10: 291–306. https://doi.org/10.1162/tacl_a_00461.
- Yosinski, Jason, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. “How Transferable Are Features in Deep Neural Networks?” *Advances in Neural Information Processing Systems* 27: 3320–28. https://proceedings.neurips.cc/paper_files/paper/2014/hash/532a2f85b6977104bc93f8580abbb330-Abstract.html.
- Zhou, Feng, and Fernando De la Torre. 2012. “Generalized Time Warping for Multi-Modal Alignment of Human Motion.” *2012 IEEE Conference on Computer Vision and Pattern Recognition*, 1282–89. <https://doi.org/10.1109/CVPR.2012.6247812>.
- Zhou, Yanqi, Tao Lei, Hanxiao Liu, et al. 2022. “Mixture-of-Experts with Expert Choice Routing.” *Advances in Neural Information Processing Systems* 35: 7103–14. <https://doi.org/10.52202/068431-0515>.
- Zhu, Xiaojin. 2015. “Machine Teaching: An Inverse Problem to Machine Learning and an Approach Toward Optimal Education.” *Proceedings of the*

Twenty-Ninth AAAI Conference on Artificial Intelligence, 4083–87. <https://doi.org/10.1609/aaai.v29i1.9761>.

Ziyin, Liu. 2024. “Symmetry Induces Structure and Constraint of Learning.” *Proceedings of the 41st International Conference on Machine Learning*, Proceedings of machine learning research, vol. 235: 62847–66. <https://proceedings.mlr.press/v235/ziyin24a.html>.

Zoph, Barret, Irwan Bello, Sameer Kumar, et al. 2022. *ST-MoE: Designing Stable and Transferable Sparse Expert Models*. <https://arxiv.org/abs/2202.08906>.